Library User Manual

AXL library family
AXM

AJINEXTEK AXT

Product Information

Full information about other AJINEXTEK products
is available by visiting our Web Site at:
Home Page : www.ajinextek.com
E-Mail ::support@ajinextek.com

Useful Contact Information

Customer Support Seoul
Tel : 82-031-436-2180~2 Fax: 82-031-436-2183

Customer Support Cheonan
Tel : 82-041-555-9771~2 Fax: 82-041-555-9773

Customer Support Deagu
Tel : 82-053-593-3700~2 Fax: 82-053-593-3703

AJINEXTEK's sales team is always available to assist you in making your decision the final choice of boards or systems is solely and wholly theresponsibility of the buyer. AJINEXTEK's entire liability in respect of the board or systems is as set out in AJINEXTEK's standard terms and conditions of sale

ⓒ Copyright 2010. AJINEXTEK co.ltd. All rights reserved.

Content

CONTENT 3

HOW TO USE MOTION LIBRARY 13

Introduction Of Motion Library 13

Introduction Of Motion Control 13
Introduction Of Motion Library 13

Initialization And Exiting 17

Initialization Of Library 17
Exiting Library 18

Motion Parameter 21

Motion Parameter Setting 21

Motion Signal 31

Motion Signal Setting 32
Universal Input/Output Signal Reading And Output Writing 38
Saving And Opening Setting Information 41

Motion Drive 44

1) One Axis 44
2) Multi Axis 44

Glossary Of Motion Drive 46

1) Servo On 46
2) Velocity Profile 46
3) Exit API At Start Point And Exit API At End Point 48
4) Relative Motion Drive And Absolute Motion Drive 48

One-Axis Drive 50

1) Position Drive 50
2) Velocity Drive 53
3) Signal Search Drive 56

4) Home Search Drive 61
5) Stop Of Motion Drive 67
6) Verification Of Motion Drive State 70

Multi-Axis Drive 76

1) Multi-Axis Position Drive 76
2) Interpolation Drive 81
3) Continuous Interpolation Drive 91

CRC Signal Setting 101

Override Drive 105

MPG Drive (Manual Pulse Generation) 109

Sync Drive 117

Gantry Drive 121

1) Setting And Cancellation Of Gantry Drive 121
2) Home Search In Gantry Drive 122

Virtual Axis Mapping 131

Interrupt Setting 134

1) Interrupt Management Method Setting 134
2) Interrupt Use Setting 137

Use Of Trigger Signal 141

Advanced Motion Drive 150

Helical Interpolation Motion 150
Spline Interpolation Motion 159
Electronic CAM [ECAM] 166
Compensation Geometric Errors 168

MOTION COMMAND MANUAL INFORMATION 171

Header File 171

Names of API in this manual 172

MOTION COMMAND QUICK LIST 174

API List **174**

Board and Module Verification API (Info) – Information 174
Virtual Axis API (Virtual) 174
Interrupt API (Interrupt) 174
Motion Parameter Setting API (Mot) -> Motor Parameter 174
Input and Output Setting API (Signal) 176
State Verification (Status) 177
Home Search API (Home) 178
Position Drive API (Move) 178
Position and Velocity Override Drive API (Override) 179
MASTER, SLAVE Drive API (Link) 180
Gantry Drive API (Gantry) 180
Interpolation Drive API (Line, Circle) 180
Continuous Interpolation Setting & Drive API (Conti) -> Continue 181
Trigger API (Trigger) 182
CRC(Remaining Pulse Clear) Drive API 182
MPG(Manul pulse Generator) Drive API 182
Helical Interpolation Setting and Drive API 183
Spline Interpolation Setting and Drive API (PCI-N804/404 only) 183
Electronic CAM Setting and Drive API 184
Compensation Geometric Errors Setting and Drive API 184

Define Sentence **185**

MOTION COMMAND FUNCTION LIST 196

Board And Module Initialization **196**

AxmInfoGetAxis 197
AxmInfoIsMotionModule 199
AxmInfoIsInvalidAxisNo 201
AxmInfoGetAxisCount 203
AxmInfoGetFirstAxisNo 205

Virtual Axis API **208**

AxmVirtualSetAxisNoMap 209

AxmVirtualGetAxisNoMap 211
AxmVirtualSetMultiAxisNoMap 213
AxmVirtualGetMultiAxisNoMap 216
AxmVirtualResetAxisMap 219

Interrupt API 220

AxlInterruptEnable 221
AxlInterruptDisable 223
AxmInterruptSetAxis 224
AxmInterruptSetAxisEnable 226
AxmInterruptGetAxisEnable 228
AxmInterruptRead 230
AxmInterruptReadAxisFlag 232
AxmInterruptSetUserEnable 234
AxmInterruptGetUserEnable 236

Motion Parameter Setting API 238

AxmMotLoadParaAll 240
AxmMotSaveParaAll 243
AxmMotSetParaLoad 246
AxmMotGetParaLoad 250
AxmMotSetPulseOutMethod 254
AxmMotGetPulseOutMethod 258
AxmMotSetEncInputMethod 260
AxmMotGetEncInputMethod 263
AxmMotSetMoveUnitPerPulse 265
AxmMotGetMoveUnitPerPulse 269
AxmMotSetDecelMode 271
AxmMotGetDecelMode 273
AxmMotSetRemainPulse 275
AxmMotGetRemainPulse 278
AxmMotSetMaxVel 280
AxmMotGetMaxVel 282
AxmMotSetAbsRelMode 284

AxmMotGetAbsRelMode 286
AxmMotSetProfileMode 288
AxmMotGetProfileMode 291
AxmMotSetAccelUnit 293
AxmMotGetAccelUnit 295
AxmMotSetMinVel 297
AxmMotGetMinVel 299
AxmMotSetAccelJerk 301
AxmMotGetAccelJerk 303
AxmMotSetDecelJerk 305
AxmMotGetDecelJerk 307

Input And Output Level Setting API 309

AxmSignalSetZphaseLevel 310
AxmSignalGetZphaseLevel 312
AxmSignalSetServoOnLevel 314
AxmSignalGetServoOnLevel 316
AxmSignalSetServoAlarmResetLevel 318
AxmSignalGetServoAlarmResetLevel 320
AxmSignalSetInpos 322
AxmSignalGetInpos 325
AxmSignalReadInpos 327
AxmSignalSetServoAlarm 329
AxmSignalGetServoAlarm 331
AxmSignalReadServoAlarm 333
AxmSignalSetLimit 335
AxmSignalGetLimit 338
AxmSignalReadLimit 340
AxmSignalSetSoftLimit 342
AxmSignalGetSoftLimit 345
AxmSignalSetStop 347
AxmSignalGetStop 349
AxmSignalReadStop 351

AxmSignalServoOn 353
AxmSignalIsServoOn 356
AxmSignalServoAlarmReset 358
AxmSignalWriteOutput 361
AxmSignalReadOutput 364
AxmSignalWriteOutputBit 366
AxmSignalReadOutputBit 369
AxmSignalReadInput 371
AxmSignalReadInputBit 374

Checking Status API 377

AxmStatusReadInMotion 378
AxmStatusReadDrivePulseCount 380
AxmStatusReadMotion 382
AxmStatusReadStop 384
AxmStatusReadMechanical 387
AxmStatusReadVel 390
AxmStatusReadPosError 392
AxmStatusReadDriveDistance 394
AxmStatusSetActPos 396
AxmStatusGetActPos 398
AxmStatusSetCmdPos 400
AxmStatusGetCmdPos 402
AxmStatusRequestServoAlarm 404
AxmStatusReadServoAlarm 406
AxmStatusGetServoAlarmString 408
AxmStatusRequestServoAlarmHistory 410
AxmStatusReadServoAlarmHistory 412
AxmStatusClearServoAlarmHistory 414

Home Search API 416

AxmHomeSetSignalLevel 420
AxmHomeGetSignalLevel 422
AxmHomeReadSignal 424

AxmHomeSetMethod 426
AxmHomeGetMethod 429
AxmHomeSetVel 432
AxmHomeGetVel 435
AxmHomeSetStart 438
AxmHomeSetResult 440
AxmHomeGetResult 445
AxmHomeGetRate 450

Position Move API 455

AxmMoveStartPos 457
AxmMovePos 460
AxmMoveVel 462
AxmMoveStartMultiVel 464
AxmMoveSignalSearch 467
AxmMoveSignalCapture 469
AxmMoveGetCapturePos 472
AxmMoveStartMultiPos 474
AxmMoveMultiPos 477
AxmMoveStop 480
AxmMoveEStop 482
AxmMoveSStop 484

Position / Velocity Override Move API 486

AxmOverridePos 487
AxmOverrideSetMaxVel 489
AxmOverrideVel 491
AxmOverrideAccelVelDecel 493
AxmOverrideVelAtPos 495

MASTER, SLAVE Move API 497

AxmLinkSetMode 498
AxmLinkGetMode 500
AxmLinkResetMode 502

Interpolation Move API (Line, Circle) 504

AxmLineMove 506
AxmCircleCenterMove 513
AxmCirclePointMove 521
AxmCircleRadiusMove 529
AxmCircleAngleMove 537

Continuous Interpolation Setting And Move API 545

AxmContiSetAxisMap 548
AxmContiGetAxisMap 551
AxmContiSetAbsRelMode 554
AxmContiGetAbsRelMode 557
AxmContiBeginNode 560
AxmContiEndNode 564
AxmContiReadFree 568
AxmContiReadIndex 572
AxmContiWriteClear 576
AxmContiStart 579
AxmContiIsMotion 583
AxmContiGetNodeNum 587
AxmContiGetTotalNodeNum 591

Trigger API 595

AxmTriggerSetTimeLevel 596
AxmTriggerGetTimeLevel 599
AxmTriggerSetAbsPeriod 602
AxmTriggerGetAbsPeriod 605
AxmTriggerSetBlock 607
AxmTriggerGetBlock 610
AxmTriggerOneShot 612
AxmTriggerSetTimerOneshot 614
AxmTriggerOnlyAbs 616
AxmTriggerSetReset 619

Gantry Move API 621

AxmGantrySetEnable 624
AxmGantryGetEnable 627
AxmGantrySetDisable 630

CRC (Remaining Pulse Clear) API 632

AxmCrcSetMaskLevel 633
AxmCrcGetMaskLevel 635
AxmCrcSetOutput 637
AxmCrcGetOutput 639

MPG Move API 641

AxmMPGSetEnable 643
AxmMPGGetEnable 646
AxmMPGSetRatio 648
AxmMPGGetRatio 650
AxmMPGReset 652

Helical Interpolation Setting And Move API 654

AxmHelixCenterMove 656
AxmHelixPointMove 662
AxmHelixRadiusMove 668
AxmHelixAngleMove 674

Spline Interpolation Setting And Move API 680

AxmSplineWrite 681

Electronic CAM Setting And Move API 686

AxmEcamSet 687
AxmEcamGet 690
AxmEcamEnableBySlave 692
AxmEcamIsSlaveEnable 694
AxmEcamEnableByMaster 696

Compensation Geometric Error Setting And Move API 699

AxmCompensationSet 700
AxmCompensationGet 704
AxmCompensationEnable 706
AxmCompensationIsEnable 708

Error Code Table Check 710

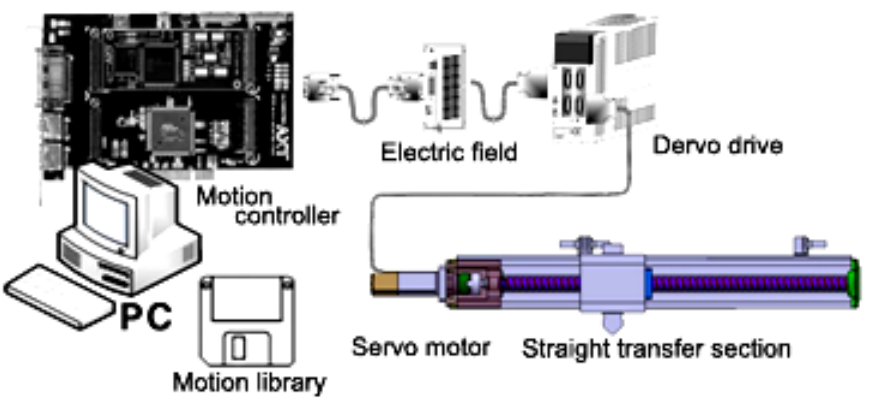
How To Use Motion Library

Introduction Of Motion Library

Introduction Of Motion Control

If you take a look at the devices that assemble or inspect accurate parts such as semiconductor, or the machine tools that process structures, what you can see obviously is the motion controller (transfer system) where actual movement occurs. Motion means movement. Motion can be explained as moves of certain object from one place to another. And, control is a concept of handling. In other words, it means that we can move an object to the direction and position that we want. Ultimately, motion control is a combination of motion and control, and can be defined as a movement of an object to the direction that we want. Following parts stated below are required for the motion control.

- Straight transfer section : mechanical part that rectilineal movement occurs
- Servo motor : device that generates power
- Servo drive : device that flows electricity to servo motor
- Motion controller : device that generates various signals to control motion
- Electric field : other connecting devices to connect motion controller and servo drive
- PC : primary device that controls motion controller by PC programming
- Motion library : API that moves motion controller



Introduction Of Motion Library

Window programming which programs on MS Windows is prevailing recently. Variety of tools are provided for the programming tools including Visual C++, C++ Builder, Delphi, Visual Basic, and so on, and AXL

provides API functions that can be utilized on all those tools. Especially, since the library functions for the motion drive provide the very subdivided functions from the specific control for the mechanical signal and universal input/output signal of general device part to the function for the one-axis and multi-axis interpolation drive function, they can be utilized to all motion controls required by users.

This user guide explains the usage of library functions for driving PCI-N804/404 motion controllers including four axes motion control LSI and SMC-2V03 motion controllers including single chip two axes motion control LSI among various motion control modules, both of which support single axis and multi axes drive as well as interpolation function. Precision control of stepping motors and servo motors is possible by using SMC-2V03 and PCI-804/404 motion libraries. Therefore, continuous motion control system can be realized without increasing the CPU load by using these motion control products, and home search, velocity and position override, linear interpolation, circular interpolation can be performed by the combination of linear and S-curve acceleration and deceleration profiles.

Basic Motion Drive

- Library initialization and exiting
- Motion parameter setting
- Motion signal setting
- Utilization of Universal Input/Output Signal
- One-axis Drive
  - Position Drive
  - Velocity Drive
  - Signal Search Drive
  - Home Search Drive
- Multi-axis drive
  - Multi-axis Position Drive
  - Multi-axis Interpolation Drive
  - Multi-axis Continuous Interpolation Drive

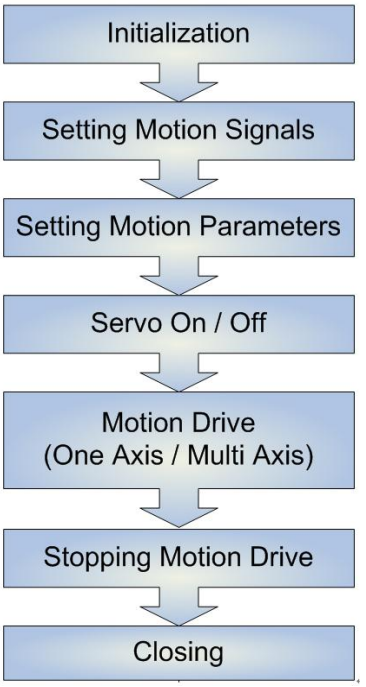
Advanced Motion Drive

- CRC Signal Setting
- Sensor Position Drive
- Override Drive
- MPG Drive
- Sync Drive
- Dynamic Link
- Gantry Drive
- Virtual Axis Mapping
- Interrupt Setting
- Use of Trigger Signal

AXM Library supports list of products.

Product	Explanation	Note
SMC-2V04	Motion Control Module Board(Combined with BPHR or BPFR)	2 controlled axis, Open loop
PCI-N804(Half size)	Motion Control Board (PCI bus)	8 controlled axis, Open loop
PCI-N404(Half size)	Motion Control Board (PCI bus)	4 controlled axis, Open loop
PCI-R1604	Network Master Board (PCI bus)	RTEX(Real-Time Express) Network Master Board Max 32 Slave Node Normally 16 axis, can be extended up to 32 controlled axis
RTEX-PM	RTEX Motion Slave Node	1 axis, pulse output (Ajinextek)
RTEX-A4N	RTEX Motion Slave Node	1 axis, servo driver (Panasonic)
PCI-R1604-MLII	Network Master Board (PCI bus)	Mechatrolink-II Network Master Board Max 30 Slave Node Normally 16 axis, can be extended up to 30 controlled axis
JEPMC-PL2910	Mechatrolink-II Motion Slave Node	2 axis, pulse output (Yaskawa)
SGDV-xxxx11A	Mechatrolink-II Motion Slave Node	1 axis, Servo Driver (Yaskawa)

Basic motion drive is divided into initialization and exiting, input/output signal, parameter setting and verification, servo on/off, signal setting drive, one-axis position drive, one-axis velocity drive, monitoring of drive state and stop state, and stopping one-axis drive and is done by the following order. On each chapter, it is explained on the basis of concept of hardware and basic API, and the sample of basic API that is necessary to the chapter.


Initialization
Setting Motion Signals
Setting Motion Parameters
Servo On / Off
Motion Drive
(One Axis / Multi Axis)
Stopping Motion Drive
Closing

Initialization And Exiting

The initialization work of AXL library is done by running AxlOpen API. When AxlOpen API runs, it initializes all library as well as base board and module that are installed internally. Hence, user must verify existence of motion module before using motion library.

Initialization Of Library

Initialization of Library

AxlOpen: AxlOpen API is an API that initializes library. When a user uses interrupt, it registers IRQ number that manage interrupt. Particularly, inputted IRQ number during the library initialization is ignored since IRQ number is generated automatically on the PCI type base board.

This API returns error code to the return value, so following can be used.

```
// Initialize library.
// 7 means IRQ. IRQ is set automatically in PCI.
if (AxlOpen(7) == AXT_RT_SUCCESS)
  printf("Library is initialized.");
```

Verification whether library is initialized or not

AxlIsOpened: AxlIsOpened API is an API that verifies whether library is initialized or not. This API returns BOOL value which is not an error code. If the library is initialized, it returns TRUE, otherwise it returns FALSE.

```
// Verify if library is initialized.
if (AxlIsOpened())
  printf("Library is initialized.");
else
  printf("Library is not initialized.");
```

Verification whether motion module exists or not

AxmInfoIsMotionModule: AxmInfoIsMotionModule API is an API that verifies whether actual motion module exists or not. This is used to verify the existence of motion module before using motion related API. To verify the existence of module, following is done.

```
// Verify if motion module exists.
DWORD dwStatus;
AxmInfoIsMotionModule(&dwStatus);
if(dwStatus == STATUS_EXIST)
  printf("Motion module exists.");
else
  printf("Motion module does not exists.");
```

Verification of the number of axis installed in the system

AxmInfoGetAxisCount: AxmInfoGetAxisCount API is an API that verifies the number of axis installed in the whole system.

```
// Verify the number of axis installed in the system.
long lAxisCount;
AxmInfoGetAxisCount(&lAxisCount);
printf("number of axis installed in the system : %d",lAxisCount);
```

AxmInfoGetFirstAxisNo: AxmInfoGetFirstAxisNo API is an API that verifies the first axis number on the specific module of specified base board.

```
// Verify the starting axis number on number 0 module of number 0 board.
long lBoardNo = 0;
long lModulePos = 0;
long lFirstAxisNo;
AxmInfoGetFirstAxisNo(lBoardNo, lModulePos, &lFirstAxisNo);
printf("Starting axis number on the 1st module of number 0 base board : %d", lFirstAxisNo);
```

AxmInfoGetAxis: AxmInfoGetAxis API is an API that verifies the information of installed axis such as board number, module number and module ID.

```
// Verify board number, module number and module ID of axis 0.
long lAxisNo = 0;
long lBoardNo, lModulePos;
DWORD dwModuleID;
AxmInfoGetAxis(lAxisNo, &lBoardNo, &lModulePos, &dwModuleID);
printf(("Axis 0 of board number : %x, Module number : %x, Module ID : %x", lBoardNo,
  lModulePos, dwModuleID);
if(AxlIsOpened())
  AfxMessageBox("Library is initialized.");
else
  AfxMessageBox("Library is not initialized.");
```

Exiting Library

AxlClose : AxlClose API is an API that ends library. Library must be ended lastly and return the assigned memory after using. If API runs normally and library ends, it returns TRUE, otherwise it returns FALSE.

```
// End library.
if (AxlClose())
  printf("Library is ended.");
```

```
// Ex1_AXM_InitAndClose.cpp : Defines the entry point for the console application.
// Initialize library, verify information of axis and end.
#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
```

```
#include "stdio.h"

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           // Verify the number of axis installed in the system.
           long lpAxisCount;
           AxmInfoGetAxisCount(&lpAxisCount);
           printf("Number of axis installed in the system : %d \n",lpAxisCount);

           // Verify board number, module position, and module ID on number axis 0.
           long lBoardNo, lModulePos;
           long lAxisNo = 0;
           DWORD dwModuleID;
           AxmInfoGetAxis(lAxisNo, &lBoardNo, &lModulePos, &dwModuleID);
           printf("on number axis 0, board number : %x, module position : %x, module
              ID : %x\n", lBoardNo,lModulePos,dwModuleID);

           // Verify the starting axis number on number 1 module of number 0 board.
           long lFirstAxisNo;
           AxmInfoGetFirstAxisNo(lBoardNo, lModulePos, &lFirstAxisNo);
           printf("on the %d th number base board, starting axis number on the module of
              the %d th : %d\n", lBoardNo,lModulePos,lFirstAxisNo);
        }
        else
           printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code
              0x%x\n",Code);
     }
     else
        printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
     printf ("AxlOpen() : ERROR  code 0x%x\n",Code);
```

```
    // End library.
    if (AxlClose())
        printf("Library is ended.\n");
    else
        printf("Library is not ended normally.\n");
}
```

Motion Parameter

Motion parameter is a setting value for the right setting of each part that constitutes motion system. Motion system, as it is mentioned before, is combined with several devices such as instrumental part, servo driver, servo motor, electric field, motion controller, and so on. Obviously, each component has its own setting parameter. Therefore, even though just one has a problem for setting, the whole system malfunctions or commits an error.

Motion parameter that is set by motion controller is as following, and each meaning shows below.

Unit per Pulse

- it is a value that sets a unit of ordered value from the command.

Start / Stop Speed

- it sets velocity when drive part starts and ends.

Pulse Out Method

- it sets type of pulse that outputs from motion controller to servo drive.

Encoder Input Method

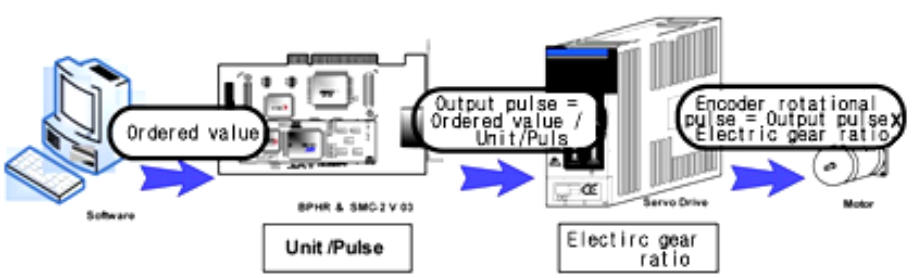
- it sets method that receives encoder signal sent from servo drive to motion controller.

Motion Parameter Setting

Parameter of motion controller is set related to setting and characteristics that each element has in the whole system. Hence, preconditions to set each parameter value should be read carefully, and set adjusting correctly compared to the system that user has.

1) Unit per Pulse

Basically, ordered value of moving distance that user assigns on the commands of library takes two steps of unit conversion as the picture below.



If a movement volume is inputted on the software, number of output pulse of motion controller by an already set parameter called unit/pulse is determined. And this output pulse is converted again by parameter called electric gear ration set on the servo drive, and the drive pulse that drives actual motor is

determined.

Unit/pulse is a parameter that is to assign unit of values given by running the motion drive API, and the API which sets this value is AxmMotSetMoveUnitPerPulse. In other words, if it is set 10 on unit and 100 on pulse, 100 pulses would be outputted to the servo drive when 10 is commanded on the motion drive API. For example, let us assume that a motion controller outputs 10,000 pulse for 1 rotation of servo motor on the instrumental part that moves 10mm per rotation of servo motor, and if you want to set the ordered value that is given on the program by mm unit, then do below.

```
// Make an ordered value of axis 0 to be mm unit.
long lAxisNo = 0;
long lPulse = 10000;      // number of pulse for 1 rotation.
double dUnit = 10;        // movement distance during 1 rotation.
AxmMotSetMoveUnitPerPulse(lAxisNo, dUnit, lPulse);
```

If you want to set the ordered value given on the program by pulse unit, then do below.

```
// Make an ordered value of axis 0 to be pulse unit.
long lAxisNo = 0;
long lPulse = 1;
double dUnit = 1;
AxmMotSetMoveUnitPerPulse(lAxisNo, dUnit, lPulse);
```

※ *Electronic gear Ratio (전자기어비)*

Unit/Pulse is a parameter that is set on the motion controller, while electric gear ratio is set on the servo drive.

[Electric gear ratio] =

[Number of encoder pulse that is generated during 1 rotation of servo motor] /

[Number of output ordered pulse of motion controller during 1 rotation of servo motor]

If 17-bit encoder is used, 131,072 of encoder pulse can be reduced to 10,000 on the motion controller based on above. That is, what a rotational accuracy of servo motor encoder lowers by an accuracy of mechanical part is an electric gear ratio.

For example, in case of 20mm ball screw, 1/2 of electric gear ratio, and 10000 of pulse number is required for 1 rotation of motor, if it is set as lPulse=10000 and dUnit=10, pulse number that outputs per moving distance of 1mm is set to 1000.



2) Start Speed

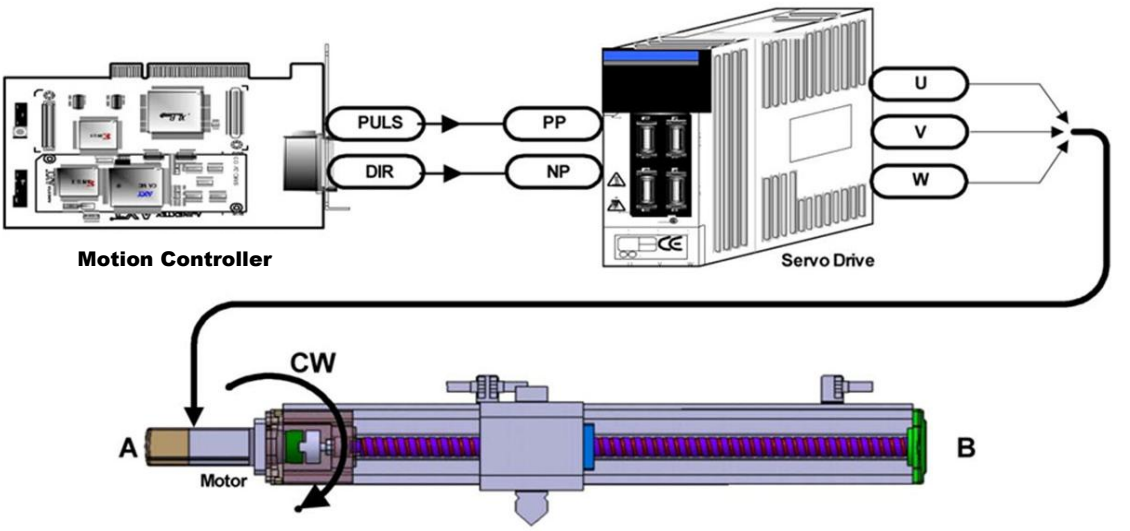
Assign an initial velocity of when motion is started. The initial velocity must be set because it is to be the reference point of motion drive, and a default value is set by 1. Using API of AxmMotSetMinVel, set as following.

```
// Set an initial velocity of axis 0 as 1.  Default : 1
long lAxisNo = 0;
double dMinVelocity = 1;
AxmMotSetMinVel(lAxisNo, dMinVelocity);
```

3) Pulse Out Method

Drive command which is ordered by user has a structure that transfers a pulse to the servo drive based on the ordered value by the motion controller showing on the picture below, and that drives the servo drive again by the servo motor.

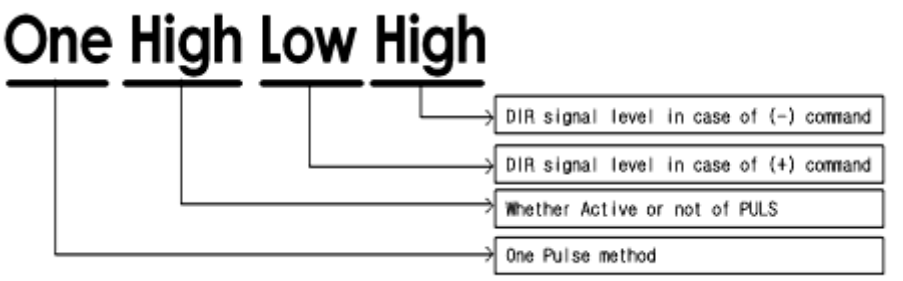
Let us assume that the motion controller outputs a pulse to the servo drive through PULS pin and DIR pin. Then PP pin and NP pin of the servo drive would receive the pulse and it would drive the servo motor as much as the concerned pulse according to the pulse signal received from the two pins. Now it is required to determine to rotate the servo motor with how much pulse from which pin and with which direction. And it will run normally if the servo drive is set identically with what the motion controller is set.

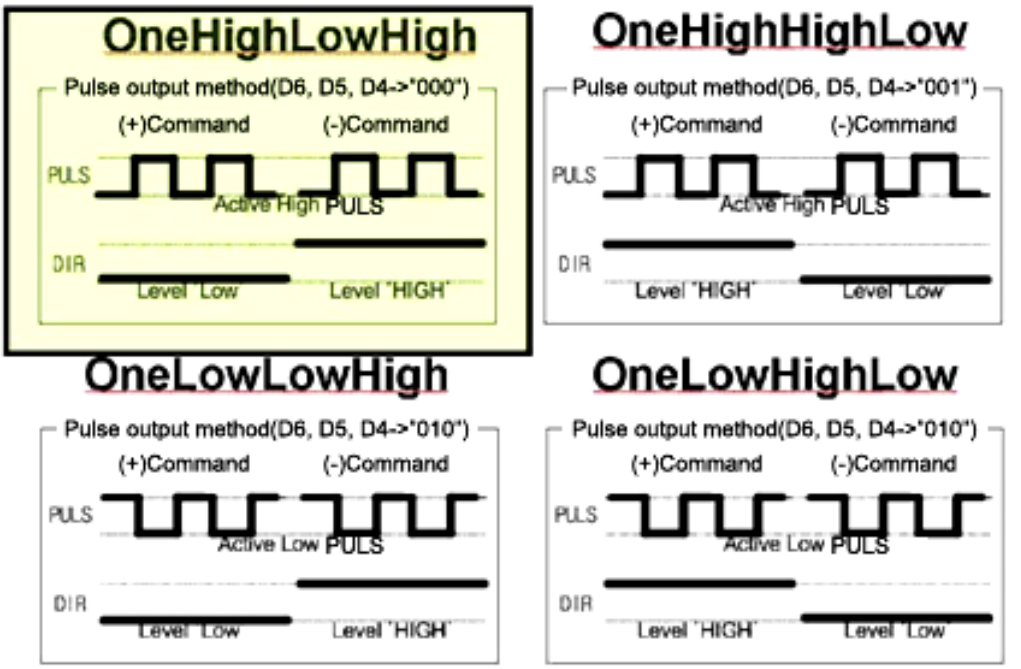


PULS and DIR pulse output method of the motion controller that outputs to the servo drive is classified into one pulse method and two pulse method.

One Pulse Method

In one pulse method, PULS pin of the motion controller outputs a pulse of the number of rotational amount and DIR pin outputs a signal level of the rotational direction. As shown on the picture below, 1 pulse output method consists of whether PULS signal and DIR are active or not and the signal level when (+) direction command is given and (−) direction command is given. Since there is a little difference on the basis of the servo drive and the servo motor, the pulse output method of the motion controller is to be set carefully based on the setting method on the servo drive.

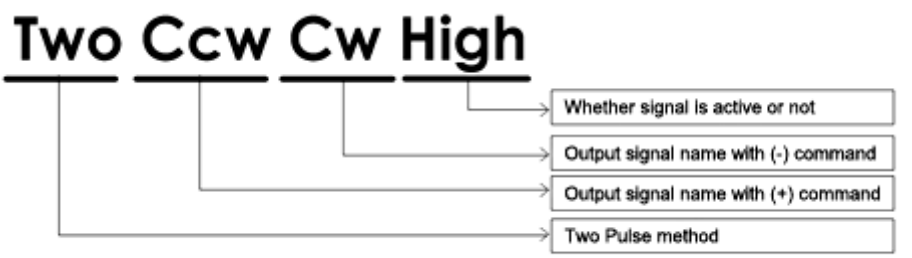


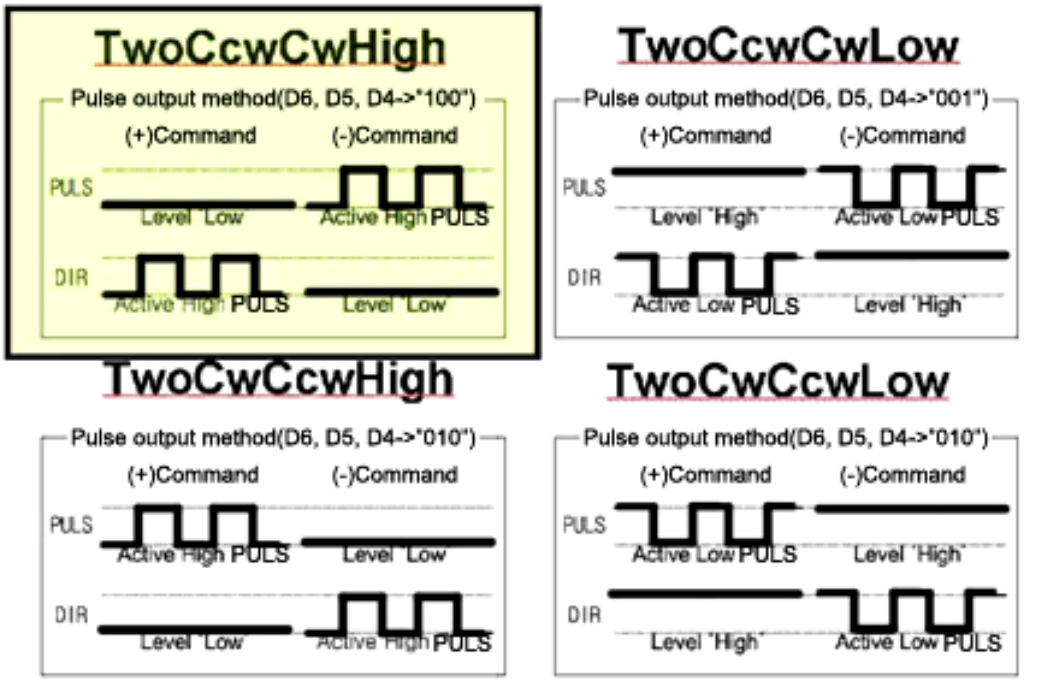


Type	Explanation
OneHighLowHigh	1 pulse method, PULSE(Active High) + direction(DIR=Low) / – direction (DIR=High)
OneHighHighLow	1 pulse method, PULSE(Active High) + direction (DIR=High) / – direction (DIR=Low)
OneLowLowHigh	1 pulse method, PULSE(Active Low) + direction (DIR=Low) / – direction (DIR=High)
OneLowHighLow	1 pulse method, PULSE(Active Low) + direction (DIR=High) / – direction (DIR=Low)

Two Pulse Method

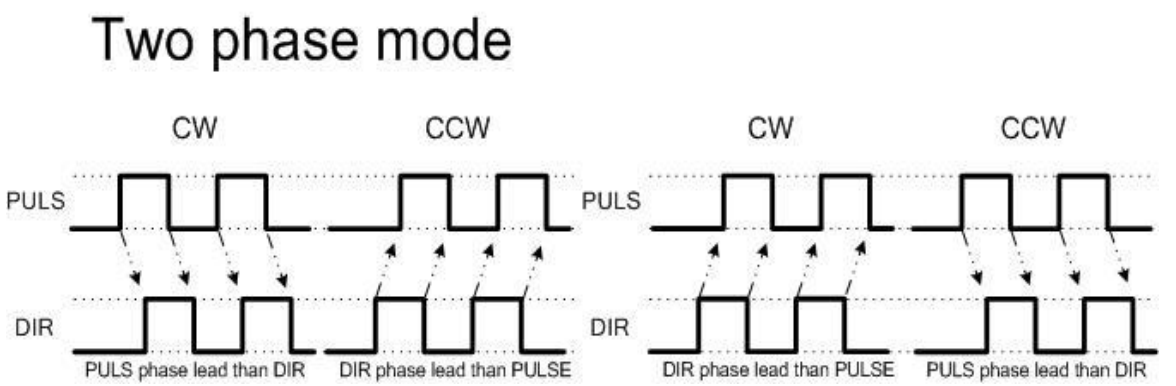
Two pulse method is an output method that is structured to show each rotational amount and rotational direction simultaneously by the pulse signal from PULS pin and DIR pin. Showing on the picture below, 2 pulse output method is structured with a part that sets a pin that a signal would be outputted when a command of (+) direction direction is given, a pin for (–) direction, and whether signal is active or not. Here, Cw means PULS pin and Ccw means DIR pin.





Type	Description
TwoCcwCwHigh	2 pulse method, PULSE(CW:−direction) DIR(CCW:+ direction), Active High
TwoCcwCwLow	2 pulse method, PULSE(CW:− direction) DIR(CCW:+ direction), Active Low
TwoCwCcwHigh	2 pulse method, PULSE(CW:+ direction) DIR(CCW:− direction), Active High
TwoCwCcwLow	2 pulse method, PULSE(CW:+ direction) DIR(CCW:− direction), Active Low

Two Phase Method



An output pulse can be set as 2 phase 4 multiplication output method, and maximum frequency of control pulse can be outputted quarterly.

Setting Pulse Output Method

User can choose a pulse output method that meets the character of his/her mechanical part and servo drive, and set a pulse output method on an assigned axis using AxmMotSetPulseOutMethod API.

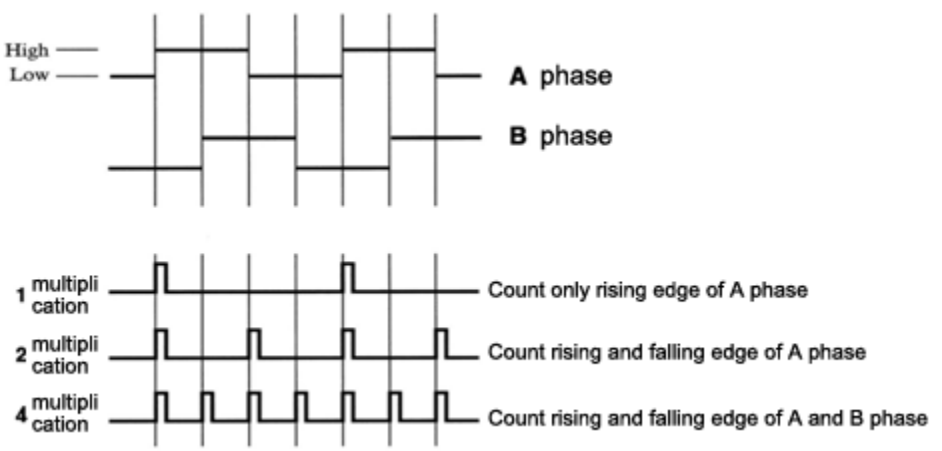
For example, let us assume that servo drive rotates servo motor to forward direction if a pulse is inputted on the PP pin and mechanical part moves a table with (−) direction, and vice versa if a pulse is inputted on the NP. When a command is given on the program, it has to be set that a pulse is outputted from DIR pin of motion controller and inputted to NP pin of servo drive with (+) command, and vice versa with (−) command. It is 2 pulse method because it runs based on a pulse that is inputted to each pin, and it requires to be set as CcwCw because a pulse is outputted on DIR pin with (+) command and outputted on PULS pin with (−) command. And it requires to be set as TwoCcwCwHigh because of setting with High Active.

```
// Pulse output method of axis 0 is to be TwoCwCcwHigh. Default : OneLowHighLow.
long lAxisNo = 0;
DWORD uMethod = TwoCwCcwHigh;
AxmMotSetPulseOutMethod(lAxisNo, uMethod);
```

4) Encoder Input Method

When servo motor rotates, pulse is outputted from encoder according to the rotational amount connected with drive axis. Generally, encoder outputs 3 types of pulses, A, B, and Z phase, and A and B phase has 90 degrees of phase difference. That is, if a motor rotates in forward direction, A phase is showed first and then a pulse on B phase is showed with 90 degrees delay. Z phase is a pulse that is generated whenever the motor rotates a turn. Rotational amount of the motor can have a feedback by counting those pulses.

Using A and B phase on the encoder, a signal which is a multiple of existing frequency can be generated, and this signal-obtaining method is called “multiplication.” A in the picture shows 2 multiplication and B shows 4 multiplication.



Generally 4 multiplication method is used, and it is set using AxmMotSetEncInputMethod API and setting values are 8 types as below.

Type	Explanation
ObverseUpDownMode	Forward direction Up/Down
ObverseSqr1Mode	Forward direction 1 multiplication
ObverseSqr2Mode	Forward direction 2multiplication
ObverseSqr4Mode	Forward direction 4multiplication
ReverseUpDownMode	Reverse direction Up/Down
ReverseSqr1Mode	Forward direction 1 multiplication
ReverseSqr2Mode	Forward direction 2 multiplication
ReverseSqr4Mode	Forward direction 4 multiplication

```
// Set 4 multiplication for the input method on the assigned axis.
long lAxisNo = 0;
DWORD uMethod = ObverseSqr4Mode;
AxmMotSetEncInputMethod(lAxisNo, uMethod);
```

```
// Ex2_AXM_SettingParameter.cpp : Defines the entry point for the console application.
// Initialize library, verify the information of axis and end.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion modules exists .\n");

           // Ordered value on axis 0 is to be mm unit.
           long lAxisNo = 0;
```

```
long lPulse = 10000;
double dUnit = 10;
AxmMotSetMoveUnitPerPulse(lAxisNo, dUnit, lPulse);
// Verify the set value.
AxmMotGetMoveUnitPerPulse(lAxisNo, &dUnit, &lPulse);
printf("UnitPerPulse setting => Unit : %d, Pulse : %f \n", dUnit, lPulse);
// Set Initial velocity on axis 0 by 1.  Default : 1.
double dMinVelocity = 1;
AxmMotSetMinVel(lAxisNo, dMinVelocity);
// Verify the set value.
AxmMotGetMinVel(lAxisNo, &dMinVelocity);
printf("MinVelocity setting => %2f \n", dMinVelocity);
// Pulse output method on axis 0 is to be TwoCwCcwHigh.
DWORD uMethod = TwoCcwCwHigh;
AxmMotSetPulseOutMethod(lAxisNo, uMethod);
// Verify the set value.
AxmMotGetPulseOutMethod(lAxisNo, &uMethod);
printf("PulseOutMethod setting => ");
switch(uMethod)
{
   case OneHighLowHigh:
      printf("OneHighLowHigh \n");
   break;
   case OneHighHighLow:
      printf("OneHighHighLow \n");
   break;
   case OneLowLowHigh:
      printf("OneLowLowHigh \n");
   break;
   case OneLowHighLow:
      printf("OneLowHighLow \n");
   break;
   case TwoCcwCwHigh:
      printf("TwoCcwCwHigh \n");
   break;
   case TwoCcwCwLow:
      printf("TwoCcwCwLow \n");
   break;
   case TwoCwCcwHigh:
      printf("TwoCwCcwHigh \n");
   break;
   case TwoCwCcwLow:
      printf("TwoCwCcwLow \n");
   break;
}

// Set 4 multiplication for the encoder input method on axis 0.
uMethod = ObverseSqr4Mode;
AxmMotSetEncInputMethod(lAxisNo, uMethod);
```

```
            // Verify the set value.
            AxmMotGetEncInputMethod(lAxisNo, &uMethod);
            printf("PulseOutMethod setting => ");
            switch(uMethod)
            {
               case ObverseUpDownMode:
                  printf("ObverseUpDownMode \n");
               break;
               case ObverseSqr1Mode:
                  printf("ObverseSqr1Mode \n");
               break;
               case ObverseSqr2Mode:
                  printf("ObverseSqr2Mode \n");
               break;
               case ObverseSqr4Mode:
                  printf("ObverseSqr4Mode \n");
               break;
               case ReverseUpDownMode:
                  printf("ReverseUpDownMode \n");
               break;
               case ReverseSqr1Mode:
                  printf("ReverseSqr1Mode \n");
               break;
               case ReverseSqr2Mode:
                  printf("ReverseSqr2Mode \n");
               break;
               case ReverseSqr4Mode:
                  printf("ReverseSqr4Mode \n");
               break;
            }
         }
         else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
               Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
      else
         printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

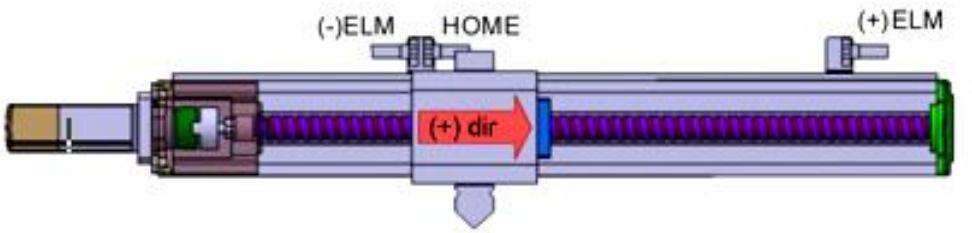
   // End Library.
   if (AxlClose())
      printf("Library is ended.\n");
   else
      printf("Library is not ended normally.\n");
}
```

Motion Signal

Signal is generally divided into motion signal, universal input signal, and universal output signal. Motion signal is structured with variety of sensors that indicate driving condition of mechanical part, and signals that show condition of servo drive. Universal signal is input signals that can be used by user arbitrarily, and universal output signal is output signals that can be used by user arbitrarily. Among the universal input/output signal, actual bit that can be used by user arbitrarily are 3 each of input and output, since super ordinate 2-bit is already used as a specific function on the motion controller.

1) Motion Signal

Motion signal is named for variety of sensors that indicate driving condition of mechanical part and signals that show condition of servo drive.



End Limit

It is a signal that is transferred to motion controller generated from a sensor which assigns transfer section of mechanical part. Showing on the picture below, it is divided into + and -, and installed on the both side. Generally, if limit signal is emitted during transfer of mechanical part, motion controller stops pulse output and mechanical part makes an emergency stop.

Inposition

It is an output signal of servo drive, and servo drive transfers inposition signal to motion controller when a position of servo motor is in the inposition range that is set on servo drive. In other words, it is a signal which is for verification whether it reached the command position or not.

Alarm

It is an output signal of servo drive, and servo drive transfers alarm signal to motion controller when an abnormal current is generated on servo motor or malfunction is occurred.

Emergency Stop

When this signal is inputted to motion controller, it stops pulse output immediately and mechanical part makes an emergency stop. This signal is usually connected with an emergency switch installed in equipment.

2) Universal Input Signal

It supports 6-bit universal input that can be utilized by user arbitrarily. User can use a signal of additional sensors using those universal input bit.

3) Universal Output Signal

It supports 6-bit universal output that can be utilized by user arbitrarily. User can apply to signal output or operation switch of actuator using those universal output bit.

Motion Signal Setting

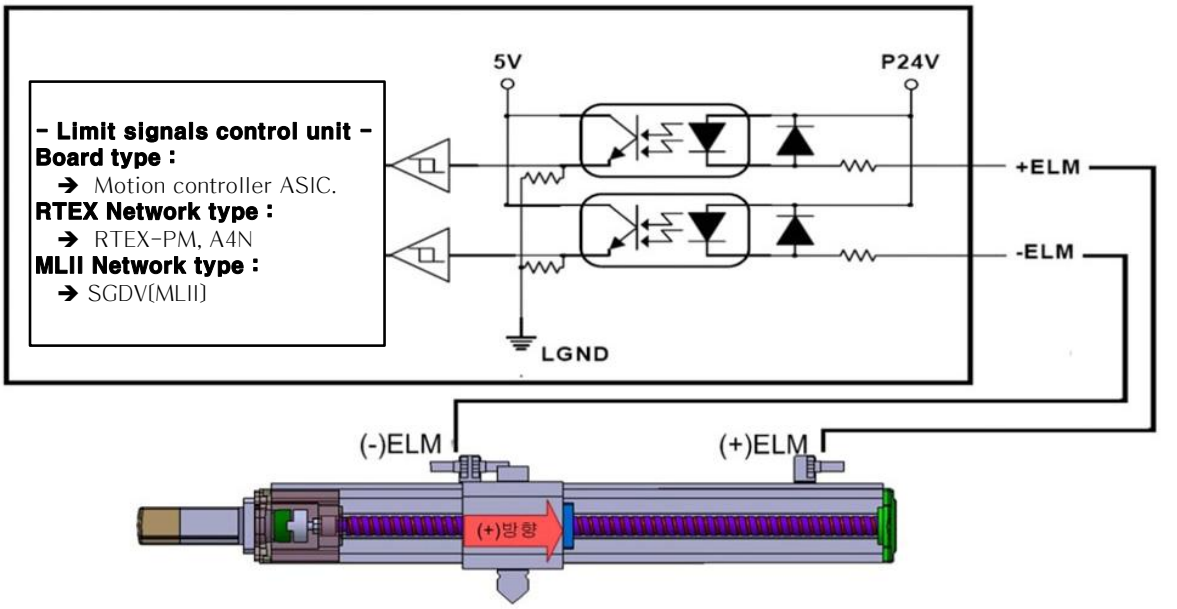
1) Signal Level and Setting for Using

Different signals are generated because of the characteristics of each mechanical part sensor, and active levels are set to meet the characteristics of those signals. There are LOW(0), HIGH(1), UNUSED(2), USED(3) for setting values, and if LOW or HIGH level is set, it senses as USED automatically. For example, if the internal signal of motion controller is HIGH when a sensor detects an object, it is set as ActiveHIGH. If it is set that each signal is not used, it does not care even when a signal is inputted. User needs to understand the existence of sensor installation of mechanical part before setting.

● *End Limit Signal*

Motion controller has an electrical insulation of external signal (24V) that comes from outside and internal signal (5V) using photo coupler to protect internal circuit.

Showing on the picture below, if the limit sensor transmits HIGH signal to B contact usually and detects transfer section and generates LOW signal, internal signal is to be HIGH as a result of reverse by photo coupler of motion controller. Therefore, this sensor has to be set as Active High.



End Limit sensor has two directions of positive(+) and negative(−) on the mechanical section. If user wants to set End Limit Level both Active High and to set to make an emergency stop when limit sensor is detected, follow below.

```
// Set active level of -End limit and +End limit on axis 0 as HIGH. Default : HIGH.
long lAxisNo = 0;
DWORD uStopMode = 0       // [0]Emergency stop, [1]Slowdown stop
DWORD uPositiveLevel = HIGH;
DWORD uNegativeLevel = HIGH;
AxmSignalSetLimit(lAxisNo, uStopMode, uPositiveLevel, uNegativeLevel);
```

● *Inposition Signal*

For the use of inposition signal and when setting Active Level, use AxmSignalSetInpos API. If it is needed to set Active Level of inposition signal to Active HIGH, follow below.

```
// Set input level of inposition signal on axis 0.      Default : HIGH.
long lAxisNo = 0;
DWORD uUse = HIGH;
AxmSignalSetInpos(lAxisNo, uUse);
```

● *Alarm Signal*

For the use of alarm signal and when setting Active Level, use AxmSignalSetServoAlarm API. If it is needed to set Active Level of alarm signal to Active LOW, follow below.

```
// Set input level of alarm signal on axis 0.                Default : HIGH.
long lAxisNo = 0;
DWORD uUse = LOW;
AxmSignalSetServoAlarm(lAxisNo, uUse);
```

● *Emergency Stop Signal*

For the use of emergency stop signal and when setting Active Level, use AxmSignalSetStop API. If it is needed to set Active Level of emergency stop signal to Active HIGH, follow below.

```
// Set input level of ESTOP on axis 0.
long lAxisNo = 0;
DWORD uStopMode = 0;    // [0] Emergency stop, [1] Slow-Down stop
DWORD uLevel = HIGH;
AxmSignalSetStop(lAxisNo, uStopMode, uLevel);
```

2) Signal Reading

● *Input Verification of End Limit Signal*

If it is needed to verify whether +/-End Limit signal is inputted or not when the mechanical part passes through End Limit sensor,use AxmSignalReadLimit API and return current state of End Limit signal to 0 or 1.

```
// Verify End Limit signal on axis 0.
long lAxisNo = 0;
DWORD uPositiveStatus, uNegativeStatus;
AxmSignalReadLimit(lAxisNo, &uPositiveStatus, &uNegativeStatus);
printf("+ELM : %x , -ELM : %x \n", uPositiveStatus, uNegativeStatus);
```

● *Input Verification of Inposition Signal*

If it is needed to verify whether inposition signal is inputted or not which is generated when the transfer part moved from motion controller to the commanded position, use AxmSignalReadInpos API.

```
// Use when verifying whether inposition signal is inputted or not.
long lAxisNo = 0;
DWORD uStatus;
AxmSignalReadInpos(lAxisNo, &uStatus);
printf("whether inposition on axis 0 exits or not : %x \n", uStatus);
```

● *Input Verification of Alarm Signal*

When the mechanical part does not move to the designated position and has malfunction, servo drive outputs alarm signal to motion controller. If it is needed to verify whether this signal is inputted or not, use AxmSignalReadServoAlarm API.

```
// Use when verifying whether alarm signal is inputted or not.
long lAxisNo = 0;
DWORD uStatus;
```

```
AxmSignalReadServoAlarm(lAxisNo, &uStatus);
printf("whether there is an alarm on axis 0 or not : %x \n", uStatus);
```

● *Input Verification of Emergency Stop Signal*

When the mechanical part stops operation urgently due to malfunction or abnormal operation, an emergency stop occurs. If it is needed to verify whether it is inputted or not, use AxmSignalReadStop API.

```
// Use when verifying whether emergency stop signal is inputted or not.
long lAxisNo = 0;
DWORD uStatus;
AxmSignalReadStop(lAxisNo, &uStatus);
printf("whether there is an emergency stop on axis 0 or not : %x \n", uStatus);
```

```
// Ex3_AXM_SettingMotionSignal.cpp:Defines the entry point for the console application.
// Set all sorts of signals of limit and alarm, and verify input state.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library is initialized.\n");

      // Inspect whether motion module exists or not.
      DWORD dwStatus;
      Code = AxmInfoIsMotionModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            printf("Motion module exists.\n");

            long lAxisNo = 0;

            // Pulse output method on axis 0 is to be TwoCwCcwHigh.
            // Default : OneLowHighLow.
            DWORD uMethod = TwoCcwCwHigh;
            AxmMotSetPulseOutMethod(lAxisNo, uMethod);
```

```
// Set Active level of -End limit and +End limit to HIGH.
// Default : HIGH.
DWORD uStopMode = 0;   // [0]Emergency Stop, [1]Slowdown Stop
DWORD uPositiveLevel = HIGH;
DWORD uNegativeLevel = HIGH;
AxmSignalSetLimit(lAxisNo, uStopMode, uPositiveLevel, uNegativeLevel);

// Set input level of inposition signal on axis 0.   Default : HIGH
DWORD uUse = HIGH;
AxmSignalSetInpos(lAxisNo, uUse);

// Set input level of alarm signal on axis 0.        Default : HIGH
uUse = LOW;
AxmSignalSetServoAlarm(lAxisNo, uUse);

// Set active input level of ESTOP on axis 0.
uStopMode = 0;          // [0] Emergency stop, [1] Slow-Down stop
DWORD uLevel = HIGH;
AxmSignalSetStop(lAxisNo, uStopMode, uLevel);

// help Message.
printf("[INFORMATION]*************** \n");
printf("[ESC] : Exit \n");
printf("**************************** \n\n");
printf("Verify input state of all sorts of signals \n\n");

BOOL fExit = FALSE;
DWORD uPositiveStatus, uNegativeStatus;
DWORD uInositionStatus, uAlarmStatus, uEStopStatus;

while (!fExit)          // Infinite loop.
{
   if (kbhit())         // Exit loop pressing any key.
   fExit = TRUE;

   // Verify End Limit signal on axis 0.
   AxmSignalReadLimit(lAxisNo, &uPositiveStatus, &uNegativeStatus);

   // Use when verifying whether inposition signal on axis 0 is inputted or not.
   AxmSignalReadInpos(lAxisNo, &uInositionStatus);

   // Use when verifying whether alarm signal is inputted or not.
   AxmSignalReadServoAlarm(lAxisNo, &uAlarmStatus);

   // Use when verifying whether emergency stop signal is inputted or not.
   AxmSignalReadStop(lAxisNo, &uEStopStatus);

   printf("\rPEnd_Limit(%x), NEnd_Limit(%x), Inposition(%x), Alarm(%x),
      EStop(%x)", uPositiveStatus, uNegativeStatus, uInositionStatus,
```

```
                uAlarmStatus, uEStopStatus);
            } // End of While()
        }
        else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
                Code);
    }
    else
        printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
    printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

  // End Library.
  if (AxlClose())
    printf("Library is ended.\n");
  else
    printf("Library is not ended normally.\n");
}
```

Universal Input/Output Signal Reading And Output Writing

In Motion Controller, user can optionally provide a general-purpose I/O. The user randomly uses general-purpose I/O signal (Universal Input/Output signal) as follows.

Product	Universal Input	Universal Output
SMC-2V04 PCI-N404/804 RTEX-PM	UIN(0) : Home Signal UIN(1) : Encoder Z phase Signal UIN(2~4) : Universal Input Signal	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2~4) : Universal Output Signal
RTEX-A4N	UIN(0) : Home Signal UIN(1~4) : Universal Input (EX-IN1~4)	UOUT(0) : Servo On/Off Signal UOUT(1) : Not Available UOUT(2~3) : Universal Output (EX-OUT1~2)
JEPMC-PL2910	UIN(0) : Home Signal (ZRNn Signal) UIN(1) : Not Available UIN(2) : Universal Input Signal (INn)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2) : Universal Output Signal (OUTn)
SGDV-xxxx11A	UIN(0) : Home Signal (EXT1 Signal) UIN(1) : Encoder Z phase(PC Signal) UIN(2) : +End limit(EXT2 Signal) UIN(3) : -End limit(EXT3 Signal)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal

State verification of universal input/output bit has a method that reads at once using AxmSignalReadInpos API or AxmReadOutput API, and that verifies the state of 5 bits from 0 to 4 individually using AxmSignalReadInputBit API or AxmSignalReadOutputBit API. And it returns 0 or 1 according to the signal state of the concerned bit.

```
// Verify the state of universal input bit signal of number 0 on axis 0.
long lAxisNo = 0;
long lBitNo = 0;
DWORD uOn;
AxmSignalReadInputBit(lAxisNo, lBitNo, &uOn);
printf("state of universal input bit signal of number 0 on axis 0 : %x",uOn);

// Verify the state of universal output bit signal of number 0 on axis 0.
AxmSignalReadOutputBit(lAxisNo, lBitNo, &uOn);
printf("state of universal output bit signal of number 0 on axis 0 : %x",uOn);
```

In order to use on/off of universal output signal, it can be set at once using AxmWriteOutput API, and can be set by bit unit using AxmSignalWriteOutputBit API. When universal output signal is HIGH, bit value is 1, and when is LOW, bit value is 0. In this manner, it is set bit of number 0~4 signal.

```
// Set universal output number 0 bit signal to 1.(Servo On).
long lAxisNo = 0;
long lBitNo = 0;
DWORD uOn = 1;
AxmSignalWriteOutputBit( lAxisNo, lBitNo, uOn);

// Set universal output number 0 bit signal to 0.(Servo Off).
```

```
uOn = 0;
AxmSignalWriteOutputBit( lAxisNo, lBitNo, uOn);
```

```
// Ex4_AXM_SettingUniversalIO.cpp:Defines the entry point for the console application.
// Set and verify universal input/output.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library is initialized.\n");

      // Inspect whether motion module exists or not.
      DWORD dwStatus;
      Code = AxmInfoIsMotionModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            printf("Motion module exists.\n");

            // help Message
            printf("[INFORMATION]*************** \n");
            printf("[ESC] : Exit \n");
            printf("[0 ~ 4] : universal output signal On / Off \n");
            printf("*************************** \n");
            long lAxisNo = 0;
            long lBitNo;
            DWORD uOn, uStatusInput[5], uStatusOutput[5];
            BOOL fExit = FALSE;
            while (!fExit)                          // Infinite loop.
            {
               if (kbhit())                         // Press any key.
               {
                  int ch = getch();
                  switch (ch)
                  {
                     case 27: fExit = TRUE;         // Esc key.
                     break;
```

```
                    case '0':   lBitNo = 0;
                    break;
                    case '1':   lBitNo = 1;
                    break;
                    case '2':   lBitNo = 2;
                    break;
                    case '3':   lBitNo = 3;
                    break;
                    case '4':   lBitNo = 4;
                    break;
                 }
                 AxmSignalReadOutputBit(lAxisNo, lBitNo, &uOn);
                 if(uOn == 1) {AxmSignalWriteOutputBit(lAxisNo, lBitNo, 0); }
                 else { AxmSignalWriteOutputBit(lAxisNo, lBitNo, 1); }
              }

              // State verification using API that verifies the information of individual
              // input/output bit.
              for(int nBitNo =0; nBitNo <5; nBitNo ++)
              {
                 AxmSignalReadInputBit(lAxisNo, nBitNo, &uStatusInput[nBitNo]);
                 AxmSignalReadOutputBit(lAxisNo, nBitNo, &uStatusOutput[nBitNo]);
              }

              printf("\rInput 0(%x), 1(%x),2(%x),3(%x),4(%x)/Output 0(%x),1(%x),2(%x),
                 3(%x),4(%x)", uStatusInput[0] , uStatusInput[1] , uStatusInput[2] ,
                 uStatusInput[3], uStatusInput[4],uStatusOutput[0],uStatusOutput[1],
                 uStatusOutput[2], uStatusOutput[3], uStatusOutput[4]);
           } // End of While()
        }
        else
           printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
              Code);
     }
     else
        printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
     printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

  // End Library.
  if (AxlClose())
     printf("\n Library is ended.\n");
  else
     printf("\n Library is not ended normally.\n");
}
```

Saving And Opening Setting Information

Every time when it is needed to draft program, it does not need to set every motion parameter and motion signal, and initialization work can be done easily by calling the setting file which is stored.

In order to save the information that is stored in the register of CAMC-IP, QI to file, use AxmMotSaveParaAll API. The API stores the information of all axis to file. If it is needed to store the setting information of all axis to SMC2V03.MOT file, follow this.

```
// Save setting information of all axis to SMC2V03.MOT file.
char *pFilename = Save_Para.MOT";
DWORD uOption = 0;
AxmMotSaveParaAll(pFilename);
```

In order to set the information that is stored in file to the register of CAMC-IP, QI, use AxmMotLoadParaAll API. The API reads the information of all axis. If it is needed to read the setting information of all axis that is stored in SMC2V03.MOT file, follow this.

```
// Read and set the setting information of all axis stored in SMC2V03.MOT file.
char *pFilename = " Save_Para.mot";
AxmMotLoadParaAll(pFilename);
```

Initial Value of Internal Variable That File Is Set

No	Signal Related Variable	Parameter Description	Setting Value
00	AXIS_NO	Axis number	Setting axis number
01	PULSE_OUT_METHOD	Setting of the pulse output method which is used as a servo pack order on the motion board	TwoCcwCwHigh
02	Enc[Encoder]	Setting of the input method of encoder signal	Sqr4Mode
03	InP[InPos]	Setting of the use of completion signal for servo pack position decision and Active Level	2
04	Alm[Alarm]	Setting of the use of servo pack alarm signal and Active Level	1
05	NLimit	Setting of the use of machine (-) limit sensor and Active Level	1
06	PLimit	Setting of the use of machine (+) limit sensor and Active Level	1
07	MinSpd	Setting of the initial drive velocity when the motor drives	1.0
08	MaxSpd	Setting of the maximum drive velocity when the motor drives	700000.0
09	HmSig	Setting of a signal to use as home sensor	4
10	HmLev	Setting of Active Level of home sensor	1
11	HmDir	Setting of the direction of initial search process during home sensor search	0
12	Zphas_Lev	Active level on encoder Z phase	1
13	Zphas_Use	Setting of the search of encoder Z phase after home sensor search	0

14	Stop Signal Mode	Mode when used ESTOP, SSTOP	0
15	Stop Signal level	Use level of ESTOP, SSTOP	0
16	VelFirst	Initial high speed search velocity during home search (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	10000.0
17	VelSecond	Velocity that comes out from the reverse direction after the 1st sensor search during home search	10000.0
18	VelThird	Velocity of research process after the 1st sensor search during home search	2000.0
19	VelLast	Setting of the final search velocity during home search [Accuracy decision of home search]	100.0
20	AccFirst	Initial high speed search acceleration during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
21	AccSecond	Acceleration that comes out from the reverse direction after the 1st sensor search during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
22	HClrTim	Waiting time before clear of Cmd, Act position after home search completion	1000.0
23	HOffset	Moving offset value during reset of machine home after home search completion	0.0
24	NSWLimit	Setting of (−) Software Limit value to apply after home search completion	0.0
25	PSWLimit	Setting of (+) Software Limit value to apply after home search completion	0.0
26	OenPulse	Setting of pulse number that requires for 1 rotation of motor [refer to servo pack electric gear ratio]	1.0
27	OneUnit	Moving amount that is moved for 1 rotation of motor [mm, degree, etc]	1
28	InitPosition	Position for initialization	200
29	InitVel	Velocity for the initialization (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	200
30	InitAccel	Acceleration for the initialization (acceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
31	InitDecel	Deceleration for the initialization (deceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
32	AbReMode	Absolute/relative mode for the initialization	0
33	ProfMode	Velocity profile mode for the initialization	4

Whenever program is written, it is easy to work initialization since there is no need to set all motion parameter and motion signal, and just call the setting file that is stored .

Basically motion related parameters are stored in MotorParaIP.mot file, but specified initial position, initial velocity, initial acceleration, initial deceleration, initial absolute and relative position mode, and initial profile mode are stored in file and used for the user's convenience. Those values can be set or called using AxmMotSetParaLoad or AxmMotGetParaLoad API. Especially, those values are loaded automatically when

other parameters initialize library, and used with store or call based on the necessity in contrast to the automatic store when library ends.

Parameter 28 – 31 can be set and brought using these APIs.

If you need to store to or callback from initial position, initial velocity, initial acceleration, and initial deceleration file for the specified axis, follow below.

```
// Specified initial position, initial velocity, initial acceleration, initial deceleration,
// initial absolute and relative position mode.
// Save initial profile mode to file.

long lAxisNo = 0;
double InitPos = 0;
double InitVel = 200;
double InitAccel = 400;
double InitDecel = 400;
AxmMotSetParaLoad(lAxisNo, InitPos, InitVel, InitAccel, InitDecel);
```

```
// Specified initial position, initial velocity, initial acceleration, initial deceleration,
// initial absolute and relative position mode.
// Call initial profile mode from file.

long lAxisNo = 0;
double InitPos, InitVel, InitAccel, InitDecel;
AxmMotGetParaLoad(lAxisNo, &InitPos, &InitVel, &InitAccel, &InitDecel);
printf("Axis %d Initial Position(%f), Initial Velocity(%f), Initial Acceleration(%f),
   Initial Deceleration(%f)", lAxisNo, InitPos, InitVel, InitAccel, InitDecel);
```

Motion Drive

Motion drive is to control a specific mechanical part that stopped or is moving using an intermediate of program. Motion drive can be divided into one-axis drive which drives one axis and multi-axis which drives more than two axes.

1) One Axis

● *Position Drive*

Position drive is the most general drive. This drive means, if the distance to move is given, movement with output of pulse that is the value calculated by velocity profile which is set on motion chip in motion module after required factors to move, such as moving velocity, acceleration/deceleration, or acceleration/deceleration time, are inputted.

This position drive is the mostly used drive API because general drive systems are designed to move the position that is already known repeatedly.

● *Velocity Drive*

Velocity drive is different from position drive which moves already assigned distance, and is a drive that moves with a constant velocity until stop command or stop signal is generated by limit sensor. This velocity drive API can be used for work that moves to (+) direction if (+) direction is pressed on jog button, and stops if the button is released.

● *Signal Search Drive*

Signal search drive is a drive that sets search target, such as limit signal or universal input/output signal previously, then moves with a constant velocity and stops when a set signal is detected. It can be used for work that is required to stop exactly on the position of end limit of home sensor of mechanical part.

● *Home Search Drive*

Home search drive means to search for home which is to be a datum of the coordination system. This drive API is provided to make home search easy by running an API after basic setting for home search.

2) Multi Axis

Generally, drive system has many cases that move several axes to a specified position at once, and for this drive, multi axis API is provided.

● *Position Drive*

This means a drive that moves with pulse output by the calculated value from motion chip in motion module

after receiving factors that are required for movement of moving velocity, acceleration/deceleration, or acceleration/deceleration time on 2 axes included in one module. This drive has a big difference from the drive using one axis drive API that two axes start to move at the same time.

● *Interpolation Drive*

This means a drive that moves from the current position to the designated position with interpolation along with a constant path of straight line or circular arc using 2 axes included in one module. Interpolation drive is generally divided into straight line interpolation drive and circular interpolation drive. Also there are general drive that follows the calculated path with velocity change of acceleration-constant velocity-deceleration starting from a point as well as continuous interpolation drive that decelerates lastly passing the designated many points without velocity change after starting from one point.

Glossary Of Motion Drive

1) Servo On

This means to turn On the servo drive, since output pulse that comes from motion card does not have influence on the actual drive if servo drive was not turned on, so servo drive must be turned On before drive.

First, ServoOn signal level has to be set using AxmSignalSetServoOnLevel API to turn ON/OFF servo drive. Most servo drives are Active HIGH, but some are not.

```
// Set ServoOn level to HIGH.
long lAxisNo = 0;
DWORD uLevel = HIGH;
AxmSignalSetServoOnLevel(lAxisNo, uLevel);
```

Next, AxmSignalServoOn API gives an on signal to servo drive and AxmSignalIsServoOn API verifies if servo drive is on or not.

```
// Turn on servo drive on axis 0.
long lAxisNo = 0;
DWORD uUse = ENABLE;
AxmSignalServoOn(lAxisNo, uUse);
```

```
// Turn off servo drive on axis 0.
long lAxisNo = 0;
DWORD uUse = DISABLE;
AxmSignalServoOn(lAxisNo, uUse);
```

```
// Verify that servo drive is on.
long lAxisNo = 0;
DWORD uUse;
AxmSignalIsServoOn(lAxisNo, &uUse);
if(uUse == ENABLE) { printf("Servo drive is on."); }
else if(uUse == DISABLE) { printf("Servo drive is off."); }
```

2) Velocity Profile

There is a process if a user give a command to move through motion drive API, mechanical part starts the drive with Start Stop Speed that is previously set, accelerates to the inputted acceleration, maintains a constant velocity when it reaches the inputted velocity, decelerates to the inputted acceleration, and then stops. Velocity profile indicates how it accelerates and decelerates with which shape of velocity change when ac/decelerated.

AXL supports trapezoid ac/deceleration, and S-curve ac/deceleration, and can make an asymmetrical profile that has different velocity when ac/decelerated. Therefore, 5 types of velocity profile below are provided, and have setting values from 0 to 4 each.

Acc/Dec Profile	Setting Value (ProfileMode)
Symmetrical trapezoid ac/deceleration	[0] SYM_TRAPEZOIDE_MODE
Asymmetrical trapezoid ac/deceleration	[1] ASYM_TRAPEZOIDE_MODE
Symmetrical Quasi-S Curve	[2] QUASI_S_CURVE_MODE
Symmetrical S curve ac/deceleration	[3] SYM_S_CURVE_MODE
Asymmetrical S curve ac/deceleration	[4] ASYM_S_CURVE_MODE

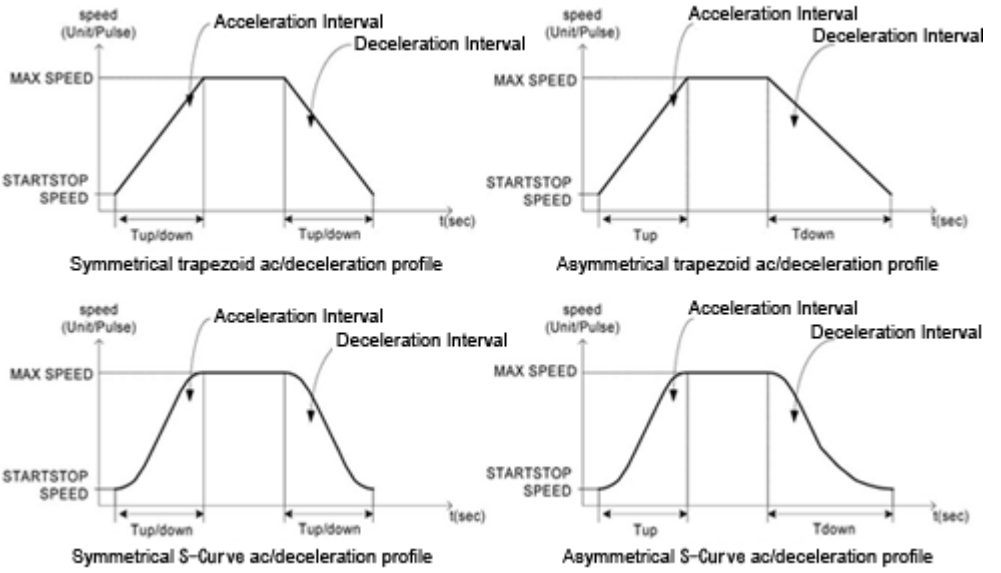
In library API, AxmMotSetProfileMode API is used to set the following.

```
// Make velocity profile on axis 0 to the symmetrical S curve ac/deceleration.
long lAxisNo = 0;
DWORD uProfileMode = 4;
AxmMotSetProfileMode(lAxisNo, uProfileMode);
```

To verify the current set value, do the following.

```
// API that reads set ac/deceleration profile.
long lAxisNo=0;
DWORD uProfileMode;
AxmMotGetProfileMode(lAxisNo, &upProfileMode);
switch(upProfileMode) {
   case 0 :  printf("current velocity profile : symmetrical trapezoid ac/deceleration");
   break;
   case 1 :  printf("current velocity profile : asymmetrical trapezoid ac/deceleration");
   break;
   case 2 :  printf("current velocity profile : symmetrical Quasi-S Curve");
   break;
   case 3 :  printf("current velocity profile : symmetrical S curve ac/deceleration");
   break;
   case 4 :  printf("current velocity profile : asymmetrical S curve ac/deceleration");
   break;
}
```

Following picture shows velocity profile of symmetrical trapezoid, asymmetrical trapezoid, symmetrical S curve, and asymmetrical S curve. On the picture, velocity change rate (acceleration) of S curve velocity profile has a smooth movement compared to trapezoid velocity profile.



3) Exit API At Start Point And Exit API At End Point

Exit API at start point and exit API at end point are that means to carry out the next command on a program after getting out of an API when a user calls the API on the program.

● *Exit API at Start Point*

This means to go on to the next command on a program as soon as the first pulse is outputted from motion chip. In other words, it means to exit an API as soon as pulse output is started. This method has an advantage that does not occupy CPU on PC and goes straight to the next command on program. This means PC can carry out other process while motion chip takes pulse output for motion control.

● *Exit API at End Point*

This method is to go on to the next command on program getting out of the API after outputting pulse that is corresponding to the command from motion chip. That is, until the end of drive, CPU on PC is occupied and it does not go on the next command on program.

Matters that require attention for pulse exit API at end point are to set whether getting out or not with inposition signal from servo drive to motion controller, and it is default that gets out without inposition signal. If a user needs more accurate control, it is recommended to use inposition signal.

4) Relative Motion Drive And Absolute Motion Drive

It is required to classify whether given drive command moves to the relative coordination or absolute

coordination. In the relative coordination system, it moves as much as the distance on the basis of current position for the distance that is given with API factor, and in the absolute coordination system, it moves as much as the distance which subtracts current position for the distance that is given. AxmMotSetAbsRelMode API is used to set, and must be used after 0 point is set conducting home search by all means because absolute coordination is structured with existence of 0 point in absolute position drive.

If relative coordination or absolute coordination is needed to be set, do as following.

```
// Set to absolute position drive.
long lAxisNo = 0;
DWORD uAbsRelMode = 0;    // [0] Absolute coordination, [1] Relative coordination
AxmMotSetAbsRelMode(lAxisNo, uAbsRelMode);
```

If current setting value needs to be verified, follow below.

```
// API which reads set moving mode.
long lAxisNo=0;
DWORD uAbsRelMode;
AxmMotGetAbsRelMode(lAxisNo, &uAbsRelMode);
if(uAbsRelMode == 0) printf("In drive of absolute position");
elseif(uAbsRelMode == 1) printf("In drive of relative position");
```

One-Axis Drive

Generally in motion drive, the basic drive is one-axis drive. One-axis drive is divided into position drive, velocity drive, signal search drive, and home search drive.

1) Position Drive

Position drive is a command to move the mechanical part to an assigned position. Axm PositionMove API and AxmMoveStartPos API are used for position drive. These two APIs make to move to the set distance or position with inputted velocity and acceleration. AxmMovePos API is an exit API at end point and carries out the next command on program after drive exits, and AxmMoveStartPos API is an exit API at start point and goes to the next command on program as soon as movement starts.

If AxmMovePos API which is an exit API at end point is to be used, follow below. As it is mentioned before, use API that sets coordination system and velocity profile separately, and set moving amount, velocity, and acceleration. In case of symmetrical profile drive as following, deceleration which is inputted is ignored and acceleration which is inputted is applied when accelerated.

```
// Move axis 0 to 100 position on the absolute coordination with symmetrical S curve drive
// mode (velocity 100, acceleration 200).
long lAxisNo = 0;
DWORD uAbsRelMode = 0;
DWORD uProfileMode = 3;
AxmMotSetAbsRelMode(lAxisNo, uAbsRelMode);
AxmMotSetProfileMode(lAxisNo, uProfileMode);

double dPosition = 100;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
AxmMovePos(lAxisNo, dPosition, dMaxVelocity, dMaxAccel, dMaxDecel);
```

If AxmMoveStartPos API which is an exit API at start point is need to be used, follow as below. API is carried out immediately after drive starts because of exit API at start point. If the next API is a drive API, the next drive command has to be given after drive exits verifying whether it is in drive or not using AxmStatusReadInMotion API by all means. This is because if it is in drive, all motion drive commands return errors and are not carried out. Here, AxmStatusReadInMotion API returns TRUE if it is in drive, and returns FALSE if not in drive.

```
// Move axis 0 to 100 position on the absolute coordination with asymmetrical S curve drive
// mode (velocity 100, acceleration 200, deceleration 100).
long lAxisNo = 0;
DWORD uAbsRelMode = 1;
DWORD uProfileMode = 4;
AxmMotSetAbsRelMode(lAxisNo, uAbsRelMode);
```

```
AxmMotSetProfileMode(lAxisNo, uProfileMode);

double dPosition = 100;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 100;
AxmMoveStartPos(lAxisNo, dPosition, dMaxVelocity, dMaxAccel, dMaxDecel);

DWORD uStatus;
AxmStatusReadInMotion(lAxisNo, &uStatus);
while(uStatus){
  AxmStatusReadInMotion(lAxisNo, &uStatus);
}
```

```
// Ex5_AXM_PositionMove.cpp : Defines the entry point for the console application.
// Move as much as 100 to position drive API and move again as much as -100, and exit.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
                  if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           // Set Active level of +End limit and -End limit on axis 0 to HIGH.
           AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
           // Set input level of inposition signal on axis 0 to HIGH.
           AxmSignalSetInpos(AXIS_0, HIGH);
           // Set input level of alarm signal on axis 0 to LOW.
```

```
        AxmSignalSetServoAlarm(AXIS_0, LOW);
        // Set Active input level of ESTOP signal on axis 0 to HIGH.
        AxmSignalSetStop(AXIS_0, 0, HIGH);

        // Make ordered value on axis 0 to be mm unit.
        AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
        // Set initial velocity on axis 0 to 1.  Default : 1
        AxmMotSetMinVel(AXIS_0, 1);
        // Make pulse output method on axis 0 to TwoCwCcwHigh.
        AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
        // Set encoder input method on designated axis to 4 multiplication.
        AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

        // Set current position to be home.
        AxmStatusSetActPos(AXIS_0, 0.0);
        AxmStatusSetCmdPos(AXIS_0, 0.0);

        // Servo On.
        AxmSignalServoOn(AXIS_0, ENABLE);

        printf("Drive symmetrical S curve of axis 0 up to 100 position with absolute
           coordination (velocity 100, acceleration 200, exit API at end point)\n");
        printf("Drive starts pressing any key.\n");
        getch();
        AxmMotSetAbsRelMode(AXIS_0, 0);
        AxmMotSetProfileMode(AXIS_0, 3);
        AxmMovePos(AXIS_0, 100, 100, 200, 200);

        printf("Drive asymmetrical S curve of axis 0 up to -100 position with relative
           coordination (velocity 100, acceleration 200, exit API at start point)\n");
        printf("Drive starts pressing any key.\n");
        getch();
        AxmMotSetAbsRelMode(AXIS_0, 1);
        AxmMotSetProfileMode(AXIS_0, 4);
        AxmMoveStartPos(AXIS_0, -100, 100, 200, 200);

        // Wait if it is in drive currently.
        DWORD uStatus;
        AxmStatusReadInMotion(AXIS_0, &uStatus);
        while(uStatus) {
           AxmStatusReadInMotion(AXIS_0, &uStatus);
        }
     }
     else
        printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
           Code);
  }
  else
     printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
```

```
    }
    else
        printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

    // Exit library.
    if (AxlClose())
        printf("Library is exited.\n");
    else
        printf("Library is not exited normally.\n");
}
```

2) Velocity Drive

Velocity drive is a type of drive that moves continuously until stop command is given with fixed velocity, acceleration, and acceleration time. Also, velocity drive API is a pulse exit API at start point that gets out of start point which starts pulse. For the velocity drive, AxmMoveVel API is used, and the API is a command to move with a specified straight velocity. Drive direction is determined by the symbol of velocity value.

```
// Start drive with symmetrical S curve velocity profile of axis 0 to (+) direction with
// velocity of 100.
long lAxisNo = 0;
DWORD uProfileMode = 3;
AxmMotSetProfileMode(lAxisNo, uProfileMode);

double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
AxmMoveVel(lAxisNo, dMaxVelocity, dMaxAccel, dMaxDecel);
```

```
// Start drive with symmetrical trapezoid velocity profile of axis 0 to (-) direction with
// velocity of 100.
long lAxisNo = 0;
DWORD uProfileMode = 0;
AxmMotSetProfileMode(lAxisNo, uProfileMode);

double dMaxVelocity = -100;
double dMaxAccel = 200;
double dMaxDecel = 200;
AxmMoveVel(lAxisNo, dMaxVelocity, dMaxAccel, dMaxDecel);
```

Like this, if velocity drive API is started, it drives until stop command is given. Stop of drive will be discussed in detail later, so do as follows.

```
// Make axis 0 drive to stop with deceleration of set deceleration.
long lAxisNo = 0;
AxmMoveSStop(lAxisNo);
```

```
// Ex6_AXM_VelocityMove.cpp : Defines the entry point for the console application.
// Drive by using velocity drive API and exit by stop command.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           // Set Active level of +End limit and -End limit on axis 0 to HIGH.
           AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
           // Set input level of inposition signal on axis 0 to HIGH.
           AxmSignalSetInpos(AXIS_0, HIGH);
           // Set input level of alarm signal on axis 0 to LOW.
           AxmSignalSetServoAlarm(AXIS_0, LOW);
           // Set Active input level of ESTOP signal on axis 0 to HIGH.
           AxmSignalSetStop(AXIS_0, 0, HIGH);

           // Make ordered value on axis 0 to be mm unit.
           AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
           // Set initial velocity on axis 0 to 1.  Default : 1
           AxmMotSetMinVel(AXIS_0, 1);
           // Make pulse output method on axis 0 to TwoCwCcwHigh.
           AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
           // Set encoder input method on designated axis to 4 multiplication.
           AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

           // Set current position to be home.
```

```
AxmStatusSetActPos(AXIS_0, 0.0);
AxmStatusSetCmdPos(AXIS_0, 0.0);

// Servo On.
AxmSignalServoOn(AXIS_0, ENABLE);

// help Message.
printf("[INFORMATION]*************** \n");
printf("[ESC] : Exit after Servo Off \n");
printf("[1] : Start velocity drive to (+) direction \n");
printf("[2] : Start velocity drive to (-) direction \n");
printf("[3] : STOP \n");
printf("*************************** \n");
DWORD uStatus;
BOOL fExit = FALSE;
while (!fExit)          // Infinite loop.
{
   if (kbhit())         // Press any key.
   {
      int ch = getch();
      switch (ch)
      {
         case 27:       // Esc key.
            fExit = TRUE;
            AxmSignalServoOn(AXIS_0, DISABLE); // Servo Off.
         break;

         case '1':
         // If not in drive, drive continuously symmetrical trapezoid velocity
         // profile to (+) direction.
            AxmStatusReadInMotion(AXIS_0, &uStatus);
            if(!uStatus)
            {
               AxmMotSetProfileMode(AXIS_0, 0);
               AxmMoveVel(AXIS_0, 100, 200, 200);
            }
         break;

         case '2':
         // If not in drive, drive continuously symmetrical trapezoid velocity
         // profile to (-) direction.
            AxmStatusReadInMotion(AXIS_0, &uStatus);
            if(!uStatus)
            {
               AxmMotSetProfileMode(AXIS_0, 3);
               AxmMoveVel(AXIS_0, -100, 200, 200);
            }
         break;
```

```
                    case '3':   // If in drive, make slow down stop.
                       AxmStatusReadInMotion(AXIS_0, &uStatus);
                       if(uStatus)
                          AxmMoveSStop(AXIS_0);
                       break;
                    }
                 }
              } // End of While()
           }
           else
              printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
                 Code);
        }
        else
           printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
     }
     else
        printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

     // Exit library.
     if (AxlClose())
        printf("Library is exited.\n");
     else
        printf("Library is not exited normally.\n");
}
```

3) Signal Search Drive

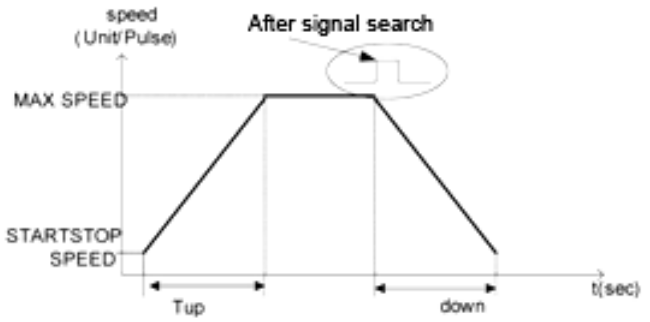
Signal search drive is a drive that sets in advance search target such as limit signal or universal input/output signal, and that stops when a set signal is detected moving with a constant velocity. Signal search drive has AxmMoveSignalSearch API that decelerates or makes an emergency stop when a specified signal is detected, and AxmMoveSignalCapture API that stores the position when a signal is detected. There are 8 signals that can be detected showing on the table below, each signal has Up Edge and Down Edge, so there are total of 16 signals.

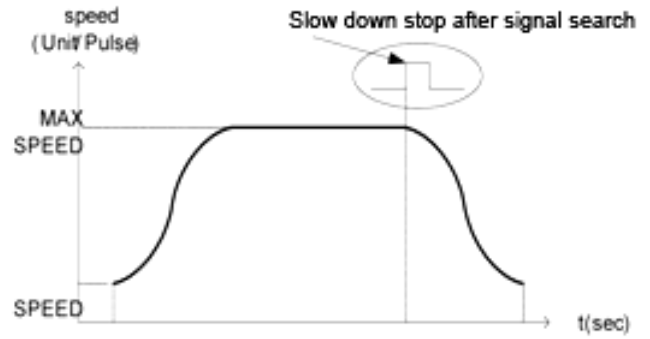
Define	Value	Search Signal
PosEndLimit	0x0	+Elm(End limit) + direction limit sensor signal
NegEndLimit	0x1	-Elm(End limit) - direction limit sensor signal
PosSloLimit	0x2	+Slm(Slow Down limit) signal - Not in used
NegSloLimit	0x3	-Slm(Slow Down limit) signal - Not in used
HomeSensor	0x4	IN0(ORG) home sensor signal
EncodZPhase	0x5	IN1(Z phase) Encoder Z phase signal
UniInput02	0x6	IN2(universal) universal input number 2 signal
UniInput03	0x7	IN3(universal) universal number 3 signal

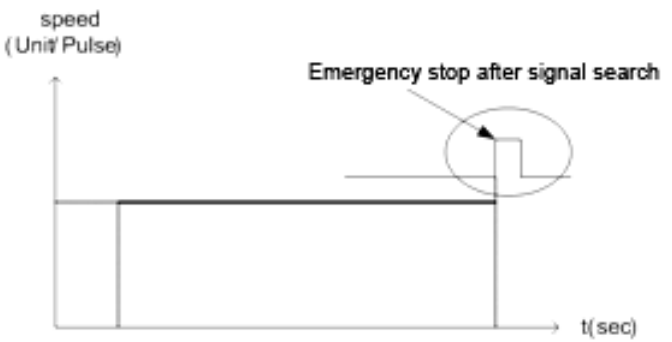
For the signal search drive, AxmMoveSignalSearch API is used, and this API is an API that makes an emergency stop or slow down stop with search of a specified input signal edge, has input of edge direction of input signal to search and input signal that is selected, drive direction, velocity and acceleration, and stop method. Drive direction is determined by the symbol of velocity value. Especially, since this API starts deceleration after corresponding signal is detected, in order to search a correct signal position, it has to be used that makes an emergency stop at the corresponding signal searching signal with low velocity and moving to the reverse direction after slow down stop in signal search drive.

```
// Search Up Edge of home sensor moving to - direction.
long lAxisNo = 0;
double dMaxVelocity = -100;
double dMaxAccel = 200;
long lDetectSignal = HomeSensor;
long lSignalEdge = UP_EDGE;
long lSignalMethod = 0;         // [0]Slowdown Stop, [1]Emergency Stop.
AxmMoveSignalSearch(lAxisNo, dMaxVelocity, dMaxAccel, lDetectSignal, lSignalEdge,
  lSignalMethod);

// Search Down Edge of home sensor moving to + direction, and make an emergency stop.
double dMaxVelocity = 20;
double dMaxAccel = 0;
long lDetectSignal = HomeSensor;
long lSignalEdge = DOWN_EDGE;
long lSignalMethod = 1;         // [0]Slowdown Stop, [1]Emergency Stop.
AxmMoveSignalSearch(lAxisNo, dMaxVelocity, dMaxAccel, lDetectSignal, lSignalEdge,
  lSignalMethod);
```







If it is simply needed not only to stop at the position that is detected signal but to verify the position of corresponding signal, use AxmMoveSignalCapture API. If this API is called, it starts to search a corresponding signal, and if signal is detected, it stores the corresponding position, and the stored position can be verified using AxmMoveGetCapturePos API. Stored position is one of Command Position or Actual Position API, and the distance can be verified using the home point.

```
// Search Up Edge of +ELM moving to + direction.
long lAxisNo = 0;
double dMaxVelocity = 100;
double dMaxAccel = 200;
long lDetectSignal = PosEndLimit;
long lSignalEdge = UP_EDGE
long lTarget = 0;          // [0] Command Position, [1] Actual Position
long lSignalMethod = 0     // [0] Slowdown Stop,  [1] Emergency Stop
```

```
AxmMoveSignalCapture(lAxisNo, dMaxVelocity, dMaxAccel, lDetectSignal, lSignalEdge, lTarget,
  lSignalMethod);

// Verify +ELM sensor position.
double dCapPotition;
AxmMoveGetCapturePos(lAxisNo, &dCapPotition);
printf("+ELM sensor position : %f", dCapPotition);
```

```
// Ex7_AXM_SignalSearchDrive.cpp : Defines the entry point for the console application.
// Move to limit sensor and home sensor using signal search drive API.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");
           // Set Active level of +End limit and -End limit on axis 0 to HIGH.
           AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
           // Set input level of inposition signal on axis 0 to HIGH.
           AxmSignalSetInpos(AXIS_0, HIGH);
           // Set input level of alarm signal on axis 0 to LOW.
           AxmSignalSetServoAlarm(AXIS_0, LOW);
           // Set Active input level of ESTOP signal on axis 0 to HIGH.
           AxmSignalSetStop(AXIS_0, 0, HIGH);

           // Make ordered value on axis 0 to be mm unit.
           AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
           // Set initial velocity on axis 0 to 1.  Default : 1
           AxmMotSetMinVel(AXIS_0, 1);
```

```
            // Make pulse output method on axis 0 to TwoCwCcwHigh.
            AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
            // Set encoder input method on designated axis to 4 multiplication.
            AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

            // Set current position to be home.
            AxmStatusSetActPos(AXIS_0, 0.0);
            AxmStatusSetCmdPos(AXIS_0, 0.0);
            // Servo On.
            AxmSignalServoOn(AXIS_0, ENABLE);

            // help Message.
            printf("[INFORMATION]*************** \n");
            printf("[ESC] : Exit \n");
            printf("[1] : Trapezoid slow down stop after moving to + direction until Up Edge
               of +ELM sensor is detected \n");
            printf("[2] : S-curve slow down stop after moving to - direction until Up Edge
               of -ELM sensor is detected \n");
            printf("[3] : Emergency stop after moving to + direction until IN/OUT5(ORG)
               DownEdge is detected \n");
            printf("*************************** \n");

            DWORD uPositiveStatus, uNegativeStatus, uHomeStatus;
            BOOL fExit = FALSE;
            while (!fExit)                                  // Infinite loop.
            {
               if (kbhit())                                 // Press any key.
               {
                  int ch = getch();
                  switch (ch)
                  {
                     case 27:                               // Esc key.
                        fExit = TRUE;
                        AxmSignalServoOn(AXIS_0, DISABLE); // Servo Off.
                     break;

                     case '1':
                        // Trapezoid drive with velocity of 100 and acceleration and
                        // deceleration of 200 on axis 0,
                        // Slow down stop after driving to + direction until Up Edge
                        // of +ELM sensor is detected.
                        AxmMotSetProfileMode(AXIS_0, 0);
                        AxmMoveSignalSearch(AXIS_0, 100, 200, PosEndLimit, UP_EDGE,0);
                     break;

                     case '2':
                        // S-curve drive with velocity of 100 and acceleration and
                        // deceleration of 200 on axis 0,
                        // Slow down stop after driving to - direction until Up Edge
```

```
                        // of -ELM sensor is detected.
                        AxmMotSetProfileMode(AXIS_0, 3);
                        AxmMoveSignalSearch(AXIS_0, -100, 200, NegEndLimit, UP_EDGE,0);
                     break;

                     case '3':
                        // Emergency stop after driving to + direction until Down
                        // Edge of home sensor is detected with velocity of 20 on axis 0.
                        AxmMoveSignalSearch(AXIS_0, 20, 0, HomeSensor, DOWN_EDGE,1);
                     break;
                  }
               }
               // Verify signal state of each sensor.
               AxmSignalReadLimit(AXIS_0, &uPositiveStatus, &uNegativeStatus);
               AxmSignalReadInputBit(AXIS_0, 0, &uHomeStatus);
               printf("\rSignal State : P_Elm(%d),N_Elm(%d),Home(%d)", uPositiveStatus,
                  uNegativeStatus, uHomeStatus);
            } // End of While()
         }
         else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
               Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("\n Library is exited.\n");
   else
      printf("\n Library is not exited normally.\n");
}
```

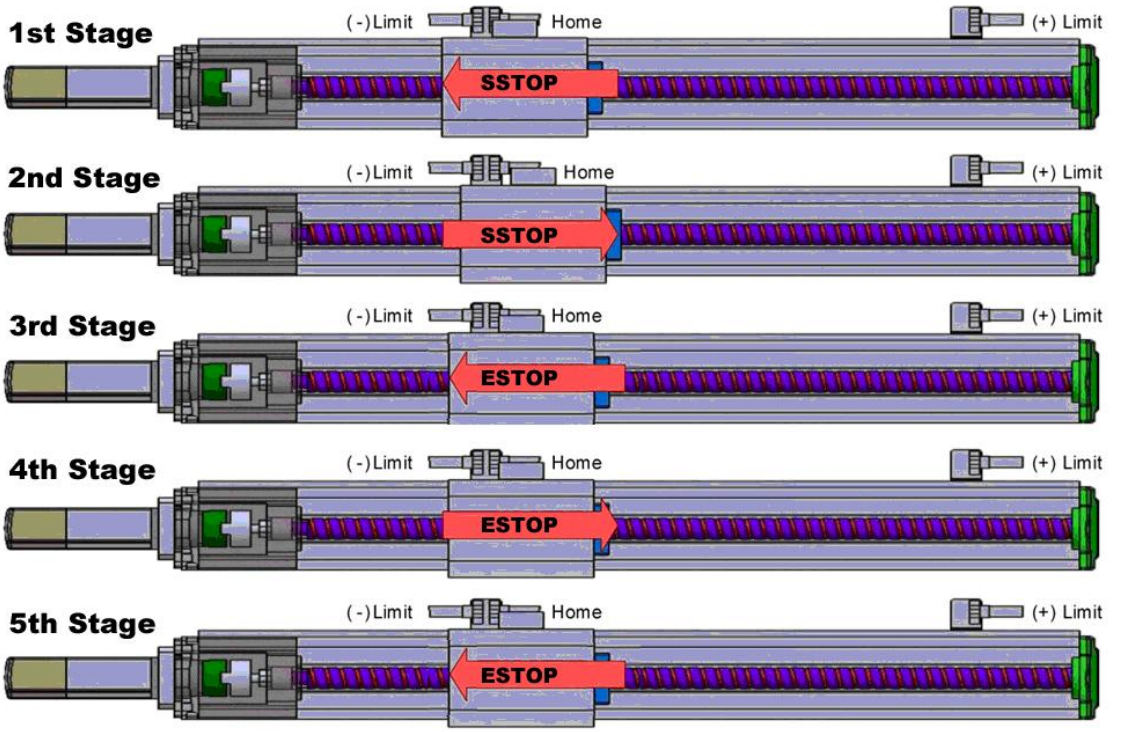
4) Home Search Drive

Even if the work for motion drive is ready with procedure of hardware initialization, parameter setting, and servo on, the state is that only the direction of coordination system is set and home which is to be a datum of coordination system is not set yet. Accurate home has to be ensured for the movement of mechanical part to a specified position which is a user designated. Home search method generally has two methods that are using signal search drive API and home search API. In order to carry out home search using signal search, since quite troublesome works are carried out, home search API is provided to make easier those process. For the use of home search API, set required data for home search using AxmHomeSetMethod and AxmHomeSetMethod API, and start home search calling AxmHomeSetStart API.

Since home search API is operated generating individual threads, if home search of various axes has to be carried out, use the API listing each home search command.

● *Home Search Method and Velocity Setting*

Home search generates various signal search order by mechanical part. But home search API of AXL has 7 stage of home search sequence below, and if home search sequence is compared with a factor of API for home search, it shows on the picture below.



1st Stage : slow down stop after searching Rising Edge signal of home sensor moving to - direction (if home sensor is already in HIGH state, omit 1st stage)

2nd Stage : slow down stop after searching Falling Edge signal of home sensor moving to + direction

3rd Stage : emergency stop after searching Rising Edge signal of home sensor moving to - direction

< In case of searching Z phase >

4th Stage : emergency stop after searching Falling Edge signal of Z phase moving to + direction

5th Stage : emergency stop after searching Rising Edge signal of Z phase moving to - direction

< In case of searching Home sensor >

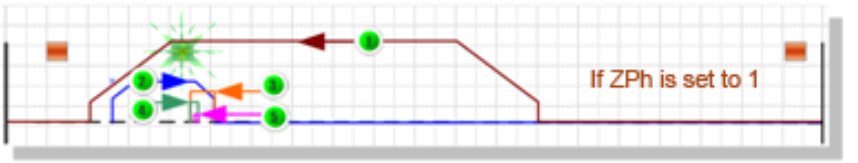
4th Stage : emergency stop after searching Falling Edge signal of home sensor moving to + direction

5th Stage : emergency stop after searching Rising Edge signal of home sensor moving to - direction

6th Stage : move additionally by Offset value if it exists

7th Stage : set to 0 after waiting for a while until vibration is decreased before setting to 0

● *Understanding of Home Search Velocity Setting by Step*



① Use of VelFirst, AccFirst

High speed search of home sensor to HmDir direction and slow down stop with HAccF deceleration

② Use of VelSecond, AccSecond

Search Down Edge of home sensor to HmDir reverse direction and slow down stop with HAccF deceleration

③ Use of VelThird

Search Up Edge of home sensor to HmDir direction and emergency stop

④ Use of VelThird

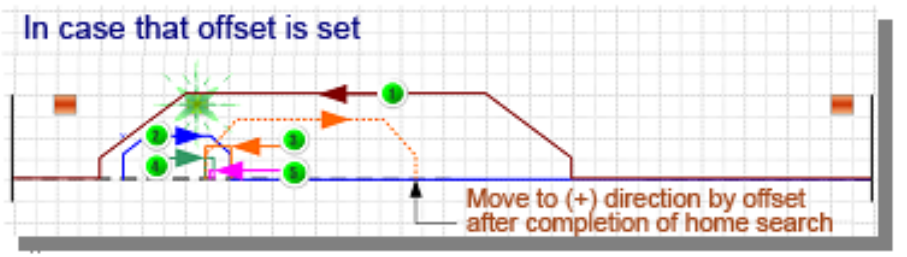
Search Down Edge on Z phase to HmDir reverse direction and emergency stop

⑤ Use of VelLast

Search Up Edge on Z phase to HmDir direction and emergency stop

● *Understanding of Offset Value Setting*

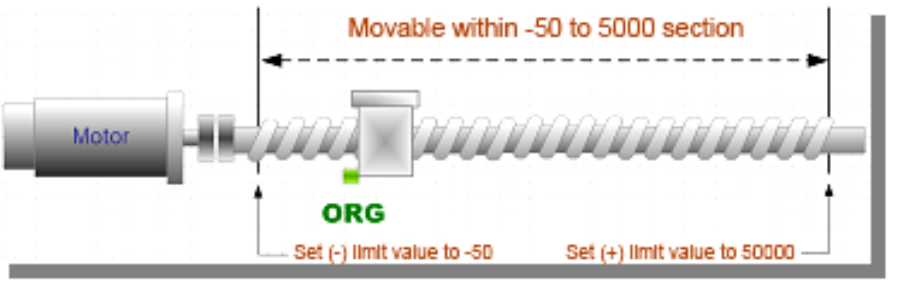
If it needs to be set home with movement to a specified position after completion of mechanical home sensor search, and ia value is set on HOffset, home will be reset after completion of movement by corresponding offset automatically after completion of home sensor search. Here, be careful to set offset within the range of (+) or (-)Limit. Otherwise, home search can not be carried out regularly.



It will be set to 0 point automatically after completion of offset movement

● *Understanding of Software Limit Value Setting*

If Soft Limit function is already set, Soft Limit will be re-set after completion of home search regularly. Soft Limit is used if Limit sensor is not installed mechanically or Limit needs to be changed by condition. Here, be careful that Soft Limit is set regularly when PLimit(+) value is bigger than NLimit(-). If NSwLimit value is equal to or bigger than PSwLimit, Soft Limit will not be set.



● *Variable Description of Home Search API*

Variable of Home Search	Explanation
HmSig	Set a signal to be used as home sensor Example) Home sensor(4), +Limit sensor(0), etc.
HmLev	Set an active level of Home Sensor Example) 0: A contact, 1: B contact
HmDir	Set initial search progress direction when home sensor detects Example) 1: (+)direction, 0: (-)direction
Zphas	Set whether detected of encoder Z phase after home sensor detect Example) 1 : To use, 0 : Not to use Use to increase home search velocity and accuracy by, on the 4th step,

	searching Down Edge of Z phase and, on the 5th step, researching Up Edge of Z phase
VelFirst	Initial high speed search velocity durign home search (if initial home sensor did not detected)
VelSecond	Velocity coming out from the reverse direction after 1st sensor search during home search
VelThird	Velocity progressing research after 1st sensor search during home search
VelLast	Final search velocity setting [determination of home search accuracy] during home search
AccFirst	Initial high speed search acceleration during home search
AccSecond	Acceleration coming out from the reverse direction after 1st sensor search during home search
HClrTim	Waiting time before clear of Command, Actual(Encoder) position after completion of home search
HOffset	Moving offset value during mechanical home reset after completion of home search

Note : When the level is not set correctly, it can operate to + direction even though it is set to - direction, and it may be a problem to find home.

```
// API that sets home search method.
long lAxisNo = 0;               // Axis number that home search will be carried out.
long nHmDir = 0;                // Moving direction of home search 1st, input (-)direction.
DWORD uHomeSignal = HomeSensor; // Use signals from signal search drive.
DWORD uZphas = 1;               // Whether Z phase detected, detect if 1, do not if 0.
double dHomeClrTime = 2000.0;   // Waiting time to set home search Enc value double.
dHomeOffset = 0.0;              // Additional movement by offset value after home.
AxmHomeSetMethod(lAxisNo,nHmDir,uHomeSignal,uZphas,dHomeClrTime,dHomeOffset);

// Set velocity that will be used during home search
double dVelFirst = 100;         // Velocity in 1st stage.
double dVelSecond = 20;         // Velocity in 2nd stage.
double dVelThird = 10;          // Velocity in 3rd stage.
double dvelLast = 10;           // Velocity for index search and accurate search (apply when
                                // Offset value moves).
double dAccFirst = 200;         // Acceleration in 1st stage.
double dAccSecond = 40;         // Acceleration in 2nd stage.
AxmHomeSetVel(lAxisNo, dVelFirst, dVelSecond, dVelThird, dvelLast, dAccFirst, dAccSecond);
```

Here, velocity of home search can be high speed in 1st stage, but it has to be a very small value in 2nd and 3rd stage because accurate work is carried out. Also, acceleration can be selected properly in 1st stage, but it is desirable to set quite big value in 2nd stage.

Sometimes, home search sequence that is provided above based on the state of mechanical part cannot be used, and in this case, make and use home search sequence that meets its own mechanical part using signal search drive API.

● *Start Home Search Drive*

So far set values that need to be set for home search are mentioned, but AxmHomeSetStart API is to carry out home search actually. If this API is called, home search is started on the assigned axis by setting value that is set previously.

```
// Verify progress rate of home search on axis 0.
long lAxisNo = 0;
AxmHomeSetStart(lAxisNo);
```

● *Verification of Home Search Progress Rate*

Once home search is carried out, progress rate of current home search can be verified. Progress rate is verified using AxmHomeGetRate API, and if it passes a specified stage, value is increased, and if it becomes 100 starting from 0, it means home search is exited.

```
// Verify progress rate of home search on axis 0.
long lAxisNo = 0;
DWORD uHomeMainStepNumber, uHomeStepNumber;
AxmHomeGetRate(lAxisNo, &uHomeMainStepNumber , &uHomeStepNumber);
printf("current progress rate : %d ", uHomeStepNumber);
// uHomeMainStepNumber is used to verify whether master, slave operates or not, in case of
// gantry.
// In case of master axis movement, it returns 0, and in case of slave movement, returns 10.
```

● *Completion and Stop of Home Search*

If home search API is carried out, home search is carried out making a new thread. If home search is exited, the result of home search needs to be returned. To verify the result of home search, AxmHomeGetResult API is used and the result below is returned.

```
// Return the result of home search.
long lAxisNo = 0;
DWORD uHomeResult;
AxmHomeGetResult(lAxisNo, &uHomeResult);
```

Define	Explanation
HOME_ERR_UNKNOWN	When home search is started with an unknown axis number
HOME_ERR_GNT_RANGE	If home search result of master and slave axis of gantry drive axis is off OffsetRange that is set
HOME_ERR_USER_BREAK	If user carried out stop command during home search
HOME_ERR_VELOCITY	If home search velocity setting is not set
HOME_ERR_AMP_FAULT	If servo pack alarm is occurred during home search
HOME_ERR_NEG_LIMIT	If (-) limit sensor is detected during home sensor search with (+) direction
HOME_ERR_POS_LIMIT	If (+) limit sensor is detected during home sensor search with (-) direction

HOME_ERR_NOT_DETECT	If home sensor is not detected
HOME_SEARCHING	If home search is in progress currently
HOME_SUCCESS	If home search is exited successfully

5) Stop Of Motion Drive

Stop of motion, which is the final stage of motion drive, has emergency stop, slow down stop, and stop method with specified deceleration. It stops motion using each API of AxmMoveEStop for emergency stop realization, AxmMoveSStop for slow down stop realization, and AxmMoveStop for stop with specified deceleration. Particularly, AxmMoveStop is to be set deceleration, and if this API is called, existing inputted deceleration is to be ineffective and it stops with newly inputted deceleration. If the deceleration is to be 0, it has same function with AxmMoveEStop, and if it is set with original deceleration, has same function with AxmMoveSStop.

```
// Make an emergency stop of motion drive on axis 0.
long lAxisNo = 0;
AxmMoveEStop(lAxisNo);
```

```
// Make slow down stop of motion drive on axis 0.
long lAxisNo = 0;
AxmMoveSStop(lAxisNo);
```

```
// Stop motion drive with deceleration of 100 on axis 0.
long lAxisNo = 0;
double dMaxDecel = 100;
AxmMoveStop(lAxisNo, dMaxDecel);
```

```
// Ex8_AXM_HomeSearchDrive.cpp : Defines the entry point for the console application.
// Carry out home search using home search API.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
```

```
{
   printf("Library is initialized.\n");

   // Inspect whether motion module exists or not.
   DWORD dwStatus;
   Code = AxmInfoIsMotionModule(&dwStatus);
   if(Code == AXT_RT_SUCCESS)
   {
      if(dwStatus == STATUS_EXIST)
      {
         printf("Motion module exists.\n");

         // Set Active level of +End limit and -End limit on axis 0 to HIGH.
         AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
         // Set input level of inposition signal on axis 0 to HIGH.
         AxmSignalSetInpos(AXIS_0, HIGH);
         // Set input level of alarm signal on axis 0 to LOW.
         AxmSignalSetServoAlarm(AXIS_0, LOW);
         // Set Active input level of ESTOP signal on axis 0 to HIGH.
         AxmSignalSetStop(AXIS_0, 0, HIGH);

         // Make ordered value on axis 0 to be mm unit.
         AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
         // Set initial velocity on axis 0 to 1.  Default : 1
         AxmMotSetMinVel(AXIS_0, 1);
         // Make pulse output method on axis 0 to TwoCwCcwHigh.
         AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
         // Set encoder input method on designated axis to 4 multiplication.
         AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

         // Servo On.
         AxmSignalServoOn(AXIS_0, ENABLE);

         // Set home search method.
         long lAxisNo = 0;
         // Axis number that home search will be carried out.
         long nHmDir = 0;
         // Moving direction of home search 1st, input (-)direction.
         DWORD uHomeSignal = HomeSensor;
         // Use signals from signal search drive.
         DWORD uZphas = 1;
         // Whether Z phase detected, detect if 1, do not if 0.
         double dHomeClrTime = 2000.0;
         // Waiting time to set home search Enc value.
         double dHomeOffset = 0.0;
         // Additional movement by offset value after home search.
         AxmHomeSetMethod(AXIS_0, nHmDir, uHomeSignal, uZphas, dHomeClrTime,
            dHomeOffset );
```

```
// Set velocity that will be used during home search.
double dVelFirst = 100;            // Velocity in 1st stage.
double dVelSecond = 20;            // Velocity in 2nd stage.
double dVelThird = 10;             // Velocity in 3rd stage.
double dvelLast = 10;   // Velocity for index search and accurate search (apply
                        // when Offset value moves).
double dAccFirst = 200;            // Acceleration in 1st stage.
double dAccSecond = 40;            // Acceleration in 2nd stage.
AxmHomeSetVel(AXIS_0, dVelFirst, dVelSecond, dVelThird, dvelLast, dAccFirst,
   dAccSecond);

// help Message.
printf("[INFORMATION]*************** \n");
printf("[ESC] : Exit \n");
printf("[1] : Start home search \n");
printf("[2] : Exit home search \n");
printf("*************************** \n");
DWORD uHomeResult;
DWORD uHomeMainStepNumber , uHomeStepNumber;
BOOL fExit = FALSE;
while (!fExit)                                  // Infinite loop.
{
   if (kbhit())                                 // Press any key.
   {
      int ch = getch();
      switch (ch)
      {
         case 27:                               // Esc key.
            fExit = TRUE;
            AxmSignalServoOn(AXIS_0, DISABLE);  // Servo Off.
         break;

         case '1':
            // Start home search if home search is not in progress on axis 0.
            AxmHomeGetResult(AXIS_0, &uHomeResult);
            if(uHomeResult)
            // Home search is in progress : 0, Home search is completed : 1.
            {
               AxmHomeSetStart(AXIS_0);
               printf("\nStart home search.\n");
            }
         break;

         case '2':
            // Exit home search if home search on axis 0 is in progress.
            AxmHomeGetResult(AXIS_0, &uHomeResult);
            if(uHomeResult == HOME_SEARCHING)
            // Home search is in progress : 0.
            {
```

```
                        AxmMoveSStop(AXIS_0);
                        printf("\nExit home search.\n");
                    }
                break;
            }
        }
        // Verify current home search progress rate.
        AxmHomeGetRate(AXIS_0, &uHomeMainStepNumber, &uHomeStepNumber);
        printf("\r Home search progress rate : %d   ", uHomeStepNumber);

      } // End of While()
    }
    else
      printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",
        Code);
  }
  else
    printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
  printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if (AxlClose())
  printf("\n Library is exited.\n");
else
  printf("\n Library is not exited normally.\n");
}
```

6) Verification Of Motion Drive State

If motion drive is carried out, mechanical part moves as much as the pulse value due to pulse generation from motion controller. In this motion drive, it is a drive and stop state monitoring that verifies information of current position or velocity, drive state, cause of stop, and so on.

● *Verification of Current Position and Ordered Position*

From motion controller, it can be verified how much of movement is commanded currently and what the actual moving distance of mechanical part which is gotten feedback from motor encoder is. When an actual position of specified axis is verified, follow below using AxmStatusGetActPos API.

```
// Verify current position on axis 0.
long lAxisNo = 0;
double dPosition;
AxmStatusGetActPos(lAxisNo, &dPosition);
printf("Actual position on axis 0 : %f", dPosition);
```

When command position is verified, do the following using AxmStatusGetCmdPos API.

```
// Verify ordered position on axis 0.
long lAxisNo = 0;
double dPosition;
AxmStatusGetCmdPos(lAxisNo, &dPosition);
printf("Ordered position on axis 0 : %f", dPosition);
```

When an error value which is the difference between command position and actual position is verified, follow below using AxmStatusReadPosError API.

```
// Verify difference between ordered position and actual position on axis 0.
long lAxisNo = 0;
double dError;
AxmStatusReadPosError(lAxisNo, &dError);
printf("Difference between ordered position and actual position on axis 0 : %f", dError);
```

● *Verification of Current Velocity*

Velocity means that obtains the velocity of current drive of mechanical part. Use AxmStatusReadVel API showing below when current drive velocity is verified.

```
// Verify current velocity on axis 0.
long lAxisNo = 0;
double dVelocity;
AxmStatusReadVel(lAxisNo, &dVelocity);
printf("Current velocity on axis 0 : %f", dVelocity);
```

● *Verification Whether Motion Drive Exists*

This is to verify pulse output state of assigned axis, and simply it means to verify that mechanical part is moving or exited drive. When it is needed to verify a movement of specified axis, use AxmStatusReadInMotion API.

```
// Verify if axis 0 drives.
long lAxisNo = 0;
DWORD uStatus;
AxmStatusReadInMotion(lAxisNo, &uStatus);
```

● *Verification of Drive State Information*

Drive state of specified axis can be read by verification of DriveStatus register on the specified axis. AxmStatusReadInMotion API is used to verify drive state of specified axis.

```
// Read drive state on axis 0.
long lAxisNo = 0;
DWORD uStatus;
AxmStatusReadMotion(lAxisNo, &uStatus);
```

Since AxmStatusReadMotion API can verify more detailed information of each CAMC-IP chip and CAMC-QI chip in addition to the information that is mentioned here, refer to the corresponding manual. Especially most used values are showed below.

※ *Mostly used Drive Status*

Define	Explanation
QIDRIVE_STATUS_0	BUSY (in DRIVE)
QIDRIVE_STATUS_1	DOWN (in deceleration DRIVE)
QIDRIVE_STATUS_2	CONST (in constant velocity DRIVE)
QIDRIVE_STATUS_3	Up (in acceleration DRIVE)
QIDRIVE_STATUS_4	In continuous drive
QIDRIVE_STATUS_5	In specified distance drive
QIDRIVE_STATUS_6	In MPG drive

● *Verification of Stop State Information*

Exit state of specified axis can be verified by verification of EndStatus register on an assigned axis. To verify the exit state of specified axis, AxmStatusReadStop API is used.

```
// Verify exit state on axis 0.
long lAxisNo = 0;
DWORD uStatus;
AxmStatusReadStop(lAxisNo, &uStatus);
```

Showing on the library manual, since AxmStatusReadStop API returns various stop state, it can be verified why it is stopped currently. If it returns 0, it means drive is exited normally, and if returns other than 0, it is exited by corresponding exit cause. Most used values are following.

※ *Mostly used End Status*

Define	Explanation
QIEND_STATUS_0	Exit by forward direction limit signal (PELM)
QIEND_STATUS_1	Exit by reverse direction limit signal (NELM)
QIEND_STATUS_4	Drive exit by forward direction soft limit emergency stop function
QIEND_STATUS_5	Drive exit by reverse direction soft limit emergency stop function
QIEND_STATUS_6	Drive exit by forward direction soft limit deceleration stop function
QIEND_STATUS_7	Drive exit by reverse direction soft limit deceleration stop function
QIEND_STATUS_8	Drive exit by servo alarm function
QIEND_STATUS_9	Drive exit by emergency stop signal input
QIEND_STATUS_10	Drive exit by emergency stop command
QIEND_STATUS_11	Drive exit by deceleration stop command

```
// Ex9_AXM_MotionStatus1.cpp : Defines the entry point for the console application.
// Verify moving distance and velocity moving to (+) direction as much as 200 using relative
// position drive API.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           // Set Active level of +End limit and -End limit on axis 0 to HIGH.
           AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
           // Set input level of inposition signal on axis 0 to HIGH.
           AxmSignalSetInpos(AXIS_0, HIGH);
           // Set input level of alarm signal on axis 0 to LOW.
           AxmSignalSetServoAlarm(AXIS_0, LOW);
           // Set Active input level of ESTOP signal on axis 0 to HIGH.
           AxmSignalSetStop(AXIS_0, 0, HIGH);

           // Make ordered value on axis 0 to be mm unit.
           AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
           // Set initial velocity on axis 0 to 1.  Default : 1
           AxmMotSetMinVel(AXIS_0, 1);
           // Make pulse output method on axis 0 to TwoCwCcwHigh.
           AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
           // Set encoder input method on designated axis to 4 multiplication.
           AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

           AxmMotSetAbsRelMode(AXIS_0, 0);     // Absolute position drive.
           AxmMotSetProfileMode(AXIS_0, 3);    // Symmetrical S-curve drive.
```

```
// Set home search method.
AxmHomeSetMethod(AXIS_0,-1,HomeSensor,1,2000.0,0.0);
// Set velocity that will be used during home search.
AxmHomeSetVel(AXIS_0, 100, 20, 10, 10, 200, 40);

// Servo On.
AxmSignalServoOn(AXIS_0, ENABLE);

// Search home.
printf("Start home search pressing any key.\n");
getch();
AxmHomeSetStart(AXIS_0);

// Verify current home search progress rate.
DWORD uHomeMainStepNumber , uHomeStepNumber;
DWORD uHomeResult;

AxmHomeGetResult(AXIS_0, &uHomeResult);
while(!uHomeResult)
{
   AxmHomeGetResult(AXIS_0, &uHomeResult);
   AxmHomeGetRate(AXIS_0, &uHomeMainStepNumber , &uHomeStepNumber);
   printf("\r In Home Search . . . %d %    ", uHomeStepNumber);
}

printf("\n[INFORMATION]*************************** \n");
printf("AP : Actual Position \n");
printf("CP : Command Position \n");
printf("Error : Actual Position - Command Position \n");
printf("CV : Command Velocity \n");
printf("****************************************** \n");
printf("Move to (+) direction as much as 200 with S-curve velocity profile on
   axis 0 (velocity:100, acceleration:200, exit API at start point)\n");
printf("Drive starts pressing any key.");
getch();
AxmMoveStartPos(AXIS_0, 200, 100, 200, 200);

// Verify the drive state during in drive.
DWORD uStatus;
double dActualPosition, dCommandPosition, dError, dVelocity;
AxmStatusReadInMotion( AXIS_0, &uStatus);
while(uStatus)
{
   AxmStatusReadInMotion( AXIS_0, &uStatus);
   AxmStatusGetActPos(AXIS_0, & dActualPosition);
   // Verify current position on axis 0.
   AxmStatusGetCmdPos(AXIS_0, & dCommandPosition);
   // Verify ordered position on axis 0.
```

```
            AxmStatusReadPosError(AXIS_0, & dError);
            // Verify difference between ordered position and actual position on axis 0.
            AxmStatusReadVel(AXIS_0, & dVelocity);
            // Verify current velocity on axis 0.
            printf("\rAP:%.1f, CP:%.1f, Error:%.1f, CV:%.1f ",       dActualPosition,
               dCommandPosition, dError, dVelocity );
         } // End of While()

         printf("\nDrive is completed.\n");
         AxmSignalServoOn(AXIS_0, DISABLE);                    // Servo Off.
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
            Code)
   }
   else
      printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
   printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if (AxlClose())
   printf("\n Library is exited.\n");
else
   printf("\n Library is not exited normally.\n");
}
```

Multi-Axis Drive

Generally, drive system moves several axes to a specified position at once in equipment, but there are many cases that each axis has related motion with a constant pattern. Multi-axis API is provided for these drives.

Multi-axis drive setting is to set one-axis drive setting for each axis that has been previously. For example, if there are 2 axes, set one-axis drive API setting to axis 0 and 1 axis each.

Multi-axis drive API is divided into multi-axis position drive that moves several axes at once with set position and velocity, and continuous interpolation drive that makes to drive several axes to a specified position along with specified path.

1) Multi-Axis Position Drive

Multi-axis position drive is what position drive of one-axis drive is carried out on two axes at the same time. Axes are started at once to the exit position of each axis that is inputted from current position, and moving velocity and acceleration of two axes are available to each setting. This multi-axis position drive has AxmMoveMultiPos API which is exit API at end point and AxmMoveStartMultiPos API which is exit API at start point.

Two API has only difference between exit API at start point and exit API at end point, and input factor is same. Here, axis number must be inputted with an order of ascending series. Following example is to move 0 and 1 axis to the position of 100 and 50 each with same velocity and acceleration. Position drive API is not an interpolation drive, and moves individually by the position, velocity, acceleration value that is set on each axis. There is no difference except starting at the same time. Obviously, 1 axis reaches the target point earlier than axis 0, and axis 0 still moves in the state that 1 axis is stopped. AxmMoveMultiPos API which is an exit API at end point comes out of API after stop of all axes that command is ordered, and carries out the next command on program.

```
// With axis 0 as a relative coordination, move to (100,50) position with symmetrical
// S-curve drive mode (velocity 100, acceleration 200).
DWORD uAbsRelMode = 0;
DWORD uProfileMode = 3;
AxmMotSetAbsRelMode(lAxisNo, uAbsRelMode);
AxmMotSetProfileMode(lAxisNo, uProfileMode);

long lArraySize = 2;
long lAxisNo[2] = {0,1};
double dPosition[2] = {100,50};
double dMaxVelocity[2] = {100,100};
double dMaxAccel[2] = {200,200};
double dMaxDecel[2] = {200,200};
AxmMoveMultiPos(lArraySize,lAxisNo,dPosition,dMaxVelocity,dMaxAccel,dMaxDecel);
```

If AxmMoveStartMultiPos API which is an exit API at start point is needed to use, follow below. The next API is carried out immediately after starting drive because of exit API at the start point, and if it is needed to

verify whether exiting drive, use AxmStatusReadInMotion API for the each axis.

```
// With axis 0 as a relative coordination, move to (100,50) position with symmetrical
// S-curve drive mode (velocity 100, acceleration 200).
DWORD uAbsRelMode = 0;
DWORD uProfileMode = 3;
AxmMotSetAbsRelMode(lAxisNo, uAbsRelMode);
AxmMotSetProfileMode(lAxisNo, uProfileMode);

long lArraySize = 2;
long lAxesNo[2] = {0,1};
double dPosition[2] = {100,50};
double dMaxVelocity[2] = {100,100};
double dMaxAccel[2] = {200,200};
double dMaxDecel[2] = {200,200};
AxmMoveStartMultiPos(lArraySize, lAxesNo, dPosition, dMaxVelocity, dMaxAccel, dMaxDecel);

// Wait until all motions are exited.
BOOL WaitForDone = TRUE;
DWORD uStatus1, uStatus2;
while(WaitForDone)
{
  AxmStatusReadInMotion(lAxesNo[0], &uStatus1);
  AxmStatusReadInMotion(lAxesNo[1], &uStatus2);
  If(uStatus1 == 0 && uStatus2 == 0)
  WaitForDone = FALSE;
}
```

```
// Ex11_AXM_MultiPositionDrive.cpp:Defines the entry point for the console application.
// Return to home again moving to (100,50) position after home setting of 2 axes
// mechanical part.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_1 1

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");
```

```
// Inspect whether motion module exists or not.
DWORD dwStatus;
Code = AxmInfoIsMotionModule(&dwStatus);
if(Code == AXT_RT_SUCCESS)
{
   if(dwStatus == STATUS_EXIST)
   {
      printf("Motion module exists.\n");

      for(int nAxisNo = 0; nAxisNo<2; nAxisNo++)
      {
         // Set Active level of +End limit and -End limit on axis 0 to HIGH.
         AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
         // Set input level of inposition signal on axis 0 to HIGH.
         AxmSignalSetInpos(nAxisNo, HIGH);
         // Set input level of alarm signal on axis 0 to LOW.
         AxmSignalSetServoAlarm(nAxisNo, LOW);
         // Set Active input level of ESTOP signal on axis 0 to HIGH.
         AxmSignalSetStop(nAxisNo, 0, HIGH);

         // Make ordered value on axis 0 to be mm unit.
         AxmMotSetMoveUnitPerPulse(nAxisNo, 10, 10000);
         // Set initial velocity on axis 0 to 1.  Default : 1
         AxmMotSetMinVel(nAxisNo, 1);
         // Make pulse output method on axis 0 to TwoCwCcwHigh.
         AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
         // Set encoder input method on designated axis to 4 multiplication.
         AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);

         // Set home search method.
         AxmHomeSetMethod(nAxisNo,-1,HomeSensor,1,2000.0,0.0);
         // Set velocity that will be used during home search.
         AxmHomeSetVel(nAxisNo, 100, 20, 10, 10, 200, 40);

         // Servo On.
         AxmSignalServoOn(nAxisNo, ENABLE);
      }

      // Search home.
      printf("Start home search pressing any key.\n");
      getch();
      AxmHomeSetStart(AXIS_0);
      AxmHomeSetStart(AXIS_1);

      // Verify current home search progress rate.
      DWORD uHomeResult_0, uHomeResult_1;
      DWORD uHomeMainStepNumber , uHomeStepNumber_0,uHomeStepNumber_1;
      AxmHomeGetResult(AXIS_0, &uHomeResult_0);
```

```
AxmHomeGetResult(AXIS_1, &uHomeResult_1);
// In home search : 0, Home search completion : 1
while(uHomeResult_0 == HOME_SEARCHING || uHomeResult_1 == HOME_SEARCHING)
{
   AxmHomeGetResult(AXIS_0, &uHomeResult_0);
   AxmHomeGetResult(AXIS_1, &uHomeResult_1);

   AxmHomeGetRate(AXIS_0, &uHomeMainStepNumber , &uHomeStepNumber_0);
   AxmHomeGetRate(AXIS_1, &uHomeMainStepNumber , &uHomeStepNumber_1);
   printf("\rIn Home Search... Axis 0: %d%, Axis 1: %d ", uHomeStepNumber_0,
      uHomeStepNumber_1);
}

long lArraySize = 2;
long lAxesNo[2] = {0,1};
double dPosition[2], dMaxVelocity[2], dMaxAccel[2], dMaxDecel[2];

printf("\nMove as much as (+)100 of axis 0 and (+)50 of axis 1 with trapezoidal
   velocity profile (exit API at end point)\n");
printf("Drive starts pressing any key.\n");
getch();
for(nAxisNo = 0; nAxisNo<1; nAxisNo++){
   AxmMotSetAbsRelMode(nAxisNo, 1);
   // Set to relative coordination.
   AxmMotSetProfileMode(nAxisNo, 0);
   // Set to symmetrical trapezoid velocity profile.
}
dPosition[0] = 100;      dPosition[1] =50;
dMaxVelocity[0] = 100;    dMaxVelocity[1] =100;
dMaxAccel[0] = 200;      dMaxAccel[1] = 200;
dMaxDecel[0] = 200;      dMaxDecel[1] =200;
AxmMoveMultiPos(lArraySize, lAxesNo, dPosition, dMaxVelocity, dMaxAccel,
   dMaxDecel);
printf("Drive is completed.\n");

printf("Move to home with S-curve velocity profile (exit API at start
   point)\n");
printf("Drive starts pressing any key.");
getch();
for(nAxisNo = 0; nAxisNo<1; nAxisNo++){
   AxmMotSetAbsRelMode(nAxisNo, 0);
   // Set to absolute coordination.
   AxmMotSetProfileMode(nAxisNo, 3);
   // Set to symmetrical S-curve velocity profile.
}
dPosition[0] = 0;         dPosition[1] =0;
dMaxVelocity[0] = 100;    dMaxVelocity[1] =100;
dMaxAccel[0] = 200;       dMaxAccel[1] = 200;
dMaxDecel[0] = 200;       dMaxDecel[1] =200;
```

```
            AxmMoveStartMultiPos(lArraySize, lAxesNo, dPosition, dMaxVelocity, dMaxAccel,
               dMaxDecel);

            // Wait until all motions are exited.
            BOOL WaitForDone = TRUE;
            DWORD uStatus0, uStatus1;
            while(WaitForDone) {
               AxmStatusReadInMotion(AXIS_0, &uStatus0);
               AxmStatusReadInMotion(AXIS_1, &uStatus1);
               if(uStatus0 == 0 && uStatus1 == 0)
                  WaitForDone = FALSE;
            }
            printf("Drive is completed.\n");

            // Servo Off.
            AxmSignalServoOn(0, DISABLE);
            AxmSignalServoOn(1, DISABLE);
         }
         else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
               Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("Library is exited.\n");
   else
      printf("Library is not exited normally.\n");
}
```

2) Interpolation Drive

Multi-axis position drive that is mentioned before is a multi-axis drive that two axes carry out each motion starting at the same time. However, it occurs that two axes move along with straight line or circular of assigned path with related movement mutually, and for those drive, straight line interpolation, circular interpolation, and continuous interpolation drive for each are provided. Since the setting of velocity profile and absolute/relative coordination drive for the interpolation drive is to be set along with an axis that has the earliest axis number among axes that had interpolation drive, please be careful to set.

● *Straight Line Interpolation Drive*

Straight line interpolation drive makes two axes included in one module to drive of straight interpolation using AxmLineMove API. In case of SMC-2V03 chip, only 2 axes interpolation is available because there are 2 axes in one chip, but PCI-N804/404 has 4 axes in one chip and arbitrary straight line interpolation of 2, 3, and 4 axis in 4 axes is available. This API drives properly to drive two axes with specified velocity and acceleration from the current position to the inputted exit point.

```
// Interpolation drive with S-curve to relative position of (100,50).
long lSize = 2;
long lAxesNo = {0,1};
double dPosition = {100,50};
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
long lCoordinate = 0;

DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;

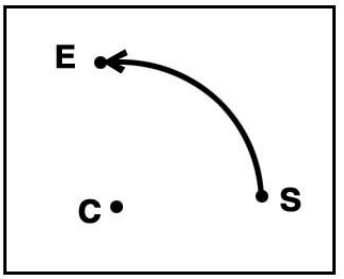
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiSetAbsRelMode(lCoordinate, uAbsRelMode);   // Set to relative position drive.

AxmLineMove(lCoordinate, dPosition, dMaxVelocity, dMaxAccel, dMaxDecel);
```

● *Circular Interpolation Drive*

Circular interpolation drive is to be interpolation drive with circular path based on the corresponding setting from the current position to the assigned position. And this circular interpolation drive APIs provide AxmCircleCenterMove, AxmCircleRadiusMove, AxmCircleAngleMove, and AxmCirclePointMove API.

- AxmCircleCenterMove



AxmCircleCenterMove API carries out circular interpolation drive from the current position (S) with input of exit point (E) and center point (C). Since this is a method to calculate circular arc using the position of three points, circle is composed of that accurate center point and exit point are inputted, and API is operated. If a wrong coordination is inputted, API will not be operated and returns FALSE. If circular interpolation drive is needed to carry out counterclockwise with center of (0, 100) from the current position (0,0) to (100,100) position, do the following.

```
// Carry out circular interpolation with center of (0, 100) from the current position (0,0)
// to (100,100) position.
long lSize = 2;
long lAxesNo = {0,1};
double dCenPos[2];
double dEndPos[2];
long lCoordinate = 0;
DWORD uAbsRelMode = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiSetAbsRelMode(lCoordinate, uAbsRelMode);   // Set to relative position drive.

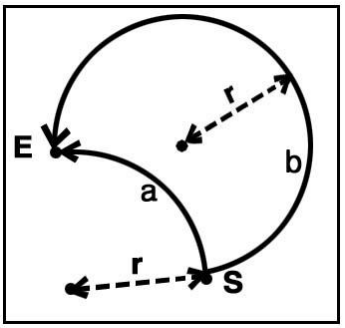
double dCenterXPosition = 0;
double dCenterYPosition = 100;
double dEndXPosition = 100;
double dEndYPosition = 100;

dCenPos[0] = dCenterXPosition;
dCenPos[1] = dCenterYPosition;

dEndPos[0] = dEndXPosition;
dEndPos[1] = dEndYPosition;

double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 1;                                    // [0] Ccw direction, [1] Cw direction
AxmCircleCenterMove(lCoordinate, lAxesNo, dCenPos, dEndPos, dMaxVelocity, dMaxAccel,
  dMaxDecel, uCWDir);
```

- AxmCircleRadiusMove



AxmCircleRadiusMove API carries out circular interpolation drive from the current position (S) with input of exit point (E) and radius (r). Since this is a method to calculate arc that passes two points and has constant radius, two circles are to be structured. Therefore, if uShortDistance value is 0 for the input radius, circle with short distance (a) is selected, and if 1, circle with long distance (b) is selected.

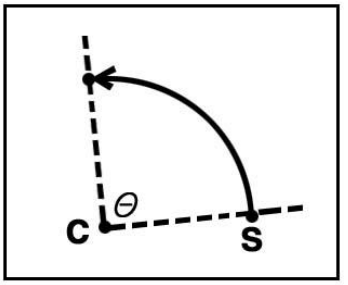
If circular interpolation is needed to carry out clockwise with long distance that has a radius of 120 from the current position to (300,200) position, do the following.

```
// Carry out circular interpolation from the current position to (300,200) position with
// long distance that has a radius of 120.
long lSize = 2;
long lAxesNo = {0,1};
double dEndPos[2];
long lCoordinate = 0;

DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiSetAbsRelMode(lCoordinate, uAbsRelMode);     // Set to relative position drive.

double dRadius = 120;
double dEndXPosition = 300
double dEndYPosition = 200;
dEndPos[0] = dEndXPosition;
dEndPos[1] = dEndYPosition;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 0;                    // [0] Cw direction, [1] Ccw direction
DWORD uShortDistance = 1;            // [0]: small(circular)distance,[1] big(circular)distance
AxmCircleRadiusMove(lCoordinate, lAxesNo, dRadius, dEndPos, dMaxVelocity, dMaxAccel,
  dMaxDecel, uCWDir, uShortDistance);
```

- AxmCircleAngleMove



AxmCircleAngleMove API carries out circular interpolation drive from the current position (S) with input of center point (C) and rotational angle (θ). This is a method to calculate circular arc using radius and angle of arc, and a circle is structured.

Circular interpolation is carried out as much as 200 degrees with center of (200,200) from the current position, and if interpolation drive is needed to carry out clockwise, do the following.

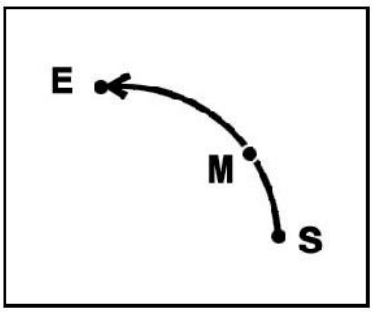
```
// Carried out circular interpolation as much as 200 degrees with center of (200,200) from
// the current position.
long lSize = 2;
long lAxesNo = {0,1};
double dCenPos[2];
long lCoordinate = 0;

DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiSetAbsRelMode (lCoordinate, uAbsRelMode);    // Set to relative position drive.

double dCenterXPosition = 200;
double dCenterYPosition = 200;

dCenPos[0] = dCenterXPosition;
dCenPos[1] = dCenterYPosition;
double dAngle = 200;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 0;                                    // [0] Cw direction, [1] Ccw direction
AxmCircleAngleMove(lCoordinate, lAxesNo, dCenPos, dAngle, dMaxVelocity, dMaxAccel,
  dMaxDecel, uCWDir);
```

- AxmCirclePointMove



AxmCirclePointMove API carries out circular interpolation drive from the current position (S) with input of middle point (M) and exit point (E). This is a method to calculate circular arc that passes three points, and a circle is structured.

If interpolation drive is needed to carry out from the current position to (150,150) passing (100,150), do the following.

```
// Carry out circular interpolation from the current position to (150,150) passing
// (100,150).
long lSize = 2;
long lAxesNo = {0,1};
double dMidPos[2];
double dEndPos[2];
long lCoordinate = 0;
DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiSetAbsRelMode(lCoordinate, uAbsRelMode);     // Set to relative position drive.

double dMidXPosition = 100;
double dMidYPosition = 50;
double dEndXPosition = 150;
double dEndYPosition = 150;

dMidPos[0] = dMidXPosition;
dMidPos[1] = dMidYPosition;

dEndPos[0] = dEndXPosition;
dEndPos[1] = dEndYPosition;

double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
AxmCirclePointMove(lCoordinate, lAxesNo, dMidPos, dEndPos, dMaxVelocity, dMaxAccel,
  dMaxDecel);
```

```
// Ex12_AXM_Interpolation.cpp : Defines the entry point for the console application.
// Start interpolation drive after setting home of 2 axes mechanical part.
#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_1 1

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");
           for(int nAxisNo = 0; nAxisNo<2; nAxisNo++)
           {
              // Set Active level of +End limit and -End limit on axis 0 to HIGH.
              AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
              // Set input level of inposition signal on axis 0 to HIGH.
              AxmSignalSetInpos(nAxisNo, HIGH);
              // Set input level of alarm signal on axis 0 to LOW.
              AxmSignalSetServoAlarm(nAxisNo, LOW);
              // Set Active input level of ESTOP signal on axis 0 to HIGH.
              AxmSignalSetStop(nAxisNo, 0, HIGH);

              // Make ordered value on axis 0 to be mm unit.
              AxmMotSetMoveUnitPerPulse(nAxisNo, 10, 10000);
              // Set initial velocity on axis 0 to 1.  Default : 1
              AxmMotSetMinVel(nAxisNo, 1);
              // Make pulse output method on axis 0 to TwoCwCcwHigh.
              AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
              // Set encoder input method on designated axis to 4 multiplication.
              AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);

              // Set home search method.
              AxmHomeSetMethod(nAxisNo,-1,HomeSensor,1,2000.0,0.0);
```

```
    // Set velocity that will be used during home search.
    AxmHomeSetVel(nAxisNo, 100, 20, 10, 10, 200, 40);
    // Servo On.
    AxmSignalServoOn(nAxisNo, ENABLE);
}

// Search home.
printf("Start home search pressing any key.\n");
getch();
AxmHomeSetStart(AXIS_0);
AxmHomeSetStart(AXIS_1);

// Verify current home search progress rate.
DWORD uHomeResult_0, uHomeResult_1;
DWORD uHomeMainStepNumber , uHomeStepNumber_0,uHomeStepNumber_1;
AxmHomeGetResult(AXIS_0, &uHomeResult_0);
AxmHomeGetResult(AXIS_1, &uHomeResult_1);
while(uHomeResult_0 == HOME_SEARCHING || uHomeResult_1 == HOME_SEARCHING)
{
    AxmHomeGetResult(AXIS_0, &uHomeResult_0);
    AxmHomeGetResult(AXIS_1, &uHomeResult_1);

    AxmHomeGetRate(AXIS_0, &uHomeMainStepNumber , &uHomeStepNumber_0);
    AxmHomeGetRate(AXIS_1, &uHomeMainStepNumber , &uHomeStepNumber_1);
    printf("\rIn Home Search... axis 0 : %d%, axis 1 : %d", uHomeStepNumber_0,
        uHomeStepNumber_1);
}

printf("\nCarry out straight line interpolation drive to relative position of
    (150,0).\n");
printf("Drive starts pressing any key.\n");
getch();
long  lSize = 2;
long  lpAxesNo[2] = {AXIS_0, AXIS_1};
double dpPosition[2] = {150, 0};
double dMaxVelocity = 50;
double dMaxAccel = 100;
double dMaxDecel = 100;
long  lCoordinate = 0;
double dCenPos[2], dMidPos[2], dEndPos[2];

AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lSize, lpAxesNo);
AxmContiSetAbsRelMode (lCoordinate, 1);      // Set to relative position drive.
AxmLineMove(lCoordinate, dpPosition, dMaxVelocity, dMaxAccel, dMaxDecel);

BOOL WaitForDone = TRUE;
DWORD uStatus0, uStatus1;
while(WaitForDone) {
```

```
        AxmStatusReadInMotion(AXIS_0, &uStatus0);
        AxmStatusReadInMotion(AXIS_1, &uStatus1);
        if(uStatus0 == 0 && uStatus1 == 0)
            WaitForDone = FALSE;
    }

    printf("\n Carry out circular interpolation drive to relative position of
        (50,50) with center of (0,50) and with Ccw direction.\n");
    printf("Drive starts pressing any key.\n");
    getch();

    dCenPos[0] = 0;    dCenPos[1] = 50;
    dEndPos[0] = 50;   dEndPos[1] = 50;
    AxmCircleCenterMove(lCoordinate, lpAxesNo, dCenPos, dEndPos, dMaxVelocity,
        dMaxAccel, dMaxDecel, 1);

    // Wait until all motions are exited.
    WaitForDone = TRUE;
    while(WaitForDone) {
        AxmStatusReadInMotion(AXIS_0, &uStatus0);
        AxmStatusReadInMotion(AXIS_1, &uStatus1);
        if(uStatus0 == 0 && uStatus1 == 0)
            WaitForDone = FALSE;
    }

    printf("\n Carry out straight line interpolation drive to relative position of
        (0,100).\n");
    printf("Drive starts pressing any key.\n");
    getch();
    dpPosition[0] = 0; dpPosition[1] = 100;
    AxmLineMove(lCoordinate, dpPosition, dMaxVelocity, dMaxAccel, dMaxDecel);
    // Wait until all motions are exited.
    WaitForDone = TRUE;
    while(WaitForDone) {
        AxmStatusReadInMotion(AXIS_0, &uStatus0);
        AxmStatusReadInMotion(AXIS_1, &uStatus1);
        if(uStatus0 == 0 && uStatus1 == 0)
            WaitForDone = FALSE;
    }

    printf("\nCarry out circular interpolation drive to relative position of (-
        50,50) having small distance with radius of 50 and with Ccw direction.\n");
    printf("Drive starts pressing any key.\n");
    getch();

    dEndPos[0] = 50; dEndPos[1] = -50;
    AxmCircleRadiusMove(lCoordinate, lpAxesNo, 50, dEndPos, dMaxVelocity, dMaxAccel,
        dMaxDecel, 0, 1);
```

```
// Wait until all motions are exited.
WaitForDone = TRUE;
while(WaitForDone) {
   AxmStatusReadInMotion(AXIS_0, &uStatus0);
   AxmStatusReadInMotion(AXIS_1, &uStatus1);
   if(uStatus0 == 0 && uStatus1 == 0)
      WaitForDone = FALSE;
}

printf("\n Carry out circular interpolation drive as much as 90 degrees with Ccw
   direction and with center of relative position of (0,-150).\n");
printf("Drive starts pressing any key.\n");
getch();

// Wait until all motions are exited.
dCenPos[0] = 0; dCenPos[1] = -150;
AxmCircleAngleMove(lCoordinate, lpAxesNo, dCenPos, 90, dMaxVelocity, dMaxAccel,
   dMaxDecel, 1);

// Wait until all motions are exited.
WaitForDone = TRUE;
while(WaitForDone) {
   AxmStatusReadInMotion(AXIS_0, &uStatus0);
   AxmStatusReadInMotion(AXIS_1, &uStatus1);
   if(uStatus0 == 0 && uStatus1 == 0)
      WaitForDone = FALSE;
}

printf("\n Carry out circular interpolation drive as much as (100,0) with Ccw
   direction having middle point of relative position of (50,50).\n");
printf("Drive starts pressing any key.\n");
getch();

// Wait until all motions are exited.
dMidPos[0] = 50;    dMidPos[1] = 50;
dEndPos[0] = 100;   dEndPos[1] = 0;
AxmCirclePointMove(lCoordinate, lpAxesNo, dMidPos, dEndPos, dMaxVelocity,
   dMaxAccel, dMaxDecel,0);

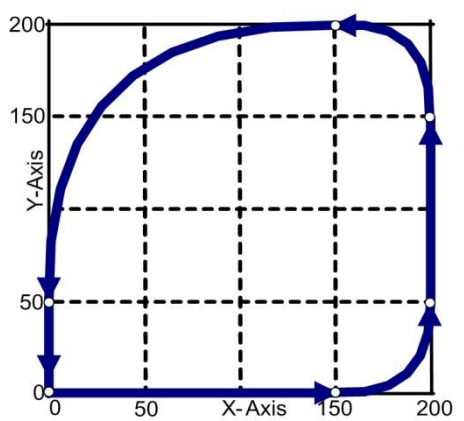
WaitForDone = TRUE;
while(WaitForDone) {
   AxmStatusReadInMotion(AXIS_0, &uStatus0);
   AxmStatusReadInMotion(AXIS_1, &uStatus1);
   if(uStatus0 == 0 && uStatus1 == 0)
      WaitForDone = FALSE;
}

printf("\nCarry out straight line interpolation drive to home.\n");
printf("Drive starts pressing any key.\n");
```

```
            getch();
            dpPosition[0] = 0; dpPosition[1] = -50;
            AxmLineMove(lCoordinate, dpPosition, dMaxVelocity, dMaxAccel, dMaxDecel);
            // Wait until all motions are exited.
            WaitForDone = TRUE;
            while(WaitForDone) {
               AxmStatusReadInMotion(AXIS_0, &uStatus0);
               AxmStatusReadInMotion(AXIS_1, &uStatus1);
               if(uStatus0 == 0 && uStatus1 == 0)
                  WaitForDone = FALSE;
            }
            // Servo Off.
            AxmSignalServoOn(AXIS_0, ENABLE);
            AxmSignalServoOn(AXIS_1, ENABLE);
         }
         else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",
               Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("Library is exited.\n");
   else
      printf("Library is not exited normally.\n");
}
```

※ *If an example of straight line interpolation drive above is carried out, following result can be verified.*



3) Continuous Interpolation Drive

● *Straight Line Continuous Interpolation Drive*

Continuous interpolation drive API is what is to drive interpolation drive mentioned before continuously, and has a drive that accelerates after starting drive, passes assigned positions with constant velocity, and decelerates and stops.

One unique thing is that, after axes to use for continuous interpolation takes mapping procedure to the new coordination system, stores commands to carry out interpolation drive continuously in the memory area of "continuous interpolation queue," if continuous interpolation start command is given later, value goes into motion control chip after coming out one by one from the stored continuous interpolation data, and carries out interpolation drive continuously. Of course, interpolation data can be inputted after interpolation drive is started.

In order to carry out continuous interpolation drive, first, axes to be used for continuous interpolation must take a process of mapping. Interpolation drive APIs that are used for continuous interpolation are to be a combination of 2 axes interpolation drives, but they have to take mapping to one coordination system that includes all axes to be used after mapping of axes. For example, if it is carried out interpolation drive of 1 and 2 axis after interpolation drive of 0 and 1 axis continuously using three axes of 0, 1, and 2, they have to take mapping to one coordination system. After mapping, set velocity profile to use in corresponding coordination system, and set absolute/relative position drive.

```
// Carry out mapping of 0,1,2 axis to coordination system of 0 number that is to use for
// continuous interpolation drive.
long lCoordinate = 0;
long lSize = 3;
long lRealAxesNo = {0,1,2};
AxmContiSetAxisMap(lCoordinate, lSize, lRealAxesNo);

// Set 0 number coordination system to relative position drive.
DWORD uAbsRelMode = 1;   // [0] : Absolute position drive, [1] : Relative position drive
AxmContiSetAbsRelMode(lCoordinate, uAbsRelMode);
```

To input straight continuous interpolation data on continuous interpolation drive queue, if AxmContiBeginNode and AxmContiEndNode API are called, it is to be AxmLineMove API. And to input circular continuous interpolation data, if AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, and AxmCircleAngleMove API are used, it is to be an API that stores in internal queue.

```
// Input data on interpolation queue to carry out line interpolation drive to (100,150,0),
// (velocity : 100, acceleration : 200).
long  lCoordinate = 0;
double dPosition = {100,150,0};
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
AxmContiBeginNode(lCoordinate);
AxmLineMove(lCoordinate, dPosition, dMaxVelocity, dMaxAccel, dMaxDecel);
```

```
AxmContiEndNode(lCoordinate);
```

```
// Input data on interpolation queue to carry out circular interpolation drive with center
// of (0,100) from the current position (0,0) to (100,100) using 0 and 1 axis.
long lSize = 2;
long lAxesNo = {0,1};
double dCenPos[2];
double dEndPos[2];
long lCoordinate = 0;
double dCenterXPosition = 0;
double dCenterYPosition = 100;
double dEndXPosition = 100;
double dEndYPosition = 100;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;

dCenPos[0] = dCenterXPosition;
dCenPos[1] = dCenterYPosition;

dEndPos[0] = dEndXPosition;
dEndPos[1] = dEndYPosition;
DWORD uCWDir = 1;               // [0] Ccw direction, [1] Cw direction
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiBeginNode(lCoordinate);
AxmCircleCenterMove(lCoordinate, lAxesNo, dCenPos, dEndPos,, dMaxVelocity, dMaxAccel,
  dMaxDecel, uCWDir);
AxmContiEndNode(lCoordinate);
```

```
// Input data on interpolation queue to carry out circular interpolation with radius of 120
// from the current position to (300,200) using 1 and 2 axis.
long lSize = 2;
long lAxesNo = {1,2};
double dEndPos[2];
long lCoordinate = 0;
double dRadius = 120;
double dEndXPosition = 300
double dEndYPosition = 200;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 0;               // [0] Ccw direction, [1] Cw direction
DWORD uShortDistance = 1;       // [0]: short(circular)distance,[1] big(circular)distance
AxmContiWriteClear(lCoordinate);
```

```
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiBeginNode(lCoordinate);
AxmCircleRadiusMove(lCoordinate, lAxesNo, dRadius, dEndPos, dMaxVelocity, dMaxAccel,
  dMaxDecel, uCWDir, uShortDistance);
AxmContiEndNode(lCoordinate);
```

```
// Input data on interpolation queue to carry out circular interpolation as much as 200
// degrees with center of (200,200) from the current position using 0 and 2 axis.
long lSize = 2;
long lAxesNo = {0,2};
double dCenPos[2];
long lCoordinate = 0;
double dCenterXPosition = 200;
double dCenterYPosition = 200;
dCenPos[0] = dCenterXPosition;
dCenPos[1] = dCenterYPosition;
double dAngle = 200;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 0;                    // [0] Ccw direction, [1] Cw direction
AxmContiBeginNode(lCoordinate);
AxmCircleAngleMove(lCoordinate, lAxesNo, dCenPos, dAngle, dMaxVelocity, dMaxAccel,
  dMaxDecel, uCWDir);
AxmContiEndNode(lCoordinate);
```

```
// Input data on interpolation queue to carry out circular interpolation from the current
// position to (150,150) passing (100,50) and using 0 and 1 axis.
long lSize = 2;
long lAxesNo = {0,1};
double dMidPos[2];
double dEndPos[2];
long lCoordinate = 0;
double dMidXPosition = 100;
double dMidYPosition = 50;
double dEndXPosition = 150;
double dEndYPosition = 150;

dMidPos[0] = dMidXPosition;
dMidPos[1] = dMidYPosition;
dEndPos[0] = dEndXPosition;
dEndPos[1] = dEndYPosition;

double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
AxmContiWriteClear(lCoordinate);
```

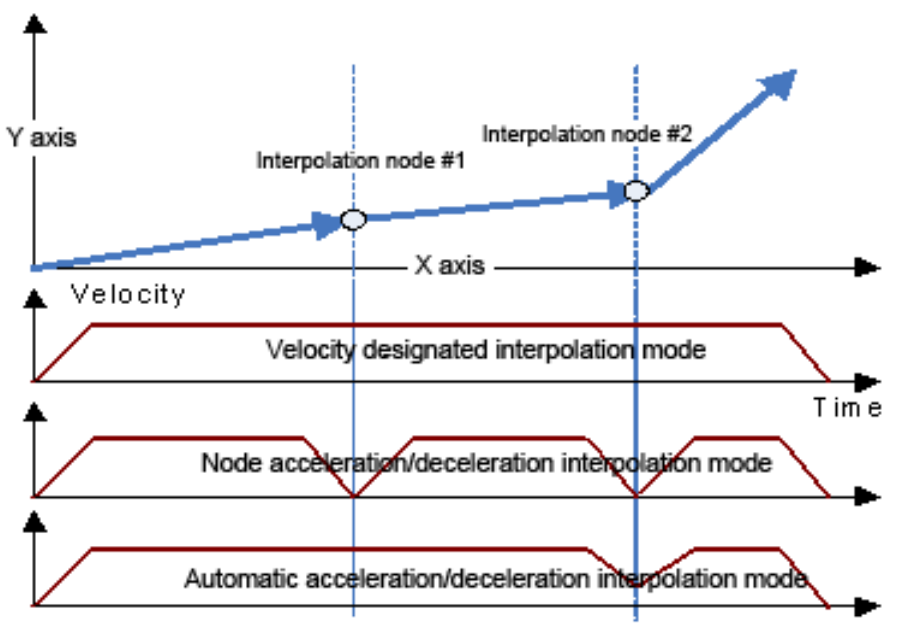
```
AxmContiSetAxisMap(lCoordinate, lSize, lAxesNo);
AxmContiBeginNode(lCoordinate);
AxmCirclePointMove(lCoordinate, lAxesNo, dMidPos, dEndPos, dEndXPosition, dEndYPosition,
  dMaxVelocity, dMaxAccel, dMaxDecel);
AxmContiEndNode(lCoordinate);
```

After input data into interpolation, when AxmContiStart API is called, continuous interpolation drive is started using the stored data. Here, velocity profile mode in drive of continuous interpolation can be designated, and there are velocity designated interpolation mode, node acceleration/deceleration interpolation mode, and automatic acceleration/deceleration interpolation mode.

Velocity designated interpolation mode means a general continuous interpolation drive that progresses continuous interpolation drive with maintaining velocity when it reaches to the designated velocity after start drive and acceleration. Node acceleration/deceleration interpolation mode is carried out at each node of continuous interpolation, and automatic acceleration/deceleration interpolation mode is to accelerate/decelerate automatically in the section of sharp curve occurrence, even though no velocity change along with continuous path.

```
// Start continuous interpolation using stored interpolation data.
long lCoordinate = 0;
DWORD dwProfileset = 0;     // (0): Velocity designated interpolation mode
// (1): node acceleration/deceleration interpolation mode
// (2): automatic acceleration/deceleration interpolation mode
 AxmContiStart(lCoordinate, dwProfileset, 0);
```

If continuous interpolation started like this is needed to exit arbitrary, use AxmMoveSStop API.



```
// Exit continuous interpolation drive.
long lCoordinate = 0
AxmMoveSStop(lCoordinate);   // Input master axis of lCoordinate into factor.
```

In addition to APIs previously mentioned, to carry out more various drive during continuous interpolation, following APIs are provided additionally. AxmContiReadFree API is provided to verify whether interpolation drive queue is cleared, and AxmContiReadIndex API is provided to verify what number of data is in drive on interpolation queue.

```
// Verify whether interpolation drive queue is cleared.
long lCoordinate = 0;
DWORD uQueueFree;
AxmContiReadFree(lCoordinate, &uQueueFree);
if(uQueueFree) { printf("Interpolation queue is cleared."); }
else { printf("Interpolation queue is not cleared."); }
```

```
// Verity what number of data is inputted into chip.
long lCoordinate = 0;
long lQueueIndex;
AxmContiReadIndex(lCoordinate, &lQueueIndex);
printf("In drive of interpolation currently Index : %d \n",lQueueIndex);
```

API is described that verifies index in drive of interpolation currently above. But, if it is in actual drive, there will be something weird. If it is in drive of interpolation with inputting 10 data, naturally when it is in first drive, it has to be 1. But it is 1 for a while and is changed to 2, and it always will have the value that is bigger by 1 than data number in actual drive. AxmContiReadIndex API which verifies index of data sent from memory to chip has the value that is bigger by 1 than actual because motion chip always has 1 more extra data besides the data in drive currently, in process of input data from interpolation data queue memory into motion chip internally.

Similar to this API, there is AxmContiGetNodeNum API that verifies node number in drive currently, and it returns data number in actual drive. Also, AxmContiGetTotalNodeNum API is used to verify total number of data that are in interpolation drive queue. If it is needed to verify what number of node is in drive currently out of what number of total interpolation drive node, do the following.

```
// Verify what number of node is in drive currently.
long lCoordinate = 0;
long lNodeNum, lNodeTotal;
AxmContiGetNodeNum(lCoordinate, &lNodeNum);
AxmContiGetTotalNodeNum(lCoordinate, &lNodeTotal);
printf("Currently %d number of node out of total of %d number is in drive of interpolation.
  \n", lNodeTotal, lNodeNum);
```

To verify whether continuous interpolation drive is in progress currently, use AxmContiIsMotion API.

```
// Verify whether it is in drive of interpolation currently.
long lCoordinate = 0;
DWORD uInMotion;
AxmContiIsMotion(lCoordinate, &uInMotion);
if(uInMotion) { printf("It is in drive of continuous interpolation."); }
```

```
else { printf("It is not in drive of continuous interpolation."); }
```

If continuous interpolation drive is exited, interpolation queue is to be clear, but sometimes it needs to clear in progress of interpolation drive. In such case, AxmContiWriteClear API is used.

```
// Clear data of interpolation queue.
long lCoordinate = 0;
AxmContiWriteClear(lCoordinate);
```

```
// Ex13_AXM_ContiInterpolation.cpp:Defines the entry point for the console application.
// Start continuous interpolation drive after home setting of 2 axes mechanical part.
#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_1 1

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.

  long lNodeTotal;
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");
           for(int nAxisNo = 0; nAxisNo<2; nAxisNo++)
           {
              // Set Active level of +End limit and -End limit on axis 0 to HIGH and
              // emergency stop.
              AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
              // Set input level of inposition signal on axis 0 to HIGH.
              AxmSignalSetInpos(nAxisNo, HIGH);
              // Set input level of alarm signal on axis 0 to LOW.
              AxmSignalSetServoAlarm(nAxisNo, LOW);
```

```
        // Set Active input level of ESTOP signal on axis 0 to HIGH.
        AxmSignalSetStop(nAxisNo, 0, HIGH);

        // Make ordered value on axis 0 to be mm unit.
        AxmMotSetMoveUnitPerPulse(nAxisNo, 10, 10000);
        // Set initial velocity on axis 0 to 1.  Default : 1
        AxmMotSetMinVel(nAxisNo, 1);
        // Make pulse output method on axis 0 to TwoCwCcwHigh.
        AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
        // Set encoder input method on designated axis to 4 multiplication.
        AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);

        // Set to relative coordination drive.
        AxmMotSetAbsRelMode(nAxisNo, 1);
        // Set to symmetrical S-curve velocity profile.
        AxmMotSetProfileMode(nAxisNo, 3);

        // Set home search method.
        AxmHomeSetMethod(nAxisNo,-1,HomeSensor,1,2000.0,0.0);
        // Set velocity that will be used during home search.
        AxmHomeSetVel(nAxisNo, 100, 20, 10, 10, 200, 40);
        // Servo On.
        AxmSignalServoOn(nAxisNo, ENABLE);
    }

    // Search home.
    printf("Start home search pressing any key.\n");
    getch();
    AxmHomeSetStart(AXIS_0);
    AxmHomeSetStart(AXIS_1);

    // Verify current home search progress rate.
    DWORD uHomeResult_0, uHomeResult_1;
    DWORD uHomeMainStepNumber , uHomeStepNumber_0,uHomeStepNumber_1;
    AxmHomeGetResult(AXIS_0, &uHomeResult_0);
    AxmHomeGetResult(AXIS_1, &uHomeResult_1);
    while(uHomeResult_0 == HOME_SEARCHING || uHomeResult_1 == HOME_SEARCHING)
    {
        AxmHomeGetResult(AXIS_0, &uHomeResult_0);
        AxmHomeGetResult(AXIS_1, &uHomeResult_1);
        AxmHomeGetRate(AXIS_0,  &uHomeMainStepNumber , &uHomeStepNumber_0);
        AxmHomeGetRate(AXIS_1,  &uHomeMainStepNumber , &uHomeStepNumber_1);
        printf("\rIn Home Search... axis 0 : %d%, axis 1 : %d % ", uHomeStepNumber_0,
            uHomeStepNumber_1);
    }

    printf("\n Move with straight line interpolation drive to relative position
        (50,25).");
    // Carry out mapping of 0 and 1 axis to coordination system of 0 number that is
```

```
// to use for continuous interpolation drive.
long lCoordinate = 0;
long lSize = 2;
long lRealAxesNo[2] = {0,1};
double dCenPos[2];  double dEndPos[2]; double dMidPos[2];
AxmContiSetAxisMap(lCoordinate, lSize, lRealAxesNo);

// Delete existing continuous interpolation data.
AxmContiWriteClear(lCoordinate);

// Set 0 number coordination system to relative position drive.
AxmContiSetAbsRelMode(lCoordinate, 1);

double dpPosition[2] = {50,25};
AxmLineMove(lCoordinate, dpPosition, 100, 200, 200);

// Wait until all motions are exited.
BOOL WaitForDone = TRUE;
DWORD uStatus0, uStatus1;
while(WaitForDone) {
   AxmStatusReadInMotion(AXIS_0, &uStatus0);
   AxmStatusReadInMotion(AXIS_1, &uStatus1);
   if(uStatus0 == 0 && uStatus1 == 0)
      WaitForDone = FALSE;
}

// Input data on interpolation queue to carry out continuous drive to relative
// position.
for(int i=0;i<5;i++)               // Repeat 5 times.
{
   // Input data on interpolation queue to move to (100,0),
   // (velocity : 100, acceleration : 200).
   AxmContiBeginNode(lCoordinate); // Start interpolation registration.

   dpPosition[0] = 100; dpPosition[1] = 0;
   AxmLineMove(lCoordinate, dpPosition, 100, 200, 200);
   dCenPos[0] = 0; dCenPos[1] = 25;
   dEndPos[0] = 25; dEndPos[1] = 25;
   // Input data on interpolation queue to carry out circular interpolation
   // counterclockwise with center of (0,25) to (25,25).
   AxmCircleCenterMove(lCoordinate, lRealAxesNo, dCenPos, dEndPos, 100, 200,
      200, 0);

   // Input data on interpolation queue to move to (0,100).
   dpPosition[0] = 0; dpPosition[1] = 100;
   AxmLineMove(lCoordinate, dpPosition, 100, 200, 200);
   dEndPos[0] = 25; dEndPos[1] = -25;
   // Input data on interpolation queue to carry out circular interpolation with
   // radius of 25 to (-25,25).
```

```
            AxmCircleRadiusMove(lCoordinate, lRealAxesNo, 25, dEndPos ,100, 200, 200, 0,
               0);

            // Input data on interpolation queue to move to (-100,0).
            dpPosition[0] = -100; dpPosition[1] = 0;
            AxmLineMove(lCoordinate, dpPosition, 100, 200, 200);
            dCenPos[0] = 0; dCenPos[1] = -25;
            // Input data on interpolation queue to carry out circular interpolation with
            // a right angle and with center of (0,-25).
            AxmCircleAngleMove(lCoordinate, lRealAxesNo, dCenPos, 90, 100, 200, 200, 0);

            // Input data on interpolation queue to move to (0,-100).
            dpPosition[0] = 0; dpPosition[1] = -100;
            AxmLineMove(lCoordinate, dpPosition, 100, 200, 200);
            dMidPos[0] = 7; dMidPos[1] = -18;
            dEndPos[0] = 25; dEndPos[1] = -25;

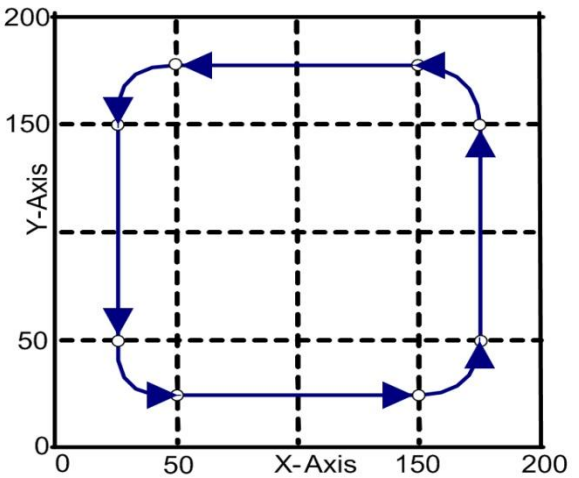
            // Input data on interpolation queue to carry out circular interpolation to
            // (25,-25) passing (7,-18) and using 0 and 1 axis.
            AxmCirclePointMove(lCoordinate, lRealAxesNo, dMidPos, dEndPos, 100, 200,
               200,0);
            AxmContiEndNode(lCoordinate);    // End of interpolation queue registration
         }

         AxmContiGetTotalNodeNum(lCoordinate, &lNodeTotal);
         printf("\n Total of %d number of node is stored currently." , lNodeTotal);
         printf("\n Start continuous interpolation with velocity specified mode using
            stored interpolation data, pressing any key.");
         getch();
         AxmContiStart(lCoordinate, 0, 0);
         printf("\n Stop immediately continuous drive pressing any key.");
         getch();
         AxmMoveSStop(0); // Make to stop master axis of lCoordinate.
         getch();
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
            Code);
   }
   else
      printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
   printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if (AxlClose())
   printf("\n Library is exited.\n");
else
```

```
    printf("\n Library is not exited normally.\n");
}
```

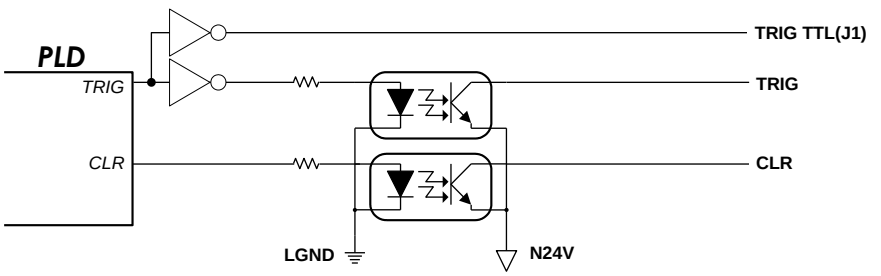
※ *If an example of straight line interpolation drive above is carried out, following result can be verified.*



CRC Signal Setting

In case of specified servo pack, at the point of drive completion or when limit sensor is detected, it occurs that deletes remaining pulse servo pack has with CRC(Current Remaining Clear) signal from outside. For this, signal pin is assigned and provided to connect to servo pack directly. Also, API is provided to set whether CRC signal is used, output level, whether limit signal is used, and so on. In addition, API is provided so that user can generate CRC signal arbitrarily.

Output level is being of reversed level of chip output, i.e. when chip output level is logic "1," CLR output is N24V, and when of logic "0," output is open collector. All the signals are isolated to external signal with photo-coupler same as universal output. These signals are outputted by external specified position or condition, and time delay is by the time passing through Photo-Coupler and by the time delay of PLD internal circuit. In case of CLR signal which is inputted PELM and NELM, approximately 6㎲~8㎲ of time delay is occurred. Since Photo Coupler output is a type of Normally Open, to obtain 24V level of voltage, user shall put on pull-up resistance. Pull-up resistance uses approximately 10㏀ of resistance.



Note) Functions CRC signal set the pulse output which is a feature available only in motion controller (PCI-N804/404, RTEX-PM).

● *Setting CRC Signal Level*

When mechanical part is put on End Limit, it can be set to generate CRC signal automatically. For this, AxmCrcSetMaskLevel API that sets CRC signal level.

```
// Set initially to use CRC on axis 0. If level setting is wrong, motor may not be operated.
// Set whether CRC signal is used on axis 0.
long lAxisNo = 0;
DWORD uLevel = HIGH;
AxmCrcSetMaskLevel(lAxisNo, uLevel, 6);        // Set 100msec of CRC signal range.
```

● *Force output setting API to Program*

No CRC signal is needed only when it is detected by limit sensor. CRC signal is needed to generate by conditions, and in this case, AxmCrcSetOutput API is used.

```
// Set initially to use CRC on axis 0. If level setting is wrong, motor may not be operated.
// Set whether CRC signal is used on axis 0.
 long lAxisNo = 0;
DWORD uLevel = HIGH;
AxmCrcSetMaskLevel(lAxisNo, uLevel);

// Send CRC signal on axis 0 by force.
DWORD uUse = ENABLE;                        // 0: DISABLE, 1:ENABLE.
AxmCrcSetOutput(lAxisNo, uUse);
```

```
// Ex14_AXM_AdvancedCRC.cpp : Defines the entry point for the console application.
// Set and verify CRC output.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           // Set Active level of +End limit and -End limit on axis 0 to HIGH.
           AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
           // Set input level of inposition signal on axis 0 to HIGH.
           AxmSignalSetInpos(AXIS_0, HIGH);
           // Set input level of alarm signal on axis 0 to LOW.
           AxmSignalSetServoAlarm(AXIS_0, LOW);
           // Set Active input level of ESTOP signal on axis 0 to HIGH.
           AxmSignalSetStop(AXIS_0, 0, HIGH);

           // Make ordered value on axis 0 to be mm unit.
```

```
AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
// Set initial velocity on axis 0 to 1.  Default : 1
AxmMotSetMinVel(AXIS_0, 1);
// Make pulse output method on axis 0 to TwoCwCcwHigh.
AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
// Set encoder input method on designated axis to 4 multiplication.
AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

// Servo On.
AxmSignalServoOn(AXIS_0, ENABLE);

// Set initially to use CRC on axis 0.
// If level setting is wrong, motor may not be operated,
// Set whether CRC signal is used on axis 0.
long lAxisNo = 0;
DWORD uLevel = HIGH;
AxmCrcSetMaskLevel(lAxisNo, uLevel, 6);

// help Message.
printf("[INFORMATION]*************** \n");
printf("[ESC] : Exit \n");
printf("[1] : Output to CRC HIGH \n");
printf("[2] : Output to CRC LOW \n");
printf("****************************** \n");
DWORD uCRCOnOff = 2;
BOOL fExit = FALSE;
while (!fExit)                  // Infinite loop.
{
   if (kbhit())                 // Press any key.
   {
      int ch = getch();
      switch (ch)
      {
      case 27:                  // Esc key.
           fExit = TRUE;
           AxmSignalServoOn(AXIS_0, DISABLE);  // Servo Off.
      break;

      case '1':         // HIGH output of CRC signal by program and by force.
            AxmCrcSetOutput(AXIS_0, ENABLE);
      break;

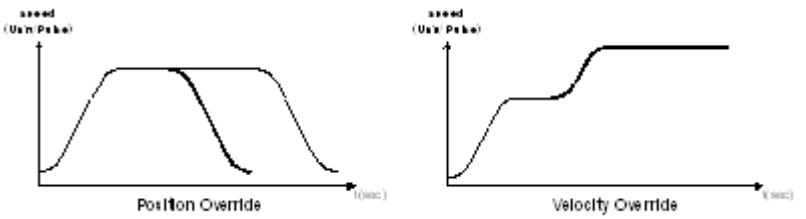
      case '2':         // LOW output of CRC signal by program and by force.
            AxmCrcSetOutput(AXIS_0, DISABLE);
      break;
      }
   }
   // Verify current CRC output.
   AxmCrcGetOutput(AXIS_0, &uCRCOnOff);
```

```
                printf("\rCRC Output : %x",uCRCOnOff);
            }     // End of While()
        }
        else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
                Code);
    }
    else
        printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
    printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

  // Exit library.
  if (AxlClose())
    printf("Library is exited.\n");
  else
    printf("Library is not exited normally.\n");
}
```

Override Drive

In this chapter, it will introduce velocity and position override drive. Override drive means change of work velocity during motion is in progress, and change of target distance during motion is in progress.



Position override uses AxmOverridePos API and velocity override uses AxmOverrideVel API. Override API must be called during in drive and is ignored when called after drive is ended. and It is available for the location override to turn the direction to the opposite only once.

```
// Move axis 0 to 100 of absolute coordination with symmetric S-curve drive mode
// (velocity 100, acceleration 200, exit API at start point).
long lAxisNo = 0;
AxmMotSetAbsRelMode(lAxisNo, 0);
AxmMotSetProfileMode(lAxisNo, 3);
AxmOverrideSetMaxVel(lAxisNo, 500);      // Set maximum velocity of override velocity.
// This API must be used to increase velocity than of initially assigned.
AxmMoveStartPos(lAxisNo, 100, 100, 200, 200);

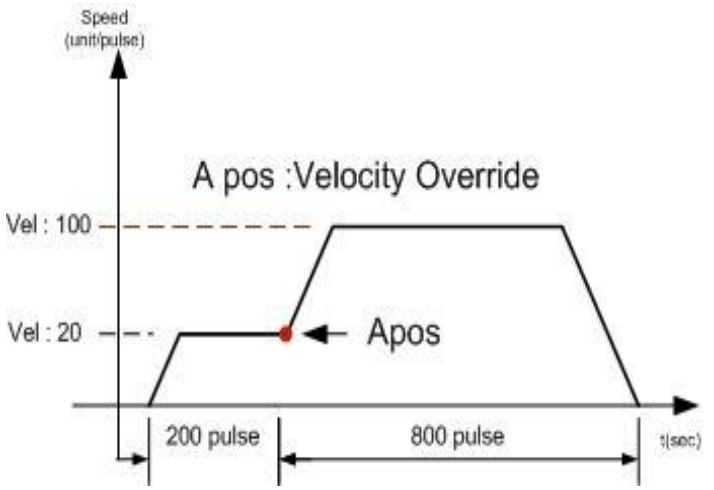
AxmOverrideVel(lAxisNo, 500);            // Adjust velocity to 500.
// Adjust to 2000 of previous absolute coordination before axis 0 drive is ended.
double dOverridePosition = 2000;
AxmOverridePos(lAxisNo, dOverridePosition);

// Adjust to 50 of previous velocity before axis 0 drive is ended.
double dOverrideVelocity = 50;
AxmOverrideVel(lAxisNo, dOverrideVelocity);
```

AxmOverrideVelAtPos API is that carries out velocity override at the position of input velocity to a certain position point and override, and can be used to override after passing a specified position. This API has a difference that is not the one that changes in drive but that starts drive with corresponding information at stop state.

```
// Move axis 0 to 1000, and carry out override with velocity of 100 when passing
// 200 position.
long lAxisNo = 0;
double dMaxVelocity = 20;
double dMaxAccel = 100;
```

```
double dMaxDecel = 100;
double dOverrideVelocity = 100;
double dOverridePosition = 200;
double dPosition = 1000;
long lTarget = 1;   // [0] Command Position , [1] Actual Position.
AxmOverrideVelAtPos(lAxisNo, dPosition, dMaxVelocity, dMaxAccel, dMaxDecel,
  dOverridePosition, dOverrideVelocity, lTarget);
```




```
// Ex15_AXM_OverrideDrive.cpp : Defines the entry point for the console application.
// Override Drive Example

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library is initialized.\n");

      // Inspect whether motion module exists or not.
      DWORD dwStatus;
      Code = AxmInfoIsMotionModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
```

```
{
   if(dwStatus == STATUS_EXIST)
   {
      printf("Motion module exists.\n");

      // Set Active level of +End limit and -End limit on axis 0 to HIGH.
      AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
      // Set input level of inposition signal on axis 0 to HIGH.
      AxmSignalSetInpos(AXIS_0, HIGH);
      // Set input level of alarm signal on axis 0 to LOW.
      AxmSignalSetServoAlarm(AXIS_0, LOW);
      // Set Active input level of ESTOP signal on axis 0 to HIGH.
      AxmSignalSetStop(AXIS_0, 0, HIGH);

      // Make ordered value on axis 0 to be mm unit.
      AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
      // Set initial velocity on axis 0 to 1.  Default : 1
      AxmMotSetMinVel(AXIS_0, 1);
      // Make pulse output method on axis 0 to TwoCwCcwHigh.
      AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
      // Set encoder input method on designated axis to 4 multiplication.
      AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

      // Use relative S-curve velocity profile.
      AxmMotSetAbsRelMode(AXIS_0, 1);
      AxmMotSetProfileMode(AXIS_0, 3);

      // Servo On.
      AxmSignalServoOn(AXIS_0, ENABLE);

      // help Message.
      printf("[INFORMATION]********************************** \n");
      printf("[ESC] : Exit \n");
      printf("[1] : Start drive with velocity of 100 towards relative coordination 100
         \n");
      printf("[2] : Override position to 200 \n");
      printf("[3] : Override velocity to 50 \n");
      printf("[4] : Start drive up to -200 and override velocity to 50 at -100 \n");
      printf("********************************************** \n");

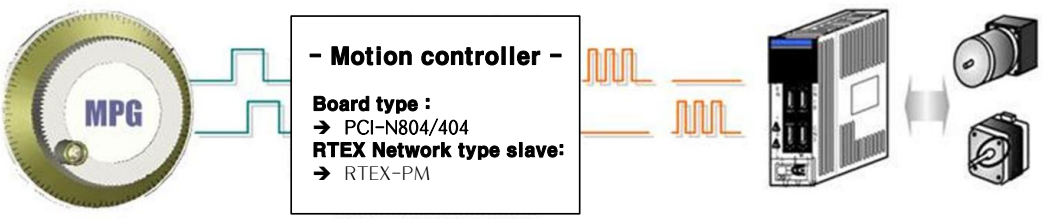
      BOOL fExit = FALSE;
      while (!fExit)                  // Infinite loop.
      {
         if (kbhit())                 // Press any key.
         {
            int ch = getch();
            switch (ch)
            {
               case 27:               // Esc key.
```

```
                        fExit = TRUE;
                        AxmSignalServoOn(AXIS_0, DISABLE);  // Servo Off.
                    break;
                    case '1':
                    // Move axis 0 to 100 position with relative coordination with symmetric
                    // S-curve drive mode.
                        AxmOverrideSetMaxVel(AXIS_0, 500);
                        // Set maximum velocity of override velocity
                        AxmMoveStartPos(AXIS_0, 100, 100, 200, 200);
                    break;
                    case '2':
                    // Adjust to 200 of coordination before drive ends on axis 0.
                        AxmOverridePos(AXIS_0, 200);
                    break;
                    case '3':
                    // Adjust velocity to 50 before drive ends on axis 0.
                        AxmOverrideVel(AXIS_0, 50);
                    break;
                    case '4':
                    // axis 0 starts drive up to 200, and override velocity to 50 when
                    // of 100 position.
                        AxmOverrideVelAtPos(AXIS_0,-200,100,200,200,-100,50, COMMAND);
                 break;
                 }
              }
           } // End of While()
        }
        else
           printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
              Code);
     }
     else
        printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
     printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

  // Exit library.
  if (AxlClose())
     printf("Library is exited.\n");
  else
     printf("Library is not exited normally.\n");
}
```

MPG Drive (Manual Pulse Generation)

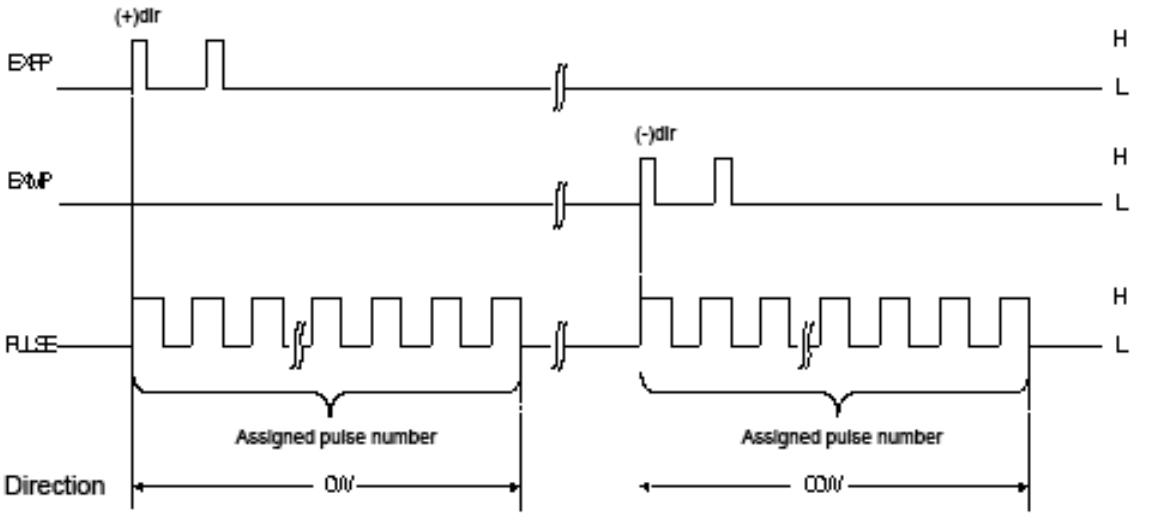
Direction signal is decided by input of EXPP, EXMP signal, and number of pulse is decided by input option setting resister. Output pulse velocity is outputted by set Object Speed.



Note) MPG drive outputs pulse which is a feature available only in PCI-N804/404, RTEX-PM.

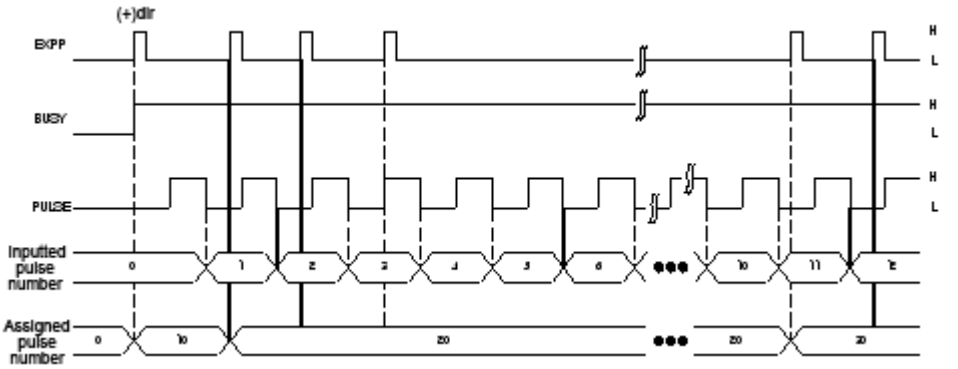
● Position drive by EXPP/EXMP (CAMC-IP)

Preset Mode outputs pulse with position drive type as much as number of pulse that is set in advance at output point of pulse. If Edge signal of EXPP/EXMP signal is inputted in drive, it carries out override as much as number of pulse that is set. Following figure corresponds to the case when EXPP/EXMP input option is single phase.



But, if Edge signal is inputted continuously, it ignores by inputted signal or decides override in comparison with outputted pulse number.

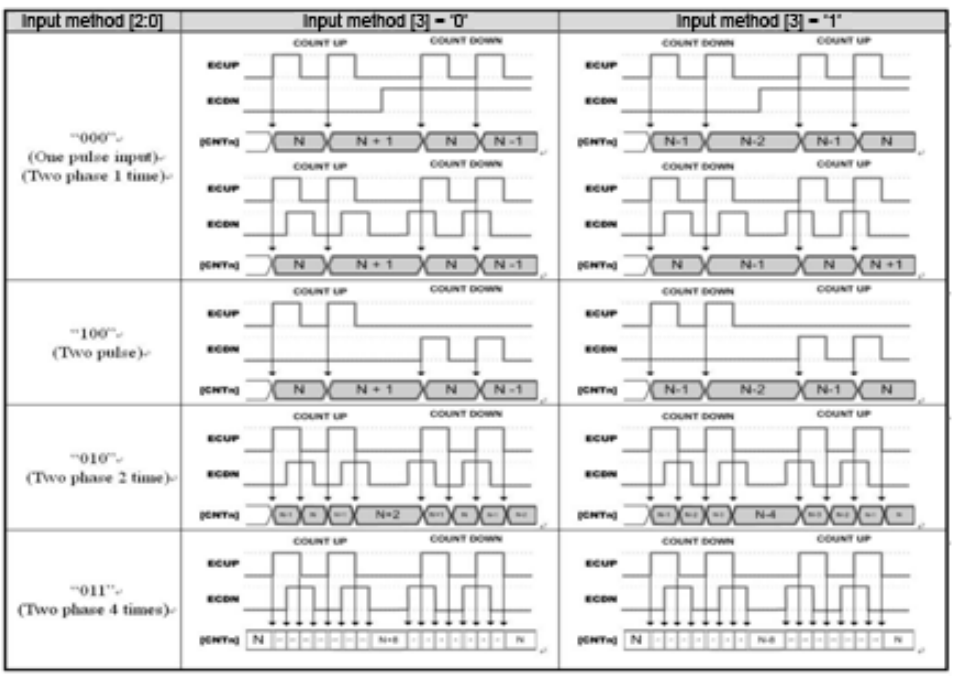
For example, when 10 of assigned pulse number is drived, it shows following figure.



● *Velocity drive by EXPP/EXMP (CAMC-QI)*

Cont Mode outputs pulse during EXPP/EXMP signal is to be Active Level. Drive type is same with Velocity Drive. While EXPP signal is Low('0') and EXMP is High('1'), it drives to (+) direction, and while EXMP signal is Low('0') and EXPP is High('1'), it drives to (-) direction.

Input pulse type

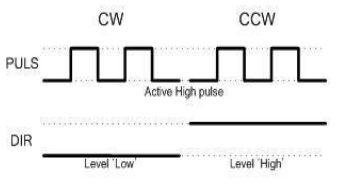
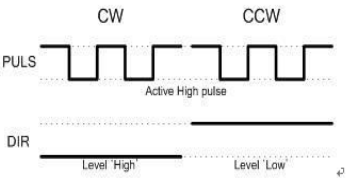
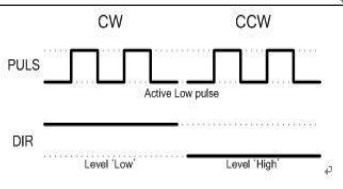
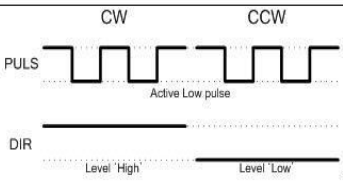
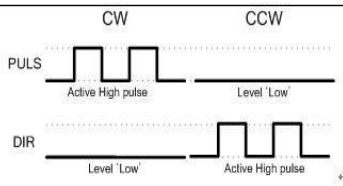
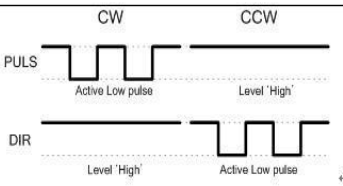
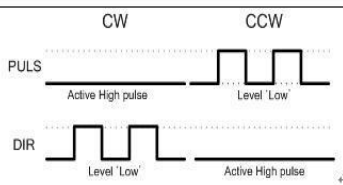
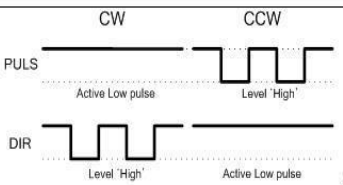
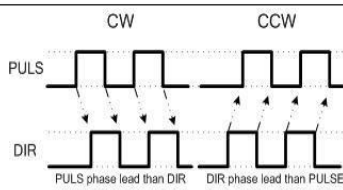
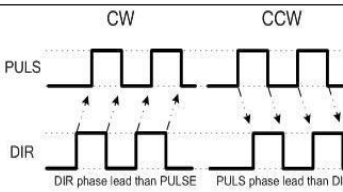


● *Interface with MPG outside*

Two signals of EXPP and EXMP is used to input signal of external rotary encoder of SMC-2V03, PCI-

N804/404 and connected with CAMC-QI chip through maximum of 1.2MHz high speed photo coupler.

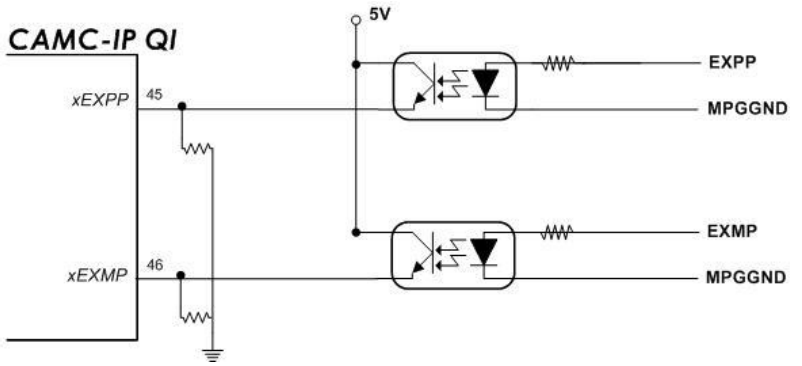
Output pulse type

[UCFG3][2~0]	[UCFG3][3] = '0'	[UCFG3][3] = '1'
"000" (One pulse mode)		
"001" (One pulse mode)		
"010" (Two pulse mode)		
"011" (Two pulse mode)		
"100" (Two phase mode)		

● *Interface with MPG outside*

Two signals of EXPP and EXMP is used to input signal of external rotary encoder of SMC-2V03 and PCI-N804/404, and connected with SMC-2V03 AND PCI-N804/404 chip through maximum of 1.2MHz high speed photo coupler.

Output pulse type



PCI-N804/404

MPG input signal is not inputted individually on each axis but used jointly by two axes. MPG signal inputted on axis 0 is used on axis 2 jointly, inputted on axis 1 is used on axis 3, inputted on axis 4 is used on axis 6, inputted on axis 5 is used on axis 7 jointly. That is, MPG signal input pin of external connect J2/J4 connected with 2/3/6/7 axes is not used.

● *EXPP/EXMP signal verification*

There is no API for verification of current state of EXPP / EXMP signal, but since Mechanical Signal of number 8 and 9 bit is connected to EXMP and EXPP each, current state can be verified through this. To verify Mechanical Signal, AxmStatusReadMechanical API is used, and EXPP and EXMP are verified as following with bit operation.

```
// Verify EXPP / EXMP signal on PCI-N804/404 chip.
AxmStatusReadMechanical(AXIS_0, &MechanicalStatus);
if(MechanicalStatus & QIMECHANICAL_EXMP_LEVEL) { bExmp = 1; } else { bExmp = 0; }
if(MechanicalStatus & QIMECHANICAL_EXPP_LEVEL) { bExpp = 1; } else { bExpp = 0; }
printf("\rEXPP : %x , EXMP : %x ",bExpp,bExmp);
```

QIMECHANICAL_EXMP_LEVEL , QIMECHANICAL_EXPP_LEVEL means

bit 11 and 12 of MechanicalStatus on PCI-N804/404.

● *Use of MPG drive API*

When MPG drive is set, from then, it drives by signal input to EXPP EXMP pin. Set as follow to use position drive mode.

```
// Set axis 0 to move by 10 per input pulse of position drive mode.
AxmMPGSetRatio(0, 10, 4096 );
AxmMPGSetEnable(0, MPG_DIFF_ONE_PHASE, 0, 10, 200, 400);

// Return Mpg signal setting on axis 0.
long lpInputMethod, lpDriveMode;
double dpVel, dpAccel, dpDecel, dpMPGPos;
double dMPGdenominator, dMPGnumerator;
```

```
AxmMPGGetEnable(0, &lpInputMethod, & lpDriveMode, &dpMPGPos, & dpVel);
AxmMPGGetRatio(0, &dMPGnumerator, &dMPGdenominator);
```

Here, dMPGdenominator = 4096 and MPGnumerator=1 mean output per 1 pulse with 1:1 as is if it is 200 pulse per rotation of MPG. If dMPGdenominator = 4096 and MPGnumerator=2, it means that send output per 2 pulses with 1:2.

Formula of MPG PULSE = ((Numerator) * (Denominator)/ 4096) is to be outputted to chip inside.

If needed to clear MPG drive, do the following.

```
// Clear MPG drive on axis 0.
long lAxisNo = 0;
AxmMPGReset(lAxisNo);
```

```
// Ex17_AXM_QIMPG.cpp : Defines the entry point for the console application.
// Start MPG drive.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           // Set Active level of +End limit and -End limit on axis 0 to HIGH.
           AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
           // Set input level of inposition signal on axis 0 to HIGH.
           AxmSignalSetInpos(AXIS_0, HIGH);
```

```
// Set input level of alarm signal on axis 0 to LOW.
AxmSignalSetServoAlarm(AXIS_0, LOW);
// Set Active input level of ESTOP signal on axis 0 to HIGH.
AxmSignalSetStop(AXIS_0, 0, HIGH);

// Make ordered value on axis 0 to be mm unit.
AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
// Set initial velocity on axis 0 to 1.  Default : 1
AxmMotSetMinVel(AXIS_0, 1);
// Make pulse output method on axis 0 to TwoCwCcwHigh.
AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
// Set encoder input method on designated axis to 4 multiplication.
AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

// Use relative position S-curve velocity profile.
AxmMotSetAbsRelMode(AXIS_0, 1);
AxmMotSetProfileMode(AXIS_0, 3);

// Servo On.
AxmSignalServoOn(AXIS_0, ENABLE);

// help Message.
printf("[INFORMATION]********************************** \n");
printf("[ESC] : Exit. \n");
printf("[1] : MPG_POS_DRV. \n");
printf("[2] : MPG_CONTI_DRV. \n");
printf("********************************************** \n");

BOOL fExit = FALSE;
DWORD MechanicalStatus;
BOOL bExmp, bExpp;
double dVelocity   = 2000;          // Setting value of velocity use.
double dAccel      = 4000;          // Setting value of acceleration use.
double dDecel      = 4000;          // Setting value of deceleration use.
long  lInputMethod;
long  lDriveMode;
long  lDirMode    =0;               // Non-used on PCI-N804/404.
long  lUserDir    =0;               // Non-used on PCI-N804/404.
double dMPGPosition = 10;           // Moving distance if MPG_PRESET_MODE mode.
double dMPGdenominator = 4095;
// During MPG(input manual pulse generation device) drive, division
// value(calculated as +1 of input value).
double dMPGnumerator = 0;
// During MPG(input manual pulse generation device) drive, multiplication
// value(calculated as +1 of input value).
while (!fExit)                      // Infinite loop.
{
   if (kbhit())                     // Press any key.
   {
```

```
            int ch = getch();
            switch (ch)
            {
               case 27:                    // Esc key.
                  fExit = TRUE;
                  AxmMPGReset(AXIS_0);    // Exit MPG drive.
                  AxmSignalServoOn(AXIS_0, DISABLE); // Servo Off.
               break;

               case '1':
                  // Exit MPG drive.
                  AxmMPGReset(AXIS_0);

                  printf("\n Start MPG drive with MPG_PRESET_MODE.\n");
                  lInputMethod= 0;        // Input signal method MPG_DIFF_ONE_PHASE (0)
                  lDriveMode = 1;         // MPG_CONTI_DRV (0), MPG_POS_DRV (1)
                  // MPG_ABS_COMMAND_POS(2), MPG_ABS_ACTUAL_POS(3),
                  // MPG_ABS_COMMAND_ZERO(4), MPG_ABS_ACTUAL_ZERO(5)

                  AxmMPGSetRatio(AXIS_0, dMPGnumerator, dMPGdenominator );
                  AxmMPGSetEnable(AXIS_0,lInputMethod,lDriveMode,dMPGPosition,
                     dVelocity, dAccel);
               break;

               case '2':
                  // Exit MPG drive.
                  AxmMPGReset(AXIS_0);

                  printf("\n Start MPG drive with MPG_CONT_MODE.\n");
                  lInputMethod = 4;
                  // Input signal method MPG_LEVEL_ONE_PHASE(4)
                  lDriveMode = 0;         // MPG_CONTI_DRV (0), MPG_POS_DRV (1)
                  // MPG_ABS_COMMAND_POS(2), MPG_ABS_ACTUAL_POS(3),
                  // MPG_ABS_COMMAND_ZERO(4), MPG_ABS_ACTUAL_ZERO(5)

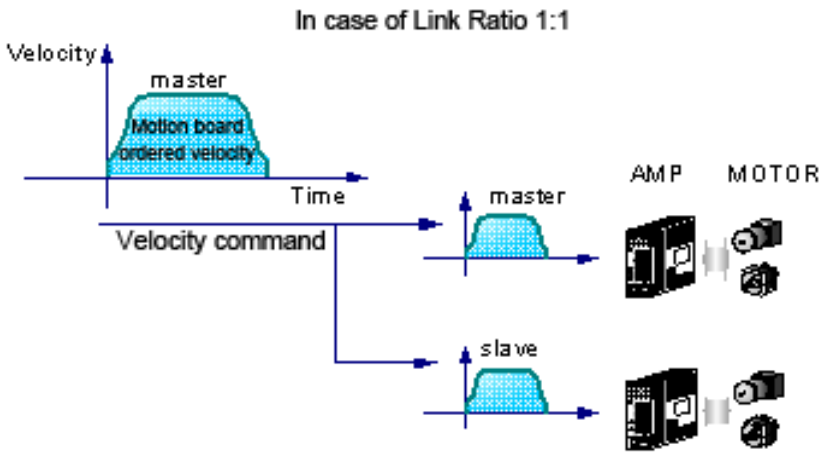
                  AxmMPGSetEnable(AXIS_0, lInputMethod, lDriveMode,dMPGPosition,
                     dVelocity, dAccel);
                  AxmMPGSetRatio(AXIS_0, dMPGnumerator, dMPGdenominator );
               break;
            }
         }
         AxmStatusReadMechanical(AXIS_0, &MechanicalStatus);
         if(MechanicalStatus & QIMECHANICAL_EXMP_LEVEL) { bExmp = 1; }
         else { bExmp = 0; }
         if(MechanicalStatus & QIMECHANICAL_EXPP_LEVEL) { bExpp = 1; }
         else { bExpp = 0; }
         printf("\rEXPP : %x , EXMP : %x ",bExpp,bExmp);
      }  // End of While()
   }
```

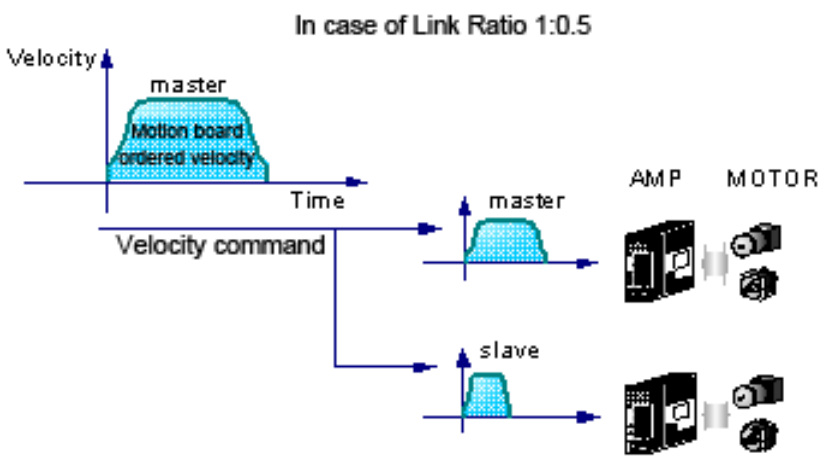
```
        else
           printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
              Code);
     }
     else
        printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
     printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

  // Exit library.
  if (AxlClose())
     printf("Library is exited.\n");
  else
     printf("Library is not exited normally.\n");
}
```

Sync Drive

Sync drive is also called Electric Gear Mode, and it is a method to control independent two axes as one axis. User set a specific master and slave axis, and set drive ratio between two axes, then slave axis moves together with drive ratio set when master axis moves.





To carry out this sync drive, it needs to set through AxmLinkSetMode API. Sync drive starts after AxmLinkSetMode API is called.

```
// Set to drive with 2:1 ratio, setting that axis 0 is to be Master and 1 axis is
// to be Slave.
long lMasterAxisNo = 0;
long lSlaveAxisNo = 1;
```

```
double dSlaveRatio = 0.5;
AxmLinkSetMode(lMasterAxisNo, lSlaveAxisNo, dSlaveRatio);

// Drive axis 0 to as much as 100.
AxmMoveStartPos(AXIS_0, 100, 100, 200, 200);
```

If set this, while axis 0 drives as much as 100, 1 axis drives along as much as 50 with 2:1 ratio. To exit sync drive, use AxmLinkResetMode API.

```
// Clear sync drive mode composed of 0 and 1 axis.
long lMasterAxisNo = 0;
AxmLinkResetMode(lMasterAxisNo);
```

```
// Ex18_AXM_ElectricGearMode.cpp : Defines the entry point for the console application.
// Carry out sync drive.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_1 1

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library is initialized.\n");

      // Inspect whether motion module exists or not.
      DWORD dwStatus;
      Code = AxmInfoIsMotionModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            printf("Motion module exists.\n");

            for(int nAxisNo = 0; nAxisNo<2; nAxisNo++)
            {
               // Set Active level of +End limit and -End limit on axis 0 to HIGH.
               AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
               // Set input level of inposition signal on axis 0 to HIGH.
               AxmSignalSetInpos(nAxisNo, HIGH);
```

```
    // Set input level of alarm signal on axis 0 to LOW.
    AxmSignalSetServoAlarm(nAxisNo, LOW);
    // Set Active input level of ESTOP signal on axis 0 to HIGH.
    AxmSignalSetStop(nAxisNo, 0, HIGH);

    // Make ordered value on axis 0 to be mm unit.
    AxmMotSetMoveUnitPerPulse(nAxisNo, 10, 10000);
    // Set initial velocity on axis 0 to 1.  Default : 1
    AxmMotSetMinVel(nAxisNo, 1);
    // Make pulse output method on axis 0 to TwoCwCcwHigh.
    AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
    // Set encoder input method on designated axis to 4 multiplication.
    AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);

    // Use relative S-curve velocity profile.
    AxmMotSetAbsRelMode(AXIS_0, 1);
    AxmMotSetProfileMode(AXIS_0, 3);

    // Servo On.
    AxmSignalServoOn(nAxisNo, ENABLE);
}

// Start sync drive.
long lMasterAxisNo = AXIS_0;
long lSlaveAxisNo = AXIS_1;
double dSlaveRatio = 0.5;
AxmLinkSetMode(lMasterAxisNo, lSlaveAxisNo,dSlaveRatio);

// help Message.
printf("[INFORMATION]********************************** \n");
printf("Sync drive is set.\nDrive starts pressing any key.\n");
printf("********************************************** \n");

// Wait until any key is pressed.
getch();
// Drive axis 0 as much as 100.
AxmMoveStartPos(AXIS_0, 100, 100, 200, 200);
// Verify if it is in drive.
DWORD uStatus;
AxmStatusReadInMotion(AXIS_0, &uStatus);
while(uStatus) {
    AxmStatusReadInMotion(AXIS_0, &uStatus);
}

// Exit sync drive.
AxmLinkResetMode(AXIS_0);

// Servo Off.
AxmSignalServoOn(AXIS_0, DISABLE);
```

```
            AxmSignalServoOn(AXIS_1, DISABLE);
         }
         else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code
               0x%x\n",Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("Library is exited.\n");
   else
      printf("Library is not exited normally.\n");
}
```

Gantry Drive

To control Gantry drive system that two axes are linked mechanically showing on the picture below, Gantry drive API is provided. Using this API, if master axis (even number axis) set with Gantry control, corresponding slave axis is synchronized with master axis and drived. Even if drive or stop command is given to slave axis after Gantry setting, all commands are ignored.



Note) It is necessary to select the two axes gantry drive on the same board.

1) Setting And Cancellation Of Gantry Drive

To set Gantry drive, using AxmGantrySetEnable API, enable Gantry drive. Gantry system has a structure that each robot is arrayed side by side and linked mechanically. And home sensor of master and slave axis has a certain value of error. If home search is carried out, error that home sensor of two axes has is compensated at last. Error generated between sensors during machine assembly is to be set to dSlOffset.

```
// Start Gantry drive using 0 and 1 axis.
long lMasterAxisNo = 0;
long lSlaveAxisNo = 1;
DWORD uGantryOn;
DWORD bSlHomeUse = FALSE;       // [0] : Select home search only for Home axis
                                // [1] : Select home search together for Home and Slave axis
double dSlOffset = 0;
// Mechanical Offset value between home sensors of master and slave.
double dSlOffsetRange = 10;      // Error limit between master home sensor and slave home
                                // sensor during home search.
AxmGantrySetEnable(lMasterAxisNo, lSlaveAxisNo, uSlHomeUse, dSlOffset, dSlOffsetRange);
// Setting value set directly by user can be read.
AxmGantryGetEnable(lMasterAxisNo, &SlHomeUse, &dSlOffset, &dSlORange, &uGatryOn);
```

After this setting, 0 and 1 axis are drived identically by command for axis 0 which is master axis. And all commands for 1 axis are ignored. For home search, carry out home search API for axis 0 at the state.

```
// Carry out home search on Gantry drive.
long MasterAxisNo=  0;
long nHmDir = -1;       // 0 : (-) direction , 1 : (+) direction.
DWORD nHmSig = 4;       // [0]PELM, [1]NELM, [2]PSLM, [3]NSLM, [4]IN0, [5]IN1, [6]IN2, [7]IN3
BOOL bZphas = TRUE;     // Whether Z phase is detected.
double dHClrTime = 100.0;   // Waiting time for encoder clear.
double dHOffset  =  0.0;
AxmHomeSetMethod(MasterAxisNo, nHmDir, nHmSig, bZphas, dHClrTime, dHOffset);

double dVelF  =  10000;   // Drive velocity towards forward direction of home search when
                          // home sensor is not detected.
double dVelA  =  1000;    // Drive velocity towards reverse direction of home search when
                          // home sensor is detected.
double dVelL  =  500;     // Velocity to be used for re-search after primary home search.
double dvelI    =   10;   // Velocity to be used during last home search.
double dAccF  =  20000;
// Acceleration and deceleration to drive towards home sensor direction of home search when
// home sensor is not detected.
double dAccA  =  10000;   // Acceleration and deceleration to drive towards reverse
                          // direction when home sensor is detected.
AxmHomeSetVel(MasterAxisNo, dVelF, dVelA, dVelL, dvelI, dAccF, dAccA);

AxmHomeSetStart(MasterAxisNo);
```

To exit Gantry drive, use AxmGantrySetDisable API.

```
// Exit Gantry drive.
long lMasterAxisNo = 0;
long lSlaveAxisNo = 1;
AxmGantrySetDisable(lMasterAxisNo, lSlaveAxisNo);
```

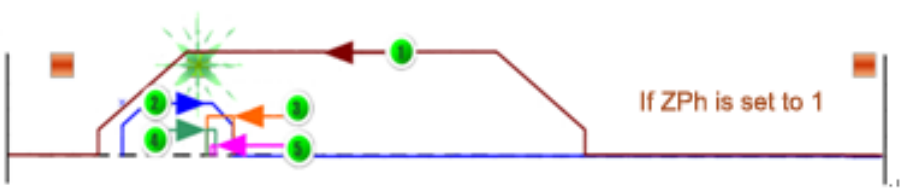
2) Home Search In Gantry Drive

Important thing in Gantry control is to carry out home search considering master and slave axis during home search. Generally, home sensor on master axis is searched first and then home sensor on slave axis is searched. At the beginning, if Gantry starts home drive after putting together with Gantry control of AxmGantrySetEnable, drive is carried out and home search is carried out using one-axis home search API for master axis.

Variable of Home Search	Explanation
HmSig	Signal setting for home sensor Example) Home sensor(4), +Limit sensor(0), etc
HmLev	Active Level setting of home sensor Example) 0: A contact, 1: B contact
HmDir	Initial search direction setting during home sensor detection Example) 1: (+) direction, 0: (−) direction
Zphas	Setting of whether encoder Z phase is detected after home sensor detection

VelFirst	Initial high speed detection velocity during home search (if initial home sensor is not detected)
VelSecond	Velocity coming out from reverse direction after primary sensor detection during home search
VelThird	Velocity progressing re-search after primary sensor detection during home search
VelLast	Final detection velocity setting during home search [decision for accuracy of home search]
AccFirst	Initial high speed detection acceleration during home search
AccSecond	Acceleration coming out from reverse direction after primary sensor detection during home search
HClrTim	Waiting time before clear of Command and Actual(Encoder) position after home search completion
HOffset	Moving Offset value during machine home re-search after home search completion
NSWLimit	(-) Software Limit value setting to apply after home search completion
PSWLimit	(+) Software Limit value setting to apply after home search completion (If NSWLimit value is greater than or same with PSWLimit value, S/W Limit is not set)

● *Understanding of Home Search Velocity setting and Home Search Sequence by Step (Master, Slave use same velocity)*



Step 1. Use VelFirst, AccFirst

Carry out high speed search of home sensor towards HmDir direction, and slow down stop with HAccF deceleration.

Step 2. Use VelSecond, AccSecond

Carry out Down Edge search of home sensor towards reverse HmDir direction, and slow down stop with HaccF deceleration.

Step 3. Use VelThird

Carry out Up Edge search of home sensor towards HmDir direction, and emergency stop.

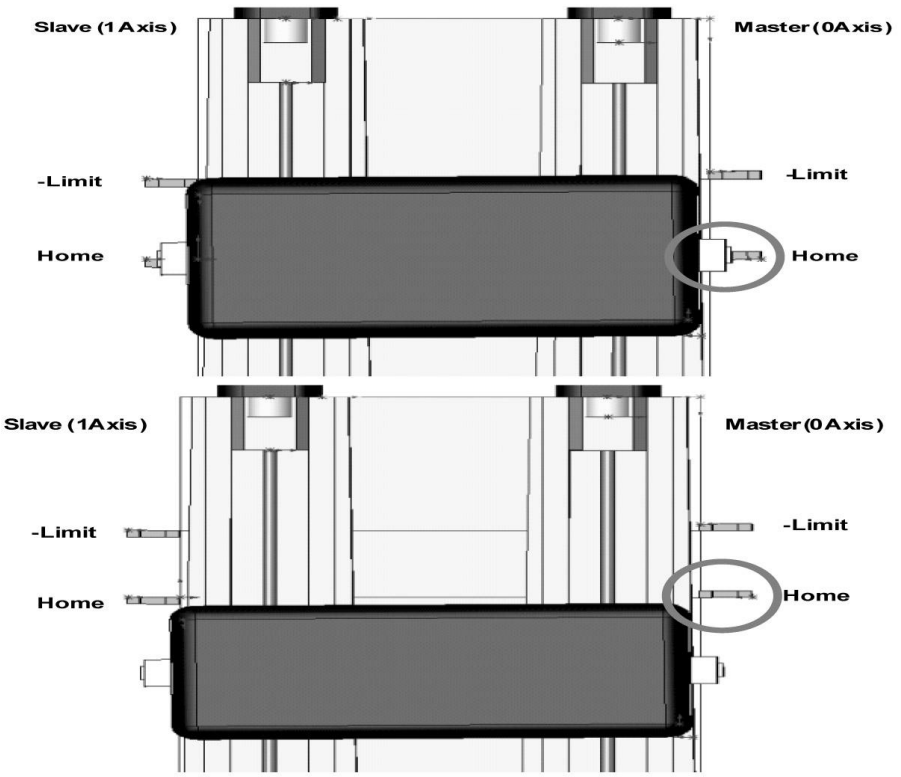
Step 4. Use VelThird

Carry out Down Edge search on Z phase towards reverse HmDir direction, and emergency stop.

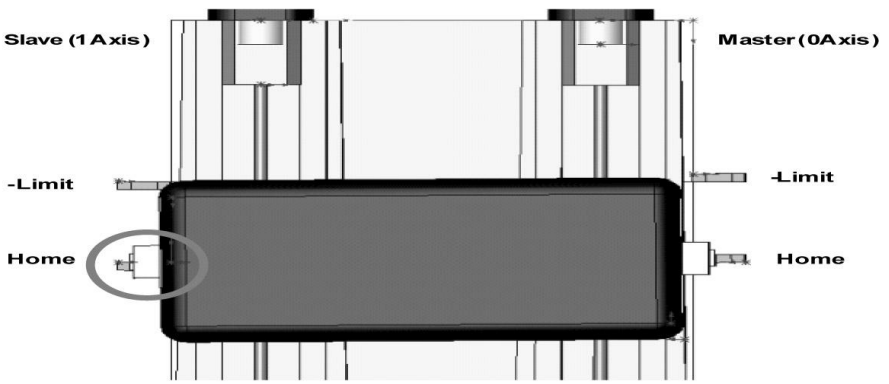
Step 5. Use VelLast

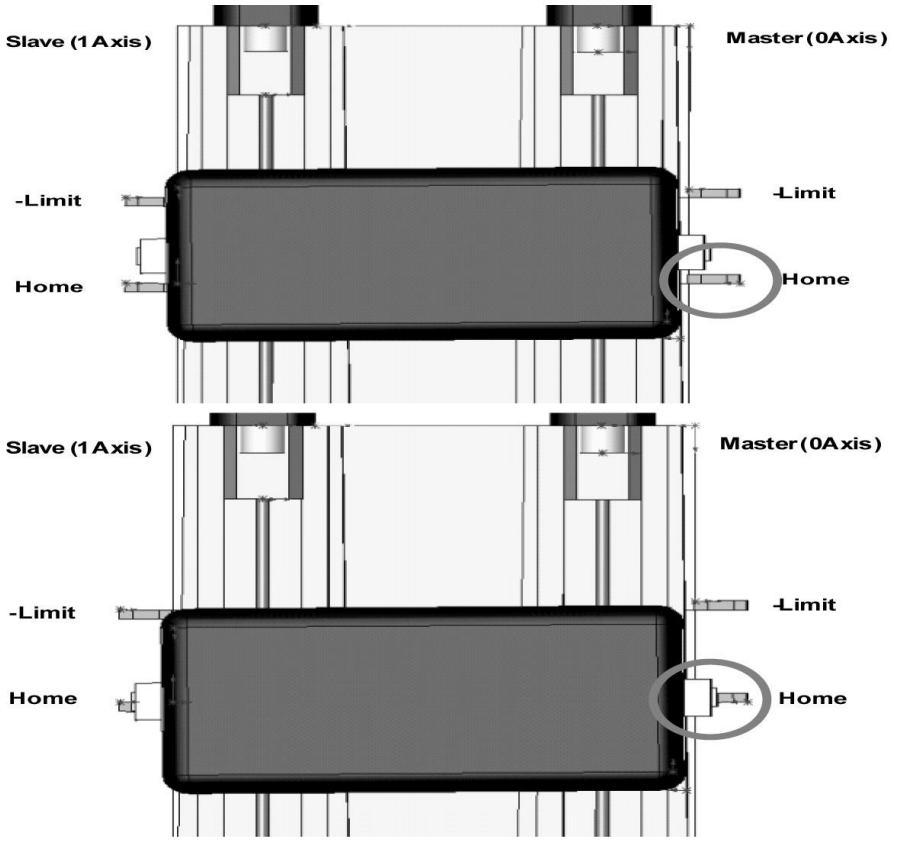
Carry out Up Edge search on Z phase towards HmDir direction, and emergency stop.

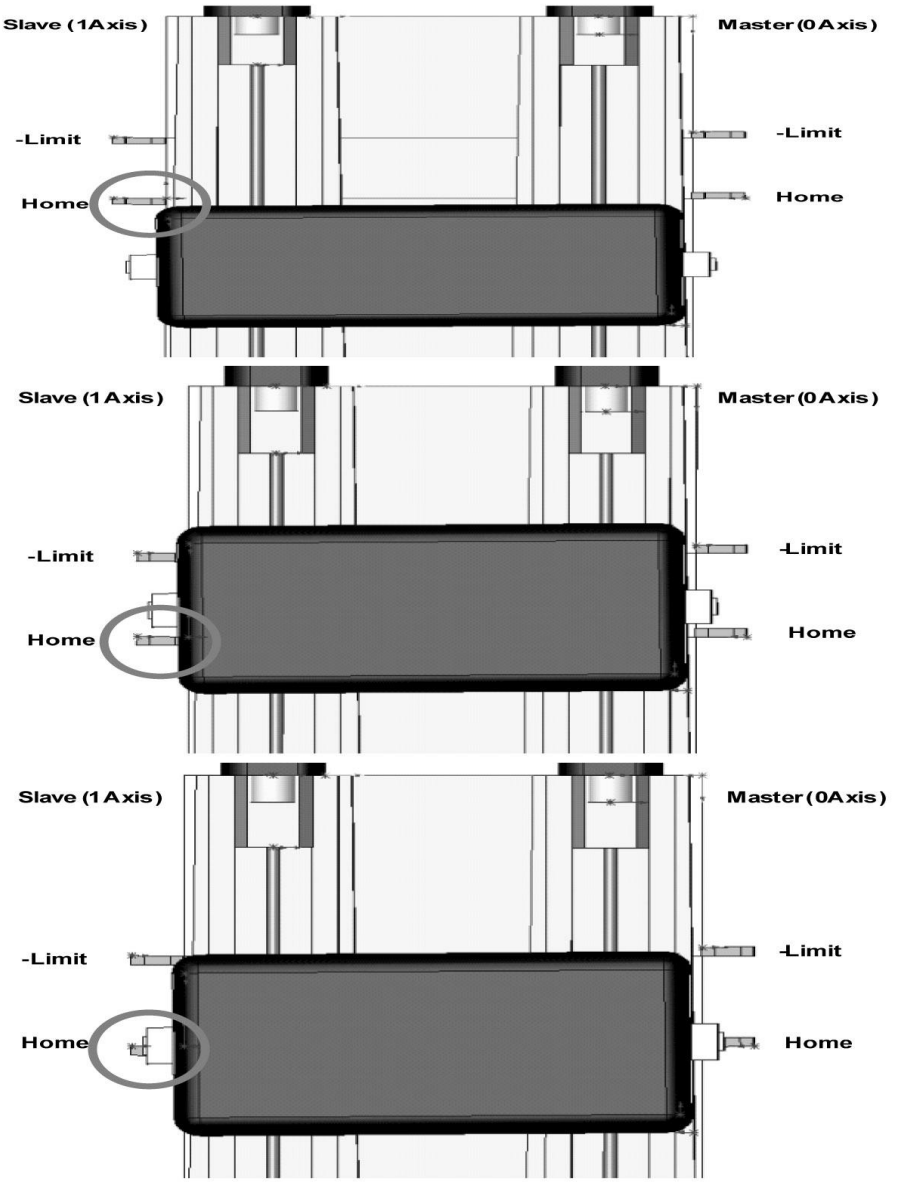
1. Search home sensor on Master axis first. Make the position of 0 only for Master axis.



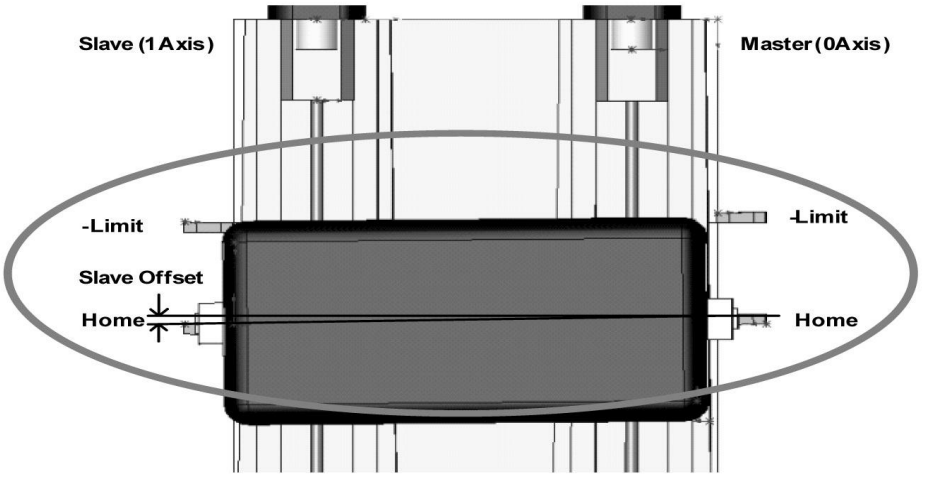
2. Search home sensor on Slave axis. Make the position of 0 only for slave axis. Then can obtain offset value of slave for master.




Slave (1Axis)
Master(0Axis)
-Limit
-Limit
Home
Home
Slave (1Axis)
Master(0Axis)
-Limit
-Limit
Home
Home

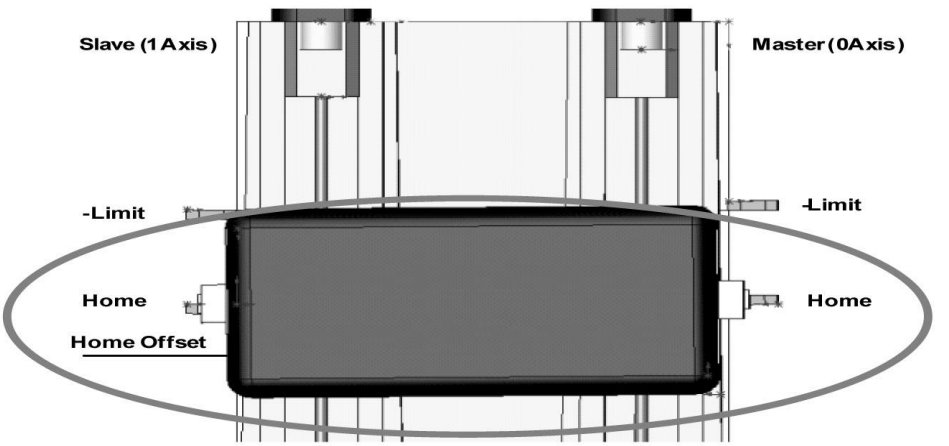


3. Release link between Master and Slave, and move Slave axis as much as Slave offset. Then send Master axis to 0. And Link again.

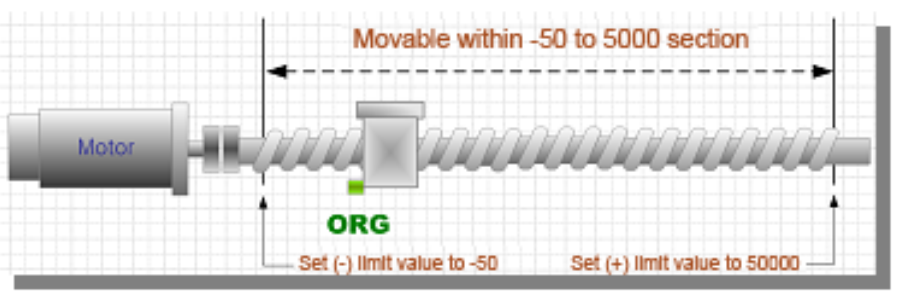


4. Move as much as home Offset, and make the position to be 0.

If there is no home Offset, do not move, and make the position to 0.



5. Carry out Software Limit setting and Gantry drive.



※ Caution: Soft Limit can be set after completion of home search regularly. Soft Limit is used if Limit sensor

is not installed mechanically or Limit needs to be changed by condition. Here, be careful that Soft Limit is set regularly when PLimit(+) value is bigger than NLimit(-). If NSwLimit value is equal to or bigger than PSwLimit, Soft Limit will not be set.

```
// Ex19_AXM_GentryDrive.cpp : Defines the entry point for the console application.
// Carry out Gantry drive.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_1 1

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           for(int nAxisNo = 0; nAxisNo<2; nAxisNo++)
           {
              // Set Active level of +End limit and -End limit on axis 0 to HIGH.
              AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
              // Set input level of inposition signal on axis 0 to HIGH.
              AxmSignalSetInpos(nAxisNo, HIGH);
              // Set input level of alarm signal on axis 0 to LOW.
              AxmSignalSetServoAlarm(nAxisNo, LOW);
              // Set Active input level of ESTOP signal on axis 0 to HIGH.
              AxmSignalSetStop(nAxisNo, 0, HIGH);

              // Make ordered value on axis 0 to be mm unit.
              AxmMotSetMoveUnitPerPulse(nAxisNo, 10, 10000);
              // Set initial velocity on axis 0 to 1.  Default : 1
              AxmMotSetMinVel(nAxisNo, 1);
              // Make pulse output method on axis 0 to TwoCwCcwHigh.
```

```
    AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
    // Set encoder input method on designated axis to 4 multiplication.
    AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);

    // Set home search method.
    AxmHomeSetMethod(nAxisNo,-1,HomeSensor,1,2000.0,0.0);
    // Set velocity that will be used during home search.
    AxmHomeSetVel(nAxisNo, 100, 20, 10, 10, 200, 40);

    // Servo On.
    AxmSignalServoOn(nAxisNo, ENABLE);
}

// Set Gantry drive.
long lMasterAxisNo = AXIS_0;
long lSlaveAxisNo = AXIS_1;
BOOL bSlHomeUse = FALSE;
// TRUE : Search slave home sensor after master home sensor is searched.
double dSlOffset = 0;     // Offset between master and slave home sensors.
double dSlOffsetRange = 10;
 // Error limit between master and slave home sensor during home search.

AxmGantrySetEnable(lMasterAxisNo, lSlaveAxisNo, bSlHomeUse, dSlOffset,
    dSlOffsetRange);

// help Message.
printf("[INFORMATION]********************************** \n");
printf("Gantry drive is set.\nStart drive pressing any key.\n");
printf("********************************************** \n");
// Wait until pressed any key.
getch();

printf("\nCarry out home search of Gantry drive.\n");
// Set home search method.
AxmHomeSetMethod(lMasterAxisNo,-1,HomeSensor,1,2000.0,0.0);
// Set velocity that will be used during home search.
AxmHomeSetVel(lMasterAxisNo, 100, 20, 10, 10, 200, 40);

//Carry out home search.
AxmHomeSetStart(lMasterAxisNo);

// Verify current home search progress rate.
DWORD uHomeMainStepNumber , uHomeStepNumber_0;
AxmHomeGetRate(lMasterAxisNo, &uHomeMainStepNumber , &uHomeStepNumber_0);
while(uHomeStepNumber_0 < 100)
{
    AxmHomeGetRate(lMasterAxisNo, &uHomeMainStepNumber , &uHomeStepNumber_0);
    printf("\r In home search... %d ", uHomeStepNumber_0);
}
```

```
            printf("\rHome search is completed \n");

            printf("Drive Master axis as much as 100.\n");
            AxmMoveStartPos(lMasterAxisNo, 100, 100, 200, 200);
            //Verify if in drive.
            DWORD uStatus;
            AxmStatusReadInMotion(lMasterAxisNo, &uStatus);
            while(uStatus) {
               AxmStatusReadInMotion(lMasterAxisNo, &uStatus);
            }

            //Exit Gantry drive.
            AxmGantrySetDisable(lMasterAxisNo,lSlaveAxisNo);

            // Servo Off.
            AxmSignalServoOn(nAxisNo, DISABLE);
            AxmSignalServoOn(nAxisNo, DISABLE);
         }
         else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
               Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("Library is exited.\n");
   else
      printf("Library is not exited normally.\n");
}
```

Virtual Axis Mapping

Virtual axis mapping is an API that gives command using arbitrary axis number, not an actual corresponding axis number, when drive command is given from library. For example, when give a command on number axis 0, give a command as number 2 axis during command. Let us suppose there are axes from number 0 to 10 in the system, and number 3 and 4 axis in the middle are removed somehow. In this case, axis number will be run moved up from number 0 to 8 in sequence. Therefore, it is occurred that axis numbers on the program have to be changed one by one, and in this case, it can be solved by mapping with original axis number without toughing program source. In this case, number 3 to 8 axes just need to change to number 5 to 10 axes.

To use this virtual axis mapping, use AxmVirtualSetAxisNoMap API mapping one axis to virtual axis or AxmVirtualSetMultiAxisNoMap API mapping various axes to virtual axes. One thing to be careful is that command on previous axis number does not operate regularly because previous actual axis number does not exist after mapping to virtual axis, and newly setting axis number shall be used after setting.

```
// Carry out mapping axis 0 to number 2 axis.
long lRealAxisNo = 0;
long lVirtualAxisNo = 2;
AxmVirtualSetAxisNoMap(lRealAxisNo, lVirtualAxisNo);
```

```
// Carry out mapping 0 and 1 axis to number 2 and 3.
long lSize = 2;
long lpRealAxesNo[2] = {0,1};
long lpVirtualAxesNo[2] = {2,3};
AxmVirtualSetMultiAxisNoMap(lSize, lpRealAxesNo, lpVirtualAxesNo);
```

To return mapped axis number to original, use AxmVirtualResetAxisMap API.

```
// Clear virtual axis mapping.
AxmVirtualResetAxisMap();
```

```
// Ex20_AXM_AxisNoMapping.cpp : Defines the entry point for the console application.
// Carry out virtual axis mapping.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_2 2

void main(void)
{
  // Initialize library.
```

```
// 7 means IRQ. IRQ is set automatically in PCI.
DWORD Code = AxlOpen(7);
if (Code == AXT_RT_SUCCESS)
{
   printf("Library is initialized.\n");

   // Inspect whether motion module exists or not.
   DWORD dwStatus;
   Code = AxmInfoIsMotionModule(&dwStatus);
   if(Code == AXT_RT_SUCCESS)
   {
      if(dwStatus == STATUS_EXIST)
      {
         printf("Motion module exists.\n");

         // Set Active level of +End limit and -End limit on axis 0 to HIGH.
         AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
         // Set input level of inposition signal on axis 0 to HIGH.
         AxmSignalSetInpos(AXIS_0, HIGH);
         // Set input level of alarm signal on axis 0 to LOW.
         AxmSignalSetServoAlarm(AXIS_0, LOW);
         // Set Active input level of ESTOP signal on axis 0 to HIGH.
         AxmSignalSetStop(AXIS_0, 0, HIGH);

         // Make ordered value on axis 0 to be mm unit.
         AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
         // Set initial velocity on axis 0 to 1.  Default : 1
         AxmMotSetMinVel(AXIS_0, 1);
         // Make pulse output method on axis 0 to TwoCwCcwHigh.
         AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
         // Set encoder input method on designated axis to 4 multiplication.
         AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

         // Use relative S-curve velocity profile.
         DWORD uAbsRelMode = 1;
         DWORD uProfileMode = 3;
         AxmMotSetAbsRelMode(AXIS_0, uAbsRelMode);    // Set to relative position drive.
         AxmMotSetProfileMode(AXIS_0, uProfileMode); // S-curve velocity.

         // Servo On.
         AxmSignalServoOn(AXIS_0, ENABLE);
         // Carry out mapping axis 0 to number 2 axis.
         long lRealAxisNo = AXIS_0;
         long lVirtualAxisNo = AXIS_2;
         AxmVirtualSetAxisNoMap(lRealAxisNo, lVirtualAxisNo);

         // help Message.
         printf("[INFORMATION]********************************** \n");
         printf("Number axis 0 is set to number 2.\nNumber 2 axis drive starts pressing
```

```
            any key.\n");
         printf("********************************************** \n");
         // Wait until pressed any key.
         getch();

         // Drive axis 0 as much as 100.
         AxmMoveStartPos(AXIS_2, 100, 100, 200, 200);
         // Verify if in drive.
         DWORD uStatus;
         AxmStatusReadInMotion(AXIS_2, &uStatus);
         while(uStatus) {
            AxmStatusReadInMotion(AXIS_2, &uStatus);
         }

         // Clear virtual axis mapping.
         AxmVirtualResetAxisMap();

         // Servo Off.
         AxmSignalServoOn(AXIS_0, DISABLE);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
            Code);
   }
   else
      printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE ) code 0x%x\n",Code);
}
else
   printf ("AxlOpen() : ERROR code 0x%x\n",Code);

// Exit library.
if (AxlClose())
   printf("Library is exited.\n");
else
   printf("Library is not exited normally.\n");
}
```

Interrupt Setting

Interrupt can be generated using various events as interrupt source generating in motion drive in AXM, if register of interrupt BAMK composed of 31 bits per axis generates each interrupt, it will be 1. Main interrupt sources are following.

Note) Interrupt setting feature is only use for PCI-N804/404.

※ *Mainly used interrupt source Status*

Define	Explanation
QIINTBANK1_1	When drive exits
QIINTBANK1_2	When drive starts
QIINTBANK1_4	counter #1 = comparator #1 event occurance
QIINTBANK2_31	If emergency stop signal is inputted
QIINTBANK2_30	If NSLM / PSLM signal is activated
QIINTBANK2_29	If NELM / PELM signal is activated
QIINTBANK1_29	If alarm signal is activated
QIINTBANK2_9	Drive and exit condition occurance using inposition function
IPINTBANK2_SSTOP_SIGNAL	If SSTOP signal is inputted
IPINTBANK2_ESTOP_SIGNAL	If ESTOP signal is inputted
IPINTBANK2_L_C_INP_Q_LESS_4	If straight line / circular interpolation data queue is to be less than 4

To use interrupt, first set how to manage interrupt on corresponding axis using AxmInterruptSetAxis API, and set use of interrupt to ENABLE using AxmInterruptSetAxisEnable API.

1) Interrupt Management Method Setting

When an event occurs, to manage this, AXM provides 3 methods; callback API, window message, and event method. Callback API method has an advantage to be reported event first of all by calling callback API immediately at the point of event occurrence, but main process is tied up until callback API exits completely. In short, if there is a work that loads in callback API, need to be careful to use. However, since event method is operated separately with main process using a specific thread watching event occurrence, this method is especially recommended because source can be used most efficiently including MultiProcessor system. To set event management method, use AxmInterruptSetAxis API and set as following.

● *Callback API method*

To use callback API method, set as following.

```
// Set event management method on number axis 0 to callback API method.
AxmInterruptSetAxis(0, NULL, NULL, OnMotionInterruptCallback, NULL);
```

OnMotionInterruptCallback is a pointer of callback API to manage interrupt. Callback API is an all area API and it is set as following. When interrupt is occurred, it calls callback API to manage interrupt automatically.

```
// Callback API to manage interrupt.
void __stdcall OnMotionInterruptCallback(long lAxisNo, DWORD dwFlag)
{
  // lAxislNo : Axis number that interrupt is occurred.
  // dwFlag : Flag that interrupt is occurred.
  printf("[INTERRUPT MESSAGE from CALLBACK method] lAxisNo: %d, dwFlag: %d \n",
     lActiveNo,bwFlag);
}
```

● *Window Message method*

To use window message method, set as following. Since window message method cannot use in C, for this part, use VC++ to describe. To use window message method, input window handle to receive a message and window message. If it does not need to use window message or needs to use default value, input 0.

```
// Set event management method on number axis 0 to window event method.
AxmInterruptSetAxis(0, this->m_hWnd, WM_MOTION_INTERRUPT, NULL, NULL);
```

Instruction shows below, OnMotionInterruptMessage API manage interrupt.

```
#define WM_MOTION_INTERRUPT 1011      // Message definition to receive interrupt message.
#define AXIS_ 0 0                     // Drive axis number.

…(Skip)…

// Connect message to API
BEGIN_MESSAGE_MAP(CExMotionInterruptDlg, CDialog)
  //{{AFX_MSG_MAP(CExMotionInterruptDlg)

ON_MESSAGE(WM_MOTION_INTERRUPT,OnMotionInterruptMessage)

  //}}AFX_MSG_MAP
END_MESSAGE_MAP()

…(Skip)…

void CExMotionInterruptDlg::OnBtnStartInterruptMessage()
{
  CString Message;

  // Set the using interrupt.
  AxlInterruptEnable();
  // Set interrupt management method.
  AxmInterruptSetAxis (AXIS_ 0, this->m_hWnd, WM_MOTION_INTERRUPT, NULL, NULL);
```

```
  // Set if interrupt permitted for specific channel.
  AxmInterruptSetAxisEnable (AXIS_ 0,ENABLE);
}

void CExMotionInterruptDlg::OnMotionInterruptMessage(long lAxislNo, DWORD dwFlag)
{
  // lAxislNo : Axis number that interrupt is occurred.
  // dwFlag : Flag that interrupt is occurred.
  printf("[INTERRUPT MESSAGE from CALLBACK method] lAxisNo : %d, bwFlag : %d \n", lAxislNo,
     dwFlag);
}
```

● *Event method*

To use event method, set as following. Since event method is operated separately with main process using a specific thread watching event occurrence, this method is especially recommended because source can be used most efficiently including MultiProcessor system.

This part also use VC++ to describe. To use event method, HANDLE type member variable m_hInterruptEvent to receive event ID is declared in advance and used.

```
// Set event management method on number 0 module to window event method.
AxmInterruptSetAxis (0,NULL,NULL,NULL,&m_hInterruptEvent);
```

Instruction shows bellow, and ThreadProc API has to manage interrupt. One thing to be careful is AxmInterruptSetAxis API must be called before thread operation.

If interrupt is occurred, interrupt event will be occurred and thread will detect this event and know that interrupt is occurred. And it verifies on which axis and of which interrupt is occurred verifying axis number that interrupt is occurred and interrupt flag information with calling AxmInterruptRead API inside thread.

```
// Header File : Required variable to use event method interrupt.
HANDLE m_hThreadHandle;
BOOL m_bThread;
HANDLE m_hInterruptEvent;
```

```
// Source File :
void CExMOTIONInterruptDlg::OnBtnStartInterruptEvent()
{
  CString Message;

  AxlInterruptEnable();

  // Set interrupt management method.
  AxmInterruptSetAxis(0,NULL,NULL,NULL,&m_hInterruptEvent);  // Event method.

  // To use event method, initialize after declaration of HANDLE pEvent member variable.
  // Make thread.
```

```
   m_bThread = TRUE;
   CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)ThreadProc, this, NULL, NULL);

   // Set interrupt permission on axis 0.
   AxmInterruptSetAxisEnable(CHANNEL_NO,ENABLE);
}

void ThreadProc(LPVOID lpData)
{
   CExMOTIONInterruptDlg *pDlg  = (CExMOTIONInterruptDlg *)lpData;
   while(pDlg->m_bThread)
   {
      if (WaitForSingleObject(pDlg->m_hInterruptEvent, INFINITE) == WAIT_OBJECT_0)
      {
         long lAxisNo;     // Axis number that interrupt is occurred.
         DWORD dwFlag;     // Flag that interrupt is occurred.
         AxmInterruptRead(lAxisNo, dwFlag);

         printf("[INTERRUPT MESSAGE from CALLBACK method] lAxisNo : %d, bwFlag : %d \n",
            lAxisNo, dwFlag);
      }
   }
}
```

2) Interrupt Use Setting

Use of interrupt is set to DISABLE as default. Therefore, interrupt will occur by setting on interrupt mask after setting of ENABLE using AxmInterruptSetAxisEnable API.

Note) AxlInterruptEnable API must be set to enable.

```
// Set interrupt to ENABLE on number axis 0.
long lAxisNo = 0;
DWORD uUse = ENABLE;
AxmInterruptSetAxisEnable(lAxisNo, uUse);
```

```
// Ex21_AXM_Interrupt.cpp : Defines the entry point for the console application.
// Generate interrupt and verify current state.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

// Callback API to manage interrupt.
```

```
void __stdcall OnMotionInterruptCallback(long lAxisNo, DWORD dwFlag)
{
   // lAxislNo : Axis number that interrupt is occurred.
   // dwFlag : Flag that interrupt is occurred.
   printf("[INTERRUPT MESSAGE from CALLBACK method] lAxisNo : %d, bwFlag : %dx \n",
      lAxisNo,dwFlag);
}

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library is initialized.\n");

      // Inspect whether motion module exists or not.
      DWORD dwStatus;
      Code = AxmInfoIsMotionModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            printf("Motion module exists.\n");

            // Set Active level of +End limit and -End limit on axis 0 to HIGH.
            AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
            // Set input level of inposition signal on axis 0 to HIGH.
            AxmSignalSetInpos(AXIS_0, HIGH);
            // Set input level of alarm signal on axis 0 to LOW.
            AxmSignalSetServoAlarm(AXIS_0, LOW);
            // Set Active input level of ESTOP signal on axis 0 to HIGH.
            AxmSignalSetStop(AXIS_0, 0, HIGH);

            // Make ordered value on axis 0 to be mm unit.
            AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
            // Set initial velocity on axis 0 to 1.  Default : 1
            AxmMotSetMinVel(AXIS_0, 1);
            // Make pulse output method on axis 0 to TwoCwCcwHigh.
            AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
            // Set encoder input method on designated axis to 4 multiplication.
            AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);

            // Use relative position S-curve velocity profile.
            DWORD uAbsRelMode = 1;
            DWORD uProfileMode = 3;
            AxmMotSetAbsRelMode(AXIS_0, uAbsRelMode);
            AxmMotSetProfileMode(AXIS_0, uProfileMode);
```

```
            // Servo On.
            AxmSignalServoOn(AXIS_0, ENABLE);

            // Set the using interrupt.
            AxlInterruptEnable();

            long lBank = 1; // For PCI-N804/404
            // Generate interrupt if motion exits.
            AxmInterruptSetUserEnable(AXIS_0, lBank, QIINTBANK2_9);

            // Set interrupt to ENABLE on number axis 0.
            AxmInterruptSetAxisEnable(AXIS_0, ENABLE);

            // Set event management method to callback API method on axis 0.
            AxmInterruptSetAxis(AXIS_0, NULL, NULL, OnMotionInterruptCallback,NULL);

            // help Message.
            printf("[INFORMATION]************************************** \n");
            printf("Generate interrupt starting drive if pressing any key.\n");
            printf("********************************************************* \n");

            // Wait until any key is pressed.
            getch();

            // Drive axis 0 as much as 100.
            AxmMoveStartPos(AXIS_0, 100, 100, 200, 200);
            // Verify if in drive.
            DWORD uStatus;
            AxmStatusReadInMotion(AXIS_0, &uStatus);
            while(uStatus) {
               AxmStatusReadInMotion(AXIS_0, &uStatus);
            }
            getch();

            // Servo Off.
            AxmSignalServoOn(AXIS_0, DISABLE);
         }
         else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",
               Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE ) code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
```

```
    if (AxlClose())
        printf("Library is exited.\n");
    else
        printf("Library is not exited normally.\n");
}
```

Use Of Trigger Signal

AXM may generate interrupt as well as 5V level of Trigger signal based on specific condition during in drive. Trigger signal comes out from TRIG signal pin, and if it is set to ActiveHIGH, it maintains 0V normally and if a signal is occurred, it changed to 5V for the set time then comes back to 0V, and vice versa for ActiveLow. Mechanical part, of course, is to be 24V. To generate trigger signal, first, set trigger signal level and duration using AxmTriggerSetTimeLevel API, and call APIs generating trigger. There are three trigger generation APIs: AxmTriggerSetAbsPeriod / AxmTriggerOnlyAbs to generate trigger on a specific condition, AxmTriggerSetBlock to generate trigger with constant interval for a specific section, and AxmTriggerOneShot / AxmTriggerSetTimerOneshot to generate trigger at once on an arbitrary point.

Note) Trigger is only available on PCI-N804/404 and RTEX-PM.

● *AxmTriggerSetTimeLevel*

When generate trigger signal, this API is of trigger output signal time setting, use of trigger output signal and Active Level, trigger datum point value input source, and interrupt output during trigger output. Trigger output signal time can be set to minimum (1 = 1usec) to maximum (40000 = 4msec), and use of trigger output signal and Active Level can be set to LOW(0), HIGH(1), UNUSED(2), and USED(3). Trigger datum position value input source is available to use Command Position(0) and Actual Position(1), and interrupt output can be set to DISABLE(0) or ENABLE(1) during trigger output.

```
// Set trigger signal level and duration on axis 0.
long lAxisNo = 0;
double dTrigTime = 100;          // 0.1ms
DWORD uTriggerLevel = HIGH;
DWORD uSelect = 0;               // Trigger occurrence in the basis of Command Position
DWORD uInterrupt = DISABLE;

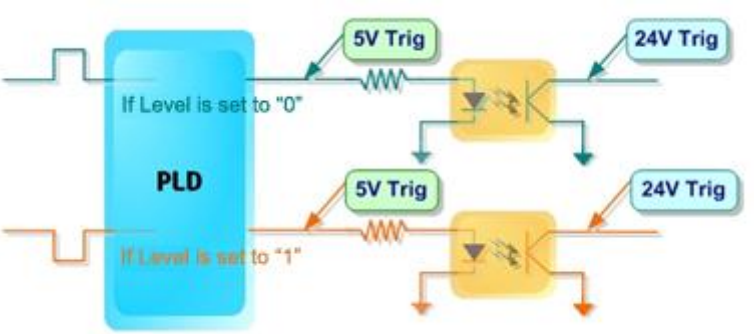
AxmTriggerSetTimeLevel(lAxisNo, dTrigTime, uTriggerLevel, uSelect, uInterrupt);
```

Low(B contact) : normally Close -> if detected, Open

High(A contact): normally Open -> if detected, Close

Falling Edge : maintain High state normally, fall to Low at trigger point and rise again to High.

Rising Edge : maintain Low state normally, rise to High at trigger point and fall again to Low.



※ Notice : In case of 5V trigger output, if Active Level is set to 0, it will be 5V when trigger is generated at 0V normally, and if case of 24V trigger, if trigger is outputted in Open state, it will be N24V. therefore, if trigger is used with 24V through terminal, it needs to be measured with 2kΩ of pull-up resistance to verify trigger output.

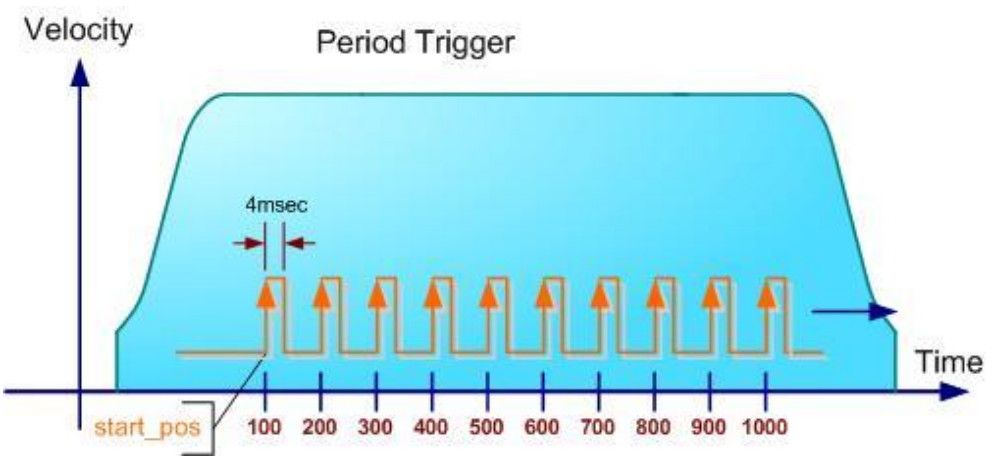
● *AxmTriggerSetAbsPeriod*

To generate trigger at a specific position, AxmTriggerSetAbsPeriod API can be used. This API generates trigger receiving trigger output method and position, and if uMethod value is to be 0, it sets trigger position value to 1000 with periodic trigger method using input trigger position value and when it moves, trigger is generated at every 1000 position (1000,2000,3000,..). And if uMethod value is to be 1, trigger is generated only at 1000 position with absolute trigger generation method and trigger is not generated at 2000, 3000, and so on.

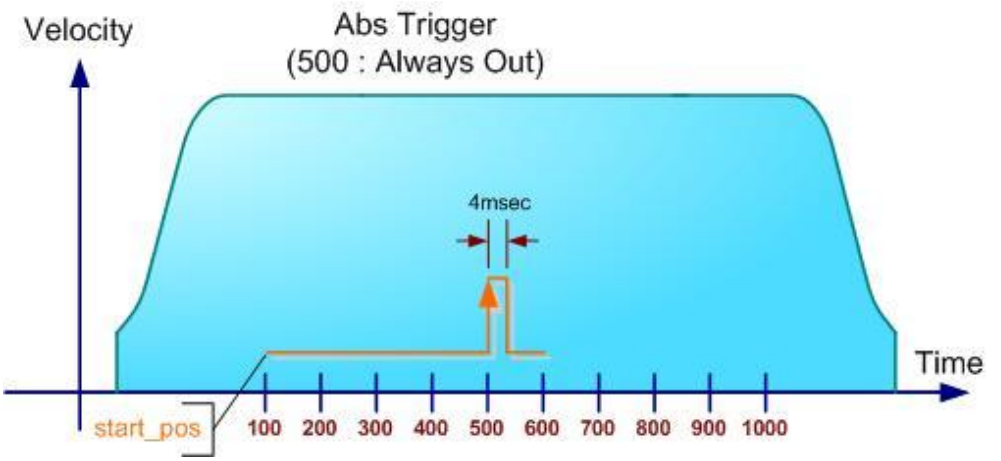
```
// Set to generate trigger at every 1000 position during moving on axis 0.
long lAxisNo = 0;
DWORD uMethod = 0; // [0] : Periodic trigger method, [1] : Absolute position trigger method
double dPosition = 1000;
AxmTriggerSetAbsPeriod(lAxisNo, uMethod, dPosition);
```

Note : When trigger mode is set to periodic mode, trigger at the time of setting is outputted. From the position started differently with AxmTriggerSetBlock API, trigger is outputted on the basis of counter change volume.

Period mode : If Position is 100, trigger output is generated at every 100 pulse.



Absolute mode : If Position is 500, trigger output is generated at 500 position unconditionally.



● *AxmTriggerOnlyAbs*

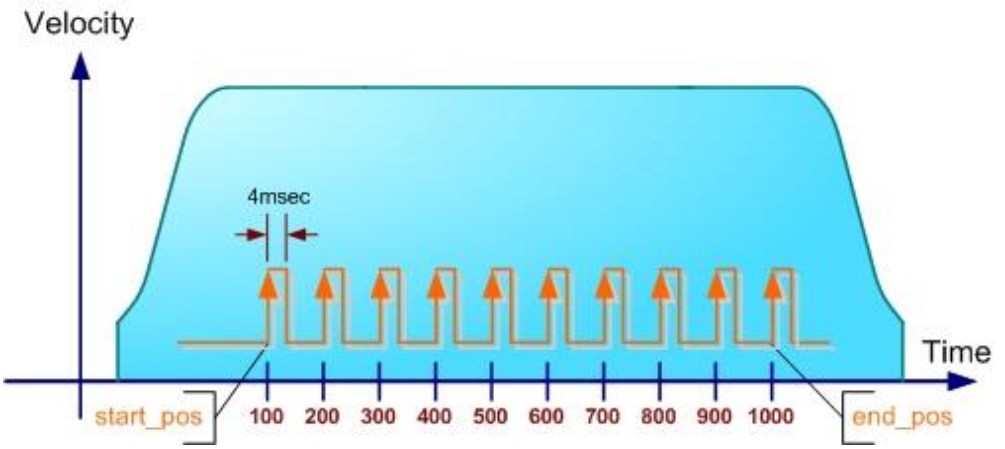
If the position to generate trigger is irregular and a lot, API has to be called repeatedly as much as trigger occurrence position with use of API previously mentioned. But this inconvenience can be reduced using AxmTriggerOnlyAbs API. This API generates trigger on the corresponding position receiving the position to generate trigger with array.

```
// Generate trigger at the assigned 10 positions during moving on axis 0.
long lAxisNo = 0;
long lTrigNum = 10;
double dTrigPos = {100, 150, 200, 250, 400, 520, 650, 700, 800, 980};
AxmTriggerOnlyAbs(lAxisNo, lTrigNum, dTrigPos);
```

● *AxmTriggerSetBlock*

This API sets start point to generate trigger and end point, and generates trigger once on a certain distance within the section.

```
// Generate trigger whenever moved as much as 100 from 300 position to 1000 position
// on axis 0.
long lAxisNo = 0;
double dStartPosition = 100;
double dEndPosition = 1000;
double dPeriodPosition = 100;
AxmTriggerSetBlock(lAxisNo, dStartPosition, dEndPosition, dPeriodPosition);
```

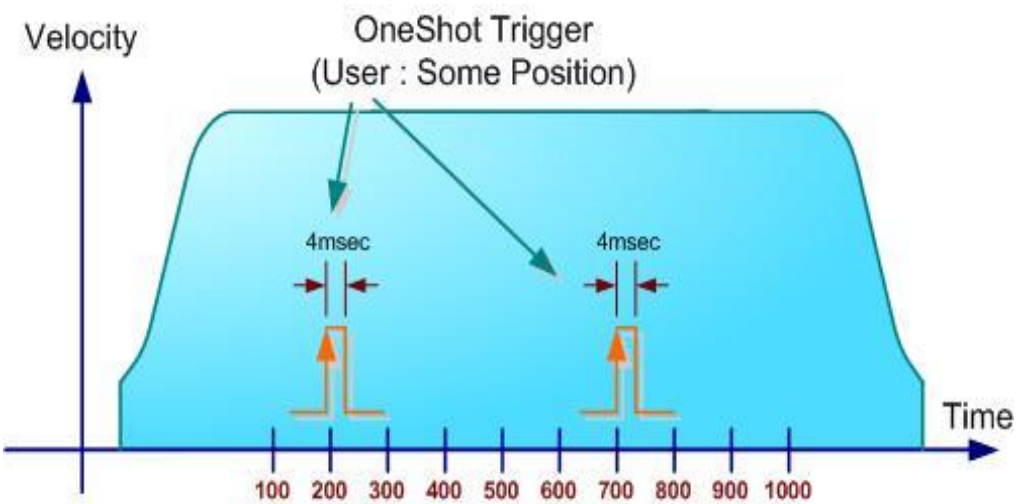


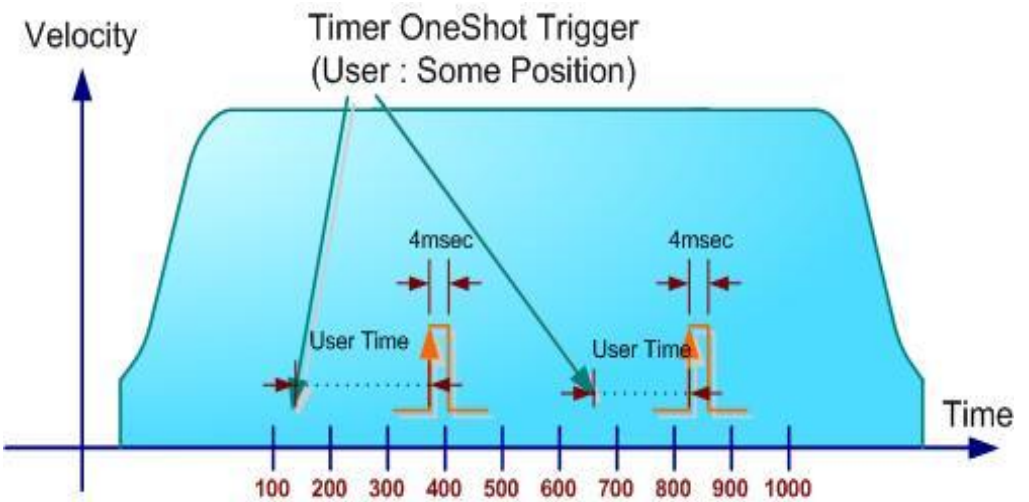
● *AxmTriggerOneShot* / *AxmTriggerSetTimerOneshot*

This API is that user outputs one trigger signal at an arbitrary point. AxmTriggerOneShot API generates trigger as soon as it is called, and AxmTriggerSetTimerOneshot API generates trigger going by a certain time after API call by input time.

```
// Generate trigger at the current point.
long lAxisNo = 0;
AxmTriggerOneShot(lAxisNo);

// Generate trigger after 1000ms(1sec).
long lAxisNo = 0;
long lmSec = 1000;
AxmTriggerSetTimerOneshot(lAxisNo, lmSec);
```






```
// Ex22_AXM_Trigger.cpp : Defines the entry point for the console application.
// Generate trigger signal with driving.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library is initialized.\n");
      // Inspect whether motion module exists or not.
      DWORD dwStatus;
      Code = AxmInfoIsMotionModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            printf("Motion module exists.\n");
            // Set Active level of +End limit and -End limit on axis 0 to HIGH.
            AxmSignalSetLimit(AXIS_0, 0, HIGH, HIGH);
            // Set input level of inposition signal on axis 0 to HIGH.
            AxmSignalSetInpos(AXIS_0, HIGH);
```

```
// Set input level of alarm signal on axis 0 to LOW.
AxmSignalSetServoAlarm(AXIS_0, LOW);
// Set Active input level of ESTOP signal on axis 0 to HIGH.
AxmSignalSetStop(AXIS_0, 0, HIGH);
// Make ordered value on axis 0 to be mm unit.
AxmMotSetMoveUnitPerPulse(AXIS_0, 10, 10000);
// Set initial velocity on axis 0 to 1.  Default : 1
AxmMotSetMinVel(AXIS_0, 1);
// Make pulse output method on axis 0 to TwoCwCcwHigh.
AxmMotSetPulseOutMethod(AXIS_0, TwoCcwCwHigh);
// Set encoder input method on designated axis to 4 multiplication.
AxmMotSetEncInputMethod(AXIS_0, ObverseSqr4Mode);
// Set to absolute position drive.
AxmMotSetAbsRelMode(AXIS_0, 0);
// Set to symmetric S-curve velocity file.
AxmMotSetProfileMode(AXIS_0, 4);
// Set home search method.
AxmHomeSetMethod(AXIS_0,-1,HomeSensor,1,2000.0,0.0);
// Set velocity that will be used during home search.
AxmHomeSetVel(AXIS_0, 100, 20, 10, 10, 200, 40);
// Servo On.
AxmSignalServoOn(AXIS_0, ENABLE);
// Search home.
AxmHomeSetStart(AXIS_0);
// Verify current home search progress rate.
DWORD uHomeMainStepNumber , uHomeStepNumber_0;
AxmHomeGetRate(AXIS_0, &uHomeMainStepNumber, &uHomeStepNumber_0);
while(uHomeStepNumber_0 < 100)
{
   AxmHomeGetRate(AXIS_0, &uHomeMainStepNumber, &uHomeStepNumber_0);
   printf("\rIn home search : %d% ", uHomeStepNumber_0);
}

// help Message.
printf("\n[INFORMATION]************************************************ \n");
printf("[ESC] : Exit \n");
printf("[1] : Generate trigger at every 50 going to 200 with (+) direction \n");
printf("[2] : Generate trigger at assigned 10 positions going to 200 with (+)
   direction \n");
printf("[3] : Generate trigger from 50 to 150 position with 10 interval going to
   200 with (+) direction \n");
printf("************************************************************** \n");

// Set trigger signal level and duration on axis 0.
double dTrigTime = 4000;           // 4ms
DWORD uTriggerLevel = HIGH;
DWORD uSelect = 0;         // Trigger generation on the basis of Command Position
DWORD uInterrupt = DISABLE;
AxmTriggerSetTimeLevel (AXIS_0, dTrigTime, uTriggerLevel, uSelect, uInterrupt);
```

```
DWORD uStatus;
BOOL fExit = FALSE;
while (!fExit)                          // Infinite loop.
{
   if (kbhit())                         // If pressing any key.
   {
      int ch = getch();
      DWORD uMethod;
      double dPosition;
      long lTrigNum = 10;
      double dTrigPos[10];
      double dStartPosition, dEndPosition, dPeriodPosition;

      switch (ch)
      {
         case 27:                       // Esc key.
            fExit = TRUE;
            AxmSignalServoOn(AXIS_0, DISABLE);
         break;

         case '1':
            // Generate trigger at every 50 position during movement.
            uMethod = 0;
            // [0] : Periodic trigger method,
            // [1] : Absolute position trigger method

            dPosition = 50;
            AxmTriggerSetAbsPeriod(AXIS_0, uMethod, dPosition);

            // Drive up to 200 position.
            AxmMoveStartPos(AXIS_0, 200, 50, 100, 100);
            AxmStatusReadInMotion(AXIS_0, &uStatus);
            while(uStatus) {
               AxmStatusReadInMotion(AXIS_0, &uStatus);
            }

            // Return again to home.
            AxmMoveStartPos(AXIS_0, 0, 100, 200, 200);
            AxmStatusReadInMotion(AXIS_0, &uStatus);
            while(uStatus) {
               AxmStatusReadInMotion(AXIS_0, &uStatus);
            }
         break;

         case '2':
            // Generate trigger at assigned 10 positions during movement.
            lTrigNum = 10;
            dTrigPos[0] = 10;  dTrigPos[1] = 15;  dTrigPos[2] = 20;
```

```
                    dTrigPos[3] = 50; dTrigPos[4] = 85;  dTrigPos[5] = 100;
                    dTrigPos[6] = 120; dTrigPos[7] = 150;
                    dTrigPos[8] = 172; dTrigPos[9] = 190;
                    AxmTriggerOnlyAbs(AXIS_0,lTrigNum, dTrigPos);

                    // Drive up to 200 position.
                    AxmMoveStartPos(AXIS_0, 200, 50, 100, 100);
                    AxmStatusReadInMotion(AXIS_0, &uStatus);
                    while(uStatus) {
                       AxmStatusReadInMotion(AXIS_0, &uStatus);
                    }

                    // Return again to home.
                    AxmMoveStartPos(AXIS_0, 0, 100, 200, 200);
                    AxmStatusReadInMotion(AXIS_0, &uStatus);
                    while(uStatus) {
                       AxmStatusReadInMotion(AXIS_0, &uStatus);
                    }
                 break;

                 case '3':
                    // Generate trigger from 50 to 150 position whenever moves every 10
                    // on axis 0.
                    dStartPosition = 50;
                    dEndPosition = 150;
                    dPeriodPosition = 10;
                    AxmTriggerSetBlock(AXIS_0, dStartPosition, dEndPosition,
                       dPeriodPosition);

                    // Drive up to 200 position.
                    AxmMoveStartPos(AXIS_0, 200, 50, 100, 100);
                    AxmStatusReadInMotion(AXIS_0, &uStatus);
                    while(uStatus) {
                       AxmStatusReadInMotion(AXIS_0, &uStatus);
                    }

                    // Return again to home.
                    AxmMoveStartPos(AXIS_0, 0, 100, 200, 200);
                    AxmStatusReadInMotion(AXIS_0, &uStatus);
                    while(uStatus) {
                       AxmStatusReadInMotion(AXIS_0, &uStatus);
                    }
                 break;
              }
           }
        }  // End of While()
     }
     else
        printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",
```

```
                Code);
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

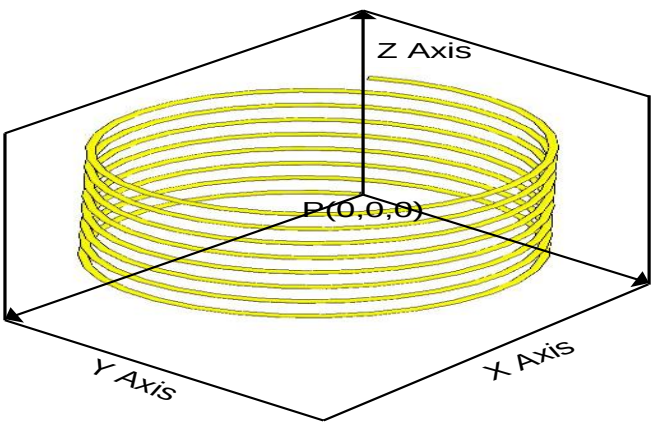
   // Exit library.
   if (AxlClose())
      printf("Library is exited.\n");
   else
      printf("Library is not exited normally.\n");
}
```

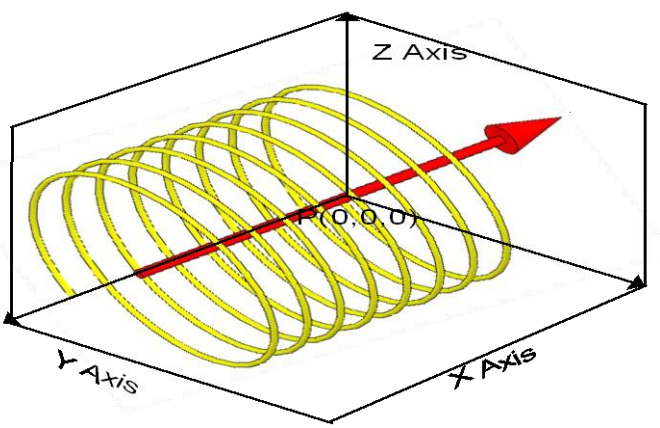
Advanced Motion Drive

So far, various APIs for general motion control are discussed. In this chapter, several functions that are must required for APIs for advanced motion drive only on CAMC-QI chip will be discussed.

Helical Interpolation Motion

Helical interpolation control is a function of driving with complex 4 axes of 2 axes circular interpolation and straight line interpolation. With this, 2 axes straight line interpolation and circular interpolation can be carried out with synchronization. It provides two APIs; API that can drive one at once, and API used together with continuous interpolation function. To drive helical interpolation, circular interpolation movement must be move with synchronization velocity and moving amount with U axis (number 4 axis). Always last axis is set to U axis, and since U axis moves with synchronization with circular interpolation, moving amount follows Z axis and velocity cannot be used differently from circular interpolation. In case of 4 axis motion board, X(0),Y(1),Z(2),U(3) are used, and for 8 axis motion board, it can be used setting X(0),Y(1),Z(2),U(3) and X2(4),Y2(5),Z2(6),U(7).

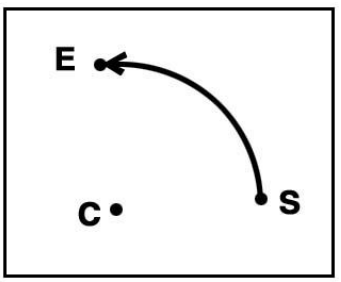




● *Helical Interpolation Drive*

Circular interpolation drive is to be interpolation drive with circular path based on the corresponding setting from the current position to the assigned position. And this circular interpolation drive APIs provide AxmHelixCenterMove, AxmHelixRadiusMove, AxmHelixAngleMove, and AxmHelixPointMove API.

- AxmHelixCenterMove

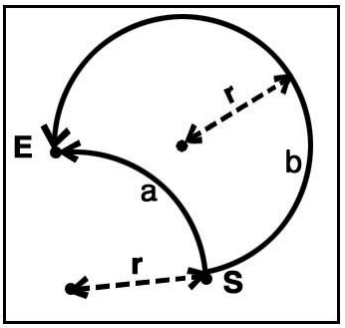


AxmHelixCenterMove API carries out circular interpolation drive from the current position (S) with input of exit point (E) and center point (C). Since this is a method to calculate circular arc using the position of three points, circle is composed of that accurate center point and exit point are inputted, and API is operated. If a wrong coordination is inputted, API will not be operated and returns FALSE. If circular interpolation drive is needed to carry out counterclockwise with center of (0, 100) from the current position (0,0) to (100,100) position, do the following.

```
// Carry out circular interpolation with center of (0, 100) from the current position (0,0)
// to (100,100) position.
long lAxisNo1 = 0;
long lAxisNo2 = 1;
DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;
AxmMotSetAbsRelMode(lAxisNo1, uAbsRelMode);    // Set to relative position drive
AxmMotSetProfileMode(lAxisNo1, uProfileMode);  // Use S-curve velocity profile

double dCenterXPosition = 0;
double dCenterYPosition = 100;
double dEndXPosition = 100;
double dEndYPosition = 100;
double dZPosition = 200;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 1;                              // [0] Ccw direction, [1] Cw direction
AxmHelixCenterMove(lCoordinate, dCenterXPosition, dCenterYPosition, dEndXPosition,
  dEndYPosition, dZPosition, dMaxVelocity, dMaxAccel, dMaxDecel, uCWDir);
```

- AxmHelixRadiusMove



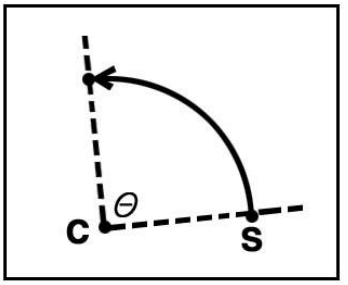
AxmHelixRadiusMove API carries out circular interpolation drive from the current position (S) with input of exit point (E) and radius (r). Since this is a method to calculate arc that passes two points and has constant radius, two circles are to be structured. Therefore, if uShortDistance value is 0 for the input radius, circle with short distance (a) is selected, and if 1, circle with long distance (b) is selected.

If circular interpolation is needed to carry out clockwise with long distance that has a radius of 120 from the current position to (300,200) position, do the following.

```
// Carry out circular interpolation from the current position to (300,200) position with
// long distance that has a radius of 120.
long lAxisNo1 = 0;
long lAxisNo2 = 1;
DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;
AxmMotSetAbsRelMode(lAxisNo1, uAbsRelMode);    // Set to relative position drive
AxmMotSetProfileMode(lAxisNo1, uProfileMode);  // Use S-curve velocity profile

double dRadius = 120;
double dEndXPosition = 300
double dEndYPosition = 200;
double dZPosition = 200;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 0;            // [0] Cw direction, [1] Ccw direction
DWORD uShortDistance = 1;    // [0]: Small (circular) distance ,[1] Big (circular) distance
AxmHelixRadiusMove (lCoordinate, dRadius, dEndXPosition, dEndYPosition, dZPosition,
  dMaxVelocity, dMaxAccel, dMaxDecel, uCWDir, uShortDistance);
```

- AxmHelixAngleMove



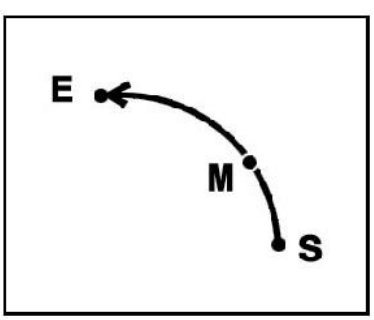
AxmHelixAngleMove API carries out circular interpolation drive from the current position (S) with input of center point (C) and rotational angle (θ). This is a method to calculate circular arc using radius and angle of arc, and a circle is structured.

Circular interpolation is carried out as much as 200 degrees with center of (200,200) from the current position, and if interpolation drive is needed to carry out clockwise, do the following.

```
// Carried out circular interpolation as much as 200 degrees with center of (200,200) from
// the current position.
long lAxisNo1 = 0;
long lAxisNo2 = 1;
DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;
AxmMotSetAbsRelMode(lAxisNo1, uAbsRelMode);       // Set to relative position drive
AxmMotSetProfileMode(lAxisNo1, uProfileMode);     // Use S-curve velocity profile

double dCenterXPosition = 200;
double dCenterYPosition = 200;
double dAngle = 200;
double dZPosition = 200;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
DWORD uCWDir = 0;                                 // [0] Cw direction, [1] Ccw direction
AxmHelixAngleMove(lCoordinate, dCenterXPosition, dCenterYPosition, dAngle, dZPosition,
  dMaxVelocity, dMaxAccel, dMaxDecel, uCWDir);
```

- AxmHelixPointMove



AxmHelixPointMove API carries out circular interpolation drive from the current position (S) with input of middle point (M) and exit point (E). This is a method to calculate circular arc that passes three points, and a circle is structured.

If interpolation drive is needed to carry out from the current position to (150,150) passing (100,150), do the following.

```
// Carry out circular interpolation from the current position to (150,150) passing
// (100,150).
long lAxisNo1 = 0;
long lAxisNo2 = 1;
DWORD uAbsRelMode = 1;
DWORD uProfileMode = 3;
AxmMotSetAbsRelMode(lAxisNo1, uAbsRelMode);       // Set to relative position drive
AxmMotSetProfileMode(lAxisNo1, uProfileMode);     // Use S-curve velocity profile

double dMidXPosition = 100;
double dMidYPosition = 50;
double dEndXPosition = 150;
double dEndYPosition = 150;
double dZPosition = 200;
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
AxmHelixPointMove(lCoordinate, dMidXPosition, dMidYPosition, dEndXPosition, dEndYPosition,
   dZPosition, dMaxVelocity, dMaxAccel, dMaxDecel);
```

These API are what stores on internal queue for helical continuous interpolation drive on the assigned coordination system when use AxmContiBeginNode and AxmContiEndNode. (Use with continuous interpolation API).

Continuous interpolation drive API is what is to drive interpolation drive mentioned before continuously, and has a drive that accelerates after starting drive, passes assigned positions with constant velocity, and decelerates and stops.

One unique thing is that, after axes to use for continuous interpolation takes mapping procedure to the new coordination system, stores commands to carry out interpolation drive continuously in the memory area of “continuous interpolation queue,” if continuous interpolation start command is given later, value goes into motion control chip after coming out one by one from the stored continuous interpolation data, and carries out interpolation drive continuously. Of course, interpolation data can be inputted after interpolation drive is started.

In order to carry out continuous interpolation drive, first, axes to be used for continuous interpolation must take a process of mapping. Interpolation drive APIs that are used for continuous interpolation are to be a combination of 2 axes interpolation drives, but they have to take mapping to one coordination system that includes all axes to be used after mapping of axes. For example, if it is carried out interpolation drive of 1 and 2 axis after interpolation drive of 0 and 1 axis continuously using three axes of 0, 1, and 2, they have to take mapping to one coordination system. After mapping, set velocity profile to use in corresponding coordination system, and set absolute/relative position drive.

```
// Ex24_AXM_Helix.cpp : Defines the entry point for the console application.
// Start helical continuous interpolation drive after home setting of 3 axes
// mechanical part.
#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_1 1
#define AXIS_2 2
#define AXIS_3 3

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           for(int nAxisNo = 0; nAxisNo<3; nAxisNo++)
           {
              // Set Active level of +End limit and -End limit on axis 0 to HIGH and
              //  emergency stop.
              AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
              // Set input level of inposition signal on axis 0 to HIGH.
              AxmSignalSetInpos(nAxisNo, HIGH);
              // Set input level of alarm signal on axis 0 to LOW.
              AxmSignalSetServoAlarm(nAxisNo, LOW);
              // Set Active input level of ESTOP signal on axis 0 to HIGH.
              AxmSignalSetStop(nAxisNo, 0, HIGH);

              // Make ordered value on axis 0 to be mm unit.
              AxmMotSetMoveUnitPerPulse(nAxisNo, 1, 1000);

              // Set initial velocity on axis 0 to 1.  Default : 1
              AxmMotSetMinVel(nAxisNo, 1);
              // Make pulse output method on axis 0 to TwoCwCcwHigh.
```

```
    AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
    // Set encoder input method on designated axis to 4 multiplication.
    AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);

    // Set to relative coordination drive.
    AxmMotSetAbsRelMode(nAxisNo, 1);
    // Set to symmetrical S-curve velocity profile.
    AxmMotSetProfileMode(nAxisNo, 3);

    // Set home search method.
    AxmHomeSetMethod(nAxisNo,-1,HomeSensor,1,2000.0,0.0);
    // Set velocity that will be used during home search.
    AxmHomeSetVel(nAxisNo, 100, 20, 10, 10, 200, 40);
    AxmStatusSetActPos(nAxisNo, 0.0);
    AxmStatusSetCmdPos(nAxisNo, 0.0);

    // Servo On.
    AxmSignalServoOn(nAxisNo, ENABLE);
}

// Search home.
printf("Start home search pressing any key.\n");
getch();
AxmHomeSetStart(AXIS_0);
AxmHomeSetStart(AXIS_1);
AxmHomeSetStart(AXIS_2);

// Verify current home search progress rate.
DWORD uHomeResult_0, uHomeResult_1, uHomeResult_2;
DWORD uHomeMainStepNumber , uHomeStepNumber_0;
DWORD uHomeStepNumber_1, uHomeStepNumber_2;
AxmHomeGetResult(AXIS_0, &uHomeResult_0);
AxmHomeGetResult(AXIS_1, &uHomeResult_1);
AxmHomeGetResult(AXIS_2, &uHomeResult_2);

while(uHomeResult_0 == HOME_SEARCHING || uHomeResult_1 == HOME_SEARCHING ||
    uHomeResult_2 == HOME_SEARCHING )
{
    AxmHomeGetResult(AXIS_0, &uHomeResult_0);
    AxmHomeGetResult(AXIS_1, &uHomeResult_1);
    AxmHomeGetResult(AXIS_2, &uHomeResult_2);

    AxmHomeGetRate(AXIS_0,  &uHomeMainStepNumber, &uHomeStepNumber_0);
    AxmHomeGetRate(AXIS_1,  &uHomeMainStepNumber, &uHomeStepNumber_1);
    AxmHomeGetRate(AXIS_2,  &uHomeMainStepNumber,  &uHomeStepNumber_2);

    printf("\r In home search... axis 0 : %d%, 1 axis : %d, 2 axis : %d% % ",
        uHomeStepNumber_0, uHomeStepNumber_1, uHomeStepNumber_2);
}
```

```
        long   lAxis[4];
        long  lCoordinate = 0;
        double dVelocity = 200;
        double dAccel = 400;

        for(int i=0; i<4; i++)
        {
           lAxis[i]     = i;
        }

        AxmContiWriteClear(lCoordinate);
        AxmContiSetAxisMap(lCoordinate, 4, lAxis);
        AxmContiSetAbsRelMode(lCoordinate, 1);

        // Input helical interpolation drive data.
        AxmContiBeginNode(lCoordinate);
        AxmHelixCenterMove(lCoordinate, 50, 50, 0, 0, 100, dVelocity, dAccel, dAccel,
           1);
        AxmHelixCenterMove(lCoordinate, 50, 50, 0, 0, 100, dVelocity, dAccel, dAccel,
           1);
        AxmHelixCenterMove(lCoordinate, 50, 50, 0, 0, 100, dVelocity, dAccel, dAccel,
           1);
        AxmHelixCenterMove(lCoordinate, 50, 50, 0, 0, 100, dVelocity, dAccel, dAccel,
           1);
        AxmHelixCenterMove(lCoordinate, 50, 50, 0, 0, 100, dVelocity, dAccel, dAccel,
           1);
        AxmHelixCenterMove(lCoordinate, 50, 50, 0, 0, 100, dVelocity, dAccel, dAccel,
           1);
        AxmContiEndNode(lCoordinate);
        AxmContiStart(lCoordinate , 0, 0);

        printf("\nExit pressing any key.\n");
        getch();
      }
      else
        printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",
           Code);
   }
   else
      printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
   printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

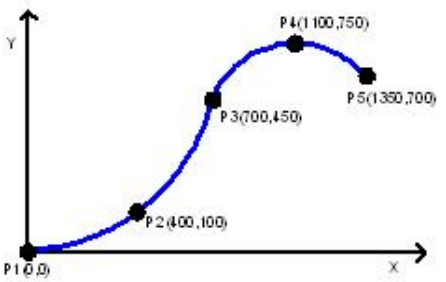
// Exit library.
if (AxlClose())
   printf("Library is exited.\n");
else
```

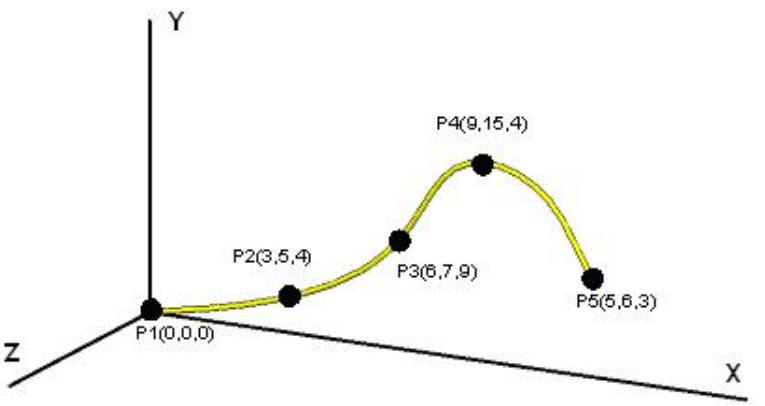
```
        printf("Library is not exited normally.\n");
}
```

Spline Interpolation Motion

Spline interpolation control provides B spline interpolation function. B spline interpolation shows below, and it changes data points assigned by user to free curve and interpolates. To drive this interpolation with direct motion, it needs to be used with continuous interpolation motion function. It provides basic X, Y, and Z axis. In case of 4 axis motion board, 3 axes out of X(0),Y(1),Z(2),U(3) can be selected and used. And, for 8 axis motion board, 3 axes out of X(0),Y(1),Z(2),U(3) in one chip and 3 axes out of X2(4),Y2(5),Z2(6),U(7) in other chip can be selected and used.

Note) Spline interpolation is only supported by PCI-N804/404.






```
// Ex25_AXM_Spline.cpp : Defines the entry point for the console application.
// Start continuous 2 dimension spline interpolation drive after home setting of 2 axes
// mechanical part.
#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

#define AXIS_0 0
#define AXIS_1 1
```

```
void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is set automatically in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library is initialized.\n");

      // Inspect whether motion module exists or not.
      DWORD dwStatus;
      Code = AxmInfoIsMotionModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            printf("Motion module exists.\n");

            for(int nAxisNo = 0; nAxisNo<2; nAxisNo++)
            {
               // Set Active level of +End limit and -End limit on axis 0 to HIGH and
               // emergency stop.
               AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
               // Set input level of inposition signal on axis 0 to HIGH.
               AxmSignalSetInpos(nAxisNo, HIGH);
               // Set input level of alarm signal on axis 0 to LOW.
               AxmSignalSetServoAlarm(nAxisNo, LOW);
               // Set Active input level of ESTOP signal on axis 0 to HIGH.
               AxmSignalSetStop(nAxisNo, 0, HIGH);

               // Make ordered value on axis 0 to be mm unit.
               AxmMotSetMoveUnitPerPulse(nAxisNo, 10, 10000);
               // Set initial velocity on axis 0 to 1.  Default : 1
               AxmMotSetMinVel(nAxisNo, 1);
               // Make pulse output method on axis 0 to TwoCwCcwHigh.
               AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
               // Set encoder input method on designated axis to 4 multiplication.
               AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);

               // Set to relative coordination drive.
               AxmMotSetAbsRelMode(nAxisNo, 1);
               // Set to symmetrical S-curve velocity profile.
               AxmMotSetProfileMode(nAxisNo, 3);

               // Set home search method.
               AxmHomeSetMethod(nAxisNo,-1,HomeSensor,1,2000.0,0.0);
               // Set velocity that will be used during home search.
               AxmHomeSetVel(nAxisNo, 100, 20, 10, 10, 200, 40);
```

```
    // Servo On.
    AxmSignalServoOn(nAxisNo, ENABLE);
}

// Search home.
printf("Start home search pressing any key.\n");
getch();
AxmHomeSetStart(AXIS_0);
AxmHomeSetStart(AXIS_1);

// Verify current home search progress rate.
DWORD uHomeResult_0, uHomeResult_1;
DWORD uHomeMainStepNumber,uHomeStepNumber_0,uHomeStepNumber_1;
AxmHomeGetResult(AXIS_0, &uHomeResult_0);
AxmHomeGetResult(AXIS_1, &uHomeResult_1);
while(uHomeResult_0 == HOME_SEARCHING || uHomeResult_1 == HOME_SEARCHING)
{
    AxmHomeGetResult(AXIS_0, &uHomeResult_0);
    AxmHomeGetResult(AXIS_1, &uHomeResult_1);

    AxmHomeGetRate(AXIS_0,  &uHomeMainStepNumber, &uHomeStepNumber_0);
    AxmHomeGetRate(AXIS_1,  &uHomeMainStepNumber, &uHomeStepNumber_1);
    printf("\r In home search... axis 0 : %d%, 1 axis : %d % ",
        uHomeStepNumber_0, uHomeStepNumber_1);
}

printf("\nMove to relative position (50,25) with straight line interpolation
    drive.");
long  lAxesNo[2] = {0,1};
double dpPosition[2] = {50,25};
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
long  lCoordinate = 0;
double dVelocity = 200;
double dAccel = 400;

AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 2, lAxesNo);
AxmContiSetAbsRelMode (lCoordinate, 1);        // Set to relative position drive
AxmLineMove(lCoordinate, dpPosition, dMaxVelocity, dMaxAccel, dMaxDecel);

// Wait until all motions are exited.
BOOL WaitForDone = TRUE;
DWORD uStatus0, uStatus1;
while(WaitForDone) {
    AxmStatusReadInMotion(AXIS_0, &uStatus0);
    AxmStatusReadInMotion(AXIS_1, &uStatus1);
    if(uStatus0 == 0 && uStatus1 == 0)
```

```
                WaitForDone = FALSE;
            }

            double dPosX[5];
            double dPosY[5];
            AxmContiWriteClear(lCoordinate);
            AxmContiSetAxisMap(lCoordinate, 2, lAxesNo);
            AxmContiSetAbsRelMode(lCoordinate, 0);

            dPosX[0] = 0   , dPosY[0] = 0;
            dPosX[1] = 450,  dPosY[1] = 100;
            dPosX[2] = 750,  dPosY[2] = 450;
            dPosX[3] = 1080, dPosY[3] = 750;
            dPosX[4] = 1300, dPosY[4] = 700;

            AxmContiBeginNode(lCoordinate);
            AxmSplineWrite(lCoordinate, 5, dPosX, dPosY, dVelocity, dAccel, dAccel, 1, 100);
            AxmContiEndNode(lCoordinate);

            printf("\nExit pressing any key.");
            getch();
        }
        else
            printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",
                Code);
    }
    else
        printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )   code 0x%x\n",Code);
  }
  else
    printf ("AxlOpen() : ERROR   code 0x%x\n",Code);

  // Exit library.
  if (AxlClose())
    printf("Library is exited.\n");
  else
    printf("Library is not exited normally.\n");
}
```

Example of 3 dimension spline interpolation

```
// Ex26_AXM_Spline2.cpp : Defines the entry point for the console application.
// Start continuous 3 dimension spline interpolation drive after home setting of 3 axes
// mechanical part.
#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"
```

```
#define AXIS_0 0
#define AXIS_1 1
#define AXIS_2 2

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is set automatically in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library is initialized.\n");

     // Inspect whether motion module exists or not.
     DWORD dwStatus;
     Code = AxmInfoIsMotionModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("Motion module exists.\n");

           for(int nAxisNo = 0; nAxisNo<3; nAxisNo++)
           {
              // Set Active level of +End limit and -End limit on axis 0 to HIGH and
              // emergency stop.
              AxmSignalSetLimit(nAxisNo, 0, HIGH, HIGH);
              // Set input level of inposition signal on axis 0 to HIGH.
              AxmSignalSetInpos(nAxisNo, HIGH);
              // Set input level of alarm signal on axis 0 to LOW.
              AxmSignalSetServoAlarm(nAxisNo, LOW);
              // Set Active input level of ESTOP signal on axis 0 to HIGH.
              AxmSignalSetStop(nAxisNo, 0, HIGH);
              // Make ordered value on axis 0 to be mm unit.
              AxmMotSetMoveUnitPerPulse(nAxisNo, 10, 10000);
              // Set initial velocity on axis 0 to 1.  Default : 1
              AxmMotSetMinVel(nAxisNo, 1);
              // Make pulse output method on axis 0 to TwoCwCcwHigh.
              AxmMotSetPulseOutMethod(nAxisNo, TwoCcwCwHigh);
              // Set encoder input method on designated axis to 4 multiplication.
              AxmMotSetEncInputMethod(nAxisNo, ObverseSqr4Mode);
              // Set home search method.
              AxmHomeSetMethod(nAxisNo,-1,HomeSensor,1,2000.0,0.0);
              // Set velocity that will be used during home search.
              AxmHomeSetVel(nAxisNo, 100, 20, 10, 10, 200, 40);
              // Servo On.
              AxmSignalServoOn(nAxisNo, ENABLE);
           }
```

```
// Search home.
printf("Start home search pressing any key.\n");
getch();
AxmHomeSetStart(AXIS_0);
AxmHomeSetStart(AXIS_1);
AxmHomeSetStart(AXIS_2);

// Verify current home search progress rate.
DWORD uHomeResult_0, uHomeResult_1, uHomeResult_2;
DWORD uHomeMainStepNumber , uHomeStepNumber_0;
DWORD uHomeStepNumber_1, uHomeStepNumber_2;
AxmHomeGetResult(AXIS_0, &uHomeResult_0);
AxmHomeGetResult(AXIS_1, &uHomeResult_1);
AxmHomeGetResult(AXIS_2, &uHomeResult_2);

while(uHomeResult_0 == HOME_SEARCHING || uHomeResult_1 == HOME_SEARCHING ||
   uHomeResult_2 == HOME_SEARCHING)
{
   AxmHomeGetResult(AXIS_0, &uHomeResult_0);
   AxmHomeGetResult(AXIS_1, &uHomeResult_1);
   AxmHomeGetResult(AXIS_2, &uHomeResult_2);
   AxmHomeGetRate(AXIS_0, &uHomeMainStepNumber , &uHomeStepNumber_0);
   AxmHomeGetRate(AXIS_1, &uHomeMainStepNumber , &uHomeStepNumber_1);
   AxmHomeGetRate(AXIS_2, &uHomeMainStepNumber , &uHomeStepNumber_2);

   printf("\r In home search... axis 0 : %d%, 1 axis : %d, 2 axis : %d % ",
      uHomeStepNumber_0, uHomeStepNumber_1, uHomeStepNumber_2);
}

printf("\n Move to relative position (100,100,100) with straight line
   interpolation drive.");
long lAxesNo[3] = {0,1,2};
double dpPosition[3] = {100,100,100};
double dMaxVelocity = 100;
double dMaxAccel = 200;
double dMaxDecel = 200;
long  lCoordinate = 0;
double dVelocity = 200;
double dAccel = 400;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 3, lAxesNo);
AxmContiSetAbsRelMode (lCoordinate, 1);        // Set to relative position drive
AxmLineMove(lCoordinate, dpPosition, dMaxVelocity, dMaxAccel, dMaxDecel);

// Wait until all motions are exited.
BOOL WaitForDone = TRUE;
DWORD uStatus0, uStatus1, uStatus2;
while(WaitForDone) {
```

```
            AxmStatusReadInMotion(AXIS_0, &uStatus0);
            AxmStatusReadInMotion(AXIS_1, &uStatus1);
            AxmStatusReadInMotion(AXIS_2, &uStatus2);
            if(uStatus0 == 0 && uStatus1 == 0 && uStatus2 == 0)
               WaitForDone = FALSE;
         }

         double dPosX[5];
         double dPosY[5];
         AxmContiWriteClear(lCoordinate);
         AxmContiSetAxisMap(lCoordinate, 3, lAxesNo);
         AxmContiSetAbsRelMode(lCoordinate, 0);

         dPosX[0] = 0   , dPosY[0] = 0;
         dPosX[1] = 450,  dPosY[1] = 100;
         dPosX[2] = 750,  dPosY[2] = 450;
         dPosX[3] = 1080, dPosY[3] = 750;
         dPosX[4] = 1300, dPosY[4] = 700;
         AxmContiBeginNode(lCoordinate);
         AxmSplineWrite(lCoordinate, 5, dPosX, dPosY, dVelocity, dAccel, dAccel, 1,  100);
         AxmContiEndNode(lCoordinate);
         printf("\nExit pressing any key.");
         getch();
      }
      else
         printf ("AxmInfoIsMotionModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",
            Code);
   }
   else
      printf ("AxmInfoIsMotionModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
   printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if (AxlClose())
   printf("Library is exited.\n");
else
   printf("Library is not exited normally.\n");
}
```

Electronic CAM [ECAM]

Electronic Cam Drive (ECAM) is driven to move the slave axis with a relationship to the master axis. In ECAM, the information of master axis is used to drive the slave axis to the position which is set by user.

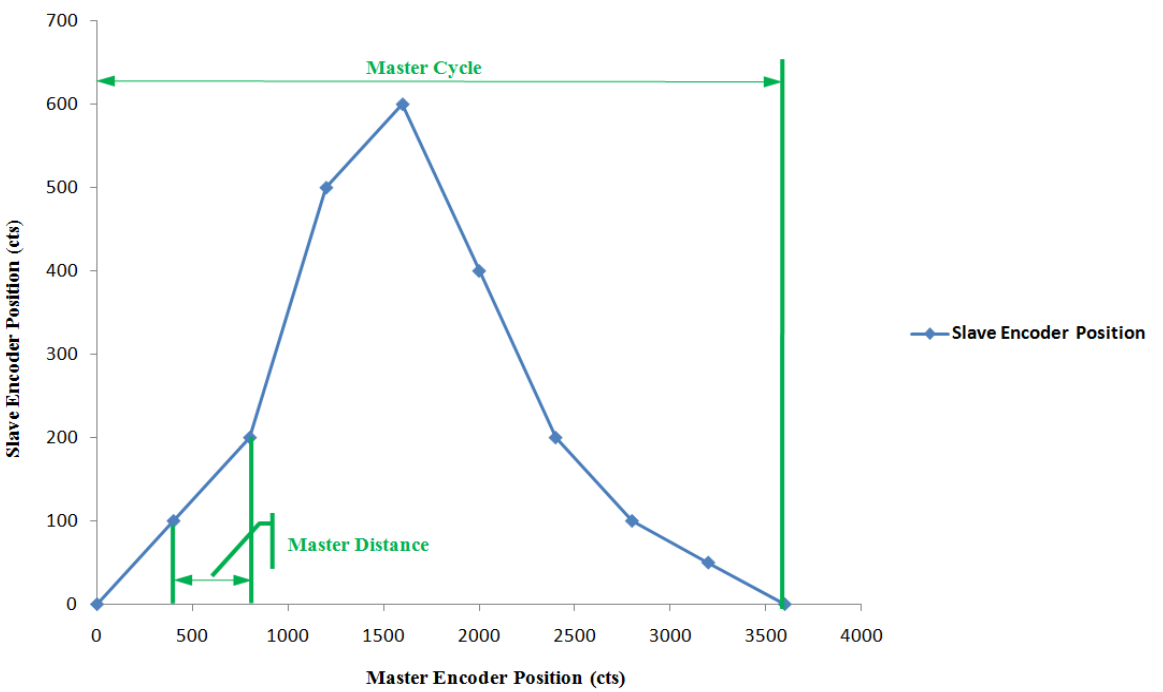
ECAM can be only used in PCI-R1604-MLII products.

The details of the master axis position information for the slave axis in following table will be set.

ECAM Point	Master Distance	Slave Position
0	0	0
1	400	100
2	800	200
3	1200	500
4	1600	600
5	2000	400
6	2400	200
7	2800	100
8	3200	50
9	3600	0

In ECAM, the travel distance of master axis and slave axis must be in period, start position and end position of the slave axis must be '0'.

As the above table, slave axis moves based on the profile in the below figure.



ECAM feature is able to set in order to use the position information based on the number of position data, the master axis position information, the slave axis position information, start position information data that is applied to the master axis. Input the slave and master position into an array that starts at '0' and the final

value of array index should be added. Up to 512 position can be set. APIs in the library include the setting position information (AxmEcamSet, AxmEcamGet), setting whether to use (AxmEcamEnableBySlave, AxmEcamIsSlaveEnable, AxmEcamEnableByMaster).

```
// Master axis : 0, Slave axis : 1, use ECAM with 10 position information.
long lAxis, lMasterAxis;
double dPos[11], dSlavePos[11];
lAxis = 1 ;
lMasterAxis = 0 ;
lNumPeriod = 10 ;
dPos[0] = 0;    dSlavePos[0] = 0;
dPos[1] = 100;   dSlavePos[1] = 9.549;
dPos[2] = 200;   dSlavePos[2] = 34.549;
dPos[3] = 300;   dSlavePos[3] = 65.45;
dPos[4] = 400;   dSlavePos[4] = 90.45;
dPos[5] = 500;   dSlavePos[5] = 100;
dPos[6] = 600;   dSlavePos[6] = 90.45;
dPos[7] = 700;   dSlavePos[7] = 65.45;
dPos[8] = 800;   dSlavePos[8] = 34.549;
dPos[9] = 900;   dSlavePos[9] = 9.549;
dPos[10] = 1000; dSlavePos[10] = 0;

AxmEcamSet(lAxis, lMasterAxis, lNumPeriod, 0, dPos, dSlavePos );
AxmEcamEnableBySlave(lAxis, ENABLE);
```

Compensation Geometric Errors

Geometric errors are regarded as the machine errors which exist under cold and warm up conditions and which do not change with time. This means that the machine repeats the same error again and again for some duration. Major geometric errors are roll, pitch, yaw and squareness errors. These types of errors originate from the manufacturing or assembly defects of different parts of the machine. In the case of assembling the machine tool, the designed state and assembly state exerts an important effect on the accuracy of the machine tool.

Compensation geometric errors is only available in PCI-R1604-MLII.

Compensation table functions are calibrated by using the input actual position in the real-time movement, the calibration value of table should be given in each axis. Real-time compensation period is passed through the communication in each cycle of motion command, and is obtained automatically to input the intermediate value of compensation table in each cycle communication without the help of the PC PCI-R1604-MLII.

The compensation table feature set the input section for receiving the compensation value to the table, the maximum input number is 512. The first value and final value in compensation table must be '0', between two entries in the table, the compensated value will be linearly interpolated to obtain the correction for the current communication cycle. X1, x2, .., xn are the compensated position and e1, e2, .., en are the corresponding position errors.

x	x1	x2	⋯	xi-1	xi	⋯	xn
e	e1	e2	⋯	ei-1	ei	⋯	en

At the point between xi and xi-1, the value is obtained as the following equation.

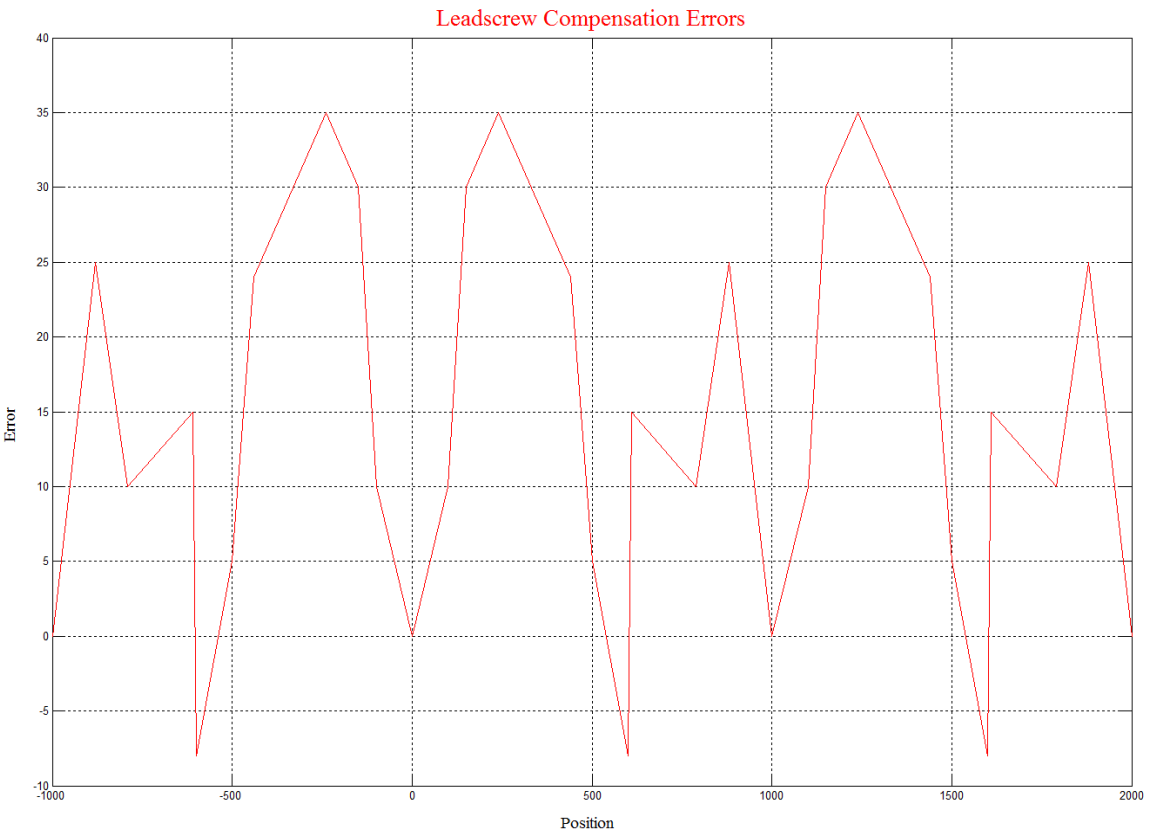
$$e = (x - x_i)\frac{e_i - e_{i-1}}{x_i - x_{i-1}} + e_i$$

Outside of this range, the uncorrected position is rolled over to within this range – essentially a modulo (remainder) operation – before the compensation is done. This permits compensation of rotary axes over several revolutions, and simple compensation for encoder eccentricity. If user defines the table which is long enough to cover the entire source motor travel, the rollover feature will never be used.

Assume that user define the entries in the positive values. However, if the motor has a travel to the opposite side-negative values, and if the rollover feature is active, these entries should be made as if they were past the positive end of the motor range. For example, if the motor travel were ± 1000 units and the table entries extends from 0 to 1000 units.

x	0	100	150	240	440	500	600	610	790	880	1000
e	0	10	30	35	24	5	-8	15	10	25	0

The actual calculated value of compensation will be shown as the below graph.



The compensation table feature reflects the actual value of servo driver as the command position while the internal position is not reflected. If the table is used to compensate, the value 500 as command position will be matched with 505 as actual position.

```
// Compensation feature is applied to axis 1, from position 1000, 10 entries,
// enable RollOver feature.
long lAxis, lNumEntry ;
double dPos[11], dCorrection[11], dStartPos;
DWORD dwRollOver ;
lAxis = 1 ;
lNumEntry = 10 ;
dStartPos = 1000 ;
dwRollOver  = 1 ;
dPos[0] = 0;     dCorrection[0] = 0;
dPos[1] = 100;   dCorrection[1] = 10;
dPos[2] = 150;   dCorrection[2] = 30;
dPos[3] = 240;   dCorrection[3] = 35;
dPos[4] = 440;   dCorrection[4] = 24;
dPos[5] = 500;   dCorrection[5] = 5;
```

```
dPos[6] = 600;   dCorrection[6] = -8;
dPos[7] = 610;   dCorrection[7] = 15;
dPos[8] = 790;   dCorrection[8] = 10;
dPos[9] = 880;   dCorrection[9] = 25;
dPos[10] = 1000; dCorrection[10] = 0;

AxmCompensationSet(lAxis, lNumEntry, dStartPos, dPos, dCorrection, dwRollOver );
AxmCompensationEnable(lAxisNo, TRUE);
```

Motion Command Manual Information

This manual is required for super ordinate motion controller including PCI-N804/404 and distributed network motion controller such as PCI-R1604/R1604-MLII to drive with Microsoft VC++6.0, Visual Basic, Borland C-Builder, Delphi language in OS environment of Windows 98, Windows NT, Windows 2000 or Windows XP, and describes library API classified by function.

Header File

C++

AXM.h

Visual Basic

AXM.bas

Delphi

AXM.pas

Names of API in this manual

Names of API in this manual can classify motion by prefix.

※ *Library API Prefix*

Axm_ : shows that API is a motion module and library exclusive use. All APIs starting with Axm are defined as AXM.h.

Set_ : sets registers or variables of chip.

Get_ : verifies variable values set by Set API in pairs with Set or reads state of chip register.

Is_ : reads state of motion drive, continuous interpolation Queue, or Servo-On signal, etc.

_Move : is a drive start command, and if an API that includes this command is carried out, pulse is outputted immediately from motion module, but at the point of that pulse output is exited, it gets out of operation API.

Start_Move : is same with move command, but at the point of that pulse is outputted from motion module after API is started, it gets out of API.

Line : sets an argument that is required for straight line interpolation drive and starts drive.

Circle : sets an argument that is required for circular interpolation drive and starts drive.

Conti_ : sets an argument that is required for continuous interpolation drive and starts drive.

Vel_ : carries out continuous drive with velocity mode or operates velocity override.

Accel_ : sets acceleration of accelerating section (unit of acceleration is PPS[Pulses/sec] if Unit/pulse is 1/1) in drive or jerk value.

Decel_ : sets acceleration of decelerating section (unit of acceleration is PPS[Pulses/sec] if Unit/pulse is 1/1) in drive or jerk value.

Sensor_ : sets and inspects parameter used for sensor position search drive.

mpg_ : sets and inspects parameter that carries out MPG drive or is required.

CRC_ : sets and inspects parameter that carries out clear of remaining pulse or is required.

Common arguments of APIs used in this manual have following meaning.

long lAxis : means a specific axis number, and is a channel (axis) number (starting from 0) obtained by initialization API. Start of channel (axis) number (starting from 0) is “0.”

DWORD uLevel : means output or input level of signal in signal setting, and has HIGH(logic '1') or LOW(logic '0') value electrically. Or it also has Enable/ Disable value of signal.

double dPos : means a position value at absolute mode, and means a distance to move at relative mode. Unit is unit distance.

double dVelocity : means moving velocity, and unit is unit distance per second. That is, if unit distance is set by 1mm, velocity value as an argument of API is to be mm/sec.

double dAcceleration : means moving acceleration (unit of acceleration is PPS[Pulses/sec] if Unit/pulse is 1/1) or accelerating time. In case of moving acceleration (unit of acceleration is PPS[Pulses/sec] if Unit/pulse is 1/1), unit is change amount of velocity per second. Here, velocity is velocity used as an argument of API. If acceleration value is to be twice of velocity value, it means that acceleration time to maximum velocity is 0.5 sec. In case of acceleration time, unit is second. Once acceleration time is set, it drives with acceleration ratio value that is set automatically. If acceleration time is 2 sec, acceleration ratio is 1/2 times value of maximum velocity, and it means acceleration time from the drive start point to maximum velocity is 2 sec.

Terminilogy List

Original Word	Axl Word
Position	Pos
Velocity	Vel
Acceleration	Accel
Deceleration	Decel
Command	Cmd
Actual	Act
Revolution	Rev
Coordinate	Coord
Mapping	Map
Information	Info
Motion Parameter	Mot
Motion Profile	
Continuous	Conti
Absolute	Abs
Relative	Rel
Manual Pulse Generation	MPG
Current Residual Clear	Crc
Number	Num

Motion Command Quick List

API List

Board and Module Verification API (Info) – Information

Function	Description
AxmInfoGetAxis	Returns board number, module position, module ID of corresponding axis.
AxmInfoIsMotionModule	Returns whether motion module exists.
AxmInfoIsInvalidAxisNo	Returns whether corresponding axis is effective.
AxmInfoGetAxisCount	Returns effentive motion axis number installed in system.
AxmInfoGetFirstAxisNo	Returns the first axis number of corresponding board and module.

Virtual Axis API (Virtual)

Function	Description
AxmVirtualSetAxisNoMap	Sets virtual axis.
AxmVirtualGetAxisNoMap	Returns virtual channel (axis) number which is set.
AxmVirtualSetMultiAxisNoMap	Sets multi-virtual axis.
AxmVirtualGetMultiAxisNoMap	Returns multi-virtual channel (axis) which is set.
AxmVirtualResetAxisMap	Clears virtual axis setting.

Interrupt API (Interrupt)

Function	Description
AxmInterruptSetAxis	Uses window message or callback API to receive interrupt message.
AxmInterruptSetAxisEnable	Sets whether interrupt on setting axis is used
AxmInterruptGetAxisEnable	Returns whether interrupt on setting axis is used
AxmInterruptRead	Reads corresponding interrupt information if interrupt is used by event method.
AxmInterruptReadAxisFlag	Returns interrupt flag value on corresponding axis.
AxmInterruptSetUserEnable	Sets whether interrupt set by user of specific axis is occurred.
AxmInterruptGetUserEnable	Verifies whether interrupt set by user of specific axis is occurred.

Motion Parameter Setting API (Mot) -> Motor Parameter

Function	Description
AxmMotLoadParaAll	Calls 33 parameter files stored with AxmMotSaveParaAll . Arbitrary change is available by user's calling.
AxmMotSaveParaAll	Stores all 33 parameters by axis for the current all axes. Store as .mot file. Call a file using AxmM otLoadParaAll.
AxmMotSetParaLoad	Use AxmMotSaveParaAll API to store information which is set on register of CAMC-IP, QI as a file. The API stores information of all axes as a file. Set 4 parameters of dInitPos, dInitVel, dInitAccel and dInitDecel only from 28 to 31 among parameters. The rest parameters are stored using AxmMotSaveParaAll API reading current state when library is closed.
AxmMotGetParaLoad	Use AxmMotSaveParaAll API to store information which is set on register of CAMC-IP, QI as a file. The API stores information of all axes as a file. The API is used to call only 4 parameters of dInitPos, dInitVel, dInitAccel and dInitDecel from 28 to 31 among parameters that information of all axes is stored.
AxmMotSetPulseOutMethod	Sets pulse output method on a specific axis.
AxmMotGetPulseOutMethod	Returns pulse output method setting on a specific axis,
AxmMotSetEncInputMethod	Sets encoder input method on a specific axis including increasing direction setting of external (actual) count on a specific axis.
AxmMotGetEncInputMethod	Returns encoder input method on a specific axis including increasing direction setting of external (actual) count on a specific axis.
AxmMotSetMoveUnitPerPulse	Sets moving distance per pulse on a specific axis.
AxmMotGetMoveUnitPerPulse	Returns moving distance per pulse on a specific axis.
AxmMotSetDecelMode	Sets deceleration start point search method on a specific axis.
AxmMotGetDecelMode	Returns deceleration start point search method on a specific axis
AxmMotSetRemainPulse	Sets remaining pulse in manual deceleration mode on a specific axis.
AxmMotGetRemainPulse	Returns remaining pulse in manual deceleration mode on a specific axis.
AxmMotSetMaxVel	Sets maximum velocity in constant speed drive API on a specific axis.
AxmMotGetMaxVel	Returns maximum velocity in constant speed drive API on a specific axis.
AxmMotSetAbsRelMode	Sets moving distance calculation mode on a specific axis. If absolute position mode (0) is set, position value used in moving API is to be absolute position value. For example, if current position value is "10" and moves from absolute position mode to "100" position, it will stop at "100" position moving as much as "+90" from the current position. If relative position mode (1) is set, position value used moving API is to be moving amount to move from the current position. For example, if current position value is "10" and moves from relative position mode to "100" position, it will stop at "110" position moving as much as "+100" from the current position.
AxmMotGetAbsRelMode	Returns moving distance calculation mode which is set on a specific axis.
AxmMotSetProfileMode	Sets drive velocity profile mode on a specific axis. Velocity profile mode has symmetrical or asymmetrical Trapezoide / S-

	Curve mode and Quasi-S mode.
AxmMotGetProfileMode	Returns drive velocity profile mode which is set on a specific axis.
AxmMotSetAccelUnit	Sets acceleration unit on a specific axis. Acceleration unit can be selected among u/s2, sec, rev/s2.
AxmMotGetAccelUnit	Returns acceleration unit which is set on a specific axis.
AxmMotSetMinVel	Sets initial velocity on a specific axis.
AxmMotGetMinVel	Returns initial velocity on a specific axis.
AxmMotSetAccelJerk	Sets acceleration jerk value on a specific axis.
AxmMotGetAccelJerk	Returns acceleration jerk value on a specific axis.
AxmMotSetDecelJerk	Sets deceleration jerk value on a specific axis.
AxmMotGetDecelJerk	Returns deceleration jerk value on a specific axis.

Input and Output Setting API (Signal)

Function	Description
AxmSignalSetZphaseLevel	Sets Z phase level on a specific axis.
AxmSignalGetZphaseLevel	Returns Z phase level on a specific axis.
AxmSignalSetServoOnLevel	Sets output level of Servo-On signal on a specific axis.
AxmSignalGetServoOnLevel	Returns output level of Servo-On signal on a specific axis.
AxmSignalSetServoAlarmResetLevel	Sets output level of Servo-Alarm Reset signal on a specific axis.
AxmSignalGetServoAlarmResetLevel	Returns output level of Servo-Alarm Reset signal on a specific axis.
AxmSignalSetInpos	Sets whether Inpositon signal is used and signal input level on a specific axis
AxmSignalGetInpos	Returns whether Inpositon signal is used and signal input level on a specific axis.
AxmSignalReadInpos	Returns Inpositon signal input state on a specific axis.
AxmSignalSetServoAlarm	Sets whether emergency stop is used and signal input level during alarm signal input on a specific axis.
AxmSignalGetServoAlarm	Returns whether emergency stop is used and signal input level during alarm signal input on a specific axis.
AxmSignalReadServoAlarm	Returns input level of alarm signal on a specific axis.
AxmSignalSetLimit	Sets whether end limit sensor is used and input level of signal on a specific axis. Available to set for decelerate stop or emergency stop during end limit sensor signal input.
AxmSignalGetLimit	Returns use of end limit sensor and signal input level and stop mode during signal input on a specific axis.
AxmSignalReadLimit	Returns input state of end limit sensor on a specific axis.
AxmSignalSetSoftLimit	Sets use of Software limit, count to use, and stop method on a specific axis.

AxmSignalGetSoftLimit	Returns use of Software limit, count to use, and stop method on a specific axis.
AxmSignalSetStop	Sets stop method (emergency stop/decelerate stop) or use of emergency stop signal.
AxmSignalGetStop	Returns stop method (emergency stop/decelerate stop) or use of emergency stop signal.
AxmSignalReadStop	Returns input state of emergency stop signal.
AxmSignalServoOn	Outputs Servo-On signal on a specific axis.
AxmSignalIsServoOn	Returns output state of Servo-On signal on a specific axis.
AxmSignalServoAlarmReset	Outputs Servo-Alarm Reset signal on a specific axis.
AxmSignalWriteOutput	Sets universal output value.
AxmSignalReadOutput	Returns universal output value.
AxmSignalWriteOutputBit	Sets universal output value by bit.
AxmSignalReadOutputBit	Returns universal output value by bit.
AxmSignalReadInput	Returns universal input value.
AxmSignalReadInputBit	Returns universal input value by bit.

State Verification (Status)

Function	Description
AxmStatusReadInMotion	Returns pulse output state on a specific axis.
AxmStatusReadDrivePulseCount	Returns drive pulse counter value on a specific axis.
AxmStatusReadMotion	Returns DriveStatus register on a specific axis.
AxmStatusReadStop	Returns EndStatus register on a specific axis.
AxmStatusReadMechanical	Returns Mechanical Signal Data on a specific axis.
AxmStatusReadVel	Reads current drive velocity on a specific axis.
AxmStatusReadPosError	Returns difference between Command Position and Actual Position on a specific axis.
AxmStatusReadDriveDistance	Verifies moving (moved) distance to final drive
AxmStatusSetActPos	Sets actual position on a specific axis.
AxmStatusGetActPos	Returns Actual position on a specific axis.
AxmStatusSetCmdPos	Sets Command position on a specific axis.
AxmStatusGetCmdPos	Returns Command position on a specific axis.
AxmStatusGetCmdPos	Return Cmd position on a specific axis.
AxmStatusRequestServoAlarm	Send the request to read the Alarm Code on a specific axis.
AxmStatusReadServoAlarm	Read the Alarm Code on a specific axis.
AxmStatusGetServoAlarmString	Return the Alarm String of error code on a specific axis.
AxmStatusRequestServoAlarmHistory	Send the request to read the Alarm History on a specific axis.

AxmStatusReadServoAlarmHistory	Read the Alarm History on a specific axis.
AxmStatusClearServoAlarmHistory	Clear the Alarm History on a specific axis.

Home Search API (Home)

Function	Description
AxmHomeSetSignalLevel	Sets Home sensor Level on a specific axis.
AxmHomeGetSignalLevel	Returns Home sensor Level on a specific axis.
AxmHomeReadSignal	Returns input state of Home signal on a specific axis.
AxmHomeSetMethod	To execute home search on corresponding axis, parameters related to home search must be set. Every home search parameters can be stored and called using AxmMotSaveParaAll and AxmMotLoadParaAll . For home search method setting, set search progressing direction, signal to be used as home, home sensor active level, whether encoder Z phase is detected, and so on.
AxmHomeGetMethod	To execute home search on corresponding axis, parameters related to home search must be set. Return parameters related to already set home.
AxmHomeSetVel	For fast and accurate home search, detect with several steps. Set velocity to be used for each step. Home search time and accuracy are determined by the setting value of velocity. Need to set home search velocity of each axis changing velocity properly by step.
AxmHomeGetVel	For fast and accurate home search, detect with several steps. Set velocity to be used for each step. Home search time and accuracy are determined by the setting value of velocity. Get each set velocity.
AxmHomeSetStart	Once home search start API is executed, be exited automatically after execution of home search in turn with automatic generation of thread that executes home search of corresponding axis inside library.
AxmHomeSetResult	Search result is set to HOME_SUCCESS after succeful execution of home search using home search API. The API can be set the result arbitrary without execution of home search by user. Home search can be executed again setting home search result to HOME_ERR_USER _BRE AK when abnormal condition occurred in normal drive after actual home search completion.
AxmHomeGetResult	Verify search result of home search API. Once home search is started, be set to HOME_SEARCHING, and if fail to home search, failure cause is set. Execute home search again after removal of failure cause.
AxmHomeGetRate	Progress rate can be verified after home search is started. If home search is completed, be returned 100 regardless of success. Home search success can be verified using GetHome Result API.

Position Drive API (Move)

Function	Description
AxmMoveStartPos	Drive with set velocity and acceleration rate up to set position with absolute/relative coordination on a specific axis. Set velocity profile in AxmMotSetProfileMode API. Get out of API at the point that pulse is outputted.
AxmMovePos	Drive with set velocity and acceleration rate up to set position with absolute/relative coordination on a specific axis. Set velocity profile in AxmMotSetProfileMode API. Get out of API at the point that pulse output is exited.
AxmMoveVel	Carry out velocity mode drive with set velocity and acceleration rate continuously. Get out of API at the point that pulse is outputted.
AxmMoveStartMultiVel	Carry out velocity mode drive with set velocity and acceleration rate for the specific multi-axis. Get out of API at the point that pulse output is started.
AxmMoveSignalSearch	Drive with set velocity and acceleration rate until set signal is detected. Get out of API at the point that pulse is outputted.
AxmMoveSignalCapture	API that moves to search the set signal and store the position on a specific axis.
AxmMoveGetCapturePos	API that verifies stored position value in 'AxmMoveSignalCapture' API.
AxmMoveStartMultiPos	Drive with set velocity and acceleration rate up to set position with absolute coordination of specific axes. Velocity profile is set in AxmMotSetProfileMode API. Get out of API at the point that pulse output is exited.
AxmMoveMultiPos	Drive with set velocity and acceleration rate up to set position with absolute coordination of specific axes. Velocity profile is set in AxmMotSetProfileMode API. Get out of API at the point that pulse is outputted.
AxmMoveStop	Make a specific axis decelerate and stop with set deceleration.
AxmMoveEStop	Make a specific axis to emergency stop.
AxmMoveSStop	Make a specific axis to decelerate and stop.

Position and Velocity Override Drive API (Override)

Function	Description
AxmOverridePos	Adjust a specific output pulse number before drive exit on a specific axis.
AxmOverrideSetMaxVel	Set maximum velocity to override before velocity override on a specific axis.
AxmOverrideVel	Make variable setting of velocity during drive on a specific axis.
AxmOverrideAccelVelDecel	Make variable setting of velocity, acceleration and deceleration on a specific axis.
AxmOverrideVelAtPos	Drive velocity mode continuously with set velocity and acceleration rate for the specific axis. Execute velocity override at the position which is set during driving. Get out of API at the point of that pulse output is started.

MASTER, SLAVE Drive API (Link)

Function	Description
AxmLinkSetMode	Set electric gear ratio between Master and Slave axis.
AxmLinkGetMode	Return electric gear ratio between Master and Slave axis.
AxmLinkResetMode	Clear setting of electric gear ratio between Master and Slave axis.

Gantry Drive API (Gantry)

Function	Description
AxmGantrySetEnable	Motion module supports gantry drive system control mechenically linked with two axes. Using this API, if master axis is set to gantry control, corresponding slave axis is synchronized with master axis and drived. Even if drive or stop command is given on slave axis after gantry setting, all commands are ignored.
AxmGantryGetEnable	Return set parameter when master axis is set to gantry control using this API.
AxmGantrySetDisable	If home search is executed after gantry control setting, slave axis executes home search after master axis executes home search. This API is used to verify error value between home on master axis and home on slave axis after home search.

Interpolation Drive API (Line, Circle)

Function	Description
AxmLineMove	This API executes multi-axis straight line interpolation drive assigning start point and end point. Get out of API after drive start. Using with AxmContiBeginNode and AxmContiEndNode , it is to be stored API in queue driving straight line interpolation assigning start point and end point on the specific coordination system. For straight line profile continuous interpolation, it stores in internal queue and starts using AxmContiStart API.
AxmCircleCenterMove	This API executes circular interpolation drive assigning start point, end point and middle point. Get out of API after drive start. Using with AxmContiBeginNode and AxmContiEndNode , it is to be stored API in queue driving circular interpolation assigning start point, end point and middle point on the specific coordination system. For profile circular continuous interpolation drive, it stores in internal queue and starts using AxmContiStart API.
AxmCirclePointMove	This API executes circular interpolation drive assigning middle point and end point. Get out of API after drive start. Using with AxmContiBeginNode and AxmContiEndNode , it is to be stored API in queue driving circular interpolation assigning middle point and end point on the specific coordination system. For profile circular

	continuous interpolation drive, it stores in internal queue and starts using AxmContiStart API.
AxmCircleRadiusMove	This API executes circular interpolation drive assigning start point, end point and radious. Get out of API after drive start. Using with AxmContiBeginNode and AxmContiEndNode , it is to be stored API in queue driving circular interpolation assigning start point, end point and radious on the specific coordination system. For profile circular continuous interpolation drive, it stores in internal queue and starts using AxmContiStart API.
AxmCircleAngleMove	This API executes circular interpolation drive assigning start point, rotational angle and radious. Get out of API after drive start. Using with AxmContiBeginNode and AxmContiEndNode , it is to be stored API in queue driving circular interpolation assigning start point, rotational angle and radious on the specific coordination system. For profile circular continuous interpolation drive, it stores in internal queue and starts using AxmContiStart API.

Continuous Interpolation Setting & Drive API (Conti) -> Continue

Function	Description
AxmContiSetAxisMap	Set continuous interpolation axis mapping on the specific coordination system.
AxmContiGetAxisMap	Return continuous interpolation axis mapping on the specific coordination system.
AxmContiSetAbsRelMode	Set continuous interpolation axis absolute/relative mode on the specific coordination system.
AxmContiGetAbsRelMode	Return continuous interpolation axis absolute/relative mode on the specific coordination system.
AxmContiBeginNode	Start registration of works to execute continuous interpolation on the specific coordination system. After calling this API, until AxmContiEndNode API is called, all motion work executed is not executed actual motion but registered to continuous interpolation motion. And when AxmContiStart API is called, registered motion is executed actually.
AxmContiEndNode	Exit registration of works to execute continuous interpolation on the specific coordination system.
AxmContiReadFree	To verify if internal queue is cleared for interpolation drive on the specific coordination system.
AxmContiReadIndex	To verify interpolation drive number stored in internal queue for interpolation drive on the specific coordination system.
AxmContiWriteClear	To clear all stored internal queue for continuous interpolation drive on the specific coordination system.
AxmContiStart	To start drive of internal continuous interpolation queue stored on the specific coordination system.
AxmContiIsMotion	To verify whether continuous interpolation drive is in drive on the

	specific coordination system.
AxmContiGetNodeNum	To verify continuous interpolation index number which is currently in drive during continuous interpolation drive on the specific coordination system.
AxmContiGetTotalNodeNum	To verify total index number of continuous interpolation drive set on the specific coordination system.

Trigger API (Trigger)

Function	Description
AxmTriggerSetTimeLevel	Set trigger signal duration, output level, and output method on a specific axis.
AxmTriggerGetTimeLevel	Return trigger signal duration, output level, and output method on a specific axis.
AxmTriggerSetAbsPeriod	Set use of trigger function, output level, position comparator, trigger signal duration and trigger output mode on a specific axis.
AxmTriggerGetAbsPeriod	Return use of trigger function, output level, position comparator, trigger signal duration and trigger output mode on a specific axis.
AxmTriggerSetBlock	Output trigger at every certain section from the start position to end position by user assignment.
AxmTriggerGetBlock	Read trigger setting value in 'AxmTriggerSetBlock' API.
AxmTriggerOneShot	Output one trigger pulse by user.
AxmTriggerSetTimerOneshot	Output one trigger pulse after seconds by user.
AxmTriggerOnlyAbs	Output infinite absolute position of absolute position trigger.
AxmTriggerSetReset	Reset trigger setting.

CRC(Remaining Pulse Clear) Drive API

Function	Description
AxmCrcSetMaskLevel	Set use of CRC signal and output level on a specific axis.
AxmCrcGetMaskLevel	Return use of CRC signal and output level on a specific axis.
AxmCrcSetOutput	Generate forced CRC signal on a specific axis.
AxmCrcGetOutput	Return whether forced CRC signal is generated on a specific axis.

MPG(Manul pulse Generator) Drive API

Function	Description
AxmMPGSetEnable	Set MPG input method, drive move mode, moving distance, velocity, etc on assigend axis.

AxmMPGGetEnable	Return MPG input method, drive move mode, moving distance, velocity, etc on assigend axis.
AxmMPGSetRatio	Set pulse ratio to move per pulse in MPG drive move mode on a specific axis.
AxmMPGGetRatio	Return pulse ratio to move per pulse in MPG drive move mode on a specific axis.
AxmMPGReset	Clear MPG drive setting on a specific axis.

Helical Interpolation Setting and Drive API

Function	Description
AxmHelixCenterMove	This API executes helical interpolation drive assigning start point, end point and middle point. Using with AxmContiBeginNode and AxmContiEndNode , it drives helical continuous interpolation assigning start point, end point and middle point on the specific coordination system. For circular continuous interpolation drive, it stores in internal queue. Start using AxmContiStart API. (use with continuous interpolation API)
AxmHelixPointMove	This API executes helical interpolation drive assigning start point, end point and radius. Using with AxmContiBeginNode and AxmContiEndNode , it drives helical continuous interpolation assigning start point and end point on the specific coordination system. For circular continuous interpolation drive, it stores in internal queue. Start using AxmContiStart API. (use with continuous interpolation API)
AxmHelixRadiusMove	This API executes helical interpolation drive assigning start point, end point and radius. Using with AxmContiBeginNode and AxmContiEndNode , it drives helical continuous interpolation assigning start point, end point and radius on the specific coordination system. For circular continuous interpolation drive, it stores in internal queue. Start using AxmContiStart API. (use with continuous interpolation API)
AxmHelixAngleMove	This API executes helical interpolation drive assigning start point, rotational angle and radius. Using with AxmContiBeginNode and AxmContiEndNode , it drives helical continuous interpolation assigning start point, rotational angle and radius on the specific coordination system. For circular continuous interpolation drive, it stores in internal queue. Start using AxmContiStart API. (use with continuous interpolation API)

Spline Interpolation Setting and Drive API (PCI-N804/404 only)

Function	Description
AxmSplineWrite	Use with AxmContiBeginNode and AxmContiEndNode . This API executes spline continuous interpolation drive. For circular continuous interpolation drive, it stores in internal queue. Start using AxmContiStart API. (use with continuous interpolation API).

Electronic CAM Setting and Drive API

Function	Description
AxmEcamSet	Set the axis for master and slave, the array of position, the start position, the number of entry.
AxmEcamGet	Return the array of position, the start position, the number of entry on the specified slave axis.
AxmEcamEnableBySlave	Enable ECAM mode on the specified slave axis before starting to drive.
AxmEcamIsSlaveEnable	Return the status of ECAM mode on the specified slave axis.
AxmEcamEnableByMaster	Enable ECAM mode on all slave axes of the specified master axis before starting to drive.

Compensation Geometric Errors Setting and Drive API

Function	Description
AxmCompensationSet	Set the array of position, the start position, the number of entry on the specified axis.
AxmCompensationGet	Return the array of position, the start position, the number of entry on the specified axis.
AxmCompensationEnable	Enable Compensation mode on the specified axis before starting to drive.
AxmCompensationIsEnable	Return the status of Compensation mode on the specified axis.

Define Sentence

It is used in AXL Motion Library, and list of define sentence is following

Header File : #include "AXHS.h"

FALSE & TRUE

Definition	Value	Explanation
FALSE	00h	False
TRUE	01h	True

AXT_USE

Definition	Value	Explanation
DISABLE	00h	Not -used
ENABLE	01h	Used

AXT_MOTION_STOPMODE_DEF

Definition	Value	Explanation
EMERGENCY_STOP	00h	Emergency stop
SLOWDOWN_STOP	01h	Decelerate stop

AXT_MOTION_EDGE_DEF

Definition	Value	Explanation
SIGNAL_DOWN_EDGE	00h	Falling edge
SIGNAL_UP_EDGE	01h	Rising edge

AXT_MOTION_SELECTION_DEF

Definition	Value	Explanation
COMMAND	00h	Command position
ACTUAL	01h	Actual position

AXT_MOTION_TRIGGER_MODE_DEF

Definition	Value	Explanation
PERIOD_MODE	00h	Trigger periodic mode
ABS_POS_MODE	01h	Trigger absolute mode

AXT_MOTION_LEVEL_MODE_DEF

Definition	Value	Explanation
LOW	00h	B Contact (NORMAL CLOSE)
HIGH	01h	A Contact (NORMAL OPEN)
UNUSED	02h	Not-used
USED	03h	Maintaine current state

AXT_MOTION_ABSREL_MODE_DEF

Definition	Value	Explanation
POS_ABS_MODE	00h	Position drive absolute mode
POS_REL_MODE	01h	Position drive relative mode

AXT_MOTION_PROFILE_MODE_DEF

Definition	Value	Explanation
SYM_TRAPEZOIDE_MODE	00h	Symmetrical Trapezode
ASYM_TRAPEZOIDE_MODE	01h	Asymmetrical Trapezode
QUASI_S_CURVE_MODE	02h	Symmetrical Quasi-S Curve (not support in PCI-N404/804)
SYM_S_CURVE_MODE	03h	Symmetrical S Curve
ASYM_S_CURVE_MODE	04h	Asymmetrical S Curve

AXT_MOTION_SIGNAL_LEVEL_DEF

Definition	Value	Explanation
INACTIVE	00h	No-activation
ACTIVE	01h	Activation

AXT_MOTION_HOME_RESULT_DEF

Definition	Value	Explanation
HOME_SUCCESS	00h	If home search is exited successfully
HOME_SEARCHING	02h	If home search is in progress currently
HOME_ERR_GNT_RANGE	10h	If home search result of master and slave axis on gantry drive axis is off the set OffsetRange
HOME_ERR_USER_BREAK	11h	If user executes a stop command during home search
HOME_ERR_VELOCITY	12h	If home search velocity is not set
HOME_ERR_AMP_FAULT	13h	If servo pack alarm occurred during home search
HOME_ERR_NEG_LIMIT	14h	If (-) limit sensor is detected during home sensor search with (+) direction

HOME_ERR_POS_LIMIT	15h	If (+) limit sensor is detected during home sensor search with (−) direction
HOME_ERR_NOT_DETECT	16h	If home sensor is not detected
HOME_ERR_UNKNOWN	FFh	If tried home search from unknown channel (axis) number (starting from 0)

AXT_MOTION_UNIV_INPUT_DEF

Definition	Value	Explanation
UIO_INP0	00h	Universal input 0
UIO_INP1	01h	Universal input 1
UIO_INP2	02h	Universal input 2
UIO_INP3	03h	Universal input 3
UIO_INP4	04h	Universal input 4

AXT_MOTION_UNIV_OUTPUT_DEF

Definition	Value	Explanation
UIO_OUT0	00h	Universal output 0
UIO_OUT1	01h	Universal output 1
UIO_OUT2	02h	Universal output 2
UIO_OUT3	03h	Universal output 3
UIO_OUT4	04h	Universal output 4

AXT_MOTION_DETECT_DOWN_START_POINT_DEF : Assignment of deceleration start point search method

Definition	Value	Explanation
AutoDetect	00h	Automatic search method
RestPulse	01h	Remaining seardh method (manual acceleration and deceleration)

AXT_MOTION_PULSE_OUTPUT_DEF

Definition	Value	Explanation
OneHighLowHigh	00h	1 pulse method, PULSE(Active High), Forward direction (DIR=Low) / Reverse direction (DIR=High)
OneHighHighLow	01h	1 pulse method, PULSE(Active High), Forward direction (DIR=High) / Reverse direction (DIR=Low)
OneLowLowHigh	02h	1 pulse method, PULSE(Active Low), Forward direction (DIR=Low) / Reverse direction (DIR=High)
OneLowHighLow	03h	1 pulse method, PULSE(Active Low), Forward direction (DIR=High) / Reverse direction (DIR=Low)
TwoCcwCwHigh	04h	2 pulse method, PULSE(CCW: Reverse direction), DIR(CW:

		Forward direction), Active High
TwoCcwCwLow	05h	2 pulse method, PULSE(CCW: Reverse direction), DIR(CW: Forward diredtion), Active Low
TwoCwCcwHigh	06h	2 pulse method, PULSE(CW: Forward diredtion), DIR(CCW: Reverse direction), Active High
TwoCwCcwLow	07h	2 pulse method, PULSE(CW: Forward diredtion), DIR(CCW: Reverse direction), Active Low
TwoPhase	08h	2 phase(90' phase difference), PULSE lead DIR(CW: Forward diredtion), PULSE lag DIR(CCW: Reverse direction)
TwoPhaseReverse	09h	2 phase(90' phase difference), PULSE lead DIR(CCW: Forward diredtion), PULSE lag DIR(CW: Reverse direction)

AXT_MOTION_EXTERNAL_COUNTER_INPUT_DEF : Encoder Input Method

Definition	Value	Explanation
ObverseUpDownMode	00h	Forward direction Up / Down
ObverseSqr1Mode	01h	Forward direction 1 multiplication
ObverseSqr2Mode	02h	Forward direction 2 multiplication
ObverseSqr4Mode	03h	Forward direction 4 multiplication
ReverseUpDownMode	04h	Reverse direction Up / Down
ReverseSqr1Mode	05h	Reverse direction 1 multiplication
ReverseSqr2Mode	06h	Reverse direction 2 multiplication
ReverseSqr4Mode	07h	Reverse direction 4 multiplication

AXT_MOTION_ACC_UNIT_DEF

Definition	Value	Explanation
UNIT_SEC2	00h	unit/sec2
SEC	01h	sec

AXT_MOTION_MOVE_DIR_DEF

Definition	Value	Explanation
DIR_CCW	00h	Counterclockwise direction
DIR_CW	01h	Clockwise direction

AXT_MOTION_RADIUS_DISTANCE_DEF

Definition	Value	Explanation
SHORT_DISTANCE	00h	Short distance circular movement
LONG_DISTANCE	01h	Long distance circular movement

AXT_MOTION_INTERPOLATION_AXIS_DEF

Definition	Value	Explanation
INTERPOLATION_AXIS2	02h	When 2 axis is used for interpolation
INTERPOLATION_AXIS3	03h	When 3 axis is used for interpolation
INTERPOLATION_AXIS4	04h	When 4 axis is used for interpolation

AXT_MOTION_CONTISTART_NODE_DEF

Definition	Value	Explanation
CONTI_NODE_VELOCITY	00h	Velocity assigned interpolation mode
CONTI_NODE_MANUAL	01h	Node acceleration and deceleration interpolation mode
CONTI_NODE_AUTO	02h	Automatic acceleration and deceleration interpolation mode

AXT_MOTION_HOME_DETECT_SIGNAL_DEF

Definition	Value	Explanation
PosEndLimit	00h	+Elm(End limit) + direction limit sensor signal
NegEndLimit	01h	-Elm(End limit) - direction limit sensor signal
PosSloLimit	02h	+Slm(Decelerate limit) signal - Not-used
NegSloLimit	03h	-Slm(Decelerate limit) signal - Not-used
HomeSensor	04h	IN0(ORG) home sensor signal
EncodZPhase	05h	IN1(Z phase) Encoder Z phase signal
UniInput02	06h	IN2(universal) universal input number 2 signal
UniInput03	07h	IN3(universal) universal input number 3 signal

AXT_MOTION_IPDETECT_DESTINATION_SIGNAL

Definition	Value	Explanation
PElmNegativeEdge	00h	+Elm(End limit) falling edge
NElmNegativeEdge	01h	-Elm(End limit) falling edge
PSlmNegativeEdge	02h	+Slm(Decelerate limit) signal - Not-used
NSlmNegativeEdge	03h	-Slm(Decelerate limit) signal - Not-used
In0DownEdge	04h	IN0(ORG) falling edge
In1DownEdge	05h	IN1(Z phase) falling edge
In2DownEdge	06h	IN2(universal) falling edge
In3DownEdge	07h	IN3(universal) falling edge
PElmPositiveEdge	00h	+Elm(End limit) rising edge

NElmPositiveEdge	01h	-Elm(End limit) rising edge
PSlmPositiveEdge	02h	+Slm(Decelerate limit) signal - Not-used
NSlmPositiveEdge	03h	-Slm(Decelerate limit) signal - Not-used
In0UpEdge	04h	IN0(ORG) rising edge
In1UpEdge	05h	IN1(Z phase) rising edge
In2UpEdge	06h	IN2(universal) rising edge
In3UpEdge	07h	IN3(universal) rising edge

AXT_MOTION_QIDETECT_DESTINATION_SIGNAL

Definition	Value	Explanation
Signal_ PosEndLimit	00h	+Elm(End limit) + direction limit sensor signal
Signal_ NegEndLimit	01h	-Elm(End limit) - direction limit sensor signal
Signal_ PosSloLimit	02h	+Slm(Decelerate limit) signal - Not-used
Signal_ NegSloLimit	03h	-Slm(Decelerate limit) signal - Not-used
Signal_ HomeSensor	04h	IN0(ORG) home sensor signal
Signal_ EncodZPhase	05h	IN1(Z phase) Encoder Z phase signal
Signal_ UniInput02	06h	IN2(universal) universal input number 2 signal
Signal_ UniInput03	07h	IN3(universal) universal input number 3 signal

AXT_MOTION_MPG_INPUT_METHOD

Definition	Value	Explanation
MPG_DIFF_ONE_PHASE	00h	MPG input method One Phase
MPG_DIFF_TWO_PHASE_1X	01h	MPG input method TwoPhase1
MPG_DIFF_TWO_PHASE_2X	02h	MPG input method TwoPhase2
MPG_DIFF_TWO_PHASE_4X	03h	MPG input method TwoPhase4
MPG_LEVEL_ONE_PHASE	04h	MPG input method Level One Phase
MPG_LEVEL_TWO_PHASE_1X	05h	MPG input method Level Two Phase1
MPG_LEVEL_TWO_PHASE_2X	06h	MPG input method Level Two Phase2
MPG_LEVEL_TWO_PHASE_4X	07h	MPG input method Level Two Phase4

AXT_MOTION_SENSOR_INPUT_METHOD

Definition	Value	Explanation
SENSOR_METHOD1	00h	General drive
SENSOR_METHOD2	01h	Slow speed drive before sensor signal detection. General drive after signal detection
SENSOR_METHOD3	02h	Slow speed drive

AXT_MOTION_HOME_CRC_SELECT

Definition	Value	Explanation
CRC_SELECT1	00h	No use of position clear, no use of remaining pulse clear
CRC_SELECT2	01h	Use of position clear, no use of remaining pulse clear
CRC_SELECT3	02h	No use of position clear, use of remaining pulse clear
CRC_SELECT4	03h	Use of position clear, use of remaining pulse clear

AXT_MOTION_QIMECHANICAL_SIGNAL_DEF : Mechanical Signal Register (PCI-N804/404, PCI-R1604/R1604-MLII)

Definition	Value	Explanation
QIMECHANICAL_PELM_LEVEL	0001h	Bit 0, +Limit emergency stop signal current state
QIMECHANICAL_NELM_LEVEL	0002h	Bit 1, -Limit emergency stop signal current state
QIMECHANICAL_PSLM_LEVEL	0004h	Bit 2, +limit decelerate stop signal current state
QIMECHANICAL_NSLM_LEVEL	0008h	Bit 3, -limit decelerate stop signal current state
QIMECHANICAL_ALARM_LEVEL	0010h	Bit 4, Alarm signal current state
QIMECHANICAL_INP_LEVEL	0020h	Bit 5, InPos signal current state
QIMECHANICAL_ESTOP_LEVEL	0040h	Bit 6, emergency stop signal(ESTOP) current state
QIMECHANICAL_ORG_LEVEL	0080h	Bit 7, home signal current state
QIMECHANICAL_ZPHASE_LEVEL	0100h	Bit 8, Z phase input signal current state
QIMECHANICAL_ECUP_LEVEL	0200h	Bit 9, ECUP terminal signal state
QIMECHANICAL_ECDN_LEVEL	0400h	Bit 10, ECDN terminal signal state
QIMECHANICAL_EXPP_LEVEL	0800h	Bit 11, EXPP terminal signal state
QIMECHANICAL_EXMP_LEVEL	1000h	Bit 12, EXMP terminal signal state
QIMECHANICAL_SQSTR1_LEVEL	2000h	Bit 13, SQSTR1 terminal signal state
QIMECHANICAL_SQSTR2_LEVEL	4000h	Bit 14, SQSTR2 terminal signal state
QIMECHANICAL_SQSTP1_LEVEL	8000h	Bit 15, SQSTP1 terminal signal state
QIMECHANICAL_SQSTP2_LEVEL	10000h	Bit 16, SQSTP2 terminal signal state
QIMECHANICAL_MODE_LEVEL	20000h	Bit 17, MODE terminal signal state

AXT_MOTION_QIEND_STATUS_DEF : Exit state of axis (PCI-N804/404, PCI-R1604, PCI-R1604-MLII)

Definition	Value	Explanation
	00000000h	No drive or initial state
QIEND_STATUS_0	00000001h	Bit 0, exit by forward direction limit signal (PELM)
QIEND_STATUS_1	00000002h	Bit 1, exit by reverse direction limit signal (NELM)
QIEND_STATUS_2	00000004h	Bit 2, exit by forward direction additional limit signal (PSLM)

QIEND_STATUS_3	00000008h	Bit 3, exit by reverse direction additional limit signal (NSLM)
QIEND_STATUS_4	00000010h	Bit 4, exit by forward direction soft limit emergency stop function
QIEND_STATUS_5	00000020h	Bit 5, exit by reverse direction soft limit emergency stop function
QIEND_STATUS_6	00000040h	Bit 6, exit by forward direction soft limit decelerate stop function
QIEND_STATUS_7	00000080h	Bit 7, exit by reverse direction soft limit decelerate stop function
QIEND_STATUS_8	00000100h	Bit 8, drive exit by servo alarm function
QIEND_STATUS_9	00000200h	Bit 9, drive exit by emergency stop signal input
QIEND_STATUS_10	00000400h	Bit 10, drive exit by emergency stop command
QIEND_STATUS_11	00000800h	Bit 11, drive exit by decelerate stop command
QIEND_STATUS_12	00001000h	Bit 12, drive exit by entire axes emergency stop command
QIEND_STATUS_13	00002000h	Bit 13, drive exit by sync stop function #1(SQSTP1)
QIEND_STATUS_14	00004000h	Bit 14, drive exit by sync stop function #2(SQSTP2)
QIEND_STATUS_15	00008000h	Bit 15, encoder input (ECUP,ECDN) error occurrence
QIEND_STATUS_16	00010000h	Bit 16, MPG input (EXPP,EXMP) error occurrence
QIEND_STATUS_17	00020000h	Bit 17, exit with successful home search
QIEND_STATUS_18	00040000h	Bit 18, exit with successful signal search
QIEND_STATUS_19	00080000h	Bit 19, drive exit by interpolation data abnormality.
QIEND_STATUS_20	00100000h	Bit 20, abnormal drive stop occurrence.
QIEND_STATUS_21	00200000h	Bit 21, MPG function block pulse buffer overflow occurrence
QIEND_STATUS_22 – 27		Bit 22,– Bit27 DON'CARE
QIEND_STATUS_28	10000000h	Bit 28, current/last move drive direction
QIEND_STATUS_29	20000000h	Bit 29, in output of remaining pulse clear
QIEND_STATUS_30	40000000h	Bit 30, abnormal drive stop cause state
QIEND_STATUS_31	80000000h	Bit 31, interpolation drive data error state.

AXT_MOTION_QIDRIVE_STATUS_DEF : Drive state of axis

Definition	Value	Explanation
QIDRIVE_STATUS_0	00000001h	Bit 0, BUSY(in drive move)
QIDRIVE_STATUS_1	00000002h	Bit 1, DOWN(in deceleration)
QIDRIVE_STATUS_2	00000004h	Bit 2, CONST(in constant speed)
QIDRIVE_STATUS_3	00000008h	Bit 3, UP(in acceleration)
QIDRIVE_STATUS_4	00000010h	Bit 4, in move of continuous drive
QIDRIVE_STATUS_5	00000020h	Bit 5, in move of assigned distance drive
QIDRIVE_STATUS_6	00000040h	Bit 6, in move of MPG drive
QIDRIVE_STATUS_7	00000080h	Bit 7, in move of home search drive
QIDRIVE_STATUS_8	00000100h	Bit 8, in move of signal search drive

QIDRIVE_STATUS_9	00000200h	Bit 9, in move of interpolation drive
QIDRIVE_STATUS_10	00000400h	Bit 10, in move of Slave drive
QIDRIVE_STATUS_11	00000800h	Bit 11, current move drive direction (different indication information in interpolation drive)
QIDRIVE_STATUS_12	00001000h	Bit 12, in waiting for servo position completion after pulse output.
QIDRIVE_STATUS_13	00002000h	Bit 13, in move of straight line interpolation drive.
QIDRIVE_STATUS_14	00004000h	Bit 14, in move of circular interpolation drive.
QIDRIVE_STATUS_15	00008000h	Bit 15, in pulse output.
QIDRIVE_STATUS_16	00010000h	Bit 16, drive reserved data number(start)(0-7)
QIDRIVE_STATUS_17	00020000h	Bit 17, drive reserved data number (middle)(0-7)
QIDRIVE_STATUS_18	00040000h	Bit 18, drive reserved data number (end)(0-7)
QIDRIVE_STATUS_19	00080000h	Bit 19, drive reversed Queue is cleared
QIDRIVE_STATUS_20	00100000h	Bit 20, drive reversed Queue is full
QIDRIVE_STATUS_21	00200000h	Bit 21, velocity mode of current move drive (start)
QIDRIVE_STATUS_22	00400000h	Bit 22, velocity mode of current move drive (end)
QIDRIVE_STATUS_23	00800000h	Bit 23, MPG buffer #1 Full
QIDRIVE_STATUS_24	01000000h	Bit 24, MPG buffer #2 Full
QIDRIVE_STATUS_25	02000000h	Bit 25, MPG buffer #3 Full
QIDRIVE_STATUS_26	04000000h	Bit 26, MPG buffer data OverFlow

AXT_MOTION_QIINTERRUPT_BANK1_DEF

Definition	Value	Explanation
QIINTBANK1_DISABLE	00000000h	INTERRUT DISABLED
QIINTBANK1_0	00000001h	Bit 0, when drive set interrupt occurrence use is exit
QIINTBANK1_1	00000002h	Bit 1, when drive is exited
QIINTBANK1_2	00000004h	Bit 2, when drive is started
QIINTBANK1_3	00000008h	Bit 3, counter #1 < comparator #1 event occurrence
QIINTBANK1_4	00000010h	Bit 4, counter #1 = comparator #1 event occurrence
QIINTBANK1_5	00000020h	Bit 5, counter #1 > comparator #1 event occurrence
QIINTBANK1_6	00000040h	Bit 6, counter #2 < comparator #2 event occurrence
QIINTBANK1_7	00000080h	Bit 7, counter #2 = comparator #2 event occurrence
QIINTBANK1_8	00000100h	Bit 8, counter #2 > comparator #2 event occurrence
QIINTBANK1_9	00000200h	Bit 9, counter #3 < comparator #3 event occurrence
QIINTBANK1_10	00000400h	Bit 10, counter #3 = comparator #3 event occurrence
QIINTBANK1_11	00000800h	Bit 11, counter #3 > comparator #3 event occurrence
QIINTBANK1_12	00001000h	Bit 12, counter #4 < comparator #4 event occurrence

QIINTBANK1_13	00002000h	Bit 13, counter #4 = comparator #4 event occurrence
QIINTBANK1_14	00004000h	Bit 14, counter #4 < comparator #4 event occurrence
QIINTBANK1_15	00008000h	Bit 15, counter #5 < comparator #5 event occurrence
QIINTBANK1_16	00010000h	Bit 16, counter #5 = comparator #5 event occurrence
QIINTBANK1_17	00020000h	Bit 17, counter #5 > comparator #5 event occurrence
QIINTBANK1_18	00040000h	Bit 18, timer #1 event occurrence
QIINTBANK1_19	00080000h	Bit 19, timer #2 event occurrence.
QIINTBANK1_20	00100000h	Bit 20, drive reservation setting Queue is cleared.
QIINTBANK1_21	00200000h	Bit 21, drive reservation setting Queue is full
QIINTBANK1_22	00400000h	Bit 22, trigger occurring distance period/absolute position Queue is cleared.
QIINTBANK1_23	00800000h	Bit 23, trigger occurring distance period/absolute position Queue is full
QIINTBANK1_24	01000000h	Bit 24, trigger signal occurrence event
QIINTBANK1_25	02000000h	Bit 25, script #1 command reservation setting Queue is cleared.
QIINTBANK1_26	04000000h	Bit 26, script #2 command reservation setting Queue is cleared.
QIINTBANK1_27	08000000h	Bit 27, script #3 initialized with execution of command reservation setting register.
QIINTBANK1_28	10000000h	Bit 28, script #4 initialized with execution of command reservation setting register.
QIINTBANK1_29	20000000h	Bit 29, servo alarm signal is permitted
QIINTBANK1_30	40000000h	Bit 30, |CNT1| – |CNT2| >= |CNT4| event occurrence.
QIINTBANK1_31	80000000h	Bit 31, interrupt occurrence command |INTGEN| execution.

AXT_MOTION_QIINTERRUPT_BANK2_DEF

Definition	Value	Explanation
QIINTBANK2_DISABLE	00000000h	INTERRUT DISABLED
QIINTBANK2_0	00000001h	Bit 0, script #1 reading command result Queue is full.
QIINTBANK2_1	00000002h	Bit 1, script #2 reading command result Queue is full.
QIINTBANK2_2	00000004h	Bit 2, script #3 reading command result register is renewed with new data
QIINTBANK2_3	00000008h	Bit 3, script #4 reading command result register is renewed with new data
QIINTBANK2_4	00000010h	Bit 4, when reservation command of script #1 is executed, command set by interrupt occurrence is executed
QIINTBANK2_5	00000020h	Bit 5, when reservation command of script #2 is executed, command set by interrupt occurrence is executed.
QIINTBANK2_6	00000040h	Bit 6, when reservation command of script #3 is executed, command set by interrupt occurrence is executed.

QIINTBANK2_7	00000080h	Bit 7, when reservation command of script #4 is executed, command set by interrupt occurrence is executed.
QIINTBANK2_8	00000100h	Bit 8, start drive
QIINTBANK2_9	00000200h	Bit 9, drive using servo position determination completion(InPos) function, exit condition occurrence.
QIINTBANK2_10	00000400h	Bit 10, event selection #1 condition occurrence to use during event counter operation..
QIINTBANK2_11	00000800h	Bit 11, event selection #2 condition occurrence to use during event counter operation.
QIINTBANK2_12	00001000h	Bit 12, SQSTR1 signal is permitted.
QIINTBANK2_13	00002000h	Bit 13, SQSTR2 signal is permitted.
QIINTBANK2_14	00004000h	Bit 14, UIO0 terminal signal is changed to '1'.
QIINTBANK2_15	00008000h	Bit 15, UIO1 terminal signal is changed to '1'.
QIINTBANK2_16	00010000h	Bit 16, UIO2 terminal signal is changed to '1'.
QIINTBANK2_17	00020000h	Bit 17, UIO3 terminal signal is changed to '1'.
QIINTBANK2_18	00040000h	Bit 18, UIO4 terminal signal is changed to '1'.
QIINTBANK2_19	00080000h	Bit 19, UIO5 terminal signal is changed to '1'.
QIINTBANK2_20	00100000h	Bit 20, UIO6 terminal signal is changed to '1'.
QIINTBANK2_21	00200000h	Bit 21, UIO7 terminal signal is changed to '1'.
QIINTBANK2_22	00400000h	Bit 22, UIO8 terminal signal is changed to '1'.
QIINTBANK2_23	00800000h	Bit 23, UIO9 terminal signal is changed to '1'.
QIINTBANK2_24	01000000h	Bit 24, UIO10 terminal signal is changed to '1'.
QIINTBANK2_25	02000000h	Bit 25, UIO11 terminal signal is changed to '1'.
QIINTBANK2_26	04000000h	Bit 26, error stop condition (LMT, ESTOP, STOP, ESTOP, CMD, ALARM) occurrence.
QIINTBANK2_27	08000000h	Bit 27, data setting error is occurred during interpolation.
QIINTBANK2_28	10000000h	Bit 28, Don't Care
QIINTBANK2_29	20000000h	Bit 29, limit signal (PELM, NELM) is inputted.
QIINTBANK2_30	40000000h	Bit 30, additional limit signal (PSLM, NSLM) is inputted.
QIINTBANK2_31	80000000h	Bit 31, emengency stop signal (ESTOP) is input.

Motion Command Function List

Board And Module Initialization

In this unit, APIs related to load or initialization required to initialize library will be introduced.

Function	Description
AxmInfoGetAxis	Returns board number, module position and module ID of corresponding axis.
AxmInfoIsMotionModule	Returns whether motion module exists.
AxmInfoIsInvalidAxisNo	Returns whether corresponding axis is effective.
AxmInfoGetAxisCount	Returns effective motion axis number installed in system.
AxmInfoGetFirstAxisNo	Returns the first axis number of corresponding board and module.

AxmInfoGetAxis

Purpose

To verify board number, module position and module ID of a specific axis.

Format

C++

```
DWORD AxmInfoGetAxis(long lAxisNo, long *lpBoardNo, long *lpModulePos,
DWORD *upModuleID);
```

Visual Basic

```
Function AxmInfoGetAxis(ByVal lAxisNo As Long, ByRef lpBoardNo As
Long, ByRef lpModulePos As Long, ByRef upModuleID As Long) As Long
```

Delphi

```
function AxmInfoGetAxis(lAxisNo : LongInt; lpBoardNo : PLongInt;
lpModulePos : PLongInt; upModuleID : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(start from 0)
[<<]lpBoardNo	Base board position value included setting axis
[<<]lpModulePos	Module position value included setting axis
[<<]upModuleID	Module ID value included setting axis :See AXHS.h (AXT_FUNC_MODULE) – [06h] PCI-N804/404, 4 Axis – [07h] RTEX A4N slave, 1 Axis – [09h] RTEX PM slave, 1 Axis – [0Ah] Mechatrolink-II SGDV slave, 1 Axis – [0Bh] Mechatrolink-II JEPMC-PL2910, 2 Axis

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Note : When only required data are needed to be called, it can be used with input of NULL on the unrequired variable.

Since board and module ID value are identical to product model number, it can be used to verify actual board and module information.

Example

C++

```
// Return board number, module position and module ID on axis 0.
Long BoardNo, ModulePos;
DWORD ModuleID;
AxmInfoGetAxis(0, &BoardNo, &ModulePos, &ModuleID);
```

Visual Basic

```
' Return board number, module position and module ID on axis 0.
Dim BoardNo , ModulePos, ModuleID As Long
AxmInfoGetAxis 0, BoardNo, ModulePos, ModuleID
```

Delphi

```
{ Return board number, module position and module ID on axis 0. }
var
  ModuleID: DWord;
  BoardNo, ModulePos: LongInt;

begin
  AxmInfoGetAxis(0, @BoardNo, @ModulePos, @ModuleID);
end;
```

See Also

AxmInfoIsMotionModule, AxmInfoIsInvalidAxisNo, AxmInfoGetAxisCount, AxmInfoGetFirstAxisNo

AxmInfoIsMotionModule

Purpose

To return if motion module is installed.

Format

C++

```
DWORD AxmInfoIsMotionModule(DWORD *upStatus);
```

Visual Basic

```
Function AxmInfoIsMotionModule(ByRef upStatus As Long) As Long
```

Delphi

```
function AxmInfoIsMotionModule(upStatus : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]upStatus	Value of whether motion module is installed or not : FALSE & TRUE – [00h] Motion module is not installed – [01h] Motion module is installed

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
* See error code Table for more information on status error codes

Description

Whether motion module is installed or not returns to *upStatus value.

Example

C++

```
// Verify whether motion module is installed.
DWORD Status;
AxmInfoIsMotionModule(&Status);
```

Visual Basic

```
' Verify whether motion module is installed.
Dim Status As Long
AxmInfoIsMotionModule Status
```

Delphi

```
{ Verify whether motion module is installed. }
var
  Status: DWord;

begin
  AxmInfoIsMotionModule(@Status);
```

```
end;
```

See Also

AxmInfoGetAxis, AxmInfoIsInvalidAxisNo, AxmInfoGetAxisCount, AxmInfoGetFirstAxisNo

AxmInfoIsInvalidAxisNo

Purpose

To return if corresponding axis is available to use.

Format

C++

```
DWORD AxmInfoIsInvalidAxisNo(long lAxisNo);
```

Visual Basic

```
Function AxmInfoIsInvalidAxisNo(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmInfoIsInvalidAxisNo(lAxisNo : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Axis number to test if available to use

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Verify if corresponding axis can be used. If not, generate error code.

Example

C++

```
// Verify whether it can be used.
DWORD   dwReturn;
dwReturn = AxmInfoIsInvalidAxisNo(9);
```

Visual Basic

```
' Verify whether it can be used.
Dim dwReturn As Long
dwReturn = AxmInfoIsInvalidAxisNo 9
```

Delphi

```
{ Verify whether it can be used. }
var
  dwReturn: Dword;

begin
  dwReturn := AxmInfoIsInvalidAxisNo(9);
end;
```

See Also

AxmInfoGetAxis, AxmInfoIsMotionModule, AxmInfoGetAxisCount, AxmInfoGetFirstAxisNo

AxmInfoGetAxisCount

Purpose

To return total axis number of installed board.

Format

C++

```
DWORD AxmInfoGetAxisCount(long *lpAxisCount);
```

Visual Basic

```
Function AxmInfoGetAxisCount(ByRef lpAxisCount As Long) As Long
```

Delphi

```
function AxmInfoGetAxisCount(lpAxisCount: PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]lpAxisCount	Total axis number of installed board

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[4051] AXT_RT_MOTION_NOT_MODULE : No motion module is installed in the system
* See error code Table for more information on status error codes

Description

Axis number of installed board returns to *lpAxisCount value.

Example

C++

```
// Verify axis number of installed motion module.
long    AxisCount;
AxmInfoGetAxisCount(&AxisCount);
```

Visual Basic

```
' Verify axis number of installed motion module.
Dim AxisCount As Long
AxmInfoGetAxisCount AxisCount
```

Delphi

```
{ Verify axis number of installed motion module. }
var
  AxisCount: LongInt;

begin
  AxmInfoGetAxisCount(@AxisCount);
```

```
end;
```

See Also

AxmInfoGetAxis, AxmInfoIsMotionModule, AxmInfoIsInvalidAxisNo, AxmInfoGetFirstAxisNo

AxmInfoGetFirstAxisNo

Purpose

To return the earliest axis number in a specific position module of corresponding base board.

Format

C++

```
DWORD AxmInfoGetFirstAxisNo(long lBoardNo, long lModulePos, long
*lpAxisNo);
```

Visual Basic

```
Function AxmInfoGetFirstAxisNo(ByVal lBoardNo As Long, ByVal
lModulePos As Long, ByRef lpAxisNo As Long) As Long
```

Delphi

```
function AxmInfoGetFirstAxisNo(lBoardNo: LongInt; lModulePos: LongInt;
lpAxisNo: PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lBoardNo	Setting base board number. Number is assigned in order of slot from 0.
[>>]lModulePos	Module number located within base board. In case of BPFR, module number from 0 to 3 are assigned, and in case of BPHR base board, module number from 0 to 1 are assigned.
[<<]lpAxisNo	Axis number value of corresponding board/module

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[1101] AXT_RT_INVALID_BOARD_NO : Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS : Invalid module position
[4051] AXT_RT_MOTION_NOT_MODULE : No motion module is installed in the system
* See error code Table for more information on status error codes

Description

Since the case of N804, N404 CAMC-QI is not the module type of board itself, return the first starting axis number on board.

Base board number is assigned in order of slot from 0. In case of BPFR, module number from 0 to 3 are assigned, and in case of BPHR base board, module number from 0 to 1 are assigned. Since motion module has 2 axes, *lpAxisNo value is to be the earliest axis number.

1. Numbering by same module is done individually.

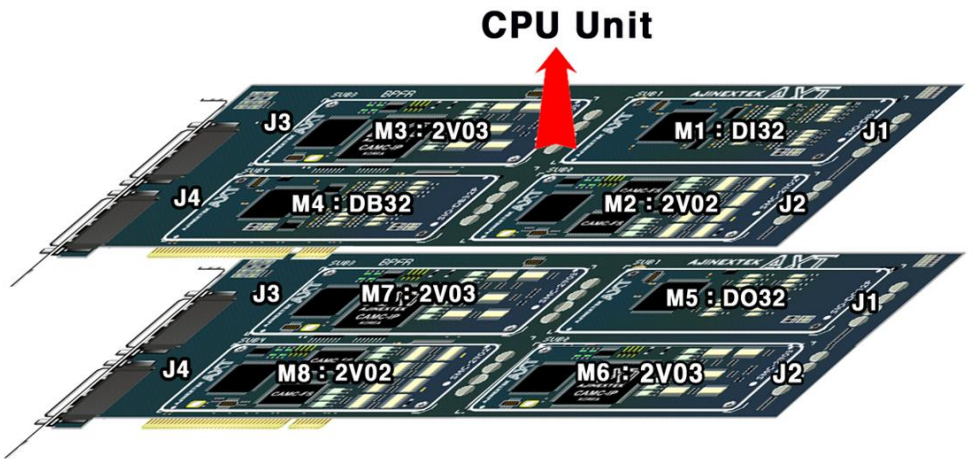
AXL provides only version of N404, N804, and 2V03 module above.

2. If there are various base boards, numbering is done from the close base board to CPU.

In general PC or IPC system, order of PCI slot is set from the close one to CPU. However, based on system setting, there are systems that start number from a distant one on CPU module, or that set each slot number arbitrary.

3. Numbering is done in order of J1 ,J2 ,J3 ,and J4 on the same base board.

Assignment of axis number and channel number.







Motion axis number connected to 2V03 module

Axis Number	0	1	2	3	4	5
Corresponding Module	M3		M6		M7	

Motion axis number connected to 2V03 module

Channel Number	00	01	02	03	···	29	30	31	32	33	34	···	45	46	47
Corresponding Module	M1						M4								

Digital Output channel number

Channel Number	0	0	···	1	1	1	1	1	1	2	···	4	4	4	4
Corresponding Module	M4							M5							

Channel Number of AIO module is determined in the same type with DIO.

Example

C++

```
// Verify the earliest axis number on a specific 0 module.
long    AxisNo;
AxmInfoGetFirstAxisNo(0, 0, &AxisNo);
```

Visual Basic

```
' Verify the earliest axis number on a specific 0 module.
Dim AxisNo As Long
AxmInfoGetFirstAxisNo 0, 0, AxisNo
```

Delphi

```
{ Verify the earliest axis number on a specific 0 module. }
var
  AxisNo: LongInt;

begin
  AxmInfoGetFirstAxisNo(0, 0, @AxisNo);
end;
```

See Also

AxmInfoGetAxis, AxmInfoIsMotionModule, AxmInfoIsInvalidAxisNo, AxmInfoGetAxisCount

Virtual Axis API

In this unit, when drive command is given from library, APIs that are to give command using arbitrary axis number not an actual corresponding axis number, will be introduced. For example, when given a command on axis 0, give a command as axis 2 during command. Let us suppose there are axes from 0 to 10 in the system, and axes 3, 4 in the middle are removed somehow. In this case, axis number will be run moved up from 0 to 8 in sequence. Therefore, axis numbers on the program have to be changed one by one, and in this case, it can be solved by mapping with original axis number without toughing program source. In this case, axes 3 to 8 just need to change to axes 5 to 10.

Function	Description
AxmVirtualSetAxisNoMap	Sets virtual axis.
AxmVirtualGetAxisNoMap	Returns virtual channel (axis) number which is set.
AxmVirtualSetMultiAxisNoMap	Sets multi-virtual axis.
AxmVirtualGetMultiAxisNoMap	Returns multi-virtual channel (axis) which is set.
AxmVirtualResetAxisMap	Clears virtual axis setting.

AxmVirtualSetAxisNoMap

Purpose

API that executes mapped actual axis number and virtual axis number.

Format

C++

```
DWORD AxmVirtualSetAxisNoMap(long lRealAxisNo, long lVirtualAxisNo);
```

Visual Basic

```
Function AxmVirtualSetAxisNoMap(ByVal lRealAxisNo As Long, ByVal
lVirtualAxisNo As Long) As Long
```

Delphi

```
function AxmVirtualSetAxisNoMap(lRealAxisNo: LongInt; lVirtualAxisNo:
LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lRealAxisNo	Actual axis number installed in system
[>>]lVirtualAxisNo	Virtual axis number for mapping

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
* See error code Table for more information on status error codes

Description

This function maps duplication of “actual” axes numbers into an array of virtual axes numbers.

After initialization, axes numbers in the range of 0 ~ (number of axis installed in the system – 1) are valid for all AXM functions, but arbitrary axes numbers can be used instead of actual installed axes numbers by using this function.

This function is made for the purpose of keeping the axes numbers assigned to the exsiting program unchanged and changing physical position of the actual actual control axes when changing the control system hardware.

Note: No more than one axis should be mapped to one virtual axis number. When there is an overlap in mapping, only the axis with lowest axis number can be controlled by the mapped virtual axis number, and other axes mapped to the same virtual axis cannot be controlled.

Example

C++

```
// Set actual axis 0 to virtual axis 5.
AxmVirtualSetAxisNoMap(0, 5);
```

```
// Verify setting virtual axis.
long    VirtualAxisNo;
AxmVirtualGetAxisNoMap(0, &VirtualAxisNo);
```

Visual Basic

```
' Set actual axis 0 to virtual axis 5.
AxmVirtualSetAxisNoMap 0, 5

' Verify setting virtual axis.
Dim VirtualAxisNo As Long
AxmVirtualGetAxisNoMap 0, VirtualAxisNo
```

Delphi

```
{ Verify setting virtual axis. }
var
  VirtualAxisNo: LongInt;

begin
  { Set actual axis 0 to virtual axis 5. }
  AxmVirtualSetAxisNoMap(0, 5);
  AxmVirtualGetAxisNoMap(0, @VirtualAxisNo);
end;
```

See Also

AxmVirtualResetAxisMap, AxmVirtualGetAxisNoMap, AxmVirtualSetMultiAxisNoMap, AxmVirtualGetMultiAxisNoMap

AxmVirtualGetAxisNoMap

Purpose

API that verifies actual axis number and mapped virtual axis number.

Format

C++

```
DWORD AxmVirtualGetAxisNoMap(long lRealAxisNo, long *lpVirtualAxisNo);
```

Visual Basic

```
Function AxmVirtualGetAxisNoMap(ByVal lRealAxisNo As Long, ByRef
lpVirtualAxisNo As Long) As Long
```

Delphi

```
function AxmVirtualGetAxisNoMap(lRealAxisNo: LongInt; lpVirtualAxisNo:
PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lRealAxisNo	Actual axis number installed in system
[<<]lpVirtualAxisNo	Virtual axis number for mapping

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

This function is to check the virtual axis number of the corresponding actual axis number mapped by the AxmVirtualSetAxisNoMap function. The return value will be -1 if the axis it not mapped, and the mapped virtual axis number will be returned when the axis is mapped.

Example

C++

```
// Set actual axis 0 to virtual axis 5.
AxmVirtualSetAxisNoMap(0, 5);

// Verify setting virtual axis.
long    VirtualAxisNo;
AxmVirtualGetAxisNoMap(0, &VirtualAxisNo);
```

Visual Basic

```
' Set actual axis 0 to virtual axis 5.
AxmVirtualSetAxisNoMap 0, 5
```

```
' Verify setting virtual axis.
Dim VirtualAxisNo As Long
AxmVirtualGetAxisNoMap 0, VirtualAxisNo
```

Delphi

```
{ Verify setting virtual axis. }
var
  VirtualAxisNo: LongInt;

begin
  { Set actual axis 0 to virtual axis 5. }
  AxmVirtualSetAxisNoMap(0, 5);
  AxmVirtualGetAxisNoMap(0, @VirtualAxisNo);
end;
```

See Also

AxmVirtualResetAxisMap, AxmVirtualSetAxisNoMap, AxmVirtualSetAxisNoMap, AxmVirtualSetMultiAxisNoMap, AxmVirtualGetMultiAxisNoMap

AxmVirtualSetMultiAxisNoMap

Purpose

API that executes mapping more than one axis of actual axis number and mapped virtual axis number.

Format

C++

```
DWORD AxmVirtualSetMultiAxisNoMap(long lSize, long *lpRealAxesNo,
long *lpVirtualAxesNo);
```

Visual Basic

```
Function AxmVirtualSetMultiAxisNoMap(ByVal lSize As Long, ByRef
lpRealAxesNo As Long, ByRef lpVirtualAxesNo As Long) As Long
```

Delphi

```
function AxmVirtualSetMultiAxisNoMap(lSize: LongInt; lpRealAxesNo:
PLongInt; lpVirtualAxesNo: PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lRealAxisNo	Axis number to set
[>>]lpRealAxesNo	Actual axis number array installed in system
[>>]lpVirtualAxesNo	Virtual axis number array for mapping

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

This function maps actual axes numbers into an array of virtual axes numbers.

After initialization, axes numbers in the range of 0 ~ (number of axis installed in the system - 1) are valid for all AXM functions, but arbitrary axes numbers can be used instead of actual installed axes numbers by using this function.

This function is made for the purpose of keeping the axes numbers assigned to the exsiting program unchanged and changing physical position of the actual actual control axes when changing the control system hardware.

Note: No more than one axis should be mapped to one virtual axis number. When there is an overlap in mapping, only the axis with lowest axis number can be controlled by the mapped virtual axis number, and other axes mapped to the same virtual axis cannot be controlled.

Example

C++

```
// Set virtual axis.
long Realaxis[4],Virtualaxis[4];
Realaxis[0] = 0;
Realaxis[1] = 1;
Realaxis[2] = 2;
Realaxis[3] = 3;

Virtualaxis[0] = 4;
Virtualaxis[1] = 5;
Virtualaxis[2] = 6;
Virtualaxis[3] = 7;
// Set axis 0 to axis 4, axis 1 to axis 5, axis 2 to axis 6, and axis 3 to axis 7.
AxmVirtualSetMultiAxisNoMap(4,Realaxis,Virtualaxis);

// Verify virtual axis setting.
long lReadRealAxisNo[4];
AxmVirtualGetMultiAxisNoMap(4, Realaxis, lReadRealAxisNo);
```

Visual Basic

```
' Set virtual axis.
Dim Realaxis(0 To 3) As Long
Dim Virtualaxis(0 To 3) As Long
Realaxis(0) = 0
Realaxis(1) = 1
Realaxis(2) = 2
Realaxis(3) = 3
Virtualaxis(0) = 4
Virtualaxis(1) = 5
Virtualaxis(2) = 6
Virtualaxis(3) = 7

AxmVirtualSetMultiAxisNoMap 4, Realaxis(0), Virtualaxis(0)

' Verify virtual axis setting.
Dim RealAxisNo (0 To 3) As Long
AxmVirtualGetMultiAxisNoMap 4, VirtualAxisNo(0), RealAxisNo(0)
```

Delphi

```
{ Set virtual axis. }
var
  Realaxis: array [0..3] of Longint;
  Virtualaxis: array [0..3] of Longint;
  RealAxisNo: array [0..3] of Longint;

begin
  Realaxis[0] := 0;
  Realaxis[1] := 1;
  Realaxis[2] := 2;
  Realaxis[3] := 3;
```

```
  Virtualaxis[0] := 4;
  Virtualaxis[1] := 5;
  Virtualaxis[2] := 6;
  Virtualaxis[3] := 7;

  AxmVirtualSetMultiAxisNoMap(4, @Realaxis, @Virtualaxis);
  { Verify virtual axis setting. }
  AxmVirtualGetMultiAxisNoMap(4, @Realaxis, @RealAxisNo);
end;
```

See Also

AxmVirtualResetAxisMap, AxmVirtualSetAxisNoMap, AxmVirtualGetAxisNoMap, AxmVirtualGetMultiAxisNoMap

AxmVirtualGetMultiAxisNoMap

Purpose

API that verifies more than one axis of actual axis number and mapped virtual axis number.

Format

C++

```
DWORD AxmVirtualGetMultiAxisNoMap(long lSize, long *lpRealAxesNo,
long *lpVirtualAxesNo);
```

Visual Basic

```
Function AxmVirtualGetMultiAxisNoMap(ByVal lSize As Long, ByRef
lpRealAxesNo As Long, ByRef lpVirtualAxesNo As Long) As Long
```

Delphi

```
function AxmVirtualGetMultiAxisNoMap(lSize: LongInt; lpRealAxesNo:
PLongInt; lpVirtualAxesNo: PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Axis number to set
[>>]lpRealAxesNo	Actual axis number array installed in system
[<<]lpVirtualAxisNo	Virtual axis number array for mapping

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Verify various virtual axis number actually mapped using AxmVirtualSetMultiAxisNoMap.

Example

C++

```
// Set virtual axis.
long Realaxis[4],Virtualaxis[4];
Realaxis[0] = 0;
Realaxis[1] = 1;
Realaxis[2] = 2;
Realaxis[3] = 3;

Virtualaxis[0] = 4;
Virtualaxis[1] = 5;
Virtualaxis[2] = 6;
```

```
Virtualaxis[3] = 7;
// Set axis 0 to axis 4, axis 1 to axis 5, axis 2 to axis 6, and axis 3 to axis 7.
AxmVirtualSetMultiAxisNoMap(4,Realaxis,Virtualaxis);

// Verify virtual axis setting.
long lReadRealAxisNo[4];
AxmVirtualGetMultiAxisNoMap(4, Realaxis, lReadRealAxisNo);
```

Visual Basic

```
' Set virtual axis.
Dim Realaxis(0 To 3) As Long
Dim Virtualaxis(0 To 3) As Long

Realaxis(0) = 0
Realaxis(1) = 1
Realaxis(2) = 2
Realaxis(3) = 3

Virtualaxis(0) = 4
Virtualaxis(1) = 5
Virtualaxis(2) = 6
Virtualaxis(3) = 7

AxmVirtualSetMultiAxisNoMap 4, Realaxis(0), Virtualaxis(0)

' Verify virtual axis setting.
Dim RealAxisNo (0 To 3) As Long
AxmVirtualGetMultiAxisNoMap 4, Realaxis(0), RealAxisNo(0)
```

Delphi

```
{ Set virtual axis. }
var
  Realaxis: array [0..3] of Longint;
  Virtualaxis: array [0..3] of Longint;
  RealAxisNo: array [0..3] of Longint;

begin
  Realaxis[0]  := 0;
  Realaxis[1]  := 1;
  Realaxis[2]  := 2;
  Realaxis[3]  := 3;

  Virtualaxis[0] := 4;
  Virtualaxis[1] := 5;
  Virtualaxis[2] := 6;
  Virtualaxis[3] := 7;

  AxmVirtualSetMultiAxisNoMap(4, @Realaxis, @Virtualaxis);
  { Verify virtual axis setting. }
```

```
  AxmVirtualGetMultiAxisNoMap(4, @Realaxis, @RealAxisNo);
end;
```

See Also

AxmVirtualResetAxisMap, AxmVirtualSetAxisNoMap, AxmVirtualGetAxisNoMap, AxmVirtualSetMultiAxisNoMap

AxmVirtualResetAxisMap

Purpose

Clear virtual axis setting that is currently set.

Format

C++

```
DWORD AxmVirtualResetAxisMap();
```

Visual Basic

```
Function AxmVirtualResetAxiMap() As Long
```

Delphi

```
function AxmVirtualResetAxisMap() : DWord; stdcall;
```

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

If virtual axis setting is cleared, axis numbers used in API are to be same with axis number installed in actual system.

Example

C++

```
// Clear virtual axis setting.
AxmVirtualResetAxisMap();
```

Visual Basic

```
' Clear virtual axis setting.
AxmVirtualResetAxisMap
```

Delphi

```
{ Clear virtual axis setting. }
begin
  AxmVirtualResetAxisMap();
end;
```

See Also

AxmVirtualSetAxisNoMap, AxmVirtualGetAxisNoMap,
AxmVirtualSetMultiAxisNoMap, AxmVirtualGetMultiAxisNoMap

Interrupt API

In this unit, interrupt related APIs are introduced. Interrupt is to inform to user when a specific condition occurs. Unlike falling method, interrupt has an advantage that does not generate load on CPU. In the environment of window, interrupt is not managed at general Application level, and it informs by event.

For example, if a user wants to use interrupt for comparison function that compares deviation of Command and Actual counter, user can make to generate interrupt when greater deviation occurred that assigned value after deviation limit assignment if user sets interrupt. AXM can generate interrupt using various events as interrupt source occurring during motion drive, and if register of interrupt BANK composed of 31-bit generates each interrupt, it becomes TRUE. When interrupt occurs, to manage this, AXM provides 3 methods; callback API, window message, and event method. Especially, since event method uses a specified thread watching if interrupt occurred or uses while sentence, it takes more system resources, but has an advantage that manages interrupt most fast.

Note) Interrupt functions are only available in the PCI-N804/404.

Function	Description
AxlInterruptEnable	In order to use the interrupt, other interrupt functions are to be used after calling this function.
AxlInterruptDisable	This function is used to disable the interrupt.
AxmInterruptSetAxis	Uses window message or callback API to receive interrupt message.
AxmInterruptSetAxisEnable	Read internal flag variable which is set in ReadInterruptFlag.
AxmInterruptGetAxisEnable	Sets whether interrupt is used of motion module on corresponding axis.
AxmInterruptRead	Returns whether interrupt is used of motion module on corresponding axis.
AxmInterruptReadAxisFlag	Returns interrupt flag value on corresponding axis.
AxmInterruptSetUserEnable	Sets whether interrupt set by user of a specific axis occurred.
AxmInterruptGetUserEnable	Verifies whether interrupt set by user of a specific axis occurred.

AxlInterruptEnable

Purpose

To enable the on-board interrup.

Format

C++

```
DWORD AxlInterruptEnable();
```

Visual Basic

```
Function AxlInterruptEnable() As Long
```

Delphi

```
function AxlInterruptEnable() : DWord; stdcall;
```

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully

* See error code Table for more information on status error codes

Description

This function must be called in order to use the on-board interrupt.

Example

C++

```
// Use window message to bring interrupt message.
BEGIN_MESSAGE_MAP(CCAMCIPDlg, CDialog)
   //}}AFX_MSG_MAP
   // Declaration
   ON_MESSAGE(WM_CAMCIP_INTERRUPT,OnTriggerInterrupt)
END_MESSAGE_MAP()

AxlInterruptEnable();
// When interrupt occurs, 'OnTriggerInterrupt()' is called.
AxmInterruptSetAxis(0, m_hWnd, NULL, NULL);
```

Visual Basic

```
' Use window message to bring interrupt message.
' Declaration
Private Sub AxtMsg1_OnMessage1(ByVal wParam As Long, ByVal lParam As Long)
' Interrupt message process script
End Sub

' 'AxtMsg1_OnMessage1()' function will be called When interrupt occurs
AxtMsg1.Message1 = WM_CAMCIP_INTERRUPT

AxlInterruptEnable
' 'AxtMsg1_OnMessage1()' function will be called When interrupt occurs
```

```
AxmInterruptSetAxis 0, Me.hWnd, WM_CAMCIP_INTERRUPT, 0
```

Delphi

```
{ Use window message to bring interrupt message. }
{ Declaration }
procedure TForm1.OnInterruptMessage(var Msg : TMessage);

begin
   { Interrupt message process script. }
   { 'AxtMsg1_OnMessage1()' function will be called When interrupt occurs. }
   AxlInterruptEnable();
   AxmInterruptSetAxis(Form1.Handle, WM_CAMCIP_INTERRUPT, nil);
end;
```

See Also

AxmInterruptSetAxisEnable, AxmInterruptGetAxisEnable, AxmInterruptRead, AxmInterruptReadAxisFlag

AxlInterruptDisable

Purpose

To disable the on-board interrup.

Format

C++

```
DWORD AxlInterruptDisable();
```

Visual Basic

```
Function AxlInterruptDisable() As Long
```

Delphi

```
function AxlInterruptDisable() : DWord; stdcall;
```

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
* See error code Table for more information on status error codes

Description

Call this function to disable the interrupt if the AxlInterruptEnable function was used to enable the interrupt.

Example

C++

```
// Disable after interrupt use.
AxlInterruptDisable();
```

Visual Basic

```
' Disable after interrupt use.
AxlInterruptDisable
```

Delphi

```
{ Disable after interrupt use.}
AxlInterruptDisable();
```

See Also

AxmInterruptSetAxisEnable, AxmInterruptGetAxisEnable, AxmInterruptRead, AxmInterruptReadAxisFlag

AxmInterruptSetAxis

Purpose

To use window message or callback API to bring interrupt message.

Format

C++

```
DWORD AxmInterruptSetAxis(long lAxisNo, HWND hWnd, DWORD uMessage,
AXT_INTERRUPT_PROC pProc, HANDLE *pEvent);
```

Visual Basic

```
Function AxmInterruptSetAxis(ByVal lAxisNo As Long, ByVal hWnd As
Long, ByVal uMessage As Long, ByVal pProc As Long, ByRef pEvent As
Long) As Long
```

Delphi

```
function AxmInterruptSetAxis(lAxisNo : LongInt; hWnd : HWND;
uMessage : DWord; pProc : AXT_INTERRUPT_PROC; pEvent : PHWND) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]hWnd	Window handle
[>>]uMessage	Window message
[>>]pProc	Callback API
[>>]pEvent	Use event

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Send window handle to bring interrupt message, and send API pointer when use callback.

Example

C++

```
// Use window message to bring interrupt message.

BEGIN_MESSAGE_MAP(CCAMCIPDlg, CDialog)
  //}}AFX_MSG_MAP
  // Declaration
```

```
  ON_MESSAGE(WM_CAMCIP_INTERRUPT,OnTriggerInterrupt)
END_MESSAGE_MAP()

// When interrupt occurs, called 'OnTriggerInterrupt()'.
AxmInterruptSetAxis(0, m_hWnd, NULL, NULL);
```

Visual Basic

```
' Use window message to bring interrupt message.
' Declaration
Private Sub AxtMsg1_OnMessage1(ByVal wParam As Long, ByVal lParam As Long)
' Interrupt message management sentence
End Sub

' When interrupt occurs, called 'AxtMsg1_OnMessage1()'.
AxtMsg1.Message1 = WM_CAMCIP_INTERRUPT

' When interrupt occurs, called 'OnTriggerInterrupt()'.
AxmInterruptSetAxis 0, Me.hWnd, WM_CAMCIP_INTERRUPT, 0
```

Delphi

```
{ Use window message to bring interrupt message. }
{ Declaration }
procedure TForm1.OnInterruptMessage(var Msg : TMessage);

begin
  { Interrupt message management sentence. }
  { When interrupt occurs, called 'OnInterruptMessage()'. }
  AxmInterruptSetAxis(Form1.Handle, WM_CAMCIP_INTERRUPT, nil);
end;
```

See Also

AxmInterruptSetAxisEnable, AxmInterruptGetAxisEnable, AxmInterruptRead, AxmInterruptReadAxisFlag

AxmInterruptSetAxisEnable

Purpose

Set whether interrupt is used on set axis.

Format

C++

```
DWORD AxmInterruptSetAxisEnable(long lAxisNo, DWORD uUse);
```

Visual Basic

```
Function AxmInterruptSetAxisEnable(ByVal lAxisNo As Long, ByVal uUse
As Long) As Long
```

Delphi

```
function AxmInterruptSetAxisEnable(lAxisNo : LongInt; uUse : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uUse	Whether interrupt is set : FALSE & TRUE – [00H] Interrupt not-used – [01H] Interrupt used

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

This function sets whether the interrupt will be used for the corresponding axis, sets whether the global interrupt condition is applied to the corresponding axis, and sets the global interrupt mask for the corresponding axis to configure.

Example

C++

```
// Set interrupt use on number axis 0.
AxmInterruptSetAxisEnable(0, ENABLE);

// Verify interrupt use setting on axis 0.
DWORD uIntUse;
AxmInterruptGetAxisEnable(0, &uIntUse);
```

Visual Basic

```
' Set interrupt use on number axis 0.
AxmInterruptSetAxisEnable 0, ENABLE

' Verify interrupt use setting on axis 0.
Dim uIntUse As Long
AxmInterruptGetAxisEnable 0, uIntUse
```

Delphi

```
{ Verify interrupt use setting on axis 0. }
var
  uIntUse : Dword;

begin
  { Set interrupt use on number axis 0. }
  AxmInterruptSetAxisEnable(0, ENABLE);

  AxmInterruptGetAxisEnable(0, @uIntUse);
end;
```

See Also

AxmInterruptSetAxis, AxmInterruptGetAxisEnable, AxmInterruptRead, AxmInterruptReadAxisFlag

AxmInterruptGetAxisEnable

Purpose

Return setting whether interrupt is used on corresponding axis.

Format

C++

```
DWORD AxmInterruptGetAxisEnable(long lAxisNo, DWORD *upUse);
```

Visual Basic

```
Function AxmInterruptGetAxisEnable(ByVal lAxisNo As Long, ByRef upUse
As Long) As Long
```

Delphi

```
function AxmInterruptGetAxisEnable(lAxisNo : LongInt; upUse :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upUse	Whether interrupt is set : FALSE & TRUE – [00H] Interrupt not-used – [01H] Interrupt used

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return setting whether interrupt is used on corresponding axis that AxmInterruptSetAxisEnable is set.

Example

C++

```
// Set interrupt use on axis 0.
AxmInterruptSetAxisEnable(0, ENABLE);

// Verify interrupt use setting on axis 0.
DWORD uIntUse;
AxmInterruptGetAxisEnable(0, &uIntUse);
```

Visual Basic

```
' Set interrupt use on axis 0.
AxmInterruptSetAxisEnable 0, ENABLE
```

```
' Verify interrupt use setting on axis 0.
Dim uIntUse As Long
AxmInterruptGetAxisEnable 0, uIntUse
```

Delphi

```
{ Verify interrupt use setting on axis 0. }
var
  uIntUse : Dword;

begin
  { Set interrupt use on axis 0. }
  AxmInterruptSetAxisEnable(0, ENABLE);

  AxmInterruptGetAxisEnable(0, @uIntUse);
end;
```

See Also

AxmInterruptSetAxis, AxmInterruptSetAxisEnable, AxmInterruptRead, AxmInterruptReadAxisFlag

AxmInterruptRead

Purpose

Return whether interrupt is used on corresponding axis.

Format

C++

```
DWORD AxmInterruptRead(long *lpAxisNo, DWORD *upFlag);
```

Visual Basic

```
Function AxmInterruptRead(ByRef lpAxisNo As Long, ByRef upFlag As
Long) As Long
```

Delphi

```
function AxmInterruptRead(lpAxisNo : PLongInt; upFlag : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lpAxisNo	Channel (axis) number(starting from 0)
[>>]upFlag	Whether interrupt is set : FALSE & TRUE – [00H] Interrupt not occurred – [01H] Interrupt occurred

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4052] AXT_RT_MOTION_NOT_INTERRUPT : Failed to read the resul of interrupt
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return occurred interrupt source verification to AxmInterruptReadAxisFlag API.

Example

C++

```
// Return whether interrupt is used on corresponding axis.
Long  upAxisNo;
DWORD upFlag;
AxmInterruptRead(&upAxisNo, &upFlag);
```

Visual Basic

```
' Return whether interrupt is used on corresponding axis.
Dim upAxisNo, upFlag As Long
AxmInterruptRead upAxisNo, upFlag
```

Delphi

```
{ Return whether interrupt is used on corresponding axis. }
var
   upFlag: DWord;
   upAxisNo: LongInt;

begin
   AxmInterruptRead(@upAxisNo, @upFlag);
end;
```

See Also

AxmInterruptSetAxis, AxmInterruptSetAxisEnable, AxmInterruptGetAxisEnable, AxmInterruptReadAxisFlag

AxmInterruptReadAxisFlag

Purpose

Return interrupt flag value on corresponding axis.

Format

C++

```
DWORD AxmInterruptReadAxisFlag(long lAxisNo, long lBank, DWORD
*upFlag);
```

Visual Basic

```
Function AxmInterruptReadAxisFlag(ByVal lAxisNo As Long, ByVal lBank
As Long, ByRef upFlag As Long) As Long
```

Delphi

```
function AxmInterruptReadAxisFlag(lAxisNo : LongInt; lBank : LongInt;
upFlag : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lBank	Interrupt bank number on corresponding axis (0: Bank1 , 1:Bank2)
[<<]upFlag	Interrupt occurrence state value : FALSE & TRUE – [00H] Interrupt not occurred – [01H] Interrupt occurred

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

32-bit interrupt flag register exists as 2 banks on motion chip installed on motion module in use. With verification of interrupt flag value of each bank, it can verify interrupt flag value that is outputted.

See in AXT_MOTION_QIINTERRUPT_BANK1_DEF, AXT_MOTION_QIINTERRUPT_BANK2_DEF

Example

C++

```
// Return interrupt flag value of number 1 bank on corresponding axis.
Long  upAxisNo;
DWORD upFlag;

AxmInterruptReadAxisFlag(&upAxisNo, 0, &upFlag);
```

Visual Basic

```
' Return interrupt flag value of number 1 bank on corresponding axis.
Dim upAxisNo, upFlag As Long
AxmInterruptReadAxisFlag upAxisNo, 0, upFlag
```

Delphi

```
{ Return interrupt flag value of number 1 bank on corresponding axis. }
var
  upFlag: DWord;
  upAxisNo: LongInt;

begin
  AxmInterruptReadAxisFlag (@upAxisNo, 0, @upFlag);
end;
```

See Also

AxmInterruptSetAxis, AxmInterruptSetAxisEnable, AxmInterruptGetAxisEnable, AxmInterruptRead

AxmInterruptSetUserEnable

Purpose

Set whether Interrupt that user wanted occurred on a specific axis.

Format

C++

```
DWORD AxmInterruptSetUserEnable(long lAxisNo, long lBank, DWORD
uInterruptNum);
```

Visual Basic

```
Function AxmInterruptSetUserEnable(ByVal lAxisNo As Long, ByVal lBank
As Long , ByVal uInterruptNum As Long) As Long
```

Delphi

```
function AxmInterruptSetUserEnable(lAxisNo : LongInt; lBank : LongInt;
uInterruptNum : DWORD) : DWord; stdcall;
```

Parameters

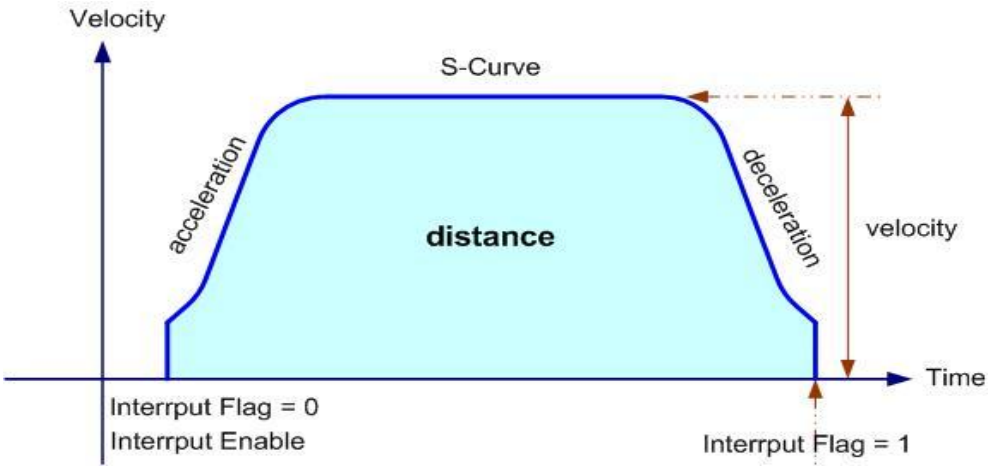
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lBank	Interrupt bank number user wanted (0,1) : AXT_MOTION_QIINTERRUPT_BANK1_DEF AXT_MOTION_QIINTERRUPT_BANK2_DEF
[>>]uInterruptNum	Interrupt bank number user wanted (0 – 31)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

When interrupt is used, set to 'ENABLE,' otherwise, set to 'DISABLE.' If interrupt is used, Active Level of interrupt output signal can be set to 'HIGH' or 'LOW'.



Example

C++

```
// Verify whether interrupt occurred after completion of pulse output on axis 0.
AxmInterruptSetUserEnable (0, 1 , QIINTBANK2_9);
DWORD dwInterruptNum
AxmInterruptGetUserEnable (0, 1 , &dwInterruptNum);
```

Visual Basic

```
' Verify whether interrupt occurred after completion of pulse output on axis 0.
AxmInterruptSetUserEnable 0, 1 , QIINTBANK2_9
Dim dwInterruptNum As Long
AxmInterruptGetUserEnable 0, 1 , dwInterruptNum
```

Delphi

```
var
   dwInterruptNum: DWord;

begin
   { Verify whether interrupt occurred after completion of pulse output on axis 0. }
   AxmInterruptSetUserEnable (0, 1, QIINTBANK2_9);
   AxmInterruptGetUserEnable (0, 1 , @dwInterruptNum);
end;
```

See Also

AxmInterruptSetAxis, AxmInterruptSetAxisEnable, AxmInterruptGetAxisEnable, AxmInterruptRead

AxmInterruptGetUserEnable

Purpose

Verify whether interrupt user wanted occurred on a specific axis.

Format

C++

```
DWORD AxmInterruptGetUserEnable(long lAxisNo, long lBank, DWORD
*upInterruptNum);
```

Visual Basic

```
Function AxmInterruptGetUserEnable(ByVal lAxisNo As Long, ByVal lBank
As Long, ByRef upInterruptNum As Long) As Long
```

Delphi

```
function AxmInterruptGetUserEnable(lAxisNo : LongInt; lBank : LongInt;
upInterruptNum : PDWord) : DWord; stdcall;
```

Parameters

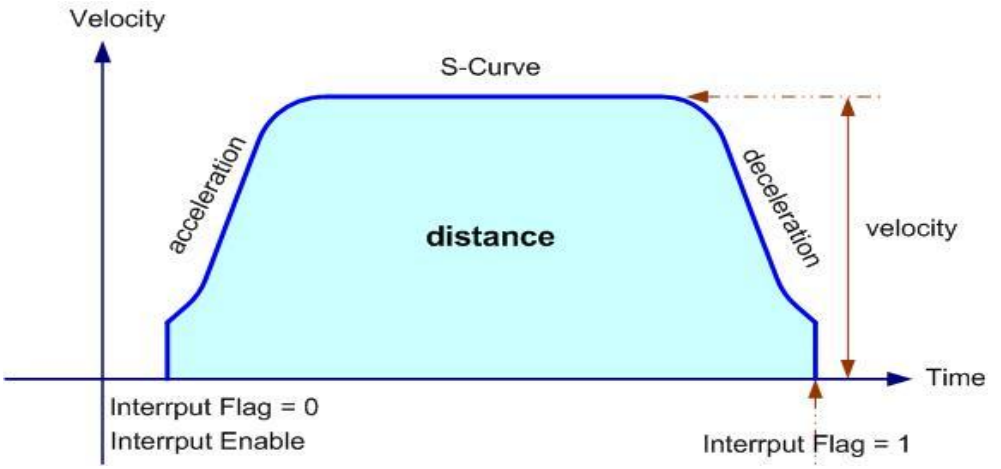
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lBank	Interrupt bank number user wanted (0,1) : AXT_MOTION_QIINTERRUPT_BANK1_DEF AXT_MOTION_QIINTERRUPT_BANK2_DEF
[<<]upInterruptNum	Interrupt bank number user wanted (0 – 31)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Use when reading 32-bit interrupt value set by 'AxmInterruptGetUserEnable'.



Example

C++

```
// Verify whether interrupt occurred after completion of pulse output on axis 0.
AxmInterruptSetUserEnable (0, 1 , QIINTBANK2_9);
DWORD dwInterruptNum
AxmInterruptGetUserEnable (0, 1 , &dwInterruptNum);
```

Visual Basic

```
' Verify whether interrupt occurred after completion of pulse output on axis 0.
AxmInterruptSetUserEnable 0, 1 , QIINTBANK2_9
Dim dwInterruptNum As Long
AxmInterruptGetUserEnable 0, 1 , dwInterruptNum
```

Delphi

```
var
   dwInterruptNum: DWord;

begin
   { Verify whether interrupt occurred after completion of pulse output on axis 0. }
   AxmInterruptSetUserEnable (0, 1, QIINTBANK2_9);
   AxmInterruptGetUserEnable (0, 1 , @dwInterruptNum);
end;
```

See Also

AxmInterruptSetAxis, AxmInterruptSetAxisEnable, AxmInterruptGetAxisEnable, AxmInterruptRead, AxmInterruptReadAxisFlag

Motion Parameter Setting API

APIs that are introduced in this unit describe all parameter setting related to motion. Before drive execution, read motion parameter APIs carefully and execute drive.

Function	Description
AxmMotLoadParaAll	Calls 33 parameter files stored with AxmMotSaveParaAll. Arbitrary change is available by user's calling
AxmMotSaveParaAll	Stores all 33 parameters by axis for the current all axes. Store as .mot file. Call a file using AxmM otLoadParaAll
AxmMotSetParaLoad	Use AxmMotSaveParaAll API to store information which is set on register of CAMC-IP, QI as a file. The API stores information of all axes as a file. Set 4 parameters of dInitPos, dInitVel, dInitAccel and dInitDecel only from 28 to 31 among parameters. The rest parameters are stored using AxmMotSaveParaAll API reading current state when library is closed
AxmMotGetParaLoad	Use AxmMotSaveParaAll API to store information which is set on register of CAMC-IP, QI as a file. The API stores information of all axes as a file. The API is used to call only 4 parameters of dInitPos, dInitVel, dInitAccel and dInitDecel from 28 to 31 among parameters that information of all axes is stored
AxmMotSetPulseOutMethod	Sets pulse output method on a specific axis
AxmMotGetPulseOutMethod	Returns pulse output method setting on a specific axis,
AxmMotSetEncInputMethod	Sets encoder input method on a specific axis including increasing direction setting of external (actual) count on a specific axis.
AxmMotGetEncInputMethod	Returns encoder input method on a specific axis including increasing direction setting of external (actual) count on a specific axis.
AxmMotSetMoveUnitPerPulse	Sets moving distance per pulse on a specific axis.
AxmMotGetMoveUnitPerPulse	Returns moving distance per pulse on a specific axis.
AxmMotSetDecelMode	Sets deceleration start point search method on a specific axis.
AxmMotGetDecelMode	Returns deceleration start point search method on a specific axis.
AxmMotSetRemainPulse	Sets remaining pulse in manual deceleration mode on a specific axis.
AxmMotGetRemainPulse	Returns remaining pulse in manual deceleration mode on a specific axis.
AxmMotSetMaxVel	Sets maximum velocity in constant speed drive API on a specific axis.
AxmMotGetMaxVel	Returns maximum velocity in constant speed drive API on a specific axis.
AxmMotSetAbsRelMode	Sets moving distance calculation mode on a specific axis. If absolute position mode (0) is set, position value used in moving API is to be absolute position value. For example, if current position value is "10" and moves from absolute position mode to "100" position, it will stop at "100" position moving as much as "+90" from the current position. If relative position mode (1) is set, position value used moving API is to be moving amount to move from the current position. For example, if current position value is "10" and moves from relative position mode to "100" position, it will stop at "110" position moving as much as "+100" from the current position.

AxmMotGetAbsRelMode	Returns moving distance calculation mode which is set on a specific axis.
AxmMotSetProfileMode	Sets drive velocity profile mode on a specific axis. Velocity profile mode has symmetrical or asymmetrical Trapezoide / S-Curve mode and Quasi-S mode.
AxmMotGetProfileMode	Returns drive velocity profile mode which is set on a specific axis.
AxmMotSetAccelUnit	Sets acceleration unit on a specific axis. Acceleration unit can be selected among u/s2, sec, rev/s2 .
AxmMotGetAccelUnit	Returns acceleration unit which is set on a specific axis.
AxmMotSetMinVel	Sets initial velocity on a specific axis.
AxmMotGetMinVel	Returns initial velocity on a specific axis.
AxmMotSetAccelJerk	Sets acceleration jerk value on a specific axis.
AxmMotGetAccelJerk	Returns acceleration jerk value on a specific axis.
AxmMotSetDecelJerk	Sets deceleration jerk value on a specific axis.
AxmMotGetDecelJerk	Returns deceleration jerk value on a specific axis.

AxmMotLoadParaAll

Purpose

To read the configuration values of all axes from the specified file and set the system with those values. Parameter files are save by AxmMotSaveParaAll function, and the saved files can be edited by the user without resitrictions.

Format

C++

```
DWORD AxmMotLoadParaAll(char *spFilePath);
```

Visual Basic

```
Function AxmMotLoadParaAll(ByRef spFilePath As String) As Long
```

Delphi

```
function AxmMotLoadParaAll(spFilePath : PChar) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]spFilePath	File name to call

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4111] AXT_RT_MOTION_INVALID_FILE_LOAD : Failed to load the file containing motion setting values
* See error code Table for more information on status error codes

Description

Read and set from *spFilePath file assigned initial value of all axes. See AxmMotSaveParaAll table for loaded parameter. No .MOT setting file at the beginning. Basic setting value is set internally. To set directly, store .MOT file using AxmMotSaveParaAll API and call to AxmMotLoadParaAll.

Initial value of internal variable that file is set.

No	Signal Related Variable	Parameter Explanation	Setting Value
00	AXIS_NO	Axis number	Setting axis number
01	PULSE_OUT_METHOD	Setting of the pulse output method which is used as a servo pack order on the motion board	TwoCcwCwHigh
02	Enc[Encoder]	Setting of the input method of encoder signal	Sqr4Mode
03	InP[InPos]	Setting of the use of completion signal for servo pack position decision and Active Level	2
04	Alm[Alarm]	Setting of the use of servo pack alarm signal and Active Level	1

05	NLimit	Setting of the use of machine (−) limit sensor and Active Level	1
06	PLimit	Setting of the use of machine (+) limit sensor and Active Level	1
07	MinSpd	Setting of the initial drive velocity when the motor drives	1.0
08	MaxSpd	Setting of the maximum drive velocity when the motor drives	700000.0
09	HmSig	Setting of a signal to use as home sensor	4
10	HmLev	Setting of Active Level of home sensor	1
11	HmDir	Setting of the direction of initial search process during home sensor search	0
12	Zphas_Lev	Active level on encoder Z phase	1
13	Zphas_Use	Setting of the search of encoder Z phase after home sensor search	0
14	Stop Signal Mode	Mode when used ESTOP, SSTOP	0
15	Stop Signal level	Use level of ESTOP, SSTOP	0
16	VelFirst	Initial high speed search velocity during home search (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	10000.0
17	VelSecond	Velocity that comes out from the reverse direction after the 1st sensor search during home search	10000.0
18	VelThird	Velocity of research process after the 1st sensor search during home search	2000.0
19	VelLast	Setting of the final search velocity during home search [Accuracy decision of home search]	100.0
20	AccFirst	Initial high speed search acceleration during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
21	AccSecond	Acceleration that comes out from the reverse direction after the 1st sensor search during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
22	HClrTim	Waiting time before clear of Cmd, Act position after home search completion	1000.0
23	HOffset	Moving offset value during reset of machine home after home search completion	0.0
24	NSWLimit	Setting of (−) Software Limit value to apply after home search completion	0.0
25	PSWLimit	Setting of (+) Software Limit value to apply after home search completion	0.0
26	OenPulse	Setting of pulse number that requires for 1 rotation of motor [refer to servo pack electric gear ratio]	1.0
27	OneUnit	Moving amount that is moved for 1 rotation of motor [mm, degree, etc]	1
28	InitPosition	Position for initialization	200

29	InitVel	Velocity for the initialization (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	200
30	InitAccel	Acceleration for the initialization (acceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
31	InitDecel	Deceleration for the initialization (deceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
32	AbsReMode	Absolute/relative mode for the initialization	0
33	ProfMode	Velocity profile mode for the initialization	4

Example

C++

```
char szFileName[50];
strcpy(szFileName, "C:\\Data\\CamcQI.mot");
AxmMotLoadParaAll(szFileName); // Read initial values of all axes on file and set.
```

Visual Basic

```
Dim szFileName As String
szFileName = "C:\\Data\\CamcQI.mot"
AxmMotLoadParaAll szFileName ' Read initial values of all axes on file and set.
```

Delphi

```
var
  szFileName : String;

begin
  szFileName := 'C:\\Data\\CamcQI.mot';
  AxmMotLoadParaAll(@szFileName); { Read initial values of all axes on file and set. }
end;
```

See Also

AxmMotSaveParaAll, AxmMotSetParaLoad, AxmMotGetParaLoad, AxmMotSetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotSaveParaAll

Purpose

Store all 33 parameters by axis for the all current axes. Store as .mot file. Call the file using AxmMotLoadParaAll.

Format

C++

```
DWORD AxmMotSaveParaAll(char *szFilePath);
```

Visual Basic

```
Function AxmMotSaveParaAll(ByRef szFilePath As String) As Long
```

Delphi

```
function AxmMotSaveParaAll(szFilePath: PChar) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]szFilePath	File name to store

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4112] AXT_RT_MOTION_INVALID_FILE_SAVE : Failed to save the file containing motion setting values
* See error code Table for more information on status error codes

Description

Store initial value of all axes in *szFilePath file. No .MOT setting file at the beginning. Basic setting value is set internally. To set directly, store .MOT file using AxmMotSaveParaAll API and call to AxmMotLoadParaAll.

Initial value of internal variable that file is set

No	Signal Related Variable	Parameter Explanation	Setting Value
00	AXIS_NO	Axis number	Setting axis number
01	PULSE_OUT_METHOD	Setting of the pulse output method which is used as a servo pack order on the motion board	TwoCcwCwHigh
02	Enc[Encoder]	Setting of the input method of encoder signal	Sqr4Mode
03	InP[InPos]	Setting of the use of completion signal for servo pack position decision and Active Level	2
04	Alm[Alarm]	Setting of the use of servo pack alarm signal and Active Level	1
05	NLimit	Setting of the use of machine (–) limit sensor and Active Level	1

06	PLimit	Setting of the use of machine (+) limit sensor and Active Level	1
07	MinSpd	Setting of the initial drive velocity when the motor drives	1.0
08	MaxSpd	Setting of the maximum drive velocity when the motor drives	700000.0
09	HmSig	Setting of a signal to use as home sensor	4
10	HmLev	Setting of Active Level of home sensor	1
11	HmDir	Setting of the direction of initial search process during home sensor search	0
12	Zphas_Lev	Active level on encoder Z phase	1
13	Zphas_Use	Setting of the search of encoder Z phase after home sensor search	0
14	Stop Signal Mode	Mode when used ESTOP, SSTOP	0
15	Stop Signal level	Use level of ESTOP, SSTOP	0
16	VelFirst	Initial high speed search velocity during home search (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	10000.0
17	VelSecond	Velocity that comes out from the reverse direction after the 1st sensor search during home search	10000.0
18	VelThird	Velocity of research process after the 1st sensor search during home search	2000.0
19	VelLast	Setting of the final search velocity during home search [Accuracy decision of home search]	100.0
20	AccFirst	Initial high speed search acceleration during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
21	AccSecond	Acceleration that comes out from the reverse direction after the 1st sensor search during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
22	HClrTim	Waiting time before clear of Cmd, Act position after home search completion	1000.0
23	HOffset	Moving offset value during reset of machine home after home search completion	0.0
24	NSWLimit	Setting of (−) Software Limit value to apply after home search completion	0.0
25	PSWLimit	Setting of (+) Software Limit value to apply after home search completion	0.0
26	OenPulse	Setting of pulse number that requires for 1 rotation of motor [refer to servo pack electric gear ratio]	1.0
27	OneUnit	Moving amount that is moved for 1 rotation of motor [mm, degree, etc]	1
28	InitPosition	Position for initialization	200
29	InitVel	Velocity for the initialization (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	200

30	InitAccel	Acceleration for the initialization (acceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
31	InitDecel	Deceleration for the initialization (deceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
32	AbsReMode	Absolute/relative mode for the initialization	0
33	ProfMode	Velocity profile mode for the initialization	4

Example

C++

```
char szFileName[50];
strcpy(szFileName, "C:\\Data\\CamCIP.mot");
AxmMotSaveParaAll(szFileName);  // Store initial values of all axes on file.
```

Visual Basic

```
Dim szFileName As String
szFileName = "C:\\Data\\CamCIP.mot"
AxmMotSaveParaAll szFileName  ' Store initial values of all axes on file.
```

Delphi

```
var
   szFileName : String;

begin
   szFileName := 'C:\\Data\\CamCIP.mot';
   AxmMotSaveParaAll(@szFileName);    { Store initial values of all axes on file. }
end;
```

See Also

AxmMotLoadParaAll, AxmMotSetParaLoad, AxmMotGetParaLoad, AxmMotSetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotSetParaLoad

Purpose

To set the parameter no. 28 to 31 – dInitPos, dInitVel, dInitAccel, dInitDecel – among the parameters saved by the AxmMotSaveParaAll function.

Format

C++

```
DWORD AxmMotSetParaLoad(long lAxisNo, double dInitPos, double
dInitVel, double dInitAccel, double dInitDecel);
```

Visual Basic

```
Function AxmMotSetParaLoad(ByVal lAxisNo As Long, ByVal dInitPos As
Double, ByVal dInitVel As Double, ByVal dInitAccel As Double, ByVal
dInitDecel As Double) As Long
```

Delphi

```
function AxmMotSetParaLoad(lAxisNo : LongInt; dInitPos : Double;
dInitVel : Double; dInitAccel : Double; dInitDecel : Double) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dInitPos	[1000.0] Initialization position (user brings this value as soon as library opens)
[>>]dInitVel	[200.0] Initialization velocity (user brings this value as soon as library opens)
[>>]dInitAccel	[400.0] Initialization acceleration (user brings this value as soon as library opens)
[>>]dInitDecel	[400.0] Initialization deceleration (user brings this value as soon as library opens)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

No .MOT setting file at the beginning. Basic setting value is set internally. To set directly, store .MOT file using AxmMotSaveParaAll API and call with AxmMotLoadParaAll. When close library, set 4 parameters with AxmMotSetParaLoad reading current state and store using AxmMotSaveParaAll API. If no setting, no stored of 4 parameters.

Initial value of internal variable that file is set

No	Signal Related Variable	Parameter Explanation	Setting Value
00	AXIS_NO	Axis number	Setting axis number
01	PULSE_OUT_METHOD	Setting of the pulse output method which is used as a servo pack order on the motion board	TwoCcwCwHigh
02	Enc[Encoder]	Setting of the input method of encoder signal	Sqr4Mode
03	InP[InPos]	Setting of the use of completion signal for servo pack position decision and Active Level	2
04	Alm[Alarm]	Setting of the use of servo pack alarm signal and Active Level	1
05	NLimit	Setting of the use of machine (–) limit sensor and Active Level	1
06	PLimit	Setting of the use of machine (+) limit sensor and Active Level	1
07	MinSpd	Setting of the initial drive velocity when the motor drives	1.0
08	MaxSpd	Setting of the maximum drive velocity when the motor drives	700000.0
09	HmSig	Setting of a signal to use as home sensor	4
10	HmLev	Setting of Active Level of home sensor	1
11	HmDir	Setting of the direction of initial search process during home sensor search	0
12	Zphas_Lev	Active level on encoder Z phase	1
13	Zphas_Use	Setting of the search of encoder Z phase after home sensor search	0
14	Stop Signal Mode	Mode when used ESTOP, SSTOP	0
15	Stop Signal level	Use level of ESTOP, SSTOP	0
16	VelFirst	Initial high speed search velocity during home search (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	10000.0
17	VelSecond	Velocity that comes out from the reverse direction after the 1st sensor search during home search	10000.0
18	VelThird	Velocity of research process after the 1st sensor search during home search	2000.0
19	VelLast	Setting of the final search velocity during home search [Accuracy decision of home search]	100.0
20	AccFirst	Initial high speed search acceleration during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
21	AccSecond	Acceleration that comes out from the reverse direction after the 1st sensor search during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
22	HClrTim	Waiting time before clear of Cmd, Act position after	1000.0

		home search completion	
23	HOffset	Moving offset value during reset of machine home after home search completion	0.0
24	NSWLimit	Setting of (−) Software Limit value to apply after home search completion	0.0
25	PSWLimit	Setting of (+) Software Limit value to apply after home search completion	0.0
26	OenPulse	Setting of pulse number that requires for 1 rotation of motor [refer to servo pack electric gear ratio]	1.0
27	OneUnit	Moving amount that is moved for 1 rotation of motor [mm, degree, etc]	1
28	InitPosition	Position for initialization	200
29	InitVel	Velocity for the initialization (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	200
30	InitAccel	Acceleration for the initialization (acceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
31	InitDecel	Deceleration for the initialization (deceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
32	AbsReMode	Absolute/relative mode for the initialization	0
33	ProfMode	Velocity profile mode for the initialization	4

Set parameters in force from 28 to 33.

Example

C++

```
double dInitpos = 1000, dInitvel = 200, dInitaccel = 400, dInitdecel = 400;
AxmMotSetParaLoad(0, dInitpos, dInitvel, dInitaccel, dInitdecel);
double dReadInitpos,dReadInitvel,dReadInitaccel,dReadInitdecel;
AxmMotGetParaLoad(0,&dReadInitpos,&dReadInitvel, &dReadInitaccel, &dReadInitdecel);
```

Visual Basic

```
Dim dInitpos ,dInitvel,dInitaccel,dInitdecel As Double
dInitpos=1000 :dInitvel = 200 :dInitaccel = 400: dInitdecel = 400
AxmMotSetParaLoad 0, dInitpos, dInitvel, dInitaccel, dInitdecel
Dim dReadInitpos,dReadInitvel,dReadInitaccel,dReadInitdecel As Double
AxmMotGetParaLoad 0, dReadInitpos, dReadInitvel, dReadInitaccel, dReadInitdecel
```

Delphi

```
var
  dInitpos,dInitvel,dInitaccel,dInitdecel : Double;
  dReadInitpos,dReadInitvel,dReadInitaccel,dReadInitdecel : Double;

begin
  dInitpos := 1000; dInitvel := 200; dInitaccel := 400; dInitdecel := 400;
  AxmMotSetParaLoad(0, dInitpos, dInitvel, dInitaccel, dInitdecel);
  AxmMotGetParaLoad(0, @dReadInitpos,@dReadInitvel, @dReadInitaccel, @dReadInitdecel);
end;
```

See Also

AxmMotLoadParaAll, AxmMotGetParaLoad, AxmMotSetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotGetParaLoad

Purpose

To get the parameter no. 28 to 33 – dInitPos, dInitVel, dInitAccel, dInitDecel, uInitAbsRelMode, and uInitProfileMode – among the parameters saved by the AxmMotSaveParaAll function.

Format

C++

```
DWORD AxmMotGetParaLoad(long lAxisNo, double *dpInitPos, double
*dpInitVel, double *dpInitAccel, double *dpInitDecel);
```

Visual Basic

```
Function AxmMotGetParaLoad(ByVal lAxisNo As Long, ByRef dpInitPos As
Double, ByRef dpInitVel As Double, ByRef dpInitAccel As Double, ByRef
dpInitDecel As Double) As Long
```

Delphi

```
function AxmMotGetParaLoad(lAxisNo : LongInt; dpInitPos : PDouble;
dpInitVel : PDouble; dpInitAccel : PDouble; dpInitDecel : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpInitPos	[1000.0] Initialization position (user brings this value as soon as library opens)
[<<]dpInitVel	[200.0] Initialization velocity (user brings this value as soon as library opens)
[<<]dpInitAccel	[400.0] Initialization acceleration (user brings this value as soon as library opens)
[<<]dpInitDecel	[400.0] Initialization deceleration (user brings this value as soon as library opens)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

*Note: When only required data are needed to bring, it is able to use with input NULL in unrequired variable.

No .MOT setting file at the beginning. Basic setting value is set internally. To set directly, store .MOT file using AxmMotSaveParaAll API and call with AxmMotLoadParaAll.

Among parameters, use only 4 parameters of dInitPos, dInitVel, dInitAccel, dInitDecel, uInitAbsRelMode, and uInitProfileMode from 28 to 31 to read set value that is set inside library currently.

Initial value of internal variable that file is set

No	Signal Related Variable	Parameter Explanation	Setting Value
00	AXIS_NO	Axis number	Setting axis number
01	PULSE_OUT_METHOD	Setting of the pulse output method which is used as a servo pack order on the motion board	TwoCcwCwHigh
02	Enc[Encoder]	Setting of the input method of encoder signal	Sqr4Mode
03	InP[InPos]	Setting of the use of completion signal for servo pack position decision and Active Level	2
04	Alm[Alarm]	Setting of the use of servo pack alarm signal and Active Level	1
05	NLimit	Setting of the use of machine (-) limit sensor and Active Level	1
06	PLimit	Setting of the use of machine (+) limit sensor and Active Level	1
07	MinSpd	Setting of the initial drive velocity when the motor drives	1.0
08	MaxSpd	Setting of the maximum drive velocity when the motor drives	700000.0
09	HmSig	Setting of a signal to use as home sensor	4
10	HmLev	Setting of Active Level of home sensor	1
11	HmDir	Setting of the direction of initial search process during home sensor search	0
12	Zphas_Lev	Active level on encoder Z phase	1
13	Zphas_Use	Setting of the search of encoder Z phase after home sensor search	0
14	Stop Signal Mode	Mode when used ESTOP, SSTOP	0
15	Stop Signal level	Use level of ESTOP, SSTOP	0
16	VelFirst	Initial high speed search velocity during home search (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	10000.0
17	VelSecond	Velocity that comes out from the reverse direction after the 1st sensor search during home search	10000.0
18	VelThird	Velocity of research process after the 1st sensor search during home search	2000.0
19	VelLast	Setting of the final search velocity during home search [Accuracy decision of home search]	100.0
20	AccFirst	Initial high speed search acceleration during home search (unit of acceleration is PPS[Pulses/sec^2] if	40000.0

		Unit/pulse is 1/1)	
21	AccSecond	Acceleration that comes out from the reverse direction after the 1st sensor search during home search (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	40000.0
22	HClrTim	Waiting time before clear of Cmd, Act position after home search completion	1000.0
23	HOffset	Moving offset value during reset of machine home after home search completion	0.0
24	NSWLimit	Setting of (−) Software Limit value to apply after home search completion	0.0
25	PSWLimit	Setting of (+) Software Limit value to apply after home search completion	0.0
26	OenPulse	Setting of pulse number that requires for 1 rotation of motor [refer to servo pack electric gear ratio]	1.0
27	OneUnit	Moving amount that is moved for 1 rotation of motor [mm, degree, etc]	1
28	InitPosition	Position for initialization	200
29	InitVel	Velocity for the initialization (unit of velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)	200
30	InitAccel	Acceleration for the initialization (acceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
31	InitDecel	Deceleration for the initialization (deceleration unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1)	400
32	AbsReMode	Absolute/relative mode for the initialization	0
33	ProfMode	Velocity profile mode for the initialization	4

Set parameters in force from 28 to 33.

Example

C++

```
double dInitpos = 1000, dInitvel = 200, dInitaccel = 400, dInitdecel = 400;
AxmMotSetParaLoad(0, dInitpos, dInitvel, dInitaccel, dInitdecel);
double dReadInitpos,dReadInitvel,dReadInitaccel,dReadInitdecel;
AxmMotGetParaLoad(0,&dReadInitpos,&dReadInitvel, &dReadInitaccel, &dReadInitdecel);
```

Visual Basic

```
Dim dInitpos ,dInitvel,dInitaccel,dInitdecel As Double
dInitpos=1000 :dInitvel = 200 :dInitaccel = 400: dInitdecel = 400
AxmMotSetParaLoad 0, dInitpos, dInitvel, dInitaccel, dInitdecel
Dim dReadInitpos,dReadInitvel,dReadInitaccel,dReadInitdecel As Double
AxmMotGetParaLoad 0, dReadInitpos, dReadInitvel, dReadInitaccel, dReadInitdecel
```

Delphi

```
var
  dInitpos,dInitvel,dInitaccel,dInitdecel : Double;
  dReadInitpos,dReadInitvel,dReadInitaccel,dReadInitdecel : Double;
```

```
begin
   dInitpos := 1000; dInitvel := 200; dInitaccel := 400; dInitdecel := 400;
   AxmMotSetParaLoad(0, dInitpos, dInitvel, dInitaccel, dInitdecel);
   AxmMotGetParaLoad(0, @dReadInitpos,@dReadInitvel, @dReadInitaccel, @dReadInitdecel);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotSetPulseOutMethod

Purpose

Set pulse output method on a specific axis.

Format

C++

```
DWORD AxmMotSetPulseOutMethod(long lAxisNo, DWORD uMethod);
```

Visual Basic

```
Function AxmMotSetPulseOutMethod(ByVal lAxisNo As Long, ByVal uMethod
As Long) As Long
```

Delphi

```
function AxmMotSetPulseOutMethod(lAxisNo : LongInt; uMethod : DWord) :
DWord; stdcall;
```

Parameters

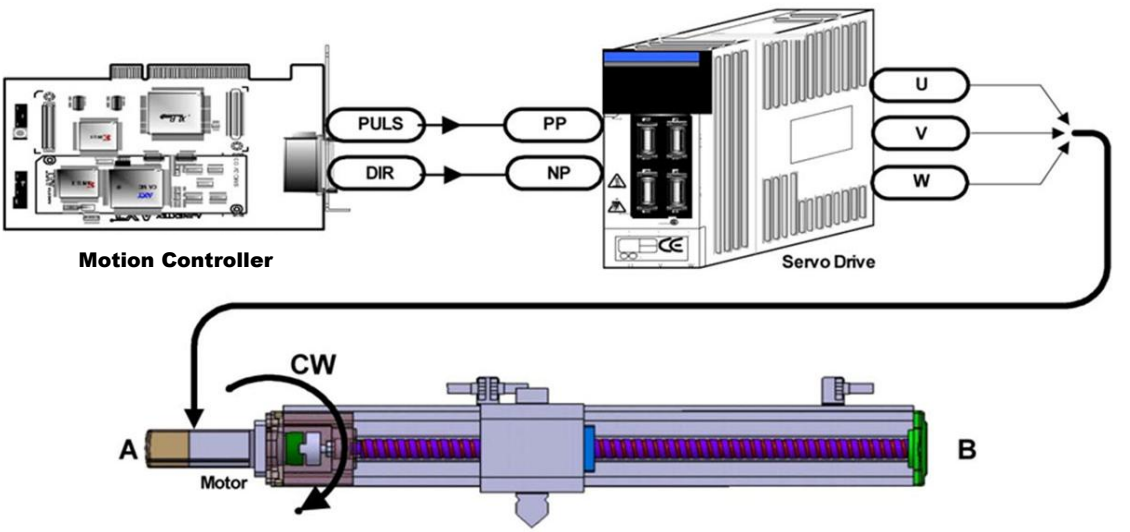
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uMethod	[04h] Pulse output method : AXT_MOTION_PULSE_OUTPUT_DEF – [00h] 1 pulse method, PULSE(Active High), forward direction(DIR=Low) / reverse direction (DIR=High) – [01h] 1 pulse method, PULSE(Active High), forward direction (DIR=High) / reverse direction (DIR=Low) – [02h] 1 pulse method, PULSE(Active Low), forward direction (DIR=Low) / reverse direction (DIR=High) – [03h] 1 pulse method, PULSE(Active Low), forward direction (DIR=High) / reverse direction (DIR=Low) – [04h] 2 pulse method, PULSE(CCW: reverse direction), DIR(CW: forward direction), Active High – [05h] 2 pulse method, PULSE(CCW: reverse direction), DIR(CW: forward direction), Active Low – [06h] 2 pulse method, PULSE(CW: forward direction), DIR(CCW: reverse direction), Active High – [07h] 2 pulse method, PULSE(CW: forward direction), DIR(CCW: reverse direction), Active Low – [08h] 2 phase (90' phase difference), PULSE lead DIR(CW: forward direction), PULSE lag DIR(CCW: reverse direction) – Non-used of IP – [09h] 2 phase (90' phase difference), PULSE lead DIR(CCW: forward direction), PULSE lag DIR(CW: reverse direction) – Non-used of IP

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

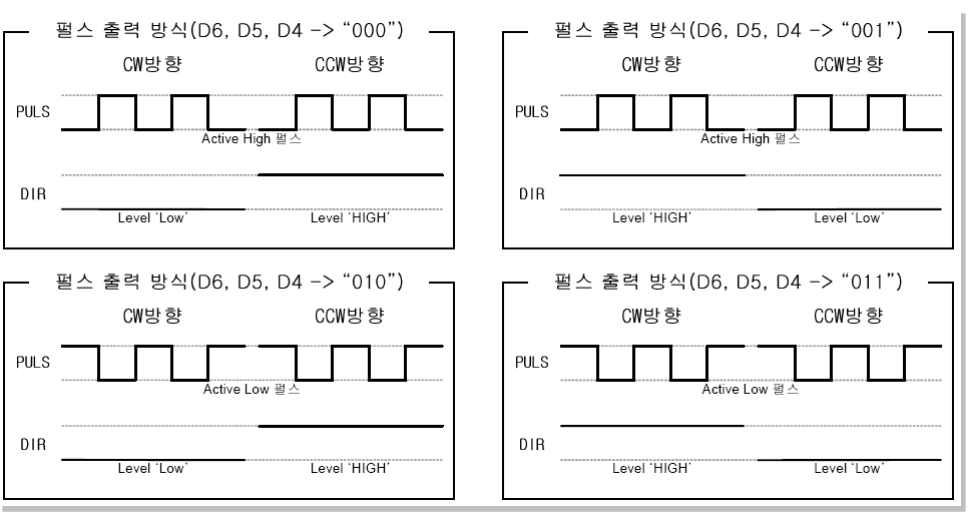
Description

There are 2 pulse output methods generally; one pulse and two pulse output method, and each is divided into 4 methods. Pulse output method must be set same with servo driver in use. Pulse output method is generally divided into 1Pulse and 2Pulse method, and each method shows below. However, IP does not provide Method 8 to 9.

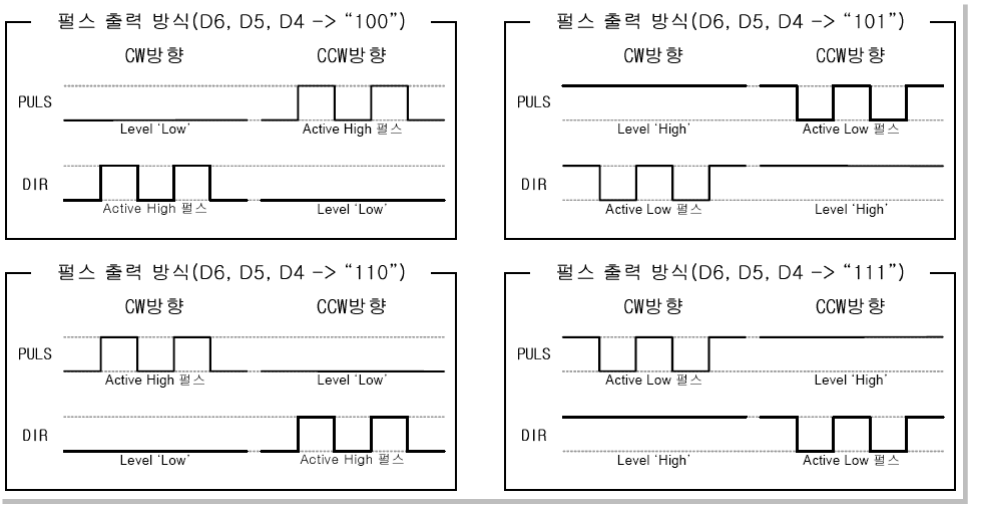


PULS, DIR pulse output method of motion controller outputted to servo drive is divided into One Pulse and Two Pulse method.

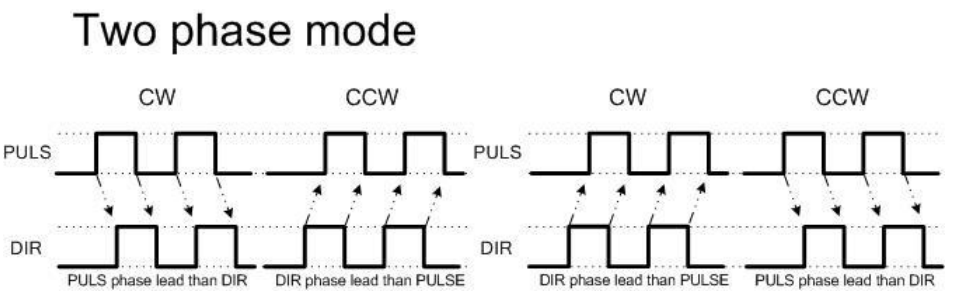
● One Pulse Method



● Two Pulse Method



● Two Phase Method



Value	Direction Signal (DIR)		Pulse Signal		Motion
	CW	CCW			
0	L	H	Active H Pulse		1 Pulse Method
1	H	L			
2	L	H	Active L Pulse		
3	H	L			
4	CW Pulse	Active H	CCW Pulse	Active H	2 Pulse Method
5		Active L		Active L	
6	CCW Pulse	Active H	CW Pulse	Active H	
7		Active L		Active L	

Example

C++

```
// Set to Two pulse method on axis 0.
AxmMotSetPulseOutMethod(0, TwoCwCcwHigh);
// Verify pulse output method on axis 0.
DWORD Method;
AxmMotGetPulseOutMethod(0, &Method);
```

Visual Basic

```
' Set to Two pulse method on axis 0.
AxmMotSetPulseOutMethod 0, TwoCwCcwHigh
' Verify pulse output method on axis 0.
Dim Method As Long
AxmMotGetPulseOutMethod 0, Method
```

Delphi

```
{ Verify pulse output method on axis 0. }
var
  Method: DWord;

begin
  { Set to Two pulse method on axis 0. }
  AxmMotSetPulseOutMethod(0, TwoCwCcwHigh);
  AxmMotGetPulseOutMethod(0, @Method);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll , AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotGetPulseOutMethod

Purpose

Return pulse output method setting on a specific axis.

Format

C++

```
DWORD AxmMotGetPulseOutMethod(long lAxisNo, DWORD *upMethod);
```

Visual Basic

```
Function AxmMotGetPulseOutMethod(ByVal lAxisNo As Long, ByRef
upMethod As Long) As Long
```

Delphi

```
function AxmMotGetPulseOutMethod(lAxisNo : LongInt; upMethod :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upMethod	[04h] Pulse output method : AXT_MOTION_PULSE_OUTPUT_DEF – [00h] 1 pulse method, PULSE(Active High), forward direction(DIR=Low) / reverse direction (DIR=High) – [01h] 1 pulse method, PULSE(Active High), forward direction (DIR=High) / reverse direction (DIR=Low) – [02h] 1 pulse method, PULSE(Active Low), forward direction (DIR=Low) / reverse direction (DIR=High) – [03h] 1 pulse method, PULSE(Active Low), forward direction (DIR=High) / reverse direction (DIR=Low) – [04h] 2 pulse method, PULSE(CCW: reverse direction), DIR(CW: forward direction), Active High – [05h] 2 pulse method, PULSE(CCW: reverse direction), DIR(CW: forward direction), Active Low – [06h] 2 pulse method, PULSE(CW: forward direction), DIR(CCW: reverse direction), Active High – [07h] 2 pulse method, PULSE(CW: forward direction), DIR(CCW: reverse direction), Active Low – [08h] 2 phase (90' phase difference), PULSE lead DIR(CW: forward direction), PULSE lag DIR(CCW: reverse direction) – Non-used of IP – [09h] 2 phase (90' phase difference), PULSE lead DIR(CCW: forward direction), PULSE lag DIR(CW: reverse direction) – Non-used of IP

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return pulse output method setting on a specific axis set with AxmMotSetPulseOutMethod.

Example

C++

```
// Set to Two pulse method on axis 0.
AxmMotSetPulseOutMethod(0, TwoCwCcwHigh);

// Verify pulse output method on axis 0.
DWORD Method;
AxmMotGetPulseOutMethod (0, &Method);
```

Visual Basic

```
' Set to Two pulse method on axis 0.
AxmMotSetPulseOutMethod 0, TwoCwCcwHigh

' Verify pulse output method on axis 0.
Dim Method As Long
AxmMotGetPulseOutMethod 0, Method
```

Delphi

```
{ Verify pulse output method on axis 0. }
var
  Method: DWord;

begin
  { Set to Two pulse method on axis 0. }
  AxmMotSetPulseOutMethod(0, TwoCwCcwHigh);
  AxmMotGetPulseOutMethod(0, @Method);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll , AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotSetEncInputMethod

Purpose

Set Encoder input method on a specific axis.

Format

C++

```
DWORD AxmMotSetEncInputMethod(long lAxisNo, DWORD uMethod);
```

Visual Basic

```
Function AxmMotSetEncInputMethod(ByVal lAxisNo As Long, ByVal uMethod
As Long) As Long
```

Delphi

```
function AxmMotSetEncInputMethod(lAxisNo : LongInt; uMethod : DWord) :
DWord; stdcall;
```

Parameters

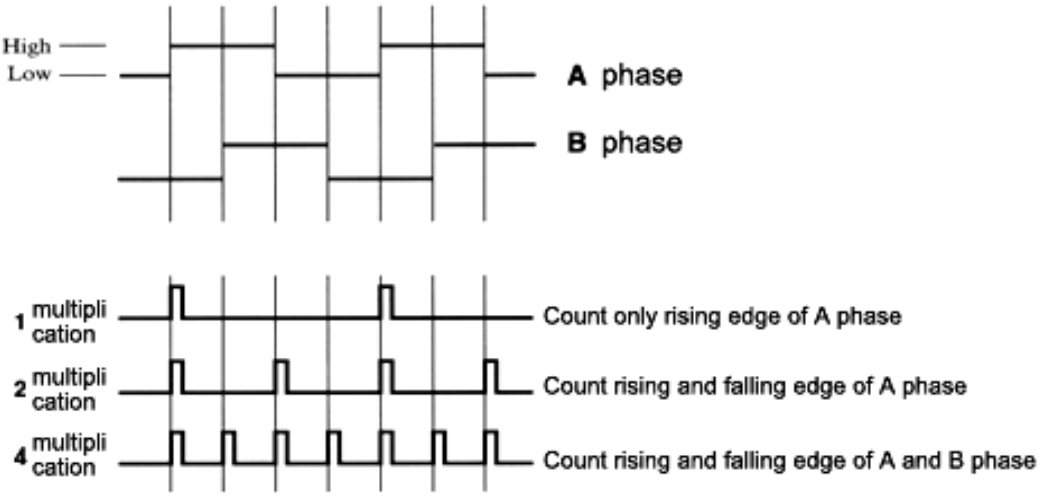
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uMethod	[03h] Encoder input method : AXT_MOTION_EXTERNAL_COUNTER_INTPUT_DEF – [00h] Forward direction Up / Down – [01h] Forward direction 1 multiplication – [02h] Forward direction 2 multiplication – [03h] Forward direction 4 multiplication – [04h] Reverse direction Up / Down – [05h] Reverse direction 1 multiplication – [06h] Reverse direction 2 multiplication – [07h] Reverse direction 4 multiplication

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Encoder input method has 8 methods, and most of driver uses 4 multiplication.



Example

C++

```
// Set Encoder input method to 4 multiplication method on axis 0.
AxmMotSetEncInputMethod(0, ObverseSqr4Mode);

// Verify Encoder input method on axis 0.
DWORD Method;
AxmMotGetEncInputMethod(0, &Method);
```

Visual Basic

```
' Set Encoder input method to 4 multiplication method on axis 0.
AxmMotSetEncInputMethod 0, ObverseSqr4Mode

' Verify Encoder input method on axis 0.
Dim Method As Long
AxmMotGetEncInputMethod 0, Method
```

Delphi

```
{ Verify Encoder input method on axis 0. }
var
  Method: DWord;

begin
  { Set Encoder input method to 4 multiplication method on axis 0. }
  AxmMotSetEncInputMethod(0, ObverseSqr4Mode);
  AxmMotGetEncInputMethod(0, @Method);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotGetEncInputMethod,

AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotGetEncInputMethod

Purpose

Return Encoder input method setting on a specific axis.

Format

C++

```
DWORD AxmMotGetEncInputMethod(long lAxisNo, DWORD *upMethod);
```

Visual Basic

```
Function AxmMotGetEncInputMethod(ByVal lAxisNo As Long, ByRef
upMethod As Long) As Long
```

Delphi

```
function AxmMotGetEncInputMethod(lAxisNo : LongInt; upMethod :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upMethod	[03h] Encoder input method: AXT_MOTION_EXTERNAL_COUNTER_INTPUT_DEF – [00h] Forward direction Up / Down – [01h] Forward direction 1 multiplication – [02h] Forward direction 2 multiplication – [03h] Forward direction 4 multiplication – [04h] Reverse direction Up / Down – [05h] Reverse direction 1 multiplication – [06h] Reverse direction 2 multiplication – [07h] Reverse direction 4 multiplication

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return encoder input method setting on a specific axis set with AxmMotSetEncInputMethod.

Example

C++

```
// Set Encoder input method to 4 multiplication method on axis 0.
AxmMotSetEncInputMethod(0, ObverseSqr4Mode);
// Verify Encoder input method on axis 0.
DWORD Method;
AxmMotGetEncInputMethod (0, &Method);
```

Visual Basic

```
' Set Encoder input method to 4 multiplication method on axis 0.
AxmMotSetEncInputMethod 0, ObverseSqr4Mode
' Verify Encoder input method on axis 0.
Dim Method As Long
AxmMotGetEncInputMethod 0, Method
```

Delphi

```
{ Verify Encoder input method on axis 0. }
var
  Method: DWord;

begin
  { Set Encoder input method to 4 multiplication method on axis 0. }
  AxmMotSetEncInputMethod(0, ObverseSqr4Mode);
  AxmMotGetEncInputMethod(0, @Method);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotSetMoveUnitPerPulse

Purpose

Set output pulse by moving distance on a specific axis.

Format

C++

```
DWORD AxmMotSetMoveUnitPerPulse(long lAxisNo, double dUnit, long lPulse);
```

Visual Basic

```
Function AxmMotSetMoveUnitPerPulse(ByVal lAxisNo As Long, ByVal dUnit As Double, ByVal lPulse As Long) As Long
```

Delphi

```
function AxmMotSetMoveUnitPerPulse(lAxisNo : LongInt; dUnit : Double; lPulse : LongInt) : DWord; stdcall;
```

Parameters

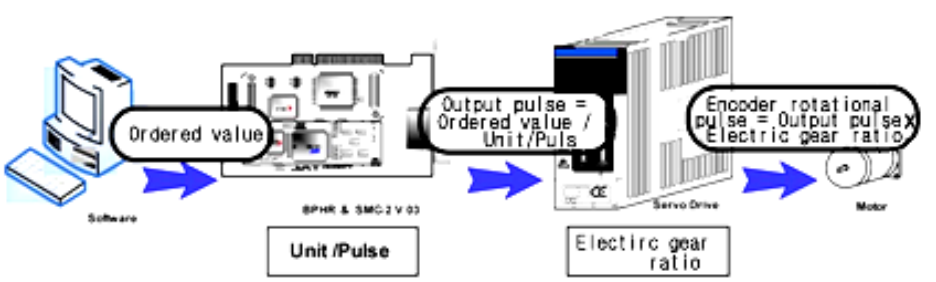
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dUnit	[1.0] Unit moving distance
[>>]lPulse	[1] Unit pulse value

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4254] AXT_RT_MOTION_MOVE_UNIT_IS_ZERO : Unit of the motion values is set as zero
* See error code Table for more information on status error codes

Description

Basically, ordered value of moving distance that user assigns on the commands of library takes two steps of unit conversion as the picture below.



If moving amount is inputted on the software, number of output pulse of motion controller by parameter which is unit/pulse and already set is determined. And this output pulse is converted again by parameter which is electric gear ration set on the servo drive, and the drive pulse that drives actual motor is determined.

Unit/pulse is a parameter that is to assign unit of values given by running the motion drive API, and the API which sets this value is AxmMotSetMoveUnitPerPulse. In other words, this means if it is set 10 on unit and 100 on pulse, when 10 is commanded on the motion drive API, 100 pulses are out to the servo drive.

Unit/pulse is a parameter that is set on the motion controller, but electric gear ratio is a parameter that is set on the servo drive.

[Electric gear ratio] =

[Number of encoder pulse that is generated during 1 rotation of servo motor] /

[Number of output ordered pulse of motion controller during 1 rotation of servo motor]

If 17-bit encoder is used, 131,072 of encoder pulse number can be reduced to 10,000 on the motion controller based on above. That is, what a rotational accuracy of servo motor encoder lowers by an accuracy of mechanical part is an electric gear ratio.

For example, in case of 20mm ball screw, 1/2 of electric gear ratio, and 10000 of pulse number is required for 1 rotation of motor, if it is set that lPulse equals 10000 and dUnit equals 10, pulse number that outputs per moving distance of 1mm is set by 1000.



Actual pulse output velocity for logical unit velocity sets Pulses/sec. Logical unit velocity means unit amount for velocity and acceleration used in a velocity specific API. But, unit of velocity may be different by user characteristic, i.e. it may be easy to indicate velocity unit with RPM for one user, and

is easy to indicate with m/sec for another user. If 3600-pulse is required pulse number for 1 rotation, input 3600 into argument lPulse. 3600/60 which is 60PPS is set internally because RPM is rotation number per minute, therefore, divided by 60. Pulse to output will be 3600-pulse per minute.

If need to set setting velocity unit to RPM(Revolution Per Minute).

ex> rpm calculation:

4500 rpm ?

If unit/ pulse = 1 : 1, it will be pulse/ sec

If need to set to 4500 rpm, 4500 / 60 sec : 75 rpm/ 1 sec

Need to know motor has which pulse per rotation. This will be known search Z phase in Encoder.

If 1 rotation:1800 pulse, it needs 75 x 1800 = 135000 pulse.

Make to move inputting Unit = 1, Pulse = 1800 in AxmMotSetMoveUnitPerPulse.

Example

C++

```
// Set pulse number outputted by moving distance on axis 0 to 100.
AxmMotSetMoveUnitPerPulse(0, 1, 100);

// Return pulse number outputted by moving distance on axis 0.
long lpPulse;
double dpUnit;

AxmMotGetMoveUnitPerPulse (0, &dpUnit, &lpPulse);
```

Visual Basic

```
' Set pulse number outputted by moving distance on axis 0 to 100.
AxmMotSetMoveUnitPerPulse 0, 1, 100

' Return pulse number outputted by moving distance on axis 0.
Dim lpPulse As Long
Dim dpUnit As Double
AxmMotGetMoveUnitPerPulse 0, dpUnit, lpPulse
```

Delphi

```
{ Return pulse number outputted by moving distance on axis 0. }
var
  lpPulse: LongInt;
  dpUnit: Double;

begin
  { Set pulse number outputted by moving distance on axis 0 to 100. }
  AxmMotSetMoveUnitPerPulse(0, 1, 100);
  AxmMotGetMoveUnitPerPulse(0, @dpUnit, @lpPulse);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotGetMoveUnitPerPulse

Purpose

Return pulse outputted by moving distance on a specific axis.

Format

C++

```
DWORD AxmMotGetMoveUnitPerPulse(long lAxisNo, double *dpUnit, long
*lpPulse);
```

Visual Basic

```
Function AxmMotGetMoveUnitPerPulse(ByVal lAxisNo As Long, ByRef
dpUnit As Double, ByRef lpPulse As Long) As Long
```

Delphi

```
function AxmMotGetMoveUnitPerPulse(lAxisNo : LongInt; dpUnit :
PDouble; lpPulse : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpUnit	[1.0] Unit moving distance
[<<]lpPulse	[1] Unit pulse value

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return pulse outputted by moving distance on a specific axis set with AxmMotSetMoveUnitPerPulse

*Note: When only required data are needed to bring, it is able to use with input NULL in no required variable.

Example

C++

```
// Set pulse number outputted by moving distance on axis 0 to 100.
AxmMotSetMoveUnitPerPulse(0, 1, 100);

// Return pulse number outputted by moving distance on axis 0.
long lpPulse;
double dpUnit;

AxmMotGetMoveUnitPerPulse (0, &dpUnit, &lpPulse);
```

Visual Basic

```
' Set pulse number outputted by moving distance on axis 0 to 100.
AxmMotSetMoveUnitPerPulse 0, 1, 100

' Return pulse number outputted by moving distance on axis 0.
Dim lpPulse As Long
Dim dpUnit As Double
AxmMotGetMoveUnitPerPulse 0, dpUnit, lpPulse
```

Delphi

```
{ Return pulse number outputted by moving distance on axis 0. }
var
  lpPulse: LongInt;
  dpUnit: Double;

begin
  { Set pulse number outputted by moving distance on axis 0 to 100. }
  AxmMotSetMoveUnitPerPulse(0, 1, 100);
  AxmMotGetMoveUnitPerPulse(0, @dpUnit, @lpPulse);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotSetDecelMode

Purpose

Set deceleration start point search method on a specific axis.

Format

C++

```
DWORD AxmMotSetDecelMode(long lAxisNo, DWORD uMethod);
```

Visual Basic

```
Function AxmMotSetDecelMode(ByVal lAxisNo As Long, ByVal uMethod As
Long) As Long
```

Delphi

```
function AxmMotSetDecelMode(lAxisNo : LongInt; uMethod : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uMethod	Deceleration start point search method : AXT_MOTION_DETECT_DOWN_START_POINT_DEF – [00h] Automatic search method – [01h] Rest search method

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

There are 2 search methods; automatic search (Autodetect) and rest search method (RestPulse). Initial value is set to automatic one.

Example

C++

```
// Set deceleration start point search method on axis 0.
AxmMotSetDecelMode(0, AutoDetect);
// Verify deceleration start point search method on axis 0.
DWORD Method;
AxmMotGetDecelMode(0, &Method);
```

Visual Basic

```
' Set deceleration start point search method on axis 0.
AxmMotSetDecelMode 0, AutoDetect
```

```
' Verify deceleration start point search method on axis 0.
Dim Method As Long
AxmMotGetDecelMode 0, Method
```

Delphi

```
{ Verify deceleration start point search method on axis 0. }
var
  Method: DWord;

begin
  { Set deceleration start point search method on axis 0. }
  AxmMotSetDecelMode(0, AutoDetect);
  AxmMotGetDecelMode(0, @Method);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotGetDecelMode, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotGetDecelMode

Purpose

Return deceleration start point search method on a specific axis.

Format

C++

```
DWORD AxmMotGetDecelMode(long lAxisNo, DWORD *upMethod);
```

Visual Basic

```
Function AxmMotGetDecelMode(ByVal lAxisNo As Long, ByRef upMethod As
Long) As Long
```

Delphi

```
function AxmMotGetDecelMode(lAxisNo : LongInt; upMethod : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upMethod	Deceleration start point search method : AXT_MOTION_DETECT_DOWN_START_POINT_DEF – [00h] Automatic search method – [01h] Rest search method

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return deceleration start point search method on a specific axis set with AxmMotSetDecelMode.

Example

C++

```
// Set search start point deceleration method.
AxmMotSetDecelMode(0, AutoDetect);

// Verify deceleration start point search method on axis 0.
DWORD Method;
AxmMotGetDecelMode(0, &Method);
```

Visual Basic

```
' Set search start point deceleration method.
AxmMotSetDecelMode 0, AutoDetect
```

```
' Verify deceleration start point search method on axis 0.
Dim Method As Long
AxmMotGetDecelMode 0, Method
```

Delphi

```
{ Verify deceleration start point search method on axis 0. }
var
  Method: DWord;

begin
  { Set search start point deceleration method. }
  AxmMotSetDecelMode(0, AutoDetect);
  AxmMotGetDecelMode(0, @Method);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetRemainPulse, AxmMotGetRemainPulse

AxmMotSetRemainPulse

Purpose

Set remaining pulse on manual deceleration mode on a specific axis.

Format

C++

```
DWORD AxmMotSetRemainPulse(long lAxisNo, DWORD uData);
```

Visual Basic

```
Function AxmMotSetRemainPulse(ByVal lAxisNo As Long, ByVal uData As
Long) As Long
```

Delphi

```
function AxmMotSetRemainPulse(lAxisNo : LongInt; uData : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uData	Set remaining pulse

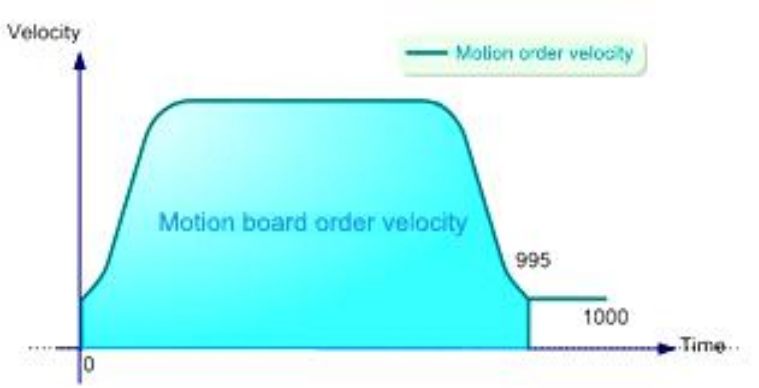
Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

When the deceleration mode is set to AutoDetect (0) by the AxmMotSetDecelMode function, this function is used to set the drive pulses after reducing the velocity of the motion.

For example, when the system is configured to move by 1000 pulses with 5 of remaining pulses using the AxmMoveStartPos function, the system will decelerate to the velocity configured by AxmMotSetMinVel function from the 995th pulse, and decelerate for the period of 5 pulses by keeping the specified velocity. This function is used to maintain deceleration at the end.



When the deceleration mode is set to RestPulse (1) by the AxmMotSetDecelMode function, this function is to set the number of remaining pulses for the deceleration segment.

For example, when the system is configured to move by 1000 pulses with 100 pulses required during acceleration, the operation is as followings depending on the configuration by the AxmMotSetRemainPulse function.

When the remaining pulse setting value is smaller than 100 : The drive will be completed before decelerating to the start stop velocity.

When the remaining pulse setting value is bigger than 100 : The drive will be completed after decelerating to start stop velocity and moving by the value bigger than 100.

Note) Since input value is pulse unit, position value (unit) should be converted to pulse unit by using the value set by AxmMotSetMoveUnitPerPulse.

Example

C++

```
// Set remaining pulse on manual deceleration mode on axis 0.
AxmMotSetRemainPulse(0, 5);
AxmMoveStartPos(0, 1000, 100, 200, 200);
// Verify remaining pulse on manual deceleration mode on axis 0.
DWORD Data;
AxmMotGetRemainPulse(0, &Data);
```

Visual Basic

```
' Set remaining pulse on manual deceleration mode on axis 0.
AxmMotSetRemainPulse 0, 5
AxmMoveStartPos 0, 1000, 100, 200, 200
' Verify remaining pulse on manual deceleration mode on axis 0.
Dim Data As Long
AxmMotGetRemainPulse 0, Data
```

Delphi

```
{ Verify remaining pulse on manual deceleration mode on axis 0. }
var
   Data: DWord;

begin
   { Set remaining pulse on manual deceleration mode on axis  0. }
   AxmMotSetRemainPulse(0, 5);
   AxmMoveStartPos(0, 1000, 100, 200, 200);
   AxmMotGetRemainPulse(0, @Data);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotGetRemainPulse

AxmMotGetRemainPulse

Purpose

Return remaining pulse on manual deceleration mode on a specific axis.

Format

C++

```
DWORD AxmMotGetRemainPulse(long lAxisNo, DWORD *upData);
```

Visual Basic

```
Function AxmMotGetRemainPulse(ByVal lAxisNo As Long, ByRef upData As
Long) As Long
```

Delphi

```
function AxmMotGetRemainPulse(lAxisNo : LongInt; upData : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upData	Set remaining pulse

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return remaining pulse on manual deceleration mode on a specific axis using AxmMotSetRemainPulse API.

Example

C++

```
// Set remaining pulse on manual deceleration mode on axis 0.
AxmMotSetRemainPulse(0, 1000);

// Verify remaining pulse on manual deceleration mode on axis 0.
DWORD Data;
AxmMotGetRemainPulse (0, &Data);
```

Visual Basic

```
' Set remaining pulse on manual deceleration mode on axis 0.
AxmMotSetRemainPulse 0, 1000
```

```
' Verify remaining pulse on manual deceleration mode on axis 0.
Dim Data As Long
AxmMotGetRemainPulse 0, Data
```

Delphi

```
{ Verify remaining pulse on manual deceleration mode on axis 0. }
var
   Data: DWord;

begin
   { Set remaining pulse on manual deceleration mode on axis 0. }
   AxmMotSetRemainPulse(0, 1000);
   AxmMotSetRemainPulse(0, @Data);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetPulseOutMethod, AxmMotGetPulseOutMethod, AxmMotSetEncInputMethod, AxmMotGetEncInputMethod, AxmMotSetMoveUnitPerPulse, AxmMotGetMoveUnitPerPulse, AxmMotSetDecelMode, AxmMotGetDecelMode, AxmMotSetRemainPulse

AxmMotSetMaxVel

Purpose

Limit maximum drive velocity of all drive API to assigned velocity.

Format

C++

```
DWORD AxmMotSetMaxVel(long lAxisNo, double dVel);
```

Visual Basic

```
Function AxmMotSetMaxVel(ByVal lAxisNo As Long, ByVal dVel As Doubel)
As Long
```

Delphi

```
function AxmMotSetMaxVel(lAxisNo : LongInt; dVel : Double) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dVel	[700000.0] Maximum drive velocity (unit of drive velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4113] AXT_RT_MOTION_INVALID_VELOCITY : Motion error occurred since the motion drive velocity value is set as zero
[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion
* See error code Table for more information on status error codes

Description

This function limits all input velocity values of drive functions for the specified axis to the velocity set by the AxmMotSetMaxVel function.

Note: The velocity can be set to up to 4MPPS for SMV-V203 product, and up to 10 MPPS for PCI-Nx04 products. The input velocity to the AxmMotSetMaxVel function is applied differently depending on the Unit/Pulse value set by AxmMotSetMoveUnitPerPulse API. For example, the maximum value that can be set is (4M/100) for SMC-2V03 and (10M/100) for PCI-Nx04 if the Unit/Pulse value is 0.01.

Example

C++

```
// Set maximum velocity of 1000 for constant speed drive on axis 0.
AxmMotSetMaxVel (0, 1000);

// Verify constant speed drive maximum velocity on axis 0.
```

```
double Vel;
AxmMotGetMaxVel (0, &Vel);
```

Visual Basic

```
' Set maximum velocity of 1000 for constant speed drive on axis 0.
AxmMotSetMaxVel 0, 1000

' Verify constant speed drive maximum velocity on axis 0.
Dim Vel As Double
AxmMotGetMaxVel 0, Vel
```

Delphi

```
{ Verify constant speed drive maximum velocity on axis 0. }
var
  Vel: Double;

begin
  { Set maximum velocity of 1000 for constant speed drive on axis 0. }
  AxmMotSetMaxVel (0, 1000);
  AxmMotGetMaxVel (0, @Vel);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotGetMaxVel

Purpose

Return maximum velocity in constant speed drive API on a specific axis.

Format

C++

```
DWORD AxmMotGetMaxVel(long lAxisNo, double *dpVel);
```

Visual Basic

```
Function AxmMotGetMaxVel(ByVal lAxisNo As Long, ByRef dpVel As Double)
As Long
```

Delphi

```
function AxmMotGetMaxVel(lAxisNo : LongInt; dpVel : PDouble) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpVel	[700000.0] Maximum drive velocity (unit of drive velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return maximum velocity in constant speed drive API on a specific axis set in AxmMotSetMaxVel.

Example

C++

```
// Set maximum velocity of 1000 for constant speed drive on axis 0.
AxmMotSetMaxVel (0, 1000);

// Verify constant speed drive maximum velocity on axis 0.
double Vel;
AxmMotGetMaxVel (0, &Vel);
```

Visual Basic

```
' Set maximum velocity of 1000 for constant speed drive on axis 0.
AxmMotSetMaxVel 0, 1000

' Verify constant speed drive maximum velocity on axis 0.
```

```
Dim Vel As Double
AxmMotGetMaxVel 0, Vel
```

Delphi

```
{ Verify constant speed drive maximum velocity on axis 0. }
var
  Vel: Double;

begin
  { Set maximum velocity of 1000 for constant speed drive on axis 0. }
  AxmMotSetMaxVel(0, 1000);
  AxmMotGetMaxVel(0, @Vel);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotSetAbsRelMode

Purpose

Set moving distance calculation mode on a specific axis.

Format

C++

```
DWORD AxmMotSetAbsRelMode(long lAxisNo, DWORD uAbsRelMode);
```

Visual Basic

```
Function AxmMotSetAbsRelMode(ByVal lAxisNo As Long, ByVal uAbsRelMode
As Long) As Long
```

Delphi

```
function AxmMotSetAbsRelMode(lAxisNo : LongInt; uAbsRelMode : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uAbsRelMode	[00h] Pulse output method : AXT_MOTION_ABSREL_MODE_DEF – [00h] Position drive absolute mode – [01h] Position drive relative mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

If absolute position mode (0) is set, position value used in moving API is to be absolute position value. For example, if current position value is “10” and moves from absolute position mode to “100” position, it will stop at “100” position moving as much as “+90” from the current position.

If relative position mode (1) is set, position value used moving API is to be moving amount to move from the current position. For example, if current position value is “10” and moves from relative position mode to “100” position, it will stop at “110” position moving as much as “+100” from the current position.

Example

C++

```
// Set axis 0 to absolute mode.
AxmMotSetAbsRelMode(0, POS_ABS_MODE);

// Verify moving mode of axis 0.
```

```
DWORD Mode;
AxmMotGetAbsRelMode(0, &Mode);
```

Visual Basic

```
' Set axis 0 to absolute mode.
AxmMotSetAbsRelMode 0, POS_ABS_MODE

' Verify moving mode of axis 0.
Dim Mode As Long
AxmMotGetAbsRelMode 0, Mode
```

Delphi

```
{ Verify moving mode of axis 0. }
var
  Mode : DWord;

begin
  { Set axis 0 to absolute mode. }
  AxmMotSetAbsRelMode(0, POS_ABS_MODE);

  AxmMotGetAbsRelMode (0, @Mode);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotGetAbsRelMode

Purpose

Return moving distance calculation mode which is set on a specific axis.

Format

C++

```
DWORD AxmMotGetAbsRelMode(long lAxisNo, DWORD *upAbsRelMode);
```

Visual Basic

```
Function AxmMotGetAbsRelMode(ByVal lAxisNo As Long, ByRef
upAbsRelMode As Long) As Long
```

Delphi

```
function AxmMotGetAbsRelMode(lAxisNo : LongInt; upAbsRelMode :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upAbsRelMode	[00h] Pulse output method : AXT_MOTION_ABSREL_MODE_DEF – [00h] Position drive absolute mode – [01h] Position drive relative mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return moving distance calculation mode which is set with AxmMotSetAbsRelMode on a specific axis.

Example

C++

```
// Set axis 0 to absolute mode.
AxmMotSetAbsRelMode(0, POS_ABS_MODE);

// Verify moving mode of axis 0.
DWORD Mode;
AxmMotGetAbsRelMode(0, &Mode);
```

Visual Basic

```
' Set axis 0 to absolute mode.
AxmMotSetAbsRelMode 0, POS_ABS_MODE
```

```
' Verify moving mode of axis 0.
Dim Mode As Long
AxmMotGetAbsRelMode 0, Mode
```

Delphi

```
{ Verify moving mode of axis 0. }
var
  Mode : DWord;

begin
  { Set axis 0 to absolute mode. }
  AxmMotSetAbsRelMode(0, POS_ABS_MODE);
  AxmMotGetAbsRelMode (0, @Mode);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotSetProfileMode

Purpose

Set drive velocity profile mode on a specific axis.

Format

C++

```
DWORD AxmMotSetProfileMode(long lAxisNo, DWORD uProfileMode);
```

Visual Basic

```
Function AxmMotSetProfileMode(ByVal lAxisNo As Long, ByVal
uProfileMode As Long) As Long
```

Delphi

```
function AxmMotSetProfileMode(lAxisNo : LongInt; uProfileMode :
DWord) : DWord; stdcall;
```

Parameters

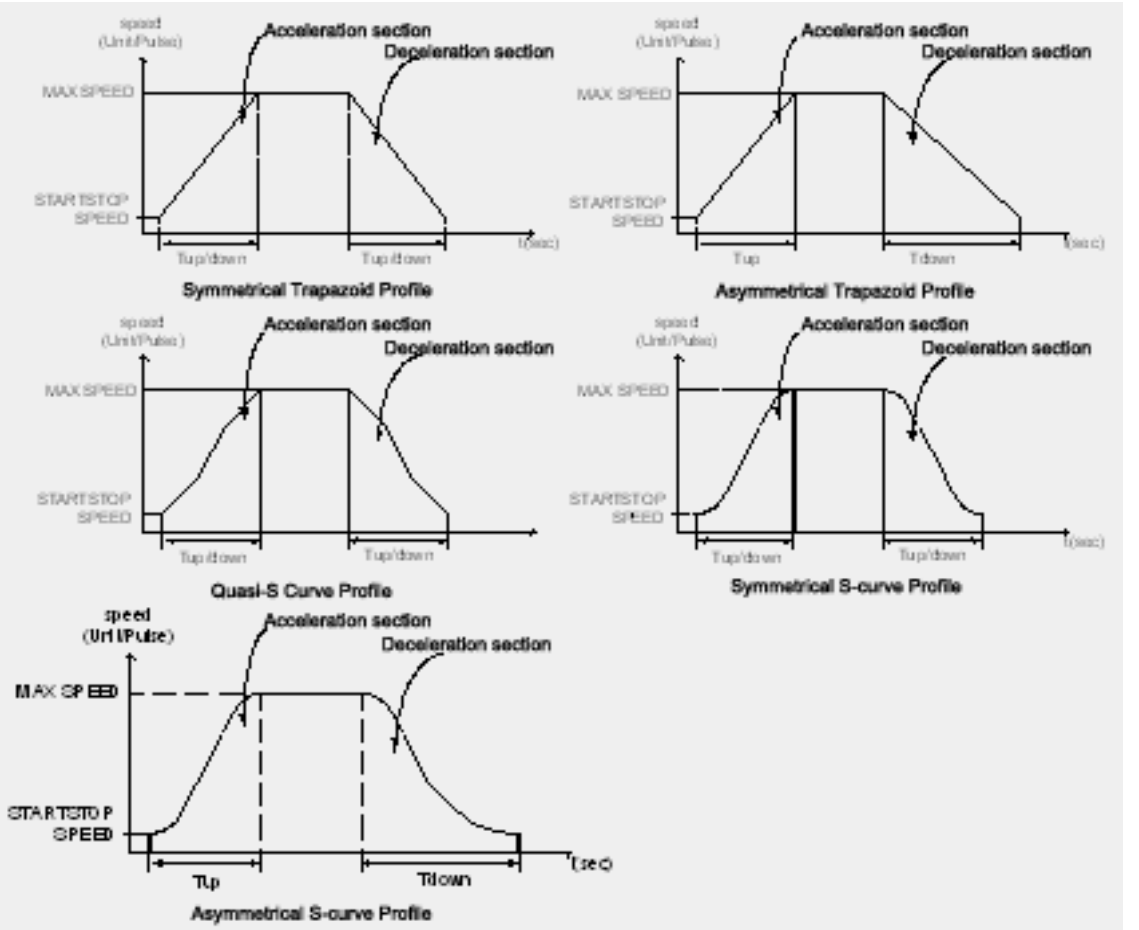
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uProfileMode	[04h] Velocity profile mode : AXT_MOTION_PROFILE_MODE_DEF – [00h] Symmetrical Trapezode – [01h] Asymmetrical Trapezode – [02h] Reserved – [03h] Symmetrical S Curve – [04h] Asymmetrical S Curve

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Velocity profile mode has 5 modes: symmetrical Trapezoid , asymmetrical Trapezoid, Quasi-S Curve mode, symmetrical S-curve and asymmetrical S-curve mode. Graph of each velocity profile mode shows below.



Example

C++

```
// Set velocity profile on axis 0 to symmetrical S-Curve.
AxmMotSetProfileMode (0, 3);

// Verify velocity profile on axis 0.
DWORD uProfile;
AxmMotGetProfileMode (0, &uProfile);
```

Visual Basic

```
' Set velocity profile on axis 0 to symmetrical S-Curve.
AxmMotSetProfileMode 0, 3

' Verify velocity profile on axis 0.
Dim uProfile As Long
AxmMotGetProfileMode 0, uProfile
```

Delphi

```
{ Verify velocity profile on axis 0. }
var
  uProfile : DWord;

begin
  { Set velocity profile on axis 0 to symmetrical S-Curve. }
  AxmMotSetProfileMode (0, 3);
  AxmMotGetProfileMode (0, @uProfile);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotGetProfileMode

Purpose

Return drive velocity profile which is set on a specific axis.

Format

C++

```
DWORD AxmMotGetProfileMode(long lAxisNo, DWORD *upProfileMode);
```

Visual Basic

```
Function AxmMotGetProfileMode(ByVal lAxisNo As Long, ByRef
upProfileMode As Long) As Long
```

Delphi

```
function AxmMotGetProfileMode(lAxisNo : LongInt; upProfileMode :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upProfileMode	[04h] Velocity profile mode : AXT_MOTION_PROFILE_MODE_DEF – [00h] Symmetrical Trapezode – [01h] Asymmetrical Trapezode – [02h] Reserved – [03h] Symmetrical S Curve – [04h] Asymmetrical S Curve

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return drive velocity profile which is set with AxmMotSetProfileMode on a specific axis.

Example

C++

```
// Set velocity profile on axis 0 to symmetrical S-Curve.
AxmMotSetProfileMode (0, SYM_S_CURVE_MODE);

// Verify velocity profile on axis 0.
DWORD uProfile;
AxmMotGetProfileMode (0, &uProfile);
```

Visual Basic

```
' Set velocity profile on axis 0 to symmetrical S-Curve.
AxmMotSetProfileMode 0, SYM_S_CURVE_MODE

' Verify velocity profile on axis 0.
Dim uProfile As Long
AxmMotGetProfileMode 0, uProfile
```

Delphi

```
{ Verify velocity profile on axis 0. }
var
  uProfile : DWord;

begin
  { Set velocity profile on axis 0 to symmetrical S-Curve. }
  AxmMotSetProfileMode (0, SYM_S_CURVE_MODE);
  AxmMotGetProfileMode (0, @uProfile);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotSetAccelUnit

Purpose

Set acceleration unit on a specific axis.

Format

C++

```
DWORD AxmMotSetAccelUnit(long lAxisNo, DWORD uAccelUnit);
```

Visual Basic

```
Function AxmMotSetAccelUnit(ByVal lAxisNo As Long, ByVal uAccelUnit
As Long) As Long
```

Delphi

```
function AxmMotSetAccelUnit(lAxisNo : LongInt; uAccelUnit : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uAccelUnit	Acceleration (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1) : AXT_MOTION_ACC_UNIT_DEF – [00h] unit/sec2 – [01h] sec

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Acceleration on a specific axis can be set to unit/sec2 calculated based on acceleration rate of moving velocity, and be set acceleration time sec (1).

Example

C++

```
// Set acceleration unit of unit/sec2on axis 0.
AxmMotSetAccelUnit (0, UNIT_SEC2);

// Verify acceleration unit on axis 0.
DWORD uAccelUnit;
AxmMotGetAccelUnit (0, &uAccelUnit);
```

Visual Basic

```
' Set acceleration unit of unit/sec2on axis 0.
AxmMotSetAccelUnit 0, UNIT_SEC2

' Verify acceleration unit on axis 0.
Dim uAccelUnit As Long
AxmMotGetAccelUnit 0, uAccelUnit
```

Delphi

```
{ Verify acceleration unit on axis 0. }
var
  uAccelUnit: DWord;

begin
  { Set acceleration unit of unit/sec2on axis 0. }
  AxmMotSetAccelUnit (0, UNIT_SEC2);

  AxmMotGetAccelUnit (0, @uAccelUnit);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotGetAccelUnit

Purpose

Return acceleration unit which is set on a specific axis.

Format

C++

```
DWORD AxmMotGetAccelUnit(long lAxisNo, DWORD *upAccelUnit);
```

Visual Basic

```
Function AxmMotGetAccelUnit(ByVal lAxisNo As Long, ByRef upAccelUnit
As Long) As Long
```

Delphi

```
function AxmMotGetAccelUnit(lAxisNo : LongInt; upAccelUnit : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upAccelUnit	Acceleration (unit of acceleration is PPS[Pulses/sec^2] if Unit/pulse is 1/1) : AXT_MOTION_ACC_UNIT_DEF – [00h] unit/sec2 – [01h] sec

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return acceleration unit which is set with AxmMotSetAccelUnit on a specific axis.

Example

C++

```
// Set acceleration unit of unit/sec on axis 0.
AxmMotSetAccelUnit (0, UNIT_SEC2);

// Verify acceleration unit on axis 0.
DWORD uAccelUnit;
AxmMotGetAccelUnit (0, &uAccelUnit);
```

Visual Basic

```
' Set acceleration unit of unit/sec on axis 0.
AxmMotSetAccelUnit 0, UNIT_SEC2
```

```
' Verify acceleration unit on axis 0.
Dim uAccelUnit As Long
AxmMotGetAccelUnit 0, uAccelUnit
```

Delphi

```
{ Verify acceleration unit on axis 0. }
var
  uAccelUnit: DWord;

begin
  { Set acceleration unit of unit/sec on axis 0. }
  AxmMotSetAccelUnit (0, UNIT_SEC2);

  AxmMotGetAccelUnit (0, @uAccelUnit);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotSetMinVel

Purpose

Set initial velocity on a specific axis.

Format

C++

```
DWORD AxmMotSetMinVel(long lAxisNo, double dMinVelocity);
```

Visual Basic

```
Function AxmMotSetMinVel(ByVal lAxisNo As Long, ByVal dMinVelocity As
Double) As Long
```

Delphi

```
function AxmMotSetMinVel(lAxisNo : LongInt; dMinVelocity : Double) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dMinVelocity	[1.0] Initial velocity (unit of initial velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Minimum velocity means start velocity of acceleration and end velocity of deceleration. If the minimum velocity is set to higher than working velocity, the system starts with the existing minimum velocity and reaches to the working velocity, and then moves with the configured minimum velocity without acceleration. The function is not needed except special cases, and the default value of minimum velocity is 1. Deviation phenomenon could be avoided by setting the minimum velocity in case of using stepping motors.

Note: If the minimum velocity is set to lower than the value of Unit/Pulse value, minimum velocity will be set to the value of Unit/Pulse since minimum unit is set to the value of Unit/Pulse.

Example

C++

```
// Set minimum velocity to 1 on axis 0.
AxmMotSetMinVel(0, 1);

// Verify minimum velocity on axis 0.
double MinVelocity;
```

```
AxmMotGetMinVel(0, &MinVelocity);
```

Visual Basic

```
' Set minimum velocity to 1 on axis 0.
AxmMotSetMinVel 0, 1

' Verify minimum velocity on axis 0.
Dim MinVelocity As Double
AxmMotGetMinVel 0, MinVelocity
```

Delphi

```
{ Verify minimum velocity on axis 0. }
var
  MinVelocity: Double;

begin
  { Set minimum velocity to 1 on axis 0. }
  AxmMotSetMinVel(0, 1);

  AxmMotGetMinVel(0, @MinVelocity);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotGetMinVel

Purpose

Return initial velocity on a specific axis.

Format

C++

```
DWORD AxmMotGetMinVel(long lAxisNo, double *dpMinVelocity);
```

Visual Basic

```
Function AxmMotGetMinVel(ByVal lAxisNo As Long, ByRef dpMinVelocity
As Double) As Long
```

Delphi

```
function AxmMotGetMinVel(lAxisNo : LongInt; dpMinVelocity : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpMinVelocity	[1.0] Initial velocity (unit of initial velocity is PPS[Pulses/sec] if Unit/pulse is 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return initial velocity set with AxmMotSetMinVel on a specific axis.

Example

C++

```
// Set initial velocity to 1 on axis 0.
AxmMotSetMinVel(0, 1);

// Verify initial velocity on axis 0.
double MinVelocity;
AxmMotGetMinVel(0, &MinVelocity);
```

Visual Basic

```
' Set initial velocity to 1 on axis 0.
AxmMotSetMinVel 0, 1

' Verify initial velocity on axis 0.
```

```
Dim MinVelocity As Double
AxmMotGetMinVel 0, MinVelocity
```

Delphi

```
{ Verify initial velocity on axis 0. }
var
  MinVelocity: Double;

begin
  { Set initial velocity to 1 on axis 0. }
  AxmMotSetMinVel(0, 1);

  AxmMotGetMinVel(0, @MinVelocity);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotSetAccelJerk

Purpose

Set acceleration jerk value on a specific axis.

Format

C++

```
DWORD AxmMotSetAccelJerk(long lAxisNo, double dAccelJerk);
```

Visual Basic

```
Function AxmMotSetAccelJerk(ByVal lAxisNo As Long, ByVal dAccelJerk
As Double) As Long
```

Delphi

```
function AxmMotSetAccelJerk(lAxisNo : LongInt; dAccelJerk : Double) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dAccelJerk	Accelerating jerk value, unit is %

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Unit is %. If it drives with S-Curve velocity profile mode, it sets section to drive with S-Curve to % between maximum velocity and start velocity during acceleration.

Example

C++

```
// Set jerk value on axis 0.
AxmMotSetAccelJerk(0, 20);
AxmMotSetDecelJerk(0, 20);

// Verify jerk value on axis 0.
double dAccelJerk, dDecelJerk;
AxmMotGetAccelJerk(0, &dAccelJerk);
AxmMotGetDecelJerk(0, &dDecelJerk);
```

Visual Basic

```
' Set jerk value on axis 0.
AxmMotSetAccelJerk 0, 20
AxmMotSetDecelJerk 0, 20

' Verify jerk value on axis 0.
Dim dAccelJerk , dDecelJerk As Double
AxmMotGetAccelJerk 0, dAccelJerk
AxmMotGetDecelJerk 0, dDecelJerk
```

Delphi

```
{ Verify jerk value on axis 0. }
var
  dAccelJerk, dDecelJerk: Double;

begin
  { Set jerk value on axis 0. }
  AxmMotSetAccelJerk(0, 20);
  AxmMotSetDecelJerk(0, 20);

  AxmMotGetAccelJerk(0, @dAccelJerk);
  AxmMotGetDecelJerk(0, @dDecelJerk);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotGetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotGetAccelJerk

Purpose

Return acceleration jerk value which is set on a specific axis.

Format

C++

```
DWORD AxmMotGetAccelJerk(long lAxisNo, double *dpAccelJerk);
```

Visual Basic

```
Function AxmMotGetAccelJerk(ByVal lAxisNo As Long, ByRef dpAccelJerk
As Double) As Long
```

Delphi

```
function AxmMotGetAccelJerk(lAxisNo : LongInt; dpAccelJerk :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpAccelJerk	Accelerating jerk value, unit is %

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Unit is %. If it drives with S-Curve velocity profile mode, it sets section to drive with S-Curve to % between maximum velocity and start velocity during acceleration.

Example

C++

```
// Set jerk value on axis 0.
AxmMotSetAccelJerk(0, 20);
AxmMotSetDecelJerk(0, 20);

// Verify jerk value on axis 0.
double dAccelJerk, dDecelJerk;
AxmMotGetAccelJerk(0, &dAccelJerk);
AxmMotGetDecelJerk(0, &dDecelJerk);
```

Visual Basic

```
' Set jerk value on axis 0.
AxmMotSetAccelJerk 0, 20
AxmMotSetDecelJerk 0, 20

' Verify jerk value on axis 0.
Dim dAccelJerk , dDecelJerk As Double
AxmMotGetAccelJerk 0, dAccelJerk
AxmMotGetDecelJerk 0, dDecelJerk
```

Delphi

```
{ Verify jerk value on axis 0. }
var
  dAccelJerk, dDecelJerk: Double;

begin
  { Set jerk value on axis 0. }
  AxmMotSetAccelJerk(0, 20);
  AxmMotSetDecelJerk(0, 20);

  AxmMotGetAccelJerk(0, @dAccelJerk);
  AxmMotGetDecelJerk(0, @dDecelJerk);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotSetDecelJerk, AxmMotGetDecelJerk

AxmMotSetDecelJerk

Purpose

Set deceleration jerk value on a specific axis.

Format

C++

```
DWORD AxmMotSetDecelJerk(long lAxisNo, double dDecelJerk);
```

Visual Basic

```
Function AxmMotSetDecelJerk(ByVal lAxisNo As Long, ByVal dDecelJerk
As Double) As Long
```

Delphi

```
function AxmMotSetDecelJerk(lAxisNo : LongInt; dDecelJerk : Double) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dDecelJerk	Decelerating jerk value, unit is %

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Unit is %. If it drives with S-Curve velocity profile mode, it sets section to drive with S-Curve to % between maximum velocity and start velocity during acceleration.

Example

C++

```
// Set jerk value on axis 0.
AxmMotSetAccelJerk(0, 20);
AxmMotSetDecelJerk(0, 20);

// Verify jerk value on axis 0.
double dAccelJerk, dDecelJerk;
AxmMotGetAccelJerk(0, &dAccelJerk);
AxmMotGetDecelJerk(0, &dDecelJerk);
```

Visual Basic

```
' Set jerk value on axis 0.
AxmMotSetAccelJerk 0, 20
AxmMotSetDecelJerk 0, 20

' Verify jerk value on axis 0.
Dim dAccelJerk , dDecelJerk As Double
AxmMotGetAccelJerk 0, dAccelJerk
AxmMotGetDecelJerk 0, dDecelJerk
```

Delphi

```
{ Verify jerk value on axis 0. }
var
  dAccelJerk, dDecelJerk: Double;

begin
  { Set jerk value on axis 0. }
  AxmMotSetAccelJerk(0, 20);
  AxmMotSetDecelJerk(0, 20);

  AxmMotGetAccelJerk(0, @dAccelJerk);
  AxmMotGetDecelJerk(0, @dDecelJerk);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotGetDecelJerk

AxmMotGetDecelJerk

Purpose

Return deceleration jerk value which is set on a specific axis.

Format

C++

```
DWORD AxmMotGetDecelJerk(long lAxisNo, double *dpDecelJerk);
```

Visual Basic

```
Function AxmMotGetDecelJerk(ByVal lAxisNo As Long, ByRef dpDecelJerk
As Double) As Long
```

Delphi

```
function AxmMotGetDecelJerk(lAxisNo : LongInt; dpDecelJerk :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpDecelJerk	Decelerating jerk value, unit is %

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Unit is %. If it drives with S-Curve velocity profile mode, it sets section to drive with S-Curve to % between maximum velocity and start velocity during acceleration.

Example

C++

```
// Set jerk value on axis 0.
AxmMotSetAccelJerk(0, 20);
AxmMotSetDecelJerk(0, 20);

// Verify jerk value on axis 0.
double dAccelJerk, dDecelJerk;
AxmMotGetAccelJerk(0, &dAccelJerk);
AxmMotGetDecelJerk(0, &dDecelJerk);
```

Visual Basic

```
' Set jerk value on axis 0.
AxmMotSetAccelJerk 0, 20
AxmMotSetDecelJerk 0, 20

' Verify jerk value on axis 0.
Dim dAccelJerk , dDecelJerk As Double
AxmMotGetAccelJerk 0, dAccelJerk
AxmMotGetDecelJerk 0, dDecelJerk
```

Delphi

```
{ Verify jerk value on axis 0. }
var
  dAccelJerk, dDecelJerk: Double;

begin
  { Set jerk value on axis 0. }
  AxmMotSetAccelJerk(0, 20);
  AxmMotSetDecelJerk(0, 20);

  AxmMotGetAccelJerk(0, @dAccelJerk);
  AxmMotGetDecelJerk(0, @dDecelJerk);
end;
```

See Also

AxmMotLoadParaAll, AxmMotSaveParaAll, AxmMotGetParaLoad, AxmMotSetParaLoad, AxmMotSetMaxVel, AxmMotGetMaxVel, AxmMotSetAbsRelMode, AxmMotGetAbsRelMode, AxmMotSetProfileMode, AxmMotGetProfileMode, AxmMotSetAccelUnit, AxmMotGetAccelUnit, AxmMotSetMinVel, AxmMotGetMinVel, AxmMotSetAccelJerk, AxmMotGetAccelJerk, AxmMotSetDecelJerk

Input And Output Level Setting API

In this unit, level by external sensors, universal input and output, and level API related to servo drive will be introduced. If level is set incorrectly, motor may not operate. Therefore, set level correctly and start drive.

Function	Description
AxmSignalSetZphaseLevel	Sets Z phase level on a specific axis.
AxmSignalGetZphaseLevel	Returns Z phase level on a specific axis.
AxmSignalSetServoOnLevel	Sets output level of Amp-On signal on a specific axis.
AxmSignalGetServoOnLevel	Returns output level of Amp-On signal on a specific axis.
AxmSignalSetServoAlarmResetLevel	Sets output level of Servo-Alarm Reset signal on a specific axis.
AxmSignalGetServoAlarmResetLevel	Returns output level of Servo-Alarm Reset signal on a specific axis.
AxmSignalSetInpos	Sets whether Inpositon signal is used and signal input level on a specific axis.
AxmSignalGetInpos	Returns whether Inpositon signal is used and signal input level on a specific axis.
AxmSignalReadInpos	Returns Inpositon signal input state on a specific axis.
AxmSignalSetServoAlarm	Sets whether emergency stop is used and signal input level during alarm signal input on a specific axis.
AxmSignalGetServoAlarm	Returns whether emergency stop is used and signal input level during alarm signal input on a specific axis.
AxmSignalReadServoAlarm	Returns input level of alarm signal on a specific axis.
AxmSignalSetLimit	Sets whether end limit sensor is used and input level of signal on a specific axis. Available to set for decelerate stop or emergency stop during end limit sensor signal input.
AxmSignalGetLimit	Returns use of end limit sensor and signal input level and stop mode during signal input on a specific axis.
AxmSignalReadLimit	Returns input state of end limit sensor on a specific axis.
AxmSignalSetSoftLimit	Sets use of Software limit, count to use, and stop method on a specific axis.
AxmSignalGetSoftLimit	Returns use of Software limit, count to use, and stop method on a specific axis.
AxmSignalSetStop	Sets stop method (emergency stop/decelerate stop) or use of emergency stop signal.
AxmSignalGetStop	Returns stop method (emergency stop/decelerate stop) or use of emergency stop signal.
AxmSignalReadStop	Returns input state of emergency stop signal.

AxmSignalSetZphaseLevel

Purpose

Set output level of Z phase input signal on a specific axis.

Format

C++

```
DWORD AxmSignalSetZphaseLevel(long lAxisNo, DWORD uLevel);
```

Visual Basic

```
Function AxmSignalSetZphaseLevel(ByVal lAxisNo As Long, ByVal uLevel
As Long) As Long
```

Delphi

```
function AxmSignalSetZphaseLevel(lAxisNo : LongInt; uLevel : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uLevel	[01h] Active Level of Z phase signal input : AXT_MOTION_LEVEL_MODE_DEF – [00h] B contact (Normal Close) – [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Set Z phase input level on an axis assigned by user.

Z phase signal means zero pulse signal of encoder. CAMC-IP, QI chip uses universal input 1 as Z phase of Encoder. If this signal is used, more accurate home all work can be executed when ORG signal is used.

If read Bit 8 using AxmStatusReadMechanical API to read Home Level, it can be verified if level is changed.

Example

C++

```
// Set Z phase level on axis 0.
AxmSignalSetZphaseLevel(0, HIGH);

// Verify Z phase level value on axis 0.
```

```
DWORD state;

AxmSignalGetZphaseLevel (0, &state);
```

Visual Basic

```
' Set Z phase level on axis 0.
AxmSignalSetZphaseLevel 0, HIGH

' Verify Z phase level value on axis 0.
Dim state As Long

AxmSignalGetZphaseLevel 0, state
```

Delphi

```
{ Verify Z phase level value on axis 0. }
var
  state: DWord;

begin
  { Set Z phase level on axis 0. }
  AxmSignalSetZphaseLevel(0, HIGH);
  AxmSignalGetZphaseLevel(0, @state);
end;
```

See Also

AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetZphaseLevel

Purpose

Return input level of Z phase signal on a specific axis.

Format

C++

```
DWORD AxmSignalGetZphaseLevel(long lAxisNo, DWORD *upLevel);
```

Visual Basic

```
Function AxmSignalGetZphaseLevel(ByVal lAxisNo As Long, ByRef upLevel
As Long) As Long
```

Delphi

```
function AxmSignalGetZphaseLevel(lAxisNo : LongInt; upLevel :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upLevel	[01h] Active Level of Z phase signal input : AXT_MOTION_LEVEL_MODE_DEF – [00h] B contact (Normal Close) – [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return Z phase input level set with 'AxmSignalSetZphaseLevel'.

Example

C++

```
// Set Z phase level on axis 0.
AxmSignalSetZphaseLevel(0, HIGH);

// Verify Z phase level value on axis 0.
DWORD state;

AxmSignalGetZphaseLevel (0, &state);
```

Visual Basic

```
' Set Z phase level on axis 0.
AxmSignalSetZphaseLevel 0, HIGH

' Verify Z phase level value on axis 0.
Dim state As Long

AxmSignalGetZphaseLevel 0, state
```

Delphi

```
{ Verify Z phase level value on axis 0. }
var
   state: DWord;

begin
   { Set Z phase level on axis 0. }
   AxmSignalSetZphaseLevel(0, HIGH);
   AxmSignalGetZphaseLevel(0, @state);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalSetServoOnLevel

Purpose

Set output level of Servo-On signal on a specific axis.

Format

C++

```
DWORD AxmSignalSetServoOnLevel(long lAxisNo, DWORD uLevel);
```

Visual Basic

```
Function AxmSignalSetServoOnLevel(ByVal lAxisNo As Long, ByVal uLevel
As Long) As Long
```

Delphi

```
function AxmSignalSetServoOnLevel(lAxisNo : LongInt; uLevel : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uLevel	[01h] Active Level of Servo-On signal: AXT_MOTION_LEVEL_MODE_DEF - [00h] B contact (Normal Close) - [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Set output level of Servo-On signal on an axis assigned by user. Use to set level because every company may have different input level of Servo-Drive.

Example

C++

```
// Set Servo-On level on axis 0.
AxmSignalSetServoOnLevel(0, HIGH);

// Verify Servo-On level value on axis 0.
DWORD state;

AxmSignalGetServoOnLevel (0, &state);
```

Visual Basic

```
' Set Servo-On level on axis 0.
AxmSignalSetServoOnLevel 0, HIGH

' Verify Servo-On level value on axis 0.
Dim state As Long

AxmSignalGetServoOnLevel 0, state
```

Delphi

```
{ Verify Servo-On level value on axis 0. }
var
   state: DWord;

begin
   { Set Servo-On level on axis 0. }
   AxmSignalSetServoOnLevel (0, HIGH);
   AxmSignalGetServoOnLevel (0, @state);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalGetServoOnLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetServoOnLevel

Purpose

Return output level setting of Servo-On signal on a specific axis.

Format

C++

```
DWORD AxmSignalGetServoOnLevel(long lAxisNo, DWORD *upLevel);
```

Visual Basic

```
Function AxmSignalGetServoOnLevel(ByVal lAxisNo As Long, ByRef
upLevel As Long) As Long
```

Delphi

```
function AxmSignalGetServoOnLevel(lAxisNo : LongInt; upLevel :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upLevel	[01h] Active Level of Servo-On signal: AXT_MOTION_LEVEL_MODE_DEF - [00h] B contact (Normal Close) - [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return output level setting of Servo-On signal set with 'AxmSignalSetServoOnLevel'.

Example

C++

```
// Set Servo-On level on axis 0.
AxmSignalSetServoOnLevel(0, HIGH);

// Verify Servo-On level value on axis 0.
DWORD state;

AxmSignalGetServoOnLevel (0, &state);
```

Visual Basic

```
' Set Servo-On level on axis 0.
AxmSignalSetServoOnLevel 0, HIGH

' Verify Servo-On level value on axis 0.
Dim state As Long

AxmSignalGetServoOnLevel 0, state
```

Delphi

```
{ Verify Servo-On level value on axis 0. }
var
   state: DWord;

begin
   { Set Servo-On level on axis 0. }
   AxmSignalSetServoOnLevel (0, HIGH);
   AxmSignalGetServoOnLevel (0, @state);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalSetServoAlarmResetLevel

Purpose

Return output level setting of Servo-On signal set with 'AxmSignalSetServoOnLevel'.

Format

C++

```
DWORD AxmSignalSetServoAlarmResetLevel(long lAxisNo, DWORD uLevel);
```

Visual Basic

```
Function AxmSignalSetServoAlarmResetLevel(ByVal lAxisNo As Long,
ByVal uLevel As Long) As Long
```

Delphi

```
function AxmSignalSetServoAlarmResetLevel(lAxisNo : LongInt; uLevel :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uLevel	[01h] Active Level Servo-Alarm Reset signal : AXT_MOTION_LEVEL_MODE_DEF - [00h] B contact (Normal Close) - [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

When alarm occurred due to abnormal operation or other reason during motion drive, clear alarm to output ServoAlarm Reset. Use to set level because every company may have different input level of Servo-Drive.

Example

C++

```
// Set Servo-Alarm Reset level on axis 0.
AxmSignalSetServoAlarmResetLevel(0, HIGH);

// Verify Servo-Alarm Reset level value on axis 0.
DWORD state;

AxmSignalGetServoAlarmResetLevel(0, &state);
```

Visual Basic

```
' Set Servo-Alarm Reset level on axis 0.
AxmSignalSetServoAlarmResetLevel 0, HIGH

' Verify Servo-Alarm Reset level value on axis 0.
Dim state As Long

AxmSignalGetServoAlarmResetLevel 0, state
```

Delphi

```
{ Verify Servo-Alarm Reset level value on axis 0. }
var
  state: DWord;

begin
  { Set Servo-Alarm Reset level on axis 0. }
  AxmSignalSetServoAlarmResetLevel(0, HIGH);
  AxmSignalGetServoAlarmResetLevel(0, @state);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit,AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetServoAlarmResetLevel

Purpose

Return output level setting of Servo-Alarm Reset signal on a specific axis.

Format

C++

```
DWORD AxmSignalGetServoAlarmResetLevel(long lAxisNo, DWORD *upLevel);
```

Visual Basic

```
Function AxmSignalGetServoAlarmResetLevel(ByVal lAxisNo As Long,
ByRef upLevel As Long) As Long
```

Delphi

```
function AxmSignalGetServoAlarmResetLevel(lAxisNo : LongInt; upLevel :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upLevel	[01h] Active Level Servo-Alarm Reset signal : AXT_MOTION_LEVEL_MODE_DEF - [00h] B contact (Normal Close) - [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return output level setting of Servo-Alarm Reset signal set with 'AxmSignalSetServoOnLevel'.

Example

C++

```
// Set Servo-Alarm Reset level on axis 0.
AxmSignalSetServoAlarmResetLevel(0, HIGH);

// Verify Servo-Alarm Reset level value on axis 0.
DWORD state;
AxmSignalGetServoAlarmResetLevel(0, &state);
```

Visual Basic

```
' Set Servo-Alarm Reset level on axis 0.
AxmSignalSetServoAlarmResetLevel 0, HIGH
```

```
' Verify Servo-Alarm Reset level value on axis 0.
Dim state As Long
AxmSignalGetServoAlarmResetLevel 0, state
```

Delphi

```
{ Verify Servo-Alarm Reset level value on axis 0. }
var
   state: DWord;

begin
   { Set Servo-Alarm Reset level on axis 0. }
   AxmSignalSetServoAlarmResetLevel(0, HIGH);
   AxmSignalGetServoAlarmResetLevel(0, @state);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalSetInpos

Purpose

Set whether InPos signal is used on a specific axis. Inposition signal is only effective when servo drive is used, and it is a signal that servo drive completed position decision movement. To use InPos function, servo drive sets InPos range with proper value because InPos range can be set on servo driver.

Format

C++

```
DWORD AxmSignalSetInpos(long lAxisNo, DWORD uUse);
```

Visual Basic

```
Function AxmSignalSetInpos(ByVal lAxisNo As Long, ByVal uUse As Long)
As Long
```

Delphi

```
function AxmSignalSetInpos(lAxisNo : LongInt; uUse : Dword) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uUse	[02h] Whether InPos signal is used : AXT_MOTION_LEVEL_MODE_DEF – [00h] B CONTACT (NORMAL CLOSE) – [01h] A CONTACT (NORMAL OPEN) – [02h] Not-used – [03h] Maintain current state

Return Values

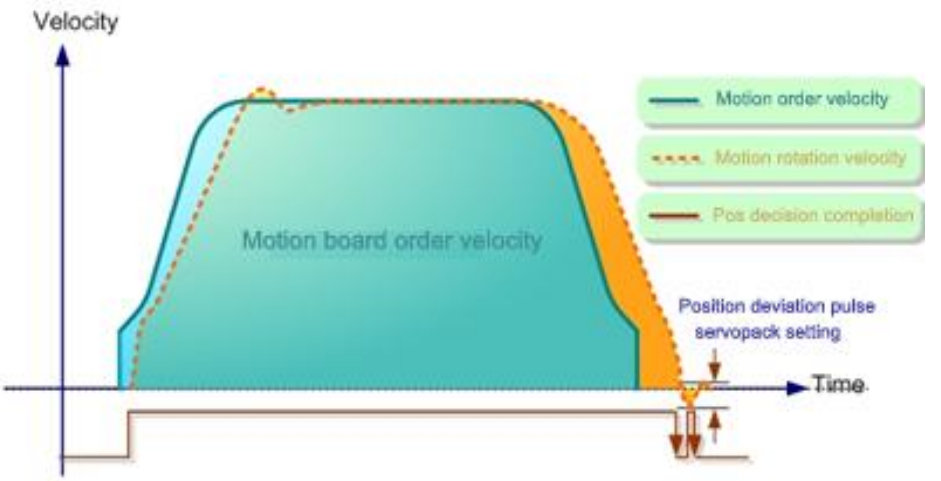
[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

When InPos signal is used, set to 'ENABLE,' otherwise, set to 'DISABLE.' If sets to 'ENABLE' Active Level of InPos signal is applied previously set level. If set Active Level of InPos signal, set to 'HIGH' or 'LOW'.

If calls this API, it recognizes that drive is ended when inPos signal (output of servo pack) comes in during motion exit. That is, move APIs operating within a loop in an API will stay in the API until inPos is to be active. InPos signal is generated when difference between command pulse input inside servo motor driver and feedback pulse input from motor is in the error range set on servo driver. Hence, this signal can be used for position decision completion signal.

Verification of InPos signal input returns AxmSignalReadInpos API.



For the reference, if step motor is used or InPos signal is not connected, it is recommended not to use by no means.

Example

C++

```
// Set whether InPos signal is used on axis 0.
AxmSignalSetInpos (0, HIGH);

// Verify whether InPos signal is used on axis 0.
DWORD   use;
AxmSignalGetInpos (0, &use);
```

Visual Basic

```
' Set whether InPos signal is used on axis 0.
AxmSignalSetInpos 0, HIGH

' Verify whether InPos signal is used on axis 0.
Dim use As Long
AxmSignalGetInpos 0, use
```

Delphi

```
{ Verify whether InPos signal is used on axis 0. }
var
  use: DWord;

begin
  { Set whether InPos signal is used on axis 0. }
  AxmSignalSetInpos (0, HIGH);
  AxmSignalGetInpos (0, @use);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetInpos

Purpose

Return whether InPos signal is used on a specific axis.

Format

C++

```
DWORD AxmSignalGetInpos(long lAxisNo, DWORD *upUse);
```

Visual Basic

```
Function AxmSignalGetInpos(ByVal lAxisNo As Long, ByRef upUse As Long)
As Long
```

Delphi

```
function AxmSignalGetInpos(lAxisNo : LongInt; upUse : PDword) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upUse	[02h] Whether InPos signal is used : AXT_MOTION_LEVEL_MODE_DEF – [00h] B CONTACT (NORMAL CLOSE) – [01h] A CONTACT (NORMAL OPEN) – [02h] Not-used – [03h] Maintain current state

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return InPos signal state set with AxmSignalSetInpos.

Return whether it is used if InPos signal assigned by user is ‘ENABLE’ or ‘DISABLE.’ If it uses InPos signal, Active Level of InPos signal can be verified.

Example

C++

```
// Set whether InPos signal is used on axis 0.
AxmSignalSetInpos (0, HIGH);

// Verify whether InPos signal is used on axis 0.
DWORD   use;
AxmSignalGetInpos (0, &use);
```

Visual Basic

```
' Set whether InPos signal is used on axis 0.
AxmSignalSetInpos 0, HIGH

' Verify whether InPos signal is used on axis 0.
Dim use As Long
AxmSignalGetInpos 0, use
```

Delphi

```
{ Verify whether InPos signal is used on axis 0. }
var
  use: DWord;

begin
  { Set whether InPos signal is used on axis 0. }
  AxmSignalSetInpos (0, HIGH);
  AxmSignalGetInpos (0, @use);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalReadInpos

Purpose

Return inposition signal input state on a specific axis.

Format

C++

```
DWORD AxmSignalReadInpos(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmSignalReadInpos(ByVal lAxisNo As Long, ByRef upStatus As Long) As Long
```

Delphi

```
function AxmSignalReadInpos(lAxisNo : LongInt; upStatus : PDword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStatus	Input state of InPos signal : AXT_MOTION_SIGNAL_LEVEL_DEF – [00h] No-activation – [01h] Activation

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return current InPos signal input state set with ' AxmSignalSetInpos'.

Example

C++

```
// Verify Inposition signal input on axis 0.
DWORD upStatus;
AxmSignalReadInpos (0, &upStatus);
```

Visual Basic

```
' Verify Inposition signal input on axis 0.
Dim upStatus As Long
AxmSignalReadInpos 0, upStatus
```

Delphi

```
{ Verify Inposition signal input on axis 0. }
var
  upStatus: DWord;

begin
  AxmSignalReadInpos (0, @upStatus);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalSetServoAlarm

Purpose

Set whether to use emergency when an alarm signal on a specific axis is inputted and the active level of the alarm signal.

Format

C++

```
DWORD AxmSignalSetServoAlarm(long lAxisNo, DWORD uUse);
```

Visual Basic

```
Function AxmSignalSetServoAlarm(ByVal lAxisNo As Long, ByVal uUse As
Long) As Long
```

Delphi

```
function AxmSignalSetServoAlarm(lAxisNo : LongInt; uUse : Dword) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uUse	[00h] Whether to use alarm signal and the active level : AXT_MOTION_LEVEL_MODE_DEF - [00h] B CONTACT (NORMAL CLOSE) - [01h] A CONTACT (NORMAL OPEN) - [02h] Not-used - [03h] Maintain current state

Note) In the network-type motion controller, the used value has the meaning as 'LOW', 'HIGH' and 'USED', except RTEX-PM board and (PCI-N804/404) uses the same

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

When alarm input is on, the motion task on the corresponding axis performs emergency stop.

Low (B contact) : Close normally -> Open if detected.

High (A contact) : Open normally -> Close if detected.

Example

C++

```
// Set whether to use emergency stop when an alarm signal on axis 0 is inputted.
AxmSignalSetServoAlarm (0, HIGH);
```

```
// Verify settings for whether to use emergency stop when an alarm signal on axis 0
// is inputted.
DWORD   use;
AxmSignalGetServoAlarm(0, &use);
```

Visual Basic

```
' Set whether to use emergency stop when an alarm signal on axis 0 is inputted.
AxmSignalSetServoAlarm 0, HIGH

' Verify settings for whether to use emergency stop when an alarm signal on axis 0
' is inputted.
Dim use As Long
AxmSignalGetServoAlarm 0, use
```

Delphi

```
{ Verify settings for whether to use emergency stop when an alarm signal on axis 0
is inputted. }
var
  use : DWord;

begin
  { Set whether to use emergency stop when an alarm signal on axis 0 is inputted. }
  AxmSignalSetServoAlarm (0, HIGH);
  AxmSignalGetServoAlarm(0, @use);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetServoAlarm

Purpose

Return whether to use emergency stop and the settings for the active level of the alarm signal when an alarm signal on a specific axis is inputted.

Format

C++

```
DWORD AxmSignalGetServoAlarm(long lAxisNo, DWORD *upUse);
```

Visual Basic

```
Function AxmSignalGetServoAlarm(ByVal lAxisNo As Long, ByRef upUse As
Long) As Long
```

Delphi

```
function AxmSignalGetServoAlarm(lAxisNo : LongInt; upUse : PDword) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upUse	[00h] Whether to use alarm signal and the active level : AXT_MOTION_LEVEL_MODE_DEF - [00h] B CONTACT (NORMAL CLOSE) - [01h] A CONTACT (NORMAL OPEN) - [02h] Not-used - [03h] Maintain current state

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the output level settings for the Servo-Alarm signal set by 'AxmSignalSetServoAlarm'.

When alarm input is on, the motion task on the corresponding axis performs emergency stop.

Low(B contact) : Close normally -> Open if detected.

High(A contact) : Open normally -> Close if detected.

Example

C++

```
// Set whether to use emergency stop when an alarm signal on axis 0 is inputted.
AxmSignalSetServoAlarm (0, HIGH);
```

```
// Verify settings for whether to use emergency stop when an alarm signal on axis 0 is
// inputted.
DWORD   use;
AxmSignalGetServoAlarm(0, &use);
```

Visual Basic

```
' Set whether to use emergency stop when an alarm signal on axis 0 is inputted.
AxmSignalSetServoAlarm 0, HIGH

' Verify settings for whether to use emergency stop when an alarm signal on axis 0 is
' inputted.
Dim use As Long
AxmSignalGetServoAlarm 0, use
```

Delphi

```
{ Verify settings for whether to use emergency stop when an alarm signal on axis 0 is
inputted. }
var
  use : DWord;

begin
  { Set whether to use emergency stop when an alarm signal on axis 0 is inputted. }
  AxmSignalSetServoAlarm (0, HIGH);
  AxmSignalGetServoAlarm(0, @use);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalReadServoAlarm

Purpose

Return input state of alarm signal on a specific axis.

Format

C++

```
DWORD AxmSignalReadServoAlarm(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmSignalReadServoAlarm(ByVal lAxisNo As Long, ByRef
upStatus As Long) As Long
```

Delphi

```
function AxmSignalReadServoAlarm(lAxisNo : LongInt; upStatus :
PDword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStatus	Input state of alarm signal : AXT_MOTION_SIGNAL_LEVEL_DEF – [00h] Non-activation – [01h] Activation

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the current input state of the Servo-Alarm signal set by 'AxmSignalSetServoAlarm'.

When alarm input is on, the motion task on the corresponding axis performs emergency stop.

upStatus returns 1 when the input signal is on, or returns 0 when it is off.

Low(B contact) : Close normally -> Open if detected.

High(A contact): Open normally -> Close if detected.

Example

C++

```
// Verify alarm signal on axis 0.
DWORD upStatus;
AxmSignalReadServoAlarm (0, &upStatus);
```

Visual Basic1

```
' Verify alarm signal on axis 0.
Dim upStatus As Long
AxmSignalReadServoAlarm 0, upStatus
```

Delphi

```
{ Verify alarm signal on axis 0. }
var
  upStatus: DWord;

begin
  AxmSignalReadServoAlarm (0, @upStatus);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalSetLimit

Purpose

Set whether to use limit sensor on a specific axis and the signal Active Level.

Format

C++

```
DWORD AxmSignalSetLimit(long lAxisNo, DWORD uStopMode, DWORD
uPositiveLevel, DWORD uNegativeLevel);
```

Visual Basic

```
Function AxmSignalSetLimit(ByVal lAxisNo As Long, ByVal uStopMode As
Long, ByVal uPositiveLevel As Long, ByVal uNegativeLevel As Long) As
Long
```

Delphi

```
function AxmSignalSetLimit(lAxisNo : LongInt; uStopMode : DWord;
uPositiveLevel : DWord; uNegativeLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uStopMode	[00h] Stop mode value after detecting limit sensor : AXT_MOTION_STOPMODE_DEF - [00h] Emergency stop - [01h] Deceleration stop
[>>]uPositiveLevel	[00h] (+) Limit Sensor Active Level : AXT_MOTION_LEVEL_MODE_DEF - [00h] B CONTACT (NORMAL CLOSE) - [01h] A CONTACT (NORMAL OPEN) - [02h] Not-used - [03h] Maintain current state
[>>]uNegativeLevel	[00h] (-) Limit Sensor Active Level : AXT_MOTION_LEVEL_MODE_DEF - [00h] B CONTACT (NORMAL CLOSE) - [01h] A CONTACT (NORMAL OPEN) - [02h] Not-used - [03h] Maintain current state

Note) In the network-type motion controller, the PositiveLevel/NegativeLevel value has the meaning as 'LOW', 'HIGH' and 'USED', except RTEX-PM board and (PCI-N804/404) uses the same.

Return Values

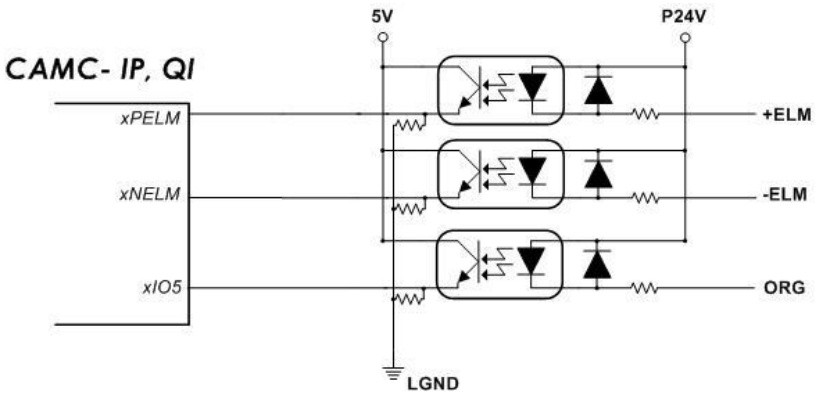
[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

If –End Limit input is on, the (–) direction motion task on the corresponding axis stops, and if +End Limit input is on, the (+) direction motion task stops.

Low(B contact) : Close normally –> Open if detected.

High(A contact): Open normally –> Close if detected.



Example

C++

```
// Set so that end limit sensor on axis 0 can be used.
AxmSignalSetLimit (0, EMERGENCY_STOP, HIGH, HIGH);

// Verify whether to use end limit sensor on axis 0.
DWORD StopMode, PositiveLevel, NegativeLevel;
AxmSignalGetLimit (0, &StopMode, &PositiveLevel, &NegativeLevel);
```

Visual Basic

```
' Set so that end limit sensor on axis 0 can be used.
AxmSignalSetLimit 0, EMERGENCY_STOP, HIGH, HIGH

' Verify whether to use end limit sensor on axis 0.
Dim StopMode , PositiveLevel, NegativeLevel As Long
AxmSignalGetLimit 0, StopMode, PositiveLevel, NegativeLevel
```

Delphi

```
{ Verify whether to use end limit sensor on axis 0. }
var
  StopMode, PositiveLevel, NegativeLevel: DWord;

begin
  { Set so that end limit sensor on axis 0 can be used. }
  AxmSignalSetLimit (0, EMERGENCY_STOP, HIGH, HIGH);
  AxmSignalGetLimit (0, @StopMode, @PositiveLevel, @NegativeLevel);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetLimit

Purpose

Return whether to use limit sensor on a specific axis and the setting value for the signal Active Level.

Format

C++

```
DWORD AxmSignalGetLimit(long lAxisNo, DWORD *upStopMode, DWORD
*upPositiveLevel, DWORD *upNegativeLevel);
```

Visual Basic

```
Function AxmSignalGetLimit(ByVal lAxisNo As Long, ByRef upStopMode As
Long, ByRef upPositiveLevel As Long, ByRef upNegativeLevel As Long)
As Long
```

Delphi

```
function AxmSignalGetLimit(lAxisNo : LongInt; upStopMode : PDword;
upPositiveLevel : PDword; upNegativeLevel : PDword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStopMode	[00h] Stop mode value after detecting limit sensor : AXT_MOTION_STOPMODE_DEF – [00h] Emergency stop – [01h] Deceleration stop
[<<]upPositiveLevel	[00h] (+) Limit Sensor Active Level : AXT_MOTION_LEVEL_MODE_DEF – [00h] B CONTACT (NORMAL CLOSE) – [01h] A CONTACT (NORMAL OPEN) – [02h] Not-used – [03h] Maintain current state
[<<]upNegativeLevel	[00h] (–) Limit Sensor Active Level : AXT_MOTION_LEVEL_MODE_DEF – [00h] B CONTACT (NORMAL CLOSE) – [01h] A CONTACT (NORMAL OPEN) – [02h] Not-used – [03h] Maintain current state

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Tip : It is possible to get only necessary data by inserting NULL into unnecessary variables.

It returns the output level settings for Pos/Neg end limit signal set by 'AxmSignalSetLimit' and returns StopMode. If –End Limit input is on, the (–) direction motion task on the corresponding axis will stop, and if +End Limit input is on, (+) direction motion task will stop.

Low(B contact) : Close normally –> Open if detected.

High(A contact): Open normally –> Close if detected.

Example

C++

```
// Set so that end limit sensor on axis 0 can be used.
AxmSignalSetLimit (0, EMERGENCY_STOP, HIGH, HIGH);

// Verify whether to use end limit sensor on axis 0.
DWORD StopMode, PositiveLevel, NegativeLevel;
AxmSignalGetLimit (0, &StopMode, &PositiveLevel, &NegativeLevel);
```

Visual Basic

```
' Set so that end limit sensor on axis 0 can be used.
AxmSignalSetLimit 0, EMERGENCY_STOP, HIGH, HIGH

' Verify whether to use end limit sensor on axis 0.
Dim StopMode , PositiveLevel, NegativeLevel As Long
AxmSignalGetLimit 0, StopMode, PositiveLevel, NegativeLevel
```

Delphi

```
{ Verify whether to use end limit sensor on axis 0. }
var
  StopMode, PositiveLevel, NegativeLevel: DWord;

begin
  { Set so that end limit sensor on axis 0 can be used. }
  AxmSignalSetLimit (0, EMERGENCY_STOP, HIGH, HIGH);
  AxmSignalGetLimit (0, @StopMode, @PositiveLevel, @NegativeLevel);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalReadLimit

Purpose

Return input state of the limit sensor on a specific axis.

Format

C++

```
DWORD AxmSignalReadLimit(long lAxisNo, DWORD *upPositiveStatus, DWORD
*upNegativeStatus);
```

Visual Basic

```
Function AxmSignalReadLimit(ByVal lAxisNo As Long, ByRef
upPositiveStatus As Long, ByRef upNegativeStatus As Long) As Long
```

Delphi

```
function AxmSignalReadLimit(lAxisNo : LongInt; upPositiveStatus :
PDword; upNegativeStatus : PDword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upPositiveStatus	(+) Limit Sensor's signal input state: AXT_MOTION_SIGNAL_LEVEL_DEF - [00h] No-activation + Limit sensor's signal is Off - [01h] Activation + Limit sensor's signal is On
[<<]upNegativeStatus	(-) Limit Sensor's signal input state: AXT_MOTION_SIGNAL_LEVEL_DEF - [00h] No-activation - Limit sensor's signal is Off - [01h] Activation - Limit sensor's signal is On

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Tip : It is possible to get only necessary data by inserting NULL into unnecessary variables.

It returns the current input state of Pos/Neg end limit signal set by 'AxmSignalSetLimit'.

If -End Limit input is on, the (-) direction motion task on the corresponding axis will stop, and if +End Limit input is on, (+) direction motion task will stop.

Low(B contact) : Close normally -> Open if detected.

High(A contact): Open normally -> Close if detected.

Example

C++

```
// Verify the input state of limit sensor on axis 0.
DWORD upPositiveLevel, upNegativeLevel;
AxmSignalReadLimit (0, &upPositiveLevel, &upNegativeLevel);
```

Visual Basic

```
' Verify the input state of limit sensor on axis 0.
Dim upPositiveLevel , upNegativeLevel As Long
AxmSignalReadLimit 0, upPositiveLevel, upNegativeLevel
```

Delphi

```
{ Verify the input state of limit sensor on axis 0. }
var
  upPositiveLevel, upNegativeLevel: DWord;

begin
  AxmSignalReadLimit (0, @upPositiveLevel, @upNegativeLevel);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalSetSoftLimit

Purpose

Set soft limit function on a specific axis.

Format

C++

```
DWORD AxmSignalSetSoftLimit(long lAxisNo, DWORD uUse, DWORD uStopMode,
DWORD uSelection, double dPositivePos, double dNegativePos);
```

Visual Basic

```
Function AxmSignalSetSoftLimit(ByVal lAxisNo As Long, ByVal uUse As
Long, ByVal uStopMode As Long, ByVal uSelection As Long, ByVal
dPositivePos As Double, ByVal dNegativePos As Double) As Long
```

Delphi

```
function AxmSignalSetSoftLimit(lAxisNo : LongInt; uUse : Dword;
uStopMode : Dword; uSelection : Dword; dPositivePos : Double;
dNegativePos : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (starting from 0)
[>>]uUse	[00h] Whether to use soft limit function : AXT_USE – [00h] Soft limit not-used – [01h] Soft limit used
[>>]uStopMode	[00h] Motion stop mode when soft limit is On : AXT_MOTION_STOPMODE_DEF – [00h] Emergency stop – [01h] Deceleration stop
[>>]uSelection	[00h] Choose current position input source : AXT_MOTION_SELECTION_DEF – [00h] Command position. Use Cmd Pos as a current position's comparator value – [01h] Actual position. Use Act Pos as a current position's comparator value
[>>]dPositivePos	[0.0] Position value of + direction soft limit
[>>]dNegativePos	[0.0] Position value of – direction soft limit

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
* See error code Table for more information on status error codes

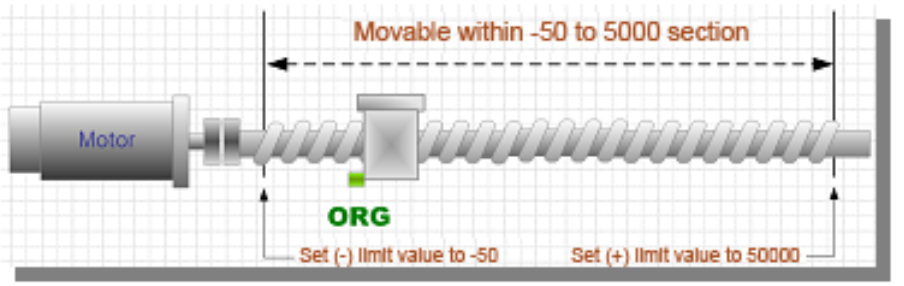
Description

Using soft limit function allows the motion movement within the range between dPositivePositiin and dNegativePos. If it gets out of range, the motion stops with the soft limit On.

Cautiion: During home search, in case it is operated by preset software limit using this API, it becomes disabled if home search stops.

N Limit : Specify stroke limit value in (−) direction.

P Limit : Specify stroke limit value in (+) direction.



Example

C++

```
// Set Soft Limit on axis 0.
AxmSignalSetSoftLimit(0, ENABLE, EMERGENCY_STOP, COMMAND, 50000, -50);

// Verify Soft Limit settings on axis 0.
DWORD upUse, upStopMode, upSelection;
double dpPositivePos, dpNegativePos;
AxmSignalGetSoftLimit (0, &upUse, &upStopMode, &upSelection, &dpPositivePos,
   &dpNegativePos);
```

Visual Basic

```
' Set Soft Limit on axis 0.
AxmSignalSetSoftLimit 0, ENABLE, EMERGENCY_STOP, COMMAND, 50000, -50

' Verify Soft Limit settings on axis 0.
Dim upUse , upStopMode , upSelection As Long
Dim dpPositivePos , dpNegativePos As Double
AxmSignalGetSoftLimit 0, upUse, upStopMode, upSelection, dpPositivePos, dpNegativePos
```

Delphi

```
{ Verify Soft Limit settings on axis 0. }
var
   upUse, upStopMode, upSelection: DWord;
   dpPositivePos, dpNegativePos: Double;

begin
   { Set Soft Limit on axis 0. }
   AxmSignalSetSoftLimit(0, ENABLE, EMERGENCY_STOP, COMMAND, 50000, -50);
```

```
  AxmSignalGetSoftLimit (0, @upUse, @upStopMode, @upSelection, @dpPositivePos,
     @dpNegativePos);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetSoftLimit

Purpose

Return whether to use soft limit function on a specific axis, position, and stop mode, etc.

Format

C++

```
DWORD AxmSignalGetSoftLimit(long lAxisNo, DWORD *upUse, DWORD
*upStopMode, DWORD *upSelection, double *dpPositivePos, double
*dpNegativePos);
```

Visual Basic

```
Function AxmSignalGetSoftLimit(ByVal lAxisNo As Long, ByRef upUse As
Long, ByRef upStopMode As Long, ByRef upSelection As Long, ByRef
dpPositivePos As Double, ByRef dpNegativePos As Double) As Long
```

Delphi

```
function AxmSignalGetSoftLimit(lAxisNo : LongInt; upUse : PDWord;
upStopMode : PDWord; upSelection : PDWord; dpPositivePos : PDouble;
dpNegativePos : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (start from 0)
[<<]upUse	[00h] Whether to use soft limit function : AXT_USE – [00h] Soft limit not-used – [01h] Soft limit used
[<<]upStopMode	[00h] Motion stop mode when soft limit is On : AXT_MOTION_STOPMODE_DEF – [00h] Emergency stop – [01h] Deceleration stop
[<<]upSelection	Choose current position input source : AXT_MOTION_SELECTION_DEF – [00h] Command position. Use Cmd Pos as a current position's comparator value – [01h] Actual position. Use Act Pos as a current position's comparator value
[<<]dpPositivePos	[0.0] Position value of + direction soft limit
[<<]dpNegativePos	[0.0] Position value of – direction soft limit

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Tip : It is possible to get only necessary data by inserting NULL into unnecessary variables.

It returns the Pos/Neg limit value set by 'AxmSignalSetSoftLimit'.

N Limit : Specify stroke limit value in (−) direction.

P Limit : Specify stroke limit value in (+) direction.

Example

C++

```
// Set Soft Limit on axis 0.
AxmSignalSetSoftLimit(0, ENABLE, EMERGENCY_STOP, CMD, 50000, -50);
// Verify Soft Limit settings on axis 0.
DWORD upUse, upStopMode, upSelection;
double dpPositivePos, dpNegativePos;
AxmSignalGetSoftLimit (0, &upUse, &upStopMode, &upSelection, &dpPositivePos,
  &dpNegativePos);
```

Visual Basic

```
' Set Soft Limit on axis 0.
AxmSignalSetSoftLimit 0, ENABLE, EMERGENCY_STOP, CMD, 50000, -50
' Verify Soft Limit settings on axis 0.
Dim upUse , upStopMode , upSelection As Long
Dim dpPositivePos , dpNegativePos As Double
AxmSignalGetSoftLimit 0, upUse, upStopMode, upSelection, dpPositivePos, dpNegativePos
```

Delphi

```
{ Verify Soft Limit settings on axis 0. }
var
  upUse, upStopMode, upSelection: DWord;
  dpPositivePos, dpNegativePos: Double;

begin
  { Set Soft Limit on axis 0. }
  AxmSignalSetSoftLimit(0, ENABLE, EMERGENCY_STOP, CMD, 50000, -50);
  AxmSignalGetSoftLimit (0, @upUse, @upStopMode, @upSelection, @dpPositivePos,
     @dpNegativePos);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalSetStop, AxmSignalGetStop, AxmSignalReadStop

AxmSignalSetStop

Purpose

Set stop mode and the level of emergency stop signal (SSTOP, ESTOP) on a specific axis.

Format

C++

```
DWORD AxmSignalSetStop(long lAxisNo, DWORD uStopMode, DWORD uLevel);
```

Visual Basic

```
Function AxmSignalSetStop(ByVal lAxisNo As Long, ByVal uStopMode As
Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxmSignalSetStop(lAxisNo : LongInt; uStopMode : Dword;
uLevel : Dword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uStopMode	[00h] Stop mode (PCI-N804/404 support emergency stop only) : AXT_MOTION_STOPMODE_DEF – [00h] Emergency stop – [01h] Deceleration stop
[>>]uLevel	[01h] Whether to use stop signal and the active level : AXT_MOTION_LEVEL_MODE_DEF – [00h] B CONTACT (NORMAL CLOSE) – [01h] A CONTACT (NORMAL OPEN) – [02h] Not-used – [03h] Maintain current state

Note) In the network-type motion controller, the Level value has the meaning as 'LOW', 'HIGH' and 'USED', except RTEX-PM board and (PCI-N804/404) uses the same.

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

User sets the level and the stop mode of emergency stop signal on the axis specified by a user. It is possible that pulse output becomes unavailable if settings are incorrect. This signal level can be verified by AxmStatusReadMechanical.

Low(B contact) : Close normally -> Open if detected.

High(A contact): Open normally -> Close if detected.

Example

C++

```
// Set whether to use stop signal on axis 0.
AxmSignalSetStop (0, EMERGENCY_STTOP, ENABLE);

// Verify settings of whether to use emergency stop signal on axis 0.
DWORD upStopMode, upLevel;
AxmSignalGetStop (0, &upStopMode, &upLevel);
```

Visual Basic

```
' Set whether to use stop signal on axis 0.
AxmSignalSetStop 0, EMERGENCY_STTOP, ENABLE

' Verify settings of whether to use emergency stop signal on axis 0.
Dim upStopMode , upLevel As Long
AxmSignalGetStop 0, upStopMode, upLevel
```

Delphi

```
{ Verify settings of whether to use emergency stop signal on axis 0. }
var
  pStopMode, upLevel: DWord;

begin
  { Set whether to use stop signal on axis 0. }
  AxmSignalSetStop (0, EMERGENCY_STTOP, ENABLE);
  AxmSignalGetStop (0, @upStopMode, @upLevel);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalGetStop, AxmSignalReadStop

AxmSignalGetStop

Purpose

Return stop mode and the level of emergency stop signal (SSTOP, ESTOP) on a specific axis.

Format

C++

```
DWORD AxmSignalGetStop(long lAxisNo, DWORD *upStopMode, DWORD
*upLevel);
```

Visual Basic

```
Function AxmSignalGetStop(ByVal lAxisNo As Long, ByRef upStopMode As
Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxmSignalGetStop(lAxisNo : LongInt; upStopMode : PDword;
upLevel : PDword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStopMode	[00h] Stop mode (PCI-N804/404 support emergency stop only) : AXT_MOTION_STOPMODE_DEF - [00h] Emergency stop - [01h] Deceleration stop
[<<]upLevel	[00h] Whether to use stop signal and the active level : AXT_MOTION_LEVEL_MODE_DEF - [00h] B CONTACT (NORMAL CLOSE) - [01h] A CONTACT (NORMAL OPEN) - [02h] Not-used - [03h] Maintain current state

Note) In the network-type motion controller, the Level value has the meaning as 'LOW', 'HIGH' and 'USED', except RTEX-PM board and (PCI-N804/404) uses the same.

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Tip : It is possible to get only necessary data by inserting NULL into unnecessary variables. It returns stop mode and level of emergency stop signal(SSTOP, ESTOP) set by 'AxmSignalSetStop'.

User sets the level and the stop mode of emergency stop signal on the axis specified by a user. It is possible that pulse output becomes unavailable if settings are incorrect. This signal level can be verified by AxmStatusReadMechanical.

Low(B contact) : Close normally -> Open if detected.

High(A contact): Open normally -> Close if detected.

Example

C++

```
// Set whether to use stop signal on axis 0.
AxmSignalSetStop (0, EMERGENCY_STTOP, ENABLE);

// Verify settings of whether to use emergency stop signal on axis 0.
DWORD upStopMode, upLevel;
AxmSignalGetStop (0, &upStopMode, &upLevel);
```

Visual Basic

```
' Set whether to use stop signal on axis 0.
AxmSignalSetStop 0, EMERGENCY_STTOP, ENABLE

' Verify settings of whether to use emergency stop signal on axis 0.
Dim upStopMode , upLevel As Long
AxmSignalGetStop 0, upStopMode, upLevel
```

Delphi

```
{ Verify settings of whether to use emergency stop signal on axis 0. }
var
  upStopMode, upLevel: DWord;

begin
  { Set whether to use stop signal on axis 0. }
  AxmSignalSetStop (0, EMERGENCY_STTOP, ENABLE);
  AxmSignalGetStop (0, @upStopMode, @upLevel);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalReadStop

AxmSignalReadStop

Purpose

Return input state of emergency stop signal on a specific axis.

Format

C++

```
DWORD AxmSignalReadStop(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmSignalReadStop(ByVal lAxisNo As Long, ByRef upStatus As
Long) As Long
```

Delphi

```
function AxmSignalReadStop(lAxisNo : LongInt; upStatus : PDword) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStatus	Pulse output method : AXT_MOTION_SIGNAL_LEVEL_DEF – [00h] No-activation (Stop signal Off) – [01h] Activation (Stop signal On)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the current input state of emergency stop signal (SSTOP, ESTOP) set by ' AxmSignalSetStop'.

Example

C++

```
// Verify the state of STOP signal on axis 0.
DWORD uStatus;
AxmSignalReadStop (0, &uStatus);
```

Visual Basic

```
' Verify the state of STOP signal on axis 0.
Dim uStatus As Long
AxmSignalReadStop 0, uStatus
```

Delphi

```
{ Verify the state of STOP signal on axis 0. }
var
  uStatus: DWord;

begin
  AxmSignalReadStop (0, @uStatus);
end;
```

See Also

AxmSignalSetZphaseLevel, AxmSignalGetZphaseLevel, AxmSignalSetServoOnLevel, AxmSignalGetServoOnLevel, AxmSignalSetServoAlarmResetLevel, AxmSignalGetServoAlarmResetLevel, AxmSignalSetInpos, AxmSignalGetInpos, AxmSignalReadInpos, AxmSignalSetServoAlarm, AxmSignalGetServoAlarm, AxmSignalReadServoAlarm, AxmSignalSetLimit, AxmSignalGetLimit, AxmSignalReadLimit, AxmSignalSetSoftLimit, AxmSignalGetSoftLimit, AxmSignalSetStop, AxmSignalGetStop

AxmSignalServoOn

Purpose

Set output of Servo-On signal on a specific axis.

Format

C++

```
DWORD AxmSignalServoOn(long lAxisNo, DWORD uOnOff);
```

Visual Basic

```
Function AxmSignalServoOn(ByVal lAxisNo As Long, ByVal uOnOff As Long)
As Long
```

Delphi

```
function AxmSignalServoOn(lAxisNo : LongInt; uOnOff : DWord) : DWord;
stdcall;
```

Parameters

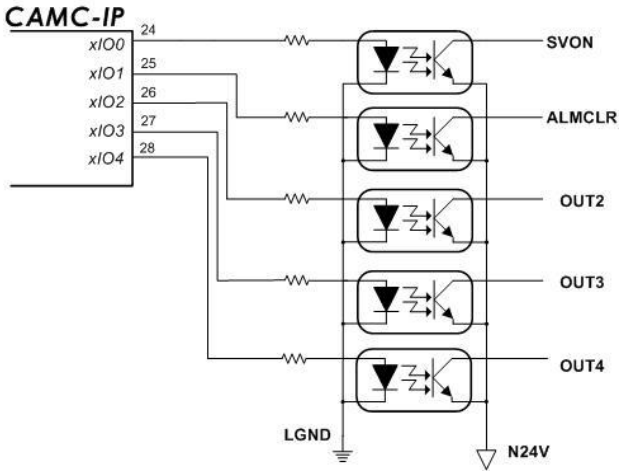
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uOnOff	[00h] Set Serve-On signal output : FALSE & TRUE - [00h] Servo-On output is Off - [01h] Servo-On output is On

Return Values

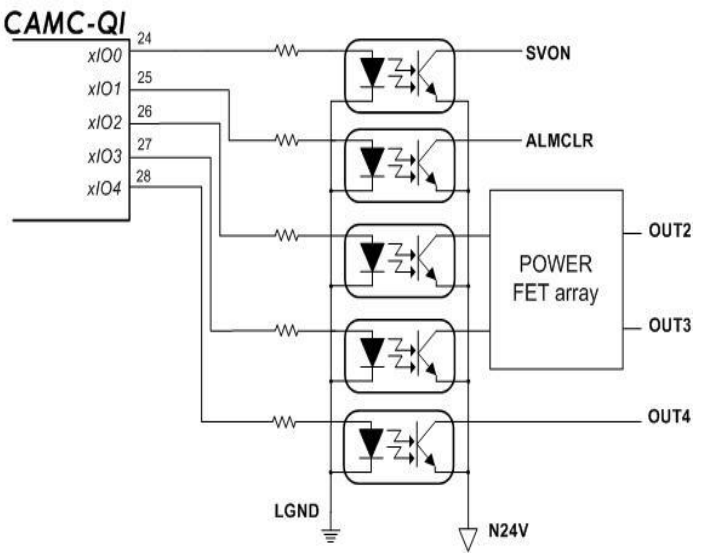
[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Servo-On signal is an external switch that is used to control ON/OFF of the servo driver when a servo driver is used. This API controls ON/OFF of the Servo-On signal. AxmSignalIsServoOn API returns the output state of the current Servo-On signal, and the active level can be set using AxmSignalSetServoOnLevel API.



The difference between IP and PCI-N804/404 is that PCI-N804/404 in the universal 2,3 is outputted enough for current to relay move because its output is from the photo coupler through the power FET.



Example

C++

```
// Servo On on axis 0.
AxmSignalServoOn(0, ENABLE);

// Verify Servo On on axis 0.
DWORD level;
AxmSignalIsServoOn(0, &level);
```

Visual Basic

```
' Servo On on axis 0.
AxmSignalServoOn 0, ENABLE

' Verify Servo On on axis 0.
Dim level As Long
AxmSignalIsServoOn 0, level
```

Delphi

```
{ Verify Servo On on axis 0. }
var
   level: DWord;

begin
   { Servo On on axis 0. }
   AxmSignalServoOn (0, ENABLE);
   AxmSignalIsServoOn (0, @level);
end;
```

See Also

AxmSignalIsServoOn, AxmSignalServoAlarmReset, AxmSignalWriteOutput, AxmSignalReadOutput, AxmSignalWriteOutputBit, AxmSignalReadOutputBit, AxmSignalReadInput, AxmSignalReadInputBit

AxmSignalIsServoOn

Purpose

Return state of Servo-On output signal on a specific axis.

Format

C++

```
DWORD AxmSignalIsServoOn(long lAxisNo, DWORD *upOnOff);
```

Visual Basic

```
Function AxmSignalIsServoOn(ByVal lAxisNo As Long, ByRef upOnOff As
Long) As Long
```

Delphi

```
function AxmSignalIsServoOn(lAxisNo : LongInt; upOnOff : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upOnOff	Set Serve-On signal output : FALSE & TRUE - [00h] Servo-On output is Off - [01h] Servo-On output is On

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the state of Servo-On output signal set by 'AxmSignalServoOn'. Servo-On signal is an external switch that is used to control ON/OFF of the servo driver when a servo driver is used. This API controls ON/OFF of the Servo-On signal. AxmSignalIsServoOn API returns the output state of the current Servo-On signal, and the active level can be set using AxmSignalSetServoOnLevel API. The universal output 0 on the board is used as servo-on and the universal output 1 is used as servo-alarm reset.

Example

C++

```
// Servo On on axis 0.
AxmSignalServoOn (0, ENABLE);

// Verify Servo On on axis 0.
DWORD level;
AxmSignalIsServoOn (0, &level);
```

Visual Basic

```
' Servo On on axis 0.
AxmSignalServoOn 0, ENABLE

' Verify Servo On on axis 0.
Dim level As Long
AxmSignalIsServoOn 0, level
```

Delphi

```
{ Verify Servo On on axis 0. }
var
   level: DWord;

begin
   { Servo On on axis 0. }
   AxmSignalServoOn (0, ENABLE);
   AxmSignalIsServoOn (0, @level);
end;
```

See Also

AxmSignalServoOn, AxmSignalServoAlarmReset, AxmSignalWriteOutput, AxmSignalReadOutput, AxmSignalWriteOutputBit, AxmSignalReadOutputBit, AxmSignalReadInput, AxmSignalReadInputBit

AxmSignalServoAlarmReset

Purpose

Set output of Servo-Alarm Reset signal on a specific axis.

Format

C++

```
DWORD AxmSignalServoAlarmReset(long lAxisNo, DWORD uOnOff);
```

Visual Basic

```
Function AxmSignalServoAlarmReset(ByVal lAxisNo As Long, ByVal uOnOff
As Long) As Long
```

Delphi

```
function AxmSignalServoAlarmReset(lAxisNo : LongInt; uOnOff : DWord) :
DWord; stdcall;
```

Parameters

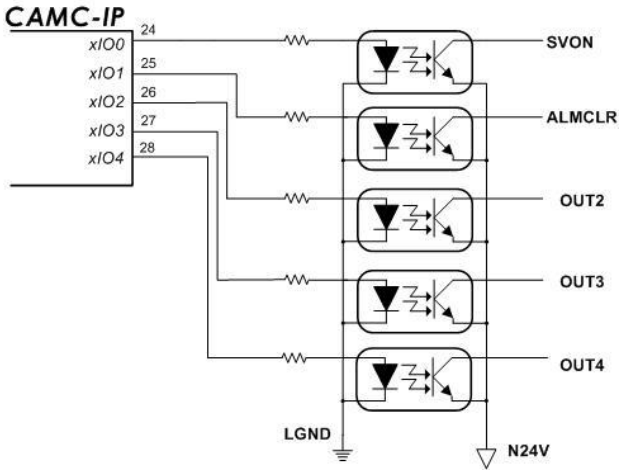
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uOnOff	Set output of Servo-Alarm Reset signal : FALSE & TRUE - [00h] Servo-Alarm Reset Output is Off - [01h] Servo-Alarm Reset Output is On

Return Values

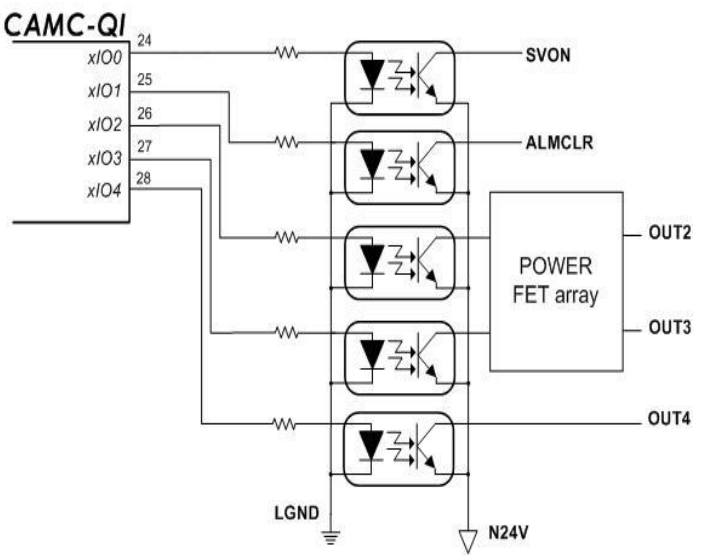
[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It is used to clear alarm in case alarm was generated in Servo Drive due to overload on a specific axis or any other reasons. The universal output 0 on the board is used as servo-on and the universal output 1 is used as servo-alarm reset.



The difference between IP and PCI-N804/404 is that PCI-n804/404 in the universal 2, 3 is outputted enough for current to relay move because its output is from the photo coupler through the power FET.



Example

C++

```
// Servo Alarm Reset on axis 0.
AxmSignalServoAlarmReset (0, ENABLE);
```

Visual Basic

```
' Servo Alarm Reset on axis 0.
AxmSignalServoAlarmReset 0, ENABLE
```

Delphi

```
begin
   { Servo Alarm Reset on axis 0. }
   AxmSignalServoAlarmReset (0, ENABLE);
end;
```

See Also

AxmSignalServoOn, AxmSignalIsServoOn, AxmSignalWriteOutput, AxmSignalReadOutput, AxmSignalWriteOutputBit, AxmSignalReadOutputBit, AxmSignalReadInput, AxmSignalReadInputBit

AxmSignalWriteOutput

Purpose

Set the value of universal output on a specific axis. This value is inputted in a hexadecimal value.

Format

C++

```
DWORD AxmSignalWriteOutput(long lAxisNo, DWORD uValue);
```

Visual Basic

```
Function AxmSignalWriteOutput(ByVal lAxisNo As Long, ByVal uValue As
Long) As Long
```

Delphi

```
function AxmSignalWriteOutput(lAxisNo : LongInt; uValue : Dword) :
DWord; stdcall;
```

Parameters

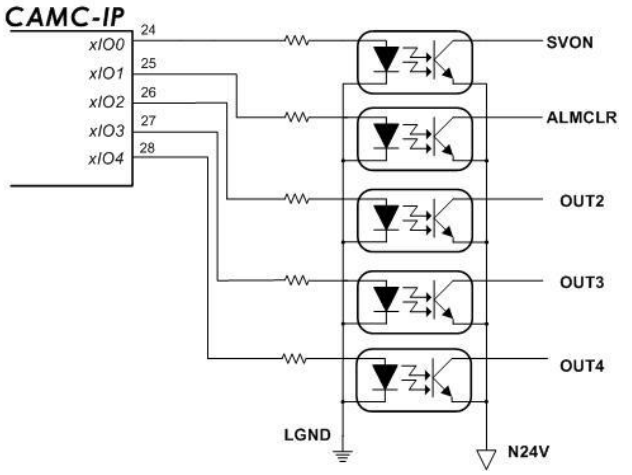
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uValue	Universal output value (Hex Value) – 0 bit : Servo On/Off – 1 bit : Remove alarm signal – 2 ~ 3 bit : General-purpose Input/Output

Return Values

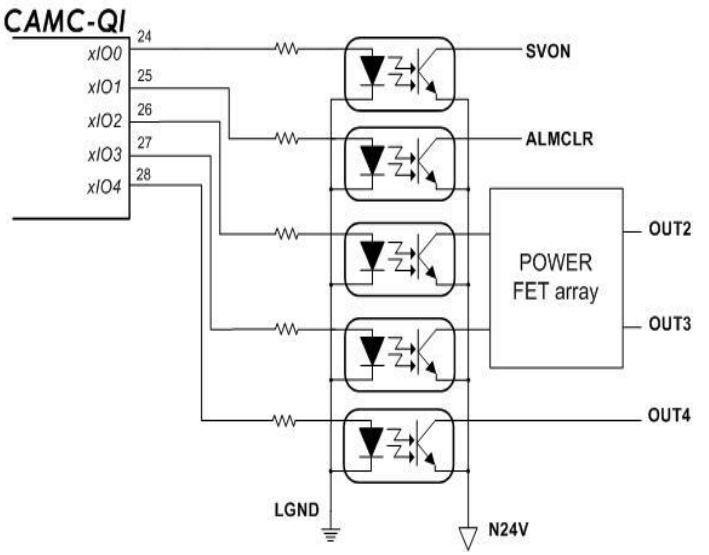
[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Universal output of the board ranges from 0 to 4. By default, universal(0) is used for Servo On, and universal(1) is used for alarm clear output signal. AxmSignalWriteOutput API is used when multiple outputs are used simultaneously, while AxmSignalWriteOutputBit API is used when only one output is necessary.



The difference between IP and PCI-N804/404 is that PCI-N804/404 in the universal 2, 3 is outputted enough for current to relay move because its output is from the photo coupler through the power FET.



Product	Universal Input	Universal Output
PCI-N804 PCI-N404 RTEX-PM	UIN(0) : Home Signal UIN(1) : Encoder Z phase Signal UIN(2~4) : Universal Input Signal	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2~4) : Universal Output Signal
RTEX-A4N	UIN(0) : Home Signal UIN(1~4) : Universal Input (EX-IN1~4)	UOUT(0) : Servo On/Off Signal UOUT(1) : Not Available UOUT(2~3) : Universal Output (EX-OUT1~2)
JEPMC-PL2910	UIN(0) : Home Signal (ZRNn Signal)	UOUT(0) : Servo On/Off Signal

	UIN(1) : Not Available UIN(2) : Universal Input Signal (INn)	UOUT(1) : Alarm Clear Signal UOUT(2) : Universal Output Signal (OUTn)
SGDV-xxxx11A	UIN(0) : Home Signal (EXT1 Signal) UIN(1) : Encoder Z phase(PC Signal) UIN(2) : +End limit(EXT2 Signal) UIN(3) : -End limit(EXT3 Signal)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal

Example

C++

```
// Set the universal output signal on axis 0.
AxmSignalWriteOutput(0, 1);

// Verify the universal output signal on axis 0.
DWORD uValue;
AxmSignalReadOutput (0, &uValue);
```

Visual Basic

```
' Set the universal output signal on axis 0.
AxmSignalWriteOutput 0, 1

' Verify the universal output signal on axis 0.
Dim uValue As Long
AxmSignalReadOutput 0, uValue
```

Delphi

```
{ Verify the universal output signal on axis 0. }
var
  uValue: DWord;

begin
  { Set the universal output signal on axis 0. }
  AxmSignalWriteOutput(0, 1);
  AxmSignalReadOutput (0, @uValue);
end;
```

See Also

AxmSignalServoOn, AxmSignalIsServoOn, AxmSignalServoAlarmReset, AxmSignalReadOutput, AxmSignalWriteOutputBit, AxmSignalReadOutputBit, AxmSignalReadInput, AxmSignalReadInputBit

AxmSignalReadOutput

Purpose

Return the value of universal output in a hexadecimal value.

Format

C++

```
DWORD AxmSignalReadOutput(long lAxisNo, DWORD *upValue);
```

Visual Basic

```
Function AxmSignalReadOutput(ByVal lAxisNo As Long, ByRef upValue As
Long) As Long
```

Delphi

```
function AxmSignalReadOutput(lAxisNo : LongInt; upValue : PDword) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upValue	Universal output value (Hex value) – 0 bit : Servo On/Off – 1 bit : Remove alarm signal – 2 ~ 3 bit : General-purpose Input/Output

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the universal output value set by 'AxmSignalWriteOutput'.

Example

C++

```
// Set the universal output signal on axis 0.
AxmSignalWriteOutput(0, 1);

// Verify the universal output signal on axis 0.
DWORD uValue;
AxmSignalReadOutput (0, &uValue);
```

Visual Basic

```
' Set the universal output signal on axis 0.
AxmSignalWriteOutput 0, 1
```

```
' Verify the universal output signal on axis 0.
Dim uValue As Long
AxmSignalReadOutput 0, uValue
```

Delphi

```
{ Verify the universal output signal on axis 0. }
var
  uValue: DWord;

begin
  { Set the universal output signal on axis 0. }
  AxmSignalWriteOutput(0, 1);
  AxmSignalReadOutput (0, @uValue);
end;
```

See Also

AxmSignalServoOn, AxmSignalIsServoOn, AxmSignalServoAlarmReset, AxmSignalWriteOutput, AxmSignalWriteOutputBit, AxmSignalReadOutputBit, AxmSignalReadInput, AxmSignalReadInputBit

AxmSignalWriteOutputBit

Purpose

Set On/Off state of a specific bit in universal output on a specific axis.

Format

C++

```
DWORD AxmSignalWriteOutputBit(long lAxisNo, long lBitNo, DWORD
uOnOff);
```

Visual Basic

```
Function AxmSignalWriteOutputBit(ByVal lAxisNo As Long, ByVal lBitNo
As Long, ByVal uOnOff As Long) As Long
```

Delphi

```
function AxmSignalWriteOutputBit(lAxisNo : LongInt; lBitNo : LongInt;
uOnOff : DWord) : DWord; stdcall;
```

Parameters

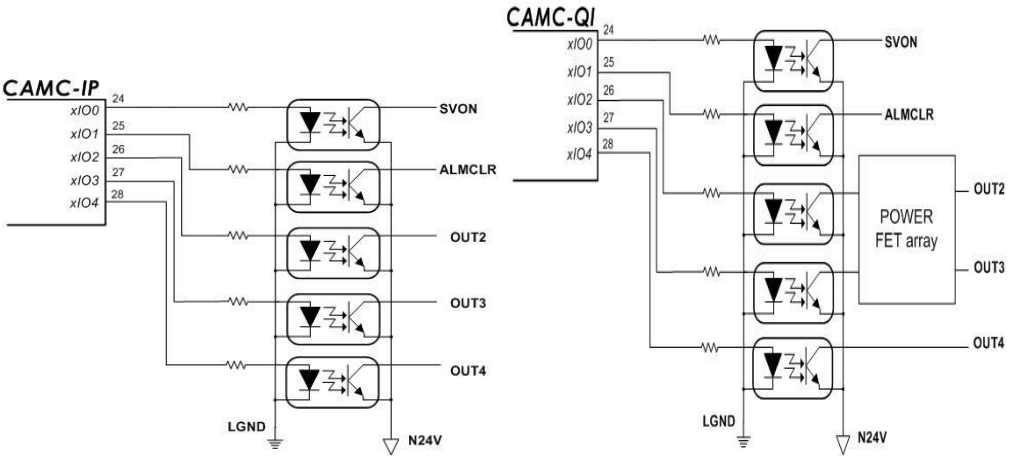
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lBitNo	Universal output bit number : AXT_MOTION_UNIV_OUTPUT_DEF – [00h] Servo On output (SVON) – [01h] Alarm clear output (ALARMCLEAR) – [02h] Universal output 2 – [03h] Universal output 3 – [04h] Universal output 4
[>>]uOnOff	Output On/Off – [00h] Output is Off (0) – [01h] Output is On (1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4105] AXT_RT_MOTION_INVALID_BIT_NO : Invalid setting for general purpose input/output bit
* See error code Table for more information on status error codes

Description

Universal output of the board ranges from 0 to 4. By default, universal(0) is used for Servo On, and universal(1) is used for alarm clear output signal. AxmSignalWriteOutput API is used when multiple outputs are used simultaneously, while AxmSignalWriteOutputBit API is used when only one output is necessary.



Product	Universal Input	Universal Output
PCI-N804 PCI-N404 RTEX-PM	UIN(0) : Home Signal UIN(1) : Encoder Z phase Signal UIN(2~4) : Universal Input Signal	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2~4) : Universal Output Signal
RTEX-A4N	UIN(0) : Home Signal UIN(1~4) : Universal Input (EX-IN1~4)	UOUT(0) : Servo On/Off Signal UOUT(1) : Not Available UOUT(2~3) : Universal Output (EX-OUT1~2)
JEPMC-PL2910	UIN(0) : Home Signal (ZRNn Signal) UIN(1) : Not Available UIN(2) : Universal Input Signal (INn)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2) : Universal Output Signal (OUTn)
SGDV-xxxx11A	UIN(0) : Home Signal (EXT1 Signal) UIN(1) : Encoder Z phase(PC Signal) UIN(2) : +End limit(EXT2 Signal) UIN(3) : -End limit(EXT3 Signal)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal

Example

C++

```
// Universal output signal 4 on axis 0 is ON.
AxmSignalWriteOutputBit(0, 4, 1);

// Verify universal output signal 4 on axis 0.
DWORD uOn;
AxmSignalReadOutputBit(0, 4, &uOn);
```

Visual Basic

```
' Universal output signal 4 on axis 0 is ON.
AxmSignalWriteOutputBit 0, 4, 1

' Verify universal output signal 4 on axis 0.
Dim uOn As Long
```

```
AxmSignalReadOutputBit 0, 4, uOn
```

Delphi

```
{ Verify universal output signal 4 on axis 0. }
var
   uOn: DWord;

begin
   { Universal output signal 4 on axis 0 is ON. }
   AxmSignalWriteOutputBit(0, 4, 1);
   AxmSignalReadOutputBit (0, 4, @uOn);
end;
```

See Also

AxmSignalServoOn, AxmSignalIsServoOn, AxmSignalServoAlarmReset, AxmSignalWriteOutput, AxmSignalReadOutput, AxmSignalReadOutputBit, AxmSignalReadInput, AxmSignalReadInputBit

AxmSignalReadOutputBit

Purpose

Return On/Off state of a specific bit in universal output on a specific axis.

Format

C++

```
DWORD AxmSignalReadOutputBit(long lAxisNo, long lBitNo, DWORD
*upOnOff);
```

Visual Basic

```
Function AxmSignalReadOutputBit(ByVal lAxisNo As Long, ByVal lBitNo
As Long, ByRef upOnOff As Long) As Long
```

Delphi

```
function AxmSignalReadOutputBit(lAxisNo : LongInt; lBitNo : LongInt;
upOnOff : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lBitNo	Universal output bit number : AXT_MOTION_UNIV_OUTPUT_DEF – [00h] Servo On output (SVON) – [01h] Alarm clear output (ALARMCLEAR) – [02h] Universal output 2 – [03h] Universal output 3 – [04h] Universal output 4
[<<]upOnOff	Output On/Off – [00h] Output is Off (0) – [01h] Output is On (1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4105] AXT_RT_MOTION_INVALID_BIT_NO : Invalid setting for general purpose input/output bit
* See error code Table for more information on status error codes

Description

It returns the universal output value set by 'AxmSignalWriteOutputBit'.

Example

C++

```
// Universal output signal 4 on axis 0 is ON.
AxmSignalWriteOutputBit(0, 4, 1);

// Verify universal output signal 4 on axis 0.
```

```
DWORD uOn;
AxmSignalReadOutputBit(0, 4, &uOn);
```

Visual Basic

```
' Universal output signal 4 on axis 0 is ON.
AxmSignalWriteOutputBit 0, 4, 1

' Verify universal output signal 4 on axis 0.
Dim uOn As Long
AxmSignalReadOutputBit 0, 4, uOn
```

Delphi

```
{ Verify universal output signal 4 on axis 0. }
var
  uOn: DWord;

begin
  { Universal output signal 4 on axis 0 is ON. }
  AxmSignalWriteOutputBit(0, 4, 1);
  AxmSignalReadOutputBit (0, 4, @uOn);
end;
```

See Also

AxmSignalServoOn, AxmSignalIsServoOn, AxmSignalServoAlarmReset, AxmSignalWriteOutput, AxmSignalReadOutput, AxmSignalWriteOutputBit, AxmSignalReadInput, AxmSignalReadInputBit

AxmSignalReadInput

Purpose

Return the value of universal input in a hexadecimal value.

Format

C++

```
DWORD AxmSignalReadInput(long lAxisNo, DWORD *upValue);
```

Visual Basic

```
Function AxmSignalReadInput(ByVal lAxisNo As Long, ByRef upValue As
Long) As Long
```

Delphi

```
function AxmSignalReadInput(lAxisNo: Long, upValue: PLong) : DWord;
stdcall;
```

Parameters

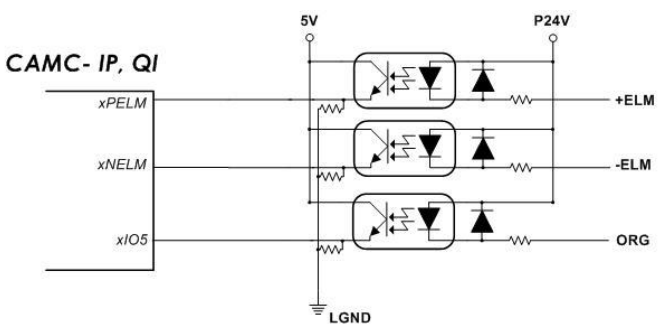
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upValue	Universal input value

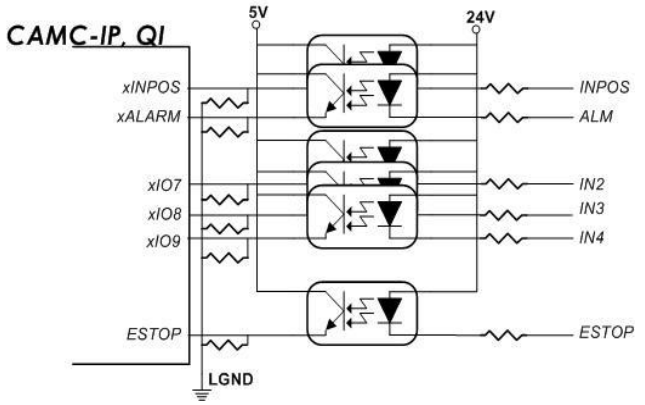
Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns universal input 0,1,2,3 in a hexadecimal value.





Product	Universal Input	Universal Output
SMC-2V04 PCI-N404/804 RTEX-PM	UIN(0) : Home Signal UIN(1) : Encoder Z phase Signal UIN(2~4) : Universal Input Signal	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2~4) : Universal Output Signal
RTEX-A4N	UIN(0) : Home Signal UIN(1~4) : Universal Input (EX-IN1~4)	UOUT(0) : Servo On/Off Signal UOUT(1) : Not Available UOUT(2~3) : Universal Output (EX-OUT1~2)
JEPMC-PL2910	UIN(0) : Home Signal (ZRNn Signal) UIN(1) : Not Available UIN(2) : Universal Input Signal (INn)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2) : Universal Output Signal (OUTn)
SGDV-xxxx11A	UIN(0) : Home Signal (EXT1 Signal) UIN(1) : Encoder Z phase(PC Signal) UIN(2) : +End limit(EXT2 Signal) UIN(3) : -End limit(EXT3 Signal)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal

Example

C++

```
// Verify universal input signal on axis 0.
DWORD uValue;
AxmSignalReadInput (0, &uValue);
```

Visual Basic

```
' Verify universal input signal on axis 0.
Dim uValue As Long
AxmSignalReadInput 0, uValue
```

Delphi

```
{ Verify universal input signal on axis 0. }
var
  uValue: DWord;

begin
  AxmSignalReadInput (0, @uValue);
```

```
end;
```

See Also

AxmSignalServoOn, AxmSignalIsServoOn, AxmSignalServoAlarmReset, AxmSignalWriteOutput, AxmSignalReadOutput, AxmSignalWriteOutputBit, AxmSignalReadOutputBit, AxmSignalReadInputBit

AxmSignalReadInputBit

Purpose

Return On/Off state of a corresponding bit in universal input on a specific axis.

Format

C++

```
DWORD AxmSignalReadInputBit(long lAxisNo, long lBitNo, DWORD *upOn);
```

Visual Basic

```
Function AxmSignalReadInputBit(ByVal lAxisNo As Long, ByVal lBitNo As
Long ,ByRef upOn As Long) As Long
```

Delphi

```
function AxmSignalReadInputBit(lAxisNo : LongInt; lBitNo : LongInt;
upOn : PDWord) : DWord; stdcall;
```

Parameters

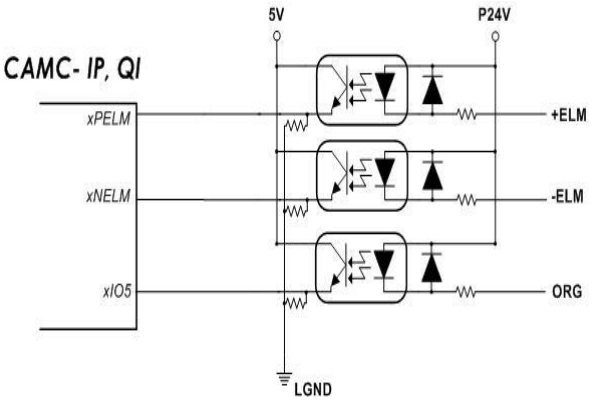
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lBitNo	Universal output bit number : AXT_MOTION_UNIV_INPUT_DEF – [00h] Home sensor input – [01h] Z phase input – [02h] Universal input 2 – [03h] Universal input 3 – [04h] Universal input 4
[<<]upOn	On/Off state of the input bit – [00h] Output is Off (0) – [01h] Output is On (1)

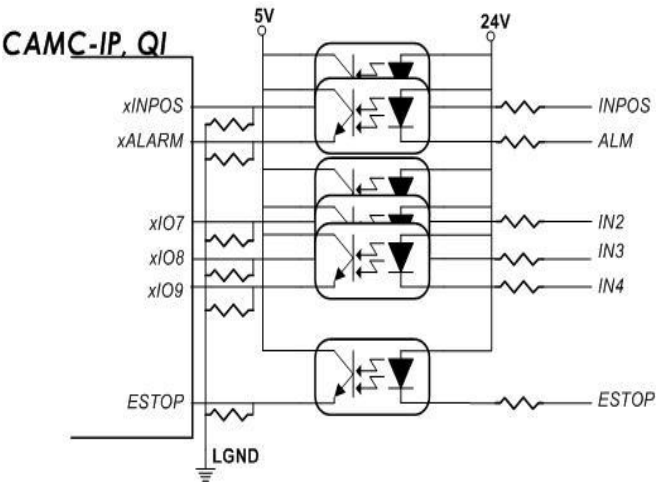
Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the corresponding bit in universal input on a specific axis in a bit value.





Product	Universal Input	Universal Output
SMC-2V04 PCI-N404/804 RTEX-PM	UIN(0) : Home Signal UIN(1) : Encoder Z phase Signal UIN(2~4) : Universal Input Signal	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2~4) : Universal Output Signal
RTEX-A4N	UIN(0) : Home Signal UIN(1~4) : Universal Input (EX-IN1~4)	UOUT(0) : Servo On/Off Signal UOUT(1) : Not Available UOUT(2~3) : Universal Output (EX-OUT1~2)
JEPMC-PL2910	UIN(0) : Home Signal (ZRNn Signal) UIN(1) : Not Available UIN(2) : Universal Input Signal (INn)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal UOUT(2) : Universal Output Signal (OUTn)
SGDV-xxxx11A	UIN(0) : Home Signal (EXT1 Signal) UIN(1) : Encoder Z phase(PC Signal) UIN(2) : +End limit(EXT2 Signal) UIN(3) : -End limit(EXT3 Signal)	UOUT(0) : Servo On/Off Signal UOUT(1) : Alarm Clear Signal

Example

C++

```
// Verify the value of universal input 4 on axis 0.
DWORD uOn;
AxmSignalReadInputBit(0, 4, &uOn);
```

Visual Basic

```
' Verify the value of universal input 4 on axis 0.
Dim uOn As Long
AxmSignalReadInputBit 0, 4, uOn
```

Delphi

```
{ Verify the value of universal input 4 on axis 0. }
var
  uOn: DWord;

begin
  AxmSignalReadInputBit (0, 4, @uOn);
end;
```

See Also

AxmSignalServoOn, AxmSignalIsServoOn, AxmSignalServoAlarmReset, AxmSignalWriteOutput, AxmSignalReadOutput, AxmSignalWriteOutputBit, AxmSignalReadOutputBit

Checking Status API

This chapter explains the APIs that are related to the status of motion control. Status APIs are groups of APIs required to watch motion status. Motion statuses contain checking the motion's velocity, acceleration, deceleration, position, and whether it is in-motion. Using the status APIs also allows checking the reason of stop when it is stopped, and the difference between Command values and Actual values.

Function	Description
AxmStatusReadInMotion	Return pulse out state on a specific axis.
AxmStatusReadDrivePulseCount	Return moving pulse counter value on a specific axis.
AxmStatusReadMotion	Return DriveStatus register on a specific axis.
AxmStatusReadStop	Return EndStatus register on a specific axis.
AxmStatusReadMechanical	Return Mechanical Signal Data on a specific axis.
AxmStatusReadVel	Read current moving velocity on a specific axis.
AxmStatusReadPosError	Return the difference between Command Position and Actual Position.
AxmStatusReadDriveDistance	Check the moving distance to the final drive.
AxmStatusSetActPos	Set Act position on a specific axis.
AxmStatusGetActPos	Return Act position on a specific axis.
AxmStatusSetCmdPos	Set Cmd position on a specific axis.
AxmStatusGetCmdPos	Return Cmd position on a specific axis.
AxmStatusRequestServoAlarm	Send the request to read the Alarm Code on a specific axis.
AxmStatusReadServoAlarm	Read the Alarm Code on a specific axis.
AxmStatusGetServoAlarmString	Return the Alarm String of error code on a specific axis.
AxmStatusRequestServoAlarmHistory	Send the request to read the Alarm History on a specific axis.
AxmStatusReadServoAlarmHistory	Read the Alarm History on a specific axis.
AxmStatusClearServoAlarmHistory	Clear the Alarm History on a specific axis.

AxmStatusReadInMotion

Purpose

Return whether the motion is completed on a specific axis.

Format

C++

```
DWORD AxmStatusReadInMotion(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmStatusReadInMotion(ByVal lAxisNo As Long, ByRef upStatus
As Long) As Long
```

Delphi

```
function AxmStatusReadInMotion(lAxisNo : LongInt; upStatus : PDWord) :
DWord; stdcall;
```

Parameters

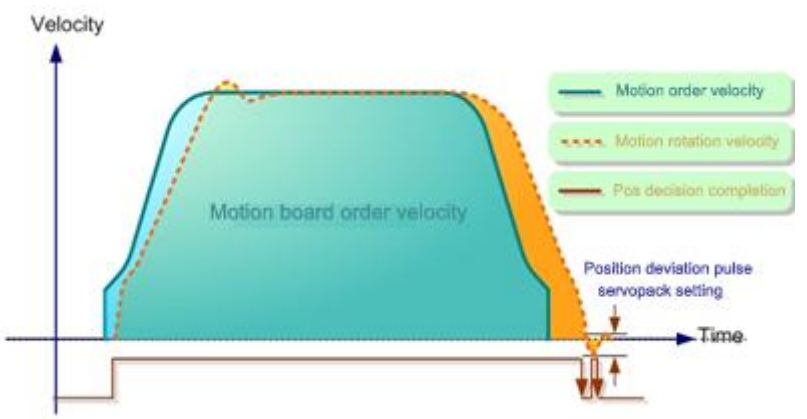
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (starting from 0)
[<<]upStatus	Motion move status : FALSE & TRUE - [00h] Not in-motion - [01h] In-motion

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

*upStatus = 1 is outputted if the corresponding axis is in motion. Otherwise, *upStatus = 0 is outputted.



▶ If Inpos input signal is set to Enable when AxmSignalSetInpos API is being used, command pulse out is not returned because it is considered that motion is not completed until INP input becomes ON although Command pulse out is completed. Please refer to AxmSignalSetInpos API regarding this matter.

Example

C++

```
// Verify the moving state on axis 0.
DWORD Status;
AxmStatusReadInMotion(0, &Status);
if (Status == FALSE)
  AfxMessageBox("The motor has been stopped.");
```

Visual Basic

```
' Verify the moving state on axis 0.
Dim Status As Long
AxmStatusReadInMotion 0, Status
If Status = FALSE Then
  MsgBox "The motor has been stopped.", vbOKCancel
End If
```

Delphi

```
{ Verify the moving state on axis 0. }
var
  Status: DWord;

begin
  AxmStatusReadInMotion(0, @Status);
  if (Status = 0) then
     Application.MessageBox('The motor has been stopped.', 'Ajinextek', MB_OK);
end;
```

See Also

AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadDrivePulseCount

Purpose

Return the value of the pulse counter from the start point to the end point for the current motion on a specific axis.

Format

C++

```
DWORD AxmStatusReadDrivePulseCount(long lAxisNo, long *lpPulse);
```

Visual Basic

```
Function AxmStatusReadDrivePulseCount(ByVal lAxisNo As Long, ByRef
lpPulse As Long) As Long
```

Delphi

```
function AxmStatusReadDrivePulseCount(lAxisNo : LongInt; lpPulse ;
PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]lpPulse	Moving pulse counter value

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the pulse counter value from the start point to the end point for the current motion on a specific axis. After motion move is completed, it maintains the value of previous out pulse counter first. Then it updates the value of pulse counter when a new motion move starts.

Note: In case of SMC-2V03 module, it has counter value after move exit; in case of N404 and N804, counter value is only shown during move and it is cleared after move exit.

Example

C++

```
// Get the pulse counter on axis 0.
long    Pulse;
AxmStatusReadDrivePulseCount (0, &Pulse);
```

Visual Basic

```
' Get the pulse counter on axis 0.
Dim Pulse As Long
```

```
AxmStatusReadDrivePulseCount 0, Pulse
```

Delphi

```
{ Get the pulse counter on axis 0. }
var
   Pulse: LongInt;

begin
   AxmStatusReadDrivePulseCount (0, @Pulse);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadMotion

Purpose

Return the value of motion moving state on a specific axis.

Format

C++

```
DWORD AxmStatusReadMotion(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmStatusReadMotion(ByVal lAxisNo As Long, ByRef upStatus As
Long) As Long
```

Delphi

```
function AxmStatusReadMotion(lAxisNo : LongInt; upStatus : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStatus	Motion move state on a specific axis : AXT_MOTION_QIDRIVE_STATUS_DEF - [00000001h] Bit 0, BUSY (In DRIVE) - [00000002h] Bit 1, DOWN (In Deceleration) - [00000004h] Bit 2, CONST(In Constant Velocity) - [00000008h] Bit 3, UP(In Acceleration) - [00000010h] Bit 4, In Continuous Drive is in move - [00000020h] Bit 5, In Preset Distance Drive is in move - [00000040h] Bit 6, In MPG Drive is in move - [00000080h] Bit 7, In Home Search Drive is in move - [00000100h] Bit 8, In Signal Search Drive is in move - [00000200h] Bit 9, In Interpolation Drive is in move - [00000400h] Bit 10, In Slave Drive is in move - [00000800h] Bit 11, Currently Moving Drive Direction (Display information is different on interpolation drive) - [00001000h] Bit 12, Waiting for Servo Position Exit Signal after Pulse Out - [00002000h] Bit 13, In Linear Interpolation Drive is in move - [00004000h] Bit 14, In Circular Interpolation Drive is in move - [00008000h] Bit 15, In Pulse Out - [00010000h] Bit 16, Number of Moving-Reserved Data(Start)(0-7) - [00020000h] Bit 17, Number of Moving-Reserved Data(Middle)(0-7) - [00040000h] Bit 18, Number of Moving-Reserved Data(End)(0-7) - [00080000h] Bit 19, Moving-Reserved Queue is empty - [00100000h] Bit 20, Moving-Reserved Queue is full - [00200000h] Bit 21, Velocity mode of current moving drive(Start) - [00400000h] Bit 22, Velocity mode of current moving drive (End) - [00800000h] Bit 23, MPG Buffer #1 Full - [01000000h] Bit 24, MPG Buffer #2 Full - [02000000h] Bit 25, MPG Buffer #3 Full - [04000000h] Bit 26, MPG Buffer Data OverFlow

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return motion move status on specific axis.

Example

C++

```
DWORD drivestatus;
// Read the moving state of axis 0.
AxmStatusReadMotion (0, &drivestatus);
if(drivestatus & QIDRIVE_STATUS_0)
{
  AfxMessageBox("Axis 0 is driven.");
}
else
{
  AfxMessageBox("Axis 0 is not driven.");
}
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadStop

Purpose

Return the state about motion exit on a specific axis.

Format

C++

```
DWORD AxmStatusReadStop(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmStatusReadStop(ByVal lAxisNo As Long, ByRef upStatus As
Long) As Long
```

Delphi

```
function AxmStatusReadStop(lAxisNo : LongInt; upStatus : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStatus	Motion exit state of an axis : AXT_MOTION_QIEND_STATUS_DEF - [00000000h] Not in move or initial state - [00000001h] Bit 0, Exit by positive limit signal(PELM) - [00000002h] Bit 1, Exit by negative limit signal(NELM) - [00000004h] Bit 2, Move exited by positive supplementary limit signal(PSLM) - [00000008h] Bit 3, Move exited by negative supplementary limit signal(NSLM) - [00000010h] Bit 4, Move exited by positive soft limit emergency stop function - [00000020h] Bit 5, Move exited by negative soft limit emergency stop function - [00000040h] Bit 6, Move exited by positive soft limit deceleration stop function - [00000080h] Bit 7, Move exited by negative soft limit deceleration stop function - [00000100h] Bit 8, Move exited by servo alarm function - [00000200h] Bit 9, Move exited by emergency stop signal input - [00000400h] Bit 10, Move exited by emergency stop command - [00000800h] Bit 11, Move exited by deceleration stop command - [00001000h] Bit 12, Move exited by emergency stop command on all axes - [00002000h] Bit 13, Move exited by sync stop function #1(SQSTP1) - [00004000h] Bit 14, Move exited by sync stop function #2(SQSTP2) - [00008000h] Bit 15, Encoder input(ECUP,ECDN) error generated - [00010000h] Bit 16, MPG input(EXPP,EXMP) error generated - [00020000h] Bit 17, Home search successfully exited - [00040000h] Bit 18, Signal search successfully exited - [00080000h] Bit 19, Move exited by interpolation data error - [00100000h] Bit 20, Abnormal move stop generated - [00200000h] Bit 21, MPG function block pulse beffer overflow generated - Bit 22, - Bit27 DON'CARE - [10000000h] Bit 28, Direction of Current/Last moving drive - [20000000h] Bit 29, Outputting remaining pulse removal signal - [40000000h] Bit 30, Cause of abnormal stop - [80000000h] Bit 31, Interpolation drive data error

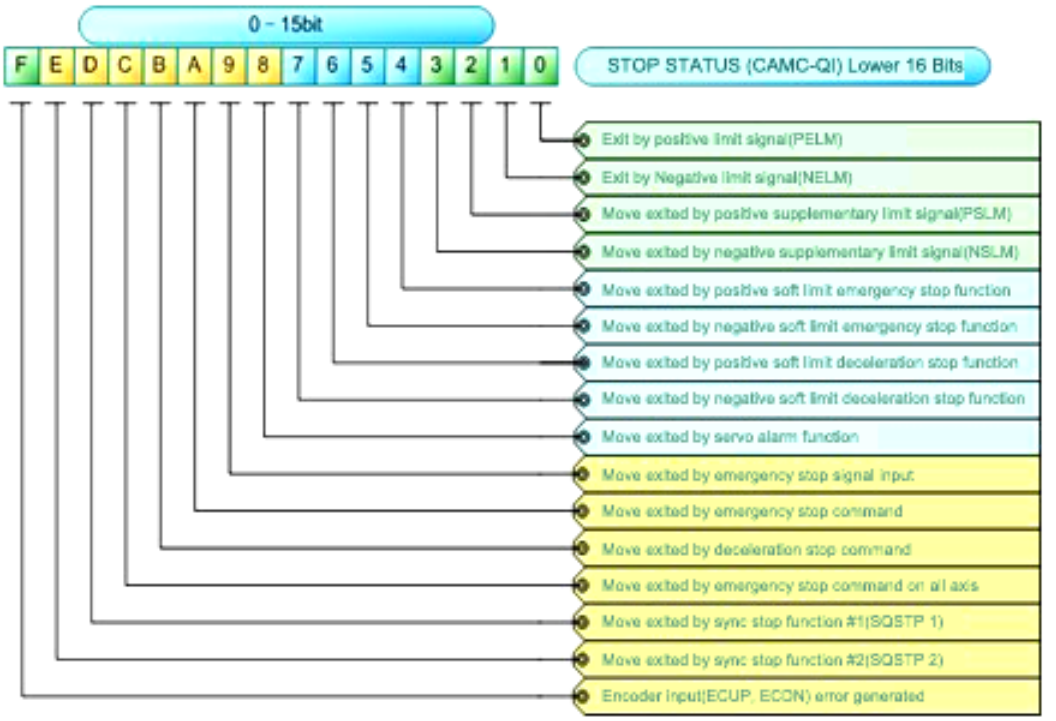
Return Values

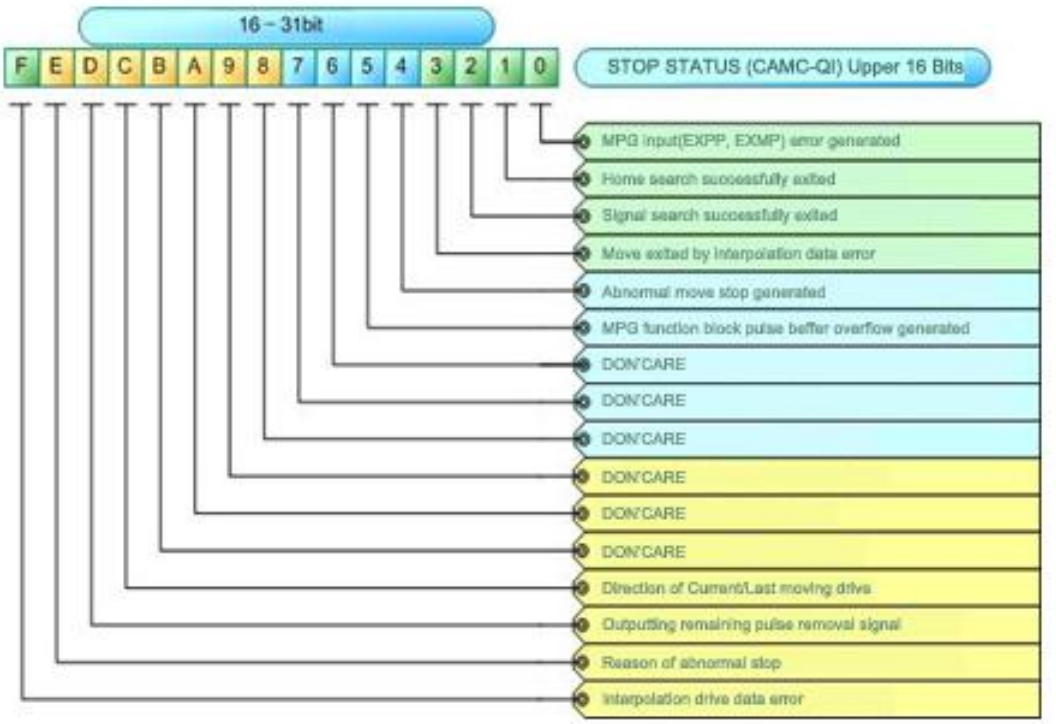
[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the value of motion exit state on a specific axis. Using this API, the information about whether motion has normally exited or stopped due to the reason such as external sensor, stop command, or motion error, can be verified.

Definition of Each Bit in Stop Status





Example

C++

```
DWORD drivestatus;
// Read the exit state of axis 0.
AxmStatusReadMotion (0, &drivestatus);
if(drivestatus & QIDRIVE_STATUS_0)
{
  AfxMessageBox("Axis 0 is driven.");
}
else
{
  AfxMessageBox("Axis 0 is not driven.");
}
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadMechanical

Purpose

Return the state of external sensors and mechanical signals on a specific axis.

Format

C++

```
DWORD AxmStatusReadMechanical(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmStatusReadMechanical(ByVal lAxisNo As Long, ByRef
upStatus As Long) As Long
```

Delphi

```
function AxmStatusReadMechanical(lAxisNo : LongInt; upStatus :
PDWord) : DWord; stdcall;
```

Parameters

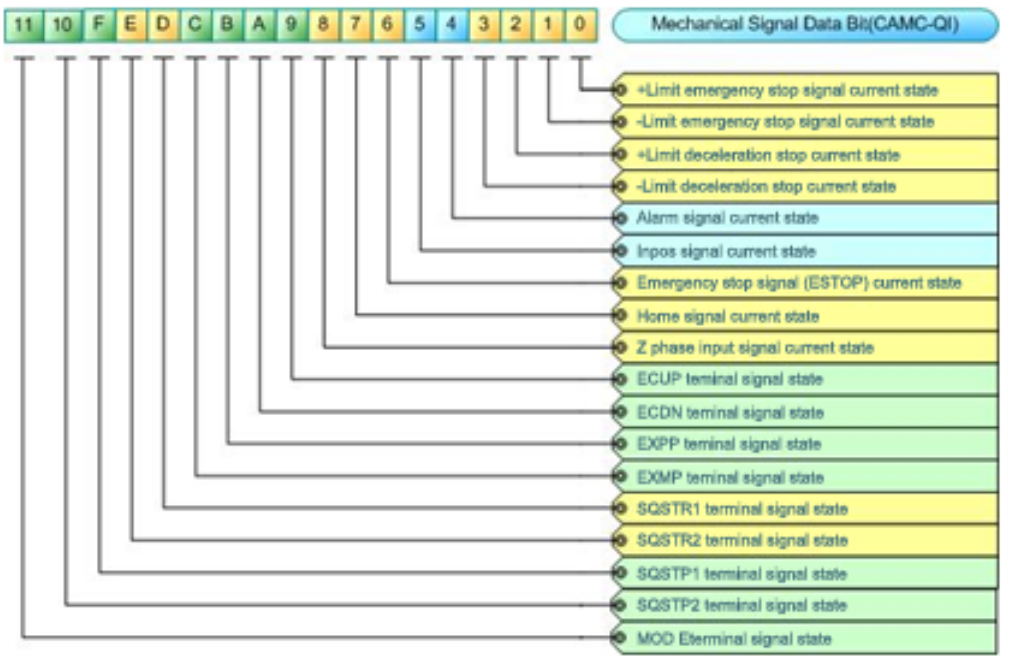
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStatus	External sensor and Motion Mechenical signal status : AXT_MOTION_QIMECHANICAL_SIGNAL_DEF – [00001h] Bit 0, +Limit emergency stop signal current state – [00002h] Bit 1, –Limit emergency stop signal current state – [00004h] Bit 2, +limit deceleration stop current state – [00008h] Bit 3, –limit deceleration stop current state – [00010h] Bit 4, Alarm signal current state – [00020h] Bit 5, InPos signal current state – [00040h] Bit 6, Emergency stop signal (ESTOP) current state – [00080h] Bit 7, Home signal current state – [00100h] Bit 8, Z phase input signal current state – [00200h] Bit 9, ECUP terminal signal state – [00400h] Bit 10, ECDN terminal signal state – [00800h] Bit 11, EXPP terminal signal state – [01000h] Bit 12, EXMP terminal signal state – [02000h] Bit 13, SQSTR1 terminal signal state – [04000h] Bit 14, SQSTR2 terminal signal state – [08000h] Bit 15, SQSTP1 terminal signal state – [10000h] Bit 16, SQSTP2 terminal signal state – [20000h] Bit 17, MODE terminal signal state

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the state of external sensors and Mechanical Signals on a specific axis. The Active Level of each state signal can be verified. The values of the Mechanical Signal state on the axis specified by a user are shown on the figure below.



Example

C++

```
// Read sensoring state of each sensor on axis 0.
DWORD Status;
AxmStatusReadMechanical (0, &Status);
```

Visual Basic

```
' Read sensoring state of each sensor on axis 0.
Dim Status As Long
AxmStatusReadMechanical 0, Status
```

Delphi

```
{ Read sensoring state of each sensor on axis 0. }
var
  Status: DWord;

begin
  AxmStatusReadMechanical (0, @Status);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadVel

Purpose

Read current moving velocity on a specific axis.

Format

C++

```
DWORD AxmStatusReadVel(long lAxisNo, Double *dpVelocity);
```

Visual Basic

```
Function AxmStatusReadVel(ByVal lAxisNo As Long, ByRef dpVelocity As
Double) As Long
```

Delphi

```
function AxmStatusReadVel(lAxisNo : LongInt; dpVelocity : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpVelocity	Current velocity (unit/sec)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It reads current moving velocity on the axis specified by a user. The unit of the velocity is unit/sec.

Example

C++

```
double     velocity;
AxmMoveStartPos(0, 1000, 100, 200, 200);

// Return the current moving velocity on a specific axis.
AxmStatusReadVel (0, &Velocity);
```

Visual Basic

```
Dim velocity As Double
AxmMoveStartPos 0, 1000, 100, 200, 200
' Return the current moving velocity on a specific axis.
AxmStatusReadVel 0, Velocity
```

Delphi

```
var
  Velocity : Double;

begin
  AxmMoveStartPos(0, 1000, 100, 200, 200);
  { Return the current moving velocity on a specific axis. }
  AxmStatusReadVel (0, @Velocity);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadPosError

Purpose

Return the difference between Command Position and Actual Position on a specific axis.

Format

C++

```
DWORD AxmStatusReadPosError(long lAxisNo, double *dpError);
```

Visual Basic

```
Function AxmStatusReadPosError(ByVal lAxisNo As Long, ByRef dpError
As Double) As Long
```

Delphi

```
function AxmStatusReadPosError(lAxisNo : LongInt; dpError : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpError	Difference between Command Position and Actual Position

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1001] AXT_RT_OPEN_ERROR : Library is not open
[1002] AXT_RT_OPEN_ALREADY : Library is already open
* See error code Table for more information on status error codes

Description

You can check if the output pulse from the axis controller specified by a user is normally operating by returning the difference between internal Command Position and Actual Position(Encoder). Step motor can also be verified if an Encoder is attached as a supplementary. This API cannot be used without Encoder.

Example

C++

```
// Verify the difference between Cmd Pos and Act Pos on axis 0.
double Error;
AxmStatusReadPosError (0, &Error);
```

Visual Basic

```
' Verify the difference between Cmd Pos and Act Pos on axis 0.
Dim Error As Double
AxmStatusReadPosError 0, Error
```

Delphi

```
{ Verify the difference between Cmd Pos and Act Pos on axis 0. }
var
  Error: Double;

begin
  AxmStatusReadPosError (0, @Error);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadDriveDistance

Purpose

Return the position driving distance from the start point to the end point of the current motion on a specific axis.

Format

C++

```
DWORD AxmStatusReadDriveDistance(long lAxisNo, double *dpUnit);
```

Visual Basic

```
Function AxmStatusReadDriveDistance(ByVal lAxisNo As Long, ByRef
dpUnit As Double) As Long
```

Delphi

```
function AxmStatusReadDriveDistance(lAxisNo : LongInt; dpUnit :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpUnit	Driving distance

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

After motion move is completed, it maintains the previous driving distance first then it updates the value of driving distance when a new motion move starts.

Note: In case of SMC-2V03 module, it has counter value after move exit; in case of N404 and N804, counter value is only shown during move and it is cleared after move exit.

Example

C++

```
// Set the current position to '0' on axis 0 and verify the output pulse.
double Unit;
AxmStatusReadDriveDistance (0, &Unit);
```

Visual Basic

```
' Set the current position to '0' on axis 0 and verify the output pulse.
Dim Unit As Double
AxmStatusReadDriveDistance 0, Unit
```

Delphi

```
{ Set the current position to '0' on axis 0 and verify the output pulse. }
var
  Unit: Double;

begin
  AxmStatusReadDriveDistance (0, @Unit);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusSetActPos

Purpose

Set Actual position(Encoder value) on a specific axis.

Format

C++

```
DWORD AxmStatusSetActPos(long lAxisNo, double dPos);
```

Visual Basic

```
Function AxmStatusSetActPos(ByVal lAxisNo As Long, ByVal dPos As
Double) As Long
```

Delphi

```
function AxmStatusSetActPos(lAxisNo : LongInt; dPos : Double) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dPos	Act position

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It can be forced to set Actual position on the axis specified by a user. It is mainly used to set the current position to 0.

Example

C++

```
// Set the current position to the encoder position '0' on axis 0.
AxmStatusSetActPos (0, 0);

// Verify encoder position on axis 0.
double Pos;
AxmStatusGetActPos (0, &Pos);
```

Visual Basic

```
' Set the current position to the encoder position '0' on axis 0.
AxmStatusSetActPos 0, 0
```

```
' Verify encoder position on axis 0.
Dim Pos As Double
AxmStatusGetActPos 0, Pos
```

Delphi

```
{ Verify encoder position on axis 0. }
var
   Pos: Double;

begin
   { Set the current position to the encoder position '0' on axis 0. }
   AxmStatusSetActPos (0, 0);
   AxmStatusGetActPos (0, @Pos);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusGetActPos

Purpose

Return Actual position(Encoder value) on a specific axis..

Format

C++

```
DWORD AxmStatusGetActPos(long lAxisNo, double *dpPos);
```

Visual Basic

```
Function AxmStatusGetActPos(ByVal lAxisNo As Long, ByRef dpPos As
Double) As Long
```

Delphi

```
function AxmStatusGetActPos(lAxisNo : LongInt; dpPos : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpPos	Act position

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the Actual position read by the encoder on the axis specified by a user.

Example

C++

```
// Set the current position to the encoder position '0' on axis 0.
AxmStatusSetActPos (0, 0);
// Verify encoder position on axis 0.
double Pos;
AxmStatusGetActPos (0, &Pos);
```

Visual Basic

```
' Set the current position to the encoder position '0' on axis 0.
AxmStatusSetActPos 0, 0
' Verify encoder position on axis 0.
Dim Pos As Double
AxmStatusGetActPos 0, Pos
```

Delphi

```
{ Verify encoder position on axis 0. }
var
   Pos: Double;

begin
   { Set the current position to the encoder position '0' on axis 0. }
   AxmStatusSetActPos (0, 0);
   AxmStatusGetActPos (0, @Pos);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusSetCmdPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusSetCmdPos

Purpose

Set Command position on a specific axis.

Format

C++

```
DWORD AxmStatusSetCmdPos(long lAxisNo, double dPos);
```

Visual Basic

```
Function AxmStatusSetCmdPos(ByVal lAxisNo As Long, ByVal dPos As
Double) As Long
```

Delphi

```
function AxmStatusSetCmdPos(lAxisNo : LongInt; dPos : Double) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dPos	Cmd position

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It can be forced to set Command position on the axis specified by a user. It is mainly used to set the current position to 0.

Example

C++

```
// Set the current position to the command position '0' on axis 0.
AxmStatusSetCmdPos (0, 0);
// Verify command position on axis 0.
double Pos;
AxmStatusGetCmdPos (0, &Pos);
```

Visual Basic

```
' Set the current position to the command position '0' on axis 0.
AxmStatusSetCmdPos 0, 0

' Verify command position on axis 0.
```

```
Dim Pos As Double
AxmStatusGetCmdPos 0, Pos
```

Delphi

```
{ Verify command position on axis 0. }
var
   Pos: Double;

begin
   { Set the current position to the command position '0' on axis 0. }
   AxmStatusSetCmdPos (0, 0);
   AxmStatusGetCmdPos (0, @Pos);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusGetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusGetCmdPos

Purpose

Return Command position on a specific axis.

Format

C++

```
DWORD AxmStatusGetCmdPos(long lAxisNo, double *dpPos);
```

Visual Basic

```
Function AxmStatusGetCmdPos(ByVal lAxisNo As Long, ByRef dpPos As
Double) As Long
```

Delphi

```
function AxmStatusGetCmdPos(lAxisNo : LongInt; dpPos : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpPos	Cmd Position

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the internal Command position of the controller on the axis specified by user.

Example

C++

```
// Set the current position to the command position '0' on axis 0.
AxmStatusSetCmdPos (0, 0);

// Verify command position on axis 0.
double Pos;
AxmStatusGetCmdPos (0, &Pos);
```

Visual Basic

```
' Set the current position to the command position '0' on axis 0.
AxmStatusSetCmdPos 0, 0

' Verify command position on axis 0.
```

```
Dim Pos As Double
AxmStatusGetCmdPos 0, Pos
```

Delphi

```
{ Verify command position on axis 0. }
var
   Pos: Double;

begin
    { Set the current position to the command position '0' on axis 0. }
   AxmStatusSetCmdPos (0, 0);
   AxmStatusGetCmdPos (0, @Pos);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusRequestServoAlarm

Purpose

Send the request to read the Alarm Code on a specific axis.

Format

C++

```
DWORD AxmStatusRequestServoAlarm(long lAxisNo);
```

Visual Basic

```
Function AxmStatusRequestServoAlarm(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmStatusRequestServoAlarm(lAxisNo : LongInt) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
* See error code Table for more information on status error codes

Description

It sends the request to read the Alarm Code on the specified axis of servo pack.

Example

C++

```
// Request the Alarm Code on axis 0.
long lAxisNo = 0;
AxmStatusRequestServoAlarm(lAxisNo);
```

Visual Basic

```
' Request the Alarm Code on axis 0.
Dim lAxisNo As Long
lAxisNo = 0

AxmStatusRequestServoAlarm lAxisNo
```

Delphi

```
{ Request the Alarm Code on axis 0. }
var
  lAxisNo : Long;

begin
```

```
  lAxisNo := 0;
  AxmStatusRequestServoAlarm(lAxisNo);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadServoAlarm

Purpose

Read the Alarm Code on a specific axis.

Format

C++

```
DWORD AxmStatusReadServoAlarm(long lAxisNo, DWORD uReturnMode, DWORD
*upAlarmCode);
```

Visual Basic

```
Function AxmStatusReadServoAlarm(ByVal lAxisNo As Long, ByVal
uReturnMode As Long, ByRef upAlarmCode As Long) As Long
```

Delphi

```
function AxmStatusReadServoAlarm(lAxisNo : LongInt; uReturnMode :
LongInt; upAlarmCode : PDword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uReturnMode	Set the operating condition – [0] Return immediately after executing – [1] Does not return to read the alarm code from servo pack – [2] Return to read the alarm code from servo pack but not the program Blocking
[<<]upAlarmCode	Alarm Code of servo pack

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4210] AXT_RT_MOTION_READ_ALARM_WAITING : The result is waiting from the alarm code of servo pack
[4211] AXT_RT_MOTION_READ_ALARM_NO_REQUEST : Servo pack returns the alarm code when the command has not been taken
[4212] AXT_RT_MOTION_READ_ALARM_TIMEOUT : Reading the alarm of servo pack is timeout (1 sec or more)
[4213] AXT_RT_MOTION_READ_ALARM_FAILED : The alarm failed to read from servo pack
* See error code Table for more information on status error codes

Description

It reads the Alarm Code on the specified axis of servo pack.

Example

C++

```
// Read the Alarm Code on axis 0, Immediate mode.
long lAxisNo = 0;
DWORD uReturnMode = 0;
DWORD *upAlarmCode;
AxmStatusReadServoAlarm(lAxisNo, uReturnMode, &upAlarmCode);
```

Visual Basic

```
' Read the Alarm Code on axis 0, Immediate mode.
Dim lAxisNo As Long
Dim uReturnMode As Long
Dim upAlarmCode As Long
lAxisNo = 0
uReturnMode = 0

AxmStatusReadServoAlarm lAxisNo, uReturnMode, upAlarmCode
```

Delphi

```
{ Read the Alarm Code on axis 0, Immediate mode. }
var
   lAxisNo : Long;
   uReturnMode : Dword;
   upAlarmCode : Dword;

begin
   lAxisNo := 0;
   uReturnMode := 0;
   AxmStatusReadServoAlarm(lAxisNo, uReturnMode, @upAlarmCode);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusRequestServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusGetServoAlarmString

Purpose

Return the Alarm String of error code on a specific axis.

Format

C++

```
DWORD AxmStatusGetServoAlarmString(long lAxisNo, DWORD uAlarmCode,
long lAlarmStringSize, char *szAlarmString);
```

Visual Basic

```
Function AxmStatusGetServoAlarmString(ByVal lAxisNo As Long, ByVal
uAlarmCode As Long, ByVal lAlarmStringSize As Long, ByRef
szAlarmString As String) As Long
```

Delphi

```
function AxmStatusGetServoAlarmString(lAxisNo : LongInt; uAlarmCode :
LongInt; lAlarmStringSize : LongInt; szAlarmString : PChar) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uAlarmCode	Alarm Code of servo pack
[>>]lAlarmStringSize	Size of Alarm String
[<<]szAlarmString	Alarm String

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4220] AXT_RT_MOTION_READ_ALARM_UNKNOWN : Unknown alarm code
[4221] AXT_RT_MOTION_READ_ALARM_FILES : Alarm information file does not exist in the predetermined position
* See error code Table for more information on status error codes

Description

Return the corresponding Alarm String of error code on a specific axis.

Example

C++

```
// Return the Alarm String on axis 0, alarm code is 4111, size is 31.
long lAxisNo = 0;
DWORD uAlarmCode = 4111;
long lAlarmStringSize = 31;
char *szAlarmString;
```

```
AxmStatusGetServoAlarmString(lAxisNo, uAlarmCode, lAlarmStringSize, &szAlarmString);
```

Visual Basic

```
' Return the Alarm String on axis 0, alarm code is 4111, size is 31.
Dim lAxisNo As Long
Dim uAlarmCode As Long
Dim lAlarmStringSize As Long
Dim szAlarmString As String

lAxisNo = 0
uAlarmCode = 4111
lAlarmStringSize = 31

AxmStatusGetServoAlarmString lAxisNo, uAlarmCode, lAlarmStringSize, szAlarmString
```

Delphi

```
{ Return the Alarm String on axis 0, alarm code is 4111, size is 31. }
var
  lAxisNo, uAlarmCode, lAlarmStringSize : Long;
  szAlarmString : String;

begin
  lAxisNo := 0;
  uAlarmCode := 4111;
  lAlarmStringSize := 31;
  AxmStatusGetServoAlarmString(lAxisNo, uAlarmCode, lAlarmStringSize, @szAlarmString);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusRequestServoAlarmHistory

Purpose

Send the request to read the Alarm History on a specific axis.

Format

C++

```
DWORD AxmStatusRequestServoAlarmHistory(long lAxisNo);
```

Visual Basic

```
Function AxmStatusRequestServoAlarmHistory(ByVal lAxisNo As Long) As
Long
```

Delphi

```
function AxmStatusRequestServoAlarmHistory(lAxisNo : LongInt) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
* See error code Table for more information on status error codes

Description

It sends the request to read the Alarm History on the specified axis of servo pack.

Example

C++

```
// Request the Alarm History on axis 0.
long lAxisNo = 0;
AxmStatusRequestServoAlarmHistory(lAxisNo);
```

Visual Basic

```
' Request the Alarm History on axis 0.
Dim lAxisNo As Long
lAxisNo = 0

AxmStatusRequestServoAlarmHistory lAxisNo
```

Delphi

```
{ Request the Alarm History on axis 0. }
var
  lAxisNo : Long;
```

```
begin
   lAxisNo := 0;
   AxmStatusRequestServoAlarmHistory(lAxisNo);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusReadServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusReadServoAlarmHistory

Purpose

Read the Alarm History on a specific axis.

Format

C++

```
DWORD AxmStatusReadServoAlarmHistory(long lAxisNo, DWORD uReturnMode,
long *lpCount, DWORD *upAlarmCode);
```

Visual Basic

```
Function AxmStatusReadServoAlarmHistory(ByVal lAxisNo As Long, ByVal
uReturnMode As Long, ByRef lpCount As Long, ByRef upAlarmCode As Long)
As Long
```

Delphi

```
function AxmStatusReadServoAlarmHistory(lAxisNo : LongInt;
uReturnMode : Dword; lpCount : PDword; upAlarmCode : PDword) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uReturnMode	Set the operating condition – [0] Return immediately after executing – [1] Does not return to read the alarm code from servo pack – [2] Return to read the alarm code from servo pack but not the program Blocking
[<<]lpCount	Read Alarm History number
[<<]upAlarmCode	Alarm Code of servo pack

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4210] AXT_RT_MOTION_READ_ALARM_WAITING : The result is waiting from the alarm code of servo pack
[4212] AXT_RT_MOTION_READ_ALARM_TIMEOUT : Reading the alarm of servo pack is timeout (1 sec or more)
[4211] AXT_RT_MOTION_READ_ALARM_NO_REQUEST : Servo pack returns the alarm code when the command has not been taken
[4213] AXT_RT_MOTION_READ_ALARM_FAILED : The alarm failed to read from servo pack
* See error code Table for more information on status error codes

Description

It reads the Alarm History on the specified axis of servo pack.

Example

C++

```
// Read the Alarm History on axis 0.
long lAxisNo = 0;
DWORD uReturnMode = 0;
long *lpCount;
DWORD *upAlarmCode;
AxmStatusReadServoAlarmHistory(lAxisNo, uReturnMode, &lpCount, &upAlarmCode);
```

Visual Basic

```
' Read the Alarm History on axis 0.
Dim lAxisNo As Long
Dim uReturnMode As Long
Dim lpCount As Long
Dim upAlarmCode As Long
lAxisNo = 0
uReturnMode = 0

AxmStatusReadServoAlarmHistory lAxisNo, uReturnMode, lpCount, upAlarmCode
```

Delphi

```
{ Read the Alarm History on axis 0. }
var
   lAxisNo, lpCount : Long;
   uReturnMode, upAlarmCode : Dword;

begin
   lAxisNo := 0;
   uReturnMode := 0;
   AxmStatusReadServoAlarmHistory(lAxisNo, uReturnMode, @lpCount, @upAlarmCode);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusClearServoAlarmHistory

AxmStatusClearServoAlarmHistory

Purpose

Clear the Alarm History on a specific axis.

Format

C++

```
DWORD AxmStatusClearServoAlarmHistory(long lAxisNo);
```

Visual Basic

```
Function AxmStatusClearServoAlarmHistory(ByVal lAxisNo As Long) As
Long
```

Delphi

```
function AxmStatusClearServoAlarmHistory(lAxisNo : LongInt) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4260] AXT_RT_PROTECTED_DURING_SERVOON : Can not be used the function or input value in servo-on status
[4261] AXT_RT_HW_ACCESS_ERROR : H/w of board access error has occurred
* See error code Table for more information on status error codes

Description

It clears the Alarm History on the specified axis of servo pack.

Example

C++

```
// Clear the Alarm History on axis 0.
long lAxisNo = 0;
AxmStatusClearServoAlarmHistory(lAxisNo);
```

Visual Basic

```
' Clear the Alarm History on axis 0.
Dim lAxisNo As Long
lAxisNo = 0

AxmStatusClearServoAlarmHistory lAxisNo
```

Delphi

```
{ Clear the Alarm History on axis 0. }
var
   lAxisNo : Long;

begin
   lAxisNo := 0;
   AxmStatusClearServoAlarmHistory(lAxisNo);
end;
```

See Also

AxmStatusReadInMotion, AxmStatusReadDrivePulseCount, AxmStatusReadMotion, AxmStatusReadStop, AxmStatusReadMechanical, AxmStatusReadVel, AxmStatusReadPosError, AxmStatusReadDriveDistance, AxmStatusSetActPos, AxmStatusGetActPos, AxmStatusSetCmdPos, AxmStatusRequestServoAlarm, AxmStatusReadServoAlarm, AxmStatusGetServoAlarmString, AxmStatusRequestServoAlarmHistory, AxmStatusReadServoAlarmHistory

Home Search API

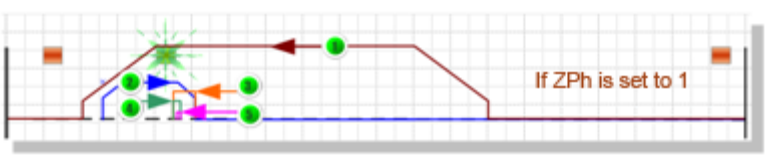
Function	Description
AxmHomeSetSignalLevel	Sets home sensor level of a specific axis.
AxmHomeGetSignalLevel	Returns home sensor level of a specific axis.
AxmHomeReadSignal	Returns state of home signal input of a specific axis.
AxmHomeSetMethod	To execute home search of the corresponding axis, the parameters related to home search must be preset. If initialization has been normally executed using AxmMotLoadParaAll API setting file, no further setup is required.. Home search method setting includes searching direction, the signal to be used as home, home sensor active level, and whether encoder Z phase is detected, etc. (Refer to the description part of AxmMotLoadParaAll for details)
AxmHomeGetMethod	To execute home search of the corresponding axis, the parameters related to home search must be preset. Returns the home-related parameters that were previously set.
AxmHomeSetVel	It detects home through multi-level steps to search it quickly and accurately, At this point, it sets the velocity to be used for each step. Time and accuracy of home search is determined based on the setting of these velocities. You can set home search velocities while adjusting velocities in each step accordingly. (Refer to the description part of AxmMotLoadParaAll for details)
AxmHomeGetVel	It detects home through multi-level steps to search it quickly and accurately, At this point, it sets the velocity to be used for each step. Time and accuracy of home search is determined based on the setting of these velocities. It gets each velocity previously set.
AxmHomeSetStart	If home search starting API is executed, the thread to execute home search of the corresponding axis is automatically created within the library. Then it sequentially executes home search and is exited.
AxmHomeSetResult	After home search is successfully executed using home search API, the search result is set to HOME SUCCESS. This API can set the result at user's discretion without executing home search. In practice, if an exceptional situation occurs during a normal move, the user can set the home search result to HOME_ERR_USER_BREAK or so to recover executing home search again.
AxmHomeGetResult	Verifies the search result of home search API. If home search starts, it is set to HOME_SEARCHING, and if it fails, the cause of failure is set. You can remove the cause of failure and restart home search.
AxmHomeGetRate	You can check the progress rate after home search starts. After home search is completed, it returns 100 regardless of whether or not it was successful. You can verify whether or not home search was successful using GetHome Result API.

For equipment move, home search needs to be executed. The important thing in the parameters related to home search is the velocity used for home search. dVelFirst is the initial velocity and dVelSecond is the velocity at which home sensor escapes to the opposite direction after its first detection. dVelThird is the

velocity used for second home search. dVelLast is the velocity used for the very last home search. If the value of dVelLast is too big, position errors may occur when home search is completed. On the other hand, if the value is too small, it takes too much time for home search. Therefore, setting a proper value is important.

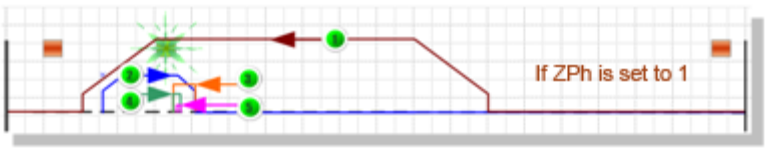
Home Search Variable	Explanation
HmSig	Sets a signal to use as home sensor (e.g.) Home Sensor(4), +Limit Sensor(0), etc.
HmLev	Sets the Active Level of Home Sensor (e.g.) 0: A contact, 1: B contact
HmDir	Sets the initial search direction in home sensor search (e.g.) 1: (+)direction, 0: (−)direction
Zphas	Sets whether to detect encoder zphase after home sensor search
VelFirst	Velocity of Initial high-speed detection in home search (when the initial home sensor is not detected)
VelSecond	Velocity to escape to the opposite direction after the first sensor detection in home search
VelThird	Velocity to execute search again after the first sensor detection in home search
VelLast	Sets the last detection velocity in home search [determine the accuracy of home search]
AccFirst	Initial high-speed detection acceleration in home search (The unit of acceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1)
AccSecond	Acceleration to escape to the opposite direction after the first sensor detection in home search (The unit of acceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1)
HClrTim	Standby time before Cmd, Act(Encoder) position are cleared after home search
HOffset	Moving offset value when resetting mechanical home after home search is completed
NSWLimit	Sets (−) Software Limit value to apply after home search is completed
PSWLimit	Sets (−) Software Limit value to apply after home search is completed (If NSWLimit is greater than or equal to PSWLimit, S/W Limit won't be set)

● *When Zph is 1*



By detecting Z phase Down Edge in Step 4 and detecting Z phase Up Edge again in Step 5, it enhances the velocity and accuracy of

Understanding home search velocity setting per each step



Use VelFirst, AccFirst

1 High-speed detection of home sensor in the direction of HmDir and deceleration stop at HAccF deceleration

Use VelSecond, AccSecond

2 Detection of Down edge of home sensor in the opposite direction of HmDir and deceleration stop at HAccF deceleration

Use VelThird

3 Detection of Up Edge of home sensor in the direction of HmDir and ememrgency stop

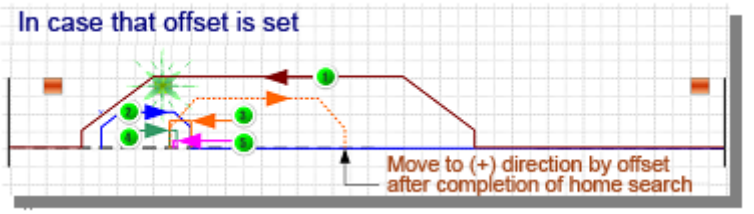
Use VelLast

4 Detection of Down Edge of Z phase in the opposite direction of HmDir and ememrgency stop

Use VelLast

5 Detection of Up Edge of Z phase in the direction of HmDir and ememrgency stop

In case that home needs to be set by moving to a specific position after mechanical home sensor detection is completed, if a value is assigned to HOffset, it automatically resets home after moving by a corresponding Offset. You should be careful not to exceed the range between (+) and (−) limits. Otherwise, home search cannot be executed normally.

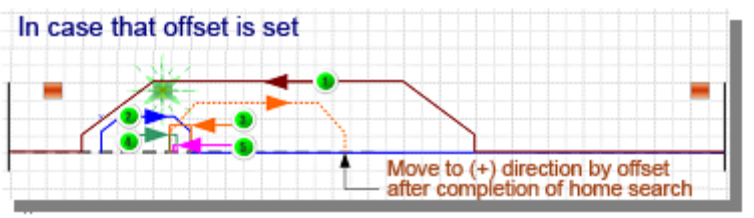


After Offset movement completing with automatic it is set with 0 pieces.

Soft limit can be set after home search is normally completed. Soft limit can be used when limit sensor is not mechanically equipped or when limit needs to be modified depending on situations. Remember that the value of PLimit(+) must be greater than that of NLimit(−) in order to set soft limit normally. If NSwLimit is greater than or equal to PSwLimit, Soft Limit will not be set.

Note: 1. During home search, software limit is enabled again when software limit is set in advance even if it is stopped by the user or outside.

2. During home search, software limit is disabled at the start when software limit is set in advance; when home search is done, enable software limit again.



AxmHomeSetSignalLevel

Purpose

Set output level of home input signal of a specific axis.

Format

C++

```
DWORD AxmHomeSetSignalLevel(long lAxisNo, DWORD uLevel);
```

Visual Basic

```
Function AxmHomeSetSignalLevel(ByVal lAxisNo As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxmHomeSetSignalLevel(lAxisNo : LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uLevel	[01h] Active Level of Home signal : AXT_MOTION_LEVEL_MODE_DEF – [00h] B contact (Normal Close) – [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Caution : Even if the level is incorrectly set in the – direction, it can still move in the + direction; It may cause a problem during home search.

It sets home input level of the axis specified by user. The chips in SMC-2V03, PCI-N804/404 use universal input 0 as home signal of encoder. In order to read home level, you can use AxmHomeReadSignal API.

Example

C++

```
// Set home input signal level on axis 0.
AxmHomeSetSignalLevel(0, HIGH);

// Verify home input signal level on axis 0.
DWORD state;
AxmHomeGetSignalLevel(0, &state);
```

Visual Basic

```
' Set home input signal level on axis 0.
AxmHomeSetSignalLevel 0, HIGH

' Verify home input signal level on axis 0.
Dim state As Long
AxmHomeGetSignalLevel 0, state
```

Delphi

```
{ Verify home input signal level on axis 0. }
var
   state: DWord;

begin
   { Set home input signal level on axis 0. }
   AxmHomeSetSignalLevel(0, HIGH);
   AxmHomeGetSignalLevel(0, @state);
end;
```

See Also

AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeGetSignalLevel

Purpose

Returns output level settings of home signal of a specific axis.

Format

C++

```
DWORD AxmHomeGetSignalLevel(long lAxisNo, DWORD *upLevel);
```

Visual Basic

```
Function AxmHomeGetSignalLevel(ByVal lAxisNo As Long, ByRef upLevel
As Long) As Long
```

Delphi

```
function AxmHomeGetSignalLevel(lAxisNo : LongInt; upLevel : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upLevel	[01h] Active Level of Home signal : AXT_MOTION_LEVEL_MODE_DEF – [00h] B contact (Normal Close) – [01h] A contact (Normal Open)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the home input level set by 'AxmHomeSetSignalLevel'.

Example

C++

```
// Set home input signal level on axis.
AxmHomeSetSignalLevel(0, HIGH);

// Verify home input signal level on axis 0.
DWORD state;
AxmHomeGetSignalLevel(0, &state);
```

Visual Basic

```
' Set home input signal level on axis.
AxmHomeSetSignalLevel 0, HIGH
```

```
' Verify home input signal level on axis 0.
Dim state As Long
AxmHomeGetSignalLevel 0, state
```

Delphi

```
{ Verify home input signal level on axis 0. }
var
  state: DWord;

begin
  { Set home input signal level on axis. }
  AxmHomeSetSignalLevel(0, HIGH);
  AxmHomeGetSignalLevel(0, @state);
end;
```

See Also

AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeReadSignal

Purpose

Return the input state of home signal of a specific axis.

Format

C++

```
DWORD AxmHomeReadSignal(long lAxisNo, DWORD *upStatus);
```

Visual Basic

```
Function AxmHomeReadSignal(ByVal lAxisNo As Long, ByRef upStatus As
Long) As Long
```

Delphi

```
function AxmHomeReadSignal(lAxisNo : LongInt; upStatus : PDword) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upStatus	Input state of a signal : AXT_MOTION_SIGNAL_LEVEL_DEF - [00h] No-activation - [01h] Activation

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the input state of the current home signal.

Example

C++

```
// Verify home signal input on axis 0.
DWORD upStatus;
AxmHomeReadSignal (0, &upStatus);
```

Visual Basic

```
' Verify home signal input on axis 0.
Dim upStatus As Long
AxmSignalReadInpos 0, upStatus
```

Delphi

```
{ Verify home signal input on axis 0. }
var
  upStatus: DWord;

begin
  AxmHomeReadSignal (0, @upStatus);
end;
```

See Also

AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeSetMethod

Purpose

To execute home search of the corresponding axis, the parameters related to home search must be preset. If initialization has been normally executed using AxmMotLoadParaAll API setting file, no further setup is required.

Home search method setting includes searching direction, the signal to be used as home, home sensor active level, and whether encoder Z phase is detected, etc. (Refer to the description part of AxmMotLoadParaAll for details)

Format

C++

```
DWORD AxmHomeSetMethod(long lAxisNo, long lHmDir, DWORD uHomeSignal,
DWORD uZphas, double dHomeClrTime, double dHomeOffset);
```

Visual Basic

```
Function AxmHomeSetMethod(ByVal lAxisNo As Long, ByVal nHmDir As Long,
ByVal uHomeSignal As Long, ByVal uZphas As Long, ByVal dHomeClrTime
As Double, ByVal dHomeOffset As Double) As Long
```

Delphi

```
function AxmHomeSetMethod(lAxisNo : LongInt; nHmDir : LongInt;
uHomeSignal : DWord; uZphas : DWord; dHomeClrTime : Double;
dHomeOffset : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lHmDir	[00h] Sets the direction to execute home search at initial stage : AXT_MOTION_MOVE_DIR_DEF – [00h] Counterclockwise – [01h] Clockwise
[>>]uHomeSignal	[04h] Sets the signal to be used for home search : AXT_MOTION_HOME_DETECT_SIGNAL_DEF – [00h] +Elm(End limit) + direction limit sensor signal – [01h] –Elm(End limit) – direction limit sensor signal – [04h] IN0(ORG) home sensor signal
[>>]uZphas	[00h] Sets whether encoder Z phasie is detected after the first home search is completed – [00h] Z phase detection non-used – [01h] Z phase detection used positive direction – [02h] Z phase detection used negative direction
[>>]dHomeClrTime	[1000.0] Sets the standby time to clear the command position and the encoder position after home search is completed [per mSec]
[>>]dHomeOffset	[0.0] Position at which home will be reset after home search is completed and is moved to mechanical home

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion
* See error code Table for more information on status error codes

Description

To execute home search of the corresponding axis, the parameters related to home search must be preset. If initialization has been normally executed using AxmMotLoadParaAll API setting file, no further setup is required. Home search method setting includes searching direction, the signal to be used as home, and whether encoder Z phase is detected, etc. Home sensor active level is independently set using AxmSignalSetHomeLevel API. (Refer to the description part of AxmMotLoadParaAll for details)

Even if the level is incorrectly set in the – direction, it can still move in the + direction; It may cause a problem during home search.

Example

C++

```
long   lHmDir = 1;      DWORD  lHmsig = 4;
DWORD  dwZphas = 0;     double dwHClrTim = 2000.0, dwHOffset = 0.0;
AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset);
AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
AxmHomeSetStart(0);
// Set home Search in the (-) direction, home sensor used, home sensor
// type A contact, Z phase detection non-used, offset non-moved.
AxmHomeGetMethod(0, &lHmDir, &lHmsig, &dwZphas, &dwHClrTim, & dwHOffset);
```

Visual Basic

```
Dim dwHOffset As Double   Dim dwHClrTim As Double
Dim lHmDir, lHmsig, dwZphas As Long
dwHClrTim = 2000#   dwHOffset = 0#
lHmDir = -1         lHmsig = 4          dwZphas = 0
AxmHomeSetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
AxmHomeSetVel 0, 10000, 5000, 500, 50, 40000, 10000
AxmHomeSetStart 0
AxmHomeGetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
```

Delphi

```
var
   dwZphas, lHmsig : DWORD;
  dwHOffset, dwHClrTim : Double;
   lHmDir : LongInt;
   velocity : Double;

begin
   lHmDir := -1;        lHmsig := 4;
   dwZphas := 0;        dwHClrTim := 2000.0;
```

```
   dwHOffset := 0.0;
   AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim,dwHOffset);
   AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
   AxmHomeSetStart(0);
   { Set home Search in the (-) direction, home sensor used, home sensor type A contact,
Z phase detection non-used, offset non-moved }
   AxmHomeGetMethod(0, @lHmDir, @lHmsig, @dwZphas, @dwHClrTim, @dwHOffset);
End;
```

See Also

AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeGetMethod

Purpose

To execute home search of the corresponding axis, the parameters related to home search must be preset. If initialization has been normally executed using AxmMotLoadParaAll API setting file, no further setup is required. It reads searching direction, the signal to be used as home, and whether encoder Z phase is detected, etc. in home search method setting. (Refer to the description part of AxmMotLoadParaAll for details)

Format

C++

```
DWORD AxmHomeGetMethod(long lAxisNo, long *lpHmDir, DWORD
*upHomeSignal, DWORD *upZphas, double *dpHomeClrTime, double
*dpHomeOffset);
```

Visual Basic

```
Function AxmHomeGetMethod(ByVal lAxisNo As Long, ByRef lpHmDir As
Long, ByRef upHomeSignal As Long, ByRef upZphas As Long, ByRef
dpHomeClrTime As Double, ByRef dpHomeOffset As Double) As Long
```

Delphi

```
function AxmHomeGetMethod(lAxisNo : LongInt; lpHmDir : PLongInt;
upHomeSignal : PDWord; upZphas : PDWord; dpHomeClrTime : PDouble;
dpHomeOffset : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]lpHmDir	[00h] Sets the direction to execute home search at initial stage : AXT_MOTION_MOVE_DIR_DEF – [00h] Counterclockwise – [01h] Clockwise
[<<]upHomeSignal	[01h] Sets the signal to be used for home search : AXT_MOTION_HOME_DETECT_SIGNAL_DEF – [00h] +Elm(End limit) + direction limit sensor signal – [01h] –Elm(End limit) – direction limit sensor signal – [02h] IN0(ORG) home sensor signal
[<<]upZphas	[00h] Sets whether encoder Z phasie is detected after the first home search is completed – [00h] Z phase detection non–used – [01h] Z phase detection used positive direction – [02h] Z phase detection used negative direction
[<<]dpHomeClrTime	[1000.0] Sets the standby time to clear the command position and the encoder position after home search is completed [per mSec]
[<<]dpHomeOffset	[0.0] Position at which home will be reset after home search is completed and is moved to mechanical home

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

To execute home search of the corresponding axis, the parameters related to home search must be preset. If initialization has been normally executed using AxmMotLoadParaAll API setting file, no further setup is required. Home search method setting includes searching direction, the signal to be used as home, and whether encoder Z phase is detected, etc. Home sensor active level is independently set using AxmSignalGetHomeLevel API. (Refer to the description part of AxmMotLoadParaAll for details)

Example

C++

```
long   lHmDir = 1;
DWORD  lHmsig =  4;
DWORD  dwZphas = 0;
double dwHClrTim = 2000.0, dwHOffset = 0.0;

AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset);
AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
AxmHomeSetStart(0);

// Set home search in the (-) direction, home sensor used, home sensor type
// A contact, Z phase detection non-used, offset non-moved.
AxmHomeGetMethod(0, &lHmDir, &lHmsig, &dwZphas, &dwHClrTim, & dwHOffset);
```

Visual Basic

```
Dim dwHOffset As Double
Dim dwHClrTim As Double
Dim lHmDir, lHmsig, dwZphas As Long
dwHClrTim = 2000#   dwHOffset = 0#
lHmDir = -1         lHmsig = 4
dwZphas = 0
' Set home search in the (-) direction, home sensor used, home sensor type
' A contact, Z phase detection non-used, offset non-moved.
AxmHomeSetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
AxmHomeSetVel 0, 10000, 5000, 500, 50, 40000, 10000
AxmHomeSetStart 0
AxmHomeGetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
```

Delphi

```
var
  dwZphas, lHmsig : DWORD;
  dwHOffset, dwHClrTim : Double;
  lHmDir : LongInt;
  velocity : Double;
```

```
begin
   lHmDir := -1;
   lHmsig := 4;
   dwZphas := 0;
   dwHClrTim := 2000.0;
   dwHOffset := 0.0;
   AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim,dwHOffset);
   AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
   AxmHomeSetStart(0);
   { Set home search in the (-) direction, home sensor used, home sensor type A contact,
Z phase detection non-used, offset non-moved. }
   AxmHomeGetMethod (0, @lHmDir, @lHmsig, @dwZphas, @dwHClrTim, @dwHOffset);
End;
```

See Also

AxmHomeSetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeSetVel

Purpose

It detects home through multi-level steps to search it quickly and accurately, At this point, it sets the velocity to be used for each step. Time and accuracy of home search is determined based on the setting of these velocities. You can set the home search velocities while adjusting velocities in each step accordingly. (Refer to the description part of AxmMotLoadParaAll for details)

Format

C++

```
DWORD AxmHomeSetVel(long lAxisNo, double dVelFirst, double dVelSecond,
double dVelThird, double dvelLast, double dAccFirst, double
dAccSecond);
```

Visual Basic

```
Function AxmHomeSetVel(ByVal lAxisNo As Long, ByVal dVelFirst As
Double, ByVal dVelSecond As Double, ByVal dVelThird As Double, ByVal
dvelLast As Double, ByVal dAccFirst As Double, ByVal dAccSecond As
Double) As Long
```

Delphi

```
function AxmHomeSetVel(lAxisNo : LongInt; dVelFirst : Double;
dVelSecond : Double; dVelThird : Double; dvelLast : Double;
dAccFirst : Double; dAccSecond : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dVelFirst	[100.0] Velocity at which to move in the direction of home search when home sensor is not detected
[>>]dVelSecond	[100.0] Velocity at which to move in the opposite direction of home search when home sensor is detected
[>>]dVelThird	[20.0] Moving velocity to be used for the second search after the first home sensor detection
[>>]dvelLast	[1.0] Moving velocity to be used for the last last home search (Velocity at which to use Z phase detection, detailed searching velocity of home sensor)
[>>]dAccFirst	[400.0] Ac/deceleration to be used when home sensor is not detected and it is moving in the direction of home search (The unit of deceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1)
[>>]dAccSecond	[400.0] Ac/deceleration to be used when home sensor is detected and it is moving in the opposite direction of home search (The unit of deceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully

[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion
[4253] AXT_RT_MOTION_VELOCITY_OUTOFBOUND : Configured velocity is either above the maximum value or below the minimum value
* See error code Table for more information on status error codes

Description

It detects home through multi-level steps to search it quickly and accurately, At this point, it sets the velocity to be used for each step. Time and accuracy of home search is determined based on the setting of these velocities. You can set the home search velocity while adjusting it in each step accordingly. (Refer to the description part of AxmMotLoadParaAll for details)

Example

C++

```
long   lHmDir = 1;
DWORD  lHmsig = 4;
DWORD  dwZphas = 0;
double dwHClrTim = 2000.0, dwHOffset = 0.0;
AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset);
AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
AxmHomeSetStart(0);
// Set home search in the (-) direction, home sensor used, home sensor
// type A contact, Z phase detection non-used, offset non-moved.
AxmHomeGetMethod (0, &lHmDir, &lHmsig, &dwZphas, &dwHClrTim, & dwHOffset);
```

Visual Basic

```
Dim dwHOffset As Double
Dim dwHClrTim As Double
Dim lHmDir, lHmsig, dwZphas As Long
dwHClrTim = 2000#
dwHOffset = 0#
lHmDir = -1
lHmsig = 4
dwZphas = 0
' Set home search in the (-) direction, home sensor used, home sensor
' type A contact, Z phase detection non-used, offset non-moved.
AxmHomeSetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
AxmHomeSetVel 0, 10000, 5000, 500, 50, 40000, 10000
AxmHomeSetStart 0
AxmHomeGetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
```

Delphi

```
var
  dwZphas, lHmsig : DWORD;
  dwHOffset, dwHClrTim : Double;
  lHmDir : LongInt;
  velocity : Double;
```

```
begin
   lHmDir := -1;
   lHmsig := 4;
   dwZphas := 0;
   dwHClrTim := 2000.0;
   dwHOffset := 0.0;
   AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim,dwHOffset);
   AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
   AxmHomeSetStart(0);
   { Set home search in the (-) direction, home sensor used, home sensor type A contact,
Z phase detection non-used, offset non-moved. }
   AxmHomeGetMethod(0, @lHmDir, @lHmsig, @dwZphas, @dwHClrTim, @dwHOffset);
End;
```

See Also

AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeGetVel

Purpose

It detects home through multi-level steps to search it quickly and accurately, At this point, it returns the velocity to be used for each step.

Format

C++

```
DWORD AxmHomeGetVel(long lAxisNo, double *dpVelFirst, double
*dpVelSecond, double *dpVelThird, double *dpVelLast, double
*dpAccFirst, double *dpAccSecond);
```

Visual Basic

```
Function AxmHomeGetVel(ByVal lAxisNo As Long, ByRef dpVelFirst As
Double, ByRef dpVelSecond As Double, ByRef dpVelThird As Double,
ByRef dpVelLast As Double, ByRef dpAccFirst As Double, ByRef
dpAccSecond As Double) As Long
```

Delphi

```
function AxmHomeGetVel(lAxisNo : LongInt; dpVelFirst : PDouble;
dpVelSecond : PDouble; dpVelThird : PDouble; dpVelLast : PDouble;
dpAccFirst : PDouble; dpAccSecond : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpVelFirst	[100.0] Velocity at which to move in the direction of home search when home sensor is not detected
[<<]dpVelSecond	[100.0] Velocity at which to move in the opposite direction of home search when home sensor is detected
[<<]dpVelThird	[20.0] Moving velocity to be used for the second search after the first home sensor detection
[<<]dpVelLast	[1.0] Moving velocity to be used for the last home search (Velocity at which to use Z phase detection, detailed searching velocity of home sensor)
[<<]dpAccFirst	[400.0] Ac/deceleration to be used when home sensor is not detected and it is moving in the direction of home search (The unit of deceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1)
[<<]dpAccSecond	[400.0] Ac/deceleration to be used when home sensor is detected and it is moving in the opposite direction of home search (The unit of deceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist

* See error code Table for more information on status error codes

Description

Tip : It is possible to get only necessary data by inserting NULL into unnecessary variables.

It detects home through multi-level steps to search it quickly and accurately, At this point, it sets the velocity to be used for each step. Time and accuracy of home search is determined based on the setting of these velocities. You can set the home search velocity while adjusting it in each step accordingly. (Refer to the description part of AxmMotLoadParaAll for details)

Example

C++

```
long   lHmDir = 1;
DWORD  lHmsig = 4;
DWORD  dwZphas = 0;
double dwHClrTim = 2000.0, dwHOffset = 0.0;
AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset);
AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
AxmHomeSetStart(0);

// Set to home search in the (-) direction, home sensor used, home sensor
// type A contact, Z phase detection non-used, offset non-moved.
AxmHomeGetMethod (0, &lHmDir, &lHmsig, &dwZphas, &dwHClrTim, & dwHOffset);
AxmHomeGetVel(0, & dVelFirst, & dVelSecond, & dVelThird, & dVelLast,
   &dAccFirst, & dAccSecond);
```

Visual Basic

```
Dim dpSlOffset As Double
Dim dSlORange As Double
Dim bSlHomeUse, bGatryOn, bMaSlChange As Long
Dim dVelFirst, dVelSecond, dVelThird,dVelLast, dAccFirst,dAccSecond As Double
Dim dwHOffset As Double
Dim dwHClrTim As Double
Dim lHmDir, lHmsig, dwZphas As Long
dwHClrTim = 2000#      dwHOffset = 0#       lHmDir = -1
lHmsig = 4             dwZphas = 0
' Set to home search in the (-) direction, home sensor used, home sensor
' type A contact, Z phase detection non-used, offset non-moved.
AxmHomeSetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
AxmHomeSetVel 0, 10000, 5000, 500, 50, 40000, 10000
AxmHomeSetStart 0
AxmHomeGetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
AxmHomeGetVel 0, dVelFirst, dVelSecond, dVelThird, dVelLast, dAccFirst, dAccSecond
```

Delphi

```
var
   dwZphas, bSlHomeUse, bGatryOn, bMaSlChange : DWORD;
   dwHOffset, dwHClrTim, dpSlOffset,dSlORange : Double;
   dVelFirst, dVelSecond, dVelThird,dVelLast, dAccFirst,dAccSecond : Double;
```

```
  lHmDir, lHmsig : LongInt;
  velocity : Double;

begin
  lHmDir := -1;
  lHmsig := 4;
  dwZphas := 0;
  dwHClrTim := 2000.0;
  dwHOffset := 0.0;
  AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim,dwHOffset);
  AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
  AxmHomeSetStart(0);
  { Set to home search in the (-) direction, home sensor used, home sensor type A
contact, Z phase detection non-used, offset non-moved. }
  AxmHomeGetMethod(0, @lHmDir, @lHmsig, @dwZphas, @dwHClrTim, @dwHOffset);
  AxmHomeGetVel (0, @dVelFirst, @dVelSecond, @dVelThird, @dVelLast,
     @dAccFirst, @dAccSecond);
End;
```

See Also

AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeSetStart

Purpose

If home search starting API is executed, the thread to execute home search of the corresponding axis is automatically created within the library. Then it sequentially executes home search and is exited.

Format

C++

```
DWORD AxmHomeSetStart(long lAxisNo);
```

Visual Basic

```
Function AxmHomeSetStart(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmHomeSetStart(lAxisNo : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0) (0 ~ n-1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion

* See error code Table for more information on status error codes

Description

If home search starting API is executed, the thread to execute home search of the corresponding axis is automatically created within the library. Then it sequentially executes home search and is exited.

Note: Even if limit sensor of opposite direction against process direction is on, it does not work without the activation of sensor of process direction. If home search is started and limit sensor of process direction is on, it moves to the next step as it thought limit sensor is detected.

Example

C++

```
DWORD uHomeResult;
if(AxmHomeSetStart (0) == AXT_RT_SUCCESS)  // Execute home search
  printf("Axis 0 can start the Home Search.");
else{
  AxmHomeGetResult (0, & uHomeResult);
  printf("Axis 0 can not start the Home Search. [Error Code:%d]", uHomeResult);
}
```

Visual Basic

```
Dim nCount As Integer
Dim strData As String
Dim uHomeResult As Long
If (AxmHomeSetStart (0) = AXT_RT_SUCCESS )Then    ' Execute home search
  MsgBox "Axis 0 can start the Home Search..", vbOKCancel
Else
  AxmHomeGetResult 0, uHomeResult
  nCount   = uHomeResult
  strData  = "Axis 0 can not start the Home Search. Error Code" + CStr(nCount) + "."
     MsgBox strData
End If
```

Delphi

```
var
  nCount : Word; strData : String;
  uHomeResult: Dword

begin
  if (AxmHomeSetStart (0) = AXT_RT_SUCCESS ) then    { Execute home search }
     Application.MessageBox('Axis 0 can start the Home Search..', 'Ajinextek', MB_OK)
  else
  begin
     AxmHomeGetResult(0, @ uHomeResult);
     nCount      := uHomeResult;
     strData     := 'Axis 0 can not start the Home Search. Error Code' +
        IntToStr(nCount) + '다.';
     Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
  end;
end;
```

See Also

AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmHomeSetResult

Purpose

It outputs the search result after home search is successfully executed using home search API.

After home search is successfully executed using home search API, the search result is set to HOME SUCCESS. This API can set the result at user's discretion without executing home search.

In practice, if an unexpected situation occurs during a normal move, the user can set the home search result to HOME_ERR_USER_BREAK or so to recover executing home research.

Format

C++

```
DWORD AxmHomeSetResult(long lAxisNo, DWORD uHomeResult);
```

Visual Basic

```
Function AxmHomeSetResult(ByVal lAxisNo As Long, ByVal uHomeResult As Long) As Long
```

Delphi

```
function AxmHomeSetResult(lAxisNo : LongInt; uHomeResult : Dword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number (start from 0) to set home search result at user's discretion
[>>]uHomeResult	Home search result : AXT_MOTION_HOME_RESULT_DEF – [01h] Home search has been successfully exited – [02h] Home search is currently in progress – [10h] Home search results of Master axis and Slave axis of gantry move axes are out of OffsetRange – [11h] User executed stop command during home search – [12h] At least one value among 4 home searching velocity setting values is less than or equal to 0 – [13h] Servo pack alarm has occurred during home search – [14h] (–) limit sensor has been detected during the home sensor search in (+) direction – [15h] (+) limit sensor has been detected during the home sensor search in (–) direction – [16h] Home sensor has not been detected – [FFh] Home search has been attempted using unknown channel(axis) number(start from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion
* See error code Table for more information on status error codes

Description

After home search is successfully executed using home search API, the search result is set to HOME SUCCESS. This API can set the result at user's discretion without executing home search.

In practice, if an unexpected situation occurs during a normal move, the user can set the home search result to HOME_ERR_USER_BREAK or so to recover executing home research.

Example

C++

```
if(AxmHomeSetResult (0, HOME_ERR_USER_BREAK) == AXT_RT_SUCCESS){
  printf("Home search result on axis 0 has been set to USER BREAK.");
}else{
  printf("Home search result setting on axis 0 has failed.");
}

BOOL bRun = TRUE;
DWORD uHomeResult;
AxmHomeSetStart (0);
while(bRun)
{
  // Output the search result after home search is successfully executed
  // using home search API.
  AxmHomeGetResult(0, &uHomeResult);
  switch(uHomeResult)
  {
     case HOME_ERR_UNKNOWN:
        printf("Unknown axis number");
        bRun=FALSE;
     break;
     case HOME_ERR_GNT_RANGE:
        printf("Out of Gantry Offset");
        bRun=FALSE;
     break;
     case HOME_ERR_USER_BREAK:
        printf("Home search stops");
        bRun=FALSE;
     break;
     case HOME_ERR_VELOCITY:
        printf("Velocity has not been set.");
        bRun=FALSE;
     break;
     case HOME_ERR_AMP_FAULT:
        printf("Servo pack alarm has occurred.");
        bRun = FALSE;
     break;
     case HOME_ERR_NEG_LIMIT:
        printf("(-) limit sensor has been detected");
        bRun = FALSE;
     break;
```

```
      case HOME_ERR_POS_LIMIT:
         printf("(+) limit sensor has been detected");
         bRun = FALSE;
      break;
      case HOME_SEARCHING:
         printf("Home search is now in progress");
         Sleep(100);
      break;
      case HOME_SUCCESS:
         printf("Home search was successful");
         bRun = FALSE;
      break;
   }
}
```

Visual Basic

```
If (AxmHomeSetResult (0, HOME_ERR_USER_BREAK) = AXT_RT_SUCCESS )Then
  MsgBox "Home search result on axis 0 has been set to USER BREAK.", vbOKCancel
Else
  MsgBox "Home search result setting on axis 0 has failed.", vbOKCancel
End If

Dim bFlag As Byte
Dim  uID As Long
Dim uHomeResult As Long
AxmHomeSetStart 0
' Output the search result after home search is successfully executed
' using home search API.
bFlag = True
Do While bFlag
AxmHomeGetResult 0, uHomeResult
uID      = uHomeResult
Select Case uID
  Case HOME_ERR_UNKNOWN:
     MsgBox "Unknown axis number", vbOKCancel
     bFlag = False
  Case HOME_ERR_GNT_RANGE:
     MsgBox "Out of Gantry Offset", vbOKCancel
     bFlag = False
  Case HOME_ERR_USER_BREAK:
     MsgBox "Home search stops", vbOKCancel
     bFlag = False
  Case HOME_ERR_VELOCITY:
     MsgBox "Velocity has not been set.", vbOKCancel
     bFlag = False
  Case HOME_ERR_AMP_FAULT:
     MsgBox "Servo pack alarm has occurred", vbOKCancel
     bFlag = False
  Case HOME_ERR_NEG_LIMIT:
```

```
      MsgBox "(-) limit sensor has been detected", vbOKCancel
      bFlag  = False
   Case HOME_ERR_POS_LIMIT:
      MsgBox "(+) limit sensor has been detected", vbOKCancel
      bFlag  = False
   Case HOME_SEARCHING:
      MsgBox "Home search is now in progress", vbOKCancel
      bFlag  = False
   Case HOME_SUCCESS:
      MsgBox "Home search was successful", vbOKCancel
      bFlag  = False
End Select
Loop
```

Delphi

```
begin
if (AxmHomeSetResult (0, HOME_ERR_USER_BREAK) = AXT_RT_SUCCESS) then
   Application.MessageBox('Home search result on axis 0 has been set to USER BREAK.',
      'Ajinextek', MB_OK)
else
   Application.MessageBox('Home search result setting on axis 0 has failed.',
      'Ajinextek', MB_OK);
End;
var
   uID : Word; bFlag : Boolean; bFlag := True;
   uHomeResult: Dword

begin
AxmHomeSetStart(0);
{ Output the search result after home search is successfully executed using home search
API.}
while bFlag do
   begin
      AxmHomeGetResult(0, @uHomeResult);
      uID   := uHomeResult;
      case (uID) of
         HOME_ERR_UNKNOWN:
            begin
               Application.MessageBox ('Unknown axis number.', 'Ajinextek', MB_OK);
               bFlag := False;
            end;
         HOME_ERR_GNT_RANGE:
            begin
               Application.MessageBox ('Out of Gantry Offset', 'Ajinextek', MB_OK);
            end;
         HOME_ERR_USER_BREAK:
            begin
               Application.MessageBox ('Home search stops.', 'Ajinextek', MB_OK);
               bFlag := False;
```

```
            end;
         HOME_ERR_VELOCITY:
            begin
               Application.MessageBox ('Velocity has not been set.', 'Ajinextek',
                  MB_OK);
               bFlag := False;
            end;
         HOME_ERR_AMP_FAULT:
            begin
               Application.MessageBox ('Servo pack alarm has occurred.', 'Ajinextek',
                  MB_OK);
               bFlag := False;
            end;
         HOME_ERR_NEG_LIMIT:
            begin
               Application.MessageBox ('(-) limit sensor has been detected.',
                  'Ajinextek', MB_OK);
               bFlag := False;
            end;
         HOME_ERR_POS_LIMIT:
            begin
               Application.MessageBox ('(+) limit sensor has been detected',
                  'Ajinextek', MB_OK);
               bFlag := False;
            end;
         HOME_SEARCHING:
            begin
               Application.MessageBox ('Home search is now in progress', 'Ajinextek',
                  MB_OK);
               bFlag := False;
            end;
         HOME_ SUCCESS:
            begin
               Application.MessageBox ('Home search was successful', 'Ajinextek',
                  MB_OK);
               bFlag := False;
            end;
      end;
   end;
end;
```

See Also

AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeGetResult, AxmHomeGetRate

AxmHomeGetResult

Purpose

It verifies the search result of home search API. If home search starts, it is set to HOME_SEARCHING, and if it fails, the cause of failure is set. You can eliminate the cause of failure and restart home search.

Format

C++

```
DWORD AxmHomeGetResult(long lAxisNo, DWORD *upHomeResult);
```

Visual Basic

```
Function AxmHomeGetResult(ByVal lAxisNo As Long, ByRef upHomeResult
As Long) As Long
```

Delphi

```
function AxmHomeGetResult(lAxisNo : LongInt; upHomeResult : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number (start from 0) to set home search result at user's discretion
[<<]upHomeResult	Home search result : AXT_MOTION_HOME_RESULT_DEF – [01h] Home search has been successfully exited – [02h] Home search is currently in progress – [10h] Home search results of Master axis and Slave axis of gantry move axes are out of OffsetRange – [11h] User executed stop command during home search – [12h] At least one value among 4 home searching velocity setting values is less than or equal to 0 – [13h] Servo pack alarm has occurred during home search – [14h] (–) limit sensor has been detected during the home sensor search in (+) direction – [15h] (+) limit sensor has been detected during the home sensor search in (–) direction – [16h] Home sensor has not been detected – [FFh] Home search has been attempted using unknown channel(axis) number(start from 0)

※ Note: After home search has succeeded, if the home search is executed again, its result is initialized and reset. If it is stopped while executing once again using stop command(AxmMoveSStop) API, it is set to HOME_USER_BREAK.

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It verifies the search result of home search API. If home search starts, it is set to HOME_SEARCHING, and if it fails, the cause of failure is set. You can eliminate the cause of failure and restart home search.

Example

C++

```
if(AxmHomeSetResult (0, HOME_ERR_USER_BREAK) == AXT_RT_SUCCESS){
  printf("Home search result on axis 0 has been set to USER BREAK.");
}else{
  printf("Home search result setting on axis 0 has failed.");
}

BOOL bRun = TRUE;
DWORD uHomeResult;
AxmHomeSetStart (0);

while(bRun)
{
  // Verify home search result.
  AxmHomeGetResult(0, &uHomeResult);
  switch(uHomeResult)
  {
  case HOME_ERR_UNKNOWN:
     printf("Unknown axis number");
     bRun = FALSE;
  break;
  case HOME_ERR_GNT_RANGE:
     printf("Out of Gantry Offset");
     bRun = FALSE;
  break;
  case HOME_ERR_USER_BREAK:
     printf("Home search stops");
     bRun = FALSE;
  break;
  case HOME_ERR_VELOCITY:
     printf("Velocity has not been set.");
     bRun = FALSE;
  break;
  case HOME_ERR_AMP_FAULT:
     printf("Servo pack alarm has occured");
     bRun = FALSE;
  break;
  case HOME_ERR_NEG_LIMIT:
     printf("(-) limit sensor has been detected");
     bRun = FALSE;
  break;
  case HOME_ERR_POS_LIMIT:
     printf("(+) limit sensor has been detected");
```

```
      bRun = FALSE;
   break;
   case HOME_SEARCHING:
      printf("Home search is now in progress");
      Sleep(100);
   break;
   case HOME_SUCCESS:
      printf("Home search was successful");
      bRun = FALSE;
   break;
}
```

Visual Basic

```
If (AxmHomeSetResult (0, HOME_ERR_USER_BREAK) = AXT_RT_SUCCESS )Then
   MsgBox "Home search result on axis 0 has been set to USER BREAK.", vbOKCancel
Else
   MsgBox "Home search result setting on axis 0 has failed.", vbOKCancel
End If

Dim bFlag As Byte
Dim  uID As Long
Dim uHomeResult As Long
AxmHomeSetStart 0

bFlag = True
Do While bFlag
' Verify home search result.
AxmHomeGetResult 0, uHomeResult
uID      = uHomeResult
Select Case uID
Case HOME_ERR_UNKNOWN:
   MsgBox "Unknown axis number", vbOKCancel
   bFlag = False
Case HOME_ERR_GNT_RANGE:
   MsgBox "Out of Gantry Offset", vbOKCancel
   bFlag = False
Case HOME_ERR_USER_BREAK:
   MsgBox "Home search stops", vbOKCancel
   bFlag = False
Case HOME_ERR_VELOCITY:
   MsgBox "Velocity has not been set.", vbOKCancel
   bFlag = False
Case HOME_ERR_AMP_FAULT:
   MsgBox "Servo pack alarm has occured", vbOKCancel
   bFlag = False
Case HOME_ERR_NEG_LIMIT:
   MsgBox "(-) limit sensor has been detected", vbOKCancel
   bFlag = False
Case HOME_ERR_POS_LIMIT:
```

```
  MsgBox "(+) limit sensor has been detected", vbOKCancel
  bFlag = False
Case HOME_SEARCHING:
  MsgBox "Home search is now in progress", vbOKCancel
  bFlag = False
Case HOME_SUCCESS:
  MsgBox "Home search was successful", vbOKCancel
  bFlag = False
End Select
Loop
```

Delphi

```
begin
if (AxmHomeSetResult (0, HOME_ERR_USER_BREAK) = AXT_RT_SUCCESS) then
  Application.MessageBox('Home search result on axis 0 has been set to USER BREAK.',
     'Ajinextek', MB_OK)
else
  Application.MessageBox('Home search result setting on axis 0 has failed.',
     'Ajinextek', MB_OK);
End;

var
  uID : Word; bFlag : Boolean; bFlag := True;
  uHomeResult: Dword

begin
AxmHomeSetStart (0);

while bFlag do
begin
  { Verify home search result. }
  AxmHomeGetResult(0, @uHomeResult);
  uID   := uHomeResult;
  case (uID) of
     HOME_ERR_UNKNOWN:
        begin
           Application.MessageBox ('Unknown axis number.', 'Ajinextek', MB_OK);
           bFlag := False;
        end;
     HOME_ERR_GNT_RANGE:
        begin
           Application.MessageBox ('Out of Gantry Offset', 'Ajinextek', MB_OK);
        end;
     HOME_ERR_USER_BREAK:
        begin
           Application.MessageBox ('Home search stops.', 'Ajinextek', MB_OK);
           bFlag := False;
        end;
     HOME_ERR_VELOCITY:
```

```
        begin
           Application.MessageBox ('Velocity has not been set.', 'Ajinextek', MB_OK);
        end;
     HOME_ERR_AMP_FAULT:
        begin
           Application.MessageBox ('Servo pack alarm has occured.', 'Ajinextek',
              MB_OK);
           bFlag := False;
        end;
     HOME_ERR_NEG_LIMIT:
        begin
           Application.MessageBox ('(-) limit sensor has been detected.', 'Ajinextek',
              MB_OK);
           bFlag := False;
        end;
     HOME_ERR_POS_LIMIT:
        begin
           Application.MessageBox ('(+) limit sensor has been detected', 'Ajinextek',
              MB_OK);
           bFlag := False;
        end;
     HOME_SEARCHING:
        begin
           Application.MessageBox ('Home search is now in progress', 'Ajinextek',
              MB_OK);
           bFlag := False;
        end;
     HOME_ SUCCESS:
        begin
           Application.MessageBox ('Home search was successful', 'Ajinextek', MB_OK);
           bFlag := False;
        end;
     end;
  end;
end;
```

See Also

AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetRate

AxmHomeGetRate

Purpose

You can check the progress rate after home search starts. After home search is completed, it returns 100 regardless of whether or not it was successful. You can verify whether or not home search was successful using GetHome Result API.

Format

C++

```
DWORD AxmHomeGetRate(long lAxisNo DWORD * upHomeMainStepNumber, DWORD
*upHomeStepNumber);
```

Visual Basic

```
Function AxmHomeGetRate(ByVal lAxisNo As Long, ByRef
upHomeMainStepNumber As Long, ByRef upHomeStepNumber As Long) As Long
```

Delphi

```
function AxmHomeGetRate(lAxisNo : LongInt; upHomeStepNumber : PDWord ;
upHomeStepNumber : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (start from 0) to verify home search progress rate
[<<]upHomeMainStepNumber	Main Step progress rate(0 – 100) – If gantry is FALSE and upHomeMainStep Number : 0, only the selected axis is in progress and the home progress rate displays upHomeStepNumber. – If gantry is TRUE and upHomeMainStep Number : 0, the master home is in progress and the master home progress rate displays upHomeStepNumber. – If gantry is TRUE and upHomeMainStep Number : 10, the slave home is in progress and the master home progress rate displays upHomeStepNumber.
[<<]upHomeStepNumber	Verifies home search progress rate – [0] When the library is not initialized; when it is an invalid channel(axis) number (start from 0); when home search is not in progress (unit:%) – [0~99] Home search progress rate(unit:%) – [100] When home search is completed(unit:%)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

You can check the progress rate after home search starts. After home search is completed, it returns 100 regardless of whether or not it was successful. You can verify whether or not home search was successful using GetHome Result API.

Example

C++

```
BOOL bRun = TRUE;
DWORD uHomeResult, uHomeStepNumber,uHomeMainStepNumber;
AxmHomeSetStart(0);

while(bRun)
{
  AxmHomeGetResult(0, &uHomeResult);
  switch(uHomeResult)
  {
  case HOME_ERR_UNKNOWN:
     printf("Unknown axis number");
     bRun  = FALSE;
  break;
  case HOME_ERR_GNT_RANGE:
     printf("Out of Gantry Offset");
     bRun = FALSE;
  break;
  case HOME_ERR_USER_BREAK:
     printf("Home search stops");
     bRun = FALSE;
  break;
  case HOME_ERR_VELOCITY:
     printf("Velocity has not been set.");
     bRun = FALSE;
  break;
  case HOME_ERR_AMP_FAULT:
     printf("Servo pack alarm has occured");
     bRun = FALSE;
  break;
  case HOME_ERR_NEG_LIMIT:
     printf("(-)limit sensor has been detected");
     bRun = FALSE;
  break;
  case HOME_ERR_POS_LIMIT:
     printf("(+)limit sensor has been detected");
     bRun = FALSE;
  break;
  case HOME_SEARCHING:
     // Verify progress rate.
     AxmHomeGetRate( 0, &uHomeMainStepNumber , &uHomeStepNumber);
     printf("\t Home search is now %d in progress", uHomeStepNumber);
     Sleep(100);
```

```
   break;
   case HOME_SUCCESS:
      printf("Home search was successful");
      bRun = FALSE;
   break;
   }
}
```

Visual Basic

```
Dim bFlag As Byte
Dim  uID As Long
Dim uHomeResult , uHomeMainStepNumber ,uHomeStepNumber As Long
bFlag = True
AxmHomeSetStart 0
Do While bFlag
AxmHomeGetResult 0, uHomeResult
uID      = uHomeResult
Select Case uID
Case HOME_ERR_UNKNOWN:
   MsgBox "Unknown axis number", vbOKCancel
   bFlag  = False
Case HOME_ERR_GNT_RANGE:
   MsgBox "Out of Gantry Offset", vbOKCancel
   bFlag  = False8
Case HOME_ERR_USER_BREAK:
   MsgBox "Home search stops", vbOKCancel
   bFlag  = False
Case HOME_ERR_VELOCITY:
   MsgBox "Velocity has not been set.", vbOKCancel
   bFlag  = False
Case HOME_ERR_AMP_FAULT:
   MsgBox "Servo pack alarm has occured", vbOKCancel
   bFlag  = False
Case HOME_ERR_NEG_LIMIT:
   MsgBox "(-)limit sensor has been detected", vbOKCancel
   bFlag  = False
Case HOME_ERR_POS_LIMIT:
   MsgBox "(+)limit sensor has been detected", vbOKCancel
   bFlag  = False
Case HOME_SEARCHING:
   ' Verify progress rate.
   AxmHomeGetRate 0, uHomeMainStepNumber , uHomeStepNumber
   nCount = uHomeStepNumber
   strData = " Home search is now in progress. The rate is " + CStr(nCount) + "."
  MsgBox strData
   bFlag  = False
Case HOME_SUCCESS:
   MsgBox "Home search was successful", vbOKCancel
   bFlag  = False
```

```
End Select
Loop
```

Delphi

```
var
  uID : Word; bFlag : Boolean; strData : String;
  uHomeResult, uHomeStepNumber, uHomeMainStepNumber : Dword
begin
bFlag := True;
AxmHomeSetStart(0);
while bFlag do
begin
  AxmHomeGetResult(0, @uHomeResult);
  uID   := uHomeResult;
  case (uID) of
     HOME_ERR_UNKNOWN:
        begin
           Application.MessageBox ('Unknown axis number.', 'Ajinextek', MB_OK);
           bFlag := False;
        end;
     HOME_ERR_GNT_RANGE:
        begin
           Application.MessageBox ('Out of Gantry Offset', 'Ajinextek', MB_OK);
        end;
     HOME_ERR_USER_BREAK:
        begin
           Application.MessageBox ('Home search stops.', 'Ajinextek', MB_OK);
           bFlag := False;
        end;
     HOME_ERR_VELOCITY:
        begin
           Application.MessageBox ('Velocity has not been set.', 'Ajinextek', MB_OK);
        end;
     HOME_ERR_AMP_FAULT:
        begin
           Application.MessageBox ('Servo pack alarm has occured.', 'Ajinextek',
              MB_OK);
           bFlag := False;
        end;
     HOME_ERR_NEG_LIMIT:
        begin
           Application.MessageBox ('(-)limit sensor has been detected.', 'Ajinextek',
              MB_OK);
           bFlag := False;
        end;
     HOME_ERR_POS_LIMIT:
        begin
           Application.MessageBox ('(+)limit sensor has been detected', 'Ajinextek',
```

```
            MB_OK);
         bFlag := False;
      end;
   HOME_SEARCHING:
      begin
         { Verify progress rate. }
         AxmHomeGetRate( 0, @ uHomeMainStepNumber ,@uHomeStepNumber);
         nCount      := uHomeStepNumber ; { Verify home search rate on axis 0. }
         strData     := ' Home search is now in progress. The rate is ' +
            IntToStr(nCount) + '.';
         Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
         bFlag := False;
      end;
   HOME_SUCCESS:
      begin
         Application.MessageBox ('Home search was successful', 'Ajinextek', MB_OK);
         bFlag := False;
      end;
   end;
 end;
end;
```

See Also

AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult

Position Move API

In this chapter, we introduce APIs related with not Interpolation Control but Single Axis, Multi Axis motion control. Single axis motion means the work that independently controls only one axis, while multi axis motion means the work that controls more than one axis. Multi axes that are not interpolation control set velocity and acceleration, and perform moving tasks. Also, if necessary, they stop motions using stop API.

The APIs are the following :

Function	Description
AxmMoveStartPos	Moves at preset velocity and acceleration rate up to the position that was set as an absolute coordinate of a specific axis. Velocity profile is set in AxmMotSetProfileMode API. Exits API at the point of pulse out.
AxmMovePos	Moves at preset velocity and acceleration rate up to the position that was set as an absolute coordinate of a specific axis. Velocity profile is set in AxmMotSetProfileMode API. Exits API at the point that pulse out is exited.
AxmMoveVel	Constantly moves in velocity mode at the velocity and acceleration rate preset on a specific axis. Exits API at the point that pulse out starts.
AxmMoveStartMultiVel	Constantly moves in velocity mode at the velocity and acceleration rate preset on specific multi axes. Exits API at the point that pulse out starts.
AxmMoveSignalSearch	Moves until the signal set to a specific velocity and acceleration rate is detected. Exits API at the point of pulse out.
AxmMoveSignalCapture	Moves in order to detect a signal set on a spcific axis and store the position.
AxmMoveGetCapturePos	Verifies the position value stored in ‘AxmMoveSignalCapture‘ API.
AxmMoveStartMultiPos	Moves at preset velocity and acceleration rate up to the positions that was set as absolute coordinates of specific axes. Velocity profile is set in AxmMotSetProfileMode API. Exits API at the point that pulse out is exited.
AxmMoveMultiPos	Moves at preset velocity and acceleration rate up to the positions that was set as absolute coordinates of specific axes. Velocity profile is set in AxmMotSetProfileMode API. Exits API at the point of pulse out.
AxmMoveStop	Deceleration stop at the deceleration that was set on a specific axis (The unit of deceleration is PPS[Pulses/sec] w Unit/pulse is set to 1/1).
AxmMoveEStop	Emergency stop a specific axis.
AxmMoveSStop	Deceleration stop a specific axis.

* Single axis move is generally the most fundamental move in motion move. Single axis drive is composed of position drive, velocity drive, signal search drive, and home search drive.

* Velocity drive is the move such that it keeps moving based on preset velocity acceleration (The unit of

acceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1) and acceleration time until exit command is called.

* Signal search drive is the move such that it initially moves at a uniform speed after presetting searching targets such as limit signal or universal input/output signal, and it stops when the preset signal is detected. Signal search drive consists of AxmMoveSignalSearch API which decelerates or emergency stops if a specific signal is detected and AxmMoveSignalCapture API which stores the position. There are 8 searchable signals as shown in the table below. Since each signal contains Up Edge and Down Edge, there are a total of 16 signals.

* Generally in equipment move systems, there are many instanes that either many axies have to be moved to a specific position at once or each axis has to move in relation with each other in some patterns. Multi axis move APIs are provided in accordance with these moves. Settings for multi axis move can be done by following the single axis move settings for all axes, as previously discussed. For example, if there are two axes, you can set the same settings as that of single axis move API to each of axis 0 and axis 1. Multi axis move APIs are divided into the following: multi axis position move that enables many axes to move at once to a specific position at a specific velocity, two axes interpolation move up to a specific position, continuous interpolation move that enables many axes to move according to specific paths.

AxmMoveStartPos

Purpose

Moves at preset velocity and acceleration rate up to a preset position of a specific axis.

Exits API at the point that pulse out starts.

Format

C++

```
DWORD AxmMoveStartPos(long lAxisNo, double dPos, double dVel, double
dAccel, double dDecel);
```

Visual Basic

```
Function AxmMoveStartPos(ByVal lAxisNo As Long, ByVal dPos As Double,
ByVal dVel As Double, ByVal dAccel As Double, ByVal dDecel As Double)
As Long
```

Delphi

```
function AxmMoveStartPos(lAxisNo : LongInt; dPos : Double; dVel :
Double; dAccel : Double; dDecel : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (start from 0)
[>>]dPos	Moving distance
[>>]dVel	Velocity
[>>]dAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dDecel	Deceleration (The unit of deceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

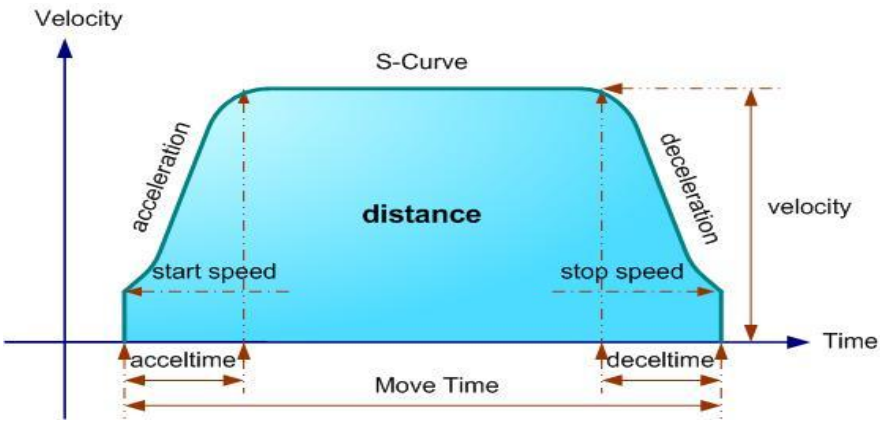
Description

It executes move by a specific distance (absolute/relative) from the current position of a specific axis. First choose absolute/relative using AxmMotSetAbsRelMode API and choose a profile using AxmMotSetProfileMode API. In absolute position mode, the setting position value becomes the absolute position value to move to, while in relative position mode, it becomes the moving distance. For example, in absolute position mode, if the position value has been set to '100', it moves to the position '100' and stop. In relative postion mode, it moves from the current postion by '100'.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

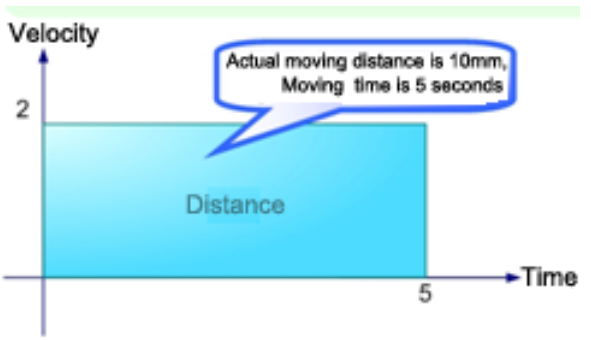
▶ In case of using master, slave by using AxmGantrySetEnable , if a command is executed on the slave axis, the command will be ignored.



Understanding velocity

Velocity = change in Distance / Change in Time
Distance = Velocity x Time

Velocity is the ratio of changes in distance to changes in time, so the unit of velocity uses mm/s or so. Note that mm is the moving distance in millimeters and s is the time in seconds. Therefore, assuming that the velocity is 2 mm/s, it means when time changes by 1 second, distance changes by 2mm.



Understanding acceleration (The unit of acceleration is PPS[Pulses/sec] when Unit/pulse is set to 1/1)

If acceleration is 0, it means no change in velocity. That is, it moves at a constant velocity. The unit of acceleration is mm/s2. Note that mm is the moving distance in millimeters and s2 is square of the time.

Acceleraion = Change in Velocity / Change in Velocity

Example

C++

```
// Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on
// the absolute coordinates of axis 0. Once the move starts, it exits API.
AxmMoveStartPos(0, 2000, 100, 200, 200);
```

Visual Basic

```
' Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on
' the absolute coordinates of axis 0. Once the move starts, it exits API.
AxmMoveStartPos 0, 2000, 100, 200, 200
```

Delphi

```
Begin
  { Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on the absolute
coordinates of axis 0. Once the move starts, it exits API. }
  AxmMoveStartPos(0, 2000, 100, 200, 200);
end;
```

See Also

AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMovePos

Purpose

Moves at preset velocity and acceleration rate up to a preset position of a specific axis.

Exits API at the point that pulse out is exited.

Format

C++

```
DWORD AxmMovePos(long lAxisNo, double dPos, double dVel, double
dAccel, double dDecel);
```

Visual Basic

```
Function AxmMovePos(ByVal lAxisNo As Long, ByVal dPos As Double,
ByVal dVel As Double, ByVal dAccel As Double, ByVal dDecel As Double)
As Long
```

Delphi

```
function AxmMovePos(lAxisNo : LongInt; dPos : Double; dVel : Double;
dAccel : Double; dDecel : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dPos	Moving distance
[>>]dVel	Velocity
[>>]dAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dDecel	Deceleration (The unit of deceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

Description

It executes move by a specific distance (absolute/relative) from the current position on a specific axis. First choose absolute/relative using AxmMotSetAbsRelMode API and choose a profile using AxmMotSetProfileMode API. It moves at preset velocity and acceleration up to a preset position of a specific axis. It exits API at the point that pulse out is exited. Since it will occupy CPU process until from the time API starts to the moment the move is exited, special caution may be required in its usage. You cannot verify whether it is in motion using AxmStatusReadInMotion API.

In absolute position mode, the setting position value becomes the absolute position value to move to, while in relative position mode, it becomes the moving distance.

For example, in absolute position mode, if the position value has been set to '100', it moves to the position '100' and stop. In relative postion mode, it moves from the current postion by '100'.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

▶ In case of using master, slave using by AxmGantrySetEnable , if a command is executed on the slave axis, the command will be ignored.

Example

C++

```
// Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on the absolute
// coordinates of axis 0. It waits until the move exits, and then exits API.
AxmMovePos (0, 2000, 100, 200, 200);
```

Visual Basic

```
' Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on the absolute
' coordinates of axis 0. It waits until the move exits, and then exits API.
AxmMovePos 0, 2000, 100, 200, 200
```

Delphi

```
begin
  { Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on the absolute
coordinates of axis 0. It waits until the move exits, and then exits API. }
  AxmMovePos (0, 2000, 100, 200, 200);
end;
```

See Also

AxmMoveStartPos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveVel

Purpose

Velocity mode move API that moves at a preset velocity on a specific axis.

Exits API at the point that pulse out starts.

Format

C++

```
DWORD AxmMoveVel(long lAxisNo, double dVel, double dAccel, double
dDecel);
```

Visual Basic

```
Function AxmMoveVel(ByVal lAxisNo As Long, ByVal dVel As Double,
ByVal dAccel As Double, ByVal dDecel As Double) As Long
```

Delphi

```
function AxmMoveVel(lAxisNo : LongInt; dVel : Double; dAccel : Double;
dDecel : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (starting from 0)
[>>]dVel	Velocity (+direction: more than 0 , -direction: less than 0)
[>>]dAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dDecel	Deceleration (The unit of deceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

Description

It initially accelerates up to the working velocity and maintain it. It can be stopped by stop API calls or external sensor signals, and it continues the motion in the specific direction. You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ In case of using master, slave by using AxmGantrySetEnable , if a command is executed on the slave axis, the command will be ignored.

Example

C++

```
// Velocity drive at velocity 100 and acceleration rate 200 on the absolute
// coordinates of axis 0.
AxmMoveVel(0, 100, 200, 200);
```

Visual Basic

```
' Velocity drive at velocity 100 and acceleration rate 200 on the absolute
' coordinates of axis 0.
AxmMoveVel 0, 100, 200, 200
```

Delphi

```
Begin
  { Velocity drive at velocity 100 and acceleration rate 200 on the absolute
coordinates of axis 0. }
  AxmMoveVel(0, 100, 200, 200);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveStartMultiVel

Purpose

Velocity mode move API that moves at a preset velocity on specific multi axes.

Exits API at the point that pulse out starts.

Format

C++

```
DWORD AxmMoveStartMultiVel(long lArraySize, long *lpAxisNo, double
*dpVel, double *dpAccel, double *dpDecel);
```

Visual Basic

```
Function AxmMoveStartMultiVel(ByVal lArraySize As Long, ByRef
lpAxisNo As Long, ByRef dpVel As Double, ByRef dpAccel As Double,
ByRef dpDecel As Double) As Long
```

Delphi

```
function AxmMoveStartMultiVel(lArraySize : LongInt; lpAxisNo :
PLongInt; dpVel : PDouble; dpAccel : PDouble; dpDecel : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lArraySize	Axis size
[>>]lpAxisNo	Channel (axis) number (start from 0) array
[>>]dpVel	Velocity (+direction: more than 0 , -direction: less than 0) array
[>>]dpAccel	Acceleration array (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dpDecel	Deceleration array (The unit of deceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

Description

Multi axis concurrent control is the function that can control multiple axis simultaneously with perfect sync. If velocity profiles have been set equally, many of the axes can be controlled with perfect synchronization of not only start/end points but also acceleration/deceleration intervals. It starts velocity move tasks of many axes simultaneously. Velocity move can accelerate up to the work velocity and maintain the work velocity. It can be stopped until a stop API is called or by external sensors;

otherwise, it continues to execute motion in the specified direction. Using this API is very helpful when multiple axes start work simultaneously and synchronized with each other.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

Example

C++

```
// Velocity drive at velocity 100, acceleration rate 200 and with S-curve.

long  lAxis[2];
double dPos[2],dVel[2];
double dAccel[2],dDecel[2];

for( int i=0; i<2; i++)
{
  AxmMotSetProfileMode(i, 4);
  AxmMotSetAbsRelMode(i, 0);
  lAxis[i]    = i;
  dVel[i] = 100;
  dAccel[i] = 200;
  dDecel[i] = 200;
}
AxmMoveStartMultiVel(2, lAxis, dVel, dAccel, dDecel);
```

Visual Basic

```
' Velocity drive at velocity 100, acceleration rate 200 and with S-curve.
Dim axes(0 To 1) As Long
Dim velocities(0 To 1) As Double
Dim accelerations(0 To 1) As Double
Dim decelerations(0 To 1) As Double
Dim i As Long

For i = 0 To 1
  AxmMotSetProfileMode i, 4
  AxmMotSetAbsRelMode i, 0
  axes(i) = i
  velocities(i) = 100
  accelerations(i) = 200
  decelerations(i) = 200
Next i

AxmMoveStartMultiVel 2, axes(0), velocities(0), accelerations(0), decelerations(0)
' Setting deceleration stop.
```

Delphi

```
var
  i : LongInt;
  lAxis : array [0..1] of LongInt;
  dVel : array [0..1] of Double;
```

```
  dAccel   : array [0..1] of Double;
  dDecel   : array [0..1] of Double;

begin
  { Velocity drive at velocity 100, acceleration rate 200 and with S-curve
  profile on the axis 0 and axis 1. }
  for i := 0 to 1 do
  begin
     AxmMotSetProfileMode(i, 4);
     AxmMotSetAbsRelMode(i, 0);
  dVel[i]  := 100;
  dAccel[i]  := 200;
  dDecel[i]  := 200;
  lAxis[i]     := i;
  end;

  AxmMoveStartMultiVel (2, @lAxis, @dVel, @dAccel, @dDecel);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveSignalSearch

Purpose

API that moves in order to detect the signal that was set on a specific axis.

Format

C++

```
DWORD AxmMoveSignalSearch(long lAxisNo, double dVel, double dAccel,
long lDetectSignal, long lSignalEdge, long lSignalMethod);
```

Visual Basic

```
Function AxmMoveSignalSearch(ByVal lAxisNo As Long, ByVal dVel As
Double, ByVal dAccel As Double, ByVal lDetectSignal As Long, ByVal
lSignalEdge As Long, ByVal lSignalMethod As Long) As Long
```

Delphi

```
function AxmMoveSignalSearch(lAxisNo : LongInt; dVel : Double;
dAccel : Double; lDetectSignal : LongInt; lSignalEdge : LongInt;
lSignalMethod : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (start from 0) array
[>>]dVel	Velocity (+direction: more than 0 , -direction: less than 0) array
[>>]dAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]lDetectSignal	Sets detection signal : AXT_MOTION_HOME_DETECT_SIGNAL_DEF - [00h] +Elm(End limit) + direction limit sensor signal - [01h] -Elm(End limit) - direction limit sensor signal - [02h] Reserved - [03h] Reserved - [04h] IN0(ORG) home sensor signal - [05h] IN1(Z) Encoder Z phase signal - [06h] IN2(universal) universal intput signal #2 - [07h] IN3(universal) universal input signal #3
[>>]lSignalEdge	Sets Edge direction of detection signal : AXT_MOTION_EDGE_DEF - [00h] Down Edge - [01h] Up Edge
[>>]lSignalMethod	Stop mode : AXT_MOTION_STOPMODE_DEF - [00h] Emergency stop - [01h] Slowdown stop

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion

[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

Description

It is the API that moves in order to search the signal that was set on a specific axis.

It can detect limit sensor signal, home sensor signal, Z phase signal of servo driver or Edges of universal input signals.

It is mainly used to implement home search move APIs.

Caution: In case of using QI, if you search IDetectSignal 0I PosEndLimit , NegEndLimit(0,1), it will detect the signal's level active state.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

Caution: If SignalMethod is used as EMERGENCY_STOP(0), the acceleration/deceleration are ignored and it accelerates at the specified velocity and emergency stops.

Example

C++

```
// If Z phase signal on axis 0 is encountered, it detects Falling Edge as it
// deceleration stops.
AxmMoveSignalSearch (0, 100, 200, 5, 0, 0);
```

Visual Basic

```
' If Z phase signal on axis 0 is encountered, it detects Falling Edge as it
' deceleration stops.
AxmMoveSignalSearch 0, 100, 200, 5, 0, 0
```

Delphi

```
Begin
  { If Z phase signal on axis 0 is encountered, it detects Falling Edge as it
deceleration stops. }
  AxmMoveSignalSearch (0, 100, 200, 5, 0, 0);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel,
AxmMoveSignalCapture,AxmMoveGetCapturePos, AxmMoveStartMultiPos,
AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveSignalCapture

Purpose

API that detects signals that were set on a specific axis and moves to store the positions.

Format

C++

```
DWORD AxmMoveSignalCapture(long lAxisNo, double dVel, double dAccel,
long lDetectSignal, long lSignalEdge, long lTarget, long
lSignalMethod);
```

Visual Basic

```
Function AxmMoveSignalCapture(ByVal lAxisNo As Long, ByVal dVel As
Double, ByVal dAccel As Double, ByVal lDetectSignal As Long, ByVal
lSignalEdge As Long, ByVal lTarget As Long, ByVal lSignalMethod As
Long) As Long
```

Delphi

```
function AxmMoveSignalCapture(lAxisNo : LongInt; dVel : Double;
dAccel : Double; lDetectSignal : LongInt; lSignalEdge : LongInt;
lTarget : LongInt; lSignalMethod : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (start from 0) array
[>>]dVel	Velocity (+direction: more than 0 , –direction: less than 0) array
[>>]dAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]lDetectSignal	Sets detection signal : AXT_MOTION_HOME_DETECT_SIGNAL_DEF – [00h] +Elm(End limit) + direction limit sensor signal – [01h] –Elm(End limit) – direction limit sensor signal – [02h] Reserved – [03h] Reserved – [04h] IN0(ORG) home sensor signal – [05h] IN1(Z) Encoder Z phase signal – [06h] IN2(universal) universal intput signal #2 – [07h] IN3(universal) universal input signal #3
[>>]lSignalEdge	Sets Edge direction of detection signal : AXT_MOTION_EDGE_DEF – [00h] Down Edge – [01h] Up Edge
[>>]lTarget	Sets to detect the location : AXT_MOTION_SELECTION_DEF – [00h] Target position (Stores Cmd position) – [01h] Actual position (ACT (1) : Stores Encoder position)
[>>]lSignalMethod	Stop mode : AXT_MOTION_STOPMODE_DEF – [00h] Emergency stop – [01h] Slowdown stop

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

Description

It is the API that detects the signal that was set on a specific axis and moves to store the position.

It can detect limit sensor signal, home sensor signal, Z phase signal of servo driver or Edges of universal input signals.

In signal search, Cmd position or Encoder position can be stored.

The stored position can be verified using 'AxmMoveGetCapturePos' API.

Caution: In case of using QI, if you search IDetectSignal 0I PosEndLimit , NegEndLimit(0,1), it will detect the signal's level active state.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

Caution: If SignalMethod is used as EMERGENCY_STOP(0), the acceleration/deceleration is ignored and it accelerates at the specified velocity and emergency stops.

Example

C++

```
// When Z phase signal Falling Edge on axis 0 is detected, it stores
// encoder position as it deceleration stops.
AxmMoveSignalCapture(0, 100, 200, 5, 0, ACTUAL, 0);
```

Visual Basic

```
' When Z phase signal Falling Edge on axis 0 is detected, it stores
' encoder position as it deceleration stops.
AxmMoveSignalCapture 0, 100, 200, 5, 0, ACTUAL, 0
```

Delphi

```
Begin
  { When Z phase signal Falling Edge on axis 0 is detected, it stores encoder position
as it deceleration stops. }
  AxmMoveSignalCapture(0, 100, 200, 5, 0, ACTUAL, 0);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveGetCapturePos,

AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveGetCapturePos

Purpose

API that verifies the position values stored in 'AxmMoveSignalCapture' API.

Format

C++

```
DWORD AxmMoveGetCapturePos(long lAxisNo, double *dpCapPos);
```

Visual Basic

```
Function AxmMoveGetCapturePos(ByVal lAxisNo As Long, ByRef dpCapPos
As Double) As Long
```

Delphi

```
function AxmMoveGetCapturePos(lAxisNo : LongInt; dpCapPos : PDouble) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpCapPos	Position value in signal search

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4162] AXT_RT_NOT_CAPTURED : Failed to capture the position
* See error code Table for more information on status error codes

Description

It is the API that verifies the position values stored in 'AxmMoveSignalCapture' API.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

Example

C++

```
// Verify position during Z phase detection.
double dPos;
AxmMoveGetCapturePos (0, &dPos);
```

Visual Basic

```
' Verify position during Z phase detection.
Dim dPos As Double
AxmMoveGetCapturePos 0, dPos
```

Delphi

```
{ Verify position during Z phase detection. }
var
   dPos : Double;

begin
   AxmMoveGetCapturePos (0, @dPos);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveStartMultiPos

Purpose

Starts move from the current position by a specific distance about multiple axes. If this API is used, multiple axes start working simultenously. This API is used when multiple axes need to start working with synchronization. Exits API at the point that pulse out starts.

Format

C++

```
DWORD AxmMoveStartMultiPos( long lArraySize, long *lpAxisNo, double
*dpPos, double *dpVel, double *dpAccel, double *dpDecel);
```

Visual Basic

```
Function AxmMoveStartMultiPos( ByVal lArraySize As Long, ByRef
lpAxisNo As Long, ByRef dpPos As Double, ByRef dpVel As Double, ByRef
dpAccel As Double, ByRef dpDecel As Double) As Long
```

Delphi

```
function AxmMoveStartMultiPos(lArraySize : LongInt; lpAxisNo :
PLongInt; dpPos : PDouble; dpVel : PDouble; dpAccel : PDouble;
dpDecel : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lArraySize	Axis size
[>>]lpAxisNo	Channel (axis) number (start from 0) array
[>>]dpPos	Moving distance array
[>>]dpVel	Velocity array
[>>]dpAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1) array
[>>]dpDecel	Deceleration (The unit of deceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1) array

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

Description

It executes move by a specific distance(absolute/relative) from the current position of one axis. First choose absolute/relative using AxmMotSetAbsRelMode API and choose a profile using AxmMotSetProfileMode API. You can verify whether it is in motion using AxmStatusReadInMotion API.

In absolute position mode, the setting position value becomes the absolute position value to move to, while in relative position mode, it becomes the moving distance. For example, if the position value has been set to '100', it moves to the position '100' and stop in absolute position mode, whereras in relative postion mode, it moves from the current postion by '100'.

Multi axis concurrent control is the function that can control multiple axis simultaneously with perfect sync. If velocity profiles have been set equally, many of the axes can be controlled with perfect synchronization of not only start/end points but also acceleration/deceleration intervals.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

Example

C++

```
// Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on
// the absolute coordinates. It exits API if the move starts.
long  lAxis[2];  double dPos[2],dVel[2], dAccel[2];
for( int i=0; i<2; i++){
AxmMotSetProfileMode(i, 3);
AxmMotSetAbsRelMode(i, 1);
lAxis[i]    = i;
dPos[i]  = 2000;
dVel[i]  = 100;
dAccel[i]    = 200; }
AxmMoveStartMultiPos (2, lAxis, dPos, dVel, dAccel, dAccel);
```

Visual Basic

```
' Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on
' the absolute coordinates. It exits API if the move starts.
Dim Distance(0 To 2) As Double
Dim velocities(0 To 2) As Double
Dim accelerations(0 To 2) As Double
Dim axis(0 To 2) As Long  Dim i As Long
For i = 0 To 2
  AxmMotSetProfileMode i, 3
  AxmMotSetAbsRelMode i, 1
  axis(i) = i
  Distance(i) = 2000
  velocities(i) = 100
  accelerations(i) = 200
Next i
AxmMoveStartMultiPos 2 , axis(0), Distance(0), velocities(0), accelerations(0),
  accelerations(0)
```

Delphi

```
{ Trapezoidal move by 2000 at velocity 100 and acceleration rate 200 on the absolute
coordinates. It exits API if the move starts. }
```

```
var
   i : LongInt;
   lAxis : array [0..1] of Longint;
  dPos : array [0..1] of Double;
  dVel : array [0..1] of Double;
   dAccel : array [0..1] of Double;
   uStopMode : DWORD;

Begin
   for i := 0 to 2 do
   begin
      AxmMotSetProfileMode(i, 3);
      AxmMotSetAbsRelMode (i, 1);
      dPos[i]   := 2000;
      dVel[i]  := 100;
      dAccel[i]     := 200;
      lAxis[i]      := i;
   end;
   AxmMoveStartMultiPos (2, @lAxis, @dPos, @dVel, @ dAccel, @dAccel);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveMultiPos

Purpose

Starts move from the current position by a specific distance about multiple axes. If this API is used, multiple axes start working simultenously. This API is used when multiple axes need to start working with synchronization. Exits API at the point that pulse out is exited.

Format

C++

```
DWORD AxmMoveMultiPos(long lArraySize, long *lpAxisNo, double *dpPos,
double *dpVel, double *dpAccel, double *dpDecel);
```

Visual Basic

```
Function AxmMoveMultiPos(ByVal lArraySize As Long, ByRef lpAxisNo As
Long, ByRef dpPos As Double, ByRef dpVel As Double, ByRef dpAccel As
Double, ByRef dpDecel As Double) As Long
```

Delphi

```
function AxmMoveMultiPos(lArraySize : LongInt; lpAxisNo : PLongInt;
dpPos : PDouble; dpVel : PDouble; dpAccel : PDouble; dpDecel :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lArraySize	Axis size
[>>]lpAxisNo	Channel (axis) number (start from 0) array
[>>]dpPos	Moving distance array
[>>]dpVel	Velocity array
[>>]dpAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1) array
[>>]dpDecel	Deceleration (The unit of deceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1) array

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
[4255] AXT_RT_MOTION_SETTING_ERROR : Invalid setting for Velocity, Acceleration, Jerk, or Profile
* See error code Table for more information on status error codes

Description

It executes move by a specific distance(absolute/relative) from the current position on multiple axes. First choose absolute/relative using AxmMotSetAbsRelMode API and choose a profile using AxmMotSetProfileMode API.

It moves at preset velocity and acceleration up to a preset position of specific axes. It exits API at the point that pulse out is exited. Since it will occupy CPU process until from the time API starts to the moment the move is exited, special caution may be required in its usage. You cannot verify whether it is in motion using AxmStatusReadInMotion API.

In absolute position mode, the setting position value becomes the absolute position value to move to, while in relative position mode, it becomes the moving distance.

For example, in absolute position mode, if the position value has been set to '100', it moves to the position '100' and stop. In relative position mode, it moves from the current position by '100'.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

Example

C++

```
// Trapezoidal move by 2000 at velocity 100, acceleration rate 200 on the absolute
// coordinates. It exits API if the move starts.
long  lAxis[2];
double dPos[2],dVel[2], dAccel[2];
for( int i=0; i<2; i++)
{
  AxmMotSetProfileMode(i, 3);
  AxmMotSetAbsRelMode(i, 1);
  lAxis[i]    = i;
  dPos[i]   = 2000;
  dVel[i]  = 100;
  dAccel[i]    = 200;
}
AxmMoveMultiPos (2, lAxis, dPos, dVel, dAccel, dAccel);
```

Visual Basic

```
' Trapezoidal move by 2000 at velocity 100, acceleration rate 200 on the absolute
' coordinates. It exits API if the move starts.
Dim Distance(0 To 2) As Double
Dim velocities(0 To 2) As Double
Dim accelerations(0 To 2) As Double
Dim axis(0 To 2) As Long
Dim i As Long
StopMode = 1; ' Set stop mode slowdown stop
For i = 0 To 2
    AxmMotSetProfileMode i, 3
    AxmMotSetAbsRelMode i, 1
    axis(i) = i
    Distance(i) = 2000
    velocities(i) = 100
    accelerations(i) = 200
Next i
```

```
AxmMoveMultiPos 2 , axis(0), Distance(0), velocities(0), accelerations(0),
  accelerations(0)
```

Delphi

```
{ Trapezoidal move by 2000 at velocity 100, acceleration rate 200 on the absolute
coordinates. It exits API if the move starts. }
var
   i : LongInt;
   lAxis : array [0..1] of Longint;
  dPos : array [0..1] of Double;
  dVel : array [0..1] of Double;
  dAccel : array [0..1] of Double;

begin
   uStopMode = 1; { Set stop mode slowdown stop }
   for i := 0 to 2 do
      begin
         AxmMotSetProfileMode(i, 3);
         AxmMotSetAbsRelMode(i, 1);
         dPos[i]   := 2000;
         dVel[i]  := 100;
         dAccel[i]     := 200;
         lAxis[i]      := i;
      end;
   AxmMoveMultiPos (2, @lAxis, @dPos, @dVel, @ dAccel, @dAccel);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveStop, AxmMoveEStop, AxmMoveSStop

AxmMoveStop

Purpose

API that enables user to stop the current motion in move at arbitrary deceleration rate.

Format

C++

```
DWORD AxmMoveStop(long lAxisNo, double dDecel);
```

Visual Basic

```
Function AxmMoveStop(ByVal lAxisNo As Long, ByVal dDecel As Double)
As Long
```

Delphi

```
function AxmMoveStop(lAxisNo : LongInt; dDecel : Double) : DWord;
stdcall;
```

Parameters

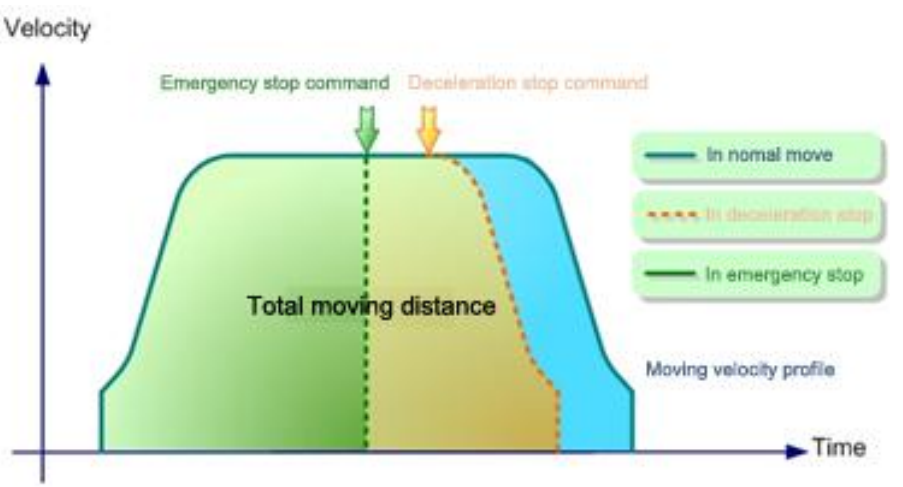
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (starting from 0)
[>>]dDecel	Deceleration rate when stopped

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It is the API that stops the current motion in move at arbitrary deceleration rate.



Example

C++

```
// Emergency stop motion move of axis 0.
AxmMoveStop(0, 0);
```

Visual Basic

```
' Emergency stop motion move of axis 0.
AxmMoveStop 0, 0
```

Delphi

```
begin
   { Emergency stop motion move of axis 0. }
   AxmMoveStop(0, 0);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveEStop, AxmMoveSStop

AxmMoveEStop

Purpose

API that emergency stops the current motion in move.

Format

C++

```
DWORD AxmMoveEStop(long lAxisNo);
```

Visual Basic

```
Function AxmMoveEStop(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmMoveEStop(lAxisNo: LongInt) : DWord; stdcall;
```

Parameters

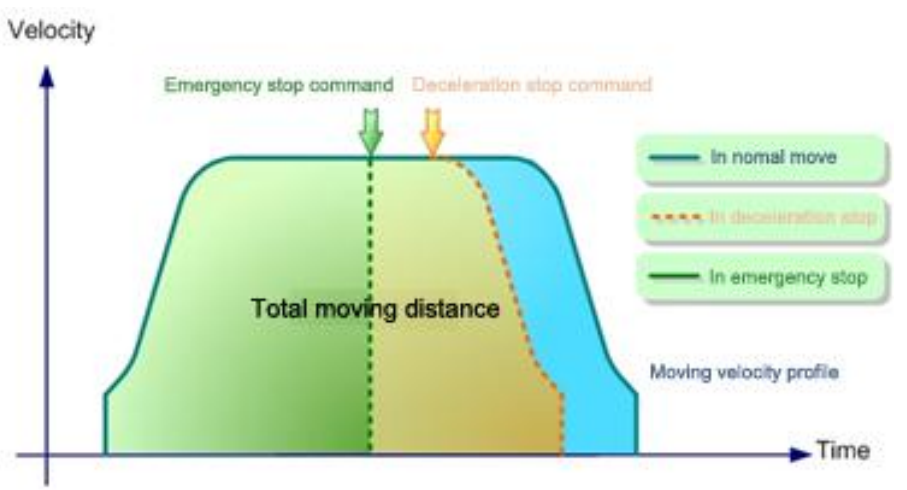
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (starting from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It is the API that emergency stops the current motion in move without deceleration.



Example

C++

```
// Emergency stop motion move of axis 0.
AxmMoveEStop(0);
```

Visual Basic

```
' Emergency stop motion move of axis 0.
AxmMoveEStop 0
```

Delphi

```
begin
   { Emergency stop motion move of axis 0. }
   AxmMoveEStop(0);
end;
```

See Also

AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveSStop

AxmMoveSStop

Purpose

API that deceleration stops the current motion in move.

Format

C++

```
DWORD AxmMoveSStop(long lAxisNo);
```

Visual Basic

```
Function AxmMoveSStop(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmMoveSStop(lAxisNo: LongInt) : DWord; stdcall;
```

Parameters

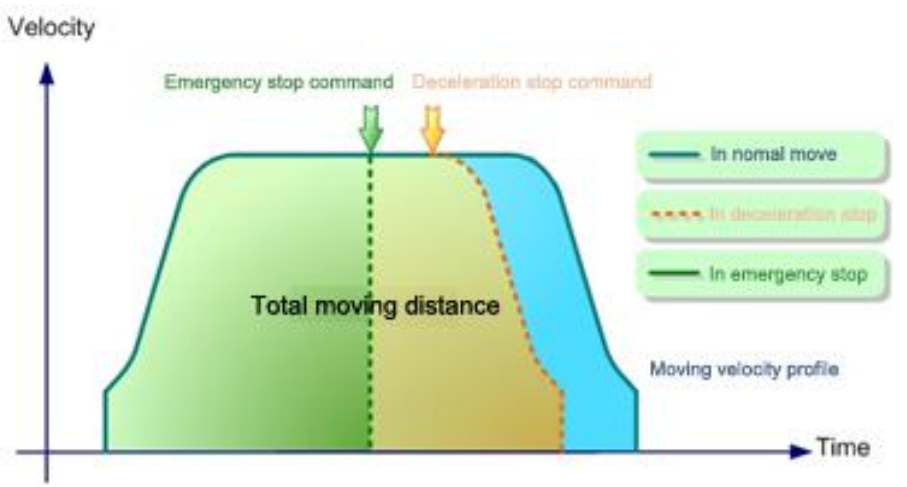
[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number (starting from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It is the API that decelerates and stops the current motion in move by the amount of the acceleration that user initially input.



Example

C++

```
// Deceleration stop motion move of axis 0.
AxmMoveSStop(0);
```

Visual Basic

```
' Deceleration stop motion move of axis 0.
AxmMoveSStop 0
```

Delphi

```
Begin
   { Deceleration stop motion move of axis 0. }
   AxmMoveSStop(0);
end;
```

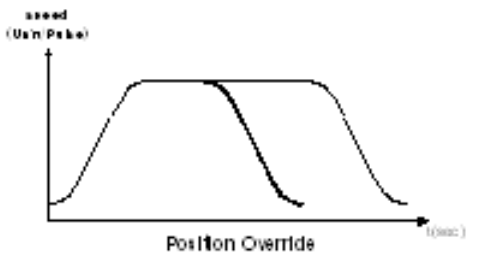
See Also

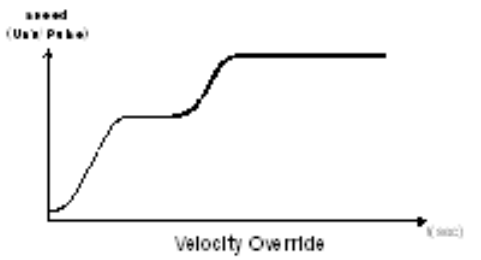
AxmMoveStartPos, AxmMovePos, AxmMoveStartMultiVel, AxmMoveSignalSearch, AxmMoveSignalCapture, AxmMoveGetCapturePos, AxmMoveStartMultiPos, AxmMoveMultiPos, AxmMoveStop, AxmMoveEStop

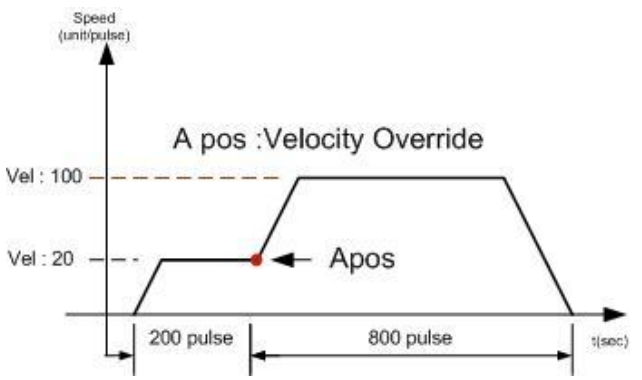
Position / Velocity Override Move API

In this chapter, we introduce velocity override move and position override move. Velocity override move means changing working velocity while motion is in progress, and position override drive means changing command distance while motion is in progress.

Function	Description
AxmOverridePos	Adjusts the number of assigned output pulses before the move of a specific axis is exited.
AxmOverrideSetMaxVel	Sets the maximum velocity to override before the velocity of a specific axis overrides.
AxmOverrideVel	Sets the velocity variably during the move of a specific axis.
AxmOverrideAccelVelDecel	Sets the velocity, acceleration and deceleration variably during the move of a specific axis.
AxmOverrideVelAtPos	Constantly moves in velocity mode at the preset velocity and acceleration rate for a specific axis. During the move, it executes velocity override at a designated position. Exits API at the point that pulse out starts.







AxmOverridePos

Purpose

API that changes preset moving distance during the motion of a specific axis.

Format

C++

```
DWORD AxmOverridePos(long lAxisNo, double dOverridePos);
```

Visual Basic

```
Function AxmOverridePos(ByVal lAxisNo As Long, ByVal dOverridePos As
Double) As Long
```

Delphi

```
function AxmOverridePos(lAxisNo : LongInt; dOverridePos : Double) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dOverridePos	Moving distance to change

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4151] AXT_RT_MOTION_ERROR_IN_NONMOTION : Not in motion drive when it should be
* See error code Table for more information on status error codes

Description

Caution: When inputting a position to override, the position must be entered as a position value in relative form because the moving distance in the moving direction is inputted as a position value in relative form.

Position override means modifying the command distance or the command coordinates while in execution with InPosition being Enable. In order to use this API, you need to use a single axis motion API such as AxmMoveStartPos while in motion.

▶ The API can be used while in motion acceleration or in constant move. Even if the command is ordered after the motion's deceleration stops, it will be ignored.

Therefore, if user's override command is ignored, you have to acknowledge that the motion has already been completed.

▶ In case that it executes linear, circular, splline, helical interpolation tasks, override cannot be used.

Note1: the position value to override is the distance value relative to current move direction; thus, position should be relative position value.

Note2: in case of SMC-2V03, it should be less than min. move distance; when move distance is over than override distance, there is emergency stop.

Note3: in case of PCI-N804/404, it should be less than initial move distance; when move distance is over than override distance, it switches to move direction opposite to override position after deceleration stop. If override occurs again, ignore it.

Example

C++

```
AxmMotSetProfileMode(0, 4);     // Asymmetric S Curve.
AxmMotSetAbsRelMode(0, 0);
AxmMoveStartPos(0, 1000, 200, 800, 800);

// Override to the position 2000 while in move of axis.
AxmOverridePos (0, 2000);
```

Visual Basic

```
AxmMotSetProfileMode 0, 4       ' Asymmetric S Curve.
AxmMotSetAbsRelMode 0, 0
AxmMoveStartPos 0, 1000, 200, 800, 800

' Override to the position 2000 while in move of axis.
AxmOverridePos 0, 2000
```

Delphi

```
begin
   { Override to the position 2000 while in move of axis. }

   AxmMotSetProfileMode(0, 4);    { Asymmetric S Curve. }
   AxmMotSetAbsRelMode(0, 0);
   AxmMoveStartPos(0, 1000, 200, 800, 800);

   AxmOverridePos (0, 2000);
end;
```

See Also

AxmOverrideSetMaxVel , AxmOverrideVel, AxmOverrideVelAtPos, AxmOverrideAccelVelDecel

AxmOverrideSetMaxVel

Purpose

Set a maximum velocity to override before overriding the velocity of a specific axis, When it is necessary to override at a higher velocity than the current velocity to move at, it sets an anticipated maximum velocity before the move. If it will override only at a lower velocity than the velocity to move at, this API doesn't have to be used.

Format

C++

```
DWORD AxmOverrideSetMaxVel(long lAxisNo, double dOverrideMaxVel);
```

Visual Basic

```
Function AxmOverrideSetMaxVel(ByVal lAxisNo As Long, ByVal
dOverrideMaxVel As Double) As Long
```

Delphi

```
function AxmOverrideSetMaxVel(lAxisNo : LongInt; dOverrideMaxVel:
Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dOverrideMaxVel	Maximum velocity to override

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion
* See error code Table for more information on status error codes

Description

It is used when it is necessary to override the velocity while a single axis motion is in progress.

In order to use this API, you need to use a single axis motion API such as AxmMoveStartPos while not in motion.

Caution : If it executes velocity override firve times, a maximum velocity must be set among them.

Example

C++

```
AxmOverrideSetMaxVel(0, 500);   // Set the maximum value of override.

AxmMotSetProfileMode(0, 4);     // Asymmetric S Curve.
AxmMotSetAbsRelMode(0, 0);
```

```
AxmMoveStartPos(0, 1000, 200, 800, 800);

// Override the velocity to 500 while in move of axis.
AxmOverrideVel (0, 500);
```

Visual Basic

```
AxmOverrideSetMaxVel 0, 500     ' Set the maximum value of override.

AxmMotSetProfileMode 0, 4       ' Asymmetric S Curve.
AxmMotSetAbsRelMode 0, 0
AxmMoveStartPos 0, 1000, 200, 800, 800

' Override the velocity to 500 while in move of axis.
AxmOverrideVel 0, 500
```

Delphi

```
begin
   AxmOverrideSetMaxVel(0, 500);    { Set the maximum value of override. }

   AxmMotSetProfileMode(0, 4);      { Asymmetric S Curve. }
   AxmMotSetAbsRelMode(0, 0);
   AxmMoveStartPos(0, 1000, 200, 800, 800);

   { Override the velocity to 500 while in move of axis. }
   AxmOverrideVel (0, 500);
end;
```

See Also

AxmOverrideVel , AxmOverridePos, AxmOverrideVelAtPos, AxmOverrideAccelVelDecel

AxmOverrideVel

Purpose

API that changes moving velocity while in motion of a specific axis.

Format

C++

```
DWORD AxmOverrideVel(long lAxisNo, double dOverrideVelocity);
```

Visual Basic

```
Function AxmOverrideVel(ByVal lAxisNo As Long, ByVal
dOverrideVelocity As Double) As Long
```

Delphi

```
function AxmOverrideVel(lAxisNo : LongInt; dOverrideVelocity:
Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dOverrideVelocity	Moving velocity to change[Unit/Sec]

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4113] AXT_RT_MOTION_INVALID_VELOCITY : Motion error occurred since the motion drive velocity value is set as zero
[4151] AXT_RT_MOTION_ERROR_IN_NONMOTION : Not in motion drive when it should be
* See error code Table for more information on status error codes

Description

It is used when it is necessary to override the velocity while a single axis motion is in progress.

In order to use this API, you need to use a single axis motion API such as AxmMoveStartPos while not in motion.

Caution: Before using AxmOverrideVel API, use AxmOverrideSetMaxVel to set the maximum velocity that is available.

EX> Assuming that velocity override is executed twice,

1. Set the higher velocity from the above two as the maximum velocity using AxmOverrideSetMaxVel.

2. Set variably the velocity of the designated axis in move where AxmMoveStartPos is appointed to be used, as the first velocity using AxmOverrideVel.

3. Set variably the velocity of the designated axis in move as the second velocity using AxmOverrideVel.

▶ The API can be used while in motion acceleration or in constant move. Even if the command is ordered after the motion's deceleration stops, it will be ignored.

Therefore, if user's override command is ignored, you have to acknowledge that the motion has already been completed.

▶ In case that it executes linear, circular, splline, helical interpolation tasks, override cannot be used.

Example

C++

```
AxmOverrideSetMaxVel(0, 500);   // Sets a maximum value of override.

AxmMotSetProfileMode(0, 4);     // Asymmetric S Curve.
AxmMotSetAbsRelMode(0, 0);
AxmMoveStartPos(0, 1000, 200, 800, 800);

// Override the velocity to 500 while in move of axis.
AxmOverrideVel (0, 500);
```

Visual Basic

```
AxmOverrideSetMaxVel 0, 500     ' Sets a maximum value of override.

AxmMotSetProfileMode 0, 4       ' Asymmetric S Curve.
AxmMotSetAbsRelMode 0, 0
AxmMoveStartPos 0, 1000, 200, 800, 800

' Override the velocity to 500 while in move of axis.
AxmOverrideVel 0, 500
```

Delphi

```
begin

  AxmOverrideSetMaxVel(0, 500);    { Sets a maximum value of override. }

  AxmMotSetProfileMode(0, 4);      { Asymmetric S Curve. }
  AxmMotSetAbsRelMode(0, 0);
  AxmMoveStartPos(0, 1000, 200, 800, 800);

  { Override the velocity to 500 while in move of axis. }
  AxmOverrideVel (0, 500);
end;
```

See Also

AxmOverrideSetMaxVel, AxmOverridePos, AxmOverrideVelAtPos, AxmOverrideAccelVelDecel

AxmOverrideAccelVelDecel

Purpose

API that changes moving velocity and accelation and decelation while in motion of a specific axis.

Format

C++

```
DWORD AxmOverrideAccelVelDecel(long lAxisNo, double dOverrideVelocity,
double dMaxAccel, double dMaxDecel);
```

Visual Basic

```
Function AxmOverrideAccelVelDecel(ByVal lAxisNo As Long, ByVal
dOverrideVelocity As Double, ByVal dMaxAccel As Double, ByVal
dMaxDecel As Double) As Long
```

Delphi

```
function AxmOverrideAccelVelDecel(lAxisNo : LongInt;
dOverrideVelocity: Double; dMaxAccel: Double; dMaxDecel: Double) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dOverrideVelocity	Moving velocity to change [Unit/Sec]
[>>]dMaxAccel	Moving accel to change [Unit/Sec^2]
[>>]dMaxDecel	Moving decel to change [Unit/Sec^2]

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4113] AXT_RT_MOTION_INVALID_VELOCITY : Motion error occurred since the motion drive velocity value is set as zero
[4151] AXT_RT_MOTION_ERROR_IN_NONMOTION : Not in motion drive when it should be
* See error code Table for more information on status error codes

Description

It is used when it is necessary to override the velocity while a single axis motion is in progress.

In order to use this API, you need to use a single axis motion API such as AxmMoveStartPos not in motion.

Caution: Before using AxmOverrideAccelVelDecel API, use AxmOverrideSetMaxVel to set the maximum velocity that is available.

EX> Assuming that velocity override is executed twice,

1. Set the higher velocity from the above two as the maximum velocity using AxmOverrideSetMaxVel.

2. Set variably the velocity of the designated axis in move where AxmMoveStartPos is appointed to be used, as the first velocity using AxmOverrideAccelVelDecel.

3. Set variably the velocity of the designated axis in move as the second velocity using AxmOverrideAccelVelDecel.

▶ The API can be used while in motion acceleration or in constant move. Even if the command is ordered after the motion's deceleration stops, it will be ignored.

Therefore, if user's override command is ignored, you have to acknowledge that the motion has already been completed.

▶ In case that it executes linear, circular, splline, helical interpolation tasks, override cannot be used.

Example

C++

```
AxmOverrideSetMaxVel(0, 500);   // Sets a maximum value of override.

AxmMotSetProfileMode(0, 4);     // Asymmetric S Curve.
AxmMotSetAbsRelMode(0, 0);
AxmMoveStartPos(0, 1000, 200, 800, 800);

// Override the velocity to 500 while in move of axis.
AxmOverrideAccelVelDecel (0, 500, 4000, 4000);
```

Visual Basic

```
AxmOverrideSetMaxVel 0, 500     ' Sets a maximum value of override.

AxmMotSetProfileMode 0, 4       ' Asymmetric S Curve.
AxmMotSetAbsRelMode 0, 0
AxmMoveStartPos 0, 1000, 200, 800, 800

' Override the velocity to 500 while in move of axis.
AxmOverrideAccelVelDecel 0, 500, 4000, 4000
```

Delphi

```
begin
  AxmOverrideSetMaxVel(0, 500);    { Sets a maximum value of override. }

  AxmMotSetProfileMode(0, 4);      { Asymmetric S Curve. }
  AxmMotSetAbsRelMode(0, 0);
  AxmMoveStartPos(0, 1000, 200, 800, 800);

  { Override the velocity to 500 while in move of axis. }
  AxmOverrideAccelVelDecel (0, 500, 4000, 4000);
end;
```

See Also

AxmOverrideSetMaxVel , AxmOverridePos, AxmOverrideVelAtPos

AxmOverrideVelAtPos

Purpose

API that changes moving velocity at the position user has specified while in motion of a specific axis.

Format

C++

```
DWORD AxmOverrideVelAtPos(long lAxisNo, double dPos, double dVel,
double dAccel, double dDecel, double dOverridePos, double
dOverrideVelocity, long lTarget);
```

Visual Basic

```
Function AxmOverrideVelAtPos(ByVal lAxisNo As Long, ByVal dPos As
Double, ByVal dVel As Double, ByVal dAccel As Double, ByVal dDecel As
Double, ByVal dOverridePos As Double, ByVal dOverrideVelocity As
Double, ByVal lTarget As Long) As Long
```

Delphi

```
function AxmOverrideVelAtPos(lAxisNo : LongInt; dPos : Double; dVel :
Double; dAccel : Double; dDecel : Double; dOverridePos : Double;
dOverrideVelocity : Double; lTarget : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dPos	Moving distance
[>>]dVel	Moving velocity (+direction: more than 0 , -direction: less than 0)
[>>]dAccel	Acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dDecel	Deceleration (The unit of deceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dOverridePos	Position where velocity will be changed
[>>]dOverrideVelocity	Moving velocity to change
[>>]lTarget	Choose position source where velocity will be changed : AXT_MOTION_SELECTION_DEF - [00h] Command position(Cmd position) - [01h] Actual position (Encoder position)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully

[4113] AXT_RT_MOTION_INVALID_VELOCITY : Motion error occurred since the motion drive velocity value is set as zero

[4151] AXT_RT_MOTION_ERROR_IN_NONMOTION : Not in motion drive when it should be

* See error code Table for more information on status error codes

Description

While moving to a designated position, it changes the velocity to the preset velocity at the position specified by user. This API is used when it needs to move either at high speed or at low speed.

Note: Before using AxmOverrideVelAtPos API, set the max. velocity that can be set by AxmOverrideSetMaxVel.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

Example

C++

```
// Move by 50000 at velocity 100 and acceleration rate 200 on the axis 0.
AxmOverrideSetMaxVel(0, 500); // Sets the override max velocity

AxmOverrideVelAtPos (0, 50000, 100, 200, 200, 10000, 500, ACTUAL);
```

Visual Basic

```
' Move by 50000 at velocity 100 and acceleration rate 200 on the axis 0.

AxmOverrideSetMaxVel 0, 500 ' Sets the override max velocity
AxmOverrideVelAtPos 0, 50000, 100, 200, 200, 10000, 500, ACTUAL
```

Delphi

```
begin
  { Move by 50000 at velocity 100 and acceleration rate 200 on the axis 0. }
  AxmOverrideSetMaxVel(0, 500); { Sets the override max velocity }

  AxmOverrideVelAtPos (0, 50000, 100, 200, 200, 10000, 500, ACTUAL);
end;
```

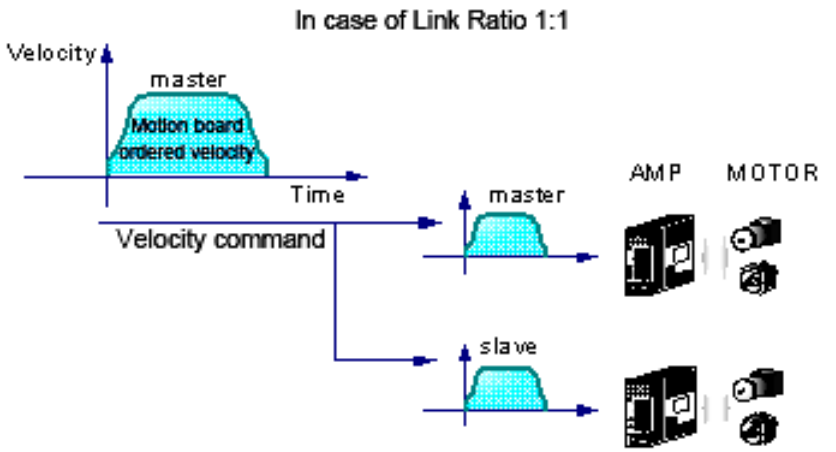
See Also

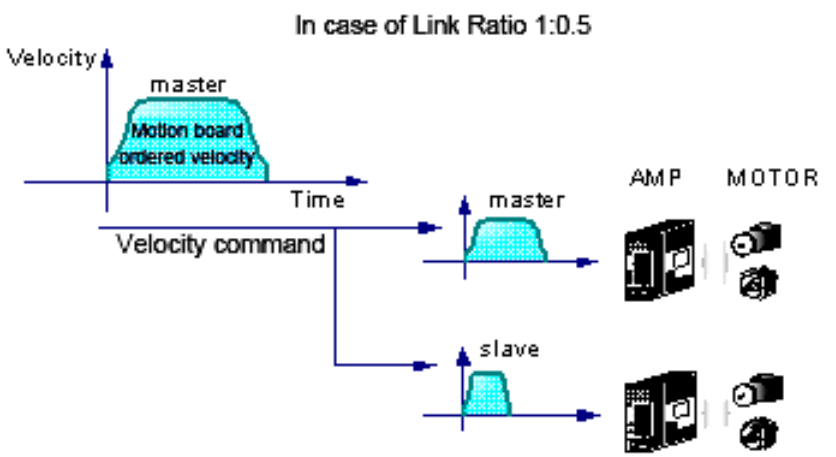
AxmOverrideSetMaxVel , AxmOverridePos, AxmOverrideVel. AxmOverrideAccelVelDecel

MASTER, SLAVE Move API

Function	Description
AxmLinkSetMode	Sets electric gear ratio between Master axis and Slave axis.
AxmLinkGetMode	Returns electric gear ratio between Master axis and Slave axis.
AxmLinkResetMode	Clear setting of electric gear ratio between Master axis and Slave axis.

Sync move, also named as electric gear mode, is the method in which independent two axes are controlled just as one axis. If user sets master axis, slave axis and the move ratio between them, the slave moves together as the master axis moves.





AxmLinkSetMode

Purpose

Set electric gear ratio between Master axis and Slave axis.

Format

C++

```
DWORD AxmLinkSetMode(long lMasterAxisNo, long lSlaveAxisNo, double
dSlaveRatio);
```

Visual Basic

```
Function AxmLinkSetMode(ByVal lMasterAxisNo As Long, ByVal
lSlaveAxisNo As Long, ByVal dSlaveRatio As Double) As Long
```

Delphi

```
function AxmLinkSetMode(lMasterAxisNo : LongInt; lSlaveAxisNo :
LongInt; dSlaveRatio : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lMasterAxisNo	Master channel(axis) number (start from 0)
[>>]lSlaveAxisNo	Slave channel(axis) number (start from 0)
[>>]dSlaveRatio	Gear ratio of slave to master axis(1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_ERROR_NOT_SAME_BOARD : Failed to initialize the motion of corresponding axis
[4163] AXT_RT_ERROR_NOT_SAME_IC : Not in the same chip
* See error code Table for more information on status error codes

Description

When a transfer command is ordered on a specific axis, this API allows that the start of the transfer motion is synchronized with the motion situation of the master axis. Set master axis and slave axis so that when the master axis moves, the slave axis moves together. Moving ratio is the ratio of a slave axis's move per one rotation of a master axis. (SlaveRatio 0 ~ 100% 0 : 0%, 0.5 : 50%, 1.0 : 100%).

Example

C++

```
// Since the ratio of master axis to slave axis is set to 0.5
// when in move, if the master moves by 1, the slave moves by 0.5.
AxmLinkSetMode(0, 1, 0.5);

// Returns electric gear ratio between Master axis and Slave axis.
double dpGearRatio;
long lpSlaveAxis;
```

```
AxmLinkGetMode(0, &lpSlaveAxis, &dpGearRatio);
```

Visual Basic

```
' Since the ratio of master axis to slave axis is set to 0.5
' when in move, if the master moves by 1, the slave moves by 0.5.
AxmLinkSetMode 0, 1, 0.5

' Returns electric gear ratio between Master axis and Slave axis.
Dim dpGearRatio As Double
Dim lpSlaveAxis As Long
AxmLinkGetMode 0, lpSlaveAxis, dpGearRatio
```

Delphi

```
{ Returns electric gear ratio between Master axis and Slave axis.}
var
   dpGearRatio: Double;
   lpSlaveAxis: LongInt;

begin
   { Since the ratio of master axis to slave axis is set to 0.5
   when in move, if the master moves by 1, the slave moves by 0.5. }
   AxmLinkSetMode(0, 1, 0.5);

   AxmLinkGetMode(0, @lpSlaveAxis, @dpGearRatio);
end;
```

See Also

AxmLinkGetMode, AxmLinkResetMode

AxmLinkGetMode

Purpose

Return electric gear ratio between Master axis and Slave axis.

Format

C++

```
DWORD AxmLinkGetMode(long lMasterAxisNo, long *lpSlaveAxisNo, double
*dpGearRatio);
```

Visual Basic

```
Function AxmLinkGetMode(ByVal lMasterAxisNo As Long, ByRef
lpSlaveAxisNo As Long, ByRef dpGearRatio As Double) As Long
```

Delphi

```
function AxmLinkGetMode(lMasterAxisNo : LongInt; lpSlaveAxisNo :
PLongInt; dpGearRatio : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lMasterAxisNo	Master channel (axis) number (start from 0)
[<<]lpSlaveAxisNo	Slave channel (axis) number
[<<]dpGearRatio	Gear ratio of slave to master axis(1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It returns the ratio between the master axis and the slave axis such that when the master axis moves, the slave axis moves together.

Moving ratio is the ratio of a slave axis's move per one rotation of a master axis.

Example

C++

```
// Since the ratio of master axis to slave axis is set to 0.5
// when in move, if the master moves by 1, the slave moves by 0.5.
AxmLinkSetMode(0, 1, 0.5);

// Returns electric gear ratio between Master axis and Slave axis.
double dpGearRatio;
long lpSlaveAxis;
AxmLinkGetMode(0, &lpSlaveAxis, &dpGearRatio);
```

Visual Basic

```
' Since the ratio of master axis to slave axis is set to 0.5
' when in move, if the master moves by 1, the slave moves by 0.5.
AxmLinkSetMode 0, 1, 0.5

' Returns electric gear ratio between Master axis and Slave axis.
Dim dpGearRatio As Double
Dim lpSlaveAxis As Long
AxmLinkGetMode 0, lpSlaveAxis, dpGearRatio
```

Delphi

```
{ Returns electric gear ratio between Master axis and Slave axis.}
var
  dpGearRatio: Double;
  lpSlaveAxis: LongInt;

begin
  { Since the ratio of master axis to slave axis is set to 0.5
  when in move, if the master moves by 1, the slave moves by 0.5.}
  AxmLinkSetMode(0, 1, 0.5);

  AxmLinkGetMode(0, @lpSlaveAxis, @dpGearRatio);
end;
```

See Also

AxmLinkSetMode, AxmLinkResetMode

AxmLinkResetMode

Purpose

Clears setting of electric gear ratio between Master axis and Slave axis.

Format

C++

```
DWORD AxmLinkResetMode(long lMasterAxisNo);
```

Visual Basic

```
Function AxmLinkResetMode(ByVal lMasterAxisNo As Long) As Long
```

Delphi

```
function AxmLinkResetMode(lMasterAxisNo : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lMasterAxisNo	Master channel (axis) number (start from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

It clears the ratio between master axis and slave axis such that when the master axis moves, the slave axis moves together.

Moving ratio is the ratio of a slave axis's move per one rotation of a master axis.

Example

C++

```
// Clear setting of electric gear ratio between Master axis and Slave axis.
AxmLinkResetMode(0);
```

Visual Basic

```
' Clear setting of electric gear ratio between Master axis and Slave axis.
AxmLinkResetMode 0
```

Delphi

```
{ Clear setting of electric gear ratio between Master axis and Slave axis.}
AxmLinkResetMode(0);
```

See Also

AxmLinkSetMode, AxmLinkGetMode

Interpolation Move API (Line, Circle)

In this chapter, we will discuss interpolation motion control. Interpolation means that more than two axes are associated with each other in order to execute the motions such as linear interpolation, circular interpolation and so on.

Concurrent control of multiple axes can be considered a same function as multi axis motion control. However, there is a big difference between them in that interpolation automatically controls the velocity of each axis related with interpolation move in order to follow the path that user requires.

For CAMC-IP, it supports 2 axes interpolation and 2 axes circular interpolation. For CAMC-QI, it can execute linear interpolation move from 2 axes to 4 axes, 2 axes circular interpolation move, 3 axes or 4 axes spline, and helical interpolation moving tasks automatically using continuous interpolation. Generally, linear interpolation, circular interpolation and helical interpolation can execute one interpolation, but spline interpolation cannot be executed without using continuous interpolation.

Besides, if universal output, triggers, interrupts supported in each node can be used during continuous interpolation, a better application equipment can be built by using velocity settings, node acceleration/deceleration and automatic acceleration/deceleration. This chapter describes how to use general interpolation for linear interpolation and circular interpolation and how to use continuous interpolation.

Function	Description
AxmLineMove	Multi axis linear interpolation moves by designating a start point and an end point. Exits API after move has started. Used in conjunction with AxmContiBeginNode, AxmContiEndNode, it specifies a start point and an end point on a specific coordination system and becomes a stored API in the linear interpolation queue. For linear profile continuous interpolation move, it is stored in the internal queue and is started by using AxmContiStart API.
AxmCircleCenterMove	Circular interpolation moves by designating a start point, a middle point and an end point. Exits API after move has started. Used in conjunction with AxmContiBeginNode, AxmContiEndNode, it specifies a start point, a middle point and an end point on a specific coordination system and becomes a stored API in the circular interpolation queue. For profile circular continuous interpolation move, it is stored in internal queue and is started by using AxmContiStart API.
AxmCirclePointMove	Circular interpolation moves by designating a middle point and an end point. Exits API after move has started. Used in conjunction with AxmContiBeginNode, AxmContiEndNode, it specifies a middle point and an end point on a specific coordination system and becomes a stored API in the circular interpolation queue. For profile circular continuous interpolation move, it is stored in internal queue and is started by using AxmContiStart API.
AxmCircleRadiusMove	Circular interpolation moves by designating a start point, an end point and a radius. Exits API after move has started. Used in conjunction with AxmContiBeginNode, AxmContiEndNode, it specifies a start point, an end point and a radius on a specific coordination system and becomes a stored API in the circular interpolation queue. For profile circular continuous interpolation move, it is stored in internal queue and is started by using AxmContiStart API.

AxmCircleAngleMove	Circular interpolation moves by designating a start point, a rotational angle and a radius. Exits API after move has started. Used in conjunction with AxmContiBeginNode, AxmContiEndNode, it specifies a start point, a rotational angle and a radius on a specific coordination system and becomes a stored API in the circular interpolation queue. For profile circular continuous interpolation move, it is stored in internal queue and is started by using AxmContiStart API.

The previously mentioned multi axis move was the multi axis move such that two axes simultaneously start and execute actions of each. However, there is an occasion such that two axes have to move according to specific paths of lines or circular arcs through co-related movements. In order to help such moves, linear interpolation, circular interpolation, continuous interpolation moves, etc. for each. Settings for velocity profile and absolute/relative coordinates of these interolation moves are designed to follow the axis that has the lowest axis number among those axes with interpolation move. Therefore, be careful in the settings.

▶Caution: Match all the axes that interpolate Unit/Pulse equally. If the Unit/Pulse are not matched with each other, all the units will be different. They must be identical to interpolation control.

▶Caution: When using a motion board with more than 4 axes motion (CAMC-IP)

In boards with more than four axes motion, two axes of Axis 0-1 are categorized as axis group 1, two axes of Axis 2-3 as axis group 2, four axes of Axis 4-5 as axis group 3, Axis 6-7 as axis group 4, and so on. Linear interpolation move and circular interpolation move among the axes in the same group are available to use, but those among the axes in different axis groups are not. It is because CAMC-IP chip is two axes and the chip cannot support these features. You must keep that in mind when setting coordinates.

▶Caution: When using a motion board with more than 8 axes motion (CAMC-QI)

In boards with more than eight axes motion, four axes are categorized as axis group 1, four axes of Axis 4-7 as axis group 2, four axes of Axis 8-11 as axis group 3, and so on. Linear interpolation move and circular interpolation move among the axes in the same group are available to use, but those among the axes in different axis groups are not. It is because CAMC-QI chip is four axes and the chip cannot support these features. You must keep that in mind when setting coordinates.

AxmLineMove

Purpose

API that multi axis linear interpolation moves by designating a start point and an end point on a coordination system.

Format

C++

```
DWORD AxmLineMove(long lCoord, double *dpPos, double dVel, double
dAccel, double dDecel);
```

Visual Basic

```
Function AxmLineMove(ByVal lCoord As Long, ByRef dpPos As Double,
ByVal dVel As Double, ByVal dAccel As Double, ByVal dDecel As Double)
As Long
```

Delphi

```
function AxmLineMove(lCoord : LongInt; dpPos : PDouble; dVel : Double;
dAccel : Double; dDecel : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordination system number (start from 0)
[>>]dpPos	Position array
[>>]dVel	Moving velocity (+direction: more than 0 , −direction: less than 0, The unit of moving velocity is PPS[Pulses/sec] when Unit/pulse is set to 1/1)
[>>]dAccel	Moving acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dDecel	Moving deceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

Description

It exits API after move has started. Used in conjunction with AxmContiBeginNode , AxmContiEndNode , it specifies a start point and an end point on a specific coordination system and becomes a stored API in the linear interpolation queue. For linear profile continuous interpolation move, it is stored in the internal queue and is started by using AxmContiStart API: for IP, two axes are available, and for QI,

two to four axes are available. Before executing this API, AxmContiSetAxisMap API does axis mapping on the axes on which interpolation tasks will be executed into map numbers. It is also applied to APIs that are related with general interpolation control, not only those related with continuous interpolation since parameter transmission is simple on them. Therefore, this API must be first used in order to map the axes that are to be used in prior to using APIs that are related with general interpolation control.

▶ By AxmSStop API and AxmEstop API, you can execute motion deceleration stop and motion emergency stop. Arguments must be stopped by inserting master axis of the Coordinate.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details. **

▶Caution: Match all the axes that interpolate Unit/Pulse equally. If the Unit/Pulse are not matched with each other, all the units will be different. They must be identical to interpolation control.

▶Caution: When using a motion board with more than 4 axes motion (CAMC-IP)

In boards with more than four axes motion, two axes of Axis 0-1 are categorized as axis group 1, two axes of Axis 2-3 as axis group 2, four axes of Axis 4-5 as axis group 3, Axis 6-7 as axis group 4, and so on. Linear interpolation move and circular interpolation move among the axes in the same group are available to use, but those among the axes in different axis groups are not. It is because CAMC-IP chip is two axes and the chip cannot support these features. You must keep that in mind when setting coordinates.

▶Caution: When using a motion board with more than 8 axes motion (CAMC-QI)

In boards with more than eight axes motion, four axes are categorized as axis group 1, four axes of Axis 4-7 as axis group 2, four axes of Axis 8-11 as axis group 3, and so on. Linear interpolation move and circular interpolation move among the axes in the same group are available to use, but those among the axes in different axis groups are not. It is because CAMC-QI chip is four axes and the chip cannot support these features. You must keep that in mind when setting coordinates.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate

	system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;

dPos[0] = 4000;
dPos[1] = 4000;
lAxis[0] = 0;
lAxis[1] = 1;

AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Relative position move.
AxmContiSetAbsRelMode(lCoordinate, 1);
// Linear interpolation move.
 AxmLineMove(lCoordinate, dPos, 200, 400, 400);
```

Visual Basic

```
Dim lCoordinate As Long
Dim Distance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim i As Long
lCoordinate = 0
 For i = 0 To 2
   axes(i) = i
 Next i
 Distance(0) = 4000
 Distance(1) = 4000
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
' Relative position move.
```

```
AxmContiSetAbsRelMode lCoordinate, 1
' Linear interpolation move.
 AxmLineMove lCoordinate, Distance(0), 200, 400, 400
```

Delphi

```
var
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dPos : array [0..1] of Double;
   lPosSize : LongInt;

begin
   lCoordinate := 0;
   lPosSize := 2;
   for i := 0 to 1 do
      begin
         lAxis[i] := i;
      end;
   dPos[0] := 4000;
   dPos[1] := 4000;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);

   { Relative position move. }
   AxmContiSetAbsRelMode(lCoordinate, 1);
   { Linear interpolation move. }
   AxmLineMove(lCoordinate, @dPos, 200, 400, 400);
end;
```

Usage examples of continuous interpolation move API >

It is the API that is stored in the internal queue for linear continuous interpolation move. This API is used in order to store interpolation commands in the internal queue when continuous interpolation move is used. This API is also used to store linear interpolation move while in continuous interpolation move. If 'AxmContiStart' API is executed after storing the interpolation motions to be executed in the internal queue, all the stored interpolation moves can be continuously executed.

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dVelocity = 200, dAccel = 400;

lAxis[0] = 0;
lAxis[1] = 1;

AxmContiWriteClear(lCoordinate);
```

```
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);

AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 800 , dPos[1] = 800;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 900 , dPos[1] = 900;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1000 , dPos[1] = 1000;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1100 , dPos[1] = 1100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
AxmContiWriteClear lCoordinate
```

```
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   i : LongInt;
   dVelocity, dAccel : Double;
   lAxis : array [0..1] of LongInt;
   dPos : array [0..1] of Double;
   lCoordinate : LongInt;
   lPosSize : LongInt;

begin
   dVelocity := 200;
   dAccel := 400;
   lPosSize := 2;
   lCoordinate := 0;
   { Register absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   for i := 0 to 1 do
   begin
      lAxis[i] := i;
   end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiBeginNode(lCoordinate);
   dPos[0] := 100 ; dPos[1] := 100;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
```

```
  dPos[0] := 200 ; dPos[1] := 200;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 300;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 500 ; dPos[1] := 500;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 600 ; dPos[1] := 600;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 700 ; dPos[1] := 700;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  { Start absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  { Start continuous interpolation move. }
  AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove, AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmCircleCenterMove

Purpose

Circular interpolation moves by designating a start point, a middle point and an end point on a specific coordination system. Exits API after move has started.

Format

C++

```
DWORD AxmCircleCenterMove(long lCoord, long *lAxisNo, double
*dCenterPos, double *dEndPos, double dVel, double dAccel, double
dDecel, DWORD uCWDir);
```

Visual Basic

```
Function AxmCircleCenterMove(ByVal lCoord As Long, ByRef lAxisNo As
Long, ByRef dCenterPos As Double, ByRef dEndPos As Double, ByVal dVel
As Double, ByVal dAccel As Double, ByVal dDecel As Double, ByVal
uCWDir As Long) As Long
```

Delphi

```
function AxmCircleCenterMove(lCoord : LongInt; lAxisNo : PLongInt;
dCenterPos : PDouble;  dEndPos : PDouble; dVel : Double; dAccel :
Double; dDecel : Double; uCWDir : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordination system number (start from 0)
[>>]lAxisNo	Channel (axis) number array (start from 0) to be used for interpolation
[>>]dCenterPos	Home position array on X axis, Y axis
[>>]dEndPos	Exit position array on X axis, Y axis
[>>]dVel	Moving velocity (+direction: more than 0 , -direction: less than 0, The unit of moving velocity is PPS[Pulses/sec] when Unit/pulse is set to 1/1)
[>>]dAccel	Moving acceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]dDecel	Moving deceleration (The unit of acceleration is PPS[Pulses/sec^2] when Unit/pulse is set to 1/1)
[>>]uCWDir	Circular direction : AXT_MOTION_MOVE_DIR_DEF - [00h] Counterclockwise - [01h] Clockwise

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board

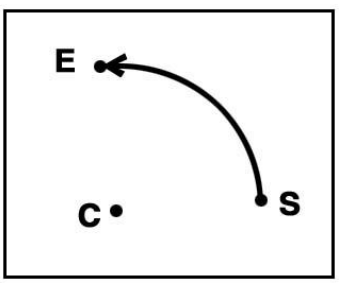
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

Description

Circular interpolation is applied to two arbitrary axes. We will explain it assuming that the two mapped axes are axis X and axis Y. Axis X is the axis that has a lower axis number and axis Y is the one that has a higher axis number. For example, if axes Z, U are the two mapped axes, then axis Z corresponds to axis X and axis U corresponds with axis Y.

Used in conjunction with AxmContiBeginNode, AxmContiEndNode , it specifies a start point, a middle point and an end point on a specific coordination system and becomes a stored API in the circular interpolation queue. For profile circular continuous interpolation move, it is stored in the internal queue and is started by using AxmContiStart API. Before executing this API, AxmContiSetAxisMap API does axis mapping on the axes on which interpolation tasks will be executed into map numbers. It is also applied to APIs that are related with general interpolation control, not only those related with continuous interpolation since parameter transmission is simple on them. Therefore, this API must be first used in order to map the axes that are to be used in prior to using APIs that are related with general interpolation control.

The current position becomes the start point of circular move, and it circular moves based on the preset middle point and the end point.



▶ By AxmSStop API and AxmEstop API, you can execute motion deceleration stop and motion emergency stop. Arguments must be stopped by inserting master axis of the Coordinate.

▶ You can verify whether it is in motion using AxmStatusReadInMotion API.

▶ When using AxmSignalSetInpos API, if Inpos input signal is set to Enable, it won't return until INP input becomes ON although command pulse out has been completed as it is regarded that motion has not been completed. Please refer to AxmSignalSetInpos API for details.

▶Caution: Match all the axes that interpolate Unit/Pulse equally. If the Unit/Pulse is not matched with each other, all the units will be different. They must be identical to interpolation control.

▶Caution: When using a motion board with more than 4 axes motion (SMC-2V03)

In boards with more than four axes motion, two axes of Axis 0-1 are categorized as axis group 1, two axes of Axis 2-3 as axis group 2, four axes of Axis 4-5 as axis group 3, Axis 6-7 as axis group 4, and so on. Linear interpolation move and circular interpolation move among the axes in the same group are available to use, but those among the axes in different axis groups are not. It is because SMC-2V03 is two axes and the board cannot support these features. You must keep that in mind when setting coordinates.

▶Caution: When using a motion board with more than 8 axes motion (PCI-N804/404)

In boards with more than eight axes motion, four axes are categorized as axis group 1, four axes of Axis 4-7 as axis group 2, four axes of Axis 8-11 as axis group 3, and so on. Linear interpolation move and circular interpolation move among the axes in the same group are available to use, but those among the axes in different axis groups are not. It is because PCI-N804/404 chip is four axes and the board cannot support these features. You must keep that in mind when setting coordinates.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

Examples of general interpolation >

C++

```
long   lAxis[2];
double dCenPos[2];
double dEndPos[2];
```

```
long   lPosSize = 2;
long lCoordinate = 0;
lAxis[0] = 0;
lAxis[1] = 1;
dCenPos[0] = 500;
dCenPos[1] = 500;
dEndPos[0] = 1000;
dEndPos[1] = 0;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);   // Relative position move.
// Circular interpolation move in CW direction.
AxmCircleCenterMove(lCoordinate, lAxis, dCenPos, dEndPos, 200, 400, 400, DIR_CW);
```

Visual Basic

```
Dim Distance(0 To 2) As Double
Dim CenDistance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim i As Long
Dim lCoordinate As Long
CenDistance(0) = 500
CenDistance(1) = 500
Distance(0) = 1000
Distance(1) = 0
lCoordinate = 0
For i = 0 To 2
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
' Circular interpolation move in CW direction.
AxmCircleCenterMove lCoordinate, axes(0), CenDistance(0), Distance(0), 200, 400, 400,
   DIR_CW
```

Delphi

```
var
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dCenPos : array [0..1] of Double;
   dEndPos : array [0..1] of Double;
   lPosSize : LongInt;

begin
   dCenPos[0] := 500;
   dCenPos[1] := 500;
   dEndPos[0] := 1000;
   dEndPos[1] := 0;
```

```
  lCoordinate := 0;
  lPosSize := 2;
  for i := 0 to 1 do
     begin
        lAxis[i] := i;
     end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
  AxmContiSetAbsRelMode(lCoordinate, 1);      { Relative position move. }
  AxmCircleCenterMove(lCoordinate, @lAxis, @dCenPos, @dEndPos, 200, 400, 400, DIR_CW);
end;
```

Examples of continuous interpolation >

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dCenPos[2];
double dEndPos[2];
double dVelocity = 200;
double dAccel = 400;
for(int i=0; i<lPosSize; i++)
{
  lAxis[i]    = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Relative position move.
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dCenPos[0] = 300 , dCenPos[1] = 300; dEndPos[0] = 400; dEndPos[1] = 400;
AxmCircleCenterMove(lCoordinate, lAxis, dCenPos, dEndPos, dVelocity, dAccel, dAccel,
  DIR_CCW);
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dCenPos[0] = 500 , dCenPos[1] = 500; dEndPos[0] = 600; dEndPos[1] = 600;
AxmCircleCenterMove(lCoordinate, lAxis, dCenPos, dEndPos, dVelocity, dAccel, dAccel,
  DIR_CCW);
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
```

```
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Relative position move.
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim dPos(0 To 1) As Double
Dim dCenPos(0 To 1) As Double
Dim lCoordinate As Long
Dim axes(0 To 1) As Long
Dim i As Long
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
dVelocity = 200
dAccel = 400
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Relative position move.
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
AxmCircleCenterMove lCoordinate, axes(0), dCenPos(0), dPos(0), dVelocity, dAccel,
  dAccel, DIR_CCW
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
AxmCircleCenterMove lCoordinate, axes(0), dCenPos(0), dPos(0), dVelocity, dAccel,
  dAccel, DIR_CCW
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  dCenPos : array [0..1] of Double;
  dEndPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;

begin
   dVelocity := 200;
   dAccel := 400;
   lPosSize := 2;
   lCoordinate := 0;
   for i := 0 to 1 do
      begin
         lAxis[i]     := i;
      end;
   { Relative position move. }
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmContiBeginNode(lCoordinate);
   dPos[0] := 100 ; dPos[1] := 100;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 200 ; dPos[1] := 200;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 300 ; dPos[1] := 300;
   dCenPos[0] := 300;  dCenPos[1] := 300;  dEndPos[0] := 400; dEndPos[1] := 400;
   AxmCircleCenterMove(lCoordinate, @lAxis, @dCenPos, @dEndPos, dVelocity, dAccel,
      dAccel, 0);
   dPos[0] := 400 ; dPos[1] := 400;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 500 ; dPos[1] := 500;
   dCenPos[0] := 500;  dCenPos[1] := 500;  dEndPos[0] := 600;  dEndPos[1] := 600;
   AxmCircleCenterMove(lCoordinate, @lAxis, @dCenPos, @dEndPos, dVelocity, dAccel,
      dAccel, 0);
   dPos[0] := 600 ; dPos[1] := 600;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 700 ; dPos[1] := 700;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   AxmContiEndNode(lCoordinate);
   { Relative position move. }
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmLineMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove, AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmCirclePointMove

Purpose

API for circular interpolation by specifying middle point and end point on the specified coordinate system. After the move starts, it exits the API.

Format

C++

```
DWORD AxmCirclePointMove(long lCoord, long *lAxisNo, double *dMidPos,
double *dEndPos, double dVel, double dAccel, double dDecel, long
lArcCircle);
```

Visual Basic

```
Function AxmCirclePointMove(ByVal lCoord As Long, ByRef lAxisNo As
Long, ByRef dMidPos As Double, ByRef dEndPos As Double, ByVal dVel As
Double, ByVal dAccel As Double, ByVal dDecel As Double, ByVal
lArcCircle As Long) As Long
```

Delphi

```
function AxmCirclePointMove(lCoord : LongInt; lAxisNo : PLongInt;
dMidPos : PDouble;  dEndPos : PDouble; dVel : Double; dAccel : Double;
dDecel : Double; lArcCircle: LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number(start from 0)
[>>]lAxisNo	Channel(axis) number(start from 0) array used for interpolation
[>>]dMidPos	Array of X middle point, and Y middle point
[>>]dEndPos	Array of X end point and Y end point
[>>]dVel	Move speed(+direction:more than 0, – direction: less than 0, move speed unit is PPS[Pulses/sec] when Unit/pulse is 1/1)
[>>]dAccel	Move accelaration(accelration unit is PPS[Pulses/sec^s] when Unit/pulse is 1/1)
[>>]dDecel	Move decelaration(decelaration unit is PPS[Pulses/sec^2] when Unit/pulse is 1/1)
[>>]lArcCircle	Arc(0), Cirlcle(1) selection value

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

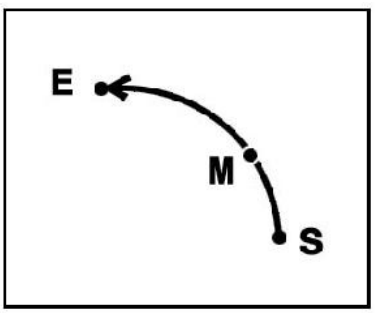
Description

Circular interpolation applies to any two axes. This explanation assumes two mapped axes as X and Y axis. X axis means lower axis number and Y axis means higher axis number in two mapped axes.

For example, when Z and U axis are two mapped axes, Z axis corresponds to X axis and U axis corresponds to Y axis.

Like AxmContiBeginNode and AxmContiEndNode , when it is being used, it becomes store API in queue which moves by specifying middle and end point in the specified coordinate system. For profile circular continuous interpolation move, it starts AxmContiStart API by saving it in the internal queue. Before executing this API, AxmContiSetAxisMap API does axis mapping which maps axes which is to execute interpolation process to axis number. Not only continuous interpolation, but also functions related to general interpolation control are applied because transmitting parameter is simple. Thus, before using APIs about general interpolation, axes for future use are mapped by this API.

Current position becomes the start point of circular move and it executes circular move going through middle point.



▶Caution: After axis mapping using AxmContiSetAxisMap, use from the lowest axis mapping. In case of circular interpolation, it is possible to move only when array it from the lowest axis.

▶ Caution: If signal of +LIMIT, -LIMIT is on, there's no move.

▶ By AxmSStop and AxmEstop API, motion deceleration stop and motion emergency stop are possible. Argument is always stopped by transmitting the first axis(the lowest actual axis number) of coordinate.

▶ AxmContiIsMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos API for further information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will be different with each other. Always set them equally, and do interpolation control.

▶Caution: When using motion board with more than 2 axes (SMC-2V03)

In the board with 8 axes by using SMC-2V03, two axes from Axis 0-1 become axis group 1, two from Axis 2- 3 become axis group 2, two from Axis 4-5 become axis group 3, and Axis6-7 become axis group 4. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with multi axes by using PCI-N804/404, 4 axes from Axis 0-3 become Axis group 1, 4 from Axis 4-7 become Axis group 2, and another 4 from Axis8-11 become Axis group 3. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

Normal interpolation move example>

C++

```
long   lAxis[2];
double dMidPos[2];
double dEndPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dVelocity = 200;
```

```
double dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
dMidPos[0] = 500;
dMidPos[1] = 500;
dEndPos[0] = 1000;
dEndPos[1] = 0;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmCirclePointMove(lCoordinate, lAxis, dMidPos, dEndPos, dVelocity, dAccel, dAccel, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim Distance(0 To 2) As Double
Dim MidDistance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim i As Long
Dim lCoordinate As Long
Distance(0) = 1000
Distance(1) = 0
MidDistance(0) = 500
MidDistance(1) = 500
dVelocity = 200
dAccel = 400
lCoordinate = 0
For i = 0 To 2
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmCirclePointMove lCoordinate, axes(0), MidDistance(0), Distance(0), dVelocity,
  dAccel, dAccel, 0
```

Delphi

```
var
  velocity, accel : Double;
  i : LongInt;
  lCoordinate : LongInt;
  lAxis : array [0..1] of Longint;
  dPos : array [0..1] of Double;
  dMidPos : array [0..1] of Double;
  dEndPos : array [0..1] of Double;
  lPosSize : LongInt;

begin
   dMidPos[0] := 500;
```

```
   dMidPos[1] := 500;
   dEndPos[0] := 1000;
   dEndPos[1] := 0;
   velocity := 200;
   accel := 400;
   lCoordinate := 0;
   lPosSize := 2;
   for i := 0 to 1 do
   begin
      lAxis[i]     := i;
   end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmCirclePointMove(lCoordinate, @lAxis, @dMidPos, @dEndPos, velocity, accel, accel,
      0);
end;
```

Example of circular interpolation move>

Start API of axis 2 circular interpolation move.

Current position becomes the start point of circular move and circular move is based on specified radius and end point.

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dMidPos[2];
double dEndPos[2];
double dVelocity = 200;
double dAccel = 400;
for(int i=0; i<lPosSize; i++)
{
  lAxis[i] = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dMidPos [0] = 300 , dMidPos [1] = 300; dEndPos[0] = 400; dEndPos[1] = 300;
AxmCirclePointMove(lCoordinate, lAxis, dMidPos, dEndPos, dVelocity, dAccel, dAccel,
  DIR_CCW);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
```

```
dMidPos[0] = 500 , dMidPos[1] = 500; dEndPos[0] = 600; dEndPos[1] = 500;
AxmCirclePointMove(lCoordinate, lAxis, dMidPos, dEndPos, dVelocity, dAccel, dAccel,
  DIR_CCW);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim dMidPos(0 To 1) As Double
Dim dEndPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
dVelocity = 200
dAccel = 400
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dMidPos(0) = 300 : dMidPos(1) = 300 : dEndPos(0) = 400 :dEndPos(1) = 300
AxmCirclePointMove lCoordinate, axes(0), dMidPos(0), dEndPos(0), dVelocity, dAccel,
  dAccel, DIR_CCW
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dMidPos(0) = 500: dMidPos(1) = 500 :dEndPos(0) = 600 :dEndPos(1) = 500
AxmCirclePointMove lCoordinate, axes(0), dMidPos(0), dEndPos(0), dVelocity, dAccel,
  dAccel, DIR_CCW
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
```

```
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  dMidPos : array [0..1] of Double;
  dEndPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;

begin
   dVelocity := 200;
   dAccel := 400;
   lPosSize := 2;
   lCoordinate := 0;
   AxmContiSetAbsRelMode(lCoordinate, 1);
   for i := 0 to 1 do
      begin
         lAxis[i] := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiBeginNode(lCoordinate);
   dPos[0] := 100 ; dPos[1] := 100;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 200 ; dPos[1] := 200;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 300 ; dPos[1] := 300;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dMidPos[0] := 300; dMidPos[1] := 300;  dEndPos[0] := 400; dEndPos[1] := 300;
   AxmCirclePointMove(lCoordinate, @lAxis, @dMidPos, @dEndPos, dVelocity, dAccel,
      dAccel, DIR_CCW);
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 400 ; dPos[1] := 400;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 500 ; dPos[1] := 500;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dMidPos[0] := 500; dMidPos[1] := 500;  dEndPos[0] := 600;  dEndPos[1] := 500;
   AxmCirclePointMove(lCoordinate, @lAxis, @dMidPos, @dEndPos, dVelocity, dAccel,
      dAccel, DIR_CCW);
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
```

```
    dPos[0] := 600 ; dPos[1] := 600;
    AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
    dPos[0] := 700 ; dPos[1] := 700;
    AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
    AxmContiEndNode(lCoordinate);
    AxmContiSetAbsRelMode(lCoordinate, 1);
    AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmLineMove, AxmCircleCenterMove, AxmCircleRadiusMove, AxmCircleAngleMove, AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmCircleRadiusMove

Purpose

To process circular interpolation move by specifying the start point, end point, and radius on the specified coordination. It exits the API after move starts.

Format

C++

```
DWORD AxmCircleRadiusMove(long lCoord, long *lAxisNo, double dRadius,
double *dEndPos, double dVel, double dAccel, double dDecel, DWORD
uCWDir, DWORD uShortDistance);
```

Visual Basic

```
Function AxmCircleRadiusMove(ByVal lCoord  As Long, ByRef lAxisNo As
Long, ByVal dRadius As Double, ByRef dEndPos As Double, ByVal dVel As
Double, ByVal dAccel As Double, ByVal dDecel As Double, ByVal uCWDir
As Long, ByVal uShortDistance As Long) As Long
```

Delphi

```
function AxmCircleRadiusMove(lCoord : LongInt; lAxisNo : PLongInt;
dRadius : Double; dEndPos : PDouble; dVel : Double; dAccel : Double;
dDecel : Double; uCWDir : DWord; uShortDistance : DWord) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number(0~ (max. axis number -1))
[>>]lAxisNo	Channe(axis)l number(0~ (max. axis number -1)) array used for interpolation
[>>]dRadius	Circular radius to move
[>>]dEndPos	Array of X axis end point and Y axis end point
[>>]dVel	Move speed (+direction:more than 0, - directoin: less than 0, move speed unit is PPS[Pulses/sec] when Unit/pulse is 1/1)
[>>]dAccel	Move accelaration (accelration unit is PPS[Pulses/sec^2] when Unit/pulse is 1/1)
[>>]dDecel	Move decelaration (decelaration unit is PPS[Pulses/sec^2] when Unit/pulse is 1/1)
[>>]uCWDir	Circular direction : AXT_MOTION_MOVE_DIR_DEF - [00h] Counterclockwise - [01h] Clockwise
[>>]uShortDistance	Set value of moving distance of circle to the middle point : AXT_MOTION_RADIUS_DISTANCE_DEF - [00h] Circular Move in short distance <a in the picture> - [01h] Circular Move in long distance <b in the picture>

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully

[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

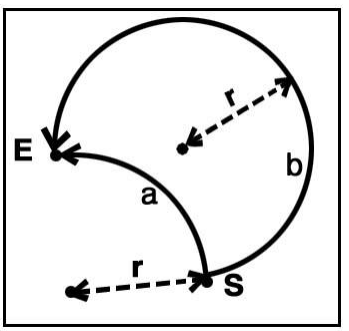
Description

Circular interpolation applies to any two axes. This explanation assumes two mapped axes are X axis and Y axis. X axis means lower axis number and Y axis means higher axis number in two mapped axes.

For example, when Z and U axis are the two mapped axes, Z axis corresponds to X axis and U axis corresponds to Y axis.

Like AxmContiBeginNode and AxmContiEndNode , when it is being used, it becomes store API in queue which moves by specifying middle and end point in the specified coordinate system. For profile circular continuous interpolation move, it starts AxmContiStart API by storing it in internal queue. Before executing this API, AxmContiSetAxisMap API does axis mapping which maps axes to execute interpolation process to axis number. Not only continuous interpolation, but also APIs related to general interpolation control are applied because transmitting parameter is simple. Thus, before using APIs about general interpolation, axes for future use are mapped by this API.

Current position becomes the start point of circular move and it executes circular move based on set radius and end point. If define circular with certain radius connection start point and end point, two center points are appeared; with this, two paths (a, b) can be defined even if it is same rotation direction.



▶Caution: After axis mapping using AxmContiSetAxisMap, use from the lowest axis mapping. In case of circular interpolation, it is possible to move only when array it from the lowest axis.

▶ Caution: If signal of +LIMIT, -LIMIT is on, there's no move.

▶ By AxmSStop and AxmEstop API, motion deceleration stop and motion emergency stop are possible. Argument is always stopped by transmitting the first axis(the lowest actual axis number) of coordinate.

▶ AxmContiIsMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos API for further information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will be different with each other. Always set them equally, and do interpolation control.

▶Caution: When using motion board with more than 2 axes (SMC-2V03)

In the board with 8 axes by using SMC-2V03, two axes from Axis 0-1 become axis group 1, two from Axis 2- 3 become axis group 2, two from Axis 4-5 become axis group 3, and Axis6-7 become axis group 4. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with multi axes by using PCI-N804/404, 4 axes from Axis 0-3 become Axis group 1, 4 from Axis 4-7 become Axis group 2, and another 4 from Axis8-11 become Axis group 3. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

Example of General Interpolation Move>

C++

```
long   lAxis[2];
double dEndPos[2];
DWORD  uShortDistance=0;
long   lPosSize = 2;
long   lCoordinate = 0;
double  dRadius = 500;
double dVelocity = 200;
double dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
dEndPos[0] = 1000;
dEndPos[1] = 0;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmCircleRadiusMove(lCoordinate, lAxis, dRadius, dEndPos, dVelocity, dAccel, dAccel, 1,
  uShortDistance);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim dRadius As Double
Dim Distance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim i As Long
Dim lCoordinate As Long
Dim uShortDistance As Long
uShortDistance = 0
Distance(0) = 1000
Distance(1) = 0
dRadius = 500
dVelocity = 200
dAccel = 400
lCoordinate = 0
For i = 0 To 2
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmCircleRadiusMove lCoordinate, axes(0), dRadius, Distance (0), dVelocity, dAccel,
  dAccel, 1, uShortDistance
```

Delphi

```
var
   xend, yend : Double;
  velocity, accel : Double;
   dRadius : Double;
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dPos : array [0..1] of Double;
   dEndPos : array [0..1] of Double;
   lPosSize : LongInt;
   uShortDistance : DWord;

begin
   dEndPos[0] := 1000;
   dEndPos[1] := 0;
   dRadius := 500;
   velocity := 200;
   accel := 400;
   lCoordinate := 0;
   lPosSize := 2;
   uShortDistance := 0;
   for i := 0 to 1 do
      begin
         lAxis[i]     := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmCircleRadiusMove(lCoordinate, @lAxis, dRadius, @dEndPos, velocity, accel, accel,
      1, uShortDistance);
end;
```

Example of Continuous Interpolation Move>

Start API of axis 2 circular interpolation move.

Current position becomes the start point of circular move and circular move is processed based on radius and end point.

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
DWORD   uShortDistance=0;
double dEndPos[2];
double dRadius = 150;
double dVelocity = 200;
double dAccel = 400;
```

```
for(int i=0; i<lPosSize; i++)
{
  lAxis[i] = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dEndPos[0] = 300; dEndPos[1] = 0;
AxmCircleRadiusMove(lCoordinate, lAxis, dRadius, dEndPos, dVelocity, dAccel, dAccel,
  DIR_CCW, uShortDistance);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dRadius = 250; dEndPos[0] = 500; dEndPos[1] = 0;
AxmCircleRadiusMove(lCoordinate, lAxis, dRadius, dEndPos, dVelocity, dAccel, dAccel,
  DIR_CCW, uShortDistance);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim dPos(0 To 1) As Double
Dim lCoordinate As Long
Dim axes(0 To 1) As Long
Dim uShortDistance As Long
Dim dRadius As Double
Dim i As Long
lPosSize = 2
lCoordinate = 0
uShortDistance = 0
For i = 0 To 1
  axes(i) = i
Next i
dVelocity = 200
dAccel = 400
dRadius = 150
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
```

```
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 300: dPos(1) = 0
AxmCircleRadiusMove lCoordinate, axes(0), dRadius, dPos(0), dVelocity, dAccel, dAccel,
  DIR_CCW, uShortDistance
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 300: dPos(1) = 0
AxmCircleRadiusMove lCoordinate, axes(0), dRadius, dPos(0), dVelocity, dAccel, dAccel,
  DIR_CCW, uShortDistance
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
  i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  uShortDistance : DWord;
  lCoordinate : LongInt;
  lPosSize : LongInt;
  dRadius : Double;

begin
  dVelocity := 200;
  dAccel := 400;
  lPosSize := 2;
  lCoordinate := 0;
  uShortDistance := 0;
  dRadius := 150;
   AxmContiSetAbsRelMode(lCoordinate, 1);
  for i := 0 to 1 do
     begin
        lAxis[i] := i;
     end;
```

```
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
  AxmContiBeginNode(lCoordinate);
  dPos[0] := 100 ; dPos[1] := 100;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 200 ; dPos[1] := 200;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 0;
  AxmCircleRadiusMove(lCoordinate, @lAxis, dRadius, @dPos, dVelocity, dAccel, dAccel,
     DIR_CCW, uShortDistance);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 0;
  AxmCircleRadiusMove(lCoordinate, @lAxis, dRadius, @dPos, dVelocity, dAccel, dAccel,
     DIR_CCW, uShortDistance);
  dPos[0] := 600 ; dPos[1] := 600;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 700 ; dPos[1] := 700;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleAngleMove, AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmCircleAngleMove

Purpose

To process circular interpolation move by specifying the center point and angle on the specified coordination. It exits the API after move starts.

Format

C++

```
DWORD AxmCircleAngleMove(long lCoord, long *lAxisNo, double
*dCenterPos, double dAngle, double dVel, double dAccel, double dDecel,
DWORD uCWDir);
```

Visual Basic

```
Function AxmCircleAngleMove(ByVal lCoord As Long, ByRef lAxisNo As
Long, ByRef dCenterPos As Double, ByVal dAngle As Double, ByVal dVel
As Double, ByVal dAccel As Double, ByVal dDecel As Double, ByVal
uCWDir As Long) As Long
```

Delphi

```
function AxmCircleAngleMove(lCoord : LongInt; lAxisNo : PLongInt;
dCenterPos: PDouble; dAngle : Double; dVel : Double; dAccel : Double;
dDecel : Double; uCWDir : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number(start from 0)
[>>]lAxisNo	Channel number(start from 0) array used for interpolation
[>>]dCenterPos	Home position array on X axis, Y axis
[>>]dAngle	Circular drive. Unit is ‘Degree(도)
[>>]dVel	Move speed(+direction:more than 0, – directoin: less than 0, move speed unit is PPS[Pulses/sec] when Unit/pulse is 1/1)
[>>]dAccel	Move accelaration (accelration unit is PPS[Pulses/sec^s] when Unit/pulse is 1/1)
[>>]dDecel	Move decelaration (decelaration unit is PPS[Pulses/sec^2] when Unit/pulse is 1/1)
[>>]uCWDir	Circular direction : AXT_MOTION_MOVE_DIR_DEF – [00h] Counterclockwise – [01h] Clockwise

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping

[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

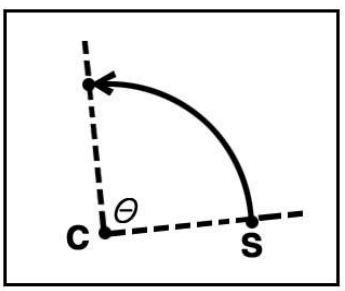
Description

Circular interpolation applies to any two axes. This explanation assumes two mapped axes are X axis and Y axis. X axis means lower axis number and Y axis means higher axis number in two mapped axes.

For example, when Z and U axis are the two mapped axes, Z axis corresponds to X axis and U axis corresponds to Y axis.

Like AxmContiBeginNode and AxmContiEndNode , when it is being used, it becomes store API in queue which moves by specifying middle and end point in the specified coordinate system. For profile circular continuous interpolation move, it starts AxmContiStart API by storing it in internal queue. Before executing this API, AxmContiSetAxisMap API does axis mapping which maps axes to execute interpolation process to axis number. Not only continuous interpolation, but also APIs related to general interpolation control are applied because transmitting parameter is simple. Thus, before using APIs about general interpolation, axes for future use are mapped by this API.

Current position becomes the start point of circular move and it executes circular move by the angle based on end point.



▶Caution: After axis mapping using AxmContiSetAxisMap, use from the lowest axis mapping. In case of circular interpolation, it is possible to move only when array it from the lowest axis.

▶ Caution: If signal of +LIMIT, -LIMIT is on, there's no move.

▶ By AxmSStop and AxmEstop API, motion deceleration stop and motion emergency stop are possible. Argument is always stopped by transmitting the first axis(the lowest actual axis number) of coordinate.

▶ AxmContiIsMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos API for further information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will be different with each other. Always set them equally, and do interpolation control.

▶Caution: When using motion board with more than 2 axes (SMC-2V03)

In the board with 8 axes by using SMC-2V03, two axes from Axis 0-1 become axis group 1, two from Axis 2- 3 become axis group 2, two from Axis 4-5 become axis group 3, and Axis6-7 become axis group 4. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with multi axes by using PCI-N804/404, 4 axes from Axis 0-3 become Axis group 1, 4 from Axis 4-7 become Axis group 2, and another 4 from Axis8-11 become Axis group 3. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

C++

```
long   lAxis[2];
double dCenPos[2];
double dAngle=90;
DWORD uCWDir=0;
double dVelocity = 200;
double dAccel = 400;
```

```
long   lPosSize = 2;
long lCoordinate = 0;
lAxis[0] = 0;
lAxis[1] = 1;
dCenPos[0] = 500;
dCenPos[1] = 500;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmCircleAngleMove(lCoordinate, lAxis, dCenPos, dAngle, dVelocity, dAccel, dAccel, 1);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim dAngle As Double
Dim CenDistance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim i As Long
Dim lCoordinate As Long
CenDistance(0) = 500
CenDistance(0) = 500
dAngle = 90
dVelocity = 200
dAccel = 400
lCoordinate = 0
For i = 0 To 2
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmCircleAngleMove lCoordinate, axes(0), CenDistance(0), dAngle, dVelocity, dAccel,
  dAccel, 1
```

Delphi

```
var
   i : LongInt;
  lCoordinate : LongInt;
  lAxis : array [0..1] of Longint;
   dAngle : Double;
  dCenPos : array [0..1] of Double;
  lPosSize : LongInt;
   dVelocity, dAccel : Double;

begin
   dAngle := 90;
   dVelocity := 200;
   dAccel := 400;
   lCoordinate := 0;
```

```
  lPosSize := 2;
  for i := 0 to 1 do
    begin
      lAxis[i] := i;
    end;
  dCenPos[0] := 500;
  dCenPos[1] := 500;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmCircleAngleMove(lCoordinate, @lAxis, @dCenPos, dAngle, dVelocity, dAccel, dAccel,
    1);
end;
```

Example of Continuous Interpolation Move>

Start APIof axis 2 circular interpolation move.

Current position becomes the start point of circular move and circular move is processed by the angle based on the end.

C++

```
long   lAxis[2];
long   lPosSize = 2;
long  lCoordinate = 0;
double dCenPos[2];
double dAngle = 180;
double dPos[2];
double dVelocity = 200;
double dAccel = 400;
for(int  i=0; i<lPosSize; i++)
{
  lAxis[i] = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dCenPos[0] = 300 , dCenPos[1] = 300;
AxmCircleAngleMove(lCoordinate, lAxis, dCenPos, dAngle, dVelocity, dAccel, dAccel,
  DIR_CCW);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dCenPos[0] = 500 , dCenPos[1] = 500;
AxmCircleAngleMove(lCoordinate, lAxis, dCenPos, dAngle, dVelocity, dAccel, dAccel,
  DIR_CCW);
dPos[0] = 600 , dPos[1] = 600;
```

```
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim dCenPos(0 To 1) As Double
Dim dAngle As Double
Dim axes(0 To 1) As Long
Dim i As Long
lPosSize = 2
lCoordinate = 0
dAngle = 180
For i = 0 To 1
  axes(i) = i
Next i
dVelocity = 200
dAccel = 400
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dCenPos(0) = 300 : dCenPos(1) = 300
AxmCircleAngleMove lCoordinate, axes(0), dCenPos(0), dAngle, dVelocity, dAccel, dAccel,
  DIR_CCW
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dCenPos(0) = 500 : dCenPos(1) = 500
AxmCircleAngleMove lCoordinate, axes(0), dCenPos(0), dAngle, dVelocity, dAccel, dAccel,
  DIR_CCW
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
```

```
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   i : LongInt;
   dVelocity, dAccel : Double;
   lAxis : array [0..1] of LongInt;
   dPos : array [0..1] of Double;
   dCenPos : array [0..1] of Double;
   dAngle : Double;
   lCoordinate : LongInt;
   lPosSize : LongInt;

begin
   dVelocity := 200;
   dAccel := 400;
   lPosSize := 2;
   lCoordinate := 0;
   dAngle := 180;
   for i := 0 to 1 do
      begin
         lAxis[i] := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmContiBeginNode(lCoordinate);
   dPos[0] := 100 ; dPos[1] := 100;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 200 ; dPos[1] := 200;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dCenPos [0] := 300;  dCenPos[1] := 300;
   AxmCircleAngleMove(lCoordinate, @lAxis, @dCenPos, dAngle, dVelocity, dAccel, dAccel,
      DIR_CCW);
   dPos[0] := 400 ; dPos[1] := 400;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dCenPos[0] := 500;  dCenPos[1] := 500;
   AxmCircleAngleMove(lCoordinate, @lAxis, @dCenPos, dAngle, dVelocity, dAccel, dAccel,
      DIR_CCW);
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 600 ; dPos[1] := 600;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 700 ; dPos[1] := 700;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 800 ; dPos[1] := 800;
   AxmContiEndNode(lCoordinate);
   AxmContiSetAbsRelMode(lCoordinate, 1);
```

```
  AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

Continuous Interpolation Setting And Move API

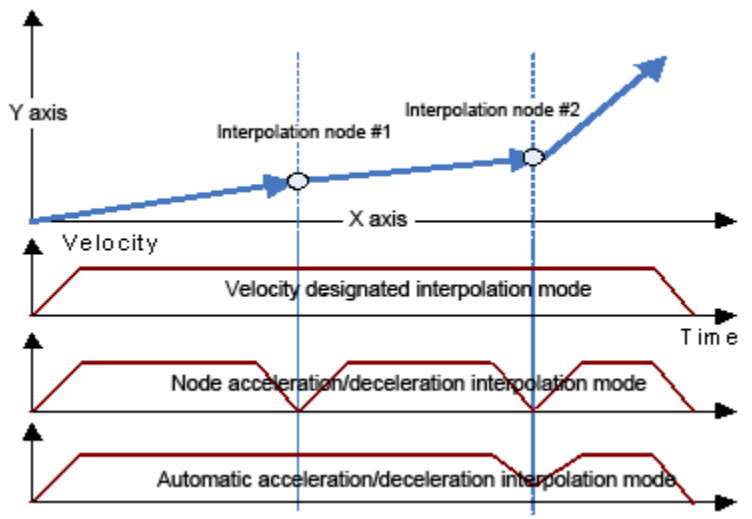
Continuous interpolation move API processes sequence of interpolation moves previously explained. It initiates and accelerates move, goes thorough specified positions in a steady speed, decelerates again at the end, and stops move.

One special thing in continuous interpolation API is that axes to be used in continuous interpolation go through mapping process to new coordinate system, commands of interpolation moves are sequentially saved in memory area called "continuous interpolation queue", when continuous interpolation initiation command is given, the stored continuous interpolation data comes out and enters into motion control chip, and interpolation moves are continuously processed. In addition, even after interpolation move is started, interpolation data can be entered into the motion control chip.

Function	Description
AxmContiSetAxisMap	Set continuous interpolation axis mapping for specified coordinate system.
AxmContiGetAxisMap	Return continuous interpolation axis mapping for specified coordinate system.
AxmContiSetAbsRelMode	Set continuous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continuous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continuous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode API are registered as continuous interpolation motion, but not processing real motion. When AxmContiStart API is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continuous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continuous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

To process continuous interpolation move, axes to be used in continuous interpolation should be mapped. Interpolation move API used in continuous interpolation is combination of axis 2 interpolation move, but when mapping axes, all axes to be used should be mapped in one coordinate system which includes all axes. For example, if you are using three axes 0,1,2 and want to process axis 0,1 interpolation move and then process axis 1,2 interpolation move, you must map axes 0,1,2 into one coordinate system. Once mapping to coordinate system is completed, you should set speed profile to be used in specified coordinate system, and select absolute/relative move.

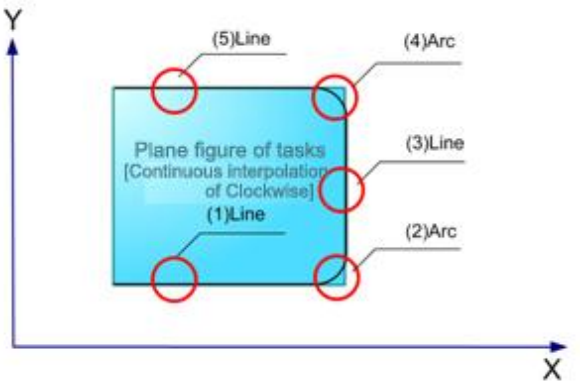
After entering data into interpolation queue, calling AxmContiStart API will start continuous interpolation move using stored data. Here, you can set speed profile mode during continuous interpolation move. There are speed specification interpolation mode and node ac/deceleration interpolation mode, and automatic ac/deceleration interpolation mode.



Speed specification interpolation mode means general continuous interpolation move in which move started, speed is accelerated, and when it reaches certain speed, it is maintained during continuous interpolation move. Node ac/deceleration interpolation mode means speed is ac/decelerated in every node. Auto ac/deceleration interpolation mode does not have speed change when it goes through continuous path, but it ac/decelerates automatically in emergency curve.

Description of continuous interpolation

List continuous interpolation motion used here is the function in which several steps of task are registered and processed collectively. Advantage of continuous interpolation motion is continuous execution of tasks without interruption of profile between tasks. Generally, in equipment, by inserting circular arc to the corner where two lines meet (such as dispensing, manufacturing, and painting), continuous control of line-> circular arc -> line -> circular arc -> line interpolation is possible.



Generally, you can use steady speed in list continuous interpolation motion, but you can also change profile by applying various speed to move nodes, which is one of the benefits of continuous interpolation move. Processing task continuously without interruption between tasks, you can change speed of tasking for certain point.

AxmContiSetAxisMap

Purpose

Set axis mapping in which axes to be used to process interpolation tasks to specified coordinate system.

Format

C++

```
DWORD AxmContiSetAxisMap(long lCoord, long lSize, long *lpRealAxisNo);
```

Visual Basic

```
Function AxmContiSetAxisMap(ByVal lCoord As Long, ByVal lSize As Long,
ByRef lpRealAxisNo As Long) As Long
```

Delphi

```
function AxmContiSetAxisMap(lCoord : LongInt; lSize : LongInt;
lpRealAxisNo : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[>>]lSize	Number of axes used in interpolation : AXT_MOTION_INTERPOLATION_AXIS_DEF – [2] When axis2 is used in interpolation – [3] When axis3 is used in interpolation – [4] When axis4 is used in interpolation
[>>]lpRealAxisNo	The axis array used in interpolation. IP : only 2 axes are possible, QI : set 2 – 4 axes Axis number to be set (0,1,2,3). For example, setting is possible within one chip, (0,1,2,3) setting possible (4,5,6,7) setting possible

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

Description

Not only continuous interpolation, but also APIs related to general interpolation control are applied because transmitting parameter is simple. Thus, before using APIs related to general interpolation, axes for future use are mapped by this API.

Caution: Always put smaller number first when axis mapping.

Here, the smallest axis becomes a master.

▶Caution: After axis mapping using AxmContiSetAxisMap API, map from the lowest axis. In case of circular interpolation, always put it to axis array from the lowest axis.

▶Caution: When using motion board with more than 2 axes (SMC-2V03)

In the board with 8 axes by using SMC-2V03, two axes from Axis 0-1 become axis group 1, two from Axis 2- 3 become axis group 2, two from Axis 4-5 become axis group 3, and Axis6-7 become axis group 4. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with multi axes by using PCI-N804/404, 4 axes from Axis 0-3 become Axis group 1, 4 from Axis 4-7 become Axis group 2, and another 4 from Axis8-11 become Axis group 3. Axes from the same group can process line interpolation move and circular interpolation move, but axes from different groups cannot. Keep in mind when specifying coordinate.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
dPos[0] = 4000;
dPos[1] = 4000;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Relative position move.
AxmContiSetAbsRelMode(lCoordinate, 1);
// Line interpolation move.
AxmLineMove(lCoordinate, dPos, 200, 400, 400);
long lAxis1, lAxis2;
AxmContiGetAxisMap(lCoordinate, & lPosSize, lAxis);
// You can identify the number of axes using lPosSize.
lAxis1 = lAxis[0]; lAxis2 = lAxis[1]; // Bring mapping axis number.
```

Visual Basic

```
Dim lCoordinate As Long
Dim Distance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim lAxis1 As Long
Dim lAxis2 As Long
Dim i As Long
lCoordinate = 0
 For i = 0 To 2
   axes(i) = i
 Next i
 Distance(0) = 4000
 Distance(1) = 4000
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
```

```
' Relative position move.
AxmContiSetAbsRelMode lCoordinate, 1
' Line interpolation move.
AxmLineMove lCoordinate, Distance(0), 200, 400, 400
AxmContiGetAxisMap lCoordinate, 2, axes (0)
' You can identify the number of axes using lPosSize.
lAxis1 =axes (0): lAxis2 = axes (1) ' Bring mapping axis number.
```

Delphi

```
var
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dPos : array [0..1] of Double;
   lPosSize : LongInt;
   lAxis1 : LongInt;
   lAxis2 : LongInt;

begin
   lCoordinate := 0;
   lPosSize := 2;
   for i := 0 to 1 do
      begin
          lAxis[i] := i;
      end;
   dPos[0] := 4000;
   dPos[1] := 4000;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
   { Relative position move. }
   AxmContiSetAbsRelMode(lCoordinate, 1);
   { Line interpolation move. }
   AxmLineMove(lCoordinate, @dPos, 200, 400, 400);
   AxmContiGetAxisMap(lCoordinate, @lPos, lAxis);
   { You can identify the number of axes using lPosSize. }
   lAxis1 := lAxis[0]; lAxis2 := lAxis[1]; { Bring mapping axis number. }
end;
```

See Also

AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiGetAxisMap

Purpose

Return axes to be used in interpolation process with axis mapping to user specified coordinate system.

Format

C++

```
DWORD AxmContiGetAxisMap(long lCoord, long *lpSize, long
*lpRealAxesNo);
```

Visual Basic

```
Function AxmContiGetAxisMap(ByVal lCoord As Long, ByRef lpSize As
Long, ByRef lpRealAxisNo As Long) As Long
```

Delphi

```
function AxmContiGetAxisMap(lCoord : LongInt; lpSize : PLongInt;
lpRealAxisNo : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[<<]lpSize	Number of axes used in interpolation (2 – 4) : AXT_MOTION_INTERPOLATION_AXIS_DEF – [2] When axis2 is used in interpolation – [3] When axis3 is used in interpolation – [4] When axis4 is used in interpolation
[<<]lpRealAxisNo	The axis array used in interpolation IP: only 2 axes are possible QI:set 2 – 4 axes. Axis number to be set(0,1,2,3). For example, setting is possible within one chip, (0,1,2,3) setting possible (4,5,6,7) setting possible

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

Return axes to process interpolation tasks set by 'AxmContiSetAxisMap' with mapping to specified coordinate system.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
```

```
long lCoordinate = 0;
dPos[0] = 4000;
dPos[1] = 4000;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Relative position move.
AxmContiSetAbsRelMode(lCoordinate, 1);
// Line interpolation move.
 AxmLineMove(lCoordinate, dPos, 200, 400, 400);
long lAxis1, lAxis2;
AxmContiGetAxisMap(lCoordinate, & lPosSize, lAxis);
// You can verify the number of axes by lPosSize.
lAxis1 = lAxis[0]; lAxis2 = lAxis[1]; // Bring the mapping axis number.
```

Visual Basic

```
Dim lCoordinate As Long
Dim Distance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim lAxis1 As Long
Dim lAxis2 As Long
Dim i As Long
lCoordinate = 0
 For i = 0 To 2
   axes(i) = i
 Next i
 Distance(0) = 4000
 Distance(1) = 4000
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
' Relative position move.
AxmContiSetAbsRelMode lCoordinate, 1
' Line interpolation move.
AxmLineMove lCoordinate, Distance(0), 200, 400, 400
AxmContiGetAxisMap lCoordinate, 2, axes (0)
' You can verify the number of axes by lPosSize.
lAxis1 =axes (0): lAxis2 = axes (1) ' Bring the mapping axis number.
```

Delphi

```
var
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dPos : array [0..1] of Double;
   lPosSize : LongInt;
   lAxis1 : LongInt;
   lAxis2 : LongInt;
```

```
begin
   lCoordinate := 0;
   lPosSize := 2;
   for i := 0 to 1 do
      begin
         lAxis[i] := i;
      end;
   dPos[0] := 4000;
   dPos[1] := 4000;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
   { Relative position move. }
   AxmContiSetAbsRelMode(lCoordinate, 1);
   { Line interpolation move. }
   AxmLineMove(lCoordinate, @dPos, 200, 400, 400);
   AxmContiGetAxisMap(lCoordinate, @lPos, lAxis);
   { You can verify the number of axes by lPosSize. }
   lAxis1 := lAxis[0]; lAxis2 := lAxis[1]; { Bring the mapping axis number. }
end;
```

See Also

AxmContiSetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiSetAbsRelMode

Purpose

Set moving distance calculation mode of specified coordinate system.

Format

C++

```
DWORD AxmContiSetAbsRelMode(long lCoord, DWORD uAbsRelMode);
```

Visual Basic

```
Function AxmContiSetAbsRelMode(ByVal lCoord As Long, ByVal
uAbsRelMode As Long) As Long
```

Delphi

```
function AxmContiSetAbsRelMode(lCoord : LongInt; uAbsRelMode :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number(start from 0)
[>>]uAbsRelMode	[00h] Moving distance calculation mode : AXT_MOTION_ABSREL_MODE_DEF – [00h] Position move absolute mode – [01h] Position move relative mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

Set the same moving distance calculation mode to all axes in coordinate system.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
dPos[0] = 4000;
dPos[1] = 4000;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Relative position move.
```

```
AxmContiSetAbsRelMode(lCoordinate, 1);
// Line interpolation move.
 AxmLineMove(lCoordinate, dPos, 200, 400, 400);
// Verify moving distance mode of axis 0.
DWORD Mode;
AxmContiGetAbsRelMode(lCoordinate, &Mode);
```

Visual Basic

```
Dim lCoordinate As Long
Dim Distance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim lAxis1 As Long
Dim lAxis2 As Long
Dim i As Long
lCoordinate = 0
For i = 0 To 2
  axes(i) = i
Next i
Distance(0) = 4000
Distance(1) = 4000
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
' Relative position move.
AxmContiSetAbsRelMode lCoordinate, 1
' Line interpolation move.
AxmLineMove lCoordinate, Distance(0), 200, 400, 400
' Verify moving distance mode of axis 0.
Dim Mode As Long
AxmContiGetAbsRelMode lCoordinate, Mode
```

Delphi

```
var
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dPos : array [0..1] of Double;
   lPosSize : LongInt;
   Mode : LongInt;

begin
   lCoordinate := 0;
   lPosSize := 2;
   for i := 0 to 1 do
      begin
         lAxis[i] := i;
      end;
   dPos[0] := 4000;
   dPos[1] := 4000;
   AxmContiWriteClear(lCoordinate);
```

```
  AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
  { Relative position move. }
  AxmContiSetAbsRelMode(lCoordinate, 1);
  { Line interpolation move. }
  AxmLineMove(lCoordinate, @dPos, 200, 400, 400);
  { Verify moving distance mode of axis 0. }
  AxmContiGetAbsRelMode(lCoordinate, @Mode);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart,AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiGetAbsRelMode

Purpose

Return moving distance calculation mode of specified coordinate system.

Format

C++

```
DWORD AxmContiGetAbsRelMode(long lCoord, DWORD *upAbsRelMode);
```

Visual Basic

```
Function AxmContiGetAbsRelMode(ByVal lCoord As Long, ByRef
upAbsRelMode As Long) As Long
```

Delphi

```
function AxmContiGetAbsRelMode(lCoord : LongInt; upAbsRelMode :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[<<]upAbsRelMode	[00h] Moving distance calculation mode : AXT_MOTION_ABSREL_MODE_DEF – [00h] Position move absolute mode – [01h] Position move relative mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

Return moving distance calculation mode of specified coordinate system set by 'AxmContiSetAbsRelMode'.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
dPos[0] = 4000;
dPos[1] = 4000;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
```

```
// Relative position move.
AxmContiSetAbsRelMode(lCoordinate, 1);
// Line interpolation move.
 AxmLineMove(lCoordinate, dPos, 200, 400, 400);
// Verify moving distance mode of axis 0.
DWORD Mode;
AxmContiGetAbsRelMode (lCoordinate, &Mode);
```

Visual Basic

```
Dim lCoordinate As Long
Dim Distance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim lAxis1 As Long
Dim lAxis2 As Long
Dim i As Long
lCoordinate = 0
For i = 0 To 2
  axes(i) = i
Next i
Distance(0) = 4000
Distance(1) = 4000
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
' Relative position move.
AxmContiSetAbsRelMode lCoordinate, 1
' Line interpolation move.
AxmLineMove lCoordinate, Distance(0), 200, 400, 400
' Verify moving distance mode of axis 0.
Dim Mode As Long
AxmContiGetAbsRelMode lCoordinate, Mode
```

Delphi

```
var
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dPos : array [0..1] of Double;
   lPosSize : LongInt;
   Mode : LongInt;

begin
   lCoordinate := 0;
   lPosSize := 2;
   for i := 0 to 1 do
      begin
          lAxis[i] := i;
   end;
   dPos[0] := 4000;
   dPos[1] := 4000;
```

```
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
  { Relative position move. }
  AxmContiSetAbsRelMode(lCoordinate, 1);
  { Line interpolation move. }
  AxmLineMove(lCoordinate, @dPos, 200, 400, 400);
  { Verify moving distance mode of axis 0. }
  AxmContiGetAbsRelMode (lCoordinate, @Mode);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiBeginNode

Purpose

To start registering tasks to be used in continuous interpolation in specified coordinate system.

Format

C++

```
DWORD AxmContiBeginNode(long lCoord);
```

Visual Basic

```
Function AxmContiBeginNode(ByVal lCoord As Long) As Long
```

Delphi

```
function AxmContiBeginNode(lCoord: LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

After this API is called, all motion processing is not an actual execution but registration as continuous interpolation motion until AxmContiEndNode API is called. When AxmContiStart API is called, registered motion will actually be started.

▶ AxmContiWriteClear Use this API to delete the registered tasks from the memory. After all continuous interpolation tasks are done, use this API for clearing memory.

▶ AxmContiIsMotion Use this API to verify whether continuous interpolation tasks are in use or not.

▶ AxmContiGetTotalNodeNum This API returns total number of registered nodes in the memory.

▶ AxmContiGetNodeNum This API verifies the index number of currently moving continuous interpolation.

▶ AxmContiReadFree This API checks whether memory is empty or not.

▶ AxmContiStart This API starts continuous interpolation task.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dVelocity = 200, dAccel = 400;
lAxis[0] = 0; lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 800 , dPos[1] = 800;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 900 , dPos[1] = 900;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1000 , dPos[1] = 1000;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1100 , dPos[1] = 1100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
```

```
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
  i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;

begin
  dVelocity := 200;
  dAccel := 400;
  lPosSize := 2;
  lCoordinate := 0;
  { Register absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  for i := 0 to 1 do
     begin
        lAxis[i] := i;
```

```
    end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
  AxmContiBeginNode(lCoordinate);
  dPos[0] := 100 ; dPos[1] := 100;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 200 ; dPos[1] := 200;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 300;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 500 ; dPos[1] := 500;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 600 ; dPos[1] := 600;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 700 ; dPos[1] := 700;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  { Start absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  { Start continuous interpolation move. }
  AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiEndNode, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiEndNode

Purpose

To end registering tasks to process continuous interpolation in specified coordinate system.

Format

C++

```
DWORD AxmContiEndNode(long lCoord);
```

Visual Basic

```
Function AxmContiEndNode(ByVal lCoord As Long) As Long
```

Delphi

```
function AxmContiEndNode(lCoord: LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

After this API is called, all motion processing is not actually executed but registered in internal queue until API is called. When AxmContiStart API is called, registered motion will actually be started.

▶ AxmContiWriteClear Use this API to delete the registered tasks from the memory. After all continuous interpolation tasks are done, use this function for clearing memory.

▶ AxmContiIsMotion Use this API to check whether continuous interpolation tasks are in use or not.

▶ AxmContiGetTotalNodeNum This API returns total number of registered nodes in the memory.

▶ AxmContiGetNodeNum This API verifies the index number of currently moving continuous interpolation.

▶ AxmContiReadFree This API checks whether memory is empty

▶ AxmContiStart This API starts continuous interpolation task.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
```

```
double dVelocity = 200, dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 800 , dPos[1] = 800;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 900 , dPos[1] = 900;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1000 , dPos[1] = 1000;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1100 , dPos[1] = 1100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
```

```
  axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;

begin
   dVelocity := 200;
   dAccel := 400;
   lPosSize := 2;
   lCoordinate := 0;
   { Register absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   for i := 0 to 1 do
      begin
         lAxis[i] := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
```

```
  AxmContiBeginNode(lCoordinate);
  dPos[0] := 100 ; dPos[1] := 100;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 200 ; dPos[1] := 200;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 300;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 500 ; dPos[1] := 500;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 600 ; dPos[1] := 600;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 700 ; dPos[1] := 700;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  { Start absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  { Start continuous interpolation move. }
  AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiReadFree

Purpose

Verify internal queue if it is empty for interpolation move.

Format

C++

```
DWORD AxmContiReadFree(long lCoord, DWORD *upQueueFree);
```

Visual Basic

```
Function AxmContiReadFree(ByVal lCoord As Long, ByRef upQueueFree As
Long) As Long
```

Delphi

```
function AxmContiReadFree(lCoord: LongInt; upQueueFree : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[<<]upQueueFree	Status of internal queue – [00h] Status of internal queue – [01h] Internal queue is empty

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

When using AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove API with AxmContiBeginNode and AxmContiEndNode , it stores tasks to queue and processes continuous interpolation to process interpolation move.

If this internal queue is empty, it says TRUE, or else it says FALSE.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dVelocity = 200, dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
```

```
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 800 , dPos[1] = 800;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 900 , dPos[1] = 900;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1000 , dPos[1] = 1000;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1100 , dPos[1] = 1100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
// Identify if internal Queue is empty for interpolation move.
DWORD ReadInterpolationQueueFree;
AxmContiReadFree (lCoordinate, &ReadInterpolationQueueFree);
if(ReadInterpolationQueueFree)  printf("Internal Queue is empty.");
else       printf("Internal Queue is not empty.");
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
lPosSize = 2
```

```
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
' Identify if internal Queue is empty for interpolation move.
Dim ReadInterpolationQueueFree As Long
AxmContiReadFree lCoordinate, ReadInterpolationQueueFree
If ReadInterpolationQueueFree Then
  MsgBox "Internal Queue is empty.", vbOKCancel
Else
  MsgBox "Internal Queue is not empty.", vbOKCancel
End If
```

Delphi

```
var
  i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;
  ReadInterpolationQueueFree : DWORD;

begin
  dVelocity := 200;       dAccel := 400;
```

```
   lPosSize := 2;
   lCoordinate := 0;
   { Register absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   for i := 0 to 1 do
      begin
          lAxis[i] := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiBeginNode(lCoordinate);
   dPos[0] := 100 ; dPos[1] := 100;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 200 ; dPos[1] := 200;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 300 ; dPos[1] := 300;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 400 ; dPos[1] := 400;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 500 ; dPos[1] := 500;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 600 ; dPos[1] := 600;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 700 ; dPos[1] := 700;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   AxmContiEndNode(lCoordinate);
   { Start absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   { Start continuous interpolation move. }
   AxmContiStart(lCoordinate, 0, 0);
   { Identify if internal Queue is empty for interpolation move. }
   AxmContiReadFree (lCoordinate, @ReadInterpolationQueueFree);
   if ReadInterpolationQueueFree = 1 then
      Application.MessageBox('Internal Queue is empty.', 'Ajinextek', MB_OK)
   else
      Application.MessageBox('Internal Queue is not empty.', 'Ajinextek', MB_OK);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiReadIndex

Purpose

Identify the number of interpolation moves stored in internal queue for processing interpolation move.

Format

C++

```
DWORD AxmContiReadIndex(long lCoord, long *lpQueueIndex);
```

Visual Basic

```
Function AxmContiReadIndex(ByVal lCoord As Long, ByRef lpQueueIndex
As Long) As Long
```

Delphi

```
function AxmContiReadIndex(lCoord: LongInt; lpQueueIndex : PLongInt) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[<<]lpQueueIndex	The number of stored interpolation queue

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

When using AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove API with AxmContiBeginNode and AxmContiEndNode , it stores tasks to queue and processes continuous interpolation to process interpolation move.

Use this API to identify the number of internal queue. One AxmLineMove API is regarded as one unit.

Example

C++

```
long ReadInterpolMotionQueueIndex;
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dVelocity = 200, dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
```

```
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 600 , dPos[1] = 600;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 700 , dPos[1] = 700;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 800 , dPos[1] = 800;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 900 , dPos[1] = 900;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1000 , dPos[1] = 1000;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 1100 , dPos[1] = 1100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
// Identify the number of interpolation move stored in internal Queue for interpolation
// move.
if(AxmContiReadIndex(lCoordinate, &ReadInterpolMotionQueueIndex) == AXT_RT_SUCCESS){
  printf(" %ld of continuous interpolation move inside.. \n",
     ReadInterpolMotionQueueIndex);
}
```

Visual Basic

```
Dim ReadInterpolMotionQueueIndex As Long
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim strData As String
Dim i As Long
dVelocity = 200
```

```
dAccel = 400
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 600: dPos(1) = 600
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 700: dPos(1) = 700
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
' Identify the number of interpolation move stored in internal Queue for interpolation
' move.
If(AxmContiReadIndex (lCoordinate, ReadInterpolMotionQueueIndex) = AXT_RT_SUCCESS) Then
  strData   = “continuous interpolation queues” + CStr(ReadInterpolMotionQueueIndex) +
“inside.”
  MsgBox strData
End If
```

Delphi

```
var
  i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;
  ReadInterpolationQueueFree : DWORD;
  ReadInterpolMotionQueueIndex,ReadInterpolMotionQueueIndex2 : longInt;
  strData : String;
```

```
begin
   dVelocity := 200;    dAccel := 400;       lPosSize := 2; lCoordinate := 0;
   { Register absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   for i := 0 to 1 do
      begin
          lAxis[i] := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiBeginNode(lCoordinate);
   dPos[0] := 100 ; dPos[1] := 100;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 200 ; dPos[1] := 200;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 300 ; dPos[1] := 300;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 400 ; dPos[1] := 400;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 500 ; dPos[1] := 500;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 600 ; dPos[1] := 600;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 700 ; dPos[1] := 700;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   AxmContiEndNode(lCoordinate);
   { Start absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   { Start continuous interpolation move. }
   AxmContiStart(lCoordinate, 0, 0);
   { Identify the number of interpolation move stored in internal Queue for
interpolation move. }
   if (AxmContiReadIndex (lCoordinate, @ReadInterpolMotionQueueIndex) = AXT_RT_SUCCESS)  t
      begin
         strData := ' continuous interpolation queue ' +
            IntToStr(ReadInterpolMotionQueueIndex) + 'inside.';
         Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
      end;
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiWriteClear

Purpose

To delete all internal queues stored for continuous interpolation move.

Format

C++

```
DWORD AxmContiWriteClear(long lCoord);
```

Visual Basic

```
Function AxmContiWriteClear(ByVal lCoord As Long) As Long
```

Delphi

```
function AxmContiWriteClear(lCoord: LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number(start from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

When using AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove API with AxmContiBeginNode and AxmContiEndNode , it stores tasks to queue and processes continuous interpolation to process interpolation move. One AxmLineMove API is regarded as one unit.

Use this API for deleting all the number of queue stored for continuous interpolation move.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
dPos[0] = 4000;
dPos[1] = 4000;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Relative position move.
```

```
AxmContiSetAbsRelMode(lCoordinate, 1);
// Line interpolation move.
 AxmLineMove(lCoordinate, dPos, 200, 400, 400);
```

Visual Basic

```
Dim lCoordinate As Long
Dim Distance(0 To 2) As Double
Dim axes(0 To 2) As Long
Dim i As Long
lCoordinate = 0
For i = 0 To 2
   axes(i) = i
Next i
Distance(0) = 4000
Distance(1) = 4000
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 2, axes(0)
' Relative position move.
AxmContiSetAbsRelMode lCoordinate, 1
' Line interpolation move.
AxmLineMove lCoordinate, Distance(0), 200, 400, 400
```

Delphi

```
var
   i : LongInt;
   lCoordinate : LongInt;
   lAxis : array [0..1] of Longint;
   dPos : array [0..1] of Double;
   lPosSize : LongInt;

begin
   lCoordinate := 0;
   lPosSize := 2;
   for i := 0 to 1 do
      begin
          lAxis[i] := i;
      end;
   dPos[0] := 4000;
   dPos[1] := 4000;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, lPosSize, @lAxis);
   { Relative position move. }
   AxmContiSetAbsRelMode(lCoordinate, 1);
   { Line interpolation move. }
   AxmLineMove(lCoordinate, @dPos, 200, 400, 400);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiStart

Purpose

To start the move of registered internal continuous interpolation queue.

Format

C++

```
DWORD AxmContiStart(long lCoord, DWORD dwProfileset, long lAngle);
```

Visual Basic

```
Function AxmContiStart(ByVal lCoord As Long, ByVal dwProfileset As
Long, ByVal lAngle As Long) As Long
```

Delphi

```
function AxmContiStart(lCoord : LongInt; dwProfileset : DWord; lAngle:
LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[>>]dwProfileset	Profile mode – [00h] Speed specified interpolation mode – [01h] Node ac/decelaration interpolation mode – [02h] Automatic ac/decelaration interpolation mode
[>>]lAngle	Angle when using automatic ac/decelation mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully

[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping

[4169] AXT_RT_ERROR_MOVE_SENSOR_CHECK : The sensor (Alarm, EMG, Limit, etc.) is detected error

* See error code Table for more information on status error codes

Description

When using AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove API with AxmContiBeginNode and AxmContiEndNode , it stores tasks to queue and processes continuous interpolation to process interpolation move. One AxmLineMove API is regarded as one unit.

Use this API to run internal queue stored for continuous interpolation move.

▶ By AxmSStop and AxmEstop API, motion deceleration stop and motion emergency stop are possible. It always stops by transmitting master axis of coordinate.

List continuous interpolation motion used here is the function in which several steps of task are registered and processed collectively. Advantage of continuous interpolation motion is continuous execution of tasks without interruption of profile between tasks. Generally in equipment, by inserting

circular arc to the corner where two lines meet (such as dispensing, manufacturing and painting), continuous control of line–> circular arc –> line –> circular arc –> line interpolation is possible.

Example

C++

```
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dVelocity = 200, dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
```

```
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;

begin
   dVelocity := 200;
   dAccel := 400;
   lPosSize := 2;
   lCoordinate := 0;
   { Register absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   for i := 0 to 1 do
      begin
          lAxis[i] := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiBeginNode(lCoordinate);
   dPos[0] := 100 ; dPos[1] := 100;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 200 ; dPos[1] := 200;
   AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
   dPos[0] := 300 ; dPos[1] := 300;
```

```
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 500 ; dPos[1] := 500;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  { Start absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  { Start continuous interpolation move. }
  AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode,, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiIsMotion

Purpose

To identify if continuous interpolation is in move or not.

Format

C++

```
DWORD AxmContiIsMotion(long lCoord, DWORD *upInMotion);
```

Visual Basic

```
Function AxmContiIsMotion(ByVal lCoord As Long, ByRef upInMotion As
Long) As Long
```

Delphi

```
function AxmContiIsMotion(lCoord: LongInt; upInMotion : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[<<]upInMotion	Identify if continuous interpolation is in move – [00h] Continuous interpolation is not in move – [01h] Continuous interpolation is in move

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

When using AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove API with AxmContiBeginNode and AxmContiEndNode , it stores tasks to queue and processes continuous interpolation to process interpolation move. One AxmLineMove API is regarded as one unit.

Use this API to identify whether coordinate system is in move when it starts continuous interpolation while internal queue stored for continuous interpolation move is running continuous interpolation.

Example

C++

```
DWORD IsContMotion;
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
```

```
double dVelocity = 200, dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
Sleep(100);
AxmContiIsMotion (lCoordinate, &IsContMotion);
if(IsContMotion ){ // Identify whether in move or not.
  MessageBox("List continuous interpolation move is in motion now\n");
}
```

Visual Basic

```
Public Declare Sub Sleep Lib "kernel32" Alias "Sleep" (ByVal dwMilliseconds As Long)
' In basic, header file should be declared to use Sleep API.
Dim IsContMotion As Long
Dim strData As String
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
AxmContiWriteClear lCoordinate
```

```
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
Sleep 100
AxmContiIsMotion lCoordinate, IsContMotion
If(IsContMotion = 1) Then
  MsgBox "List continuous interpolation move is in motion now.", vbOKCancel
End If
```

Delphi

```
var
   i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;
   IsContMotion : DWORD;

begin
   dVelocity := 200;
   dAccel := 400;
   lPosSize := 2;
   lCoordinate := 0;
   { Register absolute position move. }
   AxmContiSetAbsRelMode(lCoordinate, 0);
   for i := 0 to 1 do
      begin
          lAxis[i] := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
   AxmContiBeginNode(lCoordinate);
```

```
  dPos[0] := 100 ; dPos[1] := 100;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 200 ; dPos[1] := 200;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 300;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 500 ; dPos[1] := 500;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  { Start absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  { Start continuous interpolation move. }
  AxmContiStart(lCoordinate, 0, 0);
  Sleep(100);
  AxmContiIsMotion (lCoordinate, @IsContMotion);
  if(IsContMotion = 1 ) then  { Identify if interpolation is in motion or not. }
  begin
     Application.MessageBox('List continuous interpolation move is in motion now.',
        'Ajinextek', MB_OK);
  end ;
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmContiGetNodeNum

Purpose

To identify index number of current running continuous interpolation while continuous interpolation move.

Format

C++

```
DWORD AxmContiGetNodeNum(long lCoord, long *lpNodeNum);
```

Visual Basic

```
Function AxmContiGetNodeNum(ByVal lCoord As Long, ByRef lpNodeNum As
Long) As Long
```

Delphi

```
function AxmContiGetNodeNum(lCoord: LongInt; lpNodeNum : PlongInt) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number (start from 0)
[<<]lpNodeNum	Index number of continuous interpolation

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

When using AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove API with AxmContiBeginNode and AxmContiEndNode , it stores tasks to queue and processes continuous interpolation to process interpolation move. One AxmLineMove API is regarded as one unit.

During continuous interpolation move of internal queue stored for continuous interpolation move, it identifies index number of currently running continuous interpolation when it starts continuous interpolation move. Note that there are always 7 units in internal hardware queue in order to move.

Example

C++

```
long  lpNodeNum1;
DWORD IsContMotion;
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
```

```
double dVelocity = 200, dAccel = 400;
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
Sleep(100);
AxmContiIsMotion (lCoordinate, &IsContMotion);
if(IsContMotion ){ // Identify interpolation is in motion or not.
  AxmContiGetNodeNum (0, &lpNodeNum1);
  printf("\t Currently operating list interpolation move number is %ld\n",
     lpNodeNum1 );
}
```

Visual Basic

```
Public Declare Sub Sleep Lib "kernel32" Alias "Sleep" (ByVal dwMilliseconds As Long)
' In basic, header file should be declared to use Sleep function.
Dim IsContMotion, lpNodeNum1 As Long
Dim strData As String
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
```

```
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
Sleep 100
AxmContiIsMotion lCoordinate, IsContMotion
If(IsContMotion = 1) Then
  AxmContiGetNodeNum 0, lpNodeNum1
  strData  = "Currently operating list interpolation move number" + CStr(lpNodeNum1)
     + " is.."
  MsgBox strData
End If
```

Delphi

```
var
  i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;
  IsContMotion : DWORD;
  lpNodeNum1 : longInt;
  strData : String;

begin
  dVelocity := 200;
  dAccel := 400;
  lPosSize := 2;
  lCoordinate := 0;
  { Register absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
```

```
  for i := 0 to 1 do
     begin
         lAxis[i] := i;
     end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
  AxmContiBeginNode(lCoordinate);
  dPos[0] := 100 ; dPos[1] := 100;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 200 ; dPos[1] := 200;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 300;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 500 ; dPos[1] := 500;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  { Start absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  { Start continuous interpolation move. }
  AxmContiStart(lCoordinate, 0, 0);
  Sleep(100);
  AxmContiIsMotion (lCoordinate, @IsContMotion);
  if(IsContMotion = 1 ) then  { Identify if interpolation is in motion or not. }
     begin
        AxmContiGetNodeNum (0, @lpNodeNum1);
        strData     := 'Currently operating interpolation move number' +
           IntToStr(lpNodeNum1) + 'is..';
        Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
     end ;
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetTotalNodeNum

AxmContiGetTotalNodeNum

Purpose

To identify total number of index of specified continuous interpolation move.

Format

C++

```
DWORD AxmContiGetTotalNodeNum(long lCoord, long *lpNodeNum);
```

Visual Basic

```
Function AxmContiGetTotalNodeNum(ByVal lCoord As Long, ByRef
lpNodeNum As Long) As Long
```

Delphi

```
function AxmContiGetTotalNodeNum(lCoord: LongInt; lpNodeNum :
PlongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordinate system number(start from 0)
[<<]lpNodeNum	Total number of continuous interpolation index

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
* See error code Table for more information on status error codes

Description

When using AxmLineMove, AxmCircleCenterMove, AxmCirclePointMove, AxmCircleRadiusMove, AxmCircleAngleMove API with AxmContiBeginNode and AxmContiEndNode , it stores tasks to queue and processes continuous interpolation to process interpolation move. One AxmLineMove API is regarded as one unit.

This API identifies the total number of nodes of continuous interpolation move specified here.

Example

C++

```
long lpTotalNodeNum1;
long  lpNodeNum1;
DWORD IsContMotion;
long   lAxis[2];
double dPos[2];
long   lPosSize = 2;
long lCoordinate = 0;
double dVelocity = 200, dAccel = 400;
```

```
lAxis[0] = 0;
lAxis[1] = 1;
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, lPosSize, lAxis);
// Register absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiBeginNode(lCoordinate);
dPos[0] = 100 , dPos[1] = 100;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 200 , dPos[1] = 200;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 300 , dPos[1] = 300;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 400 , dPos[1] = 400;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
dPos[0] = 500 , dPos[1] = 500;
AxmLineMove(lCoordinate, dPos, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
// Start absolute position move.
AxmContiSetAbsRelMode(lCoordinate, 0);
// Start continuous interpolation move.
AxmContiStart(lCoordinate, 0, 0);
Sleep(100);
AxmContiIsMotion (lCoordinate, &IsContMotion);
if(IsContMotion ){     // Identify if interpolation is in motion or not.
  AxmContiGetTotalNodeNum (0, &lpTotalNodeNum1);
  printf("\t Total number of currently operating list interpolation move is %ld \n",
     lpTotalNodeNum1);
}
```

Visual Basic

```
Public Declare Sub Sleep Lib "kernel32" Alias "Sleep" (ByVal dwMilliseconds As Long)
' In basic, header file should be declared to use Sleep function.
Dim IsContMotion, lpTotalNodeNum1 As Long
Dim strData As String
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPos(0 To 1) As Double
Dim axes(0 To 1) As Long
Dim i As Long
dVelocity = 200
dAccel = 400
lPosSize = 2
lCoordinate = 0
For i = 0 To 1
  axes(i) = i
Next i
```

```
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, lPosSize, axes(0)
' Register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiBeginNode lCoordinate
dPos(0) = 100: dPos(1) = 100
AxmLineMove lCoordinate, dPos(0), dVelocity, dAccel, dAccel
dPos(0) = 200: dPos(1) = 200
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 300: dPos(1) = 300
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 400: dPos(1) = 400
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
dPos(0) = 500: dPos(1) = 500
AxmLineMove lCoordinate, dPos(0), dVelocity , dAccel, dAccel
AxmContiEndNode lCoordinate
' Start register absolute position move.
AxmContiSetAbsRelMode lCoordinate, 0
' Start continuous interpolation move.
AxmContiStart lCoordinate, 0, 0
Sleep 100
AxmContiIsMotion lCoordinate, IsContMotion
If(IsContMotion = 1) Then
  AxmContiGetTotalNodeNum 0, lpTotalNodeNum1
  strData  = "Total number of currently operating list interpolation move" +
     CStr(lpTotalNodeNum1) + " is.."
  MsgBox strData
End If
```

Delphi

```
var
  i : LongInt;
  dVelocity, dAccel : Double;
  lAxis : array [0..1] of LongInt;
  dPos : array [0..1] of Double;
  lCoordinate : LongInt;
  lPosSize : LongInt;
  IsContMotion : DWORD;
  lpTotalNodeNum1: longInt;
  strData : String;

begin
  dVelocity := 200;
  dAccel := 400;
  lPosSize := 2;
  lCoordinate := 0;
  { Register absolute position move. }
  AxmContiSetAbsRelMode(lCoordinate, 0);
  for i := 0 to 1 do
```

```
    begin
        lAxis[i] := i;
    end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 2, @lAxis);
  AxmContiBeginNode(lCoordinate);
  dPos[0] := 100 ; dPos[1] := 100;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 200 ; dPos[1] := 200;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 300 ; dPos[1] := 300;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 400 ; dPos[1] := 400;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  dPos[0] := 500 ; dPos[1] := 500;
  AxmLineMove(lCoordinate, @dPos, dVelocity, dAccel, dAccel);
  AxmContiEndNode(lCoordinate);
  { Start absolute position move. }
  AxmMotSetAbsRelMode(lCoordinate, 0);
  { Start continuous interpolation move. }
  AxmContiStart(lCoordinate, 0, 0);
  Sleep(100);
  AxmContiIsMotion (lCoordinate, @IsContMotion);
  if(IsContMotion = 1 ) then    { Identify if interpolation is in motion or not. }
    begin
      AxmContiGetTotalNodeNum (0, @ lpTotalNodeNum1);
      strData     := 'Total number of currently operating list interpolation move' +
         IntToStr(lpTotalNodeNum1) + 'is..';
      Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
  end ;
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum

Trigger API

Function	Description
AxmTriggerSetTimeLevel	Set trigger signal duration, trigger output level, and trigger output method in specified axis.
AxmTriggerGetTimeLevel	Return trigger signal duration, trigger output level, and trigger output method in specified axis.
AxmTriggerSetAbsPeriod	Set whether trigger function will be used or not, output level, position comparater, trigger signal duration, and triger output mode in specified axis.
AxmTriggerGetAbsPeriod	Return whether trigger function will be used or not, output level, position comparater, trigger signal duration, and triger output mode in specified axis.
AxmTriggerSetBlock	Output trigger for regular section from start point to endpoint which user set.
AxmTriggerGetBlock	Read trigger setting of 'AxmTriggerSetBlock' API.
AxmTriggerOneShot	User outputs one trigger pulse.
AxmTriggerSetTimerOneshot	User outputs one trigger pulse after few seconds.
AxmTriggerOnlyAbs	Output absolute position trigger infinite absolute position.
AxmTriggerSetReset	Reset trigger setting.

During motion move, AXM can send trigger signal of 5V level as well as interrupt for certain situation. Trigger signal is from signal pin, and when its setting is ActiveHIGH, it maintains 0V for normal time, 5V for trigger signal, and back to 0V. For ActiveLow it works in opposite way. Of course, it maintains 24V for mechanical part. Before making trigger signal, you may set things like trigger signal level and duration by AxmTriggerSetTimeLevel API and you may call functions making trigger for certain condition. Trigger occurring APIs include AxmTriggerSetAbsPeriod / AxmTriggerOnlyAbs API which make trigger for certain point, AxmTriggerSetBlock API which makes trigger regularly during certain period, and AxmTriggerOneShot / AxmTriggerSetTimerOneshot API which make triggers together at any time.

Note : Trigger feature is only available in PCI-N804/404 and RTEX-PM.

AxmTriggerSetTimeLevel

Purpose

To set trigger output time, trigger level, trigger output way, and whether using interrupt setting or not when trigger output for specific axis.

Format

C++

```
DWORD AxmTriggerSetTimeLevel(long lAxisNo, double dTrigTime, DWORD
uTriggerLevel, DWORD uSelect, DWORD uInterrupt);
```

Visual Basic

```
Function AxmTriggerSetTimeLevel(ByVal lAxisNo As Long, ByVal
dTrigTime As Double, ByVal uTriggerLevel As Long, ByVal uSelect As
Long ,ByVal uInterrupt As Long) As Long
```

Delphi

```
function AxmTriggerSetTimeLevel(lAxisNo : LongInt; dTrigTime : Double;
uTriggerLevel : Dword; uSelect: Dword ; uInterrupt: Dword) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[>>]dTrigTime	Trigger output signal time 1 – 50000 (50msec)
[>>]uTriggerLevel	[00h]Set whether using trigger output signal or not and Active Level : AXT_MOTION_LEVEL_MODE_DEF – [00h] B Contact(NORMAL CLOSE) Use Active Low Level Trigger Signal – [01h] A Contact(NORMAL OPEN) Use Active High Level Trigger signal – [02h] Trigger signal not in use – [03h] Trigger signal in use(maintain current status)
[>>]uSelect	[00h]Selects trigger standard position input source : AXT_MOTION_SELECTION_DEF – [00h] Command position – [01h] Actual position
[>>]uInterrupt	[00h]Whether to output interrupt when output trigger : AXT_USE – [00h] interrupt not in use – [01h] interrupt in use

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

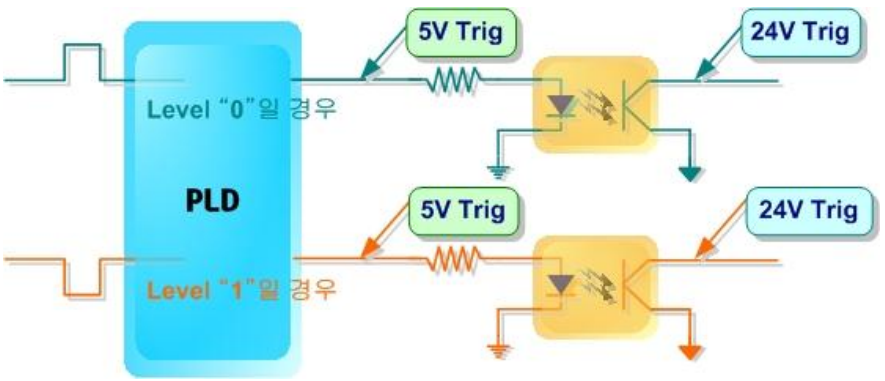
Description

Low(B contact) : normal Close –> detected Open

High(A contact): normal Open -> detected Close

Falling Edge : maintains high status for normal time, low status for trigger period, and then goes back to high status.

Riging Edge : maintains Low status for normal time, high status for trigger period, and then goes back to low status.



※ Note : In the case of 5V trigger output, it maintains 0V for normal time and 5V if trigger occurs when active level is set to 0. In the case of 24V trigger, it becomes N24V for trigger output in Open status. Thus, when using trigger with 24V with terminal strip, you need to put pull-up resistance of 2kΩ to identify trigger output.

dTrigTime : trigger output time

SMC-2V03: 1usec - maximum 4msec (set between 1 - 4000)

PCI-N804/404: 1usec - maximum 50msec (set between 1 - 50000)

Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
double dTrigTime;
DWORD dwTriggerLevel;
DWORD dwSelect;
DWORD dwInterrupt;
// Return trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerGetTimeLevel(0, &dTrigTime, &dwTriggerLevel, &dwSelect, & dwInterrupt);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
Dim dTrigTime As Double
Dim dwTriggerLevel, dwSelect, dwInterrupt As Long
' Return trigger output time, level, output method, and output interrupt of trigger
```

```
' function of axis 0.
AxmTriggerGetTimeLevel 0, dTrigTime, dwTriggerLevel, dwSelect, dwInterrupt
```

Delphi

```
var
  dwTriggerLevel, dwSelect, dwInterrupt : Dword;
  dTrigTime : Double;

begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Return trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerGetTimeLevel(0, @dTrigTime, @dwTriggerLevel, @dwSelect, @dwInterrupt);
end;
```

See Also

AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerGetTimeLevel

Purpose

Return trigger output time, trigger level, trigger output method, and whether to set output interrupt or not when outputting trigger.

Format

C++

```
DWORD AxmTriggerGetTimeLevel(long lAxisNo, double *dpTrigTime, DWORD *upTriggerLevel, DWORD *upSelect, DWORD *upInterrupt);
```

Visual Basic

```
Function AxmTriggerGetTimeLevel(ByVal lAxisNo As Long, ByRef dpTrigTime As Double, ByRef upTriggerLevel As Long, ByRef upSelect As Long , ByRef upInterrupt As Long) As Long
```

Delphi

```
function AxmTriggerGetTimeLevel(lAxisNo : LongInt; dpTrigTime : Pdouble; upTriggerLevel : PDWord; upSelect : PDWord; upInterrupt : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[<<]dpTrigTime	Trigger output signal time 1 – 50000 (50msec)
[<<]upTriggerLevel	[00h]Set whether using trigger output signal and Active Level : AXT_MOTION_LEVEL_MODE_DEF – [00h] B Contact(NORMAL CLOSE) Use Active Low Level Trigger Signal – [01h] A Contact(NORMAL OPEN) Use Active High Level Trigger signal – [02h] Trigger signal not in use – [03h] Trigger signal in use (maintain current status)
[<<]upSelect	[00h]Selects trigger standard position input source : AXT_MOTION_SELECTION_DEF – [00h] Command position – [01h] Actual position
[<<]upInterrupt	[00h]Whether to output interrupt or not when outputting trigger : AXT_USE – [00h] interrupt not in use – [01h] interrupt in use

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

cf : If you want to bring only necessary data, you can put NULL for unnecessary variable.

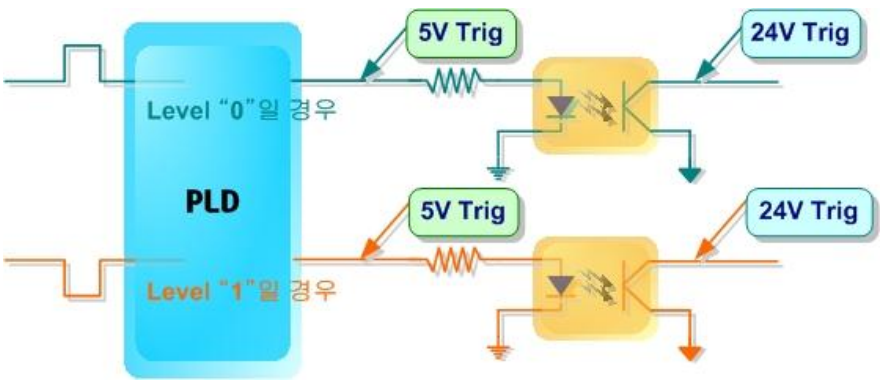
It returns trigger output time, trigger level, trigger output method, and whether to set interrupt or not when outputting trigger specified by 'AxmTriggerSetTimeLevel '.

Low(B contact) : normal Close -> detected Open

High(A contact): normal Open -> detected Close

Falling Edge : maintains high status for normal time, low status for trigger period, and then goes back to high status.

Riging Edge : maintains low status for normal time, high status for trigger period, and then goes back to low status.



※ Note : In the case of 5V trigger output, it maintains 0V for normal time and 5V when trigger occurs if active level is set to 0. In the case of 24V trigger, it becomes N24V for trigger output in Open status. Thus, when using trigger with 24V with terminal strip, you need to put pull-up resistance of 2kΩ to identify trigger output.

Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
double dTrigTime;
DWORD dwTriggerLevel;
DWORD dwSelect;
DWORD dwInterrupt;
// Return trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerGetTimeLevel(0, &dTrigTime, &dwTriggerLevel, &dwSelect, & dwInterrupt);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
Dim dTrigTime As Double
Dim dwTriggerLevel, dwSelect, dwInterrupt As Long
' Return trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
```

```
AxmTriggerGetTimeLevel 0, dTrigTime, dwTriggerLevel, dwSelect, dwInterrupt
```

Delphi

```
var
  wTriggerLevel, dwSelect, dwInterrupt : Dword;
  TrigTime : Double;

begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Return trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerGetTimeLevel(0, @dTrigTime, @dwTriggerLevel, @dwSelect, @dwInterrupt);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerSetTimerOneshot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerSetAbsPeriod

Purpose

Set trigger function for specific axis to period, absolute position mode.

Format

C++

```
DWORD AxmTriggerSetAbsPeriod(long lAxisNo, DWORD uMethod, double
dPos);
```

Visual Basic

```
Function AxmTriggerSetAbsPeriod(ByVal lAxisNo As Long, ByVal uMethod
As Long, ByVal dPos As Double) As Long
```

Delphi

```
function AxmTriggerSetAbsPeriod(lAxisNo : LongInt; uMethod : Dword;
dPos : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[>>]uMethod	[00h]Trigger output mode : AXT_MOTION_TRIGGER_MODE_DEF – [00h] Period trigger method using trigger position value – [01h] Trigger occurrence in trigger absolute position
[>>]dPos	Position value to output trigger signal (When using period: trigger outputs in each position)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

'AxmTriggerSetTimeLevel ' trigger function's trigger output time, level, output method, and whether to set interrupt for output or not are specified. You always have to use with this API. It will be in different status like following according to trigger output level.

Caution: Always set to UNIT/PULSE when setting trigger position.

If the position is set less than UNIT/PULSE, it can not be outputted on that position since min. unit is set to UNIT/PULSE.

Caution: unlike AxmTriggerSetBlock API, trigger output is based on the change in counter from the beginning point. When you set trigger mode to period mode, trigger specified will be printed.

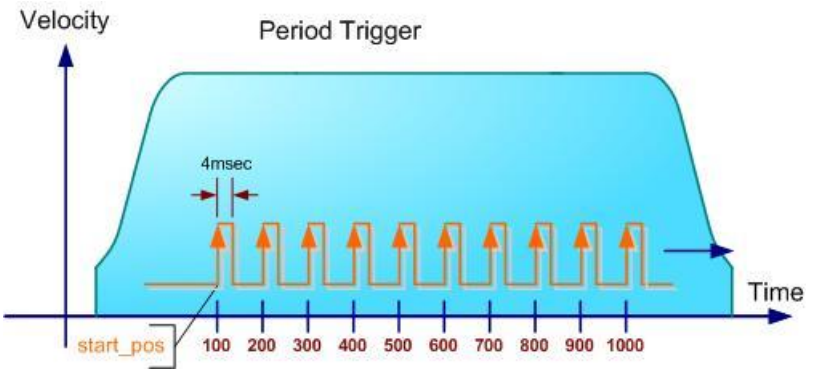
Low(B Contact) : Normal Close -> detected Open

High(A Contact) : Normal Open -> detected Close

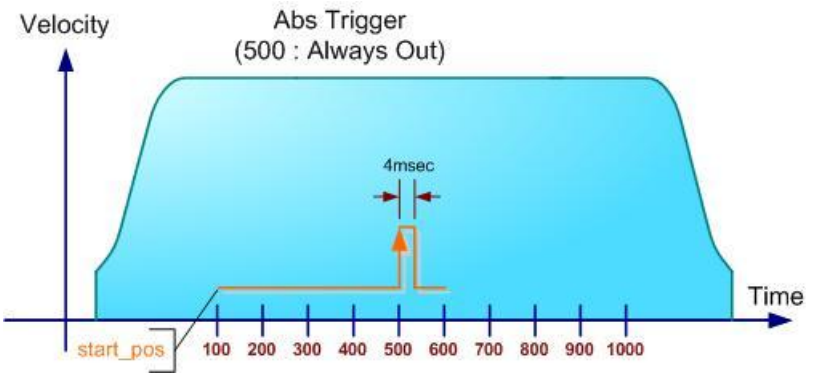
Falling Edge : maintains high status for normal time, low status for trigger period, and then goes back to high status.

Riging Edge : maintains low status for normal time, high status for trigger period, and then goes back to low status.

Period mode : if Position is 100, trigger output will be generated every 100 pulse.



Absolute mode : if Position is 500, it generates trigger output at the position of 500.



Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
// Set trigger function of axis 0 to period.
AxmTriggerSetAbsPeriod(0, PERIOD_MODE, 100);
// Identify whether using trigger function of axis 0 or not.
double uPos;
DWORD uMode;
AxmTriggerGetAbsPeriod(0, &uMode, &uPos);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
' Set trigger function of axis 0 to period.
AxmTriggerSetAbsPeriod 0, PERIOD_MODE, 100
' Identify whether using trigger function of axis 0 or not.
Dim uMode As Long
Dim uPos As Double
AxmTriggerGetAbsPeriod 0, uMode, uPos
```

Delphi

```
{ Identify whether using trigger function of axis 0 or not. }
var
  uMode: Dword;
  uPos: Double;

begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Set trigger function of axis 0 to period. }
  AxmTriggerSetAbsPeriod(0, PERIOD_MODE, 100);
  AxmTriggerGetAbsPeriod(0, @uMode, @uPos);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerSetTimerOneshot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerGetAbsPeriod

Purpose

Return period, absolute position mode of trigger function for specific axis.

Format

C++

```
DWORD AxmTriggerGetAbsPeriod(long lAxisNo, DWORD *upMethod, double
*dpPos);
```

Visual Basic

```
Function AxmTriggerGetAbsPeriod(ByVal lAxisNo As Long, ByRef upMethod
As Long, ByRef dpPos As Double) As Long
```

Delphi

```
function AxmTriggerGetAbsPeriod(lAxisNo : LongInt; upMethod : PDWord;
dpPos : Pdouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[<<]upMethod	[00h]Trigger output mode : AXT_MOTION_TRIGGER_MODE_DEF – [00h] Period trigger method using trigger position value – [01h] Trigger occurrence in trigger absolute position
[<<]dpPos	Position value to output trigger signal (When using period: trigger outputs in each position)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

cf : If you only want to bring necessary data, you can put NULL for unnecessary variable.

Return trigger functions in period, absolute position mode to specific axis set by 'AxmTriggerSetAbsPeriod ' API.

Caution: In SMC-2V03, when calling AxmTriiggerSetBlock API, ABS_POS_MODE is used in internal library, so return value for this API will be 1.

Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
```

```
// Set trigger function of axis 0 to period.
AxmTriggerSetAbsPeriod(0, PERIOD_MODE, 100);
// Identify whether using trigger function of axis 0 or not.
double uPos;
DWORD uMode;

AxmTriggerGetAbsPeriod(0, &uMode, &uPos);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
' Set trigger function of axis 0 to period.
AxmTriggerSetAbsPeriod 0, PERIOD_MODE, 100
' Identify whether using trigger function of axis 0 or not.
Dim uMode As Long
Dim uPos As Double
AxmTriggerGetAbsPeriod 0, uMode, uPos
```

Delphi

```
{ Identify whether using trigger function of axis 0 or not. }
var
  uMode: Dword;
  uPos: Double;

begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Set trigger function of axis 0 to period. }
  AxmTriggerSetAbsPeriod(0, PERIOD_MODE, 100);
  AxmTriggerGetAbsPeriod(0, @uMode, @uPos);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerSetTimerOneshot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerSetBlock

Purpose

Set trigger output for regular intervals between starting point and end point which user specified.

Format

C++

```
DWORD AxmTriggerSetBlock(long lAxisNo, double dStartPos, double
dEndPos, double dPeriodPos);
```

Visual Basic

```
Function AxmTriggerSetBlock(ByVal lAxisNo As Long, ByVal dStartPos As
Double, ByVal dEndPos As Double, ByVal dPeriodPos As Double) As Long
```

Delphi

```
function AxmTriggerSetBlock(lAxisNo : LongInt; dStartPos : Double;
dEndPos : Double; dPeriodPos : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[>>]dStartPos	Starting position of trigger signal
[>>]dEndPos	Ending position of trigger signal
[>>]dPeriodPos	Set trigger output period position

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

'AxmTriggerSetTimeLevel ' trigger function's trigger output time, level, output method, and whether to set interrupt for output or not are specified. You always have to use with this API. It will be in different status like following according to trigger output level.

Caution: Always set to UNIT/PULSE when setting trigger position.

If position is set less than UNIT/PULSE, it can not be outputted on that position since min. unit is set to UNIT/PULSE.

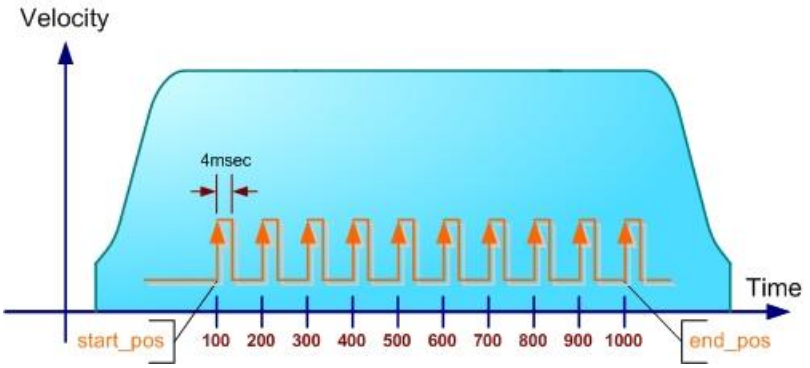
In case of SMC-2V03 module, trigger does not occur if it does not pass the trigger starting point. Also, trigger does not occur if it passes the trigger end point and enters into the range of trigger.

Low(B Contact) : Normal Close -> detected Open

High(A Contact) : Normal Open -> detected Close

Falling Edge : maintains high status for normal time, low status for trigger period, and then goes back to high status.

Riging Edge : maintains low status for normal time, high status for trigger period, and then goes back to low status.



Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
// Trigger output of 4msec for every 100 pulse between 100 and 1050, level low setting,
// interrupt enable.
AxmTriggerSetBlock(0, 100 , 1050, 100);
// Read trigger setting of AxmSetBlockTrigger API in axis 0.
double dpStartPos, dpEndPos, dpPeriodPos;
AxmTriggerGetBlock(0, &dpStartPos, &dpEndPos, &dpPeriodPos);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
' Trigger output of 4msec for every 100 pulse between 100 and 1050, level low setting,
' interrupt enable.
AxmTriggerSetBlock 0, 100, 1050, 100
' Read trigger setting of AxmSetBlockTrigger API in axis 0.
Dim dpStartPos, dpEndPos, dpPeriodPos As Double
AxmTriggerGetBlock 0, dpStartPos, dpEndPos, dpPeriodPos
```

Delphi

```
{ Read trigger setting of AxmSetBlockTrigger API in axis 0. }
var
  dpStartPos, dpEndPos, dpPeriodPos : Double;

begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function for axis 0. }
```

```
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Trigger output of 4msec for every 100 pulse between 100 and 1050, level low
setting, interrupt enable. }
  AxmTriggerSetBlock(0, 100, 1050, 100);
  AxmTriggerGetBlock(0, @dpStartPos, @dpEndPos, @dpPeriodPos);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerGetAbsPeriod, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerSetTimerOneshot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerGetBlock

Purpose

Return trigger output setting for regular intervals from start point to end point which user specified.

Format

C++

```
DWORD AxmTriggerGetBlock(long lAxisNo, double *dpStartPos, double
*dpEndPos, double *dpPeriodPos);
```

Visual Basic

```
Function AxmTriggerGetBlock(ByVal lAxisNo As Long, ByRef dpStartPos
As Double, ByRef dpEndPos As Double, ByRef dpPeriodPos As Double) As
Long
```

Delphi

```
function AxmTriggerGetBlock(lAxisNo : LongInt; dpStartPos : PDouble;
dpEndPos : PDouble; dpPeriodPos : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[<<]dpStartPos	Starting position of trigger signal
[<<]dpEndPos	Ending position of trigger signal
[<<]dpPeriodPos	Set trigger output period position

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

cf : If you only want to bring necessary data, you can put NULL for unnecessary variable.

'AxmTriggerSetBlock ' return specified trigger setting.

Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
// Trigger output of 4msec for every 100 pulse between 100 and 1050, level low setting,
// interrupt enable.
AxmTriggerSetBlock(0, 100 , 1050, 100);
```

```
// Read trigger setting of AxmSetBlockTrigger API in axis 0.
double dpStartPos, dpEndPos, dpPeriodPos;
AxmTriggerGetBlock(0, &dpStartPos, &dpEndPos, &dpPeriodPos);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
' Trigger output of 4msec for every 100 pulse between 100 and 1050, level low setting,
' interrupt enable.
AxmTriggerSetBlock 0, 100, 1050, 100
' Read trigger setting of AxmSetBlockTrigger API in axis 0.
Dim dpStartPos, dpEndPos, dpPeriodPos As Double
AxmTriggerGetBlock 0, dpStartPos, dpEndPos, dpPeriodPos
```

Delphi

```
{ Read trigger setting of AxmSetBlockTrigger API in axis 0. }
var
  dpStartPos, dpEndPos, dpPeriodPos : Double;

begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Trigger output of 4msec for every 100 pulse between 100 and 1050, level low
setting, interrupt enable. }
  AxmTriggerSetBlock(0, 100, 1050, 100);
  AxmTriggerGetBlock(0, @dpStartPos, @dpEndPos, @dpPeriodPos);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerOneShot, AxmTriggerSetTimerOneshot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerOneShot

Purpose

Output one trigger pulse at the position specified by user.

Format

C++

```
DWORD AxmTriggerOneShot(long lAxisNo);
```

Visual Basic

```
Function AxmTriggerOneShot(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmTriggerOneShot(lAxisNo : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

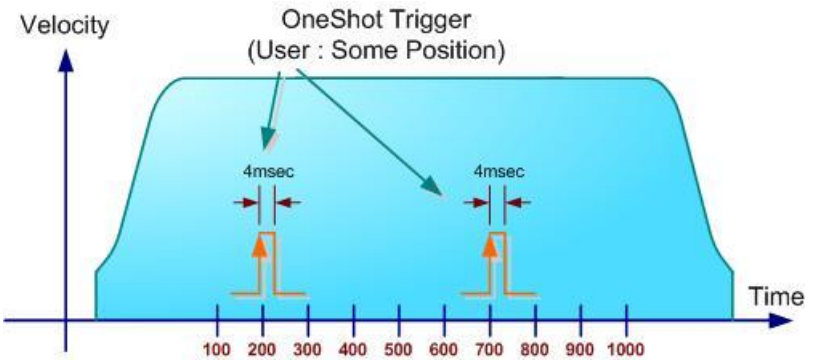
'AxmTriggerSetTimeLevel ' trigger function's trigger output time, level, output method, and whether to set interrupt for output are specified. You always have to use it with this API. It will be in different status like following according to trigger output level.

Low(B Contact) : Normal Close -> detected Open

High(A Contact) : Normal Open -> detected Close

Falling Edge : maintains high status for normal time, low status for trigger period, and then goes back to high status.

Riging Edge : maintains low status for normal time, high status for trigger period, and then goes back to low status.



Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
// Compulsorily output one trigger at axis 0.
AxmTriggerOneShot(0);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
' Compulsorily output one trigger at axis 0.
AxmTriggerOneShot 0
```

Delphi

```
begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Compulsorily output one trigger at axis 0. }
  AxmTriggerOneShot(0);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerSetTimerOneshot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerSetTimerOneshot

Purpose

Output trigger pulse at specific time specified by user after calling API.

Format

C++

```
DWORD AxmTriggerSetTimerOneshot(long lAxisNo, long lmSec);
```

Visual Basic

```
Function AxmTriggerSetTimerOneshot(ByVal lAxisNo As Long, ByVal lmSec
As Long) As Long
```

Delphi

```
function AxmTriggerSetTimerOneshot(lAxisNo : LongInt; lmSec :
LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[>>]lmSec	Time setting unit : msec

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

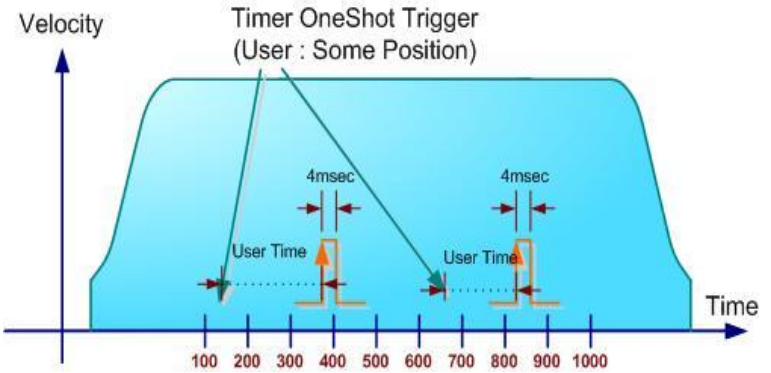
'AxmTriggerSetTimeLevel ' trigger function's trigger output time, level, output method, and whether to set interrupt for output or not are specified. You always have to use with this API. It will be in different status like following according to trigger output level.

Low(B Contact) : Normal Close -> detected Open

High(A Contact) : Normal Open -> detected Close

Falling Edge : maintains high status for normal time, low status for trigger period, and then goes back to high status.

Riging Edge : maintains low status for normal time, high status for trigger period, and then goes back to low status.



Example

C++

```
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
// Compulsorily output one trigger after 1 sec at axis 0.
AxmTriggerSetTimerOneshot(0, 1000);
```

Visual Basic

```
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
' Compulsorily output one trigger after 1 sec at axis 0.
AxmTriggerSetTimerOneshot 0, 1000
```

Delphi

```
begin
  { Set trigger output time, level, output method, and output interrupt of trigger
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Compulsorily output one trigger after 1 sec at axis 0. }
  AxmTriggerSetTimerOneshot(0, 1000);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerOnlyAbs, AxmTriggerSetReset

AxmTriggerOnlyAbs

Purpose

Output trigger pulse at absolute position specified by user regardless of the number of position.

Format

C++

```
DWORD AxmTriggerOnlyAbs(long lAxisNo, long lTrigNum, double*
dpTrigPos);
```

Visual Basic

```
Function AxmTriggerOnlyAbs(ByVal lAxisNo As Long, ByVal lTrigNum As
Long, ByRef dpTrigPos As Double) As Long
```

Delphi

```
function AxmTriggerOnlyAbs(lAxisNo : LongInt; lTrigNum : LongInt;
dpTrigPos : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)
[>>]lTrigNum	Number of trigger output position
[>>]dpTrigPos	Trigger position array

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

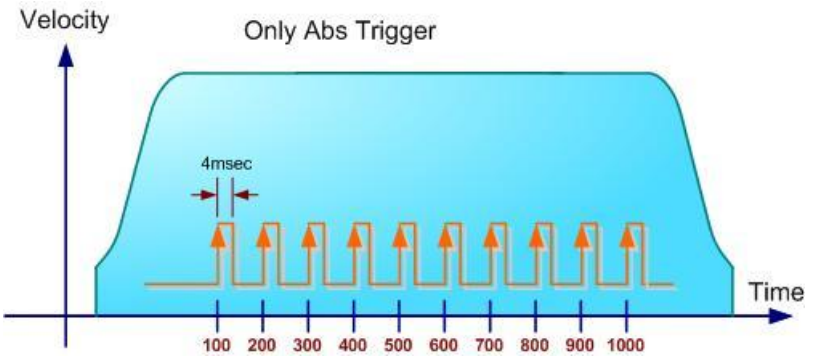
'AxmTriggerSetTimeLevel ' trigger function's trigger output time, level, output method, and whether to set interrupt for output or not are specified. You always have to use with this API. It will be in different status like following according to trigger output level.

Low(B Contact) : Normal Close -> detected Open

High(A Contact) : Normal Open -> detected Close

Falling Edge : maintains high status for normal time, low status for trigger period, and then goes back to high status.

Riging Edge : maintains low status for normal time, high status for trigger period, and then goes back to low status.



Example

C++

```
double trig_pos[10];
trig_pos[0] = 100;  trig_pos[1] = 200;  trig_pos[2] = 300;
trig_pos[3] = 400;  trig_pos[4] = 500;  trig_pos[5] = 600;
trig_pos[6] = 700;  trig_pos[7] = 800;  trig_pos[8] = 900;
trig_pos[9] = 1000;
// Set trigger output time, level, output method, and output interrupt of trigger
// function of axis 0.
AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
// Set absolute position of user at axis 0.
AxmTriggerOnlyAbs (0, 10, trig_pos);  // trig_time: 1usec ~ 4000 usec
```

Visual Basic

```
Dim trig_pos(0 To 9) As Double
trig_pos(0) = 100: trig_pos(1) = 200: trig_pos(2) = 300
trig_pos(3) = 400: trig_pos(4) = 500: trig_pos(5) = 600
trig_pos(6) = 700: trig_pos(7) = 800: trig_pos(8) = 900
trig_pos(9) = 1000
' Set trigger output time, level, output method, and output interrupt of trigger
' function of axis 0.
AxmTriggerSetTimeLevel 0, 4000, HIGH, 0, 1
' Set absolute position of user at axis 0.
AxmTriggerOnlyAbs 0, 10, trig_pos(0)  ' trig_time: 1usec ~ 4000 usec
```

Delphi

```
var
  trig_pos : array [0..9] of Double;

begin
  trig_pos[0] := 100;  trig_pos[1] := 200;  trig_pos[2] := 300;
  trig_pos[3] := 400;  trig_pos[4] := 500;  trig_pos[5] := 600;
  trig_pos[6] := 700;  trig_pos[7] := 800;  trig_pos[8] := 900;
  trig_pos[9] := 1000;
  { Set trigger output time, level, output method, and output interrupt of trigger
```

```
function of axis 0. }
  AxmTriggerSetTimeLevel(0, 4000, HIGH, 0, 1);
  { Set absolute position of user at axis 0. }
  AxmTriggerOnlyAbs (0, 10, @trig_pos); { trig_time: 1usec ~ 4000 usec}
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod, AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerSetTimerOneshot, AxmTriggerSetReset

AxmTriggerSetReset

Purpose

Release trigger output specified by user.

Format

C++

```
DWORD AxmTriggerSetReset(long lAxisNo);
```

Visual Basic

```
Function AxmTriggerSetReset(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmTriggerSetReset(lAxisNo : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel(axis) number(start from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Release trigger setting of specified axis set by 'AxmTriggerSetTimeLevel'.

Example

C++

```
// Release trigger setting of axis 0.
AxmTriggerSetReset(0);
```

Visual Basic

```
' Release trigger setting of axis 0.
AxmTriggerSetReset 0
```

Delphi

```
begin
   { Release trigger setting of axis 0. }
   AxmTriggerSetReset(0);
end;
```

See Also

AxmTriggerSetTimeLevel, AxmTriggerGetTimeLevel, AxmTriggerSetAbsPeriod,

AxmTriggerGetAbsPeriod, AxmTriggerSetBlock, AxmTriggerGetBlock, AxmTriggerOneShot, AxmTriggerSetTimerOneshot, AxmTriggerOnlyAbs

Gantry Move API

Gantry means system in which two motor axes are linked by mechanism and controlled like one axis. Generally in gantry system using 2 linear Servo motor, since if there is certain mechanical deviance between two gantries alarm may occur because of overload in Servo motor, method which completely synchronizes Command to both axes is used.

Function	Description
AxmGantrySetEnable	Motion module supports Gantry move system control in which two axes are mechanically linked. When setting master axis to gantry control using this function, slave axis will be synchronized with master axis. After gantry setting is done, all commands like move or stop command to Slave axis will be ignored.
AxmGantryGetEnable	By using this function, return parmaters specified when master axis was set to gantry control.
AxmGantrySetDisable	When executing home search after gantry control, slave axis conducts home search after master axis conducts home search. This function is used to identify error between master's home and slave's home after home search.

Two axes support gantry move function to control gantry move system which is mechanically linked like the picture below. When master axis(even channel(axis) number(starting from zero)) is set to gantry control using this function, corresponding slave axis will be synchronized with master axis in move. After gantry setting is done, all commands like move or stop command to slave axis will be ignored.



※ Note: In the case of gantry system, home sensor of master axis and slave axis will have certain error after they are assembled in the structure where each robot is put paralleled and mechanically linked. When doing home search, error of both home sensors will be compensated in the last step. It will be done by put assembly error value between sensors on dSlOffset.

The most important thing in gantry control is that master and slave axis should be considered when doing

home search. General step will be first finding master axis's home sensor and then finding slave axis's home sensor. The first thing to do is combining two by AxmGantrySetEnable , and then start home move of gantry. Then, you can use it like general home search function.

Variable of Home Search	Explanation
HmSig	Set signal to be used as home sensor ex) home sensor(4), +Limit sensor(0), etc
HmLev	Set active level of home sensor ex) 0: A Contact, 1: B Contact
HmDir	Set initial searching direction when detecting home sensor. ex) 1: (+)direction, 0: (-)direction
Zphas	Whether to detect encoder z phase after detecting home sensor
VelFirst	Initial high speed detecting speed for home search(when initial home sensor was not detected)
VelSecond	Speed for escaping to opposite direction after detecting 1st sensor in home search
VelThird	Speed of re-detecting after detecting 1st sensor in home search
VelLast	Set the final detecting speed in home search[determine detail of home search]
AccFirst	Initial high speed detecting acceleration in home search(Acceleration unit is PPS[Pulses/secgiven that Unit/pulse is 1/1)
AccSecond	Acceleration for escaping to opposite direction after detecting 1st sensor in home search(Acceleration unit is PPS[Pulses/secgiven that Unit/pulse is 1/1])
HclrTim	Waiting time before clearing Cmd, Act(Encoder) position after completion of home search
Hoffset	Offset value for moving when re-setting mechanical home after completion of home search
NSWLimit	Set (-) Software Limit which is applied after completion of home search
PSWLimit	Set (+) Software Limit which is applied after completion of home search. (If NSWLimit is equal to or bigger than PSWLimit value S/W limit will not be set)

< Understanding of Home Search Velocity setting and Home Search Sequence by Step (Master, Slave use same velocity) >

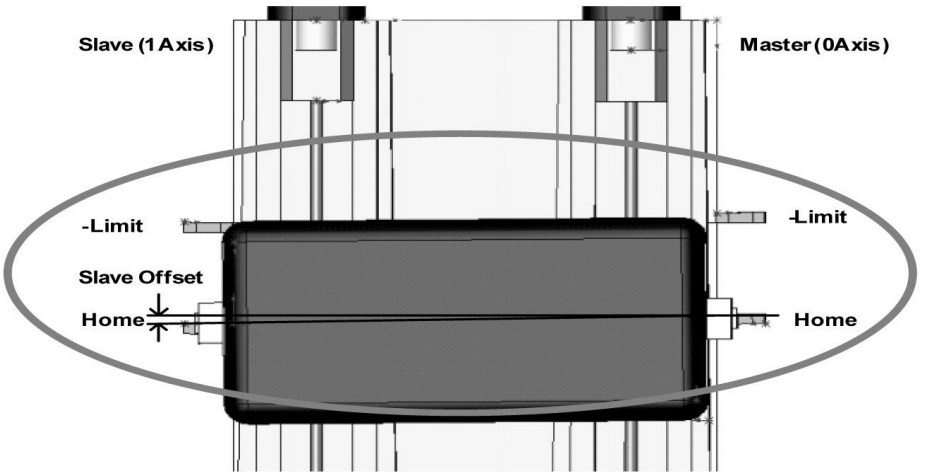
Step 1. First, find home sensor of Master axis. Set only master axis to position of 0.

Step 2. Find home sensor of Slave axis. Set only slave axis to position of 0. Then, we can get Offset value of slave against master.

Step 3. Release link between Master and Slave. Move Slave axis by Slave offset. Then send master axis to 0. Link them again.

Step 4. After moving by home offset, make position 0. If there's no home offset, set position to 0 without moving.

Step 5. Software Limit setting and Gantry move.



※ Caution : Soft limit can be set after home search is successfully processed. Soft limit can be used when limit sensor is not mechanically equiped or limit is to be changed according to situation. One thing to be careful is that PLimit(+) value should be bigger than NLimit(-) for successful setting of Soft Limit. If NSwLimit is same or more than PSwlimit, Soft Limit will not be set.

AxmGantrySetEnable

Purpose

If you set master axis to gantry control by using this API, corresponding Slave axis is synchronized with Master axis. It activates Gantry control API, and all move or stop commands to Slave axis afterwards will be ignored.

Format

C++

```
DWORD AxmGantrySetEnable(long lMasterAxisNo, long lSlaveAxisNo, DWORD uSlHomeUse, double dSlOffset, double dSlOffsetRange);
```

Visual Basic

```
Function AxmGantrySetEnable(ByVal lMasterAxisNo As Long, ByVal lSlaveAxisNo As Long, ByVal uSlHomeUse As Long, ByVal dSlOffset As Double, ByVal dSlOffsetRange As Double) As Long
```

Delphi

```
function AxmGantrySetEnable(lMasterAxisNo : LongInt; lSlaveAxisNo : LongInt; uSlHomeUse : Dword; dSlOffset : Double; dSlOffsetRange : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lMasterAxisNo	Master channel(axis) number(starting from 0)
[>>]lSlaveAxisNo	Slave channel(axis) number(starting from 0)
[>>]uSlHomeUse	Whether Slave axis will do home search like Master or not – [00h] Select home search only for Master axis – [01h] Identify selecting home search for Master axis and Slave axis together – [02h] Error price confirmation of Master axes and Slave celebration sensors
[>>]dSlOffset	Mechanical error between home sensor of master axis and home sensor of slave axis
[>>]dSlOffsetRange	Specify maximum error value between home sensor of master axis and home sensor of slave axis in home search

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE : Gantry-enable is activated again while in motion with gantry enabled
[4204] AXT_RT_MOTION_HOME_ERROR_GANTRY : Failed to execute Gantry enable during home search motion
* See error code Table for more information on status error codes

Description

.

Example

C++

```
if(AxmGantrySetEnable (0, 1, TRUE, 0, 0.0) == AXT_RT_SUCCESS )
  printf("Axis 0, axis 1 are set to Gantry Control.\n");
else
  printf("Axis 0, axis 1 are failed in setting Gantry Control.\n");
long   lHmDir = 1;
DWORD  lHmsig = 4;
DWORD  dwZphas = 0;
double dwHClrTim = 2000.0, dwHOffset = 0.0;
AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset);
AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
AxmHomeSetStart(0);
```

Visual Basic

```
If (AxmGantrySetEnable(0, 1, 1, 0#, 5#) = AXT_RT_SUCCESS) Then
  MsgBox "Axis 0, axis 1 are set to Gantry Control", vbOKCancel
Else
  MsgBox "Axis 0, axis 1 are failed in setting Gantry Control", vbOKCancel
End If
Dim dwHOffset As Double
Dim dwHClrTim As Double
Dim lHmDir, lHmsig, dwZphas As Long
dwHClrTim = 2000#
dwHOffset = 0#
lHmDir = -1
lHmsig = 4
dwZphas = 0
' home search in (-) direction, using Home Sensor, Home Sensor Type A contact, not
' using z phase detecting, not moving offset.
AxmHomeSetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
AxmHomeSetVel 0, 10000, 5000, 500, 50, 40000, 10000
AxmHomeSetStart 0
```

Delphi

```
var
  dwZphas, lHmsig : DWORD;
  dwHOffset, dwHClrTim : Double;
  lHmDir : LongInt;
  velocity : Double;

begin
  lHmDir := -1;
  lHmsig := 4;
  dwZphas := 0;
```

```
  dwHClrTim := 2000.0;
  dwHOffset := 0.0;
  if (AxmGantrySetEnable (0,1,1, 0.0, 5.0) = AXT_RT_SUCCESS) then
     Application.MessageBox('Axis 0, axis 1 are set to Gantry Control.', 'Ajinextek',
        MB_OK)
  else
     Application.MessageBox('Axis 0, axis 1 are failed in setting Gantry Control.',
        'Ajinextek', MB_OK);
  AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim,dwHOffset);
  AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
  AxmHomeSetStart(0);
end;
```

See Also

AxmGantrySetDisable, AxmGantryGetEnable, AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmGantryGetEnable

Purpose

By using this API, return parameter which was specified when setting master axis to gantry control.

Format

C++

```
DWORD AxmGantryGetEnable(long lMasterAxisNo, DWORD *upSlHomeUse,
double *dpSlOffset, double *dpSlORange, DWORD *upGatryOn);
```

Visual Basic

```
Function AxmGantryGetEnable(ByVal lMasterAxisNo As Long, ByRef
upSlHomeUse As Long, ByRef dpSlOffset As Double, ByRef dpSlORange As
Double, ByRef upGatryOn As Long) As Long
```

Delphi

```
function AxmGantryGetEnable(lMasterAxisNo : LongInt; upSlHomeUse :
PDWord; dpSlOffset : Pdouble; dpSlORange : Pdouble; upGatryOn :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lMasterAxisNo	Master channel(axis) number(starting from 0)
[<<]upSlHomeUse	Whether Slave axis will do home search like Master or not – [00h] Select home search only for Master axis – [01h] Identify selecting home search for Master axis and Slave axis together – [02h] Error price confirmation of Master axes and Slave celebration sensors
[<<]dpSlOffset	Mechanical error between home sensor of master axis and home sensor of slave axis
[<<]dpSlORange	Specify maximum error value between home sensor of master axis and home sensor of slave axis in home search
[<<]upGatryOn	Gantry ON/OFF parameter : AXT_USE – [00h] Gantry OFF – [01h] Gantry ON

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[4051] AXT_RT_MOTION_NOT_MODULE : No motion module is installed in the system
* See error code Table for more information on status error codes

Description

cf : You can put NULL value for unnecessary value to get only the necessary data.

To check the parameters related to Gantry configured by the AmxGantrySetEnable function.

Refer to the description of AsmGantrySetEnable function for the detailed explanation of parameters.

Example

C++

```
double  dOffset;
DWORD   upHomeResult0;
double    dpSlOffset,dSlORange;
DWORD   bSlHomeUse, bGatryOn;
if(AxmGantrySetEnable (0, 1, TRUE, 0, 0.0) == AXT_RT_SUCCESS )
  printf("Axis 0, axis 1 are set to Gantry Control.\n");
else
  printf("Axis 0, axis 1 are failed in setting Gantry Control.\n");
long   lHmDir = 1;
DWORD  lHmsig = 4;
DWORD  dwZphas = 0;
double dwHClrTim = 2000.0, dwHOffset = 0.0;
AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset);
AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
AxmHomeSetStart(0);
AxmGantryGetEnable(0, &bSlHomeUse, &dpSlOffset, &dSlORange, &bGatryOn);
```

Visual Basic

```
Dim  dOffset , dSlORange, dpSlOffset As Double
Dim upHomeResult0, uID As Long
Dim bSlHomeUse, bGatryOn As Long
Dim strData As String
If (AxmGantrySetEnable(0, 1, 1, 0#, 5#) = AXT_RT_SUCCESS) Then
  MsgBox "Axis 0, axis 1 are set to Gantry Control.", vbOKCancel
Else
  MsgBox "Axis 0, axis 1 are failed in setting Gantry Control.", vbOKCancel
End If
Dim dwHOffset As Double
Dim dwHClrTim As Double
Dim lHmDir, lHmsig, dwZphas As Long
dwHClrTim = 2000#
dwHOffset = 0#
lHmDir = -1
lHmsig = 4
dwZphas = 0
' home search in (-) direction, using home sensor, Home Sensor Type A contact, not
' using z phase detecting, not moving offset.
AxmHomeSetMethod 0, lHmDir, lHmsig, dwZphas, dwHClrTim, dwHOffset
AxmHomeSetVel 0, 10000, 5000, 500, 50, 40000, 10000
AxmHomeSetStart 0
AxmGantryGetEnable 0, bSlHomeUse, dpSlOffset, dSlORange, bGatryOn
```

Delphi

```
var
  dwZphas, lHmsig : DWORD;
  dwHOffset, dwHClrTim : Double;
```

```
  lHmDir : LongInt;
  velocity : Double;
  dOffset: Double;
  upHomeResult0, bMaSlChange, bSlHomeUse , bGatryOn :DWORD;
  dSlORange, dpSlOffset : Double;

begin
  lHmDir := -1;
  lHmsig := 4;
  dwZphas := 0;
  dwHClrTim := 2000.0;
  dwHOffset := 0.0;
  if (AxmGantrySetEnable (0,1,1, 0.0, 5.0) = AXT_RT_SUCCESS) then
     Application.MessageBox('Axis 0, axis 1 are set to Gantry Control.', 'Ajinextek',
        MB_OK)
  else
     Application.MessageBox('Axis 0, axis 1 are failed in setting to Gantry Control.',
        'Ajinextek', MB_OK);
  AxmHomeSetMethod(0, lHmDir, lHmsig, dwZphas, dwHClrTim,dwHOffset);
  AxmHomeSetVel(0,10000, 5000, 500, 50, 40000, 10000);
  AxmHomeSetStart(0);
  AxmGantryGetEnable(0, @bSlHomeUse, @dpSlOffset, @dSlORange, @bGatryOn);
end;
```

See Also

AxmGantrySetEnable, AxmGantrySetDisable, AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

AxmGantrySetDisable

Purpose

Release Gantry move sysem in which two axes are mechanically linked.

Format

C++

```
DWORD AxmGantrySetDisable(long lMasterAxisNo, long lSlaveAxisNo);
```

Visual Basic

```
Function AxmGantrySetDisable(ByVal lMasterAxisNo As Long, ByVal
lSlaveAxisNo As Long) As Long
```

Delphi

```
function AxmGantrySetDisable(lMasterAxisNo : LongInt; lSlaveAxisNo :
LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lMasterAxisNo	Master channel(axis) number(starting from 0)
[>>]lSlaveAxisNo	Slave channel(axis) number(starting from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4152] AXT_RT_MOTION_ERROR_IN_MOTION : Called another motion drive fuction before the completetion of current motion
[4204] AXT_RT_MOTION_HOME_ERROR_GANTRY : Failed to execute Gantry enable during home search motion
* See error code Table for more information on status error codes

Description

Release gantry move system control linked by specification of 'AxmGantrySetEnable'.

Example

C++

```
if(AxmGantrySetEnable (0, 1, TRUE, 0, 0.0) == AXT_RT_SUCCESS )
  printf("Axis 0, axis 1 are set to Gantry Control.\n");
else
  printf("Axis 0, axis 1 are failed in setting Gantry Control.\n");
// Release Gantry move system.
AxmGantrySetDisable (0, 1);
```

Visual Basic

```
If (AxmGantrySetEnable(0, 1, 1, 0#, 5#) = AXT_RT_SUCCESS) Then
  MsgBox "Axis 0, axis 1 are set to Gantry Control", vbOKCancel
```

```
Else
  MsgBox "Axis 0, axis 1 are failed in setting Gantry Control.", vbOKCancel
End If
AxmGantrySetDisable 0, 1
```

Delphi

```
begin
  if (AxmGantrySetEnable (0,1,1, 0.0, 5.0) = AXT_RT_SUCCESS) then
    Application.MessageBox('Axis 0, axis 1 are set to Gantry Control.', 'Ajinextek',
      MB_OK)
  else
    Application.MessageBox('Axis 0, axis 1 are failed in setting Gantry Control.',
      'Ajinextek', MB_OK);
  AxmGantrySetDisable (0, 1);
end;
```

See Also

AxmGantrySetEnable, AxmGantryGetEnable, AxmHomeSetMethod, AxmHomeGetMethod, AxmHomeSetVel, AxmHomeGetVel, AxmHomeSetStart, AxmHomeSetResult, AxmHomeGetResult, AxmHomeGetRate

CRC (Remaining Pulse Clear) API

Function	Description
AxmCrcSetMaskLevel	Set whether to use CRC signal and output level for specified axis.
AxmCrcGetMaskLevel	Return whether to use CRC signal and output level for specified axis.
AxmCrcSetOutput	Forcedly generate CRC signal for specific axis.
AxmCrcGetOutput	Return whether it forcedly generated CRC signal for specific axis.

In case of specific Servo pack, when move is completed or identifies limit sensor, there are some situations in which remaining pulse should be cleared by receiving CRC(Current Remaining Clear) signal from outside. For this, it provides signal pin to link directly to Servo Pack. In addition, we provide function to determine the presence of CRC signal usage, output level, and limit signal usage. User also can randomly generate CRC signal.

Output level is opposite level of chip output. That is, when chip output level is logical '1' CLR output is N24V, and when logical '0,' it is Open Collector. All these signals are Photo-Coupler like universal output, and electrically insulated from outside signal. These signals generate output by specific position or specifc condition from outside. Time delay is from time to go through Photo-Coupler and delay in PLD internal circuit. In the case of CLR signal which receives PELM and NELM as input, time delay is about 6㎲~8㎲. Because Photo Coupler output is Normally Open type output, user should attach Pull-up registance to get 24V level of voltage. Pull-up registance uses about 10㏀.

Note) CRC feature is only available in PCI-N804/404 and RTEX-PM.

AxmCrcSetMaskLevel

Purpose

Set whether to use CRC signal for specified axis.

Format

C++

```
DWORD AxmCrcSetMaskLevel(long lAxisNo, DWORD uLevel, DWORD uMethod);
```

Visual Basic

```
Function AxmCrcSetMaskLevel(ByVal lAxisNo As Long, ByVal uLevel As
Long, ByVal uMethod As Long) As Long
```

Delphi

```
function AxmCrcSetMaskLevel(lAxisNo : LongInt; uLevel : Dword;
uMethod : Dword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	channel(axis) number(starting from 0)
[>>]uLevel	[00h] Whether to use CRC signal or not : AXT_MOTION_LEVEL_MODE_DEF - [00h] B contact (NORMAL CLOSE) Use Active Low Level CRC signal - [01h] A contact (NORMAL OPEN) Use Active High Level CRC signal - [02h] CRC signal not in use (only for SMC-2V03, not for PCI-N804/404) - [03h] Maintain current status of CRC signal (only for SMC-2V03, not for PCI-N804/404)
[>>]uMethod	[00h] Remaining pulse clearing output signal pulse width (only for PCI-N804/404, not for SMC-2V03) - [00h] Not in use - [01h] Not in use - [02h] 500 uSec - [03h] 1 mSec - [04h] 10 mSec - [05h] 50 mSec - [06h] 100 mSec - [07h] Logic Level

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Set whether to use CRC signal or signal acive level for user's specified axis.

Low(B contact) : For Normal Close -> If detected Open

High(A contact): For Normal Open -> If detected Close.

Example

C++

```
// Set whether to use CRC signal on axis 0.
AxmCrcSetMaskLevel (0, LOW, 0);
// Identify whether to use CRC signal on axis 0.
DWORD uLevel, uMethod;
AxmCrcGetMaskLevel (0, &uLevel, &uMethod);
```

Visual Basic

```
' Set whether to use CRC signal on axis 0.
AxmCrcSetMaskLevel 0, LOW, 0
' Identify whether to use CRC signal on axis 0.
Dim uLevel As Long
Dim uMethod As Long
AxmCrcGetMaskLevel 0, uLevel, uMethod
```

Delphi

```
{ Identify whether to use CRC signal on axis 0. }
var
   uLevel: DWord;
   Method: DWord;

begin
   { Set whether to use CRC signal on axis 0. }
   xmCrcSetMaskLevel (0, LOW, 0);
   AxmCrcGetMaskLevel (0, @uLevel, @uMethod);
end;
```

See Also

AxmCrcGetMaskLevel, AxmCrcSetOutput, AxmCrcGetOutput

AxmCrcGetMaskLevel

Purpose

Return whether to use CRC signal or not for specific axis.

Format

C++

```
DWORD AxmCrcGetMaskLevel(long lAxisNo, DWORD *upLevel, DWORD
*upMethod);
```

Visual Basic

```
Function AxmCrcGetMaskLevel(ByVal lAxisNo As Long, ByRef upLevel As
Long, ByRef upMethod As Long) As Long
```

Delphi

```
function AxmCrcGetMaskLevel(lAxisNo : LongInt; upLevel : PDWord;
upMethod : PDword) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upLevel	[00h] Whether to use CRC signal or not : AXT_MOTION_LEVEL_MODE_DEF – [00h] B contact (NORMAL CLOSE) Use Active Low Level CRC signal – [01h] A contact (NORMAL OPEN) Use Active High Level CRC signal – [02h] CRC signal not in use (only for SMC-2V03, not for PCI-N804/404) – [03h] Maintain current status of CRC signal (only for SMC-2V03, not for PCI-N804/404)
[<<]upMethod	[00h] Remaining pulse clearing output signal pulse width (only for PCI-N804/404, not for SMC-2V03) – [00h] Not in use – [01h] Not in use – [02h] 500 uSec – [03h] 1 mSec – [04h] 10 mSec – [05h] 50 mSec – [06h] 100 mSec – [07h] Logic Level

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return whether to use CRC signal or signal active level setting for specified axis set by 'AxmCrcSetMaskLevel'

Low(B contact) : For Normal Close -> If detected Open

High(A contact): For Normal Open -> If detected Close.

Example

C++

```
// Set whether to use CRC signal on axis 0.
AxmCrcSetMaskLevel (0, LOW, 0);
// Identify whether to use CRC signal on axis 0.
DWORD uLevel, uMethod;
AxmCrcGetMaskLevel (0, &uLevel, &uMethod);
```

Visual Basic

```
' Set whether to use CRC signal on axis 0.
AxmCrcSetMaskLevel 0, LOW, 0
' Identify whether to use CRC signal on axis 0.
Dim uLevel As Long
Dim uMethod As Long
AxmCrcGetMaskLevel 0, uLevel, uMethod
```

Delphi

```
{ Identify whether to use CRC signal on axis 0. }
var
  uLevel: DWord;
  uMethod: DWord;

begin
  { Set whether to use CRC signal on axis 0. }
  AxmCrcSetMaskLevel (0, LOW, 0);
  AxmCrcGetMaskLevel (0, @uLevel, @uMethod);
end;
```

See Also

AxmCrcSetMaskLevel, AxmCrcSetOutput, AxmCrcGetOutput

AxmCrcSetOutput

Purpose

Generate forced output of CRC signal for specific axis.

Format

C++

```
DWORD AxmCrcSetOutput(long lAxisNo, DWORD uOnOff);
```

Visual Basic

```
Function AxmCrcSetOutput(ByVal lAxisNo As Long, ByVal uOnOff As Long)
As Long
```

Delphi

```
function AxmCrcSetOutput(lAxisNo : LongInt; uOnOff : DWord) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]uOnOff	Whether to generate CRC signal by Program : AXT_USE – [00h] CRC signal Off – [01h] CRC signal On

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Generate CRC signal on the specific axis when the user wants.

Low(B contact) : For Normal Close -> If detected Open

High(A contact): For Normal Open -> If detected Close.

Example

C++

```
// Generate CRC siganl on axis 0 by program.
AxmCrcSetOutput (0, ENABLE);
```

Visual Basic

```
' Generate CRC siganl on axis 0 by program.
AxmCrcSetOutput 0, ENABLE
```

Delphi

```
begin
   { Generate CRC siganl on axis 0 by program. }
   AxmCrcSetOutput (0, ENABLE);
end;
```

See Also

AxmCrcSetMaskLevel, AxmCrcGetMaskLevel

AxmCrcGetOutput

Purpose

Return whether to generate forced output of CRC signal for specific axis.

Format

C++

```
DWORD AxmCrcGetOutput(long lAxisNo, DWORD *upOnOff);
```

Visual Basic

```
Function AxmCrcGetOutput(ByVal lAxisNo As Long, ByRef upOnOff As Long)
As Long
```

Delphi

```
function AxmCrcGetOutput(lAxisNo : LongInt; upOnOff : PDWord) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]upOnOff	Whether to generate CRC signal by Program : AXT_USE – [00h] CRC signal Off – [01h] CRC signal On

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return whether to use CRC signal or signal active level for user's specified axis set by 'AxmCrcSetOutput '.

Low(B contact) : For Normal Close –> Open if detected

High(A contact): For Normal Open –> Close if detected.

Example

C++

```
// Generate CRC siganl on axis 0 by program.
AxmCrcSetOutput (0, ENABLE);
// Identify whether CRC siganl is generated on axis 0 by program.
DWORD uUse;
AxmCrcGetOutput (0, &uUse);
```

Visual Basic

```
' Generate CRC siganl on axis 0 by program.
AxmCrcSetOutput 0, ENABLE
' Identify whether CRC siganl is generated on axis 0 by program.
Dim uUse As Long
AxmCrcGetOutput 0, uUse
```

Delphi

```
var
  uUse: DWord;

begin
  { Generate CRC siganl on axis 0 by program. }
  AxmCrcSetOutput (0, ENABLE);
  AxmCrcGetOutput (0, @uUse);
end;
```

See Also

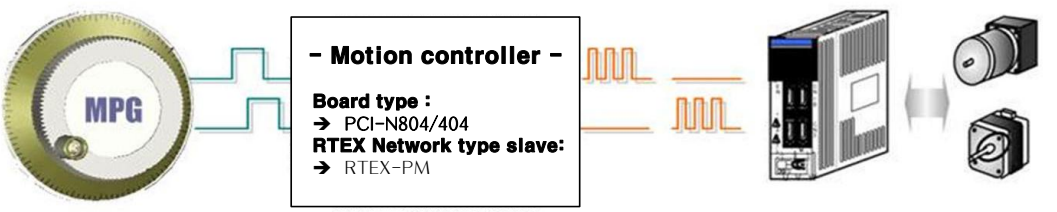
AxmCrcSetMaskLevel, AxmCrcGetMaskLevel, AxmCrcSetOutput

MPG Move API

Function	Description
AxmMPGSetEnable	Set MPG input method, drive move mode, moving distance for specific axis.
AxmMPGGetEnable	Return MPG input method, drive move mode, moving distance for specific axis.
AxmMPGSetRatio	Set pulse ratio of how much to move for a pulse in MPG drive move mode for specific axis.
AxmMPGGetRatio	Return pulse ratio of how much to move for a pulse in MPG drive move mode for specific axis.
AxmMPGReset	Relese MPG drive setting for specific axis.

Directional signal is determined by EXPP, EXMP signal, and the number of pulse by input spec setting register. The speed of output pulse is outputted by Object speed.

Note) MPG driving function that output the pulse is only available in (PCI-N804/404, RTEX-PM).

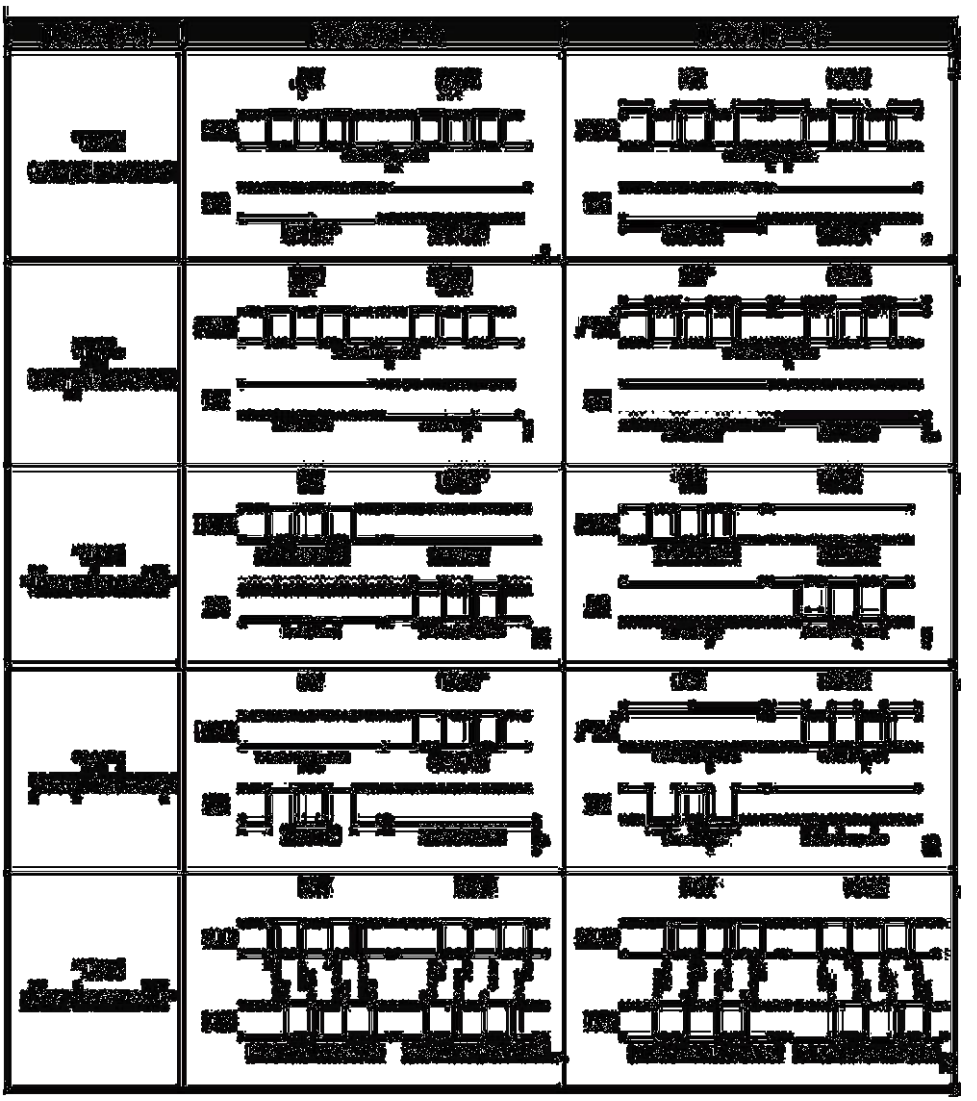


● *Interface with the outside MPG*

EXPP, EXMP signals are used an external rotary encoder to receive input signal up to 1.2 MHz high speed photo coupler.

Input pulse encoder interface is as following.

Input pulse shape



PCI-N804/404

MPG input signal is not inputted to each axis independently, but shared by both axes. MPG signal for axis 0 is shared by axis 2, MPG signal for axis 1 is shared by axis 3, MPG signal for axis 4 is shared by axis 6, and MPG signal for axis 5 is shared by axis 7. In other words, MPG signal input pin of external connector J2/J4 linked with axis 2/3/6/7 is not used.

AxmMPGSetEnable

Purpose

Set MPG input method, drive move mode, and moving distance for specific axis.

Format

C++

```
DWORD AxmMPGSetEnable(long lAxisNo, long lInputMethod, long
lDriveMode, double dMPGPos, double dVel, double dAccel);
```

Visual Basic

```
Function AxmMPGSetEnable(ByVal lAxisNo As Long, ByVal lInputMethod As
Long, ByVal lDriveMode As Long, ByVal dMPGPos As Double, ByVal dVel
As Double, ByVal dAccel As Double) As Long
```

Delphi

```
function AxmMPGSetEnable(lAxisNo : LongInt; lInputMethod : LongInt;
lDriveMode : LongInt; dMPGPos : Double; dVel: Double; dAccel:
Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lInputMethod	Input signal method : AXT_MOTION_MPG_INPUT_METHOD – [00h] MPG input type One Phase – [01h] MPG input type TwoPhase1 – [02h] MPG input type TwoPhase2 – [03h] MPG input type TwoPhase4 – [04h] MPG input type Level One Phase – [05h] MPG input type Level Two Phase1 – [06h] MPG input type Level Two Phase2 – [07h] MPG input type Level Two Phase4
[>>]lDriveMode	MPG motion mode – [00h] MPG continuous mode
[>>]dMPGPos	Moving distance for MPG_PRESET_MODE mode
[>>]dVel	Mpg velocity
[>>]dAccel	Reserved.

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Set distance, move mode, and move direction which will be used in MPG drive pulse move mode for specific axis.

Here, what dMPGdenominator = 4095, MPGnumerator=0 means is that for 200 pulses per one MPG cycle, it outputs 1pulse which is 1:1. In the case of dMPGdenominator = 4095, MPGnumerator=1 , it outpus 2 pulses which is 1:2.

Formula for internal chip output is MPG PULSE = ((Numerator) * (Denominator)/ 4096).

Caution: The results of API execution after AxmMPGSetEnable execution are different according to product.

PCI-N804/404: motion move status in the normal status before AxmMPGReset execution

SMC-2V03: motion move status only when pulse is outputted by input of MPG input signals.

Example

C++

```
// Set Mpg signal on axis 0.
AxmMPGSetEnable (0, MPG_DIFF_ONE_PHASE, 0, 10, 50, 200);
AxmMPGSetRatio(0, 1, 4096 );
// Return Mpg signal setting on axis 0.
long lpInputMethod, lpDriveMode;
double dpVel, dpAccel, dpDecel, dpMPGPos, dMPGdenominator, dMPGnumerator, dVel, dAccel;
AxmMPGGetEnable (0, &lpInputMethod, & lpDriveMode, &dpMPGPos, &dVel, & dAccel);
AxmMPGGetRatio(0, &dMPGnumerator, &dMPGdenominator);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim dDecel As Double
Dim dMPGdenominator As Double
Dim dMPGnumerator, dMPGPosition, dVel, dAccel As Double
Dim lInputMethod, lDriveMode, lDirMode, lUserDir As Long
' Set Mpg signal on axis 0.
AxmMPGSetEnable 0, MPG_DIFF_ONE_PHASE, 0, 10, 50, 200
AxmMPGSetRatio 0, 1, 4096
' Return Mpg signal setting on axis 0.
AxmMPGGetEnable 0, lInputMethod, lDriveMode, dMPGPosition, dVel, dAccel
AxmMPGGetRatio 0, dMPGnumerator, dMPGdenominator
```

Delphi

```
{ Return Mpg signal setting on axis 0. }
var
  dVelocity : Double;  dAccel    : Double;
  dDecel    : Double;  dMPGdenominator    : Double;
  dMPGnumerator, dMPGPosition, dVel : Double; lpInputMethod,lpDriveMode : longInt;

begin
  { Set Mpg signal on axis 0. }
  AxmMPGSetEnable (0, MPG_DIFF_ONE_PHASE, 0, 10, 50, 200);
```

```
   AxmMPGSetRatio(0, 1, 4096 );
   { Return Mpg signal setting on axis 0. }
   AxmMPGGetEnable (0, @lpInputMethod, @lpDriveMode, @dMPGPosition, @dVel, , @ dAccel);
   AxmMPGGetRatio(0, @dMPGnumerator, @dMPGdenominator);
end;
```

See Also

AxmMPGGetEnable, AxmMPGSetRatio, AxmMPGGetRatio, AxmMPGReset

AxmMPGGetEnable

Purpose

Return MPG input method, drive move mode, and moving distance for specific axis.

Format

C++

```
DWORD AxmMPGGetEnable(long lAxisNo, long *lpInputMethod, long
*lpDriveMode, double *dpMPGPos , double *dpVel, double *dpAccel);
```

Visual Basic

```
Function AxmMPGGetEnable(ByVal lAxisNo As Long, ByRef lpInputMethod
As Long, ByRef lpDriveMode As Long, ByRef dpMPGPos As Double, ByRef
dpVel As Double, ByRef dpAccel As Double) As Long
```

Delphi

```
function AxmMPGGetEnable(lAxisNo : LongInt; lpInputMethod : PLongInt;
lpDriveMode : PLongInt; dpMPGPos : PDouble; dpVel: PDouble; dpAccel:
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	channel(axis) number(starting from 0)
[<<]lpInputMethod	Input signal method
[<<]lpDriveMode	Mpg motion mode
[<<]dpMPGPos	Moving distance for MPG_PRESET_MODE mode
[<<]dpVel	MPG VELOCITY
[<<]dpAccel	Reserved.

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return MPG input method, drive move mode, and moving distance for specific axis set by 'AxmMPGSetEnable '.

Example

C++

```
// Set Mpg signal on axis 0.
AxmMPGSetEnable (0, MPG_DIFF_ONE_PHASE, 0, 10, 50, 200);
AxmMPGSetRatio(0, 1, 4096 );
```

```
// Return Mpg signal setting on axis 0.
long lpInputMethod, lpDriveMode;
double dpVel, dpAccel, dpDecel, dpMPGPos;
double dMPGdenominator, dMPGnumerator, dVel, dAccel;
AxmMPGGetEnable(0, &lpInputMethod, & lpDriveMode, &dpMPGPos, &dVel, & dAccel);
AxmMPGGetRatio(0, &dMPGnumerator, &dMPGdenominator);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim dDecel As Double
Dim dMPGdenominator As Double
Dim dMPGnumerator, dMPGPosition, dVel, dAccel As Double
Dim lInputMethod, lDriveMode, lDirMode, lUserDir As Long
' Set Mpg signal on axis 0.
AxmMPGSetEnable 0, MPG_DIFF_ONE_PHASE, 0, 10, 50, 200
AxmMPGSetRatio 0, 1, 4096
' Return Mpg signal setting on axis 0.
AxmMPGGetEnable 0, lInputMethod, lDriveMode, dMPGPosition, dVel, dAccel
AxmMPGGetRatio 0, dMPGnumerator, dMPGdenominator
```

Delphi

```
{ Return Mpg signal setting on axis 0. }
var
   dVelocity : Double;
   dAccel    : Double;
   dDecel    : Double;
   dMPGdenominator    : Double;
   dMPGnumerator   : Double;
   lpInputMethod,lpDriveMode : longInt;
   dMPGPosition, dVel : Double;

begin
   { Set Mpg signal on axis 0. }
   AxmMPGSetEnable (0, MPG_DIFF_ONE_PHASE, 0, 10, 50, 200);
   AxmMPGSetRatio(0, 1, 4096 );
   { Return Mpg signal setting on axis 0. }
   AxmMPGGetEnable(0, @lpInputMethod, @lpDriveMode, @dMPGPosition, @dVel, , @ dAccel);
   AxmMPGGetRatio(0, @dMPGnumerator, @dMPGdenominator);
end;
```

See Also

AxmMPGSetEnable, AxmMPGSetRatio, AxmMPGGetRatio, AxmMPGReset

AxmMPGSetRatio

Purpose

Set pulse ratio of how much to move for a pulse in MPG drive move mode for specific axis.

Format

C++

```
DWORD AxmMPGSetRatio(long lAxisNo, double dMPGnumerator, double
dMPGdenominator);
```

Visual Basic

```
Function AxmMPGSetRatio(ByVal lAxisNo As Long, ByVal dMPGnumerator As
Double, ByVal dMPGdenominator As Double) As Long
```

Delphi

```
function AxmMPGSetRatio(lAxisNo : LongInt; dMPGnumerator : Double;
dMPGdenominator : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dMPGnumerator	(Manual pulse occurrence device input) numerator value in move (input value is calculated) maximum(from 0 to 64), but only for PCI-N804/404., MPG PULSE = (dwNumerator)* (dwDenominator)/ 4096 Parameter setting will not be applied for SMC-2V03
[>>]dMPGdenominator	(Manual pulse occurrence device input) denominator value in move (input value is calculated) maximum(from 0 to 4096), but only for PCI-N804/404, Parameter setting will not be applied for SMC-2V03

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Set distance, move mode, and move direction which will be used in MPG drive pulse move mode for specific axis.

Here, what dMPGdenominator = 4095, MPGnumerator=0 means is that for 200 pulses per one MPG cycle, it outputs 1pulse which is 1:1. In the case of dMPGdenominator = 4095, MPGnumerator=1 , it outputs 2 pulses which is 1:2.

Formula for internal chip output is MPG PULSE = ((Numerator) * (Denominator)/ 4096).

Example

C++

```
// Set Mpg signal on axis 0.
AxmMPGSetEnable (0, MPG_DIFF_ONE_PHASE, 1, 10, 50, 200);
AxmMPGSetRatio(0, 1, 4096 );
// Return Mpg signal setting on axis 0.
long lpInputMethod, lpDriveMode;
double dpVel, dpAccel, dpDecel, dpMPGPos;
double dMPGdenominator, dMPGnumerator, dVel, dAccel;
AxmMPGGetEnable(0, &lpInputMethod, & lpDriveMode, &dpMPGPos, &dVel, & dAccel);
AxmMPGGetRatio(0, &dMPGnumerator, &dMPGdenominator);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim dDecel As Double
Dim dMPGdenominator As Double
Dim dMPGnumerator, dMPGPosition, dVel, dAccel As Double
Dim lInputMethod, lDriveMode, lDirMode, lUserDir As Long
' Set Mpg signal on axis 0.
AxmMPGSetEnable 0, MPG_DIFF_ONE_PHASE, 1, 10, 50, 200
AxmMPGSetRatio 0, 1, 4096
' Return Mpg signal setting on axis 0.
AxmMPGGetEnable 0, lInputMethod, lDriveMode, dMPGPosition, dVel, dAccel
AxmMPGGetRatio 0, dMPGnumerator, dMPGdenominator
```

Delphi

```
{ Return Mpg signal setting on axis 0. }
var
  dVelocity : Double;
  dAccel    : Double;
  dDecel    : Double;
  dMPGdenominator    : Double;
  dMPGnumerator   : Double;
  lpInputMethod,lpDriveMode : longInt;
  dMPGPosition, dVel : Double;

begin
  { Set Mpg signal on axis 0. }
  AxmMPGSetEnable (0, MPG_DIFF_ONE_PHASE, 1, 10, 50, 200);
  AxmMPGSetRatio(0, 1, 4096 );
  { Return Mpg signal setting on axis 0. }
  AxmMPGGetEnable(0, @lpInputMethod, @lpDriveMode, @dMPGPosition, @dVel, , @ dAccel);
  AxmMPGGetRatio(0, @dMPGnumerator, @dMPGdenominator);
end;
```

See Also

AxmMPGSetEnable, AxmMPGGetEnable, AxmMPGGetRatio, AxmMPGReset

AxmMPGGetRatio

Purpose

Return pulse ratio of how much to move for a pulse in MPG drive move mode for specific axis.

Format

C++

```
DWORD AxmMPGGetRatio(long lAxisNo, Double *dpMPGnumerator, Double
*dpMPGdenominator);
```

Visual Basic

```
Function AxmMPGGetRatio(ByVal lAxisNo As Long, ByRef dpMPGnumerator
As Double, ByRef dpMPGdenominator As Double) As Long
```

Delphi

```
function AxmMPGGetRatio(lAxisNo : LongInt; dpMPGnumerator : PDouble;
dpMPGdenominator : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dpMPGnumerator	Manual pulse occurrence device input) denominator value in move (input value is calculated) maximum(from 0 to 4096), but only for PCI-N804/404, Parameter setting will not be applied for SMC-2V03
[<<]dpMPGdenominator	(Manual pulse occurrence device input) numerator value in move(input value is calculated) maximum(from 0 to 64), but only for PCI-N804/404., MPG PULSE = (dpMPGnumerator)* (dpMPGdenominator)/ 4096 Parameter setting will not be applied for SMC-2V03

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return pulse ratio of how much to move for a pulse in MPG drive move mode for specific axis set by 'AxmMPGSetRatio'.

Example

C++

```
// Set Mpg signal on axis 0.
AxmMPGSetEnable (0, MPG_DIFF_ONE_PHASE, 1, 10, 50, 200);
AxmMPGSetRatio(0, 1, 4096 );
// Return Mpg signal setting on axis 0.
long lpInputMethod, lpDriveMode;
```

```
double dpVel, dpAccel, dpDecel, dpMPGPos;
double dMPGdenominator, dMPGnumerator, dVel, dAccel;
AxmMPGGetEnable(0, &lpInputMethod, & lpDriveMode, &dpMPGPos, &dVel, & dAccel);
AxmMPGGetRatio(0, &dMPGnumerator, &dMPGdenominator);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim dDecel As Double
Dim dMPGdenominator As Double
Dim dMPGnumerator, dMPGPosition, dVel, dAccel As Double
Dim lInputMethod, lDriveMode, lDirMode, lUserDir As Long
' Set Mpg signal on axis 0.
AxmMPGSetEnable 0, MPG_DIFF_ONE_PHASE, 1, 10, 50, 200
AxmMPGSetRatio 0, 1, 4096
' Return Mpg signal setting on axis 0.
AxmMPGGetEnable 0, lInputMethod, lDriveMode, dMPGPosition, dVel, dAccel
AxmMPGGetRatio 0, dMPGnumerator, dMPGdenominator
```

Delphi

```
{ Return Mpg signal setting on axis 0. }
var
  dVelocity : Double;
  dAccel    : Double;
  dDecel    : Double;
  dMPGdenominator    : Double;
  dMPGnumerator   : Double;
  lpInputMethod,lpDriveMode : longInt;
  dMPGPosition, dVel : Double;

begin
  { Set Mpg signal on axis 0. }
  AxmMPGSetEnable(0, MPG_DIFF_ONE_PHASE, 1, 10, 50, 200);
  AxmMPGSetRatio(0, 1, 4096 );
  { Return Mpg signal setting on axis 0. }
  AxmMPGGetEnable(0, @lpInputMethod, @lpDriveMode, @dMPGPosition, @dVel, , @ dAccel);
  AxmMPGGetRatio(0, @dMPGnumerator, @dMPGdenominator);
end;
```

See Also

AxmMPGSetEnable, AxmMPGGetEnable, AxmMPGSetRatio, AxmMPGReset

AxmMPGReset

Purpose

Release MPG drive setting for specific axis.

Format

C++

```
DWORD AxmMPGReset(long lAxisNo);
```

Visual Basic

```
Function AxmMPGReset(ByVal lAxisNo As Long) As Long
```

Delphi

```
function AxmMPGReset(lAxisNo : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Release MPG drive settings such as distance, move mode, and move direction for specific axis set by 'AxmMPGSetEnable'.

Example

C++

```
// Release MPG drive setting on axis 0.
AxmMPGReset(0);
```

Visual Basic

```
' Release MPG drive setting on axis 0.
AxmMPGReset 0
```

Delphi

```
begin
   { Release MPG drive setting on axis 0. }
   AxmMPGReset(0);
end;
```

See Also

AxmMPGSetEnable, AxmMPGGetEnable, AxmMPGSetRatio, AxmMPGGetRatio

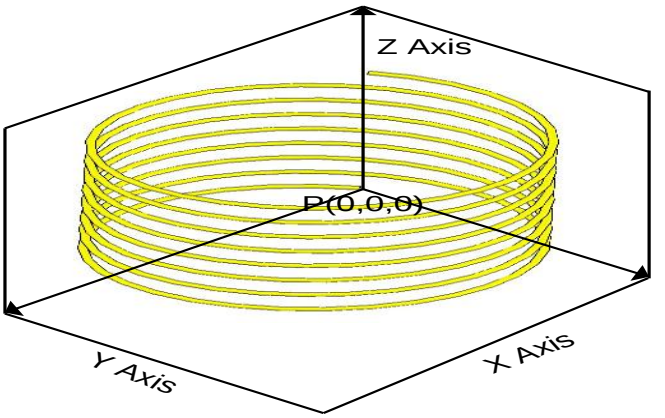
Helical Interpolation Setting And Move API

This API support only PCI-N804/404 specialized API.

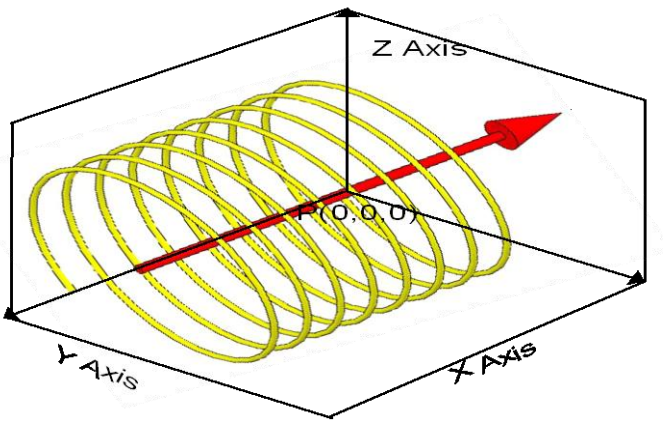
Helical interpolation control is the function which has combined 4 axes including 2 axes of circular interpolation and 2 axes of line interpolation. Thus, it can execute synchronized line and circular interpolation of 2 axes. It provides function that can execute one for a time, and that can be used with continuous interpolation function. To process helical interpolation, speed and distance of circular interpolation move should be synchronized with axis U(axis 4). The final axis should be axis U, and because axis U is moving with circular interpolation distance fallows axis Z and speed cannot be different from circular interpolation. In the case of motion board of 4 axes one group consists of axis X(0),Y(1),Z(2),U(3), and In the case of motion board of 8 axes one group consists of axis X(0),Y(1),Z(2),U(3)와 X2(4),Y2(5),Z2(6),U(7).

Function	Description
AxmHelixCenterMove	API to move helical interpolation with specification of start point, end point, and center point for specific axis. API to move helical interpolation with specification of start point, end point, and center point for specific axis in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API)
AxmHelixPointMove	API to move helical interpolation with specification of start point, end point, and radius for specific axis. API to move helical interpolation with specification of start point and center point for specific axis in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API)
AxmHelixRadiusMove	API to move helical interpolation with specification of start point, end point, and radius for specific axis. API to move helical interpolation with specification of start point, end point, and radius for specific axis in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API)
AxmHelixAngleMove	API to move helical interpolation with specification of start point, rotational angle, and radius for specific axis. API to move helical interpolation with specification of start point, rotational angle, and radius for specific axis in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API)

Ex1> Use helical interpolation with continuous interpolation



Ex2> Use helical interpolation with continuous interpolation



AxmHelixCenterMove

Purpose

API to move helical interpolation with specification of start point, end point, and center point for specific axis. It exits the function at the point of starting pulse output.

Format

C++

```
DWORD AxmHelixCenterMove(long lCoord, double dCenterXPos, double dCenterYPos, double dEndXPos, double dEndYPos, double dZPos, double dVel, double dAccel, double dDecel, DWORD uCWDir);
```

Visual Basic

```
Function AxmHelixCenterMove(ByVal lCoord As Long, ByVal dCenterXPos As Double, ByVal dCenterYPos As Double, ByVal dEndXPos As Double, ByVal dEndYPos As Double, ByVal dZPos As Double, ByVal dVel As Double, ByVal dAccel As Double, ByVal dDecel As Double, ByVal uCWDir As Long) As Long
```

Delphi

```
function AxmHelixCenterMove(lCoord : LongInt; dCenterXPos : Double; dCenterYPos : Double; dEndXPos : Double; dEndYPos : Double; dZPos : Double; dVel : Double; dAccel : Double; dDecel : Double; uCWDir : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordination System number (starting from 0)
[>>]dCenterXPos	Home position value of axis X
[>>]dCenterYPos	Home position value of axis Y
[>>]dEndXPos	End position value of axis X
[>>]dEndYPos	End position value of axis Y
[>>]dZPos	Distance to move to the direction of axis Z at a time
[>>]dVel	Move speed (Move speed unit is PPS[Pulses/sec] given that Unit/Pulse is 1/1)
[>>]dAccel	Move acceleration (accleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]dDecel	Move deceleration (decleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]uCWDir	Circular direction: AXT_MOTION_MOVE_DIR_DEF – [00h] Clockwise – [01h] Counterclockwise

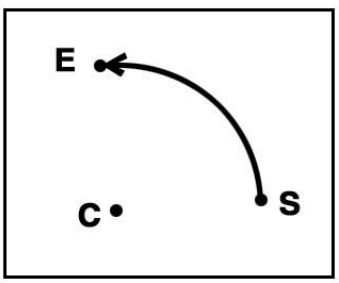
Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported

[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

Description

API to move helical interpolation with specification of start point, end point, and center point for specific axis in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API.) Current position becomes beginning point of circular move, and it process circular move on the basis of center and end point.



▶ By AxmSStop and AxmEstop API, slow down stop and emergency stop are possible. It always stops by transmitting master axis of Coordinate to parameter.

▶ AxmStatusReadInMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although Command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos for further information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will different with each other. Always equally set, and do interpolation control.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with more than 8 axes, 4 axes from Axis 0-3 becomes axis group 1, 4 axes from Axis 4-7 becomes axis group 2, and 4 axes from Axis 8-11 becomes axis group 3. Axes from the same group can process helical interpolation move, but axes from the different group cannot. The reason is that PCI-N804/404 chip has 4 axes, and it doesn't support this. Keep in mind when specify Coordinate.

▶Caution: When using Helix on continuous interpolation, Spline, linear interpolation and circular interpolation can not be used together.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.

AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

(Normal helical interpolation move example)

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[4];
long lCoordinate = 0;
for(int i=0; i<4; i++)
{
  lAxis[i] = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
  DIR_CW);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
```

```
Dim axes(0 To 3) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 3
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmHelixCenterMove lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel, 1
```

Delphi

```
Var
   dVelocity, dAccel : Double;
   i : LongInt;
   lAxis : array [0..3] of LongInt;
   lCoordinate : LongInt;

begin
   dVelocity := 2000;
   dAccel := 4000;
   lCoordinate := 0;
   for i := 0 to 3 do
   begin
      lAxis[i]     := i;
   end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
      DIR_CW);
end;
```

(continuous helical interpolation move example)

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[4];
long lCoordinate = 0;
for(int i=0; i<4; i++)
{
   lAxis[i]     = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
```

```
   DIR_CW);
AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
   DIR_CW);
AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
   DIR_CW);
AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
   DIR_CW);
AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
   DIR_CW);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate , 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
Dim axes(0 To 3) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 3
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
AxmHelixCenterMove lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel, 1
AxmHelixCenterMove lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel, 1
AxmHelixCenterMove lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel, 1
AxmHelixCenterMove lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel, 1
AxmHelixCenterMove lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel, 1
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   dVelocity, dAccel : Double;
   i : LongInt;
   lAxis : array [0..3] of LongInt;
   lCoordinate : LongInt;

begin
   dVelocity := 2000;
   dAccel := 4000;
   lCoordinate := 0;
```

```
   for i := 0 to 3 do
      begin
         lAxis[i]     := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmContiBeginNode(lCoordinate);
   AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
      DIR_CW);
   AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
      DIR_CW);
   AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
      DIR_CW);
   AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
      DIR_CW);
   AxmHelixCenterMove(lCoordinate, 500, 500, 0, 0, 100, dVelocity, dAccel, dAccel,
      DIR_CW);
   AxmContiEndNode(lCoordinate);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmHelixPointMove, AxmHelixRadiusMove, AxmHelixAngleMove, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmHelixPointMove

Purpose

API to move helical interpolation with specification of center point, end point for specific axis. It exits function at the point of beginning of pulse output.

Format

C++

```
DWORD AxmHelixPointMove(long lCoord, double dMidXPos, double dMidYPos,
double dEndXPos, double dEndYPos, double dZPos, double dVel, double
dAccel, double dDecel);
```

Visual Basic

```
Function AxmHelixPointMove(ByVal lCoord As Long, ByVal dMidXPos As
Double, ByVal dMidYPos As Double, ByVal dEndXPos As Double, ByVal
dEndYPos As Double, ByVal dZPos As Double, ByVal dVel As Double,
ByVal dAccel As Double, ByVal dDecel As Double) As Long
```

Delphi

```
function AxmHelixPointMove(lCoord : LongInt; dMidXPos : Double;
dMidYPos : Double; dEndXPos : Double; dEndYPos : Double; dZPos :
Double; dVel : Double; dAccel : Double; dDecel : Double) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordination System number (starting from 0)
[>>]dMidXPos	Midle position value of X axis
[>>]dMidYPos	Midle position value of Y axis
[>>]dEndXPos	End position value of X axis
[>>]dEndYPos	End position value of Y axis
[>>]dZPos	Distance to move to the direction of Z axis at a time
[>>]dVel	Move speed (Move speed unit is PPS[Pulses/sec] given that Unit/Pulse is 1/1)
[>>]dAccel	Move acceleration (accleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]dDecel	Move deceleration (decleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)

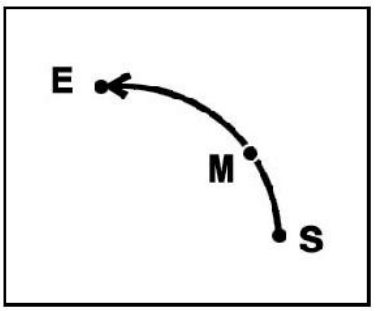
Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping

[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

Description

API to move helical interpolation with specification of start point, end point, and radius. API to move helical interpolation with specification of middle point and end point for specific axis in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API.) Current position becomes beginning point of circular move, and it process circular move going through middle point.



▶ By AxmSStop and AxmEstop API, slow down stop and emergency stop are possible. It always stops by transmitting master axis of Coordinate to parameter.

▶ AxmStatusReadInMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although Command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos for futher information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will different with each other. Always equally set, and do interpolation control.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with more than 8 axes, 4 axes from Axis 0-3 becomes axis group 1, 4 axes from Axis 4-7 becomes axis group 2, and 4 axes from Axis 8-11 becomes axis group 3. Axes from the same group can process helical Interpolation move, but axes from the different group cannot. The reason is that PCI-N804/404 chip has 4 axes, and it doesn't support this. Keep in mind when specify Coordinate.

▶Caution: When using Helix on continuous interpolation, Spline, linear interpolation and circular interpolation can not be used together.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.

AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

(Normal helical interpolation move example)

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[4];
long lCoordinate = 0;
for(int i=0; i<4; i++)
{ lAxis[i]    = i; }
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
Dim axes(0 To 3) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 3
```

```
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmHelixPointMove lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel
```

Delphi

```
var
  dVelocity, dAccel : Double;
  i : LongInt;
  lAxis : array [0..3] of LongInt;
  lCoordinate : LongInt;

begin
  dVelocity := 2000;
  dAccel := 4000;
  lCoordinate := 0;
  for i := 0 to 3 do
     begin
        lAxis[i]    := i;
     end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
end;
```

(continuous helical interpolation move example)

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[4];
long lCoordinate = 0;
for(int i=0; i<4; i++)
{
  lAxis[i]    = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate , 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
Dim axes(0 To 3) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 3
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
AxmHelixPointMove lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel
AxmHelixPointMove lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel
AxmHelixPointMove lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel
AxmHelixPointMove lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel
AxmHelixPointMove lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
  dVelocity, dAccel : Double;
  i : LongInt;
  lAxis : array [0..3] of LongInt;
  lCoordinate : LongInt;

begin
  dVelocity := 2000;
  dAccel := 4000;
  lCoordinate := 0;
  for i := 0 to 3 do
     begin
        lAxis[i]    := i;
     end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmContiBeginNode(lCoordinate);
  AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
  AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
  AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
  AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
  AxmHelixPointMove(lCoordinate, 500, 500, 1000, 0, 100, dVelocity, dAccel, dAccel);
```

```
    AxmContiEndNode(lCoordinate);
    AxmContiSetAbsRelMode(lCoordinate, 1);
    AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode , AxmHelixCenterMove, AxmHelixRadiusMove, AxmHelixAngleMove, AxmContiReadFree,AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmHelixRadiusMove

Purpose

API to move helical interpolation with specification of start point, end point, and radius for specific axis. It exits function at the point of beginning of pulse output.

Format

C++

```
DWORD AxmHelixRadiusMove(long lCoord, double dRadius, double dEndXPos,
double dEndYPos, double dZPos, double dVel, double dAccel, double
dDecel, DWORD uCWDir, DWORD uShortDistance);
```

Visual Basic

```
Function AxmHelixRadiusMove(ByVal lCoord As Long, ByVal dRadius As
Double, ByVal dEndXPos As Double, ByVal dEndYPos As Double, ByVal
dZPos As Double, ByVal dVel As Double, ByVal dAccel As Double, ByVal
dDecel As Double, ByVal uCWDir As Long, ByVal uShortDistance As Long)
As Long
```

Delphi

```
function AxmHelixRadiusMove(lCoord : LongInt; dRadius : Double;
dEndXPos : Double; dEndYPos : Double; dZPos : Double; dVel : Double;
dAccel : Double; dDecel : Double; uCWDir : DWord; uShortDistance :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordination System number(starting from 0)
[>>]dRadius	Circular radius for move
[>>]dEndXPos	End position value of X axis
[>>]dEndYPos	End position value of Y axis
[>>]dZPos	Distance to move to the direction of Z axis at a time
[>>]dVel	Move speed (Move speed unit is PPS[Pulses/sec] given that Unit/Pulse is 1/1)
[>>]dAccel	Move acceleration (accleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]dDecel	Move deceleration (decleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]uCWDir	Circular direction: AXT_MOTION_MOVE_DIR_DEF – [00h] Clockwise – [01h] Counterclockwise
[>>]uShortDistance	Distance of circle which goes to the middle point : AXT_MOTION_RADIUS_DISTANCE_DEF – [00h] Circular move for short distance – [01h] Circular move for long distance

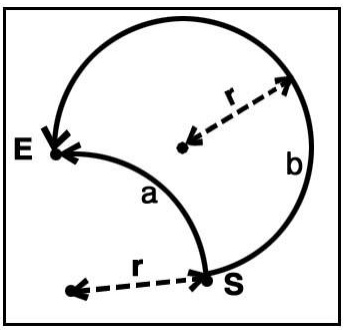
Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

Description

API to move helical interpolation with specification of start point, middle point and radius for specific axis in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API.) Current position becomes beginning point of circular move, and it process circular move in the basis of specified radius and end point.

When we define certain circular arc which connects beginning point and end point, we can have two middle points. Because of these, two paths(a,b) can be defined even if it has same rotational direction.



▶ By AxmSStop and AxmEstop function, slow down stop and emergency stop are possible. It always stops by transmitting master axis of Coordinate to parameter.

▶ AxmStatusReadInMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although Command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos for futher information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will different with each other. Always equally set, and do interpolation control.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with more than 8 axes, 4 axes from Axis 0-3 becomes axis group 1, 4 axes from Axis 4-7 becomes axis group 2, and 4 axes from Axis 8-11 becomes axis group 3. Axes from the same group can process helical interpolation move, but axes from the different group cannot. The reason is that PCI-N804/404 chip has 4 axes, and it doesn't support this. Keep in mind when specify Coordinate.

▶Caution: When using Helix on continuous interpolation, Spline, linear interpolation and circular interpolation can not be used together.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

(Normal helical interpolation move example)

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[4];
long lCoordinate = 0;
for(int i=0; i<4; i++)
{
  lAxis[i]    = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
Dim axes(0 To 3) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 3
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmHelixRadiusMove lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE
```

Delphi

```
var
  dVelocity, dAccel : Double;
  i : LongInt;
  lAxis : array [0..3] of LongInt;
  lCoordinate : LongInt;

begin
  dVelocity := 2000;
  dAccel := 4000;
  lCoordinate := 0;
  for i := 0 to 3 do
     begin
        lAxis[i]    := i;
     end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
     SHORT_DISTANCE);
end;
```

(continuous helical interpolation move example)

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[4];
long lCoordinate = 0;
for(int i=0; i<4; i++)
{
  lAxis[i]    = i;
}
```

```
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE);
AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE);
AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE);
AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE);
AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate , 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
Dim axes(0 To 3) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 3
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
AxmHelixRadiusMove lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE
AxmHelixRadiusMove lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE
AxmHelixRadiusMove lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE
AxmHelixRadiusMove lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE
AxmHelixRadiusMove lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
  SHORT_DISTANCE
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
   dVelocity, dAccel : Double;
   i : LongInt;
   lAxis : array [0..3] of LongInt;
   lCoordinate : LongInt;

begin
   dVelocity := 2000;
   dAccel := 4000;
   lCoordinate := 0;
   for i := 0 to 3 do
      begin
         lAxis[i]     := i;
   end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmContiBeginNode(lCoordinate);
   AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
      SHORT_DISTANCE);
   AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
      SHORT_DISTANCE);
   AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
      SHORT_DISTANCE);
   AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
      SHORT_DISTANCE);
   AxmHelixRadiusMove(lCoordinate, 50, 100, 0, 100, dVelocity, dAccel, dAccel, DIR_CW,
      SHORT_DISTANCE);
   AxmContiEndNode(lCoordinate);
   AxmContiSetAbsRelMode(lCoordinate, 1);
   AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmHelixCenterMove, AxmHelixPointMove, AxmHelixAngleMove, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

AxmHelixAngleMove

Purpose

API to move helical interpolation with specification of start point, rotational angle, and radius for specific axis. It exits function at the point of beginning of pulse output.

Format

C++

```
DWORD AxmHelixAngleMove(long lCoord, double dCenterXPos, double
dCenterYPos, double dAngle, double dZPos, double dVel, double dAccel,
double dDecel, DWORD uCWDir);
```

Visual Basic

```
Function AxmHelixAngleMove(ByVal lCoord As Long, ByVal dCenterXPos As
Double, ByVal dCenterYPos As Double, ByVal dAngle As Double, ByVal
dZPos As Double, ByVal dVel As Double, ByVal dAccel As Double, ByVal
dDecel As Double, ByVal uCWDir As Long) As Long
```

Delphi

```
function AxmHelixAngleMove(lCoord : LongInt; dCenterXPos : Double;
dCenterYPos : Double; dAngle : Double; dZPos : Double; dVel : Double;
dAccel : Double; dDecel : Double; uCWDir : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordination System number(starting from 0)
[>>]dCenterXPos	Middle position value of X axis
[>>]dCenterYPos	Middle position value of Y axis
[>>]dAngle	Angle for circular move. Unit is ‘Degree (도)
[>>]dZPos	Distance to move to the direction of Z axis at a time
[>>]dVel	Move speed (Move speed unit is PPS[Pulses/sec] given that Unit/Pulse is 1/1)
[>>]dAccel	Move acceleration (accleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]dDecel	Move deceleration (decleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]uCWDir	Circular direction: AXT_MOTION_MOVE_DIR_DEF – [00h] Clockwise – [01h] Counterclockwise

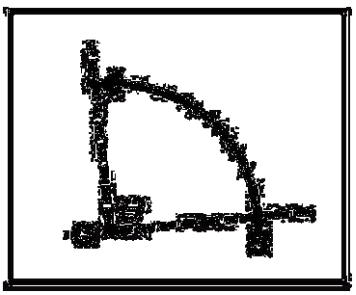
Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4158] AXT_RT_MOTION_INVALID_POSITION : Not in a valid position
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping

[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion
* See error code Table for more information on status error codes

Description

API to move helical interpolation with specification of start point, rotational angle and radius for specified coordinate system. API to move helical interpolation with specification of start point, rotational angle and radius for specific coordinate system in using with AxmContiBeginNode, AxmContiEndNode . It saves to internal Queue for circular continuous interpolation move. It starts with the use of AxmContiStart API.(It is used with continuous interpolation API.) Current position becomes beginning point of circular move, and it process circular move to specified angle in the basis of end point.



▶ By AxmSStop and AxmEstop function, slow down stop and emergency stop are possible. It always stops by transmitting master axis of Coordinate to parameter.

▶ AxmStatusReadInMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although Command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos for futher information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will different with each other. Always equally set, and do interpolation control.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with more than 8 axes, 4 axes from Axis 0-3 becomes axis group 1, 4 axes from Axis 4-7 becomes axis group 2, and 4 axes from Axis 8-11 becomes axis group 3. Axes from the same group can process helical interpolation move, but axes from the different group cannot. The reason is that PCI-N804/404 chip has 4 axes, and it doesn't support this. Keep in mind when specify Coordinate.

▶Caution: When using Helix on continuous interpolation, Spline, linear interpolation and circular interpolation can not be used together

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.

AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internal Queue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.
AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

(Normal helical interpolation move example)

C++

```
double  dVelocity = 2000, dAccel = 4000;
long    lAxis[4];
long lCoordinate = 0;

for(int  i=0; i<4; i++)
{
  lAxis[i]     = i;
}

AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
Dim axes(0 To 3) As Long
Dim i As Long
```

```
dVelocity = 2000
dAccel = 4000
lCoordinate = 0

For i = 0 To 3
   axes(i) = i
Next i

AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmHelixAngleMove lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW
```

Delphi

```
var
  dVelocity, dAccel : Double;
  i : LongInt;
  lAxis : array [0..3] of LongInt;
  lCoordinate : LongInt;

begin
  dVelocity := 2000;
  dAccel := 4000;
  lCoordinate := 0;
  for i := 0 to 3 do
     begin
        lAxis[i]    := i;
  end;
  AxmContiWriteClear(lCoordinate);
  AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel,
     DIR_CW);
end;
```

(Normal helical interpolation move example)

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[4];
long lCoordinate = 0;
for(int i=0; i<4; i++)
{
  lAxis[i]    = i;
}
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 4, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiBeginNode(lCoordinate);
AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW);
```

```
AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW);
AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW);
AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW);
AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 1);
AxmContiStart(lCoordinate , 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lCoordinate As Long
Dim axes(0 To 3) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 3
   axes(i) = i
Next i
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiBeginNode lCoordinate
AxmHelixAngleMove lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW
AxmHelixAngleMove lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW
AxmHelixAngleMove lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW
AxmHelixAngleMove lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW
AxmHelixAngleMove lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 1
AxmContiStart lCoordinate, 0, 0
```

Delphi

```
var
  dVelocity, dAccel : Double;
  i : LongInt;
  lAxis : array [0..3] of LongInt;
  lCoordinate : LongInt;

begin
  dVelocity := 2000;
  dAccel := 4000;
  lCoordinate := 0;
  for i := 0 to 3 do
     begin
        lAxis[i]     := i;
     end;
  AxmContiWriteClear(lCoordinate);
```

```
  AxmContiSetAxisMap(lCoordinate, 4, @lAxis);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmContiBeginNode(lCoordinate);
  AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel,
     DIR_CW);
  AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel,
     DIR_CW);
  AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel,
     DIR_CW);
  AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel,
     DIR_CW);
  AxmHelixAngleMove(lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel,
     DIR_CW);
  AxmContiEndNode(lCoordinate);
  AxmContiSetAbsRelMode(lCoordinate, 1);
  AxmContiStart(lCoordinate, 0, 0);
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmHelixCenterMove, AxmHelixPointMove, AxmHelixRadiusMove, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

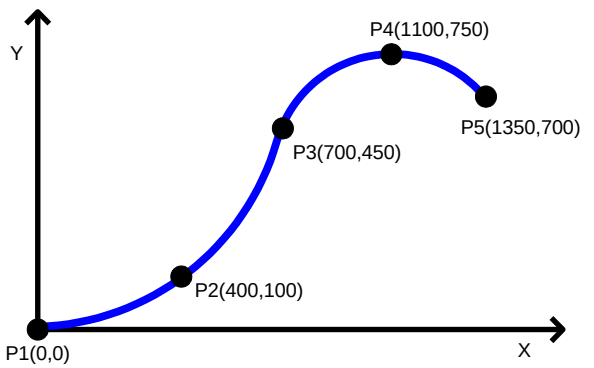
Spline Interpolation Setting And Move API

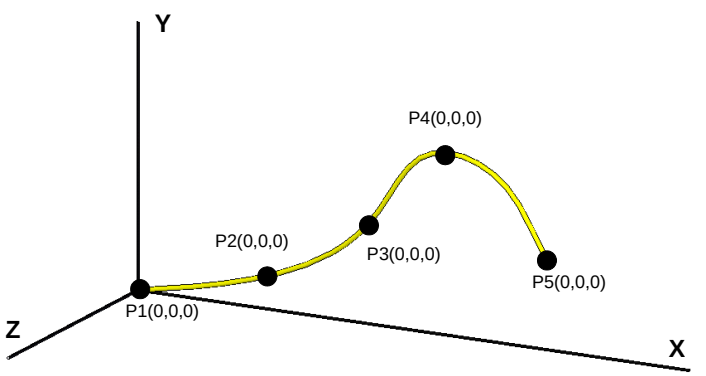
This API support only PCI-N804/404 specialized API.

Spline interpolation control provides B Spline interpolation function. B Spline interpolation executes interpolation by changing data points which the user specifys to free curve. To process this interpolation as a direct motion, it should be used with continuous interpolation motion function. It supports basically axis X, Y, Z. For 4 axes motion board it can select three axes from X(0),Y(1),Z(2),U(3). For 8 axes motion board it can selects three axes from X(0),Y(1),Z(2),U(3) in one chip and three axes from X(4),Y(5),Z(6),U(7) in another chip.

Note) Spline interpolation is supported only in (PCI-N804/404).

Function	Description
AxmSplineWrite	It is used with AxmContiBeginNode, AxmContiEndNode. It processes spline continuous inpterpolation move. It stores to internal queue for circular continuous interpolation move. It starts by using AxmContiStart API.(It is used with continuous interpolation function)





AxmSplineWrite

Purpose

Following Pos Point which user specified, it processes B Spline interpolation, which saves curve data to array user specified. This API supports spline for 2 axex and 3 axes, and used with AxmContiStart API.

Format

C++

```
DWORD AxmSplineWrite(long lCoord, long lPosSize, double *dpPosX,
double *dpPosY, double dVel, double dAccel, double dDecel, double
dPosZ, long lPointFactor);
```

Visual Basic

```
Function AxmSplineWrite(ByVal lCoord As Long, ByVal lPosSize As Long,
ByRef dpPosX As Double, ByRef dpPosY As Double, ByVal dVel As Double,
ByVal dAccel As Double, ByVal dDecel As Double, ByVal dPosZ As Double,
ByVal lPointFactor As Long) As Long
```

Delphi

```
function AxmSplineWrite(lCoord : LongInt; lPosSize : LongInt; dpPosX :
PDouble; dpPosY : PDouble; dVel : Double; dAccel : Double; dDecel :
Double; dPosZ : Double; lPointFactor: LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lCoord	Coordination System number(starting from 0)
[>>]lPosSize	The number of Points for moving(minimum 3)
[>>]dpPosX	Array of X values for moving
[>>]dpPosY	Array of Y values for moving
[>>]dVel	Move speed (Move speed unit is PPS[Pulses/sec] given that Unit/Pulse is 1/1)
[>>]dAccel	Move acceleration(accleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]dDecel	Move deceleration(decleration unit is PPS[Pulses/sec^2] given that Unit/pulse is 1/1)
[>>]dPosZ	+Z value for each position when using Z axis. Initial value is set to 0. It is used in 3 dimentional Spline.
[>>]lPointFactor	Factor of Point. Minimum is 10. Path of point deffers by Factor.

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[4160] AXT_RT_ERROR_NOT_SAME_BOARD : Not in the same board
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO : Invalid mapping number during continuous interpolation mapping
[4201] AXT_RT_MOTION_HOME_SEARCHING : Other function is called while in home search motion

* See error code Table for more information on status error codes

Description

Spline interpolation function is to automatically generate points of free curve by using BSpline interpolation technique based on points user specified. To move this directly, it should be used with continuous interpolation.

▶ AxmStatusReadInMotion API can verify whether it is on motion or not.

▶ When using AxmSignalSetInpos API, if Inpos is set to Enable, it doesn't return until INP input is set to ON although Command pulse output is completed because it considers move is not completed. Refer to AxmSignalSetInpos for futher information.

▶Caution: Axes which interpolate Unit/Pulse should be equally set. If Unit/Pulse is not equally set, their unit will different with each other. Always set equally and do interpolation control.

▶Caution: When using motion board with more than 8 (PCI-N804/404)

In the board with more than 8 axes, 4 axes from Axis 0-3 becomes axis group 1, 4 axes from Axis 4-7 becomes axis group 2, and 4 axes from Axis 8-11 becomes axis group 3. Axes from the same group can process spline interpolation move, but axes from the different group cannot. The reason is that PCI-N804/404 chip has 4 axes, and it doesn't support this. Keep in mind when specifying Coordinate.

▶Caution: when using spline on continuous interpolation, Helix, linear interpolation and circular interpolation can not be used together.

▶ Continuous Interpolation API list

Function	Description
AxmContiSetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiGetAxisMap	Setting continous interpolation axis mapping for specified Coordinate system.
AxmContiSetAbsRelMode	Setting continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiGetAbsRelMode	Return continous interpolation axis absolute/relative mode for specified coordinate system.
AxmContiBeginNode	Start registering tasks which will be processed in continous interpolation for specified coordinate system. After calling this function, all motion tasks processed before calling AxmContiEndNode function are registered as continous interpolation motion, but not processing real motion. When AxmContiStart function is called, registered motion is actually started.
AxmContiEndNode	End registering tasks to process continous interpolation in specified coordinate system.
AxmContiReadFree	Identify internal Queue is empty for interpolation move in specified coordinate system.
AxmContiReadIndex	Identify the number of interpolation move in internalQueue stored for interpolation move in specified coordinate system.
AxmContiWriteClear	Clear all of the internal Queue stored for continuous interpolation move in a specified coordinate system.
AxmContiStart	Start continuous interpolation move from internal Queue stored in a specified coordinate system.

AxmContiIsMotion	Identify countinuous interpolation move is runningor not in specified coordinate system.
AxmContiGetNodeNum	Identify the index number of currently running continuous interpolation among continous interpolation moves in specified coordinate system.
AxmContiGetTotalNodeNum	Identify total number of index of continuous interpolation move set in specified coordinate system.

Example

C++

```
double dVelocity = 2000, dAccel = 4000;
long   lAxis[3];
double dPosX[5];
double dPosY[5];
double dPosZ[5];
int    iSize = 3;
long lCoordinate = 0;
for(int i=0; i<3; i++){
  lAxis[i]    = i; }
AxmContiWriteClear(lCoordinate);
AxmContiSetAxisMap(lCoordinate, 3, lAxis);
AxmContiSetAbsRelMode(lCoordinate, 0);
dPosX[0] = 0  , dPosY[0] = 0;
dPosX[1] = 450, dPosY[1] = 100;
dPosX[2] = 750, dPosY[2] = 450;
dPosX[3] = 1080, dPosY[3] = 750;
dPosX[4] = 1300, dPosY[4] = 700;
AxmContiBeginNode(lCoordinate);
AxmSplineWrite(lCoordinate, 5, dPosX, dPosY, dVelocity, dAccel, dAccel, 1, 100);
AxmContiEndNode(lCoordinate);
AxmContiSetAbsRelMode(lCoordinate, 0);
AxmContiStart(lCoordinate, 0, 0);
```

Visual Basic

```
Dim dVelocity As Double
Dim dAccel As Double
Dim lPosSize As Long
Dim lCoordinate As Long
Dim dPosX(0 To 4) As Double
Dim dPosY(0 To 4) As Double
Dim dPosZ(0 To 4) As Double
Dim axes(0 To 2) As Long
Dim i As Long
dVelocity = 2000
dAccel = 4000
lCoordinate = 0
For i = 0 To 2
   axes(i) = i
```

```
Next i
lCoordinate = 0
AxmContiWriteClear lCoordinate
AxmContiSetAxisMap lCoordinate, 3, axes(0)
AxmContiSetAbsRelMode lCoordinate, 0
dPosX(0) = 0:   dPosY(0) = 0
dPosX(1) = 450: dPosY(1) = 100
dPosX(2) = 750: dPosY(2) = 450
dPosX(3) = 1080: dPosY(3) = 750
dPosX(4) = 1300: dPosY(4) = 700
AxmContiBeginNode lCoordinate
AxmSplineWrite lCoordinate, 5, dPosX(0), dPosY(0), dVelocity, dAccel, dAccel, 1, 100
AxmContiEndNode lCoordinate
AxmContiSetAbsRelMode lCoordinate, 0
AxmContiStart lCoordinate, 0, 0
AxmContiSetAxisMap lCoordinate, 4, axes(0)
AxmContiSetAbsRelMode lCoordinate, 1
AxmHelixAngleMove lCoordinate, 500, 500, 360, 100, dVelocity, dAccel, dAccel, DIR_CW
```

Delphi

```
var
   i : LongInt;
   dVelocity, dAccel : Double;
   lAxis : array [0..2] of LongInt;
   dPosX : array [0..4] of Double;
   dPosY : array [0..4] of Double;
   lCoordinate : LongInt;

begin
   dVelocity := 2000;
   dAccel := 4000;
   lCoordinate := 0;
   for i := 0 to 2 do
      begin
         lAxis[i]  := i;
      end;
   AxmContiWriteClear(lCoordinate);
   AxmContiSetAxisMap(lCoordinate, 3, @lAxis);
   AxmContiSetAbsRelMode(lCoordinate, 0);
   dPosX[0] := 0;   dPosY[0] := 0;
   dPosX[1] := 450;  dPosY[1] := 240;
   dPosX[2] := 750;  dPosY[2] := 610;
   dPosX[3] := 1080; dPosY[3] := 500;
   dPosX[4] := 1300; dPosY[4] := 800;
   AxmContiBeginNode(lCoordinate);
   AxmSplineWrite(lCoordinate, 5, @dPosX, @dPosY, dVelocity, dAccel, dAccel, 1, 100);
   AxmContiEndNode(lCoordinate);
   AxmContiSetAbsRelMode(lCoordinate, 0);
   AxmContiStart(lCoordinate, 0, 0);
```

```
end;
```

See Also

AxmContiSetAxisMap, AxmContiGetAxisMap, AxmContiSetAbsRelMode, AxmContiGetAbsRelMode, AxmContiBeginNode, AxmContiEndNode, AxmContiReadFree, AxmContiReadIndex, AxmContiWriteClear, AxmContiStart, AxmContiIsMotion, AxmContiGetNodeNum, AxmContiGetTotalNodeNum

Electronic CAM Setting And Move API

Ecam drive can be used only in PCI-R1604-MLII products.

ECAM coordinates a relationship between a master and a slave axis. The master axis provides a reference position, which the slave axis will follow. This slave axis will reach predefined points, creating a profile, based on the location of the master.

Function	Description
AxmEcamSet	Set the axis for master and slave, the array of position, the start position, the number of entry.
AxmEcamGet	Return the array of position, the start position, the number of entry on the specified slave axis.
AxmEcamEnableBySlave	Enable ECAM mode on the specified slave axis before starting to drive.
AxmEcamIsSlaveEnable	Return the status of ECAM mode on the specified slave axis.
AxmEcamEnableByMaster	Enable ECAM mode on all slave axes of the specified master axis before starting to drive.

AxmEcamSet

Set the axis for master and slave, the array of position, the start position, the number of entry.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmEcamSet(long lAxisNo, long lMasterAxis, long lNumEntry,
double dMasterStartPos, double *dpMasterPos, double *dpSlavePos);
```

Visual Basic

```
Function AxmEcamSet(ByVal lAxisNo As Long, ByVal lMasterAxis As Long,
ByVal lNumEntry As Long, ByVal dMasterStartPos As Double, ByRef
dpMasterPos As Double, ByRef dpSlavePos As Double) As Long
```

Delphi

```
function AxmEcamSet(lAxisNo : LongInt; lMasterAxis : LongInt;
lNumEntry : LongInt; dMasterStartPos : Double; dpMasterPos : PDouble;
dpSlavePos : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lMasterAxis	ECAM master axis (0 ~ (max axis - 1)) lAxisNo ≠ lMasterAxis
[>>]lNumEntry	The number of position entry in ECAM mode (less than 512)
[>>]dMasterStartPos	The start position of master axis in ECAM mode
[>>]dpMasterPos	The array of position for master axis in ECAM mode (dpMasterPos[0] = 0, dpMasterPos[n + 1] > dpMasterPos[n])
[>>]dpSlavePos	The array of position for slave axis in ECAM mode (dpSlavePos[0] = 0, dpSlavePos[lNumEntry] = 0)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Specify the axis for ECAM mode to set the information. Remember that one master axis can have more than one slave axis but one slave axis have only one master axis. This API supports up to 512 positions for each axis. The start position of master axis is absolute. When master axis move to the

start position in any direction, the ECAM mode can be applied. For master axis and slave axis, the position in array should be relative to the start position of master axis.

ECAM given distance of master axis change that the by the position of axis drive at thie point need input value axis number, the number of ECAM position, the master axis position where applied to the first position data (ECAM starting position), the array of position which is relative to starting position.

The first data in position array of master axis and slave axis should be '0', while the last data of slave axis must be '0'. After setting the information, ECAM mode can be used by 'AxmEcamEnableBySlave' or 'AxmEcamEnableByMaster' .

Example

C++

```
// ECAM table is applied to master axis 0, slave axis 1, start position is 0,
// data count 10.
long lAxis, lMasterAxis, lNumEntry;
double dPos[11], dSlavePos[11], dMasterStartPos, dMasterCycle;
lAxis = 1 ;
lMasterAxis = 0 ;
dMasterStartPos = 0;
lNumEntry = 10;
dPos[0] = 0;     dSlavePos [0] = 0;
dPos[1] = 100;   dSlavePos [1] = 9.549;
dPos[2] = 200;   dSlavePos [2] = 34.549;
dPos[3] = 300;   dSlavePos [3] = 65.45;
dPos[4] = 400;   dSlavePos [4] = 90.45;
dPos[5] = 500;   dSlavePos [5] = 100;
dPos[6] = 600;   dSlavePos [6] = 90.45;
dPos[7] = 700;   dSlavePos [7] = 65.45;
dPos[8] = 800;   dSlavePos [8] = 34.549;
dPos[9] = 900;   dSlavePos [9] = 9.549;
dPos[10] = 1000; dSlavePos [10] = 0;

AxmEcamSet(lAxis, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);
```

Visual Basic

```
` ECAM table is applied to master axis 0, slave axis 1, start position is 0,
` data count 10.
Dim lAxis As Long
Dim lMasterAxis As Long
Dim lNumEntry As Long
Dim dPos(0 To 11) As Double
Dim dSlavePos(0 To 11) As Double
Dim dMasterStartPos As Double
lAxis = 1
lMasterAxis = 0
lNumEntry = 10
dMasterStartPos = 0
dPos(0) = 0     dSlavePos (0) = 0
dPos(1) = 100   dSlavePos (1) = 9.549
```

```
dPos(2) = 200   dSlavePos (2) = 34.549
dPos(3) = 300   dSlavePos (3) = 65.45
dPos(4) = 400   dSlavePos (4) = 90.45
dPos(5) = 500   dSlavePos (5) = 100
dPos(6) = 600   dSlavePos (6) = 90.45
dPos(7) = 700   dSlavePos (7) = 65.45
dPos(8) = 800   dSlavePos (8) = 34.549
dPos(9) = 900   dSlavePos (9) = 9.549
dPos(10) = 1000 dSlavePos (10) = 0

AxmEcamSet lAxis, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos
```

Delphi

```
{ ECAM table is applied to master axis 0, slave axis 1, start position is 0, data count
10. }
var
   lAxis : LongInt;
   lMasterAxis : LongInt;
   lNumEntry: Longint;
   dPos : array [0..11] of Double;
   dSlavePos : array [0..11] of Double;
   dMasterStartPos : Double;

begin
   lAxis := 1 ;
   lMasterAxis := 0 ;
   lNumEntry := 10 ;
   dMasterStartPos := 0 ;

   dPos[0] := 0;     dSlavePos [0] := 0;
   dPos[1] := 100;   dSlavePos [1] := 9.549;
   dPos[2] := 200;   dSlavePos [2] := 34.549;
   dPos[3] := 300;   dSlavePos [3] := 65.45;
   dPos[4] := 400;   dSlavePos [4] := 90.45;
   dPos[5] := 500;   dSlavePos [5] := 100;
   dPos[6] := 600;   dSlavePos [6] := 90.45;
   dPos[7] := 700;   dSlavePos [7] := 65.45;
   dPos[8] := 800;   dSlavePos [8] := 34.549;
   dPos[9] := 900;   dSlavePos [9] := 9.549;
   dPos[10] := 1000; dSlavePos [10] := 0;

   AxmEcamSet (lAxis, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);
end;
```

See Also

AxmEcamGet, AxmEcamEnableBySlave, AxmEcamIsSlaveEnable, AxmEcamEnableByMaster

AxmEcamGet

Purpose

Return the array of position, the start position, the number of entry on the specified slave axis.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmEcamGet(long lAxisNo, long *lpMasterAxis, long *lpNumEntry,
double *dpMasterStartPos, double *dpMasterPos, double *dpSlavePos);
```

Visual Basic

```
Function AxmEcamGet(ByVal lAxisNo As Long, ByRef lpMasterAxis As Long,
ByRef lpNumEntry As Long, ByRef dpMasterStartPos As Double, ByRef
dpMasterPos As Double, ByRef dpSlavePos As Double) As Long
```

Delphi

```
function AxmEcamGet(lAxisNo : LongInt; lpMasterAxis : PLongInt;
lpNumEntry : PLongInt; dpMasterStartPos : PDouble; dpMasterPos :
PDouble; dpSlavePos : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]lpMasterAxis	ECAM master axis (0 ~ (max axis - 1))
[<<]lpNumEntry	The number of position entry in ECAM mode (less than 512)
[<<]dpMasterStartPos	The start position of master axis in ECAM mode
[<<]dpMasterPos	The array of position for master axis in ECAM mode
[<<]dpSlavePos	The array of position for slave axis in ECAM mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return the corresponding master axis, the number of entry, the start position of master axis, the position array of master axis and slave axis for specific axis set by 'AxmEcamSet' .

Example

C++

```
// ECAM table is applied to master axis 0, slave axis 1, start position is 0,
// data count 10.
long lAxis = 1;
long lMasterAxis;
long lNumEntry;
double dMasterStartPos;
double dPos[512];
double dSlavePos[512];
AxmEcamGet(lAxis, &lMasterAxis, &lNumEntry, &dMasterStartPos, dPos, dSlavePos);
```

Visual Basic

```
` ECAM table is applied to master axis 0, slave axis 1, start position is 0,
` data count 10.

Dim lAxis As Long
Dim lMasterAxis As Long
Dim lRetNumEntry As Long
Dim dRetPos(0 To 11) As Double
Dim dRetSlavePos(0 To 11) As Double
Dim dRetMasterStartPos As Double
lAxis = 1
AxmEcamGet lAxis, lMasterAxis, lRetNumEntry, dRetMasterStartPos, dRetPos(0),
  dRetSlavePos(0)
```

Delphi

```
{ ECAM table is applied to master axis 0, slave axis 1, start position is 0, data count
10. }
var
  lAxis : Longint;
  lRetNumEntry : Longint;
  dRetPos : array [0..11] of Double;
  dRetSlavePos : array [0..11] of Double;
  dRetMasterStartPos : Double;

Begin
  lAxis := 1;
  AxmEcamGet (lAxis, @lMasterAxis, @lRetNumEntry, @dRetMasterStartPos, dRetPos,
     dRetSlavePos );
end;
```

See Also

AxmEcamSet, AxmEcamEnableBySlave, AxmEcamIsSlaveEnable, AxmEcamEnableByMaster

AxmEcamEnableBySlave

Purpose

Enable ECAM mode on the specified slave axis before starting to drive.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmEcamEnableBySlave(long lAxisNo, DWORD dwEnable);
```

Visual Basic

```
Function AxmEcamEnableBySlave(ByVal lAxisNo As Long, ByVal dwEnable
As Long) As Long
```

Delphi

```
function AxmEcamEnableBySlave(lAxisNo : LongInt; dwEnable : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dwEnable	Set whether to use ECAM mode : AXT_USE - [00h] Not used ECAM mode - [01h] Used ECAM mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

By using this API, ECAM mode can be active or inactive on specific slave axis wherever master position moved. Before calling this function, 'AxmEcamSet' should be set to obtain the necessary information to drive.

Example

C++

```
// Activate the Ecam feature on slave axis 1.
long lAxis = 1;
AxmEcamEnableBySlave(lAxis, ENABLE );
```

Visual Basic

```
' Activate the Ecam feature on slave axis 1.
Dim lAxis As Long
```

```
lAxis = 1
AxmEcamEnableBySlave lAxis, ENABLE
```

Delphi

```
{ Activate the Ecam feature on slave axis 1. }
var
  lAxis : Longint;

begin
  lAxis := 1;
  AxmEcamEnableBySlave (lAxis, ENABLE);
end;
```

See Also

AxmEcamSet, AxmEcamGet, AxmEcamIsSlaveEnable, AxmEcamEnableByMaster

AxmEcamIsSlaveEnable

Purpose

Return the status of ECAM mode on the specified slave axis.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmEcamIsSlaveEnable(long lAxisNo, DWORD *dwpEnable);
```

Visual Basic

```
Function AxmEcamIsSlaveEnable(ByVal lAxisNo As Long, ByRef dwpEnable
As Long) As Long
```

Delphi

```
function AxmEcamIsSlaveEnable(lAxisNo : LongInt; dwpEnable : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dwpEnable	Get whether to use ECAM mode : AXT_USE - [00h] Not used ECAM mode - [01h] Used ECAM mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return the status of ECAM mode for specific slave axis set by 'AxmEcamEnableBySlave' or 'AxmEcamEnableByMaster'.

Example

C++

```
// Activate the Ecam mode on slave axis 1.
long lAxis = 1;
DWORD dwEnable;
AxmEcamIsSlaveEnable (lAxis, &dwEnable );
```

Visual Basic

```
' Activate the Ecam mode on slave axis 1.
Dim lAxis As Long
```

```
Dim dwReturn As Long
lAxis = 1
AxmEcamIsSlaveEnable lAxis, dwReturn
```

Delphi

```
{ Activate the Ecam mode on slave axis 1. }
var
   dwRetEnable : Long;
   lAxis : Longint;

begin
   lAxis := 1;
   AxmEcamIsSlaveEnable (lAxis, @dwRetEnable);
end;
```

See Also

AxmEcamSet, AxmEcamGet, AxmEcamEnableBySlave, AxmEcamEnableByMaster

AxmEcamEnableByMaster

Purpose

Enable ECAM mode on all slave axes of the specified master axis before starting to drive.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmEcamEnableByMaster(long lAxisNo, DWORD dwEnable);
```

Visual Basic

```
Function AxmEcamEnableByMaster(ByVal lAxisNo As Long, ByVal dwEnable
As Long) As Long
```

Delphi

```
function AxmEcamEnableByMaster(lAxisNo : LongInt; dwEnable : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dwEnable	Set whether to use ECAM mode : AXT_USE - [00h] Not used ECAM mode - [01h] Used ECAM mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

By using this API, ECAM mode can be active or inactive on all slave axes that are corresponding to specific master axis. Before calling this function, 'AxmEcamSet' should be set to obtain the necessary information to drive.

Example

C++

```
// ECAM table is applied to master axis 0, slave axis 1/2/3, start position is 0,
// data count 10.
long lAxis, lMasterAxis, lNumEntry;
double dPos[11], dSlavePos[11], dMasterStartPos, dMasterCycle;
lAxis = 1 ;
lMasterAxis = 0 ;
dMasterStartPos = 0;
```

```
lNumEntry = 10;
dPos[0] = 0;     dSlavePos [0] = 0;
dPos[1] = 100;   dSlavePos [1] = 9.549;
dPos[2] = 200;   dSlavePos [2] = 34.549;
dPos[3] = 300;   dSlavePos [3] = 65.45;
dPos[4] = 400;   dSlavePos [4] = 90.45;
dPos[5] = 500;   dSlavePos [5] = 100;
dPos[6] = 600;   dSlavePos [6] = 90.45;
dPos[7] = 700;   dSlavePos [7] = 65.45;
dPos[8] = 800;   dSlavePos [8] = 34.549;
dPos[9] = 900;   dSlavePos [9] = 9.549;
dPos[10] = 1000; dSlavePos [10] = 0;

AxmEcamSet(1, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);
AxmEcamSet(2, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);
AxmEcamSet(3, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);
// ECAM is enabled on all slaves with master axis # 0
AxmEcamEnableByMaster(0, ENABLE);
```

Visual Basic

```
` ECAM table is applied to master axis 0, slave axis 1/2/3, start position is 0,
` data count 10.
Dim lAxis As Long
Dim lMasterAxis As Long
Dim lNumEntry As Long
Dim dPos(0 To 11) As Double
Dim dSlavePos(0 To 11) As Double
Dim dMasterStartPos As Double
lAxis = 1
lMasterAxis = 0
lNumEntry = 10
dMasterStartPos = 0
dPos(0) = 0     dSlavePos (0) = 0
dPos(1) = 100   dSlavePos (1) = 9.549
dPos(2) = 200   dSlavePos (2) = 34.549
dPos(3) = 300   dSlavePos (3) = 65.45
dPos(4) = 400   dSlavePos (4) = 90.45
dPos(5) = 500   dSlavePos (5) = 100
dPos(6) = 600   dSlavePos (6) = 90.45
dPos(7) = 700   dSlavePos (7) = 65.45
dPos(8) = 800   dSlavePos (8) = 34.549
dPos(9) = 900   dSlavePos (9) = 9.549
dPos(10) = 1000 dSlavePos (10) = 0

AxmEcamSet 1, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos
AxmEcamSet 2, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos
AxmEcamSet 3, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos

AxmEcamEnableByMaster 0, ENABLE
```

Delphi

```
{ ECAM table is applied to master axis 0, slave axis 1/2/3, start position is 0, data
count 10. }
var
   lAxis : LongInt;
   lMasterAxis : LongInt;
   lNumEntry: Longint;
   dPos : array [0..11] of Double;
   dSlavePos : array [0..11] of Double;
   dMasterStartPos : Double;

begin
   lAxis := 1 ;
   lMasterAxis := 0 ;
   lNumEntry := 10 ;
   dMasterStartPos := 0 ;

   dPos[0] := 0;     dSlavePos [0] := 0;
   dPos[1] := 100;   dSlavePos [1] := 9.549;
   dPos[2] := 200;   dSlavePos [2] := 34.549;
   dPos[3] := 300;   dSlavePos [3] := 65.45;
   dPos[4] := 400;   dSlavePos [4] := 90.45;
   dPos[5] := 500;   dSlavePos [5] := 100;
   dPos[6] := 600;   dSlavePos [6] := 90.45;
   dPos[7] := 700;   dSlavePos [7] := 65.45;
   dPos[8] := 800;   dSlavePos [8] := 34.549;
   dPos[9] := 900;   dSlavePos [9] := 9.549;
   dPos[10] := 1000; dSlavePos [10] := 0;

   AxmEcamSet (1, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);
   AxmEcamSet (2, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);
   AxmEcamSet (3, lMasterAxis, lNumEntry, dMasterStartPos, dPos, dSlavePos);

   AxmEcamEnableByMaster(0, ENABLE);
end;
```

See Also

AxmEcamSet, AxmEcamGet, AxmEcamEnableBySlave, AxmEcamIsSlaveEnable

Compensation Geometric Error Setting And Move API

Compensation drive can be used only in PCI-R1604-MLII products.

Geometric errors are regarded as the machine errors which exist under cold and warm up conditions and which do not change with time. This means that the machine repeats the same error again and again for some duration. Major geometric errors are roll, pitch, yaw and squareness errors. These types of errors originate from the manufacturing or assembly defects of different parts of the machine. In the case of assembling the machine tool, the designed state and assembly state exerts an important effect on the accuracy of the machine tool.

Function	Description
AxmCompensationSet	Set the array of position, the start position, the number of entry on the specified axis.
AxmCompensationGet	Return the array of position, the start position, the number of entry on the specified axis.
AxmCompensationEnable	Enable Compensation mode on the specified axis before starting to drive.
AxmCompensationIsEnable	Return the status of Compensation mode on the specified axis.

AxmCompensationSet

Set the array of position, the start position, the number of entry on the specified axis.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmCompensationSet(long lAxisNo, long lNumEntry, double
dStartPos, double *dpPosition, double *dpCorrection, DWORD
dwRollOver);
```

Visual Basic

```
Function AxmCompensationSet(ByVal lAxisNo As Long, ByVal lNumEntry As
Long, ByVal dStartPos As Double, ByRef dpPosition As Double, ByRef
dpCorrection As Double, ByVal dwRollOver As Long) As Long
```

Delphi

```
function AxmCompensationSet(lAxisNo : LongInt; lNumEntry : LongInt;
dStartPos : Double; dpPosition : PDouble; dpCorrection : PDouble;
dwRollOver : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]lNumEntry	The number of entry in Compensation mode (less than < 512)
[>>]dStartPos	The start position of specific axis in Compensation mode
[>>]dpPosition	The array of position on specific axis in Compensation mode (dpPosition [0] = 0, dpPosition [n + 1] > dpPosition [n])
[>>]dpCorrection	The array of correction position on specific axis in Compensation mode (dpCorrection [0] = 0, dpCorrection [lNumEntry] = 0)
[>>]dwRollOver	Set whether to use RollOver feature : AXT_USE - [00h] Not used RollOver - [01h] Used RollOver

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Specify the axis for Compensation mode to set the information.

This API will take the distance which is subtracted from the reference position and start position. Then it will find the matching position in the table. Generally, this is between two entries in the table, so it

will linearly interpolate between these two entries to obtain the correction for the current communication cycle. There are up to 512 entries that user can apply in the table.

The distance from the first data to the last data of array is called table range. If user activates the RollOver feature, the uncorrected position is rolled over to within the range when motor move outside of this range.

Example

C++

```
// Compensation feature is applied to axis 1, from position 1000, 10 entries,
// enable RollOver feature.
long lAxis, lNumEntry ;
double dPos[11], dCorrection[11], dStartPos;
DWORD dwRollOver ;
lAxis = 1 ;
lNumEntry = 10 ;
dStartPos = 1000 ;
bRollOver = 1 ;
dPos[0] = 0;     dCorrection[0] = 0;
dPos[1] = 100;   dCorrection[1] = 10;
dPos[2] = 150;   dCorrection[2] = 30;
dPos[3] = 240;   dCorrection[3] = 35;
dPos[4] = 440;   dCorrection[4] = 24;
dPos[5] = 500;   dCorrection[5] = 5;
dPos[6] = 600;   dCorrection[6] = -8;
dPos[7] = 610;   dCorrection[7] = 15;
dPos[8] = 790;   dCorrection[8] = 10;
dPos[9] = 880;   dCorrection[9] = 25;
dPos[10] = 1000; dCorrection[10] = 0;

AxmCompensationSet(lAxis, lNumEntry, dStartPos, dPos, dCorrection, dwRollOver );
AxmCompensationEnable(lAxisNo, ENABLE);
```

Visual Basic

```
` Compensation feature is applied to axis 1, from position 1000, 10 entries,
` enable RollOver feature.
Dim lAxis As Long
Dim lNumEntry As Long
Dim dPos(0 To 11) As Double
Dim dCorrection (0 To 11) As Double
Dim dStartPos As Double
Dim dwRollOver As Long
lAxis = 1
lNumEntry = 10
dStartPos = 1000
dwRollOver = 1
dPos(0) = 0
dPos(1) = 100
dPos(2) = 150
```

```
dPos(3) = 240
dPos(4) = 440
dPos(5) = 500
dPos(6) = 600
dPos(7) = 610
dPos(8) = 790
dPos(9) = 880
dPos(10) = 1000
dCorrection(0) = 0
dCorrection(1) = 10
dCorrection(2) = 30
dCorrection(3) = 35
dCorrection(4) = 24
dCorrection(5) = 5
dCorrection(6) = -8
dCorrection(7) = 15
dCorrection(8) = 10
dCorrection(9) = 25
dCorrection(10) = 0

AxmCompensationSet lAxis, lNumEntry, dStartPos, dPos, dCorrection, dwRollOver
AxmCompensationEnable lAxisNo, ENABLE
```

Delphi

```
{ Compensation feature is applied to axis 1, from position 1000, 10 entries, enable
RollOver feature. }

var
   lAxis : LongInt;
   lNumEntry : LongInt;
   dwRollOver : DWord;
   dPos[11] : array(0..11) of Double;
   dCorrection[11] : array(0..11) of Double;
   dStartPos : Double;

begin
   lAxis := 1 ;
   lNumEntry := 10 ;
   dStartPos := 1000 ;
   bRollOver := 1 ;
   dPos(0) := 0;     dCorrection(0) := 0;
   dPos(1) := 100;   dCorrection(1) := 10;
   dPos(2) := 150;   dCorrection(2) := 30;
   dPos(3) := 240;   dCorrection(3) := 35;
   dPos(4) := 440;   dCorrection(4) := 24;
   dPos(5) := 500;   dCorrection(5) := 5;
   dPos(6) := 600;   dCorrection(6) := -8;
   dPos(7) := 610;   dCorrection(7) := 15;
   dPos(8) := 790;   dCorrection(8) := 10;
```

```
  dPos(9) := 880;   dCorrection(9) := 25;
  dPos(10) := 1000; dCorrection(10) := 0;

  AxmCompensationSet(lAxis, lNumEntry, dStartPos, dPos, dCorrection, dwRollOver );
  AxmCompensationEnable(lAxisNo, ENABLE);
end
```

See Also

AxmCompensationGet, AxmCompensationEnable, AxmCompensationIsEnable

AxmCompensationGet

Return the array of position, the start position, the number of entry on the specified axis.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmCompensationGet(long lAxisNo, long *lpNumEntry, double
*dpStartPos, double *dpPosition, double *dpCorrection, DWORD
*dwpRollOver);
```

Visual Basic

```
Function AxmCompensationGet(ByVal lAxisNo As Long, ByRef lpNumEntry
As Long, ByRef dpStartPos As Double, ByRef dpPosition As Double,
ByRef dpCorrection As Double, ByRef dwpRollOver As Long) As Long
```

Delphi

```
function AxmCompensationGet(lAxisNo : LongInt; lpNumEntry : PLongInt;
dpStartPos : PDouble; dpPosition : PDouble; dpCorrection : PDouble;
dwpRollOver : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]lpNumEntry	The number of entry in Compensation mode (less than < 512)
[<<]dpStartPos	The start position of specific axis in Compensation mode
[<<]dpPosition	The array of position on specific axis in Compensation mode (dpPosition [0] = 0, dpPosition [n + 1] > dpPosition [n])
[<<]dpCorrection	The array of correction position on specific axis in Compensation mode (dpCorrection [0] = 0, dpCorrection [lNumEntry] = 0)
[<<]dwpRollOver	Get whether to use RollOver feature : AXT_USE - [00h] Not used RollOver - [01h] Used RollOver

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return the number of entry, the start position, the position array of specific axis, the correction array of specific axis and status of RollOver feature set by 'AxmCompensationSet'.

Example

C++

```
// Compensation mode is applied to axis 0, from position 1000, 10 entries, enable
// RollOver feature.
long lAxis, lNumEntry ;
double dPos[512], dCorrection[512], dStartPos;
DWORD dwRollOver ;

lAxis = 0;
AxmCompensationGet(lAxis, &lNumEntry, &dStartPos, dPos, dCorrection, &dwRollOver );
```

Visual Basic

```
` Compensation mode is applied to axis 0, from position 1000, 10 entries, enable
` RollOver feature.
Dim lAxis As Long
Dim lNumEntry As Long
Dim dPos(0 To 11) As Double
Dim dCorrection (0 To 11) As Double
Dim dStartPos As Double
Dim dwRollOver As Long
lAxis = 0
AxmCompensationGet lAxis, lNumEntry, dStartPos, dPos, dCorrection, dwRollOver
```

Delphi

```
{ Compensation feature is applied to axis 0, from position 1000, 10 entries, enable
RollOver feature. }

var
   lAxis : LongInt;
   lNumEntry : LongInt;
   dwRollOver : DWord;
   dPos[11] : array(0..11) of Double;
   dCorrection[11] : array(0..11) of Double;
   dStartPos : Double;

begin
   lAxis := 0;
   AxmCompensationGet(lAxis, @lNumEntry, @dStartPos, dPos, dCorrection, @dwRollOver );
end
```

See Also

AxmCompensationSet, AxmCompensationEnable, AxmCompensationIsEnable

AxmCompensationEnable

Purpose

Enable Compensation mode on the specified axis before starting to drive.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmCompensationEnable(long lAxisNo, DWORD dwEnable);
```

Visual Basic

```
Function AxmCompensationEnable(ByVal lAxisNo As Long, ByVal dwEnable
As Long) As Long
```

Delphi

```
function AxmCompensationEnable(lAxisNo : LongInt; dwEnable : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[>>]dwEnable	Set whether to use Compensation mode : AXT_USE - [00h] Not used Compensation mode - [01h] Used Compensation mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

By using this API, Compensation mode can be active or inactive on specific axis. Before calling this function, 'AxmCompensationSet' should be set to obtain the necessary information to drive.

Example

C++

```
// Activate the Compensation mode on slave axis 1.
long lAxis = 1;
AxmCompensationEnable (lAxis, ENABLE );
```

Visual Basic

```
' Activate the Compensation mode on slave axis 1.
Dim lAxis As Long
lAxis = 1
```

```
AxmCompensationEnable lAxis, ENABLE
```

Delphi

```
{ Activate the Compensation mode on slave axis 1. }
Var
   lAxis : LongInt;

begin
   lAxis := 1;
   AxmCompensationEnable (lAxis, ENABLE);
end;
```

See Also

AxmCompensationSet, AxmCompensationGet, AxmCompensationIsEnable

AxmCompensationIsEnable

Purpose

Return the status of Compensation mode on the specified axis.

Note : This API can be only used in PCI-R1604-MLII.

Format

C++

```
DWORD AxmCompensationIsEnable(long lAxisNo, DWORD *dwpEnable);
```

Visual Basic

```
Function AxmCompensationIsEnable(ByVal lAxisNo As Long, ByRef
dwpEnable As Long) As Long
```

Delphi

```
function AxmCompensationIsEnable(lAxisNo : LongInt; dwpEnable :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAxisNo	Channel (axis) number(starting from 0)
[<<]dwpEnable	Get whether to use Compensation mode : AXT_USE - [00h] Not used Compensation mode - [01h] Used Compensation mode

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO : Failed to initialize the motion of corresponding axis
[4101] AXT_RT_MOTION_INVALID_AXIS_NO : Corresponding axis does not exist
* See error code Table for more information on status error codes

Description

Return the status of Compensation mode for specific axis set by 'AxmCompensationEnable'.

Example

C++

```
// Check for axis 1 about Compensation function.
long lAxis = 1;
DWORD dwEnable;
AxmCompensationIsEnable(lAxis, &dwEnable );
```

Visual Basic

```
` Check for axis 1 about Compensation function.
Dim lAxis As Long
Dim dwReturn As Long
```

```
lAxis = 1
AxmCompensationIsEnable lAxis, dwReturn
```

Delphi

```
{ Check for axis 1 about Compensation function. }
var
  dwRetEnable : Long;
  lAxis : Longint;

begin
  lAxis := 1;
  AxmCompensationIsEnable(lAxis, @dwRetEnable);
end;
```

See Also

AxmCompensationSet, AxmCompensationGet, AxmCompensationEnable

Error Code Table Check

Error coded returned from the library functions can be used to check the errors generated during the execution of library functions and to process the result accordingly. Error code returned from each of the library functions indicates reasons of failure of corresponding function, and you can check the error code to understand the cause of failure. In general, it is good idea to check the code when the function returns the code other than AXT_RT_SUCCESS for the purpose of easier debugging.

Error Code	Description
[0000] AXT_RT_SUCCESS	API function executed successfully
[1001] AXT_RT_OPEN_ERROR	Library is not open
[1002] AXT_RT_OPEN_ALREADY	Library is already open
[1053] AXT_RT_NOT_OPEN	Failed to initialize the library
[1054] AXT_RT_NOT_SUPPORT_VERSION	Hardware not supported
[1101] AXT_RT_INVALID_BOARD_NO	Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS	Invalid module position
[1103] AXT_RT_INVALID_LEVEL	Invalid level
[1151] AXT_RT_ERROR_VERSION_READ	Failed to read library version
[1152] AXT_RT_NETWORK_ERROR	Slave nodes is not connected to the network or connection is faulty
[1153] AXT_RT_NETWORK_LOCK_MISMATCH	Stored connection information and the actual connection information is incorrect. Lock of the board in EzConfig information does not match with the actual connection status. Lock information for reconstruction of a slave existing Ezconfig should give clear in the Unlock
[1160] AXT_RT_1ST_BELOW_MIN_VALUE	First parameter is below the minimum value
[1161] AXT_RT_1ST_ABOVE_MAX_VALUE	First parameter is above the maximum value
[1170] AXT_RT_2ND_BELOW_MIN_VALUE	Second parameter is below the minimum value
[1171] AXT_RT_2ND_ABOVE_MAX_VALUE	Second parameter is above the maximum value
[1180] AXT_RT_3RD_BELOW_MIN_VALUE	Third parameter is below the minimum value
[1181] AXT_RT_3RD_ABOVE_MAX_VALUE	Third parameter is above the maximum value
[1190] AXT_RT_4TH_BELOW_MIN_VALUE	Fourth parameter is below the minimum value
[1191] AXT_RT_4TH_ABOVE_MAX_VALUE	Fourth parameter is above the maximum value
[1200] AXT_RT_5TH_BELOW_MIN_VALUE	Fifth parameter is below the minimum value
[1201] AXT_RT_5TH_ABOVE_MAX_VALUE	Fifth parameter is above the maximum value
[1210] AXT_RT_6TH_BELOW_MIN_VALUE	Sixth parameter is below the minimum value
[1211] AXT_RT_6TH_ABOVE_MAX_VALUE	Sixth parameter is above the maximum value
[1220] AXT_RT_7TH_BELOW_MIN_VALUE	Seventh parameter is below the minimum value
[1221] AXT_RT_7TH_ABOVE_MAX_VALUE	Seventh parameter is above the maximum value

[1230] AXT_RT_8TH_BELOW_MIN_VALUE	Eighth parameter is below the minimum value
[1231] AXT_RT_8TH_ABOVE_MAX_VALUE	Eighth parameter is above the maximum value
[1240] AXT_RT_9TH_BELOW_MIN_VALUE	Nineth parameter is below the minimum value
[1241] AXT_RT_9TH_ABOVE_MAX_VALUE	Nineth parameter is above the maximum value
[1250] AXT_RT_10TH_BELOW_MIN_VALUE	Tenth parameter is below the minimum value
[1251] AXT_RT_10TH_ABOVE_MAX_VALUE	Tenth parameter is above the maximum value
[2001] AXT_RT_AIO_OPEN_ERROR	Failed to open AIO module
[2051] AXT_RT_AIO_NOT_MODULE	No AIO module
[2052] AXT_RT_AIO_NOT_EVENT	Failed to read AIO event
[2101] AXT_RT_AIO_INVALID_MODULE_NO	Invalid AIO module
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO	Invalid AIO channel number
[2106] AXT_RT_AIO_INVALID_USE	Can not use AIO
[2107] AXT_RT_AIO_INVALID_TRIGGER_MODE	Invalid trigger mode
[3001] AXT_RT_DIO_OPEN_ERROR	Failed to open DIO module
[3051] AXT_RT_DIO_NOT_MODULE	No DIO module
[3052] AXT_RT_DIO_NOT_INTERRUPT	DIO Interrupt not set
[3101] AXT_RT_DIO_INVALID_MODULE_NO	Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO	Invalid DIO OFFSET number
[3103] AXT_RT_DIO_INVALID_LEVEL	Invalid DIO level
[3104] AXT_RT_DIO_INVALID_MODE	Invalid DIO mode
[3105] AXT_RT_DIO_INVALID_VALUE	Invalid setting value
[3106] AXT_RT_DIO_INVALID_USE	Can not use DIO API function
[4001] AXT_RT_MOTION_OPEN_ERROR	Failed to open motion library
[4051] AXT_RT_MOTION_NOT_MODULE	No motion module is installed in the system
[4052] AXT_RT_MOTION_NOT_INTERRUPT	Failed to read the resul of interrupt
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO	Failed to initialize the motion of corresponding axis
[4054] AXT_RT_MOTION_NOT_IN_CONT_INTERPOL	Command to stop continuous interpolation while not in continuous interpolation motion
[4055] AXT_RT_MOTION_NOT_PARA_READ	Failed to load parameters for home return drive
[4101] AXT_RT_MOTION_INVALID_AXIS_NO	Corresponding axis does not exist
[4102] AXT_RT_MOTION_INVALID_METHOD	Invalid setting for corresponding axis drive
[4103] AXT_RT_MOTION_INVALID_USE	Invalid setting for 'uUse' parameter
[4104] AXT_RT_MOTION_INVALID_LEVEL	Invalid setting for 'uLevel' parameter
[4105] AXT_RT_MOTION_INVALID_BIT_NO	Invalid setting for general purpose input/output bit
[4106] AXT_RT_MOTION_INVALID_STOP_MODE	Invalid setting values for motion stop mode
[4107] AXT_RT_MOTION_INVALID_TRIGGER_MODE	Invalid setting for trigger setting mode

[4108] AXT_RT_MOTION_INVALID_TRIGGER_LEVEL	Invalid setting for trigger output level
[4109] AXT_RT_MOTION_INVALID_SELECTION	'uSelection' parameter is set to a value other than COMMAND or ACTUAL
[4110] AXT_RT_MOTION_INVALID_TIME	Invalid setting for Trigger output time value
[4111] AXT_RT_MOTION_INVALID_FILE_LOAD	Failed to load the file containing motion setting values
[4112] AXT_RT_MOTION_INVALID_FILE_SAVE	Failed to save the file containing motion setting values
[4113] AXT_RT_MOTION_INVALID_VELOCITY	Motion error occurred since the motion drive velocity value is set as zero
[4114] AXT_RT_MOTION_INVALID_ACCELTIME	Motion error occurred since motion drive acceleration time value is set as zero
[4115] AXT_RT_MOTION_INVALID_PULSE_VALUE	Input pulse value is set to a value less than zero when setting motion unit
[4116] AXT_RT_MOTION_INVALID_NODE_NUMBER	Called position or velocity override function while not in motion
[4117] AXT_RT_MOTION_INVALID_TARGET	Flag indicating the cause of multi axis motion stop
[4151] AXT_RT_MOTION_ERROR_IN_NONMOTION	Not in motion drive when it should be
[4152] AXT_RT_MOTION_ERROR_IN_MOTION	Called another motion drive fuction before the completetion of current motion
[4153] AXT_RT_MOTION_ERROR	Error occurred during the execution of multi axis motion stop function
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE	Gantry-enable is activated again while in motion with gantry enabled
[4155] AXT_RT_MOTION_ERROR_GANTRY_AXIS	Invalid input of gantry axis master channel (axis) number (starting from zero)
[4156] AXT_RT_MOTION_ERROR_MASTER_SERVOON	Servo ON is not enabled for the master axis
[4157] AXT_RT_MOTION_ERROR_SLAVE_SERVOON	Servo ON is not enabled for the slavve axis
[4158] AXT_RT_MOTION_INVALID_POSITION	Not in a valid position
[4159] AXT_RT_ERROR_NOT_SAME_MODULE	Not in the same module
[4160] AXT_RT_ERROR_NOT_SAME_BOARD	Not in the same board
[4161] AXT_RT_ERROR_NOT_SAME_PRODUCT	When products are different each other
[4162] AXT_RT_NOT_CAPTURED	Failed to capture the position
[4163] AXT_RT_ERROR_NOT_SAME_IC	Not in the same chip
[4164] AXT_RT_ERROR_NOT_GEARMODE	Failed to change to the gear mode
[4165] AXT_ERROR_CONTI_INVALID_AXIS_NO	Invalid axis during continuous interpolation axix mapping
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO	Invalid mapping number during continuous interpolation mapping
[4167] AXT_ERROR_CONTI_EMPTY_MAP_NO	Continuous interpolation mapping number is empty
[4168] AXT_RT_MOTION_ERROR_CACULATION	Error in calculation
[4169] AXT_RT_ERROR_MOVE_SENSOR_CHECK	Continuous interpolation error sensor (Alarm, EMG, Limit, etc) is detected

[4170] AXT_ERROR_HELICAL_INVALID_AXIS_NO	Invalid axis for helical axis mapping
[4171] AXT_ERROR_HELICAL_INVALID_MAP_NO	Invalid mapping number for helical mapping
[4172] AXT_ERROR_HELICAL_EMPTY_MAP_NO	Helical mapping number is empty
[4180] AXT_ERROR_SPLINE_INVALID_AXIS_NO	Invalid axis for spline axis mapping
[4181] AXT_ERROR_SPLINE_INVALID_MAP_NO	Invalid mapping number for spline mapping
[4182] AXT_ERROR_SPLINE_EMPTY_MAP_NO	Spline Mapping number is empty
[4183] AXT_ERROR_SPLINE_NUM_ERROR	Spline point value is inapplicable
[4184] AXT_RT_MOTION_INTERPOL_VALUE	Invalid input value for interpolation
[4185] AXT_RT_ERROR_NOT_CONTIBEGIN	CONTIBEGIN function is not called in continuous interpolation
[4186] AXT_RT_ERROR_NOT_CONTIEND	CONTIEND function is not called in continuous interpolation
[4201] AXT_RT_MOTION_HOME_SEARCHING	Other function is called while in home search motion
[4202] AXT_RT_MOTION_HOME_ERROR_SEARCHING	Home search motion is fored to stop by the user or something from external
[4203] AXT_RT_MOTION_HOME_ERROR_START	Failed to start home search drive due to intialization problem
[4204] AXT_RT_MOTION_HOME_ERROR_GANTRY	Failed to execute Gantry enable during home search motion
[4251] AXT_RT_MOTION_POS_OUTOFBOUND	Configured position is either above the maximum value or below the minimum value
[4252] AXT_RT_MOTION_PROFILE_INVALID	Invalid setting for velocity profile
[4253] AXT_RT_MOTION_VELOCITY_OUTOFBOUND	Configured velocity is either above the maximum value or below the minimum value
[4254] AXT_RT_MOTION_MOVE_UNIT_IS_ZERO	Unit of the motion values is set as zero
[4255] AXT_RT_MOTION_SETTING_ERROR	Invalid setting for Velocity, Acceleration, Jerk, or Profile
[4256] AXT_RT_MOTION_IN_CONT_INTERPOL	Execuation of motion start or restart function before the completion of current continous interpolated motion
[4257] AXT_RT_MOTION_DISABLE_TRIGGER	Trigger output is disabled
[4258] AXT_RT_MOTION_INVALID_CONT_INDEX	Invalid setting for continuous interpolation index value
[4259] AXT_RT_MOTION_CONT_QUEUE_FULL	Continuous Interpolation queue of motion chip is full
[4260] AXT_RT_PROTECTED_DURING_SERVOON	Can not be used the function or input value in servo-on status
[4261] AXT_RT_HW_ACCESS_ERROR	H/w of board access error has occurred. Check the status of the board mounting system or battery operation to check the status of power input

The contents of this manual can be changed without notice. The name of company, institution, production, people and event used in the examples are not real data. None of actual company, institution, product, people, or events are intended to be connected, and it shall not be interpreted that way. It is user's responsibility to comply with the corresponding copyright law. Separately from the right in the copyright, any part of this manual shall not be copied in any form or means (electronic, mechanical, paper copy, disk copy, or other method), shall not be coped for other purpose, and shall not be stored, introduced, or transmitted without explicit written approval of Ajinextek Co., Ltd.

Ajinextek Co., Ltd reserves all rights including patents, trademark right, copyright, or other intellectual property associated with the contents of this manual. Except the right explicitly given to you from Ajinextek Co., Ltd. by the written end user license agreement, providing this manual does not allow you to use any of patents, copyright, or other intellectual properties.